

Γιώργος Ν. Παναγιώτου

Μια εισαγωγή στην Python για Μεταλλειολόγους & άλλους Μηχανικούς

```
import numpy as np
distance_from_blast = 200 # m
weight_of_explosives = 100 #kg
type_of_blast = 'Mine' # 'Mine' or 'Quarry'
if type_of_blast == 'Mine':
    h_factor = 1140.
elif type_of_blast == 'Quarry':
    h_factor = 1090.
if type_of_blast == 'Construction':
    h_factor = 1730.
h_factor = -1.60
ppv_usb = h_factor*((distance_from_blast/weight_of_explosives)**2)
type_of_rock = 'Sedimentary' # 'Sedimentary' or 'Igneous'
if type_of_rock == 'Sedimentary':
    ppv_din = 969*(weight_of_explosives**0.5*(distance_from_blast**-1.5))
elif type_of_rock == 'Igneous':
    ppv_din = 206*(weight_of_explosives**0.5*(distance_from_blast**-1.3))
```



ΓΙΩΡΓΟΣ Ν. ΠΑΝΑΓΙΩΤΟΥ

Καθηγητής, Εργαστήριο Εξόρυξης Πετρωμάτων
Σχολή Μηχανικών Μεταλλείων – Μεταλλουργών
Εθνικό Μετσόβιο Πολυτεχνείο

*Μια εισαγωγή στην Python
για Μεταλλειολόγους
&
άλλους Μηχανικούς*



Μια εισαγωγή στην Python για Μεταλλειολόγους & άλλους Μηχανικούς

Συγγραφή

Γιώργος Ν. Παναγιώτου

Συντελεστές έκδοσης

Γλωσσική Επιμέλεια: Δημήτρης Καλλιάρas

Γραφιστική Επιμέλεια: Αντώνιος Κ. Κοκολάκης

Κεντρική ομάδα υποστήριξης

Γλωσσικός Έλεγχος: Γεωργία Τριανταφυλλίδου

Γραφιστικός Έλεγχος: Ηλίας Τσιώνης

Βιβλιοθηκονομική Επεξεργασία: Αλέξανδρος Ηλιάκης



Το παρόν έργο αδειοδοτείται υπό τους όρους της άδειας Creative Commons Αναφορά Δημιουργού - Μη Εμπορική Χρήση - Παρόμοια Διανομή 4.0. Για να δείτε ένα αντίγραφο της άδειας αυτής επισκεφτείτε τον ιστότοπο <https://creativecommons.org/licenses/by-nc-sa/4.0/deed.el>

Αν τυχόν κάποιο τμήμα του έργου διατίθεται με διαφορετικό καθεστώς αδειοδότησης, αυτό αναφέρεται ρητά και ειδικώς στην οικεία θέση.

ΚΑΛΛΙΠΟΣ

Εθνικό Μετσόβιο Πολυτεχνείο

Ηρώων Πολυτεχνείου 9, 15780 Ζωγράφου

www.kallipos.gr

ISBN: 978-618-5667-71-9

Βιβλιογραφική Αναφορά: Παναγιώτου, Γ. (2022). *Μια εισαγωγή στην Python για Μεταλλειολόγους & άλλους Μηχανικούς* [Προπτυχιακό εγχειρίδιο]. Κάλλιπος, Ανοικτές Ακαδημαϊκές Εκδόσεις. <https://dx.doi.org/10.57713/kallipos-99>

Αφιερώνεται στους δασκάλους μου

Πίνακας Περιεχομένων

Πίνακας Περιεχομένων	5
Πρόλογος.....	11
01 Εισαγωγή.....	12
01.1 Γενικά.....	12
01.2 Σημειώσεις για τον αναγνώστη	14
01.3 Διαμοιρασμός κώδικα	15
01.4 Παροράματα και ημαρτημένα.....	15
02 Δονήσεις από ανατινάξεις	16
02.1 Γενικά.....	16
02.2 Μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους	16
02.3 Ασφαλή όρια μεγέθους δονήσεων	17
02.4 Εκτίμηση του μεγέθους των δονήσεων.....	18
03 Η γλώσσα Python	20
04 Καλωσορίσατε στην Python.....	22
04.1 Γενικά.....	22
04.2 Το 1 ^ο στάδιο: Ο τίτλος.....	22
04.3 Το 2 ^ο στάδιο: Οι υπολογισμοί.....	23
04.3.1 Περί μεταβλητών.....	23
04.3.2 Περί αλγεβρικών πράξεων	26
04.3.3 Περί αριθμών.....	27
04.3.4 Υπολογισμοί με μεταβλητές.....	29
04.4 Το 3 ^ο στάδιο: Η παρουσίαση των αποτελεσμάτων	30
04.4.1 Φορμαρισμένα αποτελέσματα.....	31
04.5 Το 4 ^ο στάδιο: Εισαγωγή δεδομένων από το πληκτρολόγιο	33
04.6 Ακρίβεια δεκαδικών αριθμών	35
04.7 Συνοψίζοντας... ..	36
04.8 Ο κώδικας στο τέλος του Κεφαλαίου 04	36
05 Βρόχοι επανάληψης	38
05.1 Γενικά.....	38
05.2 Η εντολή επανάληψης for.....	39
05.3 Η εντολή επανάληψης while	41
05.4 Οι εντολές break και continue	42
05.5 Σύγκριση των εντολών επανάληψης for & while.....	44
05.6 Τελεστές σύγκρισης	44
05.7 Επανάληψη με την εντολή for & την range ()	44
05.8 Ο κώδικας στο τέλος του Κεφαλαίου 05	46
06 Γραφικές παραστάσεις	48

06.1 Γενικά.....	48
06.2 Εγκατάσταση εξωτερικών βιβλιοθηκών	49
06.2.1 Η εντολή <code>import</code>	49
06.3 Διαμορφώνοντας το διάγραμμα	51
06.4 Αποθήκευση αποτελεσμάτων σε αρχείο	53
06.5 Ο κώδικας στο τέλος του Κεφαλαίου 06	54
07 Έλεγχος ροής εκτέλεσης κώδικα.....	57
07.1 Γενικά.....	57
07.2 Οι εντολές <code>if-elif-else</code>	57
07.3 Υπολογισμός της <code>ppn</code> κατά USBM και DIN.....	59
07.4 Αμυντικός προγραμματισμός.....	60
07.5 Διαγράμματα: πολλές μεταβλητές σε ένα διάγραμμα.....	62
07.5.1 Χρώματα	66
07.5.2 Στυλ γραμμών	68
07.5.3 Πάχος γραμμών	68
07.5.4 Σημάδια	68
07.6 Ο κώδικας στο τέλος του Κεφαλαίου 07	70
08 Δομές δεδομένων	72
08.1 Γενικά.....	72
08.2 Πλειάδες.....	72
08.3 Λίστες.....	75
08.3.1 Μέθοδοι στις λίστες	76
08.3.2 Αλλαγές, διαγραφές και προσθήκες στοιχείων	78
08.4 Λεξικά	79
08.4.1 Αντληση δεδομένων από λεξικά	81
08.4.2 Τα δεδομένα ξεχωριστά από τους αλγορίθμους.....	82
08.5 Σύνολα.....	83
08.6 Ο κώδικας στο τέλος του κεφαλαίου 08	84
09 Συναρτήσεις.....	86
09.1 Γενικά.....	86
09.2 Υπολογισμός εμβαδού ακανόνιστου σχήματος	86
09.3 Το <code>return</code>	90
09.4 Παράμετροι και μεταβλητές συνάρτησης	92
09.5 Συναρτήσεις <code>lambda</code>	95
09.6 Η συνάρτηση <code>main()</code>	96
10 Αναδιάρθρωση του κώδικα <code>BLASTppv.py</code>	98
10.1 Γενικά.....	98
10.2 Δημιουργία κωδικού πρόσβασης	98
10.3 Κρυπτογράφηση κωδικού πρόσβασης	100

10.4 Είσοδος του χρήστη στο πρόγραμμα	101
10.5 Καλωσόρισμα του χρήστη	104
10.6 Βάση δεδομένων χρηστών του προγράμματος	105
10.7 Εισαγωγή δεδομένων του προβλήματος	107
10.8 Εκτίμηση της ppn με τη μέθοδο του USBM.....	112
10.9 Εκτίμηση της ppn κατά το πρότυπο DIN.....	113
10.10 Εμφάνιση αποτελεσμάτων	113
10.11 Γραφική παράσταση της σχέσης ppn - απόσταση	114
10.12 Διάγραμμα συμμόρφωσης με τον ΚΜΛΕ.....	115
10.13 Η συνάρτηση <code>main()</code>	120
10.14 Ο κώδικας στο τέλος του Κεφαλαίου 10	120
10.15 Φορητότητα προγραμμάτων Python	129
11 Επίλυση εξισώσεων.....	131
11.1 Γενικά.....	131
11.2 Επίλυση εξισώσεων: βιβλιοθήκη <code>SymPy</code>	131
11.2.1 Εξισώσεις με έναν άγνωστο	132
11.2.2 Συστήματα εξισώσεων	135
11.2.3 Παράγωγοι συναρτήσεων.....	136
11.2.4 Ολοκληρώματα συναρτήσεων	137
11.2.5 Συνήθεις διαφορικές εξισώσεις.....	140
11.3 Επίλυση εξισώσεων: βιβλιοθήκη <code>SciPy</code>	142
11.3.1 Εξισώσεις με έναν άγνωστο	143
11.3.2 Συστήματα εξισώσεων	145
11.3.3 Παράγωγοι συναρτήσεων.....	146
11.3.4 Ολοκληρώματα συναρτήσεων	147
11.3.4.1 Υπολογισμός όγκου εκσκαφών	148
11.3.5 Συνήθεις διαφορικές εξισώσεις.....	151
11.4 Επίλυση εξισώσεων: Μέθοδος της διχοτόμησης	155
11.5 Περί ανοχής λάθους στις αριθμητικές μεθόδους.....	157
12 Επεξεργασία δεδομένων.....	159
12.1 Γενικά.....	159
12.2 Καταγραφή δεδομένων σε αρχεία.....	159
12.1.1 Αρχεία <code>.xlsx</code>	159
12.1.2 Αρχεία <code>.csv</code>	160
12.2 Η βιβλιοθήκη <code>Pandas</code>	166
12.3 Οπτικοποίηση δεδομένων	170
12.4 Καμπύλη προσαρμογής.....	174
12.4.1 Ο συντελεστής συσχέτισης Pearson r	174
12.4.2 Χάραξη της καμπύλης προσαρμογής.....	175

12.4.3 Καμπύλη προσαρμογής και επίπεδο εμπιστοσύνης	177
12.5 Ο κώδικας στο τέλος του Κεφαλαίου 12	180
13 Βελτιστοποίηση.....	184
13.1 Γενικά.....	184
13.2 Ελαχιστοποίηση συνάρτησης μορφής $f(x)$	184
13.3 Βελτιστοποίηση συνάρτησης πολλών μεταβλητών υπό συνθήκες	188
13.3.1 Το προς βελτιστοποίηση πρόβλημα	189
13.3.2 Κατάστρωση του σχεδίου επίλυσης.....	190
13.3.3 Η μέθοδος Simplex: <code>Scipy</code>	190
13.4 Βελτιστοποίηση συνάρτησης ακέραιων μεταβλητών υπό συνθήκες: <code>PuLP</code>	193
13.4.1 Το προς βελτιστοποίηση πρόβλημα	193
13.4.2 Κατάστρωση του σχεδίου επίλυσης.....	194
13.4.3 Βελτιστοποίηση με τον αλγόριθμο CBC: <code>PuLP</code>	195
14 Διαχείριση έργου.....	198
14.1 Γενικά.....	198
14.2 Το προς διαχείριση έργο.....	198
14.3 Δικτυωτός χρονικός προγραμματισμός έργων: CPM	199
14.3.1 Το πρόγραμμα <code>cpm.exe</code>	201
14.3.2 Το πρόγραμμα <code>CPM.py</code>	202
14.4 Δικτυωτός χρονικός προγραμματισμός έργων: PERT	208
14.4.1 Το πρόγραμμα <code>pert.exe</code>	209
14.4.2 Το πρόγραμμα <code>PERT.py</code>	209
14.5 Το διάγραμμα παρακολούθησης εκτέλεσης έργου Gantt	213
15 Χημικοί τύποι ενώσεων.....	219
15.1 Γενικά.....	219
15.2 Γραμματική ανάλυση	219
15.2.1 Το module: <code>PyParsing</code>	219
15.2.2 Το module: <code>Chemparse</code>	221
15.3 Ο κώδικας στο τέλος του Κεφαλαίου 15	221
16 Αντικειμενοστραφής προγραμματισμός	223
16.1 Γενικά.....	223
16.2 Κλάσεις και αντικείμενα	223
16.3 Κληρονομικότητα.....	225
16.4 Ο κώδικας στο τέλος του Κεφαλαίου 16	226
17 Γραφική διεπαφή χρήστη	228
17.1 Γενικά.....	228
17.2 Η βιβλιοθήκη <code>Tkinter</code>	228
17.3 Πλάνο διαμόρφωσης του κώδικα <code>BLASTppvGUI.py</code>	229
17.4 Το βασικό GUI παράθυρο.....	230

17.5 Το GUI παράθυρο του BLASTppvGUI .py	233
17.5.1 Το widget Label.....	242
17.5.2 Το widget Entry.....	242
17.5.3 Το widget Radiobutton	243
17.5.4 Το widget Scale.....	244
17.5.5 Το widget Button	245
17.5.6 Τα widgets Canvas & PhotoImage	246
17.5.7 Το widget simpledialog.askfloat	250
17.5.8 Η βιβλιοθήκη fpdf	252
17.5.9 Η βιβλιοθήκη os	256
17.5.10 Το widget messagebox.showerror.....	257
17.6 Επιλογή χρωμάτων με τη βοήθεια της Tkinter	258
17.7 Τα megawidgets της βιβλιοθήκης Pmw	261
18 Προσομοίωση συστημάτων	263
18.1 Γενικά.....	263
18.2 Περί προσομοίωσης	263
18.3 Στοχαστική προσομοίωση διακριτών γεγονότων.....	265
18.4 Το πείραμα της βελόνας του Buffon.....	265
18.5 Γεννήτριες ψευδο-τυχαίων αριθμών	268
18.6 Στατιστικές κατανομές.....	270
18.7 Προσαρμογή δεδομένων σε κατανομή: fitter.....	270
18.8 Η βιβλιοθήκη Salabim.....	276
18.9 Προσομοίωση συστήματος φόρτωσης.....	277
18.9.1 Προσομοίωση του φορτωτή ως component	278
18.9.2 Προσομοίωση του φορτωτή ως resource.....	285
18.9.3 Προσομοίωση της φόρτωσης με πολλούς φορτωτές	290
18.10 Προσομοίωση συστήματος φόρτωσης-μεταφοράς.....	291
18.11 Animation συστήματος φόρτωσης-μεταφοράς	297
18.12 Οι στατιστικές κατανομές της βιβλιοθήκης Salabim	305
18.12.1 Η χρήση της κανονικής κατανομής στην προσομοίωση.....	306
18.13 Τεχνικές προσομοίωσης συστημάτων.....	307
19 Δεδομένα από το Web.....	312
19.1 Γενικά.....	312
19.2 Άδεια λήψης δεδομένων από το Open Weather Map	312
19.3 Ανάπτυξη της εφαρμογής.....	312
20 Περί πυθωνικής μουσικής.....	320
20.1 Γενικά.....	320
20.2 Ταλαντώσεις και ήχοι.....	321

20.3 Ο ήχος των πλήκτρων ενός πιάνου	323
20.4 Σύνθεση μουσικής με τη βιβλιοθήκη <code>MIDIUtil</code>	326
20.5 Η Python στη μουσική	328
21 Για το τέλος	329
22 Βιβλιογραφία	331

Πρόλογος

Ο κύριος σκοπός αυτού του βιβλίου είναι να εισάγει τους σπουδαστές της Σχολής Μηχανικών Μεταλλείων – Μεταλλουργών του Εθνικού Μετσόβιου Πολυτεχνείου, αλλά και άλλων Σχολών Μηχανικών, στον προγραμματισμό των ηλεκτρονικών υπολογιστών χρησιμοποιώντας τη γλώσσα προγραμματισμού Python, η οποία είναι εγκαταστημένη σε υπολογιστές που λειτουργούν με το λειτουργικό σύστημα Windows 10 της Microsoft.

Το βιβλίο είναι ανοικτής πρόσβασης (open-access) και εκδίδεται με άδεια Creative Commons από τη Δράση Κάλλιπος [<https://www.kallipos.gr>] για τους ακόλουθους λόγους:

- Ο κώδικας των προγραμμάτων που περιέχονται στο βιβλίο αναπτύχθηκε χρησιμοποιώντας λογισμικό ανοικτής πρόσβασης.
- Χρησιμοποιήθηκαν, επίσης, πακέτα λογισμικού είτε ανοικτής πρόσβασης είτε με δωρεάν εκχώρηση χρήσης, με μοναδική εξαίρεση το Microsoft Excel.
- Η τεκμηρίωση του περιεχομένου του βιβλίου και η βιβλιογραφία που χρησιμοποιήθηκε, με δύο μόνο εξαιρέσεις, βασίζονται σε πηγές ανοικτής πρόσβασης, οι οποίες είναι όλες διαθέσιμες στο Διαδίκτυο.

Έτσι, ο συγγραφέας ακολούθησε το *δωρεάν έλαβετε, δωρεάν δότε*.

Ευχαριστίες εκφράζονται προς τους συνεργάτες-συναδέλφους μου στο Εργαστήριο Εξόρυξης Πετρωμάτων του Εθνικού Μετσόβιου Πολυτεχνείου, Μάνο Τσιαβό, Μαρία Μπασάνου, Κατερίνα Αδάμ και Θόδωρο Μιχαλακόπουλο, οι οποίοι διάβασαν το σύνολο ή επιμέρους κεφάλαια του βιβλίου και έκαναν εύστοχες παρατηρήσεις, καθώς και προς τους Δημήτρη Καλλιάρια για τη γλωσσική επιμέλεια και Αντώνη Κοκολάκη για τη γραφιστική επιμέλεια της έκδοσης του βιβλίου.

Καθηγητής Γιώργος Ν. Παναγιώτου

Αθήνα, Μάρτιος 2022

01 Εισαγωγή

01.1 Γενικά

Στο εύλογο ερώτημα, γιατί ο σπουδαστής της Σχολής Μηχανικών Μεταλλείων – Μεταλλουργών (ΣΜΜΜ) να εκπαιδεύεται στον προγραμματισμό των ηλεκτρονικών υπολογιστών, την απάντηση δίνουν τα παρακάτω τρία αποσπάσματα κειμένου (στη γλώσσα του πρωτοτύπου):

1. “More individuals, employed in an increasing range of activity, are becoming involved with computers in their daily work, mining engineers and metallurgists included.”
2. “The aim is not to produce an expert in the computing field. The undergraduates are being educated in the technologies of mining and metallurgy to which most of the student’s time and effort must be devoted. However, these technologies use computers, and the emphasis should therefore be on the applications of computers.”
3. “The new graduate should have just sufficient knowledge of computers to use them as a tool, to know the limits of what he or she can achieve without specialized assistance, and to know what can be achieved with specialized assistance in the computing field.”

που περιλαμβάνονται στο άρθρο του A.M. Henderson “*The Role of Computers in Preparing Graduates for the Mineral Industry*”, το οποίο παρουσιάστηκε στο συνέδριο APCOM 77 (15th International Symposium on the Application of Computers and Operations Research in the Mineral Industries, Brisbane, Australia, 4-8 July 1977).

Οι ίδιοι λόγοι, οι οποίοι γράφτηκαν το 1977, όταν ο συγγραφέας ήταν τεταρτοετής σπουδαστής στη ΣΜΜΜ, ισχύουν και σήμερα!

Και βέβαια ισχύουν, τροποποιημένοι κατά περίπτωση, και για τους σπουδαστές Σχολών Μηχανικών άλλων ειδικοτήτων.

Στην αγορά αλλά και στο Διαδίκτυο υπάρχουν πολλά και καλά βιβλία που παρουσιάζουν-διδάσκουν τη γλώσσα Python για προγραμματιστές όλων των επιπέδων, από αρχάριους έως πολύ πεπειραμένους.

Λίγα όμως είναι εκείνα τα οποία δομούν την παρουσίαση της Python μέσα από την ανάπτυξη προγραμμάτων που επιλύουν πραγματικά προβλήματα Μηχανικού. Ούτε ένα από αυτά δεν ασχολείται με αντικείμενα της ειδικότητας του Μεταλλειολόγου Μηχανικού.

Το βιβλίο απευθύνεται τόσο στον πρωτοετή σπουδαστή όσο και στον Μηχανικό της πράξης, χωρίς να είναι απαραίτητη προηγούμενη εμπειρία τους στον προγραμματισμό υπολογιστών, αν και είναι καλοδεχούμενη.

Στο πρώτο μέρος του βιβλίου (Κεφάλαια 02 έως 10), η εισαγωγή στον προγραμματισμό γίνεται σταδιακά, ακολουθώντας τα βήματα που πρέπει να γίνουν για να επιλυθεί ένα πραγματικό μεταλλευτικό πρόβλημα, γράφοντας τμηματικά τον κώδικα ενός προγράμματος στην Python, που θα αναπτυχθεί για τον σκοπό αυτό.

Το μεταλλευτικό πρόβλημα αναφέρεται στην εκτίμηση των επιπτώσεων των δονήσεων του εδάφους, οι οποίες δημιουργούνται στον περιβάλλοντα χώρο ενός υπαίθριου μεταλλείου, λατομείου ή τεχνικού έργου, κατά την εξόρυξη των πετρωμάτων με χρήση εκρηκτικών υλών, δηλαδή με ανατινάξεις.

Η μελέτη του προβλήματος των επιπτώσεων των ανατινάξεων στον περιβάλλοντα χώρο είναι αντικείμενο μαθήματος που, συνήθως, διδάσκεται σε μεγαλύτερο εξάμηνο απ’ ότι το εξάμηνο που διδάσκεται ο προγραμματισμός ηλεκτρονικών υπολογιστών. Έτσι, για τις ανάγκες της ύλης αυτού του βιβλίου, παρουσιάζονται στο Κεφάλαιο 02 *Δονήσεις από ανατινάξεις* όλα τα στοιχεία της σχετικής θεωρίας των ανατινάξεων, τα οποία απαιτούνται για την υπολογιστική αντιμετώπιση του προβλήματος των δονήσεων του εδάφους.

Με την προσέγγιση αυτή δίνεται, επίσης, στον σπουδαστή των *μικρών εξαμήνων* η ευκαιρία να αντιληφθεί ότι ο προγραμματισμός των υπολογιστών είναι ένα εξαιρετικά χρήσιμο εργαλείο για την επίλυση *πραγματικών* προβλημάτων, αντίστοιχα εκείνων τα οποία θα αντιμετωπίσει στην καριέρα του ως Μηχανικός.

Όταν θα φθάσει στο τέλος του Κεφαλαίου 10 *Αναδιάρθρωση του κώδικα BLASTprn.py*, ο αναγνώστης θα έχει μάθει να βασικά στοιχεία και τις τεχνικές προγραμματισμού της γλώσσας Python και θα έχει αναπτύξει τον κώδικα ενός προγράμματος το οποίο:

- Θα δέχεται τις τιμές των παραμέτρων εισόδου (input data) που απαιτούνται για την επίλυση του προβλήματος,
- Θα κάνει τους υπολογισμούς, με βάση όσα προβλέπονται στη σχετική με το πρόβλημα μεταλλευτική θεωρία, μέσω αλγορίθμων,
- Θα εμφανίζει τα αριθμητικά αποτελέσματα της επίλυσης του προβλήματος (output),
- Θα κατασκευάζει χρήσιμα για τον Μηχανικό διαγράμματα,
- Θα μπορούν να το χρησιμοποιούν μόνο οι χρήστες που γνωρίζουν τον κωδικό πρόσβασης σε αυτό (password protected), οι οποίοι και θα καταχωρούνται σε μια βάση δεδομένων.

Το δεύτερο μέρος του βιβλίου (Κεφάλαια 11 έως 20), με το να πραγματεύεται αυτοτελή αντικείμενα θεματολογίας Μηχανικού, αποσκοπεί στη διεύρυνση των γνώσεων του αναγνώστη, με επιπλέον στοιχεία της γλώσσας Python, με τεχνικές προγραμματισμού και κυρίως με τη χρήση εξειδικευμένων βιβλιοθηκών, εσωτερικών και εξωτερικών (third-party), της Python.

Από αυτά τα αντικείμενα, ορισμένα είναι γενικού ενδιαφέροντος, όπως η επίλυση εξισώσεων, συστημάτων εξισώσεων, διαφορικών εξισώσεων και ολοκληρωμάτων, καθώς και η κατασκευή της γραφικής διεπαφής υπολογιστή-χρήστη.

Υπάρχουν όμως και Κεφάλαια που πραγματεύονται εξειδικευμένα αντικείμενα, όπως την επεξεργασία δεδομένων που προκύπτουν από μετρήσεις μεγεθών, τη βελτιστοποίηση και τον γραμμικό προγραμματισμό, τη διαχείριση έργου και την προσομοίωση συστημάτων. Αυτά απευθύνονται, κυρίως, στους σπουδαστές των μεγαλύτερων εξαμήνων που θα διδαχθούν την ύλη των αντικειμένων αυτών σε συναφή μαθήματα της Σχολής τους ή στους Μηχανικούς που θέλουν να τα εφαρμόσουν στην πράξη.

Το τελευταίο Κεφάλαιο του βιβλίου δίνει τη δυνατότητα στον αναγνώστη να μετατρέψει τον υπολογιστή του σε μουσικό όργανο και να συνθέσει τη δική του μουσική με τη βοήθεια της Python.

Από διδακτικής άποψης, το βιβλίο ακολουθεί τις αρχές της *μάθησης μέσω της εξάσκησης*, με μια διαδικασία η οποία στηρίζεται στον *εποικοδομητισμό* ή *οικοδομισμό* (constructivism) και υποστηρίζει ότι οι άνθρωποι δημιουργούν γνώση και νοήματα από μια αλληλεπίδραση μεταξύ των εμπειριών τους και των ιδεών τους [βλέπε: [https://el.wikipedia.org/wiki/Οικοδομισμός_\(παιδαγωγική\)](https://el.wikipedia.org/wiki/Οικοδομισμός_(παιδαγωγική))].

Στο βιβλίο αυτό, η διδασκαλία του προγραμματισμού των υπολογιστών με χρήση της γλώσσας Python γίνεται *μέσω της επίλυσης προβλημάτων* (problem based learning) και έτσι τα στοιχεία της γλώσσας Python δεν παρουσιάζονται με τον συνήθη και *εξαντλητικό* τρόπο, που συναντάμε στα περισσότερα εγχειρίδια της γλώσσας, *αλλά εκεί που πρέπει να χρησιμοποιηθούν* (Just-in-Time) και στην *έκταση του απαιτείται* για την επίλυση του προβλήματος.

Τα προβλήματα που χρησιμοποιούνται για την εκμάθηση της Python είναι προβλήματα Μηχανικού. Έτσι ο σπουδαστής δεν θα *εξαντληθεί* με παραδείγματα από τον χώρο των μαθηματικών, που είναι τόσο δημοφιλή σε εγχειρίδια όλων των γλωσσών προγραμματισμού. Παραδείγματα με αντικείμενα τη συνάρτηση Fibonacci, την εύρεση των πρώτων αριθμών, το κόσκινο του Ερατοσθένη ή τη μετατροπή των δεκαδικών αριθμών σε δυαδικούς και τούμπαλιν, δεν υπάρχουν σε αυτό το βιβλίο.

Το βιβλίο, επειδή γράφτηκε από Μεταλλειολόγο Μηχανικό ο οποίος χρησιμοποιεί γλώσσες προγραμματισμού για την επίλυση μεταλλευτικών προβλημάτων επί 45 σχεδόν χρόνια, έχει περισσότερο πρακτικό χαρακτήρα.

Για τον λόγο αυτό, δεν θα πρέπει να θεωρηθεί κείμενο που πραγματεύεται το αντικείμενο της *Επιστήμης των Υπολογιστών* ή *Εγχειρίδιο της Γλώσσας Προγραμματισμού Python*. Υπάρχει πληθώρα καλών βιβλίων, στο εμπόριο και στο Διαδίκτυο, στα οποία μπορεί να ανατρέξει ο αναγνώστης που ενδιαφέρεται να εμβαθύνει στα αντικείμενα αυτά.

Κατά τη συγγραφή του βιβλίου, επειδή η φιλοδοξία του συγγραφέα είναι να αποτελέσει αυτό ένα *Εγχειρίδιο Μηχανικού*, καταβλήθηκε προσπάθεια ώστε το κείμενο να έχει σαφήνεια και ακρίβεια, θυσιάζοντας, σε πολλές περιπτώσεις, το όποιο *γλωσσικό κάλλος* διαθέτει ο συγγραφέας του.

Εν τέλει, ο σκοπός αυτού του βιβλίου είναι πολύ ταπεινός: να παρουσιάσει με απλό τρόπο τις βασικές έννοιες και τεχνικές προγραμματισμού της γλώσσας Python, για να μπορεί ο αναγνώστης του να γράψει, γρήγορα και εύκολα, χρήσιμα προγράμματα, μια εμπειρία που θα είναι ένα εξαιρετικό εφόδιο στην καριέρα του ως Μηχανικός.

01.2 Σημειώσεις για τον αναγνώστη

1. Ορισμένοι όροι που χρησιμοποιούνται στον προγραμματισμό με τη γλώσσα Python παρουσιάζονται στο κείμενο στην αγγλική γλώσσα, γιατί έτσι είναι πιο κατανοητοί (καθιερωμένοι) μεταξύ των μελών της κοινότητας των προγραμματιστών της Python, των Pythonistas.
2. Επίσης, χρησιμοποιούνται και τα Greeklish...
3. Τα στοιχεία της γλώσσας Python (εντολές, μεταβλητές, συναρτήσεις, παράμετροι κ.ά.) καθώς και τα URL (web addresses) των διαδικτυακών πηγών, όταν εμφανίζονται εντός του κυρίως σώματος του κειμένου είναι γραμμένα με γραμματοσειρά Courier.
4. Στις εξισώσεις και συναρτήσεις ως σύμβολο του πολλαπλασιασμού χρησιμοποιείται ο αστερίσκος * και της διαίρεσης ή κλάσματος το /.
5. Στους αριθμούς, για τον διαχωρισμό του δεκαδικού μέρους χρησιμοποιείται η τελεία (.) και το κόμμα (,) για τον διαχωρισμό των χιλιάδων.
6. Εξισώσεις και συναρτήσεις που επαναλαμβάνονται στο κείμενο διατηρούν την αρχική τους αρίθμηση.
7. Λέξεις ή προτάσεις στις οποίες ο συγγραφέας θέλει να δώσει έμφαση είναι γραμμένες με *πλάγια γραφή* (italic).
8. Ο κώδικας (script) των προγραμμάτων της Python, που ενσωματώνεται στο κείμενο του βιβλίου, ακολουθεί τις χρωματικές επιλογές του Integrated Development Environment (IDE) PyCharm Pro, στο οποίο και αναπτύχθηκε, πριν γίνει η μεταφορά του στο κείμενο.
9. Ό,τι εμφανίζεται στην οθόνη του υπολογιστή κατά το τρέξιμο του κώδικα που περιλαμβάνεται στο βιβλίο, δηλαδή τα δεδομένα εισόδου (input) και τα αποτελέσματα (output), στο κείμενο παρουσιάζονται εντός έγχρωμου γαλάζιου πλαισίου. Τον κανόνα αυτό δεν ακολουθούν τα γραφικά που δημιουργούνται από τον κώδικα, καθώς και οι αποτυπώσεις (print screen) της οθόνης.
10. Στο κείμενο του βιβλίου και στα σημεία, στα οποία αυτό κρίνεται σκόπιμο, γίνεται πολύ συχνά παραπομπή σε διαδικτυακούς τόπους (websites), των οποίων οι διευθύνσεις (URL) ήταν ενεργές τον Μάρτιο 2022. Εάν στο διάβα του χρόνου αυτές αλλάξουν, μια έρευνα στο Διαδίκτυο θα εντοπίσει τη νέα τους θέση.

11. Τα Κεφάλαια του βιβλίου διαβάζονται με τη σειρά από το Κεφάλαιο 02 έως το Κεφάλαιο 10. Τα υπόλοιπα Κεφάλαια είναι γενικώς ανεξάρτητα μεταξύ τους, αλλά κάνουν συχνές αναφορές σε προηγούμενα Κεφάλαια.

01.3 Διαμοιρασμός κώδικα

Ο κώδικας των προγραμμάτων που περιλαμβάνονται στο βιβλίο έχει δοκιμαστεί και τρέχει στην έκδοση (release) Python 3.7.9.

Τα αρχεία του κώδικα είναι διαθέσιμα, μαζί με τα συνοδευτικά αρχεία που είναι απαραίτητα για την εκτέλεσή του, στο website του βιβλίου.

Στο ίδιο website ο αναγνώστης θα βρει και όλα τα άλλα αρχεία που μνημονεύονται στο βιβλίο.

Τα αρχεία είναι ταξινομημένα ανά Κεφάλαιο και περιέχονται στο αντίστοιχο directory του Κεφαλαίου.

01.4 Παροράματα και ημαρτημένα

Τα λάθη είναι για τους ανθρώπους...

Ο συγγραφέας θα εκτιμούσε ιδιαίτερα εάν ο αναγνώστης του υποδείξει τυχόν λάθη, στο κείμενο ή τους κώδικες των προγραμμάτων, στέλνοντας email στο: panagiotou@metal.ntua.gr

02 Δονήσεις από ανατινάξεις

02.1 Γενικά

Κατά την εξόρυξη των πετρωμάτων με χρήση εκρηκτικών υλών (ανατίναξη πετρωμάτων), η χημική ενέργεια της εκρηκτικής ύλης που αποτελεί τη γόμωση των διατρημάτων της ανατίναξης, μετατρέπεται σε μηχανική. Τη θέση της ανατίναξης την ονομάζουμε *μέτωπο ανατίναξης*.

Ένα μέρος αυτής της μηχανικής ενέργειας αναλώνεται για τη θραύση και για τη μετακίνηση του όγκου του πετρώματος που θα εξορυχθεί. Η υπόλοιπη ενέργεια, το ποσοστό της οποίας εξαρτάται κυρίως από τα χαρακτηριστικά του σχεδίου της ανατίναξης, διαχέεται στον χώρο που περιβάλλει το μέτωπο ανατίναξης και εκδηλώνεται, κυρίως, υπό τη μορφή *δονήσεων του εδάφους και αέριου ωστικού κύματος*.

Οι δονήσεις του εδάφους είναι το αποτέλεσμα της διάδοσης ελαστικών τασικών κυματοπαλμών στη μάζα του πετρώματος και στην επιφάνεια του εδάφους. Χαρακτηρίζονται, κυρίως, από τα μεγέθη της *μέγιστης ταχύτητας δόνησης σωματιδίων του εδάφους* (peak particle velocity – ppv) και της *συχνότητας* (frequency) του κυματοπαλμού.

Το αέριο ωστικό κύμα είναι και αυτό ένας κυματοπαλμός, που διαδίδεται στον ατμοσφαιρικό αέρα και χαρακτηρίζεται από τα μεγέθη της *υπερπίεσης* (overpressure) και της *συχνότητας*. Σε αυτό θα αναφερθούμε στο τέλος του βιβλίου, στο Κεφάλαιο 20 *Περί πυθωνικής μουσικής*.

02.2 Μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους

Η μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους είναι δυνατό να εκτιμηθεί πριν την πραγματοποίηση μιας ανατίναξης, αλλά και να μετρηθεί, με τη βοήθεια δονησιογράφων, σε σημεία ενδιαφέροντος κατά την εκτέλεση της ανατίναξης. Οι δονησιογράφοι καταγράφουν, επίσης, τη συχνότητα των κυματοπαλμών των δονήσεων και την υπερπίεση και συχνότητα του αέριου ωστικού κύματος.

Από εκτεταμένη έρευνα που πραγματοποιήθηκε, κυρίως από το Γραφείο Μεταλλείων των ΗΠΑ (USBM), διαπιστώθηκε ότι η μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (ppv), που αναμένεται σε ένα σημείο, το οποίο απέχει από το μέτωπο ανατίναξης απόσταση (*d*), και η ποσότητα της εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα (*w*), συνδέονται με τη σχέση:

$$ppv = h * (d / \sqrt{w})^b \quad [2.1]$$

όπου:

ppv : μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (mm/s)

d : απόσταση από το μέτωπο ανατίναξης στο σημείο ενδιαφέροντος (m)

w : μέγιστη ποσότητα εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα (kg)

h : h-factor, συντελεστής που εξαρτάται από τον τύπο της ανατίναξης

b : b-factor, συντελεστής που εξαρτάται από τις γεωλογικές/τοπογραφικές συνθήκες μεταξύ του μετώπου ανατίναξης και του σημείου ενδιαφέροντος.

Ο Πίνακας 2.1 παρουσιάζει τις τιμές των h-factor και b-factor για ορισμένους τύπους ανατινάξεων.

Πίνακας 2.1. Τιμές των h-factor και b-factor ανατινάξεων σε υπαίθρια μέτωπα.

Τύπος ανατίναξης	h-factor	b-factor
Μεταλλεία	1140	-1.60
Λατομεία	1090	-1.82
Τεχνικά Έργα	1730	-1.60

Τη μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (prv), μπορούμε επίσης να την εκτιμήσουμε σύμφωνα με το Γερμανικό Πρότυπο DIN 4150-1:2001, λαμβάνοντας υπόψη το είδος του πετρώματος, που αποτελεί το υπέδαφος μεταξύ του μετώπου ανατίναξης και του σημείου ενδιαφέροντος, από τις σχέσεις:

$$\text{Ιζηματογενή Πετρώματα: } prv = 969 * L^{0.6} * R^{-1.5} \quad [2.2]$$

$$\text{Πυριγενή Πετρώματα: } prv = 206 * L^{0.8} * R^{-1.3} \quad [2.3]$$

όπου:

prv : μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (mm/s),

L : ποσότητα εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα (kg),

R : απόσταση από το μέτωπο ανατίναξης στο σημείο ενδιαφέροντος (m).

02.3 Ασφαλή όρια μεγέθους δονήσεων

Προκειμένου να εξασφαλιστεί ότι οι δονήσεις του εδάφους από τις ανατινάξεις δεν θα προκαλέσουν ζημιές σε παρακείμενες κατασκευές, έχουν θεσπιστεί από τις αρχές των χωρών κανονισμοί, οι οποίοι ορίζουν τα κατά περίπτωση επιτρεπόμενα όρια της μέγιστης ταχύτητας δόνησης σωματιδίων του εδάφους στη θέση της κατασκευής.

Ένας από τους πλέον αποδεκτούς, διεθνώς, κανονισμούς είναι αυτός του Γερμανικού Προτύπου DIN 4150-3/1999 και της νεότερης έκδοσής του, DIN 4150-3/2016, ο οποίος προσδιορίζει το μέγιστο επιτρεπόμενο όριο της μέγιστης ταχύτητας δόνησης των σωματιδίων του εδάφους (prv) στη θέση μιας κατασκευής, ως συνάρτηση της συχνότητας (frequency) του κυματοπαλμού της δόνησης και του είδους της κατασκευής.

Στη χώρα μας, ο σχετικός κανονισμός που ισχύει αναφέρεται στο άρθρο 88 του Κανονισμού Μεταλλευτικών και Λατομικών Εργασιών και υιοθετεί το Γερμανικό Πρότυπο DIN 4150-3/1999.

Ειδικότερα, ο Κανονισμός Μεταλλευτικών και Λατομικών Εργασιών (ΚΜΛΕ), που δημοσιεύθηκε στο ΦΕΚ Β' 1227/14.6.2011, στο ΚΕΦΑΛΑΙΟ ΙΧ Επιφανειακές εκσκαφές – Ασφάλεια επιφάνειας – Προστασία περιβάλλοντος και στο άρθρο 88 Προστασία περιβάλλοντος χώρου από τις δονήσεις, το ωστικό κύμα των εκρήξεων και τους θορύβους, μεταξύ άλλων, αναφέρει:

«Η μέγιστη ταχύτητα δονήσεως των σωματιδίων του εδάφους στη θέση του κτίσματος (σε mm/sec) σε συνάρτηση με τη συχνότητα και ανάλογα με το είδος της κατασκευής δεν πρέπει να υπερβαίνει τα όρια που καθορίζονται στον παρακάτω πίνακα, σε συνδυασμό με το διάγραμμα που ακολουθεί (σύμφωνα με το γερμανικό πρότυπο DIN 4150-3/1999).»

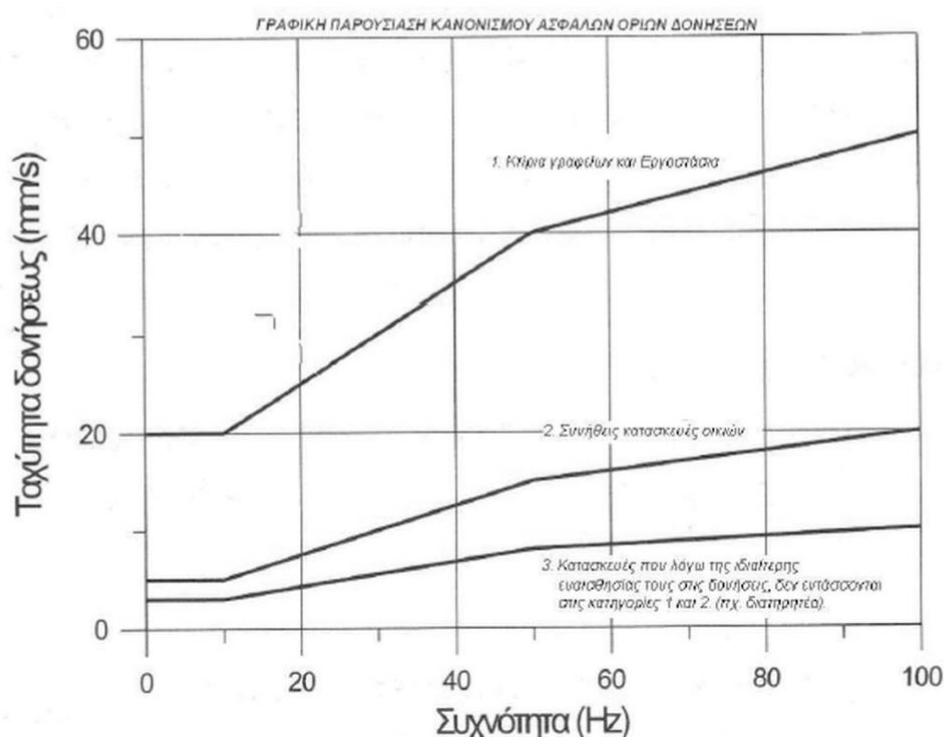
Ο πίνακας και το διάγραμμα που αναφέρονται στο άρθρο 88 του ΚΜΛΕ παρουσιάζονται στην Εικόνα 2.1.

Σημειώστε ότι οι μέγιστες επιτρεπόμενες τιμές ταχύτητας δόνησης σωματιδίων του εδάφους που προβλέπονται στο Πρότυπο DIN 4150-3/1999 διατηρούνται ίδιες και στο εν ισχύ σήμερα Πρότυπο DIN 4150-3/2016.

Σημείωση: Μπορείτε να κατεβάσετε το κείμενο του ΚΜΛΕ από το website του Εθνικού Τυπογραφείου:
https://www.et.gr/api/DownloadFeksApi/?fek_pdf=20110201227

ΠΙΝΑΚΑΣ
ΜΕΓΙΣΤΩΝ ΕΠΙΤΡΕΠΟΜΕΝΩΝ ΤΙΜΩΝ ΤΗΣ ΤΑΧΥΤΗΤΑΣ ΔΟΝΗΣΗΣ ΤΩΝ ΣΩΜΑΤΙΔΙΩΝ
ΤΟΥ ΕΔΑΦΟΥΣ ΣΕ ΣΥΝΑΡΤΗΣΗ ΜΕ ΤΗ ΣΥΧΝΟΤΗΤΑ, ΓΙΑ ΔΙΑΦΟΡΑ ΕΙΔΗ ΚΑΤΑΣΚΕΥΩΝ

Είδος Κατασκευής	Ταχύτητα δόνησεως (mm/sec)			
	<10 Hz	10 - 50 Hz	50 - 100 Hz	>100Hz
1. Κτίρια γραφείων και Εργοστάσια	20	20-40	40-50	50
2. Συνήθεις κατασκευές οικιών	5	5-15	15-20	20
3. Κατασκευές που λόγω της ιδιαίτερης ευαισθησίας τους στις δονήσεις δεν εντάσσονται στις κατηγορίες 1 και 2 (π.χ. διατηρητέα)	3	3-8	8-10	10



Εικόνα 2.1. Πίνακας και διάγραμμα μέγιστων επιτρεπόμενων τιμών της ταχύτητας δόνησης σωματιδίων του εδάφους σε συνάρτηση με τη συχνότητα, για διάφορα είδη κατασκευών [Πηγή: Υ.Α. Δ7/Α/οικ. 12050/2223/2011 (Φ.Ε.Κ. 1227/Β' 14.6.2011)].

02.4 Εκτίμηση του μεγέθους των δονήσεων

Όταν εκπονούμε την Τεχνική Μελέτη για την εκμετάλλευση ενός υπαίθριου μεταλλείου ή λατομείου καθώς και για την εκτέλεση ενός τεχνικού έργου, όπου προβλέπεται η εξόρυξη πετρωμάτων με χρήση εκρηκτικών υλών, πρέπει να συμπεριλάβουμε, μεταξύ των άλλων, και τη μελέτη του σχεδίου των ανατινάξεων. Για το σχέδιο των ανατινάξεων που θα προτείνουμε είμαστε υποχρεωμένοι:

- να εκτιμήσουμε το μέγεθος των δονήσεων που θα δημιουργηθούν από την ανατίναξη και στη συνέχεια,
- να ελέγξουμε εάν το μέγεθος των δονήσεων είναι εντός των επιτρεπόμενων ορίων που προβλέπονται από τον ΚΜΛΕ για τις κατασκευές οι οποίες βρίσκονται περίξ του μεταλλείου, λατομείου ή τεχνικού έργου.

Μόνο όταν οι δονήσεις είναι εντός των ορίων που προβλέπονται από τον ΚΜΛΕ οι αρχές θα μας δώσουν άδεια εκμετάλλευσης ή εκτέλεσης του έργου.

Επειδή κατά τη διάρκεια της λειτουργίας ενός μεταλλείου-λατομείου ή της εκτέλεσης ενός τεχνικού έργου, τα σχέδια ανατίναξης και η θέση του μετώπου διαρκώς μεταβάλλονται ή και νέες κατασκευές ανεγείρονται στον περιβάλλοντα χώρο, οι παραπάνω υπολογισμοί πρέπει να επαναλαμβάνονται με νέα δεδομένα.

Θα ήταν, επομένως, χρήσιμο για τον μηχανικό ανατινάξεων η ανάπτυξη ενός προγράμματος υπολογιστή, που θα αυτοματοποιούσε τους σχετικούς υπολογισμούς, μειώνοντας τον χρόνο εκτέλεσης των πράξεων και ταυτόχρονα την πιθανότητα λαθών κατά την πληκτρολόγηση στην αριθμομηχανή-calculator.

Το πρόγραμμα θα έπαιρνε ως δεδομένα εισόδου τις παραμέτρους του σχεδίου της ανατίναξης και του κυματοπαλμού της δόνησης, καθώς και τις θέσεις και τα είδη των κατασκευών, θα εφάρμοζε τους αλγορίθμους των εξισώσεων που εκτιμούν τη μέγιστη ταχύτητα δόνησης των σωματιδίων του εδάφους (*ρρν*) και θα έλεγχε το αποτέλεσμα των υπολογισμών με τα επιτρεπόμενα όρια του Πίνακα/Διαγράμματος του ΚΜΛΕ που παρουσιάζονται στην Εικόνα 2.1.

Εάν το πρόγραμμα, εκτός από το να του δίνει το αποτέλεσμα, του έκανε και μια παραμετρική ανάλυση, δίνοντάς του πίνακες με τη μεταβολή της μέγιστης ταχύτητας δόνησης των σωματιδίων του εδάφους για διαφορετικές αποστάσεις και ποσότητες εκρηκτικών υλών ή ακόμα καλύτερα, εάν δημιουργούσε τα αντίστοιχα διαγράμματα μεταβολής, τότε το όφελος που θα είχε στην εργασία του θα έκανε γρήγορα την *απόσβεση* του κόπου και του χρόνου που αφιέρωσε για την ανάπτυξη του προγράμματος.

Ένα τέτοιο πρόγραμμα θα αναπτύξουμε στα επόμενα Κεφάλαια του βιβλίου!

03 Η γλώσσα Python

Η γλώσσα Python είναι μια σύγχρονη γλώσσα προγραμματισμού υψηλού επιπέδου, ανοικτού κώδικα (open source) και γενικής χρήσης.

Αναπτύχθηκε από τον Guido van Rossum, στις αρχές της δεκαετίας του '90 (Version 1.0 Ιανουάριος 1994) και το όνομά της προέρχεται από την κωμική σειρά της τηλεόρασης του BBC *Monty Python's Flying Circus*, γιατί ο van Rossum, λάτρης της σειράς, ήθελε να της δώσει ένα σύντομο, μοναδικό και ελαφρώς μυστηριώδες όνομα.

Η Python είναι εύκολη στην εκμάθηση, είναι συνοπτική, δηλαδή με λίγες γραμμές κώδικα προγραμματίζονται εργασίες για τις οποίες σε άλλες γλώσσες θα απαιτούνταν πολύ περισσότερες γραμμές κώδικα, και παρέχει ισχυρές προγραμματιστικές δυνατότητες, που μπορούν να τις αξιοποιήσουν τόσο οι αρχάριοι όσο και οι έμπειροι προγραμματιστές.

Ένα από τα πλεονεκτήματα της γλώσσας είναι η ύπαρξη εξαιρετικά μεγάλου αριθμού έτοιμων βιβλιοθηκών, εσωτερικών ή εξωτερικών (third-party), που διευκολύνουν τον προγραμματισμό για την εκτέλεση ειδικών εργασιών, όπως είναι οι γραφικές παραστάσεις, υπολογισμοί μαθηματικοί και στατιστικοί, διαχείριση βάσεων δεδομένων, προσομοίωση, web-development κ.ά.

Όντας γλώσσα ανοικτού κώδικα, ο interpreter της Python παρέχεται δωρεάν στους χρήστες, όπως και οι βιβλιοθήκες της. Η διαχείριση και υποστήριξη της γλώσσας γίνεται από τον μη κερδοσκοπικό οργανισμό Python Software Foundation [<https://www.python.org>]. Από αυτό το website μπορείτε να την κατεβάσετε (download) και να την εγκαταστήσετε στον υπολογιστή σας. Εκεί επίσης θα βρείτε για να κατεβάσετε, πάντα δωρεάν, και εξαιρετικά βιβλία εκμάθησης της Python!

Εκδόσεις της Python είναι διαθέσιμες για τα λειτουργικά συστήματα, Windows, Unix/Linux, Mac OS X, κ.ά., αλλά στο βιβλίο αυτό θα χρησιμοποιήσουμε την έκδοση της Python για Windows 10.

Στο website της Python Software Foundation θα βρείτε τις διάφορες εκδόσεις (versions) της Python για Windows 10 οι οποίες είναι διαθέσιμες για download. Η πρόταση του συγγραφέα είναι να επιλέξετε μια stable έκδοση η οποία να βρίσκεται σε maintenance status *security* και όχι *bugfix*.

Σημειώστε, επίσης, ότι ορισμένες εξωτερικές βιβλιοθήκες μπορεί να μην υποστηρίζονται, τη στιγμή που θέλετε να τις χρησιμοποιήσετε, από τις νεότερες εκδόσεις της Python. Ο κώδικας που περιλαμβάνεται στο βιβλίο αυτό είναι κατάλληλος, δηλαδή έχει δοκιμαστεί, για την έκδοση (release) Python 3.7.9.

Για να γράψουμε τον κώδικα ενός προγράμματος, να βρούμε τα λάθη που κάναμε στη σύνταξή του ή και τη λογική του και να τρέξουμε το πρόγραμμά μας, συνήθως χρησιμοποιούμε ένα *Ολοκληρωμένο Περιβάλλον Ανάπτυξης* (Integrated Development Environment - IDE), το οποίο μας προσφέρει ένα γραφικό περιβάλλον και υποστήριξη στον προγραμματισμό, εκσφαλμάτωση (debugging) και εκτέλεση των προγραμμάτων που αναπτύσσουμε.

Η Python συμπεριλαμβάνει ένα δικό της IDE που ονομάζεται IDLE (Integrated DeveLopment Environment), αλλά ο συγγραφέας βρήκε καταλληλότερο για τις ανάγκες του και χρησιμοποιεί το PyCharm της JetBrains s.r.o.

Μπορείτε να κατεβάσετε το Community version του PyCharm δωρεάν από το website: <https://www.jetbrains.com/pycharm>

Τοπικό περιβάλλον Python για Μηχανικούς

Για να δημιουργήσουμε στον υπολογιστή μας ένα τοπικό περιβάλλον ανάπτυξης επιστημονικών προγραμμάτων Python (local Python environment for scientific computing) ακολουθούμε μία από τις παρακάτω μεθοδολογίες – στρατηγικές:

1. Κάνουμε εγκατάσταση της version/release του interpreter της Python που επιθυμούμε από το website: <https://www.python.org/downloads> (η Python αυτή είναι γνωστή και ως CPython Distribution).

Επίσης, εάν δεν θέλουμε να χρησιμοποιήσουμε τον IDLE της Python για να γράφουμε τα προγράμματά μας, κάνουμε εγκατάσταση και του IDE της αρεσκείας μας.

Στη συνέχεια, και όταν αυτό απαιτείται από τον κώδικα του προγράμματος που αναπτύσσουμε, κάνουμε εγκατάσταση στον υπολογιστή μας τις εξωτερικές βιβλιοθήκες που θα χρησιμοποιήσουμε σε αυτό. Τη μεθοδολογία αυτή θα ακολουθήσουμε σε αυτό το βιβλίο. Πολλά IDE, όπως το PyCharm, διαθέτουν την ευκολία να κάνουμε την εγκατάσταση εξωτερικών βιβλιοθηκών μέσα από το περιβάλλον τους.

2. Κάνουμε την εγκατάσταση ενός ready-to-use περιβάλλοντος ανάπτυξης, όπως το Anaconda Python Distribution. Η Individual Edition παρέχεται δωρεάν και μπορείτε να την κατεβάσετε από το website: <https://www.anaconda.com/products/individual>

Μαζί με την εγκατάσταση του interpreter την Python, το Anaconda εγκαθιστά στο υπολογιστή μας και μια πληθώρα εξωτερικών βιβλιοθηκών, πολλές από τις οποίες μπορεί να μην τις χρησιμοποιήσουμε ποτέ...

Όσες βιβλιοθήκες δεν εγκαθίστανται αυτόματα από το Anaconda και θέλουμε να τις χρησιμοποιήσουμε στον κώδικα των προγραμμάτων μας, πρέπει να τις εγκαταστήσουμε εμείς.

04 Καλωσορίσατε στην Python

04.1 Γενικά

Στο Κεφάλαιο αυτό θα αρχίσουμε να χτίζουμε, σταδιακά, τις γνώσεις μας στην Python προσπαθώντας να υπολογίσουμε τη μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (ppv) μιας ανατίναξης, χρησιμοποιώντας τα στοιχεία της σχετικής με το θέμα τεχνολογίας των ανατινάξεων που παρουσιάστηκαν στο Κεφάλαιο 02 Δονήσεις από ανατινάξεις.

04.2 Το 1^ο στάδιο: Ο τίτλος

Γράφουμε τον παρακάτω κώδικα:

```
# Welcome to Python!
# Το BLASTppv.py είναι ένα programma pov
# ypologizei tin ppv mias anatinaxis se metalleia/latomeia/erga
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # Ο Titlos
```

Εάν τρέξουμε το παραπάνω πρόγραμμα (ναι, είναι ένα πρόγραμμα στην Python!), παίρνουμε το παρακάτω αποτέλεσμα (output) στην οθόνη:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
```

Ας εξηγήσουμε τι έχει γίνει:

Στον κώδικα που γράψαμε, οι 3 πρώτες γραμμές αρχίζουν με το σύμβολο # και μετά το καλωσόρισμα στην Python περιλαμβάνουν πληροφορίες για το τι κάνει το πρόγραμμά μας. Επίσης, μας πληροφορούν ότι το όνομα που αποφασίσαμε να δώσουμε στο πρόγραμμά μας είναι BLASTppv και ότι θα το σώσουμε σε ένα αρχείο με όνομα BLASTppv.py (όλα τα αρχεία προγραμμάτων Python έχουν επέκταση .py).

Ο διερμηνέας (interpreter) της Python, διαβάζει και εκτελεί τα προγράμματα Python, γραμμή – γραμμή, από τα αριστερά προς τα δεξιά και από την αρχή μέχρι το τέλος του κώδικα. Εάν συναντήσει το σύμβολο #, ότι βρίσκεται μετά από αυτό σε μια γραμμή του κώδικα το θεωρεί σχόλιο (comment) και δεν το διαβάζει ούτε το εκτελεί!

Επομένως, ο κώδικας που γράψαμε μετά το σύμβολο # είναι *άχρηστος* για την Python, αλλά εξαιρετικά χρήσιμος για τον προγραμματιστή, γιατί του λέει (ή θυμίζει) τι κάνει ο κώδικας που έχει γράψει. Στην 4^η γραμμή π.χ. το σχόλιο μάς θυμίζει ότι η γραμμή αυτή γράφει τον τίτλο του προγράμματός μας στο output.

Καλό είναι να συνηθίσετε να γράφετε (πολλά) σχόλια όταν προγραμματίζετε, ώστε να μπορείτε να θυμάστε τι έχετε γράψει στον κώδικά σας αλλά και γιατί το γράψατε έτσι (π.χ. τι μονάδες μέτρησης έχουν οι παράμετροι ενός προβλήματος, την πηγή μιας εξίσωσης που χρησιμοποιήσατε κ.ά.).

Η μόνη εντολή Python που έχει ο κώδικάς μας είναι το `print()`, που μεταφέρει στην οθόνη το μήνυμα που βρίσκεται μέσα στις παρενθέσεις `()`.

Στον κώδικά μας το μήνυμα αυτό είναι ένα string (συμβολοσειρά - ακολουθία χαρακτήρων) που αποτελείται από τις λέξεις και τα κενά μεταξύ των λέξεων: PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS, και πρέπει να βρίσκεται εντός εισαγωγικών, μονών ' ', διπλών " " ή και τριπλών εισαγωγικών ''' ''' ή """ """.

Με τη συνάρτηση `print()` εμφανίζουμε τα αποτελέσματα των υπολογισμών μας στην οθόνη του υπολογιστή μας, με απλό ή πιο σύνθετο τρόπο, όπως θα δούμε με λεπτομέρεια στη συνέχεια.

Έτσι λοιπόν, γράψαμε και εκτελέσαμε το πρώτο στάδιο του προγράμματος BLASTppv.py που μας παρουσιάζει στην οθόνη του υπολογιστή μας τον τίτλο του.

04.3 Το 2^ο στάδιο: Οι υπολογισμοί

Στο 2^ο στάδιο θα υπολογίσουμε τη μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους σε απόσταση 200 m από το μέτωπο μιας ανατίναξης σε ασβεστολιθικούς σχηματισμούς (ιζηματογενές πέτρωμα), σύμφωνα με το Γερμανικό Πρότυπο DIN 4150-1:2001. Η ποσότητα της εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα είναι 100 kg.

Στον κώδικα του 1^{ου} Σταδίου προσθέτουμε τις παρακάτω γραμμές:

```
print("Distance = 200 m")
print('Explosives = 100 kg')
ppv = 969*100**0.6*200**-1.5 # DIN 4150-1:2001 Exisosi [2.2]
print('ppv [mm/s] =', ppv)
```

Εάν ξανατρέξουμε το πρόγραμμα, παίρνουμε το παρακάτω output στην οθόνη:

```
PROGRAMMA ΥΠΟΛΟΓΙΣΜΟΥ ΤΗΣ PPV ΑΝΑΤΙΝΑΧΗΣ
Distance = 200 m
Explosives = 100 kg
ppv [mm/s] = 5.429736867032171
```

Ας εξηγήσουμε τον κώδικα που προσθέσαμε:

Οι δύο πρώτες γραμμές με την εντολή `print()` δεν κάνουν τίποτε άλλο από το να μεταφέρουν στην οθόνη, κάτω από τον τίτλο, τα δεδομένα του προβλήματός μας (Απόσταση = 200m, Εκρηκτικά = 100 kg).

Η τρίτη γραμμή του κώδικα μοιάζει να είναι μια εξίσωση. Τουλάχιστον αυτό μας λέει το σχόλιο στο τέλος της γραμμής, ότι δηλαδή είναι η Εξίσωση [2.2] του Κεφαλαίου 02 που κάνει τον υπολογισμό της μέγιστης ταχύτητας δόνησης σωματιδίων του εδάφους για ιζηματογενή πετρώματα, σύμφωνα με το Γερμανικό Πρότυπο DIN 4150-1:2001.

ΟΧΙ, το σύμβολο `=` στην Python, και σε άλλες γλώσσες προγραμματισμού, δεν δηλώνει την ισότητα των όσων βρίσκονται εκατέρωθεν του `=`. Είναι ένας τελεστής *εκχώρησης* (assignment) και με αυτόν εκχωρούμε (αποδίδουμε) στο αριστερό σκέλος του `=` την τιμή του δεξιού σκέλους. Το αριστερό σκέλος (το `ppv` στο πρόγραμμά μας) είναι μια *μεταβλητή* (variable) με όνομα `ppv`.

04.3.1 Περί μεταβλητών

Η μεταβλητή είναι ένα όνομα που αναφέρεται σε μια τιμή ή για να είμαστε πιο ακριβείς, είναι το όνομα της θέσης στη μνήμη του υπολογιστή μας όπου η Python έχει εκχωρήσει την τιμή της μεταβλητής.

Εάν γράψουμε:

```
x = 1
```

η Python δημιουργεί μια θέση στη μνήμη του υπολογιστή (τη μεταβλητή), την ονοματίζει `x` και της εκχωρεί την τιμή 1, δηλαδή τον ακέραιο αριθμό ένα.

Εάν στη συνέχεια γράψουμε:

```
x = x + 1
```

η Python εκχωρεί στη μεταβλητή `x` την τιμή που ήδη αυτή είχε, δηλαδή το ένα, αυξημένη κατά τον ακέραιο αριθμό ένα, δηλαδή η τιμή της μεταβλητής `x` είναι τώρα δύο (2).

Εδώ κατανοούμε στην πράξη γιατί το σύμβολο `=` στον προγραμματισμό δεν σημαίνει ισότητα των δύο μερών, γιατί εάν ίσχυε, τότε σύμφωνα με την άλγεβρα:

```
x = x + 1 → x - x = 1 → 0 = 1 ☹️
```


Εάν γράψουμε:

```
y = 1.2
z = 'George'
```

έχουμε δημιουργήσει άλλες δύο μεταβλητές, την `y` που έχει τιμή τον δεκαδικό αριθμό 1.2 και τη `z` που έχει τιμή τη συμβολοσειρά `George`.

Η εκχώρηση σε μια μεταβλητή τιμής της μορφής συμβολοσειράς γίνεται πάντα εντός μονών (`' '`), διπλών (`" "`) ή και τριπλών εισαγωγικών (`''' '''` ή `""" """`). Τα διπλά εισαγωγικά `" "` είναι χρήσιμα όταν η συμβολοσειρά που θέλουμε να γράψουμε περιλαμβάνει και μονό εισαγωγικό π.χ. `"George's Office"`. Τα τριπλά εισαγωγικά τα χρησιμοποιούμε όταν η συμβολοσειρά καταλαμβάνει περισσότερες της μίας γραμμές κώδικα. Επίσης, μπορούμε να χρησιμοποιήσουμε, ελεύθερα, μονά και διπλά εισαγωγικά μέσα σε τριπλά.

Η Python, κατά την εκχώρηση των τιμών στις μεταβλητές, καταλαβαίνει την κατηγορία (class) της μεταβλητής, δηλαδή εάν η μεταβλητή είναι *ακέραια* (integer – int), *δεκαδική ή κινητής υποδιαστολής* (floating point – float) ή *συμβολοσειρά* (string – str).

Στην Python δεν δηλώνουμε ρητά την κατηγορία των δεδομένων, δηλαδή των τιμών των μεταβλητών που χρησιμοποιούμε. Η Python κάνει τον έλεγχο κατά την εκτέλεση του προγράμματος και η ιδιότητα αυτή ονομάζεται *δυναμική απόδοση τύπων* (dynamic typing). Η Python είναι μια *dynamically typed language*!

Για του λόγου το αληθές, γράφουμε στη συνέχεια των παραπάνω τις παρακάτω εντολές:

```
print(type(x))
print(type(y))
print(type(z))
```

τρέχουμε το πρόγραμμα και στο output θα δούμε σε ποια κατηγορία (class) ανήκουν οι μεταβλητές `x`, `y`, `z`:

```
<class 'int'>
<class 'float'>
<class 'str'>
```

Οι τιμές των μεταβλητών μπορούν να αλλάξουν όσες φορές θέλουμε ή απαιτείται κατά την εκτέλεση του προγράμματός μας, εξ ου και ο όρος *μεταβλητή*, και μαζί με την τιμή αλλάζει και η κατηγορία της μεταβλητής.

Εάν αλλάξουμε τις τιμές των μεταβλητών `x`, `y`, `z` (όπως παρακάτω) και αναζητήσουμε το είδος στο οποίο ανήκουν μετά την αλλαγή:

```
x = y
y = 'George'
z = 3
print(type(x))
print(type(y))
print(type(z))
```

θα πάρουμε στο output:

```
<class 'float'>
<class 'str'>
<class 'int'>
```

Τα ονόματα των μεταβλητών μπορούν να έχουν έναν ή περισσότερους χαρακτήρες. Η Python δέχεται *μακρόσυρτα* ονόματα μεταβλητών, αλλά οι υπερβολές ποτέ δεν κάνουν καλό...

Το όνομα που θα επιλέξουμε να δώσουμε σε μια μεταβλητή καλό είναι να αντιπροσωπεύει κάτι σχετικό με τη χρήση της ή την ιδιότητα της τιμής που θα εκχωρηθεί σε αυτήν.

Οι χαρακτήρες που σχηματίζουν το όνομα της μεταβλητής μπορεί να είναι: τα πεζά και κεφαλαία γράμματα του λατινικού αλφάβητου, οι αριθμοί 0 έως 9 και ο ειδικός χαρακτήρας `_` (κάτω παύλα - underscore), ενώ μεταξύ των χαρακτήρων δεν επιτρέπονται κενά.

Τα ονόματα των μεταβλητών πρέπει να αρχίζουν ΠΑΝΤΑ με ένα γράμμα ή τον χαρακτήρα `_` (underscore). Μεταβλητή με όνομα `a1` είναι δεκτή, με όνομα `1a` δεν είναι δεκτή και το μήνυμα που θα πάρουμε από την Python θα είναι:

```
SyntaxError: invalid syntax.
```

Εάν αποφασίσετε να επιλέξετε όνομα μεταβλητής με έναν χαρακτήρα, ποτέ να μην χρησιμοποιήσετε το `'I'` (κεφαλαίο γράμμα 'eye'), το `'l'` (πεζό γράμμα 'el') και το `'O'` (κεφαλαίο γράμμα 'oh'). Τα δύο πρώτα το μάτι μας δεν μπορεί να τα ξεχωρίσει σε ορισμένες γραμματοσειρές (η Python μπορεί...), το τρίτο εύκολα μπερδεύεται με το 0 (μηδέν).

Τα παρακάτω στυλ ονομάτων μεταβλητών είναι συνηθισμένα στην κοινότητα των προγραμματιστών Python:

- `b` (single lowercase letter),
- `B` (single uppercase letter),
- `Lowercase`,
- `lower_case_with_underscores`,
- `UPPERCASE`,
- `UPPER_CASE_WITH_UNDERSCORES`,
- `CapitalizedWords`.

Ανεξάρτητα από το στυλ που θα επιλέξετε να ακολουθήσετε, σημασία έχει το όνομα που δίνετε σε μια μεταβλητή να υποδηλώνει το *περιεχόμενό της*, π.χ. για τη μεταβλητή που αντιστοιχεί στην ποσότητα εκρηκτικής ύλης κάποιος θα μπορούσε να επιλέξει ως όνομα το `w` (όπως το σύμβολο της στην Εξίσωση [2.1]), στο βιβλίο αυτό όμως θα επιλέξουμε το όνομα `weight_of_explosives`. Η διαφορά είναι προφανής...

Επίσης, πρέπει να έχουμε υπόψη μας ότι τα ονόματα των μεταβλητών είναι *case sensitive*, δηλαδή η Python διακρίνει τα πεζά από τα κεφαλαία γράμματα και για παράδειγμα οι μεταβλητές με ονόματα `ppn`, `Ppn` και `PPV` είναι τρεις διαφορετικές μεταβλητές και ΟΧΙ μια (ίδια) μεταβλητή με τρεις διαφορετικές γραφές.

Τέλος, η Python έχει δεσμευμένες για δικιά της χρήση ορισμένες λέξεις - κλειδιά (keywords). Για την Python 3.7 αυτές είναι οι ακόλουθες 33:

```
False, None, True, and, as, assert, break, class, continue, def, del,
elif, else, except, finally, for, from, global, if, import, in, is,
lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield
```

Οι λέξεις-κλειδιά δεν μπορούν να χρησιμοποιηθούν ως ονόματα μεταβλητών.

Σημείωση: Για τον σπουδαστή που θα διερωτηθεί εάν μπορεί να χρησιμοποιήσει γράμματα του ελληνικού αλφαβήτου στα ονόματα μεταβλητών, η απάντηση είναι ΝΑΙ, όπως φαίνεται και στο παράδειγμα που ακολουθεί:

```
ένα = 1
Δύο = 2
ΤΡΙΑ = ένα + Δύο
print('ΤΡΙΑ =', ΤΡΙΑ)
```

```
ΤΡΙΑ = 3
```

και μάλιστα γράμματα με τόνους!

Η συμβουλή του συγγραφέα είναι να μην χρησιμοποιείτε το ελληνικό αλφάβητο, αλλά το λατινικό και αν κρίνετε απαραίτητο ότι πρέπει να ονοματίσετε τις μεταβλητές σας με *άρωμα Ελλάδας* κάντε το στα Greeklish π.χ. οι μεταβλητές ένα, δύο και ΤΡΙΑ θα μπορούσαν να γραφτούν ena, Dyo και TRIA αντίστοιχα.

04.3.2 Περί αλγεβρικών πράξεων

Στο πρόγραμμά μας, η γραμμή του κώδικα:

```
prv = 969*100**0.6*200**-1.5 # DIN 4150-1:2001 Exisosi [2.2]
```

ορίζει τη μεταβλητή prv, που γνωρίζουμε ότι είναι η συντομογραφία του peak particle velocity (μέγιστη ταχύτητα σωματιδίων) και εκχωρεί σε αυτή την τιμή η οποία προκύπτει από την εκτέλεση των πράξεων μεταξύ των αριθμών που βρίσκονται στα δεξιά του = .

Δηλαδή, στην Εξίσωση [2.2] έχουμε αντικαταστήσει το L και το R με τις τιμές του προβλήματος: 100 (kg εκρηκτικών) και 200 (m απόσταση) αντίστοιχα, και η Python κάνει τις πράξεις μεταξύ των αριθμών (σταθερών) καταλαβαίνοντας ότι όταν βλέπει ένα * πρέπει να κάνει πολλαπλασιασμό και όταν βλέπει δύο συνεχόμενα ** πρέπει να υψώσει σε δύναμη.

Οι τελεστές (operators) που χρησιμοποιεί η Python για να κάνει αλγεβρικές πράξεις είναι:

+	Πρόσθεση αριθμών (Addition).
-	Αφαίρεση ενός αριθμού από έναν άλλο (Subtraction).
*	Γινόμενο δύο αριθμών (Multiplication).
**	Ύψωση αριθμού σε δύναμη (Exponentiation).
/	Διαίρεση δύο αριθμών (Division).
//	Διαίρεση δύο αριθμών στρογγυλοποιημένη προς τα κάτω (Floor Division).
%	Υπόλοιπο διαίρεσης δύο αριθμών (Modulo).

Για την κατανόηση της χρήσης των δύο τελευταίων τελεστών δίνονται τα παρακάτω παραδείγματα:

ΠΡΑΞΗ	ΑΠΟΤΕΛΕΣΜΑ
8//2	4
8%2	0
8.9//2	4.0
8.9%2	0.90000000000000004

Στην τελευταία πράξη, ενώ όλοι θα περιμέναμε ως αποτέλεσμα το 0.9, η Python μας δίνει ένα νούμερο με 16 δεκαδικά ψηφία που είναι βέβαια πολύ-πολύ κοντά στην αναμενόμενη τιμή, αλλά όχι 0.9! Αυτό είναι μια *ιδιαιτερότητα* που αφορά την ακρίβεια των δεκαδικών αριθμών στα υπολογιστικά συστήματα και για την οποία θα μιλήσουμε στην παράγραφο 04.6 *Ακρίβεια δεκαδικών αριθμών*.

Εκείνο που έχει πολύ μεγάλη σημασία να κατανοήσουμε είναι η σειρά/προτεραιότητα με την οποία η Python εκτελεί τις πράξεις όταν υπάρχουν περισσότεροι του ενός τελεστές σε μια αλγεβρική παράσταση, όπως στην περίπτωση της Εξίσωσης [2.2].

Οι *παρενθέσεις* () έχουν τη μεγαλύτερη προτεραιότητα και αναγκάζουν την Python να κάνει τους υπολογισμούς με τη σειρά που θέλουμε. Ό,τι βρίσκεται μέσα σε ένα ζεύγος παρενθέσεων υπολογίζεται πρώτα, αρχίζοντας από τις εσωτερικές παρενθέσεις προς τις εξωτερικές. Επίσης, χρησιμοποιώντας παρενθέσεις κάνουμε τις αλγεβρικές παραστάσεις πιο εύκολα αναγνώσιμες, χωρίς να επηρεάζεται το αποτέλεσμα των πράξεων.

Ο συγγραφέας δεν θα έγραφε τον κώδικά του έτσι:

```
prv = 969*100**0.6*200**-1.5 # DIN 4150-1:2001 Exisosi [2.2]
```

αλλά έτσι:

```
ppv = 969*(100**0.6)*(200**-1.5) # DIN 4150-1:2001 Exisosi [2.2]
```

παρόλο που γνωρίζει ότι η Python θα υπολογίσει το ίδιο αποτέλεσμα των πράξεων, είτε υπάρχουν οι παρενθέσεις είτε όχι.

Εκείνο που πρέπει να προσέχει κάποιος όταν χρησιμοποιεί παρενθέσεις είναι ότι πρέπει να κλείνει καθεμία παρένθεση που ανοίγει ή αλλιώς διατυπωμένο, σε μια παράσταση όσες είναι οι αριστερές παρενθέσεις, τόσες ακριβώς πρέπει να είναι και οι δεξιές παρενθέσεις. Αυτό ισχύει και για όλες τις εντολές της Python όπου χρησιμοποιούνται παρενθέσεις!

Η *ύψωση σε δύναμη* έχει την επόμενη μεγαλύτερη προτεραιότητα. Ακολουθούν, με την ίδια μεταξύ τους προτεραιότητα, ο *πολλαπλασιασμός* και η *διαίρεση*. Τελευταίες, με την ίδια μεταξύ τους προτεραιότητα, εκτελούνται οι πράξεις της *πρόσθεσης* και της *αφαίρεσης*.

Τελεστές με την ίδια προτεραιότητα, δηλαδή οι *, / και +, - υπολογίζονται με προτεραιότητα από αριστερά προς τα δεξιά.

Από πλευράς στυλ πληκτρολόγησης, έχει επικρατήσει αριστερά-δεξιά των *, / και ** να μην υπάρχει κενό διάστημα, ενώ αριστερά-δεξιά των + και - αφήνουμε ένα κενό διάστημα.

Οδηγίες για το στυλ γραφής κώδικα στην Python δίνονται στο *PEP 8 -- Style Guide for Python Code* [<https://www.python.org/dev/peps/pep-0008/>].

Ο τελεστής + χρησιμοποιείται και για την *πρόσθεση* συμβολοσειρών, δημιουργώντας μια αλληλουχία συμβολοσειρών, δηλαδή, δημιουργεί μια νέα συμβολοσειρά, όπου η αρχή της δεύτερης βρίσκεται μετά το τέλος της πρώτης, π.χ.

```
'George' + 'Panagiotou' → 'GeorgePanagiotou'
```

και

```
'George' + 'Panagiotou' → 'George Panagiotou'
```

(παρατηρήστε την ύπαρξη ή όχι κενού διαστήματος πριν από το γράμμα P.)

Με την ίδια λογική λειτουργεί στις συμβολοσειρές και ο τελεστής *, μόνο που στην περίπτωση αυτή ο ένας από τους πολλαπλασιαστέους πρέπει να είναι συμβολοσειρά και ο άλλος ακέραιος, π.χ.

```
2*'Kalimera!' → 'Kalimera!Kalimera!'
```

Επειδή η πιθανότητα να περιλαμβάνονται αλγεβρικές πράξεις στα προγράμματά σας είναι πολύ μεγάλη, ως Μηχανικοί θα πρέπει να εξοικειωθείτε με τον τρόπο που πρέπει να τις εκφράζετε σε κώδικα της Python. Σε περίπτωση αμφιβολιών, για να τσεκάρετε εάν η Python εκτελεί τις πράξεις σύμφωνα με την επιθυμία σας, ιδίως στις πιο πολύπλοκες αλγεβρικές παραστάσεις, δεν υπάρχει καλύτερος τρόπος από το να επιβεβαιώσετε το αποτέλεσμα που σας δίνει η Python με το αποτέλεσμα που υπολογίζετε εσείς με την αριθμομηχανή σας. Η χρήση παρενθέσεων διευκολύνει πολύ την αποφυγή τέτοιου είδους λαθών...

04.3.3 Περί αριθμών

Η Python χρησιμοποιεί αριθμούς που ανήκουν στις εξής κατηγορίες (classes):

Κατηγορία ακέραιων αριθμών (integer numbers)

Είναι αριθμοί χωρίς υποδιαστολή, θετικοί ή αρνητικοί.

Εάν γράψουμε 100 η Python θα καταλάβει ότι το 100 είναι ακέραιος αριθμός και εάν τον εκχωρήσουμε σε μια μεταβλητή, η μεταβλητή αυτή θα είναι μια ακέραια μεταβλητή (int).

Εάν γράψουμε 100.0 ή 100. η Python θα καταλάβει ότι το 100.0 ή 100. είναι αριθμός δεκαδικός ή κινητής υποδιαστολής και εάν τον εκχωρήσουμε σε μια μεταβλητή, η μεταβλητή αυτή θα είναι μια μεταβλητή κινητής υποδιαστολής (float).

Η Python δεν έχει περιορισμό στο μέγεθος των ακέραιων αριθμών. Εάν κάνετε την πράξη 123^{123} και το αποτέλεσμα το εκχωρήσετε σε μια μεταβλητή με όνομα `Big_Integer`:

```
Big_Integer = 123**123
print('Big_Integer =', Big_Integer)
```

η μεταβλητή αυτή θα έχει τιμή:

```
Big_Integer =
11437436793461719009988029522806627674621807845185022977588797505236950478566689
64466065683652015421696499747277306288423453431965811348959199428208744498372120
99476648958359023796078549041949007807220625356526926729664064846685758382803707
100766740220839267
```

Κατηγορία δεκαδικών αριθμών ή κινητής υποδιαστολής (floating point numbers)

Είναι αριθμοί, θετικοί ή αρνητικοί, που έχουν δεκαδικό μέρος ή μόνο την υποδιαστολή. Όταν λέμε υποδιαστολή, εννοούμε την τελεία (.) και όχι το κόμμα (,).

Εάν γράψουμε 1, 2 αντί του σωστού 1.2 η Python θα καταλάβει ότι γράψαμε τους αριθμούς 1 και 2 και εάν τους εκχωρήσουμε σε μια μεταβλητή, η μεταβλητή αυτή θα είναι μια μεταβλητή της κατηγορίας *πλειάδας* (tuple):

```
float_wrong = 1,2
print('float_wrong = ', float_wrong)
print(type(float_wrong))
```

```
float_wrong = (1, 2)
<class 'tuple'>
```

και εάν δεν αντιληφθούμε το λάθος μας και στη συνέχεια κάνουμε αλγεβρικές πράξεις με τη μεταβλητή αυτή:

```
float_WRONG = float_wrong * 3
print('float_WRONG = ', float_WRONG)
```

η Python θα κάνει πράξεις με πλειάδες και όχι με δεκαδικούς αριθμούς. Έτσι, αντί του αναμενόμενου αποτελέσματος `float_WRONG = 3.6` θα εμφανίσει στην οθόνη ως αποτέλεσμα:

```
float_WRONG = (1, 2, 1, 2, 1, 2)
```

Για τις πλειάδες (tuples) θα μιλήσουμε στην παράγραφο 08.2 *Πλειάδες*.

Η Python τους πολύ μικρούς και τους πολύ μεγάλους δεκαδικούς αριθμούς δεν τους γράφει ολόκληρους, αλλά σε επιστημονική μορφή (π.χ. 3.141416×10^5). Εάν κάνουμε την πράξη 123.0^{123} και το αποτέλεσμα το εκχωρήσουμε σε μια μεταβλητή:

```
Big_Float = 123.0**123
print('Big_Float =', Big_Float)
```

η μεταβλητή αυτή θα έχει τιμή:

```
Big_Float = 1.1437436793461719e+257
```

όπου το σύμβολο `e+257` σημαίνει ότι ο αριθμός που βρίσκεται αριστερά του, δηλαδή το 1.1437436793461719 πρέπει να πολλαπλασιαστεί με το 10^{257} .

Με την ίδια λογική μπορούμε να εκχωρήσουμε σε μια μεταβλητή τιμή γράφοντάς την σε επιστημονική μορφή, π.χ.

```
a = 2e+10
print(type(a))
print('a = ', a)
```

```
<class 'float'>
a = 20000000000.0
```

```
b = 3.14e-10
print(type(b))
print('b = ', b)
```

```
<class 'float'>
b = 3.14e-10
```

Παρατηρήστε ότι ενώ είχαμε εκχωρήσει στη μεταβλητή *a* ακέραια τιμή γραμμένη με την επιστημονική μορφή (2e+10), η Python καταχώρησε τη μεταβλητή αυτή στην κατηγορία των float και όχι στην κατηγορία των int, γιατί στην Python η επιστημονική μορφή των αριθμών είναι όμοια με τη μορφή αριθμών κινητής υποδιαστολής (float). Επίσης, παρατηρήστε ότι μετά το *e* πρέπει να γράψουμε το πρόσημο του εκθέτη (+ ή -). Το πρόσημο + δεν είναι απαραίτητο, θα μπορούσαμε να γράψουμε και *a* = 2e10, αλλά γράφοντάς το *a* = 2e+10 ακολουθούμε μια καλή πρακτική.

Πράξεις μεταξύ ακέραιων και δεκαδικών αριθμών

Εάν κάνουμε αλγεβρικές πράξεις μεταξύ ακέραιων και δεκαδικών αριθμών, το αποτέλεσμα θα είναι δεκαδικός αριθμός. Κατ' ανάλογο τρόπο, εάν κάνουμε πράξεις μεταξύ μεταβλητών κατηγορίας int και μεταβλητών κατηγορίας float, το δε αποτέλεσμα το εκχωρήσουμε σε μια νέα μεταβλητή, η μεταβλητή αυτή θα είναι κατηγορίας float.

Κατηγορία μιγαδικών αριθμών (complex numbers)

Η Python μπορεί να χειριστεί και μιγαδικούς αριθμούς, με τους οποίους δεν θα ασχοληθούμε στο βιβλίο αυτό, και θα περιοριστούμε στο να δούμε πώς εκχωρούνται σε μια μεταβλητή.

Για παράδειγμα, ο μιγαδικός αριθμός $2 + 3j$, με πραγματικό μέρος 2 και φανταστικό μέρος 3 εκχωρείται στη μεταβλητή *C*, είτε όπως στο παράδειγμα:

```
C = 2 + 3j
print(type(C))
print('C = ', C)
```

```
<class 'complex'>
C = (2+3j)
```

είτε με τη βοήθεια της συνάρτησης `complex()`, όπως στο παράδειγμα:

```
c = complex(2,3)
print(type(c))
print('c = ', c)
```

```
<class 'complex'>
c = (2+3j)
```

04.3.4 Υπολογισμοί με μεταβλητές

Στο πρόγραμμά μας, στη γραμμή του κώδικα:

```
ppv = 969*100**0.6*200**-1.5 # DIN 4150-1:2001 Exisosi [2.2]
```

έχουμε αντικαταστήσει το L και το R της Εξίσωσης [2.2] με τις τιμές του προβλήματος 100 (kg εκρηκτικών) και 200 (m απόσταση) αντίστοιχα. Αυτό δεν είναι λάθος, αλλά προγραμματιστικά δεν είναι λογικό, γιατί εάν θελήσουμε να υπολογίσουμε την ppv σε μια θέση που απέχει απόσταση 300 m από τον χώρο της ανατίναξης, θα πρέπει να επεμβούμε στον κώδικα και να αλλάξουμε το 200 σε 300. Το ίδιο ισχύει και για τυχόν αλλαγές στην ποσότητα των εκρηκτικών.

Το λογικό είναι να δημιουργήσουμε δύο νέες μεταβλητές και να εκχωρήσουμε σε αυτές τις τιμές της απόστασης από το μέτωπο ανατίναξης και της ποσότητας των εκρηκτικών:

```
distance_from_blast = 200 # m
weight_of_explosives = 100 # kg
```

και στη συνέχεια στη γραμμή του κώδικα που υπολογίζει την ppv να χρησιμοποιούμε τα ονόματα των μεταβλητών αυτών, αντί των αριθμητικών τιμών τους:

```
ppv = 969*(weight_of_explosives**0.6)*(distance_from_blast**-1.5)
```

Έτσι από εδώ και στο εξής, όταν θέλουμε να υπολογίσουμε την ppv για διαφορετικές τιμές των παραμέτρων L και το R της Εξίσωσης [2.2], θα αλλάζουμε τις τιμές των μεταβλητών `weight_of_explosives` και `distance_from_blast` και όχι τις αντίστοιχες τιμές στον κώδικα της αλγεβρικής παράστασης που υπολογίζει την ppv.

Τέλος, στη γραμμή του κώδικα η οποία εμφανίζει στην οθόνη το αποτέλεσμα του υπολογισμού της ppv:

```
print('ppv [mm/s] =', ppv)
```

κάνουμε χρήση του ονόματος της μεταβλητής `ppv`. Δηλαδή με την εντολή `print()` ζητάμε από την Python να εμφανίσει στην οθόνη το κείμενο-συμβολοσειρά που βρίσκεται μεταξύ των `' '` και μετά την τιμή της μεταβλητής `ppv`. Προσέξτε το απαραίτητο κόμμα που υπάρχει μεταξύ των δύο αυτών οντοτήτων.

04.4 Το 3^ο στάδιο: Η παρουσίαση των αποτελεσμάτων

Τα αποτελέσματα των υπολογισμών μας πρέπει να είναι κατανοητά όχι μόνο σε εμάς που γράψαμε το πρόγραμμα, αλλά και σε εκείνους που θα τα χρησιμοποιήσουν ή θα τα αξιολογήσουν. Επομένως η παρουσίαση των αποτελεσμάτων πρέπει να γίνεται με τέτοιο τρόπο ώστε να είναι πλήρως κατανοητά, να μην δημιουργούν παρερμηνείες, να μην έχουν ασάφειες, ενώ αυτός που τα διαβάζει πρέπει να γνωρίζει όχι μόνον τα δεδομένα από τα οποία προέκυψαν τα αποτελέσματα, αλλά και το πώς.

Με βάση τα παραπάνω, θα εμπλουτίσουμε το πρόγραμμά μας με τις παρακάτω μεταβλητές:

```
ppv_prediction_equation = 'DIN 4150-1:2001'
type_of_rock = 'Limestone'
```

οι οποίες μαζί με τις:

```
distance_from_blast = 200 # m
weight_of_explosives = 100 # kg
```

που έχουμε ήδη γράψει στον κώδικά μας, περιγράφουν πλήρως τα δεδομένα του προβλήματος.

Δηλαδή, ότι θα υπολογίσουμε τη μέγιστη ταχύτητα δόνησης των σωματιδίων του εδάφους σε θέση που απέχει 200 m από το μέτωπο μιας ανατίναξης, στην οποία θα εκραγούν ταυτόχρονα 100 kg εκρηκτικής ύλης, σύμφωνα με την εξίσωση του Γερμανικού Προτύπου DIN 4150-1:2001 για ιζηματογενή πετρώματα, γιατί το πέτρωμά μας είναι ασβεστόλιθος (Limestone) και επομένως θα χρησιμοποιήσουμε στον υπολογισμό της ppv την Εξίσωση [2.2].

Εδώ κάνουμε μια λογική υπέρβαση. Εμείς γνωρίζουμε από τη Γεωλογία ότι ο ασβεστόλιθος είναι ιζηματογενές πέτρωμα ενώ η Python δεν το γνωρίζει... Στη συνέχεια όμως θα δούμε πώς μπορεί η Python να

επιλέγει μόνη της την Εξίσωση [2.2] (για ιζηματογενή πετρώματα) ή την Εξίσωση [2.3] (για πυριγενή πετρώματα) ανάλογα με τον τύπο του πετρώματος.

Η γραμμή του κώδικα:

```
print('ppv [mm/s] =', ppv)
```

εμφανίζει στην οθόνη το αποτέλεσμα των υπολογισμών μας:

```
ppv [mm/s] = 5.429736867032171
```

ότι δηλαδή η ppv, σε μονάδες μέτρησης mm/s είναι 5.429736867032171 με ακρίβεια 15 δεκαδικών ψηφίων! Η ακρίβεια αυτή είναι υπερβολική και ένας Μηχανικός που βλέπει ένα τέτοιο αποτέλεσμα θα αρχίσει να αμφισβητεί το κατά πόσον αυτός που έγραψε το πρόγραμμα έχει και πολλή *επαφή* με το αντικείμενο...

Θα τροποποιήσουμε τη γραμμή του κώδικα ώστε να μας δίνει ακρίβεια δύο (2) δεκαδικών ψηφίων, που είναι πολύ ικανοποιητική, ως εξής:

```
print('ppv [mm/s] = {:.2f}'.format(ppv))
```

Θα τρέξουμε το πρόγραμμα και το νέο αποτέλεσμα που θα δούμε στην οθόνη είναι:

```
ppv [mm/s] = 5.43
```

που είναι πιο λογικό. Παρατηρήστε ότι η Python έκανε και στρογγυλοποίηση, το 5.42973... το έκανε 5.43 !

04.4.1 Φορμαρισμένα αποτελέσματα

Με την εντολή `print()` μπορούμε να φορμάρουμε τα αποτελέσματά μας χρησιμοποιώντας τη String Format Method ως εξής:

```
print('ppv [mm/s] = {:.2f}'.format(ppv))
```

Στη συμβολοσειρά που βρίσκεται εντός των `()` της `print` και εντός των `' '` έχουμε γράψει: το λεκτικό που χαρακτηρίζει το αποτέλεσμα, `ppv [mm/s] =`, μετά εντός των `{ }` τον τρόπο που θέλουμε να εμφανιστεί το αποτέλεσμά μας, δηλαδή ότι θα είναι δεκαδικός αριθμός— `float (f)` με 2 δεκαδικά ψηφία μετά την υποδιαστολή `(.2)` και ότι το αποτέλεσμά μας είναι η τιμή μιας μεταβλητής `(:)`. Το όνομα της μεταβλητής (`ppv`) βρίσκεται εντός των `()` της συνάρτησης `format()`, η οποία γράφεται αμέσως μετά τα `' '` της συμβολοσειράς και χωρίζεται από αυτήν με μια τελεία `(.)`.

Εάν η τιμή της μεταβλητής είναι ακέραιος αριθμός (`integer`), τότε γράφουμε `{:d}` ενώ εάν έχει ως τιμή μια συμβολοσειρά (`string`), γράφουμε `{:s}`.

Με βάση όσα αναφέραμε παραπάνω θα γράψουμε τον παρακάτω κώδικα παρουσίασης των αποτελεσμάτων του προγράμματος:

```
print('INPUT DATA:')
print('PPV Prediction Equation : {:s}'.format(ppv_prediction_equation))
print('Type of Rock : {:s}'.format(type_of_rock))
print('Distance from Blast [m] : {:d}'.format(distance_from_blast))
print('Weight of Explosives [kg]: {:.1f}'.format(weight_of_explosives))
print('OUTPUT:')
print('Peak Particle Velocity - ppv [mm/s] = {:.2f}'.format(ppv))
```

Εάν ξανατρέξουμε το πρόγραμμα, θα εμφανιστεί το παρακάτω αποτέλεσμα στην οθόνη:


```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
INPUT DATA:
PPV Prediction Equation : DIN 4150-1:2001
Type of Rock : Limestone
Distance from Blast [m] : 200
Weight of Explosives [kg]: 100.0
OUTPUT:
Peak Particle Velocity - ppv [mm/s] = 5.43
```

Χρησιμοποιώντας τη String Format Method, έχουμε πολλές δυνατότητες μορφοποίησης των ορισμάτων της `print()`. Για παράδειγμα, θα μπορούσαμε να είχαμε τα αποτελέσματά μας υπό μορφή μιας πρότασης κειμένου γράφοντας την κάτωθι γραμμή κώδικα:

```
print('The peak particle velocity (ppv) at a distance of {:.1f} m \n\
from a blastsite, when a charge of {:.1f} kg is detonated, \n\
based on the {:s} prediction equation for {:s} \n\
rock, is {:.2f} mm/s'.format(distance_from_blast, weight_of_explosives,
ppv_prediction_equation, type_of_rock, ppv))
```

και θα είχαμε το αποτέλεσμα:

```
The peak particle velocity (ppv) at a distance of 200.0 m
from a blastsite, when a charge of 100.0 kg is detonated,
based on the DIN 4150-1:2001 prediction equation for Limestone
rock, is 5.43 mm/s
```

Παρατηρήστε ότι μέσα στις `()` της `print()` χρησιμοποιήσαμε δύο χαρακτήρες διαφυγής (escape characters): τον χαρακτήρα `\n` οποίος εμφανίζει τη συμβολοσειρά που ακολουθεί σε μια νέα γραμμή και τον χαρακτήρα `\` ο οποίος μας επιτρέπει να γράφουμε τον μακρόσυρτο κώδικα μιας εντολής Python σε περισσότερες της μίας γραμμές.

Σύμφωνα με το *PEP 8 -- Style Guide for Python Code*, το μέγιστο μήκος μιας γραμμής κώδικα στην Python πρέπει να περιορίζεται στους 79 χαρακτήρες.

Επίσης, παρατηρήστε ότι οι χαρακτήρες διαφυγής (υπάρχουν και άλλοι εκτός από τους δύο που μόλις αναφέρθηκαν) δεν εμφανίζονται στο output, αλλά λειτουργούν ως εντολές προς τον interpreter της Python.

Τέλος, παρατηρήστε ότι εντός των `()` του `format()` δεν υπάρχει ο χαρακτήρας διαφυγής `\` παρόλο που συνεχίσαμε τον κώδικα σε νέα γραμμή, γιατί η απαίτηση αυτή καλύπτεται από τις `()`. Τα IDE, όπως ο PyCharm, αυτό το κάνουν αυτόματα.

Αυτό που πρέπει να προσέχουμε στην περίπτωση που δηλώνουμε πολλές μεταβλητές σε μια εντολή `print()` είναι να τις γράφουμε στο `.format()` με τη σειρά που έχουμε προβλέψει να παρουσιαστούν στις θέσεις των `{}`, χωρίζοντάς τις με κόμμα, γιατί αλλιώς η `print()` θα μας δώσει αλλόκοτα αποτελέσματα.

Με την `print()` μπορούμε να φορμάρουμε τα αποτελέσματα και με τη μέθοδο f-string, την οποία ο συγγραφέας δεν συνηθίζει να χρησιμοποιεί, ως εξής:

```
print(f'ppv [mm/s] = {ppv:.2f}')
```

δηλαδή, αντί για το `.format()` έχουμε το γράμμα `f` μπροστά από τα `' '` του string και το όνομα της μεταβλητής (`ppv`) γράφεται εντός των `{}` μπροστά από το `:`.

Τέλος, θα μπορούσαμε να είχαμε τα αποτελέσματά μας υπό μορφή μιας πρότασης ελληνικού κειμένου, γράφοντας την κάτωθι γραμμή κώδικα:

```
print('Η μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (ppv) \n\
σε απόσταση {:.1f} m από τον χώρο της ανατίναξης, \n\
όταν εκρήγνυνται {:.1f} kg εκρηκτικής ύλης, με βάση \n\
```



```
την εξίσωση πρόβλεψης του {:s} για πέτρωμα {:s}, \n\
είναι {:.2f} mm/s'.format(distance_from_blast, weight_of_explosives,
                             ppv_prediction_equation, type_of_rock, ppv))
```

οπότε στην οθόνη θα εμφανιζόταν το αποτέλεσμα:

Η μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (ppv)
σε απόσταση 200.0 m από τον χώρο της ανατίναξης,
όταν εκρήγνυνται 100.0 kg εκρηκτικής ύλης, με βάση
την εξίσωση πρόβλεψης του DIN 4150-1:2001 για πέτρωμα Limestone,
είναι 5.43 mm/s

04.5 Το 4^ο στάδιο: Εισαγωγή δεδομένων από το πληκτρολόγιο

Το πρόγραμμά μας, στο στάδιο της μέχρι τώρα ανάπτυξής του, είναι ένα έτοιμο πρόγραμμα Python, δηλαδή έχει δεδομένα εισόδου, των οποίων οι τιμές αποδίδονται σε μεταβλητές, έχει αλγόριθμο που κάνει πράξεις με μεταβλητές, υπολογίζει το ζητούμενο μέγεθος (την ppv) και έχει εντολές που μας παρουσιάζουν μορμαρισμένα τόσο τα δεδομένα εισόδου όσο και το αποτέλεσμα των υπολογισμών.

Το πρόγραμμά μας όμως δεν είναι λειτουργικό. Ο λόγος είναι ότι η εκχώρηση τιμών στις δύο βασικές μεταβλητές των δεδομένων μας, δηλαδή στη μεταβλητή `distance_from_blast`, που αντιπροσωπεύει την απόσταση του σημείου στο οποίο θέλουμε να υπολογίσουμε την ppv από το μέτωπο ανατίναξης, και στη μεταβλητή `weight_of_explosives`, που αντιπροσωπεύει την ποσότητα της εκρηκτικής ύλης, είναι *ενσωματωμένη* στον κώδικα του προγράμματος, με τις γραμμές κώδικα:

```
distance_from_blast = 200 # m
weight_of_explosives = 100 # kg
```

Εάν θελήσουμε να υπολογίσουμε την ppv με διαφορετικά δεδομένα απόστασης ή ποσότητας εκρηκτικής ύλης, θα πρέπει να επεμβούμε στον κώδικα του προγράμματος και να αλλάξουμε τις τιμές 200 ή 100 αντίστοιχα με τις νέες τιμές. Αυτό δεν είναι λειτουργικό και είναι επικίνδυνο γιατί μπορεί, από απροσεξία, να κάνουμε και άλλες αλλαγές στον κώδικα της Python, τέτοιες που να μην ξανατρέχει σωστά το πρόγραμμα ή τουλάχιστον όπως έτρεχε πριν.

Το λογικό είναι, όταν τρέχουμε το πρόγραμμα να σας *ζητάει* αυτό τις τιμές των δεδομένων που χρειάζεται για να κάνει τον υπολογισμό της ppv. Αυτό γίνεται με τη βοήθεια της συνάρτησης `input()`.

Εάν γράψουμε αντί των:

```
distance_from_blast = 200 # m
weight_of_explosives = 100 # kg
```

τις παρακάτω γραμμές κώδικα:

```
distance_from_blast = input('Distance from Blast [m] :') # m
weight_of_explosives = input('Weight of Explosives [kg] :') # kg
```

και ξανατρέξουμε το πρόγραμμα, θα παρατηρήσουμε ότι στην οθόνη εμφανίζεται το παρακάτω μήνυμα-προτροπή (prompt):

Distance from Blast [m] :

και το πρόγραμμα σταματάει την εκτέλεσή του σε αυτό το σημείο. Να τι έχει συμβεί:

Κάθε φορά που καλείται η συνάρτηση `input()` σε ένα πρόγραμμα Python, το πρόγραμμα σταματάει να εκτελείται και περιμένει από τον χρήστη να πληκτρολογήσει κάτι. Προφανώς, αυτό που θα πληκτρολογήσουμε είναι ένας αριθμός, π.χ. το 200, που ανταποκρίνεται στο μήνυμα-προτροπή και είναι η απόσταση του σημείου στο οποίο θέλουμε να υπολογίσουμε την ppv από το μέτωπο ανατίναξης, σε μέτρα.

Όταν πατήσουμε στη συνέχεια το πλήκτρο Enter, το πρόγραμμα θα συνεχίσει την εκτέλεσή του και θα εκχωρήσει στη μεταβλητή `distance_from_blast` την τιμή 200 που πληκτρολογήσαμε.

Αμέσως μετά, από κάτω, εμφανίζεται στην οθόνη το μήνυμα-προτροπή για να πληκτρολογήσουμε την ποσότητα της εκρηκτικής ύλης, σε κιλά :

```
Distance from Blast [m] :200
Weight of Explosives [kg] :
```

Θα πληκτρολογήσουμε π.χ. τον αριθμό 100 για την ποσότητα των εκρηκτικών σε κιλά και στη συνέχεια το πλήκτρο Enter. Το πρόγραμμα θα συνεχίσει να τρέχει και θα εκχωρήσει στη μεταβλητή `weight_of_explosives` την τιμή 100 που πληκτρολογήσαμε.

Και μετά έρχεται η καταστροφή! Το πρόγραμμα *κρασάρει* και ο debugger της Python εμφανίζει τη *διάγνωση* ότι στη γραμμή του κώδικα:

```
ppv = 969*(weight_of_explosives**0.6)*(distance_from_blast**-1.5)
```

υπάρχει σοβαρό σφάλμα του τύπου:

```
TypeError: unsupported operand type(s) for ** : 'str' and 'float'
```

δηλαδή, ότι στη γραμμή του κώδικα που υπολογίζει την τιμή της `ppv` και ειδικότερα στην πράξη της ύψωσης σε δύναμη (`**`) συμμετέχουν κατηγορίες τελεστών (`operand type(s)`) οι οποίες δεν υποστηρίζονται από την πράξη της ύψωσης σε δύναμη. Μάλιστα, οι τύποι των τελεστών που εμπλέκονται στο σφάλμα είναι `'str'` και `'float'`. Πράγματι, είναι προφανές ότι δεν μπορούμε να υψώσουμε μια συμβολοσειρά σε δύναμη με εκθέτη δεκαδικό αριθμό ή το αντίστροφο.

Επομένως το σφάλμα εντοπίζεται στην κατηγορία-class των μεταβλητών `distance_from_blast` και `weight_of_explosives`, αφού είναι δεδομένο ότι οι εκθέτες 0.6 και -1.5 είναι εξ'ορισμού κατηγορίας `'float'`.

Οι μεταβλητές αυτές, παρόλο που εμείς με το πληκτρολόγιο, μέσω της συνάρτησης `input()` τις εκχωρήσαμε τους ακέραιους αριθμούς 200 και 100 αντίστοιχα, εμφανίζονται να έχουν εκχωρημένες τιμές κατηγορίας `'str'`, όπως προκύπτει και από τον σχετικό έλεγχο που μπορείτε να κάνετε:

```
print(type(distance_from_blast))
```

```
<class 'str'>
```

```
print(type(weight_of_explosives))
```

```
<class 'str'>
```

Αυτό συμβαίνει γιατί στην Python, οτιδήποτε πληκτρολογήσουμε σε ένα μήνυμα-προτροπή της συνάρτησης `input()` εκχωρείται στην αντίστοιχη μεταβλητή ως *συμβολοσειρά*!

Το *πρόβλημα* που δημιουργήθηκε λύνεται όμως εύκολα με τη βοήθεια των συναρτήσεων `int()` και `float()` που διαθέτει η Python.

Η συνάρτηση `int()` μετατρέπει τις συμβολοσειρές αριθμών και τους δεκαδικούς αριθμούς που βρίσκονται εντός των `()` σε ακέραιους αριθμούς. Στους δεκαδικούς αριθμούς απλά κόβει το δεκαδικό μέρος.

Η συνάρτηση `float()` μετατρέπει τις συμβολοσειρές αριθμών και τους ακέραιους αριθμούς που βρίσκονται εντός των `()` σε δεκαδικούς αριθμούς.

Για να τρέξει σωστά το πρόγραμμά μας πρέπει να αντικαταστήσουμε τις γραμμές κώδικα:

```
distance_from_blast = input('Distance from Blast [m] :') # m
weight_of_explosives = input('Weight of Explosives [kg] :') # kg
```

με αυτές:

```
distance_from_blast = float(input('Distance from Blast [m] :'))
weight_of_explosives = float(input('Weight of Explosives [kg] :'))
```

Επιλέξαμε να χρησιμοποιήσουμε τη συνάρτηση `float()` αντί της `int()`, γιατί τόσο η απόσταση όσο και η ποσότητα των εκρηκτικών υλών είναι μεγέθη των οποίων η τιμή μπορεί να έχει και δεκαδικό μέρος. Θα επιλέγαμε τη συνάρτηση `int()` αντί της `float()` εάν το μέγεθος, την τιμή του οποίου θα πληκτρολογήσουμε, μετριέται σε ακέραιες τιμές, π.χ. ο αριθμός των ανατινάξεων (δεν μπορούμε να έχουμε μισή ανατίναξη...).

Ξανατρέχουμε το πρόγραμμα και, με απογοήτευση..., βλέπουμε ότι η Python διέγινε ένα καινούριο σφάλμα που δεν το έβγαζε την προηγούμενη φορά που τρέξαμε το πρόγραμμα, στη γραμμή κώδικα:

```
print('Distance from Blast [m] : { :d}'.format(distance_from_blast))
```

και η διάγνωση του σφάλματος είναι:

```
ValueError: Unknown format code 'd' for object of type 'float'
```

δηλαδή, ότι στη γραμμή του κώδικα που εμφανίζει στην οθόνη το δεδομένο του προβλήματός μας `Distance from Blast [m] :`, ενώ εμείς λέμε στην Python να εμφανίσει την τιμή της μεταβλητής `distance_from_blast`, η οποία είναι κατηγορίας `float` (γιατί έτσι αποφασίσαμε να μετατρέπεται η συμβολοσειρά που πληκτρολογούμε ως `input` για το μέγεθος της απόστασης), η Python αναμένει να φορμάρει, λόγω του τελεστή `{ :d}`, μια τιμή κατηγορίας `int`.

Διορθώνουμε το σφάλμα αντικαθιστώντας τον τελεστή `{ :d}` με τον `{ :.1f}`. Τώρα το πρόγραμμά μας θα τρέχει χωρίς σφάλματα.

Τέλος, υπάρχει και η συνάρτηση `str()` που μετατρέπει σε συμβολοσειρές τους αριθμούς που βρίσκονται εντός των `()`, ακέραιους ή δεκαδικούς.

Οι συναρτήσεις `int()` και `float()` είναι εξαιρετικά χρήσιμες, αλλά δεν κάνουν και θαύματα... Οι συμβολοσειρές τις οποίες θα μετατρέψουν σε ακέραιους ή δεκαδικούς αριθμούς πρέπει να έχουν εμφάνιση αριθμού!

Επομένως, εάν για παράδειγμα γράψετε:

```
this_year = int(MMXXII)
```

μην περιμένετε να εκχωρηθεί στη μεταβλητή `this_year` η ακέραια τιμή 2022. Εάν δεν το έχετε ήδη αντιληφθεί, το `MMXXII` είναι ο αριθμός 2022 γραμμένος στο ρωμαϊκό-λατινικό σύστημα αρίθμησης.

04.6 Ακρίβεια δεκαδικών αριθμών

Οι ηλεκτρονικοί υπολογιστές και κατά συνέπεια οι γλώσσες προγραμματισμού, όπως η Python, με τις οποίες γράφουμε κώδικα για να τον εκτελέσουν οι υπολογιστές, όταν χειρίζονται δεκαδικούς αριθμούς (αριθμούς κινητής υποδιαστολής - floating point numbers) δεν δουλεύουν με ακρίβεια δεκαδικών ψηφίων αλλά με ακρίβεια σημαντικών ψηφίων.

Για να καταλάβετε τι συμβαίνει, γράφουμε τις παρακάτω γραμμές κώδικα που υπολογίζουν το άθροισμα $a + b$ των μεταβλητών `a` και `b`, στις οποίες έχουμε εκχωρήσει τις δεκαδικές τιμές 0.1 και 0.2 αντίστοιχα, και στη συνέχεια εμφανίζουν στην οθόνη τις τιμές των μεταβλητών και του αθροίσματός τους, με ακρίβεια 20 δεκαδικών ψηφίων:

```
a = 0.1
b = 0.2
a_plus_b = a + b
print('a      = {:.20f}'.format(a))
```

```
print('b      = {:.20f}'.format(b))
print('a + b = {:.20f}'.format(a_plus_b))
```

```
a      = 0.10000000000000000555
b      = 0.20000000000000001110
a + b  = 0.30000000000000004441
```

Από ό,τι βλέπετε η τιμή 0.1 που εμείς εκχωρήσαμε στη μεταβλητή `a` αποδόθηκε σωστά μέχρι το 17^ο δεκαδικό ψηφίο, η τιμή 0.2 που εκχωρήσαμε στη μεταβλητή `b` αποδόθηκε σωστά μέχρι το 16^ο δεκαδικό ψηφίο, και το αναμενόμενο άθροισμα 0.3 των `a + b` αποδίδεται σωστά μέχρι το 16^ο δεκαδικό ψηφίο.

Γενικά, η Python αποδίδει σωστά τους αριθμούς μέχρι 16 δεκαδικά ψηφία.

Λαμβάνοντας αυτά υπόψη, θα πρέπει να είμαστε πολύ προσεκτικοί όταν κάνουμε συγκρίσεις ισότητας μεταβλητών κατηγορίας `float` γιατί το αποτέλεσμα της σύγκρισης μπορεί να βγαίνει λάθος, ενώ φαινομενικά είναι σωστό.

Για παράδειγμα, εάν ζητήσουμε από την Python να συγκρίνει εάν η τιμή του αθροίσματος `a + b` είναι ίση με 0.3 θα σας πει ότι είναι λάθος (`False`). Ο λόγος είναι απλός, το 0.3 για την Python είναι ο αριθμός 0.29999999999999998890, πολύ-πολύ κοντά στο 0.30000000000000004441 του αθροίσματός σας αλλά όχι ίσο...

04.7 Συνοψίζοντας...

Στο Κεφάλαιο αυτό γνωριμίς με την Python γράψαμε το πρώτο μας λειτουργικό πρόγραμμα που κάνει τους υπολογισμούς για την επίλυση ενός μεταλλευτικού προβλήματος, όχι ιδιαίτερα σύνθετου, αλλά πραγματικού.

Μάθαμε να γράφουμε τους αλγορίθμους που απαιτούνται για την επίλυσή του και να εισάγουμε τα δεδομένα του προβλήματος από το πληκτρολόγιο. Δημιουργήσαμε, δηλαδή, μια στοιχειώδη διεπαφή (interface) του χρήστη με το πρόγραμμα, και τέλος μάθαμε να παρουσιάζουμε τα αποτελέσματα των υπολογισμών μας μορμαρισμένα ώστε να είναι ευανάγνωστα στον χρήστη.

Για να υλοποιήσουμε τις παραπάνω εργασίες σε κώδικα, αλλά και τα επιπλέον παραδείγματα χρήσης της γλώσσας Python, χρησιμοποιήσαμε λίγες εντολές και συγκεκριμένα:

τις ενσωματωμένες στην Python συναρτήσεις:

`print()`, `format()`, `input()`, `float()`, `int()`, `str()` και την `type()` για να βλέπουμε την κατηγορία στην οποία ανήκουν οι μεταβλητές,

τους τελεστές των αλγεβρικών πράξεων (`+`, `-`, `*`, `/`, `**`, `//`, `%`), τους τελεστές μορφοποίησης `{:d}`, `{:nf}` και `{:s}`, καθώς και τους χαρακτήρες διαφυγής `\n` και `\.`

Τέλος, και αυτό είναι ένα σημαντικό μέρος της διαδικασίας ανάπτυξης προγραμμάτων υπολογιστή, είδαμε πώς μπορούμε να κάνουμε αποσφαλμάτωση ή εκσφαλμάτωση (debugging), δηλαδή να βρίσκουμε και να διορθώνουμε τα σφάλματα (bugs) του κώδικα που γράφουμε και τα οποία μας επισημαίνει ο debugger (αποσφαλματωτής) της Python.

04.8 Ο κώδικας στο τέλος του Κεφαλαίου 04

Ο κώδικας του προγράμματός μας, αφού διαγράψουμε τις γραμμές κώδικα με τις οποίες πειραματιστήκαμε εξερευνώντας διάφορα στοιχεία της Python, αποτελείται από τις παρακάτω γραμμές:

```
# To BLASTppv.py einai ena programma pou
# ypologizei tin ppv mias anatinaxis se metalleia/latomeia/erga
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # O Titlos
ppv_prediction_equation = 'DIN 4150-1:2001'
type_of_rock = 'Limestone'
distance_from_blast = float(input('Distance from Blast [m] :')) # m
```

```

weight_of_explosives = float(input('Weight of Explosives [kg] :')) # kg
ppv = 969*(weight_of_explosives**0.6)*(distance_from_blast**-1.5) # DIN
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # O Titlos
print('INPUT DATA:')
print('PPV Prediction Equation : {:s}'.format(ppv_prediction_equation))
print('Type of Rock : {:s}'.format(type_of_rock))
print('Distance from Blast [m] : {:.1f}'.format(distance_from_blast))
print('Weight of Explosives [kg]: {:.1f}'.format(weight_of_explosives))
print('OUTPUT:')
print('Peak Particle Velocity - ppv [mm/s] = {:.2f}'.format(ppv))

```

Αποθηκεύουμε τον *πηγαίο κώδικα* του προγράμματος, που στην κοινότητα της Python τον λέμε script (σενάριο), σε αρχείο με όνομα `Chapter_4_BLASTppv.py`

Εάν τρέξουμε το `Chapter_4_BLASTppv.py` με τιμές απόστασης από τον χώρο της ανατίναξης 200 m και ποσότητας εκρηκτικών 100 kg, στην οθόνη θα εμφανιστεί το παρακάτω αποτέλεσμα των υπολογισμών μας:

```

PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
INPUT DATA:
PPV Prediction Equation : DIN 4150-1:2001
Type of Rock : Limestone
Distance from Blast [m] : 200.0
Weight of Explosives [kg]: 100.0
OUTPUT:
Peak Particle Velocity - ppv [mm/s] = 5.43

```

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_4_BLASTppv.py`

05 Βρόχοι επανάληψης

05.1 Γενικά

Ο μηχανικός-χρήστης του προγράμματος `Chapter_4_BLASTppv.py` είναι πρακτικός άνθρωπος και θα τον ενδιέφερε το πρόγραμμα να έχει τη δυνατότητα, εκτός από το να υπολογίζει τη μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (ppv) σε μια συγκεκριμένη απόσταση από το μέτωπο ανατίναξης, να του δημιουργεί και έναν πίνακα αποτελεσμάτων.

Ο πίνακας θέλει να έχει δύο στήλες: την απόσταση από το μέτωπο ανατίναξης, για αποστάσεις από 100 m έως 1000 m ανά 100 m και την αναμενόμενη ppv για αυτές τις αποστάσεις.

Ένας τρόπος για να κάνει τον πίνακα είναι να τρέξει το πρόγραμμα 10 φορές, εκχωρώντας διαδοχικά στη μεταβλητή `distance_from_blast` τις τιμές 100, 200, 300, 400, 500, 600, 700, 800, 900 και 1000 και να σημειώνει την τιμή της ppv που υπολογίζει κάθε φορά το πρόγραμμα. Μετά, με τη βοήθεια π.χ. του Excel του Microsoft Office, μπορεί να διαμορφώσει τον ζητούμενο πίνακα.

Αυτός ο τρόπος όμως δεν είναι λογικός για κάποιον που γνωρίζει προγραμματισμό. Θα επεκτείνουμε, λοιπόν, τις δυνατότητες του προγράμματος ώστε να δημιουργεί αυτόματα τον ζητούμενο πίνακα, προσθέτοντας σε αυτό τις παρακάτω γραμμές κώδικα:

```
print('{:s}'.format(61*'-'))
print('Distance from Blast [m]    Peak Particle Velocity - ppv [mm/s]')
print('{:s}'.format(61*'-'))
for distance_from_blast in (100, 200, 300, 400, 500, 600, 700, 800, 900, 1000):
    ppv = 969*(weight_of_explosives**0.6)* \
        (distance_from_blast**-1.5)
    print('{:s} {:6.1f} {:s} {:6.2f}'.format(5*' ',
                                             distance_from_blast, 22*' ', ppv))
```

Εάν τρέξουμε το πρόγραμμα, μετά την προσθήκη των παραπάνω γραμμών κώδικα, εμφανίζεται στην οθόνη ο ζητούμενος πίνακας:

Distance from Blast [m]	Peak Particle Velocity - ppv [mm/s]

100.0	15.36
200.0	5.43
300.0	3.96
400.0	1.92
500.0	1.37
600.0	1.04
700.0	0.83
800.0	0.68
900.0	0.57
1000.0	0.49

Στη συνέχεια θα εξηγήσουμε τον τρόπο με τον οποίο η Python δημιούργησε τον πίνακα.

Η γραμμή κώδικα:

```
print('{:s}'.format(61*'-'))
```

που υπάρχει εις διπλούν στον κώδικα, εμφάνισε στην οθόνη μια διακεκομμένη γραμμή (-----) πάνω και κάτω από την επικεφαλίδα του πίνακα. Η επικεφαλίδα εμφανίζεται με τη γραμμή κώδικα:

```
print('Distance from Blast [m]    Peak Particle Velocity - ppv [mm/s]')
```

Όπως βλέπετε, το μήκος των γραμμών είναι το ίδιο με το μήκος του κειμένου της επικεφαλίδας, δηλαδή 61 διαστήματα, και επειδή το font που χρησιμοποιούμε στη γραμματοσειρά του κώδικα είναι `Courier`, ήτοι ένα `monospace font` (δηλαδή κάθε γράμμα καταλαμβάνει διάστημα ίσου πλάτους), οι γραμμές και η επικεφαλίδα *αλφαδιάζονται* τέλεια. Για λόγους σύγκρισης, το κείμενο του βιβλίου είναι γραμμένο με font `Calibri`, το οποίο είναι ένα `proportional font` και το διάστημα που καταλαμβάνει κάθε γράμμα δεν έχει σταθερό πλάτος.

Τον αριθμό των διαστημάτων της επικεφαλίδας μπορεί κάποιος να τον υπολογίσει μετρώντας τους χαρακτήρες και τα κενά διαστήματα της συμβολοσειράς:

```
Distance from Blast [m]    Peak Particle Velocity - ppv [mm/s]
```

Υπάρχει όμως και ένας πιο εύκολος, αλάνθαστος, *Pythonic* τρόπος υπολογισμού.

Γράφουμε τον βοηθητικό κώδικα:

```
spaces = len('Distance from Blast [m]    Peak Particle Velocity - ppv [mm/s]')
print('spaces =', spaces)
```

και παίρνουμε ως αποτέλεσμα το:

```
spaces = 61
```

Η συνάρτηση `len()` υπολογίζει το μήκος μιας συμβολοσειράς, δηλαδή μετράει τους χαρακτήρες και τα κενά μιας συμβολοσειράς που περικλείονται μεταξύ των `' '`.

Με την εντολή:

```
print('{:s}'.format(61*'-'))
```

λέμε στη Python να μας εμφανίσει ως string την τιμή `61*'-'`, δηλαδή το γινόμενο του 61 επί τη μεταβλητή string `'-'`, ήτοι 61 παύλες τη μία δίπλα στην άλλη. Θυμηθείτε ότι η Python μπορεί να κάνει τις πράξεις του πολλαπλασιασμού και της πρόσθεσης όχι μόνο μεταξύ αριθμητικών μεταβλητών, αλλά και μεταξύ μεταβλητών κατηγορίας string.

Οι υπόλοιπες γραμμές του κώδικα υπολογίζουν την `ppv` σε διαφορετικές αποστάσεις από το μέτωπο ανατίναξης με τη βοήθεια της εντολής `for`.

05.2 Η εντολή επανάληψης `for`

Για να δημιουργήσουμε τον ζητούμενο πίνακα πρέπει να υπολογίσουμε την αναμενόμενη `ppv` σε αποστάσεις από 100 m έως 1000 m ανά 100 m, δηλαδή πρέπει να κάνουμε τον υπολογισμό της `ppv` 10 φορές. Η εντολή `for` μας επιτρέπει να εκτελούμε μια συγκεκριμένη ομάδα εντολών, που την ονομάζουμε μπλοκ, για έναν συγκεκριμένο αριθμό επαναλήψεων. Ένα τέτοιο μπλοκ εντολών, του οποίου επαναλαμβάνεται η εκτέλεση, ονομάζεται *βρόχος* (*loop*).

Η επικεφαλίδα ενός βρόχου `for` ξεκινάει με τη δεσμευμένη λέξη-κλειδί της Python `for` ακολουθούμενη από το όνομα μιας μεταβλητής, την `distance_from_blast` στην περίπτωσή μας, τη δεσμευμένη λέξη-κλειδί `in`, τις τιμές που θα εκχωρηθούν διαδοχικά στη μεταβλητή, εντός `()`, στην περίπτωσή μας είναι οι τιμές `(100, 200, 300, 400, 500, 600, 700, 800, 900, 1000)` και τελειώνει, απαραίτητως, με μια άνω-κάτω τελεία `:`

Στις γραμμές κώδικα κάτω από την επικεφαλίδα ενός βρόχου γράφεται το μπλοκ των εντολών που συμμετέχουν στις επαναλήψεις. Οι εντολές αυτές πρέπει να είναι μετατοπισμένες προς τα δεξιά. Η μετατόπιση – εσοχή (*indentation*), την οποία ρυθμίζει αυτόματα η Python και τα IDE όταν πληκτρολογήσουμε `Enter` μετά την άνω-κάτω τελεία, είναι τέσσερα (4) κενά διαστήματα.

Όταν τελειώσουμε το γράψιμο των εντολών του μπλοκ, η πρώτη εντολή που ευθυγραμμίζεται με το αριστερό περιθώριο, δηλαδή είναι αλφαδιασμένη με το `for`, είναι η πρώτη εντολή έξω από το μπλοκ των εντολών του βρόχου και δεν συμμετέχει στις επαναλήψεις.

Με τη γραμμή κώδικα:

```
for distance_from_blast in (100, 200, 300, 400, 500, 600, 700, 800, 900, 1000):
```

η Python θα εκχωρήσει στη μεταβλητή `distance_from_blast` διαδοχικά τις τιμές 100, 200, 300, 400, 500, 600, 700, 800, 900 και 1000, και μετά από καθεμία εκχώρηση θα εκτελέσει τις εντολές του βρόχου:

```
    ppv = 969*(weight_of_explosives**0.6)* \
          (distance_from_blast**-1.5)
    print('{:s} {:6.1f} {:s} {:6.2f}'.format(5*' ',
                                             distance_from_blast, 22*' ', ppv))
```

δηλαδή θα υπολογίσει την `ppv` και θα εμφανίσει στη συνέχεια σε μία γραμμή τις τιμές των μεταβλητών `distance_from_blast` και `ppv`.

Μετά την εκχώρηση της τιμής 1000 στη μεταβλητή `distance_from_blast`, η Python θα εκτελέσει την πρώτη εντολή έξω από το μπλοκ των εντολών του βρόχου, εάν υπάρχει, ή θα σταματήσει την εκτέλεση του προγράμματος.

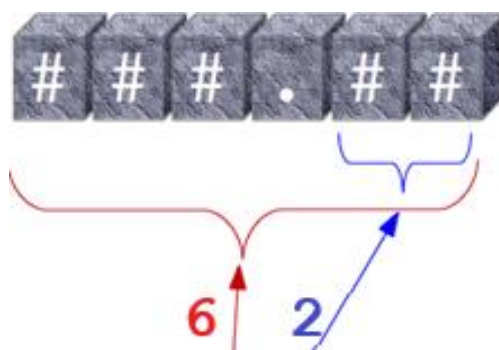
Η εντολή:

```
    print('{:s} {:6.1f} {:s} {:6.2f}'.format(5*' ',
                                             distance_from_blast, 22*' ', ppv))
```

μας επιτρέπει να φορμάρουμε στοιχισμένα τα νούμερα του πίνακά μας.

Κάθε φορά που εκχωρείται τιμή στη μεταβλητή `distance_from_blast` θα εμφανίζονται σε μία ξεχωριστή γραμμή από την `print()` τα εξής: 5 κενά διαστήματα με τον τελεστή `{:s}`, η τιμή της απόστασης που εκχωρήθηκε με ακρίβεια ενός δεκαδικού ψηφίου με τον τελεστή `{:6.1f}`, 22 κενά διαστήματα με τον τελεστή `{:s}`, και η τιμή της μεταβλητής `ppv`, που υπολογίστηκε προηγουμένως, με ακρίβεια δύο δεκαδικών ψηφίων με τον τελεστή `{:6.2f}`.

Παρατηρήστε ότι τον τελεστή φορμαρίσματος δεκαδικών αριθμών, στην παράγραφο 04.4.1 *Φορμαρισμένα αποτελέσματα* τον χρησιμοποιήσαμε με τη μορφή `{:.2f}`, δηλαδή ζητήσαμε να εμφανίζει τον δεκαδικό αριθμό με δύο δεκαδικά ψηφία, ενώ τώρα τον χρησιμοποιούμε στη μορφή `{:6.2f}`, δηλαδή ζητάμε να εμφανίζει μεν τον δεκαδικό αριθμό με δύο δεκαδικά ψηφία, αλλά όλος ο αριθμός, ήτοι το ακέραιο μέρος, το δεκαδικό μέρος και η υποδιαστολή, να έχουν εύρος 6 ψηφίων (Εικόνα 5.1). Εάν ο δεκαδικός αριθμός μαζί με την υποδιαστολή έχουν εύρος μικρότερο των 6 ψηφίων, τότε μπροστά από το ακέραιο μέρος προστίθενται κενά διαστήματα. Με αυτόν τον τρόπο επιτυγχάνουμε την τέλεια στοίχιση των τιμών του πίνακα.



Εικόνα 5.1. Επεξήγηση του τελεστή μορφοποίησης `{:6.2f}`

Οι τιμές εκχώρησης στη μεταβλητή `distance_from_blast` που δίνονται από την εντολή `for` την επικεφαλίδα του βρόχου (100, 200, 300, 400, 500, 600, 700, 800, 900, 1000), στην

περίπτωσή μας είναι υπό τη μορφή μιας *δομής δεδομένων (data structure)*, η οποία ονομάζεται *πλειάδα (tuple)*. Το περιεχόμενο μιας πλειάδας γράφεται εντός () και μεταξύ των μελών της βάζουμε κόμμα.

Άλλες δομές δεδομένων της Python είναι οι *λίστες (lists)* και τα *λεξικά (dictionaries)*.

Στη θέση της πλειάδας θα μπορούσαμε να είχαμε την αντίστοιχη λίστα τιμών αποστάσεων: [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000] με το ίδιο αποτέλεσμα. Το περιεχόμενο μιας λίστας γράφεται εντός [] και μεταξύ των μελών της βάζουμε κόμμα.

Οι δομές δεδομένων στην Python είναι ένα σύνολο δεδομένων (τιμών) μαζί με ένα σύνολο επιτρεπτών λειτουργιών (πράξεων) επί αυτών των δεδομένων. Τις δομές δεδομένων θα τις παρουσιάσουμε συγκεντρωτικά στο Κεφάλαιο 08 *Δομές δεδομένων*.

05.3 Η εντολή επανάληψης while

Τη δημιουργία του ζητούμενου πίνακα μπορούμε να την υλοποιήσουμε προγραμματιστικά στην Python και με την εντολή επανάληψης `while`. Η υλοποίηση είναι πιο σύνθετη, αλλά μας εισάγει σε μια βασική τεχνική του προγραμματισμού, την επαναλαμβανόμενη εκτέλεση τμήματος του κώδικα υπό συνθήκη. Με την εντολή `while` η Python εκτελεί επαναλαμβανόμενα τις εντολές του μπλοκ ενός βρόχου, όσο μια συνθήκη (προϋπόθεση) παραμένει αληθής.

Στη συνέχεια, θα επεκτείνουμε τις δυνατότητες του προγράμματός μας να δημιουργεί αυτόματα τον ζητούμενο πίνακα, με τη χρήση της εντολής `while`, προσθέτοντας τις παρακάτω γραμμές κώδικα:

```
distance_start_value = float(input('Distance start-value [m] :')) # m
distance_end_value = float(input('Distance end-value [m] :')) # m
distance_step_value = float(input('Distance step-value [m] :')) # m
print('{:s}'.format(61*'-'))
print('Distance from Blast [m]    Peak Particle Velocity - ppv [mm/s]')
print('{:s}'.format(61*'-'))
distance_from_blast = distance_start_value
while distance_from_blast <= distance_end_value:
    ppv = 969*(weight_of_explosives**0.6)*(distance_from_blast**-1.5)
    print('{:s} {:6.1f} {:s} {:6.2f}'.format(5*' ',
                                             distance_from_blast, 22*' ', ppv))
    distance_from_blast = distance_from_blast + distance_step_value
```

Οι πρώτες 3 γραμμές του κώδικα μας επιτρέπουν να εισάγουμε από το πληκτρολόγιο και να εκχωρήσουμε τιμές απόστασης, υπό μορφή δεκαδικού αριθμού, στις παρακάτω μεταβλητές:

`distance_start_value` : η αρχική τιμή απόστασης του πίνακα, π.χ. 100 m,

`distance_end_value` : η τελική τιμή απόστασης του πίνακα, π.χ. 1000 m,

`distance_step_value` : το βήμα μεταβολής των τιμών της απόστασης, π.χ. 100 m.

Οι επόμενες 3 γραμμές του κώδικα μας εμφανίζουν την επικεφαλίδα του πίνακα και τις διακεκομμένες γραμμές πάνω και κάτω από αυτήν, όπως τις έχουμε ήδη χρησιμοποιήσει στην υλοποίηση με την εντολή `for`.

Στη συνέχεια, εκχωρούμε στη μεταβλητή `distance_from_blast`, που χρησιμοποιείται για τον υπολογισμό της `ppv`, την αρχική τιμή απόστασης του πίνακα, μέσω της μεταβλητής `distance_start_value`.

Οι υπόλοιπες γραμμές του κώδικα είναι ένας βρόχος `while`.

Ο βρόχος `while` ξεκινάει με μία επικεφαλίδα που περιλαμβάνει τη δεσμευμένη λέξη `while`, μία συνθήκη (μία *Boolean* έκφραση) και τελειώνει με μία άνω-κάτω τελεία. Στη συνέχεια ακολουθεί σε εσοχή ένα μπλοκ εντολών βρόχου, οι οποίες εκτελούνται επαναλαμβανόμενα όσο η συνθήκη της επικεφαλίδας είναι αληθής

(True). Όταν η συνθήκη γίνει ψευδής (False), το μπλοκ εντολών παύει να εκτελείται και η ροή εκτέλεσης των εντολών μεταφέρεται στην πρώτη εντολή μετά τον βρόχο `while`.

Στον κώδικά μας, η συνθήκη είναι: η τιμή της μεταβλητής `distance_from_blast` να είναι μικρότερη ή ίση (`<=`) από την τιμή της μεταβλητής `distance_end_value`. Όταν γίνει μεγαλύτερη, τότε η συνθήκη θα καταστεί ψευδής.

Η αρχική τιμή της μεταβλητής `distance_from_blast` είναι ίδια με την τιμή της μεταβλητής `distance_start_value`, δηλαδή 100 και έχει εκχωρηθεί εκτός του βρόχου `while` με μια εντολή *αρχικοποίησης*. Επομένως, την πρώτη φορά που ελέγχεται η συνθήκη του βρόχου `while`, επειδή είναι αληθής, γίνεται η εκτέλεση του μπλοκ εντολών, δηλαδή υπολογίζεται η *ppv* για την τιμή απόστασης των 100 m και εμφανίζεται φορμαρισμένη η πρώτη γραμμή τιμών του πίνακα.

Εάν στη συνέχεια δεν αλλάζουμε την τιμή της μεταβλητής `distance_from_blast`, στους επόμενους ελέγχους της συνθήκης του βρόχου `while` το αποτέλεσμα θα είναι πάντα αληθές, θα εμφανίζεται δηλαδή φορμαρισμένη η πρώτη γραμμή τιμών του πίνακα για την τιμή απόστασης των 100 m, και αυτό θα συνεχιστεί επ' άπειρον, γιατί έχουμε δημιουργήσει έναν *ατέρμονα βρόχο* (*infinite loop*).

Για τον λόγο αυτό, στην τελευταία γραμμή του μπλοκ εντολών του βρόχου κάνουμε την αλλαγή στην τιμή της μεταβλητής `distance_from_blast`, δηλαδή στην τρέχουσα τιμή της προσθέτουμε το βήμα μεταβολής των τιμών της απόστασης, που έχει ήδη εκχωρηθεί στη μεταβλητή `distance_step_value`, ήτοι 100m.

Με αυτή τη νέα τιμή της μεταβλητής `distance_from_blast` γίνεται ο επόμενος έλεγχος της συνθήκης, κ.ο.κ., μέχρις ότου στην τιμή της μεταβλητής `distance_from_blast` εκχωρηθεί η τιμή 1100 ($1000 + 100$) η οποία και καθιστά τον έλεγχο της συνθήκης ψευδή.

Τότε, οι εντολές του μπλοκ του βρόχου παύουν να εκτελούνται και η ροή του προγράμματος μεταφέρεται στην πρώτη γραμμή κώδικα εκτός του βρόχου, εάν υπάρχει, ή σταματά η εκτέλεση του προγράμματος.

Εάν μια συνθήκη είναι ψευδής κατά την πρώτη φορά που αυτή ελέγχεται, το μπλοκ εντολών του βρόχου `while` δεν εκτελείται ποτέ.

05.4 Οι εντολές `break` και `continue`

Με τις εντολές επανάληψης `for` και `while` σχετίζονται οι εντολές `break` και `continue` που είναι και δεσμευμένες λέξεις της Python.

Η εντολή `break` χρησιμοποιείται για να κάνουμε άμεση έξοδο από οποιοδήποτε σημείο εντός ενός βρόχου.

Ας υποθέσουμε ότι δεν θέλουμε το πρόγραμμά μας να δημιουργεί πίνακες με περισσότερες από 10 γραμμές τιμών, ανεξάρτητα των τιμών που εισάγει ο χρήστης από το πληκτρολόγιο για τις μεταβλητές `distance_end_value` και `distance_step_value`.

Για να το υλοποιήσουμε αυτό, τροποποιούμε τον κώδικά μας και έχουμε τον παρακάτω:

```
distance_start_value = float(input('Distance start-value [m] :')) # m
distance_end_value = float(input('Distance end-value [m] :')) # m
distance_step_value = float(input('Distance step-value [m] :')) # m
print('{:s}'.format(61*'-'))
print('Distance from Blast [m]    Peak Particle Velocity - ppv [mm/s]')
print('{:s}'.format(61*'-'))
distance_from_blast = distance_start_value
i = 0
while distance_from_blast <= distance_end_value:
    ppv = 969*(weight_of_explosives**0.6)*(distance_from_blast**-1.5)
    print('{:s} {:6.1f} {:s} {:6.2f}'.format(5*' ',
                                             distance_from_blast, 22*' ', ppv))
```

```

distance_from_blast = distance_from_blast + distance_step_value
i = i + 1
if i == 10:
    break

```

Στον νέο μας κώδικα εισάγουμε μια βοηθητική μεταβλητή, την *i*, που θα τη χρησιμοποιήσουμε ως μια μεταβλητή μετρητή (*counter*) και την αρχικοποιούμε δίνοντάς της την ακέραια τιμή 0. Στη συνέχεια, μέσα στον βρόχο *while*, με την εντολή *i = i + 1* αυξάνουμε την τιμή της κατά μία μονάδα, κάθε φορά που εκτελείται μια επανάληψη των εντολών του βρόχου.

Η εντολή:

```
if i == 10:
```

ελέγχει τότε η τιμή της μεταβλητής *i* θα γίνει ίση με τον αριθμό 10 και όταν αυτό συμβεί, η εντολή *break* που ακολουθεί *σπάει το σερί* των επαναλήψεων, γιατί έχουν ήδη υπολογιστεί και εμφανιστεί οι 10 επιτρεπόμενες γραμμές τιμών του πίνακα.

Έτσι, εάν ο χρήστης εισάγει με το πληκτρολόγιο για τις μεταβλητές *distance_start_value*, *distance_end_value* και *distance_step_value* τις τιμές 100, 2000 και 100 αντίστοιχα και τρέξει το πρόγραμμα, θα εμφανιστεί ένας πίνακας με 10 και όχι με 20 γραμμές τιμών.

Την εντολή *continue* τη χρησιμοποιούμε για να μεταφέρουμε τη ροή του προγράμματος, από οποιοδήποτε σημείο εντός ενός βρόχου, στη συνθήκη της επικεφαλίδας του βρόχου.

Ας υποθέσουμε ότι δεν θέλουμε το πρόγραμμά μας να δημιουργεί πίνακες με τιμή απόστασης από το μέτωπο ανατίναξης, δηλαδή με τιμή της μεταβλητής *distance_from_blast*, μικρότερη των 100 m, δηλαδή η πρώτη γραμμή τιμών του πίνακα να είναι για απόσταση 100 m.

Για να το υλοποιήσουμε αυτό, τροποποιούμε τον κώδικά μας και έχουμε τον παρακάτω:

```

distance_start_value = float(input('Distance start-value [m] :')) # m
distance_end_value = float(input('Distance end-value [m] :')) # m
distance_step_value = float(input('Distance step-value [m] :')) # m
print('{:s}'.format(61*'-'))
print('Distance from Blast [m]    Peak Particle Velocity - ppv [mm/s]')
print('{:s}'.format(61*'-'))
distance_from_blast = distance_start_value
while distance_from_blast <= distance_end_value:
    if distance_from_blast < 100.:
        distance_from_blast = 100.
        continue
    ppv = 969*(weight_of_explosives**0.6)*(distance_from_blast**-1.5)
    print('{:s} {:6.1f} {:s} {:6.2f}'.format(5*' ',
                                             distance_from_blast, 22*' ', ppv))
    distance_from_blast = distance_from_blast + distance_step_value

```

Στον νέο κώδικα η πρώτη γραμμή μετά την επικεφαλίδα του βρόχου *while*:

```
if distance_from_blast < 100.:
```

ελέγχει εάν η τιμή της μεταβλητής *distance_from_blast* είναι μικρότερη του 100 και εάν αυτό είναι αληθές, τότε με τη γραμμή κώδικα που ακολουθεί:

```
distance_from_blast = 100.
```

εκχωρεί στη μεταβλητή την τιμή 100.

Αμέσως μετά, η εντολή *continue* μεταφέρει τη ροή του προγράμματος στη συνθήκη της επικεφαλίδας του βρόχου *while* και ξεκινά νέα επανάληψη.

Στη νέα επανάληψη, η τιμή της μεταβλητής *distance_from_blast* δεν έχει πλέον τιμή μικρότερη του 100 και ο έλεγχος *if distance_from_blast < 100.:* δίνει ψευδές αποτέλεσμα. Τότε, η ροή του

προγράμματος μεταφέρεται στις γραμμές κώδικα του βρόχου `while` που υπολογίζουν την `rrn` και εμφανίζουν τη γραμμή των τιμών του πίνακα.

Παρατηρήστε ότι, εάν ο χρήστης πληκτρολογήσει για τις μεταβλητές `distance_start_value`, `distance_end_value` και `distance_step_value` τις τιμές 1, 1000 και 100 αντίστοιχα, και τρέξει το πρόγραμμα, τότε θα εμφανιστεί ένας πίνακας με τιμές αποστάσεων 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000 και όχι ένας πίνακας με τιμές αποστάσεων 1, 101, 201, 301, 401, 501, 601, 701, 801, 901.

Στην παράγραφο αυτή για να δείξουμε τη χρήση των εντολών `break` και `continue` χρησιμοποιήσαμε και την εντολή `if` με την οποία επιτυγχάνουμε τον έλεγχο της ροής εκτέλεσης ενός προγράμματος υπό συνθήκες. Σε αυτή θα αναφερθούμε εκτενώς στην παράγραφο 07.2 Οι εντολές `if-elif-else`.

05.5 Σύγκριση των εντολών επανάληψης `for` & `while`

Με την εντολή `for` ο αριθμός των επαναλήψεων είναι προκαθορισμένος.

Με την εντολή `while` ο αριθμός των επαναλήψεων δεν είναι προκαθορισμένος, αλλά αυτές εκτελούνται μέχρις ότου η συνθήκη της επικεφαλίδας του βρόχου γίνει ψευδής. Και βέβαια πάντα ελλοχεύει ο κίνδυνος να έχουμε ατέρμονα βρόχο κατά την εκτέλεση ενός απρόσεκτα γραμμένου προγράμματος...

05.6 Τελεστές σύγκρισης

Στην εφαρμογή της εντολής `while` χρησιμοποιήσαμε *τελεστές σύγκρισης* (Boolean). Η Python για να κάνει σύγκριση μεταξύ μεταβλητών, μεταβλητών και αριθμητικών τιμών ή μεταξύ λογικών εκφράσεων, διαθέτει τους παρακάτω τελεστές:

<code>x == y</code>	x είναι ίσο με y
<code>x != y</code>	x δεν είναι ίσο με y
<code>x > y</code>	x είναι μεγαλύτερο από y
<code>x < y</code>	x είναι μικρότερο από y
<code>x >= y</code>	x είναι μεγαλύτερο ή ίσο με y
<code>x <= y</code>	x είναι μικρότερο ή ίσο με y

Οι τελεστές σύγκρισης, όπως και οι τελεστές αριθμητικών πράξεων, έχουν σειρά προτεραιότητας στην εκτέλεσή τους. Η σειρά με την οποία θα αποτιμηθούν οι τελεστές σύγκρισης αποκτά ιδιαίτερη σημασία όταν εμπλέκονται περισσότεροι του ενός τελεστές σε μια λογική παράσταση.

Οι παραστάσεις μέσα σε παρενθέσεις αποτιμώνται πρώτες.

Η προτεραιότητα των τελεστών, από τη μεγαλύτερη στη μικρότερη, είναι η εξής:

`<`, `>`, `<=`, `>=`, `!=`, `==`

Τέλος, σημειώστε ότι δεν υπάρχουν τελεστές σύγκρισης με σύμβολα `=<` και `=>`.

05.7 Επανάληψη με την εντολή `for` & την `range()`

Στην επικεφαλίδα του βρόχου `for` εκχωρήσαμε τιμές στη μεταβλητή `distance_from_blast`, οι οποίες αποτελούσαν το περιεχόμενο μιας πλειάδας, δηλαδή (100, 200, 300, 400, 500, 600, 700,

800, 900, 1000) ή μιας λίστας, δηλαδή [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000].

Αντί της πλειάδας ή της λίστας, μπορούμε να χρησιμοποιήσουμε τη συνάρτηση της Python `range()`.

Η συνάρτηση `range()` μας επιτρέπει να δημιουργούμε μια σειρά ακέραιων αριθμών εντός συγκεκριμένου εύρους (range). Ανάλογα με τον αριθμό των ορισμάτων (arguments) που θα δώσουμε στην `range()` θα λάβουμε μια σειρά ακέραιων αριθμών με προσδιορισμένη αρχή, τέλος και βήμα αύξησης μεταξύ των διαδοχικών ακέραιων αριθμών.

Στη γενική της μορφή δέχεται τρία ορίσματα, που είναι ακέραιοι αριθμοί ή ακέραιες μεταβλητές:

`range(start, stop, step)`.

Εάν δηλώσουμε μόνον ένα όρισμα, αυτό είναι το `stop` και η συνάρτηση έχει τη μορφή `range(stop)`. Η σειρά των ακέραιων αριθμών που θα δημιουργήσει θα αρχίζει από το μηδέν (0) και θα τελειώνει στον ακέραιο που αντιστοιχεί στη διαφορά `stop - 1`, δηλαδή δεν περιλαμβάνει την τιμή του `stop`. Π.χ. η συνάρτηση `range(3)` δίνει τους αριθμούς 0, 1, 2.

Εάν δηλώσουμε δύο ορίσματα, αυτά είναι το `start` και το `stop` και η συνάρτηση έχει τη μορφή `range(start, stop)`. Η σειρά των ακέραιων αριθμών που θα δημιουργήσει θα αρχίζει από το `start` και θα τελειώνει στον ακέραιο που αντιστοιχεί στο `stop - 1`, δηλαδή δεν περιλαμβάνει την τιμή του `stop`. Π.χ. η συνάρτηση `range(3, 9)` δίνει τους αριθμούς 3, 4, 5, 6, 7, 8.

Εάν δηλώσουμε και τα τρία ορίσματα, η συνάρτηση έχει τη μορφή `range(start, stop, step)`. Η σειρά των ακέραιων αριθμών που θα δημιουργήσει θα αρχίζει από το `start`, θα τελειώνει στον ακέραιο που αντιστοιχεί στο `stop - 1`, δηλαδή δεν περιλαμβάνει την τιμή του `stop` και οι διαδοχικοί αριθμοί θα έχουν βήμα ίσο με το `step`. Π.χ. η συνάρτηση `range(3, 10, 3)` δίνει τους αριθμούς 3, 6, 9. Σημειώστε ότι η τιμή του `step` δεν μπορεί να είναι μηδέν.

Το πρόβλημα με τη χρήση της συνάρτησης `range()` είναι ότι δεν δέχεται ως ορίσματα δεκαδικούς αριθμούς. Δέχεται ακέραιους (χωρίς την υποδιαστολή) αρνητικούς ή θετικούς και το μηδέν (εκτός ως τιμή του `step`).

Σημειώστε ότι για να ξεπεράσουμε αυτόν τον περιορισμό στη χρήση της συνάρτησης `range()`, μπορούμε να χρησιμοποιήσουμε τη συνάρτηση `arange()`, η οποία όμως δεν είναι ενσωματωμένη συνάρτηση της Python, δηλαδή δεν ανήκει στη standard βιβλιοθήκη συναρτήσεων της Python, αλλά σε μια εξαιρετικά χρήσιμη third-party (εξωτερική) βιβλιοθήκη της Python, τη NumPy. Στα επόμενα Κεφάλαια του βιβλίου θα κάνουμε εκτεταμένη χρήση των συναρτήσεων της βιβλιοθήκης NumPy.

Ο κώδικας για τη δημιουργία του πίνακα τιμών απόστασης – ppv με χρήση της εντολής `for` και της συνάρτησης `range()` είναι ο ακόλουθος:

```
distance_start_value = int(float(input('Distance start-value [m] :')))
distance_end_value = int(float(input('Distance end-value [m] :')))
distance_step_value = int(float(input('Distance step-value [m] :')))
print('{:s}'.format(61*'-'))
print('Distance from Blast [m]    Peak Particle Velocity - ppv [mm/s]')
print('{:s}'.format(61*'-'))
for distance_from_blast in range(distance_start_value,
                                distance_end_value + 1, distance_step_value):
    ppv = 969*(weight_of_explosives**0.6)* \
          (distance_from_blast**-1.5)
    print('{:s} {:6.1f} {:s} {:6.2f}'.format(5*' ',
                                              distance_from_blast, 22*' ', ppv))
```

Επειδή η συνάρτηση `range()` δέχεται μόνο ακέραιους αριθμούς, τις τιμές που εκχωρούμε στις μεταβλητές `distance_start_value`, `distance_end_value` και `distance_step_value` και τις οποίες πληκτρολογεί ο χρήστης (θυμηθείτε ότι για την Python είναι συμβολοσειρές), τις μετατρέπουμε

πρώτα σε δεκαδικούς αριθμούς (float) και στη συνέχεια σε ακέραιους (int), αποκόπτοντας την υποδιαστολή και το τυχόν δεκαδικό μέρος. Έτσι εάν ο χρήστης πληκτρολογήσει την τιμή 100. ή την τιμή 100.1, στη μεταβλητή θα εκχωρηθεί η ακέραια τιμή 100 που είναι ένα όρισμα αποδεκτό από τη συνάρτηση range().

Εάν αντί να ακολουθήσουμε την παραπάνω διαδικασία, γράφαμε την πρώτη γραμμή του κώδικα έτσι:

```
distance_start_value = int(input('Distance start-value [m] :'))
```

και ο χρήστης πληκτρολογούσε τιμή για τη μεταβλητή distance_start_value το 100, δεν θα υπήρχε κανένα πρόβλημα. Αν όμως πληκτρολογούσε την τιμή 100. τότε επειδή η Python την αντιλαμβάνεται ως μια συμβολοσειρά που περιέχει και το σύμβολο της υποδιαστολής, δεν μπορεί να τη μετατρέψει σε ακέραιο αριθμό και ο interpreter σταματά την εκτέλεση του προγράμματος με την ένδειξη λάθους:

```
ValueError: invalid literal for int() with base 10: '100.'
```

Τέλος, σημειώστε ότι ως ορίσματα στη συνάρτηση range() στην επικεφαλίδα του βρόχου for δηλώσαμε τις τιμές των ακέραιων μεταβλητών distance_start_value, distance_end_value + 1 και distance_step_value. Δώσαμε δηλαδή τιμή στο όρισμα stop ίση με την τιμή της μεταβλητής distance_end_value αυξημένη κατά 1, ώστε ο πίνακάς μας να έχει ως τιμή της απόστασης, στην τελευταία γραμμή του, την τιμή της μεταβλητής distance_end_value που θα πληκτρολογήσουμε και όχι την τιμή distance_end_value - 1.

05.8 Ο κώδικας στο τέλος του Κεφαλαίου 05

Ο κώδικας του προγράμματός μας, αφού διαγράψουμε τις γραμμές κώδικα με τις οποίες πειραματιστήκαμε εξερευνώντας διάφορα στοιχεία της Python, δίνεται στη συνέχεια.

Για την υλοποίηση της δημιουργίας του πίνακα τιμών απόστασης από το μέτωπο ανατίναξης και τιμών αναμενόμενης ppv επιλέξαμε να κάνουμε χρήση της εντολής for και της συνάρτησης range(). Επίσης, για λόγους ομοιομορφίας οι τιμές του πίνακα παρουσιάζονται με ένα δεκαδικό ψηφίο.

```
# To BLASTppv.py einai ena programma pou
# ypologizei tin ppv mias anatinaxis se metalleia/latomeia/erga
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # O Titlos
ppv_prediction_equation = 'DIN 4150-1:2001'
type_of_rock = 'Limestone'
distance_from_blast = float(input('Distance from Blast [m] :')) # m
weight_of_explosives = float(input('Weight of Explosives [kg] :')) # kg
ppv = 969*(weight_of_explosives**0.6)*(distance_from_blast**-1.5) # DIN
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # O Titlos
print('INPUT DATA:')
print('PPV Prediction Equation : {:s}'.format(ppv_prediction_equation))
print('Type of Rock : {:s}'.format(type_of_rock))
print('Distance from Blast [m] : {:.1f}'.format(distance_from_blast))
print('Weight of Explosives [kg]: {:.1f}'.format(weight_of_explosives))
print('OUTPUT:')
print('Peak Particle Velocity - ppv [mm/s] = {:.2f}'.format(ppv))
# Table Distance vs ppv
distance_start_value = int(float(input('Distance start-value [m] :')))
distance_end_value = int(float(input('Distance end-value [m] :')))
distance_step_value = int(float(input('Distance step-value [m] :')))
print('{:s}'.format(61*'-'))
print('Distance from Blast [m] Peak Particle Velocity - ppv [mm/s]')
print('{:s}'.format(61*'-'))
for distance_from_blast in range(distance_start_value,
                                distance_end_value + 1, distance_step_value):
    ppv = 969*(weight_of_explosives**0.6)* \
        (distance_from_blast**-1.5)
```

```
print('{:s} {:6.1f} {:s} {:6.1f}'.format(5*' ',
                                         distance_from_blast, 22*' ', ppv))
```

Αποθηκεύουμε το script του προγράμματός μας σε αρχείο με όνομα `Chapter_5_BLASTppv.py`

Εάν τρέξουμε το πρόγραμμα με τιμές απόστασης από το μέτωπο ανατίναξης 200 m, ποσότητα εκρηκτικών 100 kg, και τιμές απόστασης για τον πίνακα απόσταση-ppv: αρχική τιμή 100 m, τελική τιμή 1000 m και βήμα μεταβολής 100 m, στην οθόνη θα εμφανιστεί το παρακάτω αποτέλεσμα των υπολογισμών μας:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
INPUT DATA:
PPV Prediction Equation : DIN 4150-1:2001
Type of Rock           : Limestone
Distance from Blast [m] : 200.0
Weight of Explosives [kg]: 100.0
OUTPUT:
Peak Particle Velocity - ppv [mm/s] = 5.43
-----
Distance from Blast [m]   Peak Particle Velocity - ppv [mm/s]
-----
100.0                     15.4
200.0                     5.4
300.0                     3.0
400.0                     1.9
500.0                     1.4
600.0                     1.0
700.0                     0.8
800.0                     0.7
900.0                     0.6
1000.0                    0.5
```

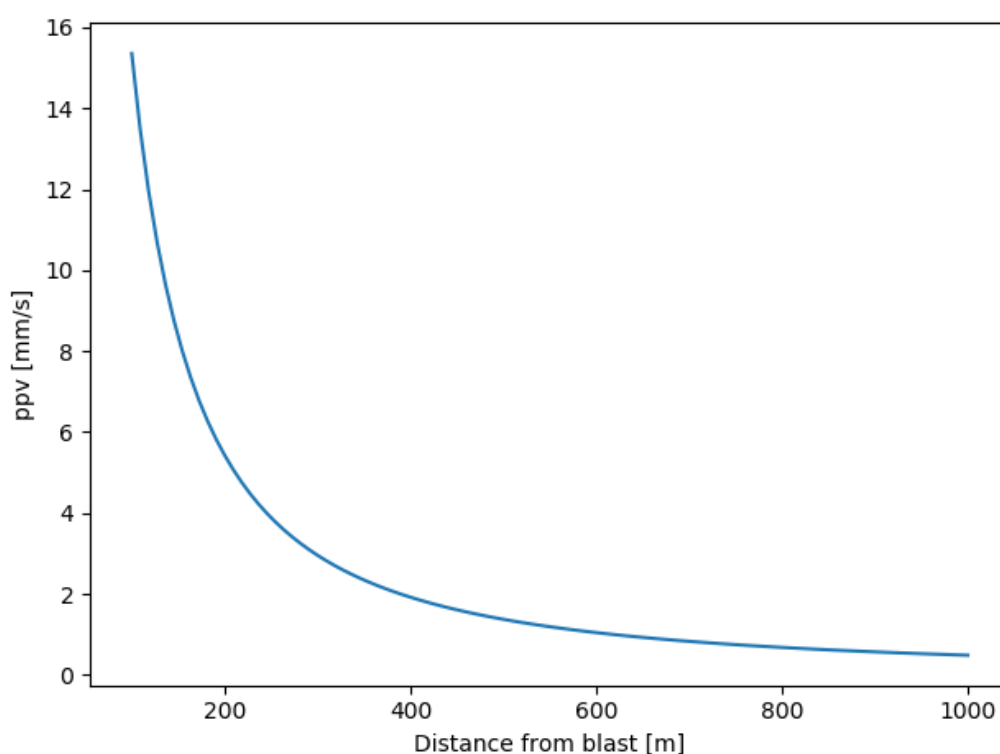
Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_5_BLASTppv.py`

06 Γραφικές παραστάσεις

06.1 Γενικά

Μελετώντας τις τιμές του πίνακα που μας δίνει την αναμενόμενη *ppn* συναρτήσει της απόστασης από το μέτωπο ανατίναξης και αποτελεί μέρος του *output* του προγράμματός μας, όπως φαίνεται στο τέλος της παραγράφου 05.8 Ο κώδικας στο τέλος του Κεφαλαίου 05, αντιλαμβανόμαστε ότι η *ppn* δεν μεταβάλλεται γραμμικά συναρτήσει της απόστασης. Αυτό άλλωστε ήταν προφανές και από τη μορφή της Εξίσωσης [2.2].

Η μεταβολή αυτή θα γινόταν καλύτερα αντιληπτή στον χρήστη του προγράμματός μας, εάν κατασκευάζαμε τη γραφική παράσταση (διάγραμμα) μεταβολής της *ppn* συναρτήσει της απόστασης, το οποίο παρουσιάζεται στην Εικόνα 6.1.



Εικόνα 6.1. Γραφική παράσταση της μεταβολής της *ppn* συναρτήσει της απόστασης από το μέτωπο ανατίναξης [Πηγή: αρχείο *ppn_vs_distance.png*].

Το διάγραμμα αυτό το κατασκευάσαμε χρησιμοποιώντας συναρτήσεις από δύο εξαιρετικά χρήσιμες για τον προγραμματισμό εφαρμογών Μηχανικού *εξωτερικές βιβλιοθήκες* (packages) της Python, τη Matplotlib (<https://matplotlib.org>) και τη NumPy (<https://numpy.org>).

Η Matplotlib είναι μια βιβλιοθήκη δημιουργίας διδιάστατων γραφικών παραστάσεων υψηλής ποιότητας. Η NumPy είναι η βασική εξωτερική μαθηματική βιβλιοθήκη της Python, η οποία χρησιμοποιείται συστηματικά για scientific computing.

Η Matplotlib συνήθως χρησιμοποιείται σε συνεργασία με τη NumPy, επειδή η δεύτερη μας χρειάζεται για να κάνει τη δουλειά με τους μαθηματικούς υπολογισμούς που χρησιμοποιούνται στη δημιουργία των γραφημάτων.

Ο κώδικας με τον οποίο κατασκευάσαμε τη γραφική παράσταση της Εικόνας 6.1 είναι:

```
import numpy as np
import matplotlib.pyplot as plt
```



```
x = np.linspace(100, 1000, 100)
y = 969*(weight_of_explosives**0.6)*(x**-1.5)
plt.plot(x, y)
plt.xlabel('Distance from blast [m]')
plt.ylabel('ppv [mm/s]')
plt.savefig('ppv_vs_distance.png')
plt.show()
```

Στις λίγες αυτές γραμμές κώδικα έχουμε αρκετά καινούρια στοιχεία της γλώσσας Python που θα τα δούμε στη συνέχεια.

06.2 Εγκατάσταση εξωτερικών βιβλιοθηκών

Οι βιβλιοθήκες (packages) της Python διακρίνονται σε *εσωτερικές* και *εξωτερικές* (third-party). Οι εσωτερικές βιβλιοθήκες (π.χ. η `math`, η `tkinter` κ.ά.) περιλαμβάνονται στην *Python Standard Library* και εγκαθίστανται στον υπολογιστή μας μαζί με την εγκατάσταση του interpreter της γλώσσας Python. Οι εξωτερικές βιβλιοθήκες (π.χ. η `numpy` και η `matplotlib`) πρέπει να εγκατασταθούν στον υπολογιστή από εμάς.

Η εγκατάσταση των εξωτερικών βιβλιοθηκών `numpy` και `matplotlib` γίνεται με τη βοήθεια του `pip`, που είναι ο *package installer* της Python ως εξής:

```
pip install numpy
pip install matplotlib
```

Πολλά IDE, όπως το PyCharm, διαθέτουν ένα facility που κάνει την εγκατάσταση των εξωτερικών βιβλιοθηκών. Είναι καλή πρακτική, πριν κάνετε την εγκατάσταση μιας εξωτερικής βιβλιοθήκης να διαβάσετε τις σχετικές οδηγίες στο website της βιβλιοθήκης (π.χ. στο <https://numpy.org/install> για τη `numpy`).

06.2.1 Η εντολή `import`

Με την εντολή `import` εισάγουμε στον κώδικα ενός προγράμματος της Python εσωτερικές ή εξωτερικές βιβλιοθήκες. Αυτό είναι απαραίτητο να γίνει για να μπορέσουμε στη συνέχεια να χρησιμοποιήσουμε στον κώδικα τις συναρτήσεις αυτών των βιβλιοθηκών.

Για καθεμία βιβλιοθήκη που εισάγουμε, γράφουμε ξεχωριστή εντολή `import` και οι εντολές αυτές τοποθετούνται, συνήθως, στην αρχή του προγράμματος για να μπορούμε να τις εντοπίζουμε εύκολα.

Οποσδήποτε, η εντολή `import` μιας βιβλιοθήκης πρέπει να έχει γραφτεί πριν από τις γραμμές κώδικα με τις οποίες γίνονται κλήσεις σε συναρτήσεις αυτής της βιβλιοθήκης.

Την εντολή `import` μπορούμε να τη χρησιμοποιήσουμε με διάφορους τρόπους.

Εάν γράψουμε:

```
from numpy import cos, sqrt
```

τότε από τη βιβλιοθήκη `numpy` εισάγουμε ΜΟΝΟ τις συναρτήσεις `cos()` και `sqrt()`, οι οποίες υπολογίζουν το συνημίτονο και την τετραγωνική ρίζα αντίστοιχα. Δεν μπορούμε όμως να χρησιμοποιήσουμε στον κώδικά μας τις υπόλοιπες συναρτήσεις της βιβλιοθήκης `numpy`.

Εάν γράψουμε:

```
from numpy import *
```

τότε από τη βιβλιοθήκη `numpy` εισάγουμε ΟΛΕΣ τις συναρτήσεις της βιβλιοθήκης και μπορούμε να χρησιμοποιήσουμε όποιες από αυτές θέλουμε στον κώδικά μας.

Ο συγγραφέας του βιβλίου δεν συνιστά τη χρήση αυτής της τεχνικής `import`, γιατί δημιουργεί τις προϋποθέσεις να έχουμε λάθη στον κώδικά μας, ιδίως όταν καλούμε με την τεχνική αυτή περισσότερες της μίας βιβλιοθήκες.

Αυτό οφείλεται στο γεγονός ότι πολλές βιβλιοθήκες διαθέτουν συναρτήσεις που έχουν το ίδιο όνομα, κάνουν την ίδια δουλειά, αλλά δέχονται διαφορετικά ορίσματα.

Για παράδειγμα, η βιβλιοθήκη (εξωτερική) `numpy` και η βιβλιοθήκη (εσωτερική) `math` διαθέτουν και οι δύο τη συνάρτηση `exp()`, η οποία υπολογίζει το e^m όπου m ένας πραγματικός αριθμός.

Εάν γράψουμε:

```
from numpy import *
x_numpy = exp([0, 1, 2.5])
print('x_numpy =', x_numpy)
```

η Python με τη συνάρτηση `exp()` της βιβλιοθήκης `numpy` υπολογίζει το e^m όπου m οι αριθμοί που περιέχονται στη λίστα `[0, 1, 2.5]` και το αποτέλεσμα θα είναι μία λίστα:

```
x_numpy = [ 1.          2.71828183 12.18249396]
```

Εάν γράψουμε:

```
from math import *
x_math = exp([0, 1, 2.5])
print('x_math =', x_math)
```

η Python, με τη συνάρτηση `exp()` της βιβλιοθήκης `math`, δίνει ως αποτέλεσμα το:

```
x_math = exp([0, 1, 2.5])
TypeError: must be real number, not list
```

Δηλαδή ο debugger της Python μας βγάζει λάθος, γιατί δεν μπορεί να υπολογίσει με τη συνάρτηση `exp()` της βιβλιοθήκης `math` το e^m των μελών μιας λίστας, αλλά μόνο ανεξάρτητων πραγματικών αριθμών.

Το πρόβλημα δημιουργείται όταν γράψουμε:

```
from numpy import *
from math import *
x_numpy = exp([0, 1, 2.5])
print('x_numpy =', x_numpy)
x_math = exp(2.5)
print('x_math =', x_math)
```

δηλαδή κάνουμε `import` και των δύο βιβλιοθηκών `numpy` και `math`, που και οι δύο διαθέτουν τη συνάρτηση `exp()` αλλά με διαφορετικά χαρακτηριστικά ορισμάτων η καθεμία. Το αποτέλεσμα των υπολογισμών είναι ένα μήνυμα λάθους:

```
x_numpy = exp([0, 1, 2.5])
TypeError: must be real number, not list
```

γιατί η Python χρησιμοποιεί, και στις δύο περιπτώσεις, τη συνάρτηση `exp()` της βιβλιοθήκης `math` που κάναμε `import` τελευταία. Τώρα πλέον είναι κατανοητός ο κίνδυνος που ελλοχεύει με τη χρήση εντολών `import` του τύπου:

```
from «βιβλιοθήκη» import *
```

Εάν γράψουμε:

```
import numpy
```

τότε κάνουμε `import` τη βιβλιοθήκη `numpy` με τη *standard* μέθοδο.

Το πρόβλημα που έχουμε με τη μέθοδο αυτή είναι ότι στον κώδικά μας κάθε φορά που γράφουμε μια συνάρτηση πρέπει να αναφέρουμε τη βιβλιοθήκη από την οποία προέρχεται με prefix το όνομά της. Π.χ. τη συνάρτηση `cos()` πρέπει να τη γράψουμε `numpy.cos()`.

Το θετικό είναι ότι με τον τρόπο αυτό γνωρίζουμε τη βιβλιοθήκη προέλευσης καθεμιάς συνάρτησης που χρησιμοποιούμε στον κώδικα. Το αρνητικό είναι ότι η μέθοδος αυτή αυξάνει τον φόρτο πληκτρολόγησης.

Εάν γράψουμε:

```
import numpy as np
```

τότε κάνουμε `import` τη βιβλιοθήκη `numpy` με τη *standard* μέθοδο, αλλά ταυτόχρονα δίνουμε στη βιβλιοθήκη το σύντομο παρατσούκλι (*nickname*) `np`. Με τον τρόπο αυτό μειώνεται ο φόρτος πληκτρολόγησης, γιατί τώρα τη συνάρτηση `cos()` θα τη γράψουμε `np.cos()`.

Έτσι στον κώδικα του προγράμματός μας με τις εντολές:

```
import numpy as np
import matplotlib.pyplot as plt
```

κάνουμε `import` τη βιβλιοθήκη `numpy`, τις συναρτήσεις της οποίας θα γράφουμε με prefix το `np.` και το module `pyplot` της βιβλιοθήκης `matplotlib`, τις συναρτήσεις του οποίου θα γράφουμε με prefix το `plt.` Παρατηρήστε ότι μεταξύ του prefix και του ονόματος της συνάρτησης υπάρχει τελεία.

06.3 Διαμορφώνοντας το διάγραμμα

Με τη γραμμή κώδικα:

```
x = np.linspace(100, 1000, 100)
```

ορίζουμε τη μεταβλητή `x` που αντιπροσωπεύει τις τιμές του οριζόντιου άξονα `x` του διαγράμματος, δηλαδή του άξονα της απόστασης. Στη μεταβλητή `x` εκχωρούμε τιμές που προκύπτουν από τη συνάρτηση `linspace()` της βιβλιοθήκης `numpy`.

Η συνάρτηση `np.linspace(start, stop, num)` κάνει ισοδιαμερισμό του διαστήματος `[start, stop]`, δημιουργώντας ένα array (matrix, μαθηματικό πίνακα) που αποτελείται από `num` αριθμούς, οι οποίοι ισαπέχουν μεταξύ τους. Π.χ. η συνάρτηση `np.linspace(1, 10, 10)` δημιουργεί το array `[1. 2. 3. 4. 5. 6. 7. 8. 9. 10.]`.

Επομένως, με τη γραμμή κώδικα:

```
x = np.linspace(100, 1000, 100)
```

δημιουργούμε ένα array που έχει 100 τιμές από το 100 έως και το 1000.

Με τη γραμμή κώδικα:

```
y = 969*(weight_of_explosives**0.6)*(x**-1.5)
```

ορίζουμε τη μεταβλητή `y` που αντιπροσωπεύει τις τιμές του κάθετου άξονα `y` του διαγράμματος, δηλαδή του άξονα της `ppv`. Στη μεταβλητή `y` εκχωρούμε τιμές που προκύπτουν από τον υπολογισμό των πράξεων δεξιά του `=`. Επειδή η μεταβλητή `x`, που είναι array, συμμετέχει στον υπολογισμό αυτό, αυτό έχει ως αποτέλεσμα η μεταβλητή `y` να είναι και αυτή ένα array, με τιμές τόσες, όσες έχει η μεταβλητή `x`.

Με τις γραμμές κώδικα:

```
plt.plot(x, y)
plt.xlabel('Distance from blast [m]')
plt.ylabel('ppv [mm/s]')
plt.savefig('ppv_vs_distance.png')
plt.show()
```

καλούμε μια σειρά από συναρτήσεις του module `pyplot` της βιβλιοθήκης `matplotlib`, οι οποίες διαδοχικά κάνουν τις εξής εργασίες:

`plt.plot(x, y)` Σχεδιάζει την καμπύλη η οποία ενώνει τα σημεία (x, y) που δημιουργούνται από τα ζεύγη τιμών των `array x` και `y`, χωρίς να εμφανίζει τα σημεία αυτά στο διάγραμμα. Η `pyplot` έχει βέβαια τη δυνατότητα να το κάνει, αλλά τα σημεία είναι τόσα πολλά (100) που θα χάλαν την εμφάνιση της καμπύλης.

`plt.xlabel('Distance from blast [m]')` Γράφει την επιγραφή (label) `Distance from blast [m]` στον οριζόντιο άξονα των `x`.

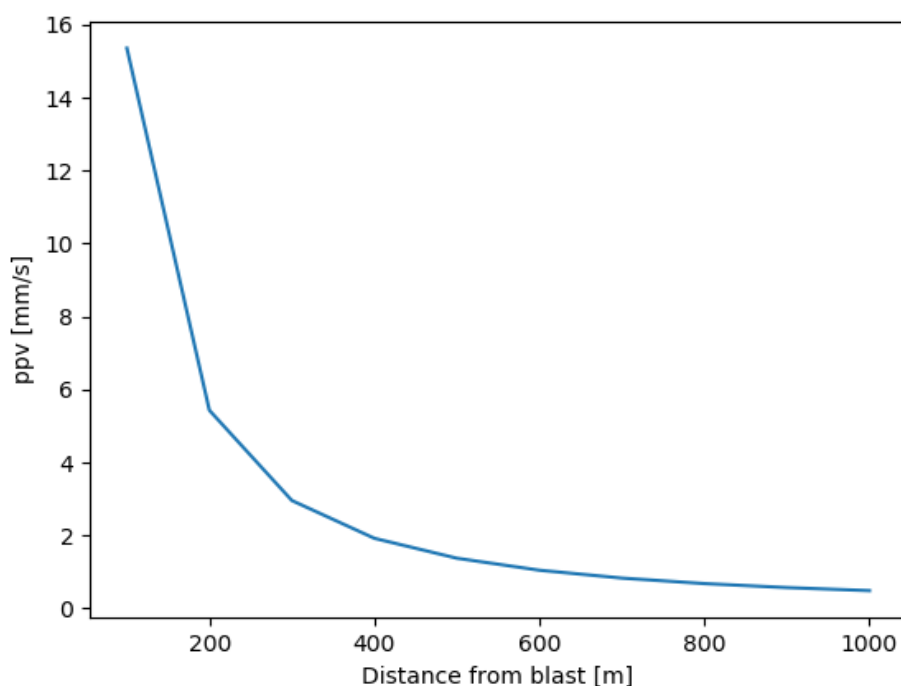
`plt.ylabel('ppv [mm/s]')` Γράφει την επιγραφή (label) `ppv [mm/s]` στον κάθετο άξονα των `y`.

`plt.savefig('ppv_vs_distance.png')` Σώζει το διάγραμμα με όνομα `ppv_vs_distance` σε αρχείο τύπου `png`. Ο `png` είναι ο default τύπος αρχείου εικόνας στον οποίο σώζει ένα αρχείο η `savefig`, αλλά μπορούμε να επιλέξουμε και άλλους τύπους αρχείων εικόνας, όπως τα `jpg` και `jpeg`.

`plt.show()` Παρουσιάζει την εικόνα της γραφικής παράστασης στην οθόνη.

Παρατήρηση: Εάν αντιστρέψετε τη σειρά των δύο τελευταίων εντολών, δηλαδή γράψετε πρώτα την `plt.show()` και μετά την `plt.savefig('ppv_vs_distance.png')` η γραφική παράσταση δεν θα σωθεί στο αρχείο που επιλέξατε!

Εάν αναρωτιέστε γιατί χρησιμοποιήσαμε τόσα πολλά σημεία (100) για τη χάραξη της καμπύλης και δεν περιοριστήκαμε στον αριθμό των 10 σημείων απόστασης, όπως κάναμε όταν δημιουργήσαμε τον πίνακα τιμών απόστασης – αναμενόμενης `ppv`, η απάντηση δίνεται *γραφικά* από το διάγραμμα της Εικόνας 6.2.



Εικόνα 6.2. Γραφική παράσταση της μεταβολής της `ppv` συναρτήσει της απόστασης, με 10 σημεία.

Για τη χάραξη της καμπύλης της Εικόνας 6.2 χρησιμοποιήσαμε τη γραμμή κώδικα `x = np.linspace(100, 1000, 10)` αντί της `x = np.linspace(100, 1000, 100)` με αποτέλεσμα η καμπύλη να είναι μια τεθλασμένη γραμμή, ενώ η καμπύλη της Εικόνας 6.1, που κατασκευάστηκε με τη γραμμή κώδικα `x = np.linspace(100, 1000, 100)`, έχει υποστεί εξομάλυνση, λόγω των πολλών ζευγών τιμών (x, y) που χρησιμοποιήσαμε για τον σχεδιασμό της.

Η βιβλιοθήκη `matplotlib` έχει πολλές δυνατότητες και επιτρέπει στον Μηχανικό προγραμματιστή να σχεδιάσει γραφικές παραστάσεις διαφόρων τύπων και μορφών, εξαιρετικής ποιότητας, επιλέγοντας από μια

πολύ μεγάλη συλλογή. Σχετικά με την ποικιλία των γραφημάτων που μπορεί να σχεδιάσει η matplotlib βλέπε στο website της βιβλιοθήκης: <https://matplotlib.org/stable/gallery/index.html>

06.4 Αποθήκευση αποτελεσμάτων σε αρχείο

Τη γραφική παράσταση της μεταβολής της ppv συναρτήσει της απόστασης την έχουμε αποθηκεύσει στο αρχείο `ppv_vs_distance.png` και επομένως μπορούμε να την ενσωματώσουμε, αργότερα, σε ένα κείμενο που γράφουμε π.χ. στο Word. Στην παράγραφο αυτή θα μάθουμε πώς να αποθηκεύσουμε σε αρχείο και τα αποτελέσματα των υπολογισμών του προγράμματός μας.

Προσθέτουμε τις παρακάτω γραμμές στο τέλος του κώδικα που παρουσιάσαμε στην παράγραφο 05.8 Ο κώδικας στο τέλος του Κεφαλαίου 05 και σώσαμε στο αρχείο `Chapter_5_BLASTppv.py`:

```
f = open('Chapter_6_BLASTppv_results.txt', 'w')
print('PROGRAMMA ΥΠΟΛΟΓΙΣΜΟΥ ΤΗΣ PPV ΑΝΑΤΙΝΑΧΗΣ', file=f)
print('INPUT DATA:', file=f)
print('PPV Prediction Equation : {:s}'.format\
      (ppv_prediction_equation), file=f)
print('Type of Rock : {:s}'.format(type_of_rock), file=f)
print('Distance from Blast [m] : {:.1f}'.format(distance_from_blast), \
      file=f)
print('Weight of Explosives [kg]: {:.1f}'.format(weight_of_explosives), \
      file=f)
print('OUTPUT:', file=f)
print('Peak Particle Velocity - ppv [mm/s] = {:.2f}'.format(ppv), \
      file=f)
f.close()
```

Για να γράψουμε δεδομένα σε ένα αρχείο πρέπει πρώτα να το *δημιουργήσουμε*, να το *ανοίξουμε*, μετά να *γράψουμε* στο αρχείο τα δεδομένα που θέλουμε και τέλος να το *κλείσουμε*.

Με τη γραμμή κώδικα:

```
f = open('Chapter_6_BLASTppv_results.txt', 'w')
```

η συνάρτηση `open()` δημιουργεί ένα αντικείμενο αρχείου, το `f`, επί του οποίου μπορούμε να εφαρμόσουμε μεθόδους. Η συνάρτηση `open()` δέχεται δύο ορίσματα: το πρώτο όρισμα είναι το όνομα του αρχείου που δημιουργούμε και ανοίγουμε και το δεύτερο όρισμα είναι ο τρόπος ανοίγματος (mode). Το mode `'w'` (write) σημαίνει ότι ανοίγουμε το αρχείο για να γράψουμε κάτι σε αυτό.

Το όνομα του αρχείου που επιλέξαμε είναι `Chapter_6_BLASTppv_results.txt` και έχει extension `.txt`, δηλαδή είναι ένα αρχείο κειμένου ώστε να μπορούμε να το ανοίγουμε π.χ. με το Notepad των Windows και να κάνουμε εργασίες copy-paste.

Το όνομα του αρχείου μπορεί να δίνεται με την πλήρη αναφορά του *μονοπατιού* του, π.χ. `C:\Documents\George\Chapter_6_BLASTppv_results.txt` ενώ χωρίς την αναφορά του μονοπατιού, εξ ορισμού, ο φάκελος του αρχείου θα είναι ο φάκελος στον οποίο βρίσκεται το script του προγράμματός μας.

Αν το αρχείο που ανοίγουμε για γράψιμο υπάρχει ήδη στον υπολογιστή μας, π.χ. από προηγούμενο τρέξιμο του προγράμματός μας, τότε τα δεδομένα που περιέχει σβήνονται όλα και αντικαθίστανται από τα νέα που θα του γράψουμε.

Με τη γραμμή κώδικα:

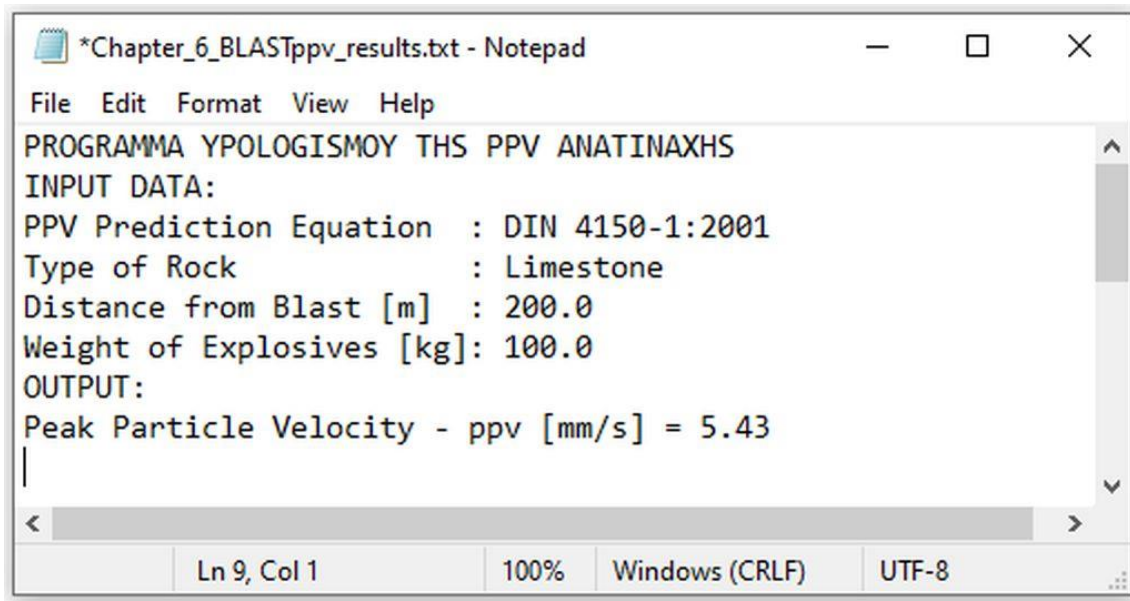
```
f.close()
```

γίνεται το κλείσιμο του αρχείου εφαρμόζοντας στο αντικείμενο του αρχείου, το `f`, τη μέθοδο `close()`.

Μεταξύ των γραμμών του κώδικα που ανοίγουν και στη συνέχεια κλείνουν το αρχείο `Chapter_6_BLASTppv_results.txt` παρεμβάλλονται οι γραμμές κώδικα με τις οποίες γράφουμε στο αρχείο τα αποτελέσματα των υπολογισμών του προγράμματός μας.

Πρόκειται για τις εντολές `print()` που χρησιμοποιήσαμε για να εμφανίσουμε τα αποτελέσματα του προγράμματός μας στην οθόνη, με την προσθήκη του ορίσματος `file=f`. Δηλαδή, τώρα δίνουμε οδηγία στην Python να κατευθύνει το `output` των εντολών `print()` στο αντικείμενο του αρχείου, το `f`, και όχι στην οθόνη.

Στην Εικόνα 6.3 παρουσιάζεται το αρχείο `Chapter_6_BLASTppv_results.txt` όπως φαίνεται ανοιγμένο στο Notepad των Windows.



Εικόνα 6.3. Το αρχείο `Chapter_6_BLASTppv_results.txt` ανοιγμένο στο Notepad των Windows.

Σημειώστε ότι ο συνηθής τρόπος εγγραφής δεδομένων σε ένα αρχείο είναι με χρήση της μεθόδου `write()`. Για παράδειγμα, γράφουμε:

```
f = open('myfile.txt', 'w')
f.write('Peak Particle Velocity - ppv [mm/s]\n')
f.write(str(5.43) + '\n')
f.close()
```

Το πρόβλημα με τη μέθοδο `write()`, και για αυτό τον λόγο δεν τη χρησιμοποιήσαμε στο πρόγραμμά μας, είναι ότι το όρισμά της πρέπει να είναι συμβολοσειρά. Αν θέλουμε να γράψουμε αριθμούς, θα πρέπει πρώτα να τους μετατρέψουμε σε συμβολοσειρές με τη βοήθεια της συνάρτησης `str()`.

Επίσης, για να αλλάξουμε γραμμή σε μια εγγραφή στο αρχείο με τη μέθοδο `write()`, χρησιμοποιούμε τον χαρακτήρα διαφυγής `\n`. Παρατηρήστε τον τρόπο χρήσης του `\n` στην 3^η γραμμή του κώδικα!

Το περιεχόμενο του αρχείου `myfile.txt` είναι:

```
Peak Particle Velocity - ppv [mm/s]
5.43
```

06.5 Ο κώδικας στο τέλος του Κεφαλαίου 06

Ο κώδικας του προγράμματός μας στο τέλος αυτού του κεφαλαίου, αφού διαγράψουμε τις γραμμές κώδικα με τις οποίες πειραματιστήκαμε εξερευνώντας διάφορα στοιχεία της Python, δίνεται στη συνέχεια.

Στον κώδικα έχουν γίνει ανακατατάξεις στη θέση των γραμμών των εντολών και έχει προστεθεί εντολή ώστε να αποθηκεύεται σε αρχείο και ο πίνακας τιμών απόστασης - αναμενόμενης `ppv`.

```

# To BLASTppv.py einai ena programma poy
# ypologizei tin ppv mias anatinaxis se metalleia/latomeia/erga
# Import of Packages
import matplotlib.pyplot as plt
import numpy as np
#
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # O Titlos
ppv_prediction_equation = 'DIN 4150-1:2001'
type_of_rock = 'Limestone'
distance_from_blast = float(input('Distance from Blast [m] :')) # m
weight_of_explosives = float(input('Weight of Explosives [kg] :')) # kg
ppv = 969*(weight_of_explosives**0.6)*(distance_from_blast**-1.5) # DIN
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # O Titlos
print('INPUT DATA:')
print('PPV Prediction Equation : {:s}'.format(ppv_prediction_equation))
print('Type of Rock : {:s}'.format(type_of_rock))
print('Distance from Blast [m] : {:.1f}'.format(distance_from_blast))
print('Weight of Explosives [kg]: {:.1f}'.format(weight_of_explosives))
print('OUTPUT:')
print('Peak Particle Velocity - ppv [mm/s] = {:.2f}'.format(ppv))
# Write results in a file
f = open('Chapter_6_BLASTppv_results.txt', 'w')
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS', file=f)
print('INPUT DATA:', file=f)
print('PPV Prediction Equation : {:s}'.format\
(ppv_prediction_equation), file=f)
print('Type of Rock : {:s}'.format(type_of_rock), file=f)
print('Distance from Blast [m] : {:.1f}'.format(distance_from_blast),\
file=f)
print('Weight of Explosives [kg]: {:.1f}'.format(weight_of_explosives),\
file=f)
print('OUTPUT:', file=f)
print('Peak Particle Velocity - ppv [mm/s] = {:.2f}'.format(ppv),\
file=f)
# Table Distance vs ppv
distance_start_value = int(float(input('Distance start-value [m] :')))
distance_end_value = int(float(input('Distance end-value [m] :')))
distance_step_value = int(float(input('Distance step-value [m] :')))
print('{:s}'.format(61*'-'))
print('Distance from Blast [m] Peak Particle Velocity - ppv [mm/s]')
print('{:s}'.format(61*'-'))
print('{:s}'.format(61*'-'), file=f)
print('Distance from Blast [m] Peak Particle Velocity - ppv [mm/s]',\
file=f)
print('{:s}'.format(61*'-'), file=f)
for distance_from_blast in range(distance_start_value,
distance_end_value + 1, distance_step_value):
    ppv = 969*(weight_of_explosives**0.6)* \
(distance_from_blast**-1.5)
    print('{:s} {:6.1f} {:s} {:6.1f}'.format(5*' ',
distance_from_blast, 22*' ', ppv))
# Write table in a file
print('{:s} {:6.1f} {:s} {:6.1f}'.format(5 * ' ',
distance_from_blast, 22 * ' ', ppv), file=f)
f.close()
# Plot Distance vs ppv
x = np.linspace(100, 1000, 100)
y = 969*(weight_of_explosives**0.6)*(x**-1.5)
plt.plot(x, y)
plt.xlabel('Distance from blast [m]')
plt.ylabel('ppv [mm/s]')

```



```
plt.savefig('ppv_vs_distance.png')
plt.show()
```

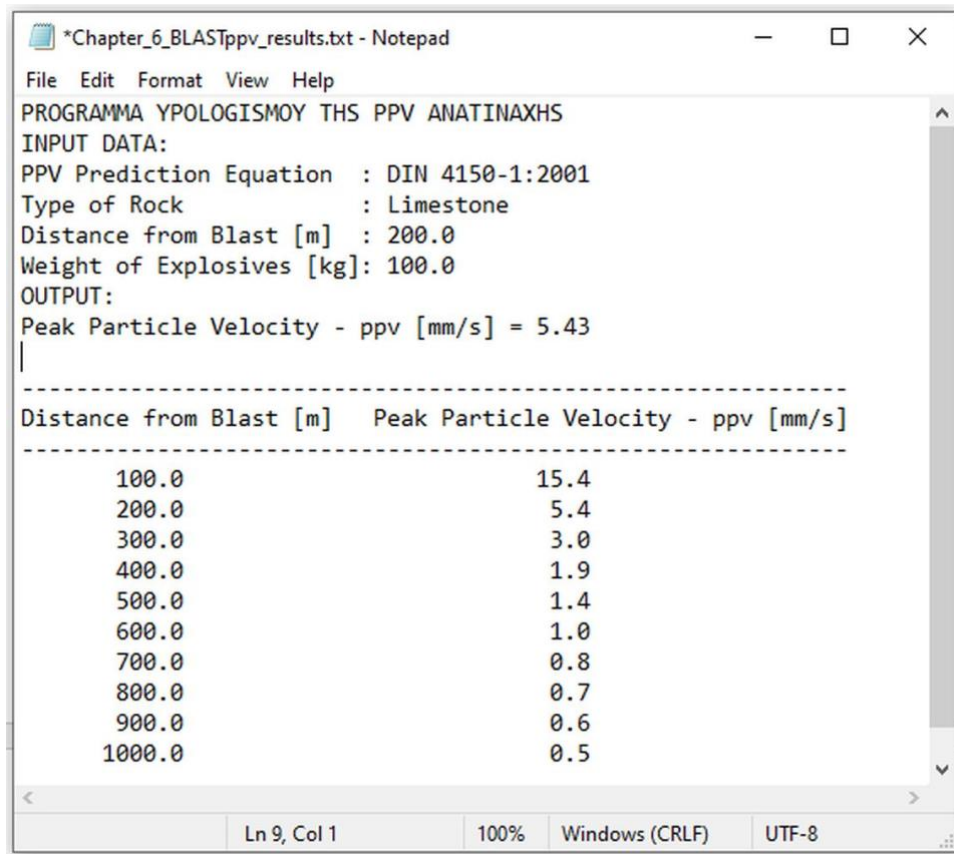
Αποθηκεύουμε το script του προγράμματός μας σε αρχείο με όνομα `Chapter_6_BLASTppv.py`.

Εάν τρέξουμε το πρόγραμμα με τιμές απόστασης από το μέτωπο ανατίναξης 200 m, ποσότητα εκρηκτικών 100 kg, και τιμές απόστασης για τη δημιουργία του πίνακα απόστασης-ppv: αρχική τιμή 100 m, τελική τιμή 1000 m και βήμα μεταβολής 100 m, στην οθόνη θα εμφανιστεί το αποτέλεσμα των υπολογισμών μας.

Επιπλέον, το πρόγραμμα δημιουργεί και σώζει δύο αρχεία:

1. το `Chapter_6_BLASTppv_results.txt`, που περιέχει τα αποτελέσματα των υπολογισμών (Εικόνα 6.4) και
2. το `ppv_vs_distance.png`, το οποίο περιέχει την εικόνα της γραφικής παράστασης της μεταβολής της ppv συναρτήσει της απόστασης (Εικόνα 6.1).

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_6_BLASTppv.py` μαζί με τα δύο παραπάνω αρχεία αποτελεσμάτων.



```
*Chapter_6_BLASTppv_results.txt - Notepad
File Edit Format View Help
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
INPUT DATA:
PPV Prediction Equation : DIN 4150-1:2001
Type of Rock : Limestone
Distance from Blast [m] : 200.0
Weight of Explosives [kg]: 100.0
OUTPUT:
Peak Particle Velocity - ppv [mm/s] = 5.43
|
-----
Distance from Blast [m]    Peak Particle Velocity - ppv [mm/s]
-----
100.0                      15.4
200.0                      5.4
300.0                      3.0
400.0                      1.9
500.0                      1.4
600.0                      1.0
700.0                      0.8
800.0                      0.7
900.0                      0.6
1000.0                     0.5
```

Εικόνα 6.4. Τα αποτελέσματα του προγράμματος `Chapter_6_BLASTppv.py` αποθηκευμένα στο αρχείο `Chapter_6_BLASTppv_results.txt` (ανοιγμένο στο Notepad των Windows).

07 Έλεγχος ροής εκτέλεσης κώδικα

07.1 Γενικά

Στη μέχρι τώρα ανάπτυξη του προγράμματος υπολογισμού της *rrn* χρησιμοποιήσαμε μόνο την Εξίσωση [2.2], η οποία υπολογίζει την *rrn* σύμφωνα με το πρότυπο DIN για ιζηματογενή πετρώματα (sedimentary rocks). Εάν η περιοχή στην οποία διαδίδεται ο κυματοπαλμός της ανατίναξης καλύπτεται από πυριγενή πετρώματα (igneous rocks), τότε για τον υπολογισμό της *rrn* πρέπει να χρησιμοποιήσουμε την Εξίσωση [2.3].

Επίσης, στην παράγραφο 02.2 *Μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους* είδαμε ότι την *rrn* μπορούμε να την υπολογίσουμε και με την Εξίσωση [2.1] που προτείνεται από το USBM. Στην περίπτωση αυτή οι παράμετροι *h-factor* και *b-factor* της Εξίσωσης [2.1] λαμβάνουν τιμές που εξαρτώνται από τον τύπο της ανατίναξης, δηλαδή εάν η ανατίναξη πραγματοποιείται σε Μεταλλείο (Mine), Λατομείο (Quarry) ή Τεχνικό Έργο (Construction).

Στη συνέχεια αυτού του Κεφαλαίου θα επεκτείνουμε τις δυνατότητες του προγράμματός μας, ώστε αυτό να υπολογίζει την *rrn* λαμβάνοντας υπόψη όλες τις επιλογές που μας παρέχει η σχετική θεωρία των δονήσεων από ανατινάξεις, η οποία παρουσιάστηκε στην παράγραφο 02.2 *Μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους*.

07.2 Οι εντολές `if-elif-else`

Οι εντολές `if-elif-else` χρησιμοποιούνται για να ελέγχουμε τη ροή εκτέλεσης του κώδικα ενός προγράμματος, εκτελώντας τμήματά του μόνο όταν ικανοποιούνται συγκεκριμένες συνθήκες. Δηλαδή, ελέγχεται μια συνθήκη και ανάλογα με το αποτέλεσμα του ελέγχου (Αληθές ή Ψευδές) εκτελείται ή δεν εκτελείται ένα μπλοκ εντολών του κώδικα.

Την εντολή `if` τη γνωρίσαμε ήδη στην παράγραφο 05.4 *Οι εντολές `break` και `continue`*, όταν είδαμε τη χρήση των εντολών `break` και `continue`.

Σε αυτό το Κεφάλαιο θα δούμε τη χρήση της ομάδας των εντολών `if-elif-else` σε πλήρη έκταση, με εφαρμογή στο πρόγραμμα υπολογισμού της μέγιστης ταχύτητας δόνησης σωματιδίων του εδάφους από μια ανατίναξη.

Στην απλούστερη εκδοχή, ο έλεγχος ροής επιτυγχάνεται με το ζεύγος εντολών `if-else` :

```
if 10 < 0:
    print('10 is less than 0')
    # End of True-block
else:
    print('10 is greater than 0')
    # End of False-block
```

Ο έλεγχος της ροής ξεκινάει με μία επικεφαλίδα που περιλαμβάνει τη δεσμευμένη λέξη `if`, μία συνθήκη (μία *Boolean* έκφραση) και τελειώνει με μία άνω-κάτω τελεία. Στη συνέχεια, ακολουθεί σε εσοχή ένα μπλοκ εντολών οι οποίες εκτελούνται εάν η συνθήκη της επικεφαλίδας είναι αληθής (`True`).

Εάν η συνθήκη είναι ψευδής (`False`), η ροή εκτέλεσης μεταφέρεται στο μπλοκ εντολών που είναι σε εσοχή κάτω από την επικεφαλίδα με τη δεσμευμένη λέξη `else` που ακολουθείται από μία άνω-κάτω τελεία.

Στη συνέχεια εκτελείται η πρώτη εντολή του κώδικα εκτός του μπλοκ.

Στο παραπάνω παράδειγμα είναι προφανές ότι η συνθήκη είναι πάντα ψευδής (`False`) και στην οθόνη εμφανίζεται το κείμενο:

10 is greater than 0

Ο κλάδος του `else` είναι προαιρετικός, δηλαδή μπορούμε να έχουμε μόνον τον κλάδο του `if` και να εκτελείται ένα μπλοκ εντολών στην περίπτωση που η συνθήκη είναι αληθής. Με τη μορφή αυτή χρησιμοποιήσαμε το `if` στην παράγραφο 05.4 Οι εντολές *break* και *continue*.

Στην πλήρη εκδοχή, ο έλεγχος ροής επιτυγχάνεται με τις εντολές `if-elif-else` ως εξής:

```
number = 0
if number < 0:
    print('number is less than 0')
    # End of True-block_1
elif number == 0:
    print('number is 0')
    # End of True-block_2
else:
    print('number is greater than 0')
    # End of False-block
```

Στο παράδειγμα αυτό θέλουμε να ελέγξουμε εάν η τιμή της μεταβλητής `number` είναι μικρότερη, ίση ή μεγαλύτερη του μηδενός.

Μεταξύ των κλάδων `if` και `else` έχουμε προσθέσει έναν κλάδο `elif` (else if), ο οποίος περιλαμβάνει τη δεσμευμένη λέξη `elif`, μία συνθήκη (μία *Boolean* έκφραση) και τελειώνει με μία άνω-κάτω τελεία. Μετά ακολουθεί, σε εσοχή, ένα μπλοκ εντολών οι οποίες εκτελούνται εάν η συνθήκη της επικεφαλίδας του κλάδου `elif` είναι αληθής (`True`).

Στη μεταβλητή `number` έχουμε εκχωρήσει την ακέραια τιμή 0 και όπως είναι αναμενόμενο, ως αποτέλεσμα του ελέγχου στην οθόνη θα εμφανιστεί το κείμενο:

number is 0

δηλαδή θα εκτελεστεί το μπλοκ εντολών του κλάδου `elif`.

Δεν υπάρχει όριο στο πλήθος των εντολών που μπορούν να υπάρχουν εντός των μπλοκ των `if`, `elif` και `else`, πρέπει όμως να υπάρχει τουλάχιστον μία εντολή. Επίσης, δεν υπάρχει όριο στο πλήθος των κλάδων `elif` που ακολουθούν μια εντολή `if`.

Πολλές φορές, ιδίως στη φάση ανάπτυξης ενός προγράμματος, δεν γνωρίζουμε ποιες ακριβώς εντολές θα πρέπει να γράψουμε σε ένα μπλοκ εντολών `if-elif-else`. Επειδή όμως πρέπει να γράψουμε τουλάχιστον μία, τότε γράφουμε την εντολή, δεσμευμένη λέξη της Python, `pass` που σημαίνει *πάσο*.

Τέλος, η Python μας δίνει τη δυνατότητα να έχουμε και εμφωλευμένες (nested) εντολές `if-elif-else`.

Για παράδειγμα, εάν τρέξουμε τις παρακάτω γραμμές κώδικα:

```
number = 7
if number < 0:
    print('number is less than 0')
else:
    if number == 0:
        print('number is 0')
    elif number > 0:
        print('number is greater than 0')
    else:
        pass
```

το κείμενο που θα εμφανιστεί στη οθόνη είναι:

number is greater than 0

Σημείωση: Στην παράγραφο 04.6 *Ακρίβεια δεκαδικών αριθμών* αναφέραμε ότι θα πρέπει να είμαστε πολύ προσεκτικοί όταν κάνουμε συγκρίσεις ισότητας μεταβλητών κατηγορίας `float` λόγω των θεμάτων ακρίβειας που υπάρχουν με τους δεκαδικούς αριθμούς. Η σύγκριση με μεταβλητές κατηγορίας `int` και ακέραιους αριθμούς, όπως στην περίπτωση των παραπάνω παραδειγμάτων, είναι ασφαλής, γιατί δεν υφίστανται θέματα ακρίβειας.

07.3 Υπολογισμός της *ppv* κατά USBM και DIN

Για να υπολογίσει το πρόγραμμά μας την *ppv* σύμφωνα με τη μέθοδο που προτείνεται από το USBM (Εξίσωση [2.1]) αλλά και κατά το πρότυπο DIN (Εξισώσεις [2.2] και [2.3]), γράφουμε τις παρακάτω γραμμές κώδικα:

```
import numpy as np
distance_from_blast = 200 # m
weight_of_explosives = 100 #kg
# USBM Method
type_of_blast = 'Mine' # 'Mine' or 'Quarry' or 'Construction'
if type_of_blast == 'Mine':
    h_factor = 1140.
    b_factor = -1.60
elif type_of_blast == 'Quarry':
    h_factor = 1090.
    b_factor = -1.82
elif type_of_blast == 'Construction':
    h_factor = 1730.
    b_factor = -1.60
else:
    pass
ppv_usbm = h_factor*((distance_from_blast/ \
                        np.sqrt(weight_of_explosives))**b_factor)
# DIN Standard
type_of_rock = 'Sedimentary' # 'Sedimentary' or 'Igneous'
if type_of_rock == 'Sedimentary':
    ppv_din = 969*(weight_of_explosives**0.6)* \
              (distance_from_blast**-1.5)
elif type_of_rock == 'Igneous':
    ppv_din = 206*(weight_of_explosives**0.8)* \
              (distance_from_blast**-1.3)
else:
    pass
# Output
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # O Titlos
print('INPUT DATA:')
print('Type of Blast           : {:s}'.format(type_of_blast))
print('Type of Rock            : {:s}'.format(type_of_rock))
print('Distance from Blast [m]  : {:.1f}'.format(distance_from_blast))
print('Weight of Explosives [kg]: {:.1f}'.format(weight_of_explosives))
print('OUTPUT:')
print('Peak Particle Velocity (USBM) [mm/s] = {:.2f}'.format(ppv_usbm))
print('Peak Particle Velocity (DIN) [mm/s]  = {:.2f}'.format(ppv_din))
```

Οι γραμμές κώδικα κάτω από τη γραμμή σχολίου `# USBM Method` υπολογίζουν την *ppv* κατά τη μέθοδο του USBM, δηλαδή με την Εξίσωση [2.1].

Επειδή στην Εξίσωση [2.1] οι παράμετροι *h-factor* και *b-factor* λαμβάνουν τιμές που εξαρτώνται από τον τύπο της ανατίναξης, ορίζουμε τη μεταβλητή `type_of_blast`, κατηγορίας `str`, στην οποία μπορούμε να εκχωρούμε τις τιμές `Mine` ή `Quarry` ή `Construction`, δηλαδή ότι η ανατίναξη πραγματοποιείται σε Μεταλλείο (`Mine`) ή Λατομείο (`Quarry`) ή Τεχνικό Έργο (`Construction`).

Στη συνέχεια με τη βοήθεια μιας ομάδας εντολών `if-elif-else` εκχωρούμε στις μεταβλητές `h-factor` και `b-factor` τις τιμές που αυτές λαμβάνουν ανάλογα με τον τύπο της ανατίναξης, σύμφωνα με τον Πίνακα 2.1 της παραγράφου 02.2 *Μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους*.

Η γραμμή κώδικα μετά το μπλοκ εντολών του `else` υπολογίζει την `ppv` κατά τη μέθοδο του USBM και εκχωρεί την τιμή της στη μεταβλητή `ppv_usb`.

Οι γραμμές κώδικα κάτω από τη γραμμή σχολίου `# DIN Standard` υπολογίζουν την `ppv` κατά το πρότυπο DIN, δηλαδή την Εξίσωση [2.2], εάν το πέτρωμα είναι ιζηματογενές ή την Εξίσωση [2.3], εάν το πέτρωμα είναι πυριγενές.

Για τον σκοπό αυτό ορίζουμε τη μεταβλητή `type_of_rock`, κατηγορίας `str`, στην οποία μπορούμε να εκχωρούμε τις τιμές `Sedimentary` ή `Igneous`, δηλαδή ότι το πέτρωμα είναι Ιζηματογενές (`Sedimentary`) ή Πυριγενές (`Igneous`).

Στη συνέχεια, με τη βοήθεια μιας ομάδας εντολών `if-elif-else` εκχωρούμε στη μεταβλητή `ppv_din` την τιμή που προκύπτει από την εφαρμογή της Εξίσωσης [2.2], εάν η μεταβλητή `type_of_rock` έχει τιμή `Sedimentary` ή την τιμή που προκύπτει από την εφαρμογή της Εξίσωσης [2.3], εάν η μεταβλητή `type_of_rock` έχει τιμή `Igneous`.

Παρατήρηση: Και στις δύο περιπτώσεις εφαρμογής της ομάδας εντολών `if-elif-else` ο κλάδος των `else` είναι περιττός, δεν κάνει κάτι, γι' αυτό και η μόνη εντολή που έχει είναι το `pass`. Ο συγγραφέας του βιβλίου προτιμά να προγραμματίζει γράφοντας το πλήρες σεντ των εντολών `if-elif-else`, γιατί κάνουν τον κώδικα περισσότερο κατανοητό (τουλάχιστον σε αυτόν...).

Στον κώδικα έχουμε ήδη εκχωρήσει τις τιμές `type_of_blast = 'Mine'` και `type_of_rock = 'Sedimentary'` (στη συνέχεια θα δούμε πώς θα τις εισάγουμε ως δεδομένα `input` από το πληκτρολόγιο) και εάν τρέξουμε το πρόγραμμα με τιμές απόστασης από το μέτωπο ανατίναξης 200 m και ποσότητα εκρηκτικών 100 kg, το αποτέλεσμα είναι:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
INPUT DATA:
Type of Blast           : Mine
Type of Rock            : Sedimentary
Distance from Blast [m] : 200.0
Weight of Explosives [kg]: 100.0
OUTPUT:
Peak Particle Velocity (USBM) [mm/s] = 9.45
Peak Particle Velocity (DIN) [mm/s] = 5.43
```

07.4 Αμυντικός προγραμματισμός

Οι τιμές των μεταβλητών `type_of_blast` και `type_of_rock` είναι λογικό να εισάγονται από το πληκτρολόγιο και όχι μέσω εντολών εκχώρησης στον κώδικα του προγράμματος.

Για τον λόγο αυτό διαγράφουμε τις γραμμές κώδικα:

```
type_of_blast = 'Mine'    # 'Mine' or 'Quarry' or 'Construction'
type_of_rock = 'Sedimentary' # 'Sedimentary' or 'Igneous'
```

και γράφουμε στην αρχή του κώδικα, τις παρακάτω γραμμές κώδικα:

```
type_of_blast = input('Type of Blast [Mine, Quarry or Construction] :')
assert type_of_blast in ['Mine', 'Quarry', 'Construction']
type_of_rock = input('Type of Rock [Sedimentary or Igneous] :')
assert type_of_rock in ['Sedimentary', 'Igneous']
```

Με την πρώτη εντολή `input()` εκχωρούμε μέσω του πληκτρολογίου την τιμή της μεταβλητής `type_of_blast`, κατηγορίας `str`, η οποία ΠΡΕΠΕΙ να είναι μία εκ των συμβολοσειρών: `Mine` ή `Quarry` ή `Construction`.

Με τη δεύτερη εντολή `input()` εκχωρούμε μέσω του πληκτρολογίου την τιμή της μεταβλητής `type_of_rock`, κατηγορίας `str`, η οποία ΠΡΕΠΕΙ να είναι μία εκ των συμβολοσειρών: `Sedimentary` ή `Igneous`.

Εάν ο χρήστης του προγράμματος πληκτρολογήσει οποιοδήποτε άλλο όνομα-συμβολοσειρά, π.χ. κάνει λάθος και αντί για `Mine` πληκτρολογήσει `Mines`, το πρόγραμμα, μετά την εισαγωγή των δεδομένων, θα *κρασάρει!*

Για να μην συμβεί αυτό πρέπει να *λάβουμε τα μέτρα μας* έγκαιρα, στο στάδιο της εισαγωγής των δεδομένων. Αυτή η τεχνική ονομάζεται *αμυντικός προγραμματισμός* (defensive programming).

Ο πιο συνηθισμένος τρόπος αμυντικού προγραμματισμού είναι να προσθέσουμε στον κώδικα *διαβεβαιώσεις* για ό,τι έχει προηγηθεί. Ελέγχουμε, δηλαδή, ότι όσες εντολές έχουν εκτελεστεί μέχρι μια εντολή διαβεβαίωσης είναι OK.

Αυτή τη δουλειά κάνει η γραμμή κώδικα:

```
assert type_of_blast in ['Mine', 'Quarry', 'Construction']
```

όπου η εντολή `assert` (δεσμευμένη λέξη της Python) ελέγχει-διαβεβαιώνει ότι η τιμή της μεταβλητής `type_of_blast` που πληκτρολογούμε εμπεριέχεται στις τιμές-μέλη της λίστας `['Mine', 'Quarry', 'Construction']`.

Εάν το αποτέλεσμα του ελέγχου είναι αληθές (`True`), η Python προχωράει στην επόμενη εντολή. Εάν το αποτέλεσμα του ελέγχου είναι ψευδές (`False`), ο debugger της Python βγάζει μήνυμα λάθους και τερματίζει την εκτέλεση του προγράμματος.

Έστω ότι έχουμε πληκτρολογήσει την απόσταση από το μέτωπο ανατίναξης και το βάρος της εκρηκτικής ύλης, αλλά κατά την πληκτρολόγηση του τύπου της ανατίναξης, αντί για το σωστό και αναμενόμενο `Mine` πληκτρολογούμε το `Mines`.

Η εντολή `assert` θα διαπιστώσει το λάθος μας και, αφού εμφανιστεί στην οθόνη μήνυμα προσδιορισμού του λάθους, η εκτέλεση του προγράμματος θα τερματιστεί.

Ακολουθεί το σχετικό output της οθόνης:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
Distance from Blast [m] :200
Weight of Explosives [kg] :100
Type of Blast [Mine, Quarry or Construction] :Mines
    assert type_of_blast in ['Mine', 'Quarry', 'Construction']
AssertionError
```

Εάν δεν είχαμε κάνει την πρόβλεψη να χρησιμοποιήσουμε την εντολή `assert` και τρέχαμε το πρόγραμμα, θα προχωρούσαμε στην ολοκλήρωση της εισαγωγής των δεδομένων και μετά το πρόγραμμα θα σταματούσε την εκτέλεσή του με ένδειξη λάθους, όπως εμφανίζεται στο σχετικό output της οθόνης που ακολουθεί:

```

PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
Distance from Blast [m] :200
Weight of Explosives [kg] :100
Type of Blast [Mine, Quarry or Construction] :Mines
Type of Rock [Sedimentary or Igneous] :Igneous
    ppv_usb = h_factor*((distance_from_blast/ \
NameError: name 'h_factor' is not defined

```

Όπως μας λέει το μήνυμα λάθους, το πρόβλημα εντοπίζεται στη γραμμή κώδικα:

```
ppv_usb = h_factor*((distance_from_blast/ \
```

όπου η μεταβλητή με όνομα `h_factor` εμφανίζεται ότι δεν έχει οριστεί.

Ο λόγος που συμβαίνει αυτό είναι απλός.

Η μεταβλητή `type_of_blast` έλαβε κατά την πληκτρολόγηση την τιμή εκχώρησης `Mines`. Η τιμή αυτή στη συνέχεια συμμετείχε στη συνθήκη των κλάδων της ομάδας εντολών `if-elif-else`, με τους οποίους γίνεται εκχώρηση τιμών στις μεταβλητές `h_factor` και `b_factor` ανάλογα με τον τύπο της ανατίναξης.

Επειδή η συμβολοσειρά `Mines` δεν είναι ίδια (`==`) με καμία από τις αποδεκτές τιμές `'Mine'`, `'Quarry'`, `'Construction'` των κλάδων `if-elif`, μεταφερόμαστε στον κλάδο `else` από τον οποίο βγαίνουμε αμέσως με την εντολή `pass` χωρίς να γίνει εκχώρηση τιμών στις μεταβλητές `h_factor` και `b_factor`.

Όταν έρθει η σειρά να εκτελεστεί η πρώτη γραμμή κώδικα μετά την ομάδα εντολών `if-elif-else`, η Python, με ανύπαρκτη τιμή στη μεταβλητή με όνομα `h_factor`, δεν μπορεί να υπολογίσει την τιμή της μεταβλητής `ppv_usb` και το πρόγραμμα *κρασάρει*.

Την ίδια δουλειά κάνει η γραμμή κώδικα:

```
assert type_of_rock in ['Sedimentary', 'Igneous']
```

όπου η εντολή `assert` ελέγχει-διαβεβαιώνει ότι η τιμή της μεταβλητής `type_of_rock` που πληκτρολογούμε εμπεριέχεται στις τιμές-μέλη της λίστας `['Sedimentary', 'Igneous']`.

Συμπέρασμα: «Ο χρόνος που γλιτώνεις μετρώντας *προσεκτικά* ένα κομμάτι ξύλο πριν το κόψεις, είναι πολύ περισσότερος από τον χρόνο που παίρνει η μέτρηση.» (Γνωμικό Αμερικάνων ξυλουργών).

07.5 Διαγράμματα: πολλές μεταβλητές σε ένα διάγραμμα

Στην παράγραφο 06.3 *Διαμορφώνοντας το διάγραμμα* είδαμε πώς μπορούμε να σχεδιάσουμε διαγράμματα δύο μεταβλητών (x, y) με τη βοήθεια του module `pyplot` της βιβλιοθήκης `matplotlib`.

Τώρα θα κατασκευάσουμε μια γραφική παράσταση που θα περιλαμβάνει και τις δύο καμπύλες μεταβολής των ppv κατά USBM και κατά DIN, συναρτήσει της απόστασης από το μέτωπο ανατίναξης.

Γράφουμε τις παρακάτω γραμμές κώδικα.

Επισημάνση!: Επειδή τις περισσότερες γραμμές από τον κώδικα που ακολουθεί τις έχουμε ήδη γράψει στον κώδικα για τον υπολογισμό της ppv κατά USBM και κατά DIN, στην παράγραφο 07.3 *Υπολογισμός της ppv κατά USBM και DIN*, οι νέες προσθήκες επισημαίνονται με κίτρινο χρώμα.

Νέες γραμμές κώδικα είναι επίσης και όλες όσες είναι κάτω από τη γραμμή σχολίου `# Plot Distance vs ppv_usb & ppv_din`

```

import matplotlib.pyplot as plt
x = np.linspace(100, 1000, 100)
# USBM Method
if type_of_blast == 'Mine':
    h_factor = 1140.

```

```

        b_factor = -1.60
    elif type_of_blast == 'Quarry':
        h_factor = 1090.
        b_factor = -1.82
    elif type_of_blast == 'Construction':
        h_factor = 1730.
        b_factor = -1.60
    else:
        pass
    ppv_usbm = h_factor*((distance_from_blast/ \
                           np.sqrt(weight_of_explosives))**b_factor)
    y_ppv_usbm = h_factor*((x/np.sqrt(weight_of_explosives))**b_factor)
    # DIN Standard
    if type_of_rock == 'Sedimentary':
        ppv_din = 969*(weight_of_explosives**0.6)* \
                  (distance_from_blast**-1.5)
        y_ppv_din = 969*(weight_of_explosives**0.6)*(x**-1.5)
    elif type_of_rock == 'Igneous':
        ppv_din = 206*(weight_of_explosives**0.8)* \
                  (distance_from_blast**-1.3)
        y_ppv_din = 206*(weight_of_explosives**0.8)*(x**-1.3)
    else:
        pass
    # Plot Distance vs ppv_usb & ppv_din
    plt.plot(x, y_ppv_usbm, 'r-')
    plt.plot(x, y_ppv_din, 'b--')
    plt.xlabel('Distance from blast [m]')
    plt.ylabel('ppv [mm/s]')
    plt.xticks(np.arange(min(x), max(x)+1, 100))
    plt.yticks(np.arange(0, max(y_ppv_usbm)+1, 1))
    plt.legend(['ppv-USBM', 'ppv-DIN'])
    plt.title('Distance vs ppv-USBM & ppv-DIN')
    plt.grid(True)
    plt.savefig('ppv_vs_distance_x2.png')
    plt.show()

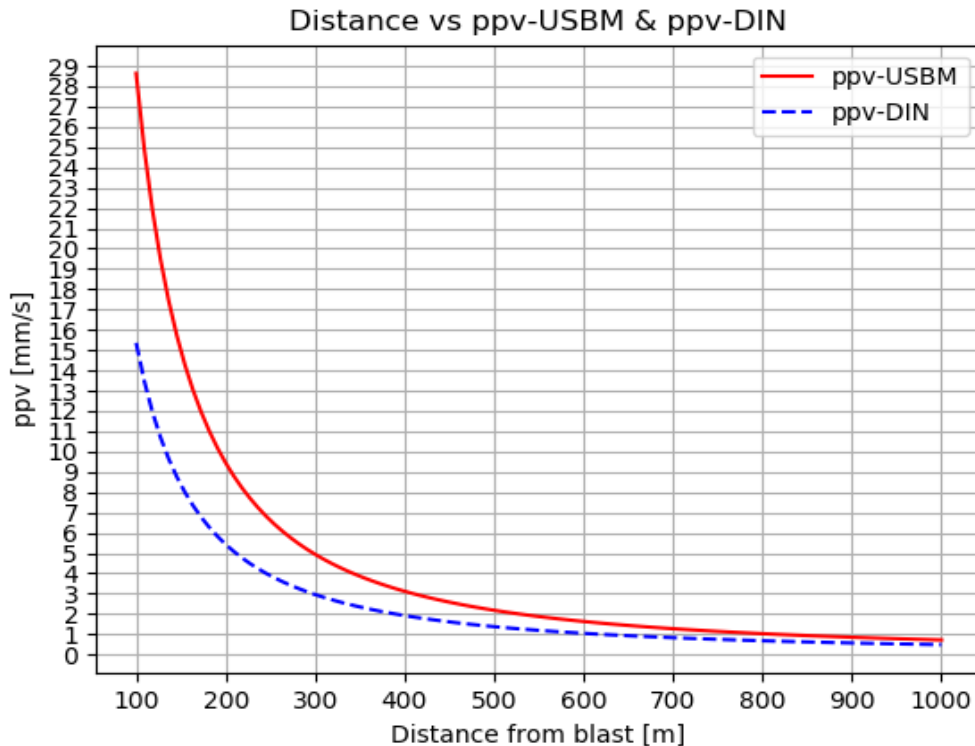
```

Στην Εικόνα 7.1 παρουσιάζεται το διάγραμμα που κατασκευάζεται από τον παραπάνω κώδικα.

Με τη γραμμή κώδικα:

```
x = np.linspace(100, 1000, 100)
```

δημιουργούμε ένα array που έχει 100 τιμές από το 100 έως και το 1000. Πρόκειται για τις αποστάσεις, σε μέτρα, για τις οποίες θα υπολογίσουμε τις μεταβλητές y_{ppv_usbm} και y_{ppv_din} , οι οποίες στη συνέχεια θα χρησιμοποιηθούν για την κατασκευή των δύο καμπυλών μεταβολής των ppv κατά USBM και κατά DIN αντίστοιχα, συναρτήσει της απόστασης.



Εικόνα 7.1. Γραφική παράσταση της μεταβολής της ppv κατά USBM και κατά DIN συναρτήσει της απόστασης από το μέτωπο ανατίναξης.

Με τη γραμμή κώδικα:

```
y_ppv_usb = h_factor*((x/np.sqrt(weight_of_explosives))**b_factor)
```

υπολογίζουμε τη μεταβλητή `y_ppv_usb`, που έχει μορφή array, με την οποία θα δημιουργήσουμε την καμπύλη μεταβολής της ppv κατά USBM.

Με τις γραμμές κώδικα:

```
y_ppv_din = 969*(weight_of_explosives**0.6)*(x**-1.5)
```

και

```
y_ppv_din = 206*(weight_of_explosives**0.8)*(x**-1.3)
```

υπολογίζουμε τη μεταβλητή `y_ppv_din`, που έχει μορφή array, με την οποία θα δημιουργήσουμε την καμπύλη μεταβολής της ppv κατά DIN. Προσέξτε τη θέση τους στον κώδικα!

Στο τμήμα του κώδικα κάτω από τη γραμμή σχολίου *# Plot Distance vs ppv_usb & ppv_din*, που σχεδιάζει το διάγραμμα, οι παρακάτω συναρτήσεις του module `pyplot` της βιβλιοθήκης `matplotlib` κάνουν τις εξής εργασίες:

`plt.plot(x, y_ppv_usb, 'r-')` Σχεδιάζει την καμπύλη η οποία ενώνει τα σημεία (`x`, `y_ppv_usb`), τα οποία δημιουργούνται από τα ζεύγη τιμών των array `x` και `y_ppv_usb`, χωρίς να εμφανίζει τα σημεία αυτά. Σε αντίθεση με τις καμπύλες που σχεδιάσαμε στην παράγραφο 06.3 *Διαμορφώνοντας το διάγραμμα*, όπου η `pyplot` χρησιμοποιούσε τις default τιμές της (ήταν στον *αυτόματο...*), εδώ με το όρισμα `r-` ζητάμε η καμπύλη να έχει κόκκινο χρώμα και να είναι συμπαγής.

`plt.plot(x, y_ppv_din, 'b--')` Σχεδιάζει την καμπύλη η οποία ενώνει τα σημεία (`x`, `y_ppv_din`) που δημιουργούνται από τα ζεύγη τιμών των array `x` και `y_ppv_din`, χωρίς να εμφανίζει τα σημεία αυτά, ενώ με το όρισμα `b--` ζητάμε η καμπύλη να έχει μπλε χρώμα και να είναι διακεκομμένη.

`plt.xlabel('Distance from blast [m]')` και `plt.ylabel('ppv [mm/s]')` Γράφουν τις επιγραφές (labels) στον οριζόντιο και τον κάθετο άξονα του διαγράμματος αντίστοιχα.

`plt.xticks(np.arange(min(x), max(x)+1, 100))` Τοποθετεί στον οριζόντιο άξονα τα σημάδια υποδιαίρεσης (ticks) και τους αριθμούς τους. Η επιλογή μας είναι το πρώτο tick να είναι η ελάχιστη τιμή του array `x` (δηλαδή το 100), το τελευταίο tick να είναι η μέγιστη τιμή του array `x` (δηλαδή το 1000) συν 1, και τα ενδιάμεσα να απέχουν μεταξύ τους 100 μονάδες (δηλαδή 100 μέτρα).

Παρατήρηση: Εάν αντί για `max(x)+1` γράφαμε `max(x)`, στον οριζόντιο άξονα δεν θα εμφανιζόταν ο αριθμός 1000 !

`plt.yticks(np.arange(0, max(y_ppv_usbm)+1, 1))` Τοποθετεί στον κάθετο άξονα τα σημάδια υποδιαίρεσης (ticks) και τους αριθμούς τους. Η επιλογή μας είναι το πρώτο tick να είναι το 0, το τελευταίο tick να είναι η μέγιστη τιμή του array `y_ppv_usbm` συν 1, και τα ενδιάμεσα να απέχουν μεταξύ τους 1 μονάδα (δηλαδή 1 mm/s).

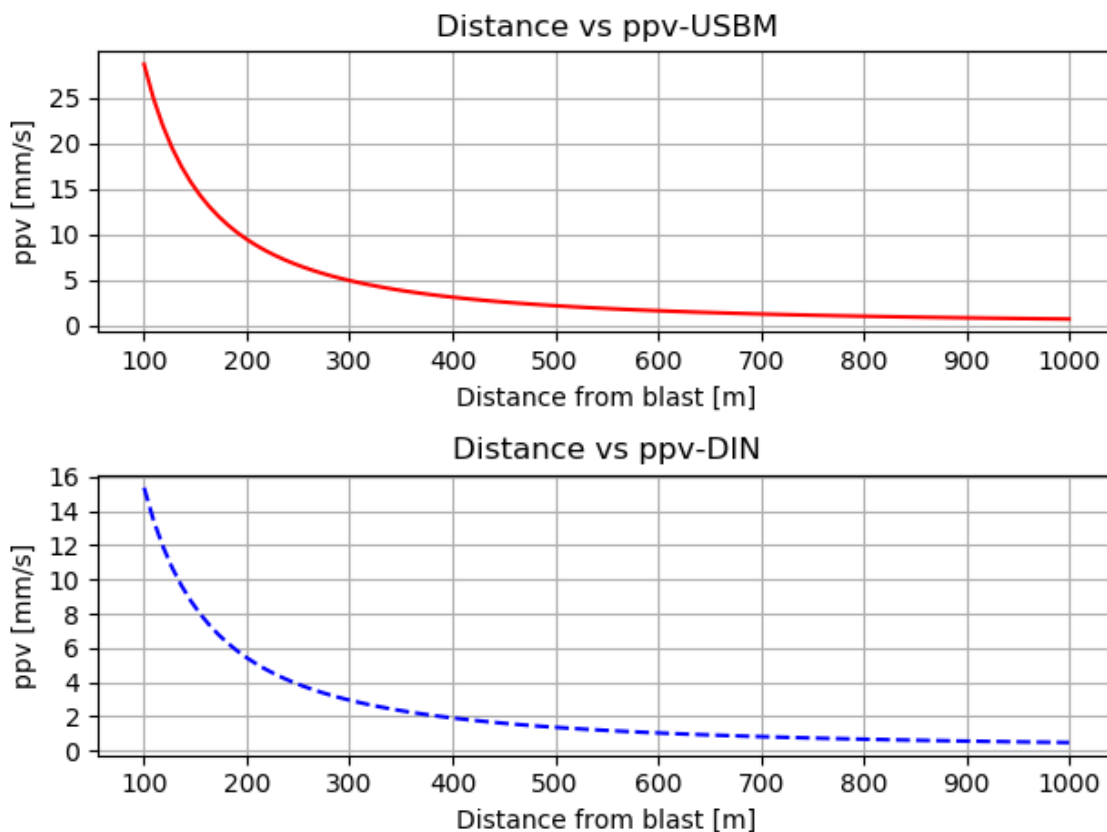
`plt.legend(['ppv-USBM', 'ppv-DIN'])` Τοποθετεί στο διάγραμμα το υπόμνημα (legend) για να γνωρίζουμε τι αντιπροσωπεύει καθεμία καμπύλη. Εδώ πρέπει να προσέξετε ώστε η σειρά με την οποία θα γράψετε το λεκτικό κάθε καμπύλης να ταυτίζεται με τη σειρά που σχεδιάσατε τις καμπύλες.

`plt.grid(True)` Τοποθετεί στο διάγραμμα ένα πλέγμα (grid), το οποίο διευκολύνει το μάτι στην εύρεση των τιμών στους άξονες που αντιστοιχούν σε σημεία των καμπυλών.

`plt.savefig('ppv_vs_distance_x2.png')` Σώζει το διάγραμμα που κατασκευάσαμε με όνομα `ppv_vs_distance_x2` σε αρχείο τύπου `png`.

`plt.show()` Παρουσιάζει την εικόνα της γραφικής παράστασης (Εικόνα 7.1) στην οθόνη.

Υπάρχει επίσης και η δυνατότητα να σχεδιάσουμε τις καμπύλες των `ppv` συναρτήσει της απόστασης, σε δύο ανεξάρτητες γραφικές παραστάσεις (subplots) αλλά στο ίδιο διάγραμμα, όπως φαίνεται στην Εικόνα 7.2.



Εικόνα 7.2. Γραφικές παραστάσεις της μεταβολής της `ppv` κατά USBM και κατά DIN συναρτήσει της απόστασης με τη μορφή των subplots.

Ο κώδικας με τον οποίο υλοποιήσαμε το διάγραμμα της Εικόνας 7.2 δίνεται στη συνέχεια:

```
# Plot Distance vs ppv_usb & Distance vs ppv_din (subplots)
plt.subplot(2, 1, 1) # ppv-USBM
plt.plot(x, y_ppv_usb, 'r-')
plt.xlabel('Distance from blast [m]')
plt.ylabel('ppv [mm/s]')
plt.xticks(np.arange(min(x), max(x)+1, 100))
plt.yticks(np.arange(0, max(y_ppv_usb)+1, 5))
plt.title('Distance vs ppv-USBM')
plt.grid(True)
#
plt.subplot(2, 1, 2) # ppv-DIN
plt.plot(x, y_ppv_din, 'b--')
plt.xlabel('Distance from blast [m]')
plt.ylabel('ppv [mm/s]')
plt.xticks(np.arange(min(x), max(x)+1, 100))
plt.yticks(np.arange(0, max(y_ppv_din)+1, 2))
plt.title('Distance vs ppv-DIN')
plt.grid(True)
#
plt.tight_layout()
plt.savefig('ppv_vs_distance_subplots.png')
plt.show()
```

Οι καινούριες συναρτήσεις του module pyplot της βιβλιοθήκης matplotlib που χρησιμοποιήσαμε στον παραπάνω κώδικα είναι:

`plt.subplot(2, 1, 1)` Δημιουργεί έναν *κάναβο* από 2 γραμμές και 1 στήλη και στη συνέχεια αναμένει τις οδηγίες – γραμμές κώδικα που ακολουθούν, για να σχεδιάσει την 1^η γραφική παράσταση (subplot 1).

`plt.subplot(2, 1, 2)` Στον *κάναβο* των 2 γραμμών και 1 στήλης αναμένει τις οδηγίες που ακολουθούν, για να σχεδιάσει τη 2^η γραφική παράσταση (subplot 2).

`plt.tight_layout()` Προσαρμόζει αυτόματα τις παραμέτρους των subplots 1 & 2 ώστε να *χωράνε* στις διαστάσεις του διαγράμματος.

Τέλος, παρατηρήστε ότι στις συναρτήσεις `plt.yticks()` έχουμε αυξήσει τις αποστάσεις που μπαίνουν τα σημάδια υποδιαίρεσης (ticks), γιατί αλλιώς οι αριθμοί τους θα γράφονταν ό ένας πάνω στον άλλο. Επίσης, σημειώστε ότι με τη συνάρτηση `plt.savefig()` σώζονται και οι δύο γραφικές παραστάσεις σε ένα αρχείο.

Όπως θα έχετε αντιληφθεί, η βιβλιοθήκη matplotlib μας προσφέρει πολλά *εργαλεία* για να κατασκευάσουμε γραφικές παραστάσεις απλές ή σύνθετες, οι οποίες μπορεί να έχουν ποικίλες μορφές εμφάνισης. Όποιος ενδιαφέρεται να εξερευνήσει όλες τις δυνατότητες της βιβλιοθήκης θα πρέπει να συμβουλευτεί το *User's Guide*, της version της matplotlib που έχει εγκαταστήσει στον υπολογιστή του, από το επίσημο website της βιβλιοθήκης <https://matplotlib.org/>

Στη συνέχεια παρατίθενται μερικές μόνο από τις δυνατότητες μορφοποίησης των διαγραμμάτων που διαθέτει το module pyplot της βιβλιοθήκης matplotlib.

07.5.1 Χρώματα

Τα βασικά χρώματα είναι (Εικόνα 7.3):

- `b` : blue (μπλε),
- `g` : green (πράσινο),
- `r` : red (κόκκινο),
- `c` : cyan (γαλάζιο),

- m : magenta (φούξια),
- y : yellow (κίτρινο),
- k : black (μαύρο),
- w : white (λευκό).

Base Colors



Εικόνα 7.3. Τα βασικά χρώματα της βιβλιοθήκης Matplotlib.

Για όποιον θέλει να έχει περισσότερες επιλογές, η βιβλιοθήκη Matplotlib διαθέτει και μια πλήρη *παλέτα χρωμάτων* (Cascading Style Sheets Colors – CSS Colors), η οποία φαίνεται στην Εικόνα 7.4.

CSS Colors

	black		bisque		forestgreen		slategrey
	dimgray		darkorange		limegreen		lightsteelblue
	dimgrey		burlywood		darkgreen		cornflowerblue
	gray		antiquewhite		green		royalblue
	grey		tan		lime		ghostwhite
	darkgray		navajowhite		seagreen		lavender
	darkgrey		blanchedalmond		mediumseagreen		midnightblue
	silver		papayawhip		springgreen		navy
	lightgray		moccasin		mintcream		darkblue
	lightgrey		orange		mediumspringgreen		mediumblue
	gainsboro		wheat		mediumaquamarine		blue
	whitesmoke		oldlace		aquamarine		slateblue
	white		floralwhite		turquoise		darkslateblue
	snow		darkgoldenrod		lightseagreen		mediumslateblue
	rosybrown		goldenrod		mediumturquoise		mediumpurple
	lightcoral		cornsilk		azure		rebeccapurple
	indianred		gold		lightcyan		blueviolet
	brown		lemonchiffon		paleturquoise		indigo
	firebrick		khaki		darkslategray		darkorchid
	maroon		palegoldenrod		darkslategrey		darkviolet
	darkred		darkkhaki		teal		mediumorchid
	red		ivory		darkcyan		thistle
	mistyrose		beige		aqua		plum
	salmon		lightyellow		cyan		violet
	tomato		lightgoldenrodyellow		darkturquoise		purple
	darksalmon		olive		cadetblue		darkmagenta
	coral		yellow		powderblue		fuchsia
	orangered		olivedrab		lightblue		magenta
	lightsalmon		yellowgreen		deepskyblue		orchid
	sienna		darkolivegreen		skyblue		mediumvioletred
	seashell		greenyellow		lightskyblue		deeppink
	chocolate		chartreuse		steelblue		hotpink
	saddlebrown		lawngreen		aliceblue		lavenderblush
	sandybrown		honeydew		dodgerblue		palevioletred
	peachpuff		darkseagreen		lightslategray		crimson
	peru		palegreen		lightslategrey		pink
	linen		lightgreen		slategray		lightpink

Εικόνα 7.4. Η πλήρης παλέτα χρωμάτων (Cascading Style Sheets Colors – CSS Colors) της βιβλιοθήκης Matplotlib.

07.5.2 Στυλ γραμμών

Τα βασικά στυλ γραμμών που χρησιμοποιεί η Matplotlib για τη σχεδίαση των καμπυλών των γραφικών παραστάσεων είναι (Εικόνα 7.5):

- '-' 'solid' line style (συμπαγής γραμμή),
- '--' 'dashed' line style (διακεκομμένη γραμμή από παύλες),
- '-.' 'dash-dot' line style (διακεκομμένη γραμμή από παύλες-τελείες),
- ':' 'dotted' line style (διακεκομμένη γραμμή από τελείες).



Εικόνα 7.5. Τα βασικά στυλ γραμμών της βιβλιοθήκης Matplotlib.

07.5.3 Πάχος γραμμών

Το default πάχος γραμμών που χρησιμοποιεί η Matplotlib για τη σχεδίαση των γραφικών παραστάσεων είναι 1.5 pixels. Με το πάχος αυτό σχεδιάσαμε όλες τις γραφικές παραστάσεις μας μέχρι στιγμής.

Το πάχος αλλάζει με το όρισμα `linewidth=k` της συνάρτησης `plot()`, όπου `k` αριθμός κατηγορίας `float`. Π.χ. η συνάρτηση: `plot(x, y, linewidth=2.5)` χαράζει τη γραμμή της καμπύλης των `x, y` με πάχος 2.5 pixels.

07.5.4 Σημάδια

Οι γραφικές παραστάσεις που σχεδιάζουμε μπορεί να έχουν *σημάδια* (markers) στα σημεία τομής των τιμών των `x, y` ή σημάδια και γραμμές ή μόνο γραμμές. Στα διαγράμματα που έχουμε κατασκευάσει μέχρι τώρα δεν χρησιμοποιήσαμε σημάδια, γιατί τα σημεία `x, y` ήταν πολλά και η γραφική παράσταση θα ήταν άσχημη.

Τα βασικά γεμισμένα σημάδια (filled markers) που χρησιμοποιεί η Matplotlib παρουσιάζονται στην Εικόνα 7.6.

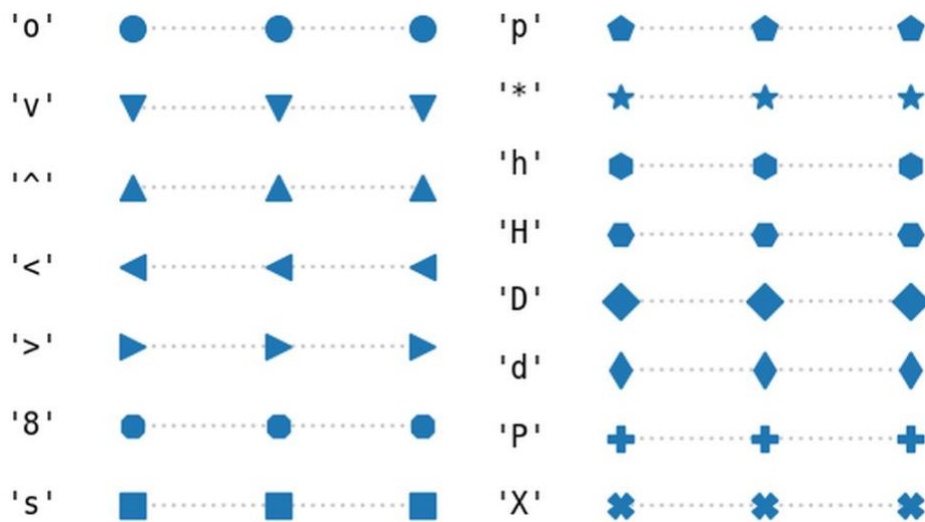
Με το όρισμα `markersize=k`, όπου `k` αριθμός κατηγορίας `float`, της συνάρτησης `plot()` μπορούμε να επιλέξουμε το μέγεθος του σημαδιού που θα χρησιμοποιήσουμε.

Με το όρισμα `markerfacecolor='color-code'`, όπου `color-code` ο κωδικός ονομασίας του χρώματος, της συνάρτησης `plot()` μπορούμε να επιλέξουμε το χρώμα του σώματος σημαδιού που θα χρησιμοποιήσουμε.

Τη συνάρτηση `plot()` μπορούμε να τη γράψουμε και με αναλυτικό-περιγραφικό τρόπο.

Ο παρακάτω κώδικας σχεδιάζει τη γραφική παράσταση της Εικόνας 7.7:

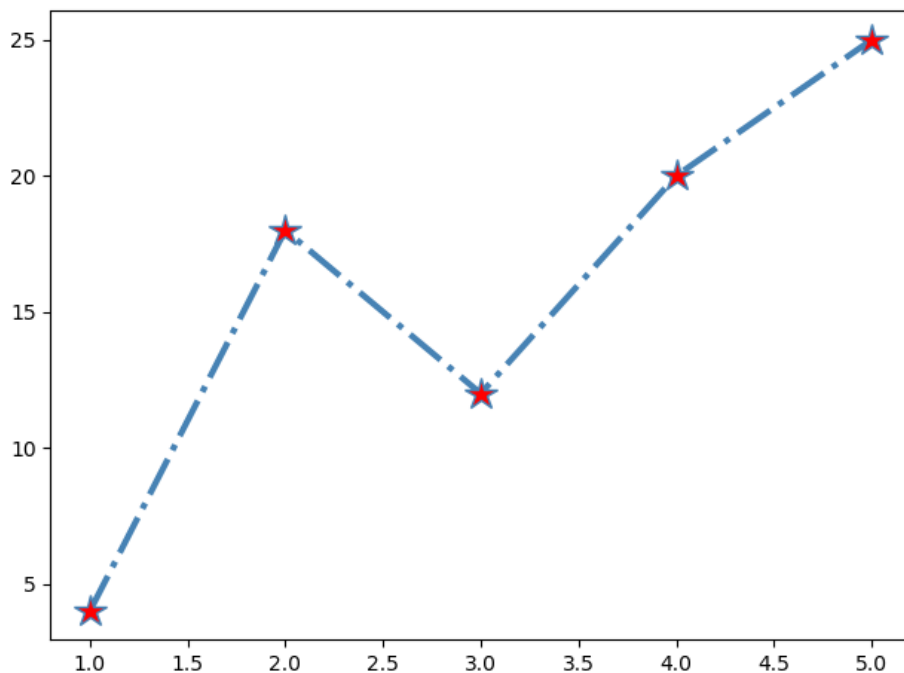
```
x = [1, 2, 3, 4, 5]
y = [4, 18, 12, 20, 25]
plt.plot(x, y, color='steelblue', marker='*', linestyle='-.-', \
         linewidth=3, markerfacecolor='r', markersize=16)
plt.show()
```



Εικόνα 7.6. Τα βασικά γεμισμένα σημάδια (filled markers) της βιβλιοθήκης Matplotlib.

Δηλαδή, σχεδιάζει στη γραφική παράσταση των x , y που λαμβάνουν τιμές από τις λίστες-array $[1, 2, 3, 4, 5]$ και $[4, 18, 12, 20, 25]$ αντίστοιχα, τοποθετώντας στα σημεία (x , y) σημάδια σχήματος αστεριού με χρώμα σώματος κόκκινο και μέγεθος 16 pixels. Τα σημάδια ενώνονται με διακεκομμένη γραμμή από παύλες-τελείες χρώματος steelblue και πάχους 3 pixels.

Παρατηρήστε ότι το περίγραμμα των αστεριών έχει το χρώμα της γραμμής, μπορούμε όμως να το αλλάξουμε. Εάν δεν είχαμε ορίσει το χρώμα του σώματος των αστεριών σε κόκκινο, τότε θα είχαν το χρώμα της γραμμής, δηλαδή steelblue.



Εικόνα 7.7. Γραφική παράσταση με σημεία και χρώματα.

07.6 Ο κώδικας στο τέλος του Κεφαλαίου 07

Ο κώδικας του προγράμματός μας στο τέλος αυτού του Κεφαλαίου, αφού διαγράψουμε τις γραμμές κώδικα με τις οποίες πειραματιστήκαμε εξερευνώντας διάφορα στοιχεία της Python, δίνεται στη συνέχεια.

Στον κώδικα, προκειμένου αυτός να γίνει πιο διαχειρίσιμος, δεν υπάρχουν οι εντολές που υπολογίζουν τον πίνακα τιμών απόστασης - αναμενόμενης ppv, καθώς και εκείνες που αποθηκεύουν τα αποτελέσματα του προγράμματος σε αρχείο. Οι εντολές αυτές υπάρχουν στον κώδικα του script που αποθηκεύσαμε στο αρχείο με όνομα Chapter_6_BLASTppv.py και μπορούν να ενσωματωθούν, εάν απαιτείται, εύκολα με κατάλληλη τροποποίηση του κώδικα.

```
# To BLASTppv.py einai ena programma pou
# ypologizei tin ppv mias anatinaxis se metalleia/latomeia/erga
# Import of Packages
import matplotlib.pyplot as plt
import numpy as np
#
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # O Titlos
distance_from_blast = float(input('Distance from Blast [m] :')) # m
weight_of_explosives = float(input('Weight of Explosives [kg] :')) # kg
type_of_blast = input('Type of Blast [Mine, Quarry or Construction] :')
assert type_of_blast in ['Mine', 'Quarry', 'Construction']
type_of_rock = input('Type of Rock [Sedimentary or Igneous] :')
assert type_of_rock in ['Sedimentary', 'Igneous']
x = np.linspace(100, 1000, 100)
# USBM Method
if type_of_blast == 'Mine':
    h_factor = 1140.
    b_factor = -1.60
elif type_of_blast == 'Quarry':
    h_factor = 1090.
    b_factor = -1.82
elif type_of_blast == 'Construction':
    h_factor = 1730.
    b_factor = -1.60
else:
    pass
ppv_usbm = h_factor*((distance_from_blast/ \
                        np.sqrt(weight_of_explosives))**b_factor)
y_ppv_usbm = h_factor*((x/np.sqrt(weight_of_explosives))**b_factor)
# DIN Standard
if type_of_rock == 'Sedimentary':
    ppv_din = 969*(weight_of_explosives**0.6)* \
              (distance_from_blast**-1.5)
    y_ppv_din = 969*(weight_of_explosives**0.6)*(x**-1.5)
elif type_of_rock == 'Igneous':
    ppv_din = 206*(weight_of_explosives**0.8)* \
              (distance_from_blast**-1.3)
    y_ppv_din = 206*(weight_of_explosives**0.8)*(x**-1.3)
else:
    pass
# Plot Distance vs ppv_usb & ppv_din
plt.plot(x, y_ppv_usbm, 'r-')
plt.plot(x, y_ppv_din, 'b--')
plt.xlabel('Distance from blast [m]')
plt.ylabel('ppv [mm/s]')
plt.xticks(np.arange(min(x), max(x)+1, 100))
plt.yticks(np.arange(0, max(y_ppv_usbm)+1, 1))
plt.legend(['ppv-USBM', 'ppv-DIN'])
plt.title('Distance vs ppv-USBM & ppv-DIN')
```

```

plt.grid(True)
plt.savefig('ppv_vs_distance_x2.png')
plt.show()
# Output on screen
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # 0 Titlos
print('INPUT DATA:')
print('Type of Blast          : {:s}'.format(type_of_blast))
print('Type of Rock           : {:s}'.format(type_of_rock))
print('Distance from Blast [m] : {:.1f}'.format(distance_from_blast))
print('Weight of Explosives [kg]: {:.1f}'.format(weight_of_explosives))
print('OUTPUT:')
print('Peak Particle Velocity (USBM) [mm/s] = {:.2f}'.format(ppv_usbm))
print('Peak Particle Velocity (DIN) [mm/s]  = {:.2f}'.format(ppv_din))
# End of program

```

Αποθηκεύουμε το script αυτής της version του προγράμματός μας σε αρχείο με όνομα Chapter_7_BLASTppv.py

Εάν τρέξουμε το πρόγραμμα με τα παρακάτω δεδομένα:

```

PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
Distance from Blast [m] :200
Weight of Explosives [kg] :100
Type of Blast [Mine, Quarry or Construction] :Mine
Type of Rock [Sedimentary or Igneous] :Sedimentary

```

στην οθόνη θα εμφανιστεί το αποτέλεσμα των υπολογισμών μας.

```

PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
INPUT DATA:
Type of Blast          : Mine
Type of Rock           : Sedimentary
Distance from Blast [m] : 200.0
Weight of Explosives [kg]: 100.0
OUTPUT:
Peak Particle Velocity (USBM) [mm/s] = 9.45
Peak Particle Velocity (DIN) [mm/s]  = 5.43

```

Επιπλέον, το πρόγραμμα δημιουργεί τη γραφική παράσταση της μεταβολής της ppv-USBM και ppv-DIN συναρτήσει της απόστασης (Εικόνα 7.1) και σώζει την εικόνα της στο αρχείο ppv_vs_distance_x2.png

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο Chapter_7_BLASTppv.py μαζί με το αρχείο της γραφικής παράστασης.

08 Δομές δεδομένων

08.1 Γενικά

Στον προγραμματισμό, *δεδομένα (data)* ονομάζουμε τις διαθέσιμες πληροφορίες που έχουμε και πρόκειται να τις χρησιμοποιήσουμε για την επίλυση ενός προβλήματος με τη βοήθεια ενός προγράμματος.

Τα δεδομένα για να χρησιμοποιηθούν αποτελεσματικά από τις γλώσσες προγραμματισμού είναι οργανωμένα σε *δομές δεδομένων (data structures)*. Οι δομές δεδομένων είναι ένα σύνολο δεδομένων (τιμών) μαζί με ένα σύνολο επιτρεπτών λειτουργιών (πράξεων) επί αυτών των δεδομένων.

Στην παράγραφο 05.2 Η εντολή επανάληψης *for*, όταν είδαμε τη χρήση της εντολής *for*, χρησιμοποιήσαμε δύο μορφές δομών δεδομένων της Python, την πλειάδα και τη λίστα. Επίσης, έχουμε χρησιμοποιήσει εκτεταμένα τις συμβολοσειρές, που μπορούν να θεωρηθούν και αυτές μια δομή δεδομένων, αφού αποτελούν μία *οργανωμένη διάταξη* αλφαριθμητικών χαρακτήρων.

Οι *πλειάδες*, οι *λίστες*, τα *λεξικά* και τα *σύνολα* είναι οι βασικές δομές δεδομένων της Python και θα τις παρουσιάσουμε σε αυτό το Κεφάλαιο.

08.2 Πλειάδες

Η *πλειάδα (tuple)* είναι μια διατεταγμένη συλλογή-ακολουθία τιμών. Οι τιμές-μέλη μιας πλειάδας ονομάζονται *στοιχεία (elements, items)* και μπορεί να είναι οποιουδήποτε τύπου (αριθμοί, συμβολοσειρές, άλλες πλειάδες ή λίστες). Το γεγονός ότι τα μέλη μιας πλειάδας είναι διατεταγμένα σημαίνει ότι κάθε στοιχείο της έχει τη δική του θέση και η θέση αυτή αντιστοιχεί σε έναν δείκτη.

Τα στοιχεία της πλειάδας χωρίζονται με κόμμα, και χωρίς να είναι αναγκαίο, είναι σύνηθες να περικλείουμε τις πλειάδες με παρενθέσεις `()`. Π.χ. οι εντολές:

```
distance_from_blast = (100, 200, 300, 400, 500, 600, 700, 800, 900, 1000)
```

και:

```
distance_from_blast = 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000
```

είναι ισοδύναμες εντολές εκχώρησης στη μεταβλητή `distance_from_blast` της πλειάδας `(100, 200, 300, 400, 500, 600, 700, 800, 900, 1000)`. Ο συγγραφέας του βιβλίου χρησιμοποιεί πάντα παρενθέσεις στις πλειάδες, γιατί κάνουν πιο ευανάγνωστο τον κώδικα.

Το κύριο χαρακτηριστικό των πλειάδων είναι ότι είναι *αμετάβλητες* ή *αμετάτρεπτες (immutable)*. Δηλαδή, από τη στιγμή που θα οριστούν δεν μπορούμε να αλλάξουμε τα στοιχεία-τιμές τους.

Επομένως, η χρήση τους περιορίζεται στις περιπτώσεις όπου πρόκειται να χρησιμοποιηθεί μια ακολουθία τιμών, η οποία δεν πρόκειται να αλλάξει σε μεταγενέστερο στάδιο του κώδικά μας. Εάν επιθυμούμε να κάνουμε σε μια ακολουθία τιμών αλλαγές στα στοιχεία της, τότε στην περίπτωση αυτή θα χρησιμοποιήσουμε μια άλλη δομή δεδομένων, τη λίστα (βλέπε παράγραφο 08.3 *Λίστες*).

Εάν θέλουμε να δημιουργήσουμε, για κάποιο λόγο, μια *άδεια πλειάδα* θα γράψουμε μόνο τις παρενθέσεις:

```
empty_tuple = ()
```

Εάν θέλουμε να δημιουργήσουμε μια πλειάδα με ένα μόνο στοιχείο, μετά το μοναδικό στοιχείο πρέπει να βάλουμε κόμμα, γιατί αλλιώς η Python θα νομίσει ότι είναι αριθμός ή συμβολοσειρά, π.χ.

```
single_element_tuple = (100,)
```

Εάν θέλουμε να μάθουμε τον αριθμό των στοιχείων που περιέχει μια πλειάδα, δηλαδή το μήκος της πλειάδας, εκτός από το να τα μετρήσουμε..., μπορούμε να χρησιμοποιήσουμε τη συνάρτηση `len()`.

Εάν γράψουμε και τρέξουμε τον παρακάτω κώδικα:


```
distance_from_blast = (100, 200, 300, 400, 500, 600, 700, 800, 900, 1000)
empty_tuple = ()
single_element_tuple = (100,)
print('Length of distance_from_blast:', len(distance_from_blast))
print('Length of empty_tuple:', len(empty_tuple))
print('Length of single_element_tuple:', len(single_element_tuple))
```

το αποτέλεσμα είναι:

```
Length of distance_from_blast: 10
Length of empty_tuple: 0
Length of single_element_tuple: 1
```

Με τις ενσωματωμένες συναρτήσεις `max()` και `min()` βρίσκουμε ποια είναι η μέγιστη και η ελάχιστη τιμή μιας πλειάδας. Για παράδειγμα, εάν γράψουμε:

```
num_tuple = (100, 110.5, 3.14)
str_tuple = ['abc', 'abd', 'abe']
print(max(num_tuple))
print(min(num_tuple))
print(max(str_tuple))
print(min(str_tuple))
```

στην οθόνη θα εμφανιστεί το παρακάτω αποτέλεσμα:

```
110.5
3.14
abe
abc
```

Παρατηρήστε πώς λειτουργούν οι συναρτήσεις `max()` και `min()` όταν οι τιμές των στοιχείων της πλειάδας είναι κατηγορίας `str`, η σύγκριση των strings στην Python γίνεται λεξικογραφικά!

Η προσπέλαση στην τιμή ενός στοιχείου της πλειάδας γίνεται με τη βοήθεια του δείκτη του στοιχείου. Ο δείκτης είναι ένας ακέραιος αριθμός και γράφεται εντός `[]`. Π.χ. εάν γράψουμε:

```
print(distance_from_blast[0])
print(distance_from_blast[9])
```

στην οθόνη μας θα εμφανιστεί η τιμή του 1^{ου} και του 10^{ου} (τελευταίου) στοιχείου της πλειάδας `distance_from_blast`:

```
100
1000
```

Βλέπουμε, δηλαδή, ότι η αρίθμηση των δεικτών αρχίζει από το 0 και τελειώνει στον αριθμό που αντιστοιχεί στο μήκος της πλειάδας μείον 1. Αυτό φαίνεται παραστατικά στη δεύτερη γραμμή του Πίνακα 8.1.

Πίνακας 8.1. Αντιστοίχιση στοιχείων και δεικτών πλειάδας ή λίστας.

distance_from_blast	100	200	300	400	500	600	700	800	900	1000
Δείκτης (+)	0	1	2	3	4	5	6	7	8	9
Δείκτης (-)	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Ο λόγος για τον οποίο ο δείκτης ξεκινά από το 0 είναι γιατί η πλειάδα αποθηκεύεται στη μνήμη του υπολογιστή σειριακά και ο δείκτης δείχνει σε ποια μετάθεση μέσα στη μνήμη βρίσκεται το συγκεκριμένο στοιχείο της πλειάδας. Για το πρώτο στοιχείο δεν χρειάζεται να μετατοπιστεί καθόλου ο δείκτης, άρα είναι ο δείκτης `[0]`, για το δεύτερο στοιχείο θα μετατοπιστεί κατά μία θέση, άρα είναι ο δείκτης `[1]`, κ.ο.κ.

Οι δείκτες μπορεί να έχουν και αρνητική αρίθμηση. Έτσι, εάν γράψουμε:

```
print(distance_from_blast[-1])
print(distance_from_blast[-10])
```

στην οθόνη μας θα εμφανιστεί η τιμή του 10^{ου} και του 1^{ου} στοιχείου της πλειάδας `distance_from_blast`:

```
1000
100
```

Αυτό φαίνεται παραστατικά στην τρίτη γραμμή του Πίνακα 8.1.

Τέλος, εάν γράψουμε:

```
print(distance_from_blast[10])
```

ένα συνηθισμένο λάθος που κάνουμε όταν θέλουμε να προσπελάσουμε το τελευταίο στοιχείο μιας πλειάδας με 10 στοιχεία, αντί του σωστού `[9]`, ο debugger της Python θα μας βγάλει την παρακάτω ένδειξη λάθους:

```
print(distance_from_blast[10])
IndexError: tuple index out of range
```

Δηλαδή, ο δείκτης που γράψαμε είναι εκτός του εύρους των δεικτών της πλειάδας `distance_from_blast` που είναι από `[0]` έως `[9]`.

Εκτός από την προσπέλαση στα στοιχεία μιας πλειάδας *ατομικά*, με τη χρήση του τελεστή των δεικτών `[]`, μπορούμε να προσπελάσουμε μια σειρά στοιχείων με τη βοήθεια του *τελεστή φέτας* (slice).

Ο τελεστής φέτας `[n:m]` μας επιστρέφει μια νέα πλειάδα που αποτελείται από το n-στό στοιχείο έως το m-1 στοιχείο της αρχικής πλειάδας. Π.χ. εάν γράψουμε:

```
sliced_distance_from_blast = distance_from_blast[1:5]
print('sliced_distance_from_blast:', sliced_distance_from_blast)
```

στην οθόνη μας θα εμφανιστεί η νέα πλειάδα `sliced_distance_from_blast` που αποτελείται από το 2^ο έως και το 5^ο στοιχείο της αρχικής πλειάδας `distance_from_blast`, γιατί στον τελεστή φέτας `[1:5]` το 1 αντιστοιχεί στο στοιχείο με δείκτη `[1]` που έχει τιμή 200 και το 5 αντιστοιχεί στο στοιχείο με δείκτη 5-1 δηλαδή `[4]` που έχει τιμή 500:

```
sliced_distance_from_blast: (200, 300, 400, 500)
```

Τελειώνοντας την παρουσίαση των πλειάδων, θα αναφέρουμε ότι υπάρχει και η συνάρτηση `tuple()` η οποία:

- εάν της δώσουμε ως όρισμα μια συμβολοσειρά, μας δημιουργεί μια πλειάδα με στοιχεία τους χαρακτήρες της συμβολοσειράς,
- εάν της δώσουμε ως όρισμα μια λίστα, μας δημιουργεί μια πλειάδα με στοιχεία τα στοιχεία της λίστας.

Έτσι, εάν γράψουμε:

```
x= tuple('George')
print(x)
y = tuple([1, 2, 3])
print(y)
```

το αποτέλεσμα στην οθόνη θα είναι:

```
('G', 'e', 'o', 'r', 'g', 'e')  
(1, 2, 3)
```

08.3 Λίστες

Η *λίστα* (list) είναι μια διατεταγμένη συλλογή-ακολουθία τιμών. Οι τιμές-μέλη μιας λίστας ονομάζονται *στοιχεία* (elements, items) και μπορεί να είναι οποιουδήποτε τύπου (αριθμοί, συμβολοσειρές, πλειάδες ή άλλες λίστες). Το γεγονός ότι τα μέλη μιας λίστας είναι διατεταγμένα σημαίνει ότι κάθε στοιχείο της έχει τη δική του θέση, και η θέση αυτή αντιστοιχεί σε έναν δείκτη.

Βλέπουμε ότι μέχρι στιγμής οι λίστες είναι όμοιες με τις πλειάδες. Η μεγάλη διαφορά τους είναι ότι οι λίστες είναι *μετατρέπτες* (mutable), σε αντίθεση με τις πλειάδες που είναι *αμετάτρεπτες* (immutable).

Δηλαδή, από τη στιγμή που θα οριστούν οι λίστες, μπορούμε, σε μεταγενέστερο στάδιο του κώδικά μας, να αλλάξουμε τα στοιχεία τους, να προσθέσουμε νέα ή να αφαιρέσουμε. Το γεγονός αυτό τις καθιστά εξαιρετικά χρήσιμες στον προγραμματισμό.

Τα στοιχεία της λίστας χωρίζονται με κόμμα και πρέπει να περικλείουμε τις λίστες με τετράγωνες αγκύλες []. π.χ.

```
distance_from_blast = [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
```

Εάν θέλουμε να δημιουργήσουμε μια *άδεια* λίστα, θα γράψουμε μόνο τις αγκύλες: `empty_list = []`.

Εάν θέλουμε να δημιουργήσουμε μια λίστα με ένα μοναδικό στοιχείο, μετά το στοιχείο (σε αντίθεση με τις πλειάδες) δεν είναι αναγκαίο να βάλουμε κόμμα. π.χ. οι εντολές `single_element_list = [100,]` και `single_element_list = [100]` είναι ισοδύναμες.

Εάν θέλουμε να μάθουμε τον αριθμό των στοιχείων που περιέχει μια λίστα, δηλαδή το μήκος της λίστας, μπορούμε να χρησιμοποιήσουμε τη συνάρτηση `len()`.

Με τις συναρτήσεις `max()` και `min()` βρίσκουμε ποια είναι η μέγιστη και η ελάχιστη τιμή μιας λίστας, με τον ίδιο τρόπο που είδαμε στην περίπτωση των πλειάδων.

Η προσπέλαση στην τιμή ενός στοιχείου της λίστας γίνεται με τη βοήθεια του δείκτη του στοιχείου. Ο δείκτης είναι ακέραιος αριθμός και γράφεται εντός []. Η χρήση των δεικτών μιας λίστας ακολουθεί την ίδια ακριβώς λογική των δεικτών των πλειάδων, την οποία παρουσιάσαμε στην παράγραφο 08.2 *Πλειάδες* (βλέπε και Πίνακα 8.1) και δεν θα την επαναλάβουμε εδώ. Το ίδιο ισχύει και για τον τελεστή φέτας μιας λίστας.

Το γεγονός ότι οι λίστες είναι μετατρέπτες, μας επιτρέπει να κάνουμε πράξεις επί αυτών ή των στοιχείων τους.

Έστω ότι έχουμε τις λίστες:

```
distance_from_blast_1 = [100, 200, 300, 400, 500]
```

και

```
distance_from_blast_2 = [600, 700, 800, 900, 1000]
```

το άθροισμα των δύο λιστών:

```
add_lists = distance_from_blast_1 + distance_from_blast_2
```

είναι: `[100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]`

ενώ το γινόμενο μιας λίστας επί έναν ακέραιο αριθμό δημιουργεί μια νέα λίστα με στοιχεία, τα στοιχεία της αρχικής λίστας επαναλαμβανόμενα τόσες φορές, όσες αντιστοιχούν στον ακέραιο αριθμό. Π.χ. το αποτέλεσμα του γινομένου:

```
multiply_list = 2*distance_from_blast_1
```

είναι: `[100, 200, 300, 400, 500, 100, 200, 300, 400, 500]`

Επίσης, με χρήση της ενσωματωμένης συνάρτησης `sum()` μπορούμε να βρούμε το άθροισμα των στοιχείων μιας λίστας, εφόσον αυτά είναι αριθμοί, Π.χ. η εντολή:

```
sum_list = sum(distance_from_blast_1)
```

θα δώσει ως αποτέλεσμα το: 1500

Οι πράξεις επί των στοιχείων μιας λίστας γίνονται *διατρέχοντας* τη λίστα με τη βοήθεια της εντολής `for` και του τελεστή `in`.

Επίσης, μπορούμε να κάνουμε όλες τις πράξεις (πρόσθεση, πολλαπλασιασμό, διαίρεση, ύψωση σε δύναμη, κλπ.) ή να εφαρμόσουμε συναρτήσεις επί των στοιχείων μιας λίστας π.χ. `sqrt()`, `cos()` κ.ά.

Για παράδειγμα, με την παρακάτω εντολή προσθέτουμε τον αριθμό 10 σε κάθε στοιχείο της λίστας `distance_from_blast_1`

```
add_to_list_item = [i + 10 for i in distance_from_blast_1]
```

και το αποτέλεσμα είναι η νέα λίστα: `[110, 210, 310, 410, 510]`

Με τη χρήση της ενσωματωμένης συνάρτησης `zip()` μπορούμε να προσπελάσουμε *παράλληλα* τα στοιχεία δύο ή περισσότερων λιστών που έχουν ίδιο μήκος. Για παράδειγμα, με την εντολή:

```
zip_lists = [i + k for i, k in zip(distance_from_blast_1, distance_from_blast_2)]
```

παίρνουμε το πρώτο στοιχείο από την πρώτη λίστα (`distance_from_blast_1`) και το πρώτο στοιχείο από τη δεύτερη λίστα (`distance_from_blast_2`), τα προσθέτουμε μεταξύ τους και το άθροισμα είναι το πρώτο στοιχείο μιας νέας λίστας, της `zip_lists`. Η ίδια διαδικασία επαναλαμβάνεται με τα δεύτερα, τρίτα, κ.ο.κ. στοιχεία των λιστών, ώστε το τελικό αποτέλεσμα να είναι η λίστα:

`[700, 900, 1100, 1300, 1500]`

Κατ' αναλογία με τη συνάρτηση `tuple()` που είδαμε στις πλειάδες, στις λίστες χρησιμοποιούμε τη συνάρτηση `list()` η οποία:

- εάν της δώσουμε ως όρισμα μια συμβολοσειρά, μας δημιουργεί μια λίστα με στοιχεία τους χαρακτήρες της συμβολοσειράς,
- εάν της δώσουμε ως όρισμα μια πλειάδα, μας δημιουργεί μια λίστα με στοιχεία τα στοιχεία της πλειάδας.

Έτσι, εάν γράψουμε:

```
x= list('George')
print(x)
y = list((1, 2, 3))
print(y)
```

το αποτέλεσμα στην οθόνη θα είναι:

```
['G', 'e', 'o', 'r', 'g', 'e']
[1, 2, 3]
```

08.3.1 Μέθοδοι στις λίστες

Στις λίστες μπορούμε να εφαρμόσουμε μια σειρά από χρήσιμες μεθόδους.

Η μέθοδος `split()` δημιουργεί μια λίστα από μια συμβολοσειρά κατά διαφορετικό τρόπο απ' ότι η συνάρτηση `list()`, π.χ. εάν γράψουμε:

```
name = 'George Panagiotou'
y = name.split()
print(y)
x = list(name)
print(x)
```

το αποτέλεσμα στην οθόνη θα είναι:

```
['George', 'Panagiotou']
['G', 'e', 'o', 'r', 'g', 'e', ' ', 'P', 'a', 'n', 'a', 'g', 'i', 'o', 't', 'o', 'u']
```

Παρατηρούμε ότι η μέθοδος `split()` δημιουργεί μια νέα λίστα με στοιχεία τις λέξεις (δηλαδή τους συνεχόμενους χαρακτήρες) της συμβολοσειράς. Ο χαρακτήρας *κενό διάστημα* (whitespace) θεωρείται ως το όριο των λέξεων. Αντιθέτως, η συνάρτηση `list()` θεωρεί τον χαρακτήρα *κενό διάστημα* στοιχείο της λίστας, όπως φαίνεται στη δεύτερη λίστα (`x`) που εμφανίζεται στην οθόνη.

Η μέθοδος `join()` είναι η αντίστροφη της `split()` και συνενώνει τα στοιχεία μιας λίστας που είναι συμβολοσειρές, κάνοντας χρήση ενός *οριοθέτη* (delimiter), ο οποίος προσδιορίζει εάν η συνένωση θα είναι με κενό ή όχι. π.χ. εάν γράψουμε:

```
name_list = ['George', 'Panagiotou']
delimiter_1 = ''
full_name_1 = delimiter_1.join(name_list)
print(full_name_1)
delimiter_2 = ' '
full_name_2 = delimiter_2.join(name_list)
print(full_name_2)
```

το αποτέλεσμα στην οθόνη θα είναι:

```
GeorgePanagiotou
George Panagiotou
```

Η μέθοδος `append()` προσθέτει ένα στοιχείο στο τέλος μιας λίστας. Π.χ. εάν θέλουμε να προσθέσουμε στη λίστα `name_list` το πατρώνυμο γράφουμε:

```
name_list.append('Nikolaos')
```

και η νέα λίστα θα είναι: `['George', 'Panagiotou', 'Nikolaos']`

Η μέθοδος `pop(i)` αφαιρεί από μια λίστα το στοιχείο που αντιστοιχεί στον δείκτη `[i]` και επιστρέφει την τιμή αυτού του στοιχείου. Π.χ. εάν γράψουμε:

```
full_name_list = ['George', 'Panagiotou', 'Nikolaos']
print(full_name_list.pop(2))
print(full_name_list)
```

το αποτέλεσμα στην οθόνη θα είναι:

```
Nikolaos
['George', 'Panagiotou']
```

Παρατηρούμε ότι η μέθοδος `pop(2)` αφενός μεν μας επέστρεψε την τιμή του στοιχείου που αντιστοιχεί στον δείκτη `[2]` και μας την εμφάνισε στην οθόνη, με την εντολή `print(full_name_list.pop(2))`, αφετέρου δε αφαίρεσε το στοιχείο αυτό από την αρχική λίστα, όπως προκύπτει από την εμφάνιση του αποτελέσματος της εντολής `print(full_name_list)` στην οθόνη.

Εάν εφαρμόσουμε σε μια λίστα τη μέθοδο `pop(i)` χωρίς τιμή δείκτη, δηλαδή έτσι: `pop()`, τότε αφαιρεί το τελευταίο στοιχείο της λίστας.

Η μέθοδος `sort()` ταξινομεί τα στοιχεία κατά αύξουσα σειρά, αριθμητική ή αλφαβητική (λεξικογραφική). Π.χ. εάν γράψουμε:

```
full_name_list = ['George', 'Panagiotou', 'Nikolaos']
full_name_list.sort()
print(full_name_list)
num_list = [1, 7, 2, 16]
num_list.sort()
print(num_list)
```

το αποτέλεσμα στην οθόνη θα είναι:

```
['George', 'Nikolaos', 'Panagiotou']
[1, 2, 7, 16]
```

Παρατήρηση: Η μέθοδος `sort()`, σε αντίθεση με τη μέθοδο `pop(i)`, δεν επιστρέφει κάτι, και για τον λόγο αυτό δεν μπορώ να γράψω `print(num_list.sort())`. Εάν το κάνω, το αποτέλεσμα θα είναι: `None` δηλαδή *Τίποτα*.

08.3.2 Αλλαγές, διαγραφές και προσθήκες στοιχείων

Έστω ότι έχουμε τη λίστα:

```
distance_from_blast = [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
```

Για να αλλάξουμε την τιμή ενός στοιχείου της λίστας, εκχωρούμε στο στοιχείο αυτό, που το ονοματίζουμε με τον δείκτη του, τη νέα τιμή. Π.χ. εάν γράψουμε:

```
distance_from_blast[4] = 'here'
```

αντικαθιστούμε την τιμή 500 του 5^{ου} στοιχείου της λίστας, με την τιμή της συμβολοσειράς `here` και η νέα λίστα είναι:

```
[100, 200, 300, 400, 'here', 600, 700, 800, 900, 1000]
```

Για να αλλάξουμε την τιμές περισσότερων του ενός στοιχείων της λίστας, ονοματίζουμε τα στοιχεία αυτά με τον *τελεστή φέτας* και τους δίνουμε τις νέες τιμές υπό μορφή λίστας. Π.χ. εάν γράψουμε:

```
distance_from_blast[4:7] = ['here', 'till', 'there']
```

η νέα λίστα είναι:

```
[100, 200, 300, 400, 'here', 'till', 'there', 800, 900, 1000]
```

Για να αφαιρέσουμε-διαγράψουμε ένα ή περισσότερα στοιχεία από μια λίστα εκχωρούμε στη θέση των στοιχείων αυτών μια κενή λίστα. Π.χ. εάν γράψουμε:

```
distance_from_blast[4:7] = []
```

η νέα λίστα είναι:

```
[100, 200, 300, 400, 800, 900, 1000]
```

Εάν στη νέα μας λίστα θέλουμε να επαναφέρουμε τις αρχικές τιμές, δηλαδή να προσθέσουμε τρία στοιχεία με τιμές 400, 500, 600 παρεμβάλλοντάς τα μεταξύ των στοιχείων με τιμές 400 και 800, θα γράψουμε:

```
distance_from_blast[4:4] = [500, 600, 700]
```

και η νέα λίστα είναι πλέον ίδια με την αρχική:

```
[100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
```

Τελειώνοντας, να πούμε ότι διαγραφή ενός ή περισσότερων στοιχείων μιας λίστας γίνεται και με τη χρήση του τελεστή `del`. π.χ. γράφοντας:

```
del distance_from_blast[4:7]
```

και το αποτέλεσμα θα είναι το ίδιο με εκείνο που πήραμε χρησιμοποιώντας την εντολή:

```
distance_from_blast[4:7] = []
```

08.4 Λεξικά

Το **λεξικό** (dictionary) είναι μια *μη-διατεταγμένη* συλλογή σύνθετων τιμών δεδομένων. Κάθε στοιχείο-μέλος ενός λεξικού αποτελείται από ένα ζευγάρι τιμών που έχει τη μορφή *κλειδί:τιμή* (key:value). Τα στοιχεία του λεξικού χωρίζονται με κόμμα και είναι αναγκαίο να περικλείουμε τα λεξικά σε *άγκιστρα* { }.

Το κλειδί είναι συνήθως μια συμβολοσειρά, αλλά μπορεί να χρησιμοποιηθούν και αριθμοί ή πλειάδες, δηλαδή μόνο αμετάβλητα αντικείμενα. Δεν επιτρέπεται σε ένα λεξικό να υπάρχουν δύο ή περισσότερα ίδια κλειδιά, δηλαδή κάθε κλειδί είναι μοναδικό!

Σε κάθε κλειδί αντιστοιχεί μία τιμή. Η τιμή μπορεί να είναι οποιουδήποτε τύπου: αριθμοί, συμβολοσειρές, πλειάδες, λίστες ή άλλα λεξικά.

Το γεγονός ότι τα στοιχεία ενός λεξικού είναι *μη-διατεταγμένα* σημαίνει ότι κάθε στοιχείο του δεν έχει τη δική του θέση, και κατά συνέπεια δεν είναι δυνατή η προσπέλαση των στοιχείων του λεξικού με τη βοήθεια δεικτών, όπως γίνεται στις πλειάδες και τις λίστες. Η προσπέλαση ενός ή περισσότερων στοιχείων του λεξικού γίνεται με τη χρήση των κλειδιών.

Τα λεξικά είναι *μετατρεπτά* (mutable) και μπορούμε εύκολα να προσθέσουμε, να διαγράψουμε στοιχεία ή να αλλάξουμε τις τιμές τους. Το γεγονός αυτό τα καθιστά εξαιρετικά χρήσιμα στον προγραμματισμό.

Για να δημιουργήσουμε ένα *άδειο* λεξικό θα γράψουμε μόνο τα άγκιστρα: `empty_dictionary = {}`

Για να αρχίσουμε να *γεμίζουμε* το άδειο λεξικό θα γράψουμε εντολές του τύπου:

```
empty_dictionary['H'] = 1.00794
```

Εάν θέλουμε να προσθέσουμε περισσότερα του ενός στοιχεία στο λεξικό, μπορούμε να χρησιμοποιήσουμε τη μέθοδο `update()` όπως παρακάτω (προσέξτε τις παρενθέσεις και τα άγκιστρα):

```
empty_dictionary.update({'O': 15.9994, 'N': 14.0067})
```

Εάν τώρα ζητήσουμε από την Python να μας εμφανίσει στην οθόνη το περιεχόμενο του λεξικού `empty_dictionary`, με την εντολή `print(empty_dictionary)`, το αποτέλεσμα θα είναι:

```
{'H': 1.00794, 'O': 15.9994, 'N': 14.0067}
```

Το λεξικό μας πλέον απέκτησε περιεχόμενο!

Ο τελεστής `del` αφαιρεί ένα ζευγάρι κλειδιού-τιμής από ένα λεξικό. Για παράδειγμα, με την εντολή:

```
del empty_dictionary['N']
```

αφαιρούμε από το λεξικό το στοιχείο που έχει κλειδί το `'N'` και το περιεχόμενο του λεξικού τώρα είναι:

```
{'H': 1.00794, 'O': 15.9994}
```

Ο συνηθισμένος τρόπος για να δημιουργήσουμε από την αρχή ένα λεξικό, του οποίου γνωρίζουμε το περιεχόμενο, είναι να το πληκτρολογήσουμε στον κώδικά μας.

Με τον τρόπο αυτό ο συγγραφέας του βιβλίου δημιούργησε το λεξικό `elements_AW_Dix` που περιέχει τα ατομικά βάρη των χημικών στοιχείων και το οποίο χρησιμοποιεί στα προγράμματα που αναπτύσσει:

```
# Atomic Weights of Chemical Elements
# {'symbol': relative atomic mass (in g/atom)}
# (Source: NIST-JANAF Thermochemical Tables, 4th Edition, Jan1998)
```

```
elements_AW_Dix = {
    'Ac': 227.0278, 'Ag': 107.8682, 'Al': 26.98154, 'Ar': 39.948,
    'As': 74.9216, 'Au': 196.9665, 'B': 10.81, 'Ba': 137.33,
    'Be': 9.01218, 'Bi': 208.9804, 'Br': 79.904, 'C': 12.011,
    'Ca': 40.08, 'Cd': 112.418, 'Ce': 140.12, 'Cl': 35.453,
    'Co': 58.9312, 'Cr': 51.996, 'Cs': 132.9054, 'Cu': 63.546,
    'D': 2.014102, 'Dy': 162.50, 'Er': 167.26, 'Eu': 151.96,
    'F': 18.998403, 'Fe': 55.847, 'Ga': 69.72, 'Gd': 157.25,
    'Ge': 72.59, 'H': 1.00794, 'He': 4.00260, 'Hf': 178.49,
    'Hg': 200.59, 'Ho': 164.9304, 'I': 126.9045, 'In': 114.82,
    'Ir': 192.22, 'K': 39.0983, 'Kr': 83.80, 'La': 138.9055,
    'Li': 6.941, 'Lu': 174.967, 'Mg': 24.305, 'Mn': 54.9380,
    'Mo': 95.94, 'N': 14.0067, 'Na': 22.98977, 'Nb': 92.9064,
    'Nd': 144.24, 'Ne': 20.179, 'Ni': 58.69, 'Np': 237.0482,
    'O': 15.9994, 'Os': 190.2, 'P': 30.97376, 'Pa': 231.0359,
    'Pb': 207.2, 'Pr': 140.9077, 'Pt': 195.0, 'Ra': 226.0254,
    'Rb': 85.4678, 'Re': 186.207, 'Rh': 102.9055, 'Ru': 101.07,
    'S': 32.06, 'Sb': 121.75, 'Sc': 44.9559, 'Se': 78.96,
    'Si': 28.0855, 'Sm': 150.36, 'Sn': 118.69, 'Sr': 87.62,
    'Ta': 180.9479, 'Tb': 158.9254, 'Te': 127.60, 'Th': 232.0381,
    'Ti': 47.88, 'Tl': 204.383, 'Tm': 168.9342, 'U': 238.0289,
    'V': 50.9415, 'W': 183.85, 'Xe': 131.29, 'Y': 88.9059,
    'Yb': 173.04, 'Zn': 65.38, 'Zr': 91.22}
```

Εάν θέλουμε να μάθουμε τον αριθμό των στοιχείων που περιέχει το λεξικό, δηλαδή των χημικών στοιχείων που περιέχει το λεξικό `elements_AW_Dix`, θα χρησιμοποιήσουμε τη συνάρτηση `len()`. Γράφοντας:

```
print(len(elements_AW_Dix))
```

παίρνουμε ως αποτέλεσμα τον αριθμό 87, αντί του αναμενόμενου αριθμού των 118 στοιχείων που υπάρχουν στην τελευταία έκδοση του περιοδικού πίνακα των στοιχείων.

Πράγματι το λεξικό `elements_AW_Dix` δεν περιέχει τα ραδιενεργά χημικά στοιχεία, των οποίων το ατομικό βάρος (για την ακρίβεια η *σχετική ατομική μάζα*) δεν μπορεί να προσδιοριστεί επακριβώς, εάν δεν είναι γνωστή η προέλευσή τους, καθώς και τα στοιχεία που έχουν προστεθεί στον περιοδικό πίνακα τα τελευταία χρόνια. Στην τελευταία του έκδοση (2018), ο περιοδικός πίνακας περιλαμβάνει 118 χημικά στοιχεία [<https://iupac.org/what-we-do/periodic-table-of-elements>].

Ο τελεστής `in` ελέγχει αν υπάρχει ένα συγκεκριμένο κλειδί σε ένα λεξικό. Δεν μπορεί να ελέγξει εάν υπάρχει μια συγκεκριμένη τιμή!

Έτσι, εάν αναζητήσουμε το ατομικό βάρος του συνθετικού ραδιενεργού χημικού στοιχείου με σύμβολο 'Og' που ονομάστηκε *Oganesson* προς τιμή του Ρώσου πυρηνικού φυσικού *Yuri Tsolakovich Oganessian* το 2016, και είναι το 118^ο (τελευταίο) στοιχείο του περιοδικού πίνακα, με την εντολή:

```
print('Og' in elements_AW_Dix)
```

το αποτέλεσμα θα είναι: `False` δηλαδή δεν υπάρχει στο λεξικό `elements_AW_Dix`

Ένας άλλος τρόπος με τον οποίο μπορούμε να δημιουργήσουμε ένα λεξικό είναι με τη χρήση της ενσωματωμένης συνάρτησης `dict()`. Για παράδειγμα, οι εντολές:

```
some_AW = dict(Ac = 227.0278, Ag = 107.8682, Al = 26.98154)
print(some_AW)
```

δημιουργούν και εμφανίζουν στην οθόνη το λεξικό:


```
{'Ac': 227.0278, 'Ag': 107.8682, 'Al': 26.98154}
```

Με τη μέθοδο `keys()` επιστρέφουμε μια *εικόνα* (view) των κλειδιών ενός λεξικού. Για παράδειγμα, οι εντολές:

```
keys_some_AW = some_AW.keys()
print(keys_some_AW)
```

εμφανίζουν στην οθόνη τα κλειδιά:

```
dict_keys(['Ac', 'Ag', 'Al'])
```

Με τη μέθοδο `values()` επιστρέφουμε μια *εικόνα* (view) των τιμών ενός λεξικού. Για παράδειγμα, οι εντολές:

```
values_some_AW = some_AW.values()
print(values_some_AW)
```

εμφανίζουν στην οθόνη τις τιμές:

```
dict_values([227.0278, 107.8682, 26.98154])
```

Με τη μέθοδο `items()` επιστρέφουμε μια *εικόνα* (view) των ζευγαριών κλειδιών-τιμών ενός λεξικού. Για παράδειγμα, οι εντολές:

```
items_some_AW = some_AW.items()
print(items_some_AW)
```

εμφανίζουν στην οθόνη τα ζευγάρια κλειδιών-τιμών:

```
dict_items([('Ac', 227.0278), ('Ag', 107.8682), ('Al', 26.98154)])
```

08.4.1 Άντληση δεδομένων από λεξικά

Ο μπετονίτης είναι ένα βιομηχανικό ορυκτό ιδιαίτερης σημασίας για τη χώρα μας. Η εξόρυξη πραγματοποιείται κατά κύριο λόγο στα ορυχεία της νήσου Μήλου και σε μικρότερες ποσότητες στη νήσο Κίμωλο. Η ελληνική παραγωγή μπετονίτη κατατάσσεται πρώτη σε μέγεθος πανευρωπαϊκά και δεύτερη παγκόσμια. Ο χημικός τύπος του μπετονίτη είναι $\text{Al}_2\text{H}_2\text{Na}_2\text{O}_{13}\text{Si}_4$.

Θα υπολογίσουμε το Μοριακό Βάρος (MW) του μπετονίτη αντλώντας δεδομένα, δηλαδή τα ατομικά βάρη των στοιχείων του χημικού τύπου του, από το λεξικό `elements_AW_Dix`.

Κάτω από τον κώδικα του λεξικού `elements_AW_Dix`, γράφουμε τις παρακάτω γραμμές κώδικα:

```
# BENTONITE Al2H2Na2O13Si4 molecular weight
bentonite_elements = ['Al', 'H', 'Na', 'O', 'Si']
bentonite_atoms = [2, 2, 2, 13, 4]
aw_bentonite_elements = []
for i in bentonite_elements:
    aw_bentonite_elements.append(elements_AW_Dix[i])
mw_bentonite = sum([k*m for k, m in zip(bentonite_atoms,
                                       aw_bentonite_elements)])
print('MW of Bentonite [g] = {:.3f}'.format(mw_bentonite))
```

και το αποτέλεσμα που εμφανίζεται στην οθόνη είναι:

```
MW of Bentonite [g] = 422.293
```

Στη συνέχεια, θα εξηγήσουμε τον τρόπο με τον οποίο υπολογίσαμε το MW του μπετονίτη.

Καταρχάς, δημιουργήσαμε τρεις λίστες:

- την `bentonite_elements` στην οποία εκχωρήσαμε ως τιμές τα σύμβολα των χημικών στοιχείων του μπετονίτη, με τη σειρά που εμφανίζονται στον χημικό του τύπο,
- την `bentonite_atoms` στην οποία εκχωρήσαμε ως τιμές τον αριθμό των ατόμων των χημικών στοιχείων του μπετονίτη, με τη σειρά που εμφανίζονται στη λίστα `bentonite_elements`,
- την κενή λίστα `aw_bentonite_elements` στην οποία θα συμπεριλάβουμε, στη συνέχεια, τα ατομικά βάρη των στοιχείων που εμφανίζονται στον χημικό τύπο του μπετονίτη, με την ίδια σειρά που έχουν στη λίστα `bentonite_elements`.

Το τελευταίο, το επιτυγχάνουμε με την εντολή επανάληψης `for`, με την οποία για κάθε στοιχείο (`i`) που υπάρχει στη λίστα `bentonite_elements` επιλέγουμε το ατομικό βάρος αυτού του στοιχείου από το λεξικό `elements_AW_Dix` και με τη μέθοδο `append()` το παραθέτουμε στο τέλος των τιμών της λίστας `aw_bentonite_elements`.

Με την εντολή:

```
mw_bentonite = sum([k*m for k, m in zip(bentonite_atoms, aw_bentonite_elements)])
```

κάνουμε ταυτόχρονα τα εξής:

Με τη χρήση της ενσωματωμένης συνάρτησης `zip()` πολλαπλασιάζουμε παράλληλα, δηλαδή το 1^ο με το 1^ο, το 2^ο με το 2^ο κ.ο.κ., τα στοιχεία των λιστών `bentonite_atoms` και `aw_bentonite_elements`.

Στη συνέχεια, με τη βοήθεια της συνάρτησης `sum()` προσθέτουμε τα επιμέρους γινόμενα. Το άθροισμά τους εκχωρείται στη μεταβλητή `mw_bentonite` και είναι το ζητούμενο μοριακό βάρος του μπετονίτη.

Το πρόγραμμα που υπολογίζει το μοριακό βάρος του μπετονίτη το αποθηκεύουμε στο αρχείο με όνομα `Chapter_8_MW_bentonite.py`

Στο Κεφάλαιο 15 *Χημικοί τύποι ενώσεων* θα γράψουμε προγράμματα που θα υπολογίζουν το μοριακό βάρος μιας χημικής ένωσης με έναν πιο αυτοματοποιημένο τρόπο, δίνοντας στο πρόγραμμα ως δεδομένα μόνο τον χημικό τύπο της ένωσης και βέβαια το λεξικό των ατομικών βαρών των χημικών στοιχείων.

Το script του κώδικα, στο οποίο περιλαμβάνεται και το λεξικό των ατομικών βαρών των χημικών στοιχείων `elements_AW_Dix`, βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_8_MW_bentonite.py`

08.4.2 Τα δεδομένα ξεχωριστά από τους αλγορίθμους

Επιστρέφοντας στο πρόγραμμα υπολογισμού της `rrn` και στο στάδιο ανάπτυξης που το αφήσαμε στην παράγραφο 07.6 *Ο κώδικας στο τέλος του Κεφαλαίου 07*, δηλαδή στο script του αρχείου `Chapter_7_BLASTppn.py`, παρατηρούμε ότι δεν ικανοποιεί μια κλασική αρχή του προγραμματισμού: τα δεδομένα να είναι ξεχωριστά από τους αλγορίθμους.

Πράγματι, όταν εφαρμόζουμε τη μέθοδο του USBM για τον υπολογισμό της `rrn`, χρησιμοποιούμε τις εντολές ελέγχου ροής εκτέλεσης `if-elif-else` για να εκχωρήσουμε στις μεταβλητές `h-factor` και `b-factor` τις τιμές που αυτές λαμβάνουν ανάλογα με τον τύπο της ανατίναξης.

Το ίδιο κάνουμε και όταν εφαρμόζουμε τη μέθοδο του προτύπου DIN και χρησιμοποιούμε εναλλακτικά την Εξίσωση [2.2] ή την Εξίσωση [2.3] ανάλογα με τον τύπο του πετρώματος.

Ο πιο σωστός και Pythonic τρόπος είναι να υλοποιήσουμε τον κώδικά μας με τη χρήση λεξικών, στα οποία θα συμπεριλάβουμε τις τιμές των μεταβλητών `h-factor` και `b-factor` και τις αριθμητικές σταθερές των Εξισώσεων [2.2] και [2.3].

Ο Πίνακας 2.1 της παραγράφου 02.2 *Μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους*, που μας δίνει τις τιμές των μεταβλητών `h-factor` και `b-factor` ανάλογα με τον τύπο της ανατίναξης, γράφεται υπό τη μορφή ενός λεξικού ως εξής:

```
usbm_ppv_Dix = {'Mine': [1140., -1.60], 'Quarry': [1090., -1.82],
                'Construction': [1730., -1.60]}
```

δηλαδή, οι τιμές των κλειδιών έχουν τη μορφή λίστας, όπου το πρώτο στοιχείο της λίστας είναι η τιμή της μεταβλητής *h-factor* και το δεύτερο στοιχείο της λίστας είναι η τιμή της μεταβλητής *b-factor*.

Κατ' ανάλογο τρόπο, στο λεξικό:

```
din_ppv_Dix = {'Sedimentary': [969, 0.6, -1.5],
               'Igneous': [206, 0.8, -1.3]}
```

οι τιμές των κλειδιών, υπό μορφή λίστας, είναι οι αριθμητικές σταθερές των Εξισώσεων [2.2] και [2.3] που αντιστοιχούν στον τύπο του πετρώματος.

Για τον υπολογισμό της *ppv* σύμφωνα με τη μέθοδο του USBM, η σχετική εντολή διαμορφώνεται κατάλληλα ώστε, ανάλογα με την τιμή που πληκτρολογούμε για τη μεταβλητή *type_of_blast*, να επιλέγονται οι αντίστοιχες τιμές των *h-factor* και *b-factor*:

```
# USBM Method
ppv_usbm = usbm_ppv_Dix[type_of_blast][0]*((distance_from_blast/
                                             np.sqrt(weight_of_explosives))
                                             **usbm_ppv_Dix[type_of_blast][1])
```

Κατ' ανάλογο τρόπο διαμορφώνεται η εντολή που υπολογίζει την *ppv* σύμφωνα με το πρότυπο DIN:

```
ppv_din = din_ppv_Dix[type_of_rock][0]* \
          (weight_of_explosives**din_ppv_Dix[type_of_rock][1])* \
          (distance_from_blast**din_ppv_Dix[type_of_rock][2])
```

Τέλος, οι μεταβλητές *y_ppv_usbm* και *y_ppv_din*, με τις οποίες δημιουργούνται οι καμπύλες μεταβολής της *ppv* συναρτήσει της απόστασης, υπολογίζονται τώρα από τις γραμμές κώδικα:

```
y_ppv_usbm = usbm_ppv_Dix[type_of_blast][0]* \
              ((x/np.sqrt(weight_of_explosives))
               **usbm_ppv_Dix[type_of_blast][1])
```

και

```
y_ppv_din = din_ppv_Dix[type_of_rock][0]* \
              (weight_of_explosives**din_ppv_Dix[type_of_rock][1])* \
              (x**din_ppv_Dix[type_of_rock][2])
```

08.5 Σύνολα

Το *σύνολο* (set) είναι μια μη-διατεταγμένη συλλογή - ακολουθία τιμών. Τα σύνολα είναι *μετατρέπτα* (mutable). Οι τιμές-μέλη ενός συνόλου ονομάζονται *στοιχεία* (elements, items) και πρέπει πρώτον να είναι αμετάτρεπτα και δεύτερον καθένα από αυτά να εμφανίζεται μόνο μία φορά.

Κατά συνέπεια, τα στοιχεία ενός συνόλου μπορεί να είναι αριθμοί, συμβολοσειρές ή πλειάδες, αλλά δεν μπορεί να είναι λίστες, λεξικά ή άλλα σύνολα. Το γεγονός ότι τα μέλη ενός συνόλου είναι μη-διατεταγμένα σημαίνει ότι κάθε στοιχείο του δεν έχει τη δική του θέση και κατά συνέπεια δεν είναι προσπελάσιμο μέσω ενός δείκτη. Τα σύνολα χρησιμοποιούνται στην Python για να αποδώσουμε δομές δεδομένων κατ' ανάλογο τρόπο με εκείνο των συνόλων στα μαθηματικά.

Στο βιβλίο αυτό δεν θα χρησιμοποιήσουμε δομές δεδομένων μορφής συνόλων. Στη συνέχεια, παρατίθενται λίγα χαρακτηριστικά των συνόλων για λόγους πληρότητας του αντικειμένου αυτού του κεφαλαίου.

Τα στοιχεία ενός συνόλου γράφονται εντός αγκυλών {} π.χ *a* = {0, 1, 2}

Η ενσωματωμένη συνάρτηση `set()` μετατρέπει μια συμβολοσειρά σε σύνολο. Π.χ.

```
b = set('George')
print(b)
```

```
{'G', 'g', 'e', 'r', 'o'}
```

Παρατηρήστε ότι τα στοιχεία του συνόλου `b` δεν ακολουθούν τη σειρά των χαρακτήρων-συμβόλων στη λέξη `George` και ότι επίσης *χάθηκε* το ένα `'e'`.

Στην Python με τα σύνολα μπορούμε να κάνουμε τις γνωστές πράξεις των μαθηματικών συνόλων, π.χ. ένωση (union), τομή (intersection) κ.ά. χρησιμοποιώντας τις κατάλληλες κατά περίπτωση συναρτήσεις, όπως φαίνεται στο παράδειγμα που ακολουθεί:

```
b = set('George')
print(b)
c = set('Panagiotou')
print(c)
b_union_c = b.union(c)
print('Union =', b_union_c)
b_intersection_c = b.intersection(c)
print('Intersection =', b_intersection_c)
```

τα αποτελέσματα του οποίου, όπως εμφανίζονται στην οθόνη, είναι:

```
{'G', 'o', 'e', 'r', 'g'}
{'u', 'P', 'i', 'n', 'o', 'g', 't', 'a'}
Union = {'u', 'G', 'P', 'o', 'i', 'n', 'e', 'r', 'g', 't', 'a'}
Intersection = {'o', 'g'}
```

Για περισσότερες λεπτομέρειες επί των συνόλων ανατρέξτε στο εγχειρίδιο της Python [<https://docs.python.org/3/tutorial/datastructures.html>].

08.6 Ο κώδικας στο τέλος του κεφαλαίου 08

Ο κώδικας του προγράμματός μας στο τέλος αυτού του Κεφαλαίου, αφού διαγράψουμε τις γραμμές κώδικα με τις οποίες πειραματιστήκαμε εξερευνώντας διάφορα στοιχεία της Python, δίνεται στη συνέχεια.

```
# To BLASTppv.py einai ena programma pou
# ypologizei tin ppv mias anatinaxis se metalleia/latomeia/erga
# Import of Packages
import matplotlib.pyplot as plt
import numpy as np
# Dictionaries
usbm_ppv_Dix = {'Mine': [1140., -1.60], 'Quarry': [1090., -1.82],
                'Construction': [1730., -1.60]}
din_ppv_Dix = {'Sedimentary': [969, 0.6, -1.5],
               'Igneous': [206, 0.8, -1.3]}
# Data input
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # O Titlos
distance_from_blast = float(input('Distance from Blast [m] :')) # m
weight_of_explosives = float(input('Weight of Explosives [kg] :')) # kg
type_of_blast = input('Type of Blast [Mine, Quarry or Construction] :')
assert type_of_blast in ['Mine', 'Quarry', 'Construction']
type_of_rock = input('Type of Rock [Sedimentary or Igneous] :')
assert type_of_rock in ['Sedimentary', 'Igneous']
x = np.linspace(100, 1000, 100)
# USBM Method
ppv_usbm = usbm_ppv_Dix[type_of_blast][0]*((distance_from_blast/
                                             np.sqrt(weight_of_explosives))
                                             **usbm_ppv_Dix[type_of_blast][1])
```

```

y_ppv_usbm = usbm_ppv_Dix[type_of_blast][0]* \
              ((x/np.sqrt(weight_of_explosives))
              **usbm_ppv_Dix[type_of_blast][1])

# DIN Standard
ppv_din = din_ppv_Dix[type_of_rock][0]* \
          (weight_of_explosives**din_ppv_Dix[type_of_rock][1])* \
          (distance_from_blast**din_ppv_Dix[type_of_rock][2])
y_ppv_din = din_ppv_Dix[type_of_rock][0]* \
            (weight_of_explosives**din_ppv_Dix[type_of_rock][1])* \
            (x**din_ppv_Dix[type_of_rock][2])

# Plot Distance vs ppv_usb & ppv_din
plt.plot(x, y_ppv_usbm, 'r-')
plt.plot(x, y_ppv_din, 'b--')
plt.xlabel('Distance from blast [m]')
plt.ylabel('ppv [mm/s]')
plt.xticks(np.arange(min(x), max(x)+1, 100))
plt.yticks(np.arange(0, max(y_ppv_usbm)+1, 1))
plt.legend(['ppv-USBM', 'ppv-DIN'])
plt.title('Distance vs ppv-USBM & ppv-DIN')
plt.grid(True)
plt.savefig('ppv_vs_distance_x2.png')
plt.show()

# Output on screen
print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS') # Ο Titlos
print('INPUT DATA:')
print('Type of Blast          : {:s}'.format(type_of_blast))
print('Type of Rock           : {:s}'.format(type_of_rock))
print('Distance from Blast [m] : {:.1f}'.format(distance_from_blast))
print('Weight of Explosives [kg]: {:.1f}'.format(weight_of_explosives))
print('OUTPUT:')
print('Peak Particle Velocity (USBM) [mm/s] = {:.2f}'.format(ppv_usbm))
print('Peak Particle Velocity (DIN) [mm/s]  = {:.2f}'.format(ppv_din))
# End of program

```

Αποθηκεύουμε το script του προγράμματός μας σε αρχείο με όνομα Chapter_8_BLASTppv.py

Το πρόγραμμά μας στη μορφή που έχει στο αρχείο Chapter_8_BLASTppv.py δεν κάνει κάτι διαφορετικό, σε σύγκριση με εκείνο της παραγράφου 07.6 Ο κώδικας στο τέλος του Κεφαλαίου 07 που αποθηκεύσαμε στο αρχείο Chapter_7_BLASTppv.py, και επομένως δίνει και τα ίδια αποτελέσματα, τα οποία δεν επαναλαμβάνονται εδώ.

Το script όμως του προγράμματός μας έχει γίνει τώρα πιο συμπαγές (compact), έχει δομές δεδομένων, τα δεδομένα είναι ξεχωριστά από τους αλγορίθμους και ίσως είναι πιο όμορφο...

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο Chapter_8_BLASTppv.py

09 Συναρτήσεις

09.1 Γενικά

Συναρτήσεις στα προηγούμενα Κεφάλαια του βιβλίου έχουμε δει πολλές. Έχουμε χρησιμοποιήσει ενσωματωμένες συναρτήσεις της Python, όπως τις `print()`, `range()`, `input()` κ.ά. και συναρτήσεις από τις εξωτερικές βιβλιοθήκες (packages) της Python, Matplotlib και NumPy. Όλες αυτές οι συναρτήσεις δημιουργήθηκαν-γράφτηκαν από άλλους προγραμματιστές και εμείς απλά τις χρησιμοποιούμε.

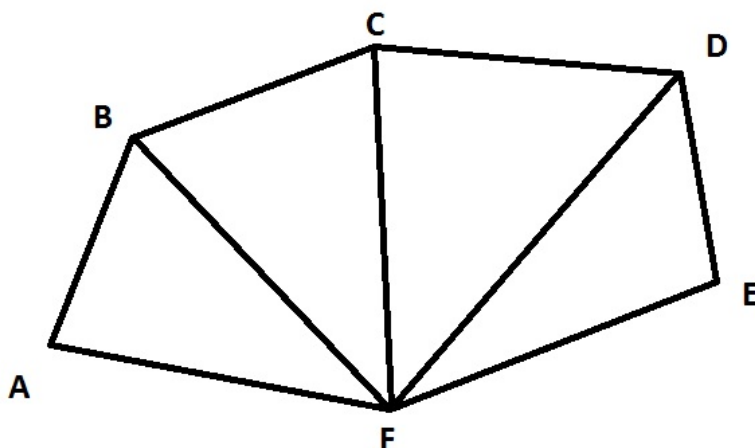
Στο Κεφάλαιο αυτό θα μάθουμε να δημιουργούμε δικές μας συναρτήσεις, τις οποίες θα μπορούμε να τις εκτελούμε, καλώντας τις με το όνομά τους, σε οποιοδήποτε σημείο του προγράμματός μας και όσες φορές θέλουμε.

Στην Python, πολύ απλά, *συνάρτηση* (function) είναι ένα μπλοκ κώδικα που τρέχει μόνο όταν το καλούμε σε κάποιο άλλο σημείο του προγράμματός μας.

09.2 Υπολογισμός εμβαδού ακανόνιστου σχήματος

Ως Μηχανικοί πρέπει πολλές φορές να υπολογίσουμε το εμβαδόν μιας έκτασης που έχει ακανόνιστο σχήμα, όπως η έκταση του πολυγώνου ABCDEF που φαίνεται στην Εικόνα 9.1.

Μια συνήθης μέθοδος είναι να χωρίσουμε το πολύγωνο σε τρίγωνα, να υπολογίσουμε το εμβαδόν καθενός τριγώνου ξεχωριστά και τέλος να αθροίσουμε τα εμβαδά των τριγώνων.



Εικόνα 9.1. Διαίρεση πολυγώνου σε τρίγωνα για τον υπολογισμό του εμβαδού του.

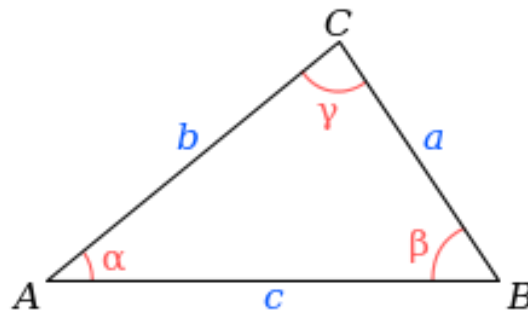
Για να υπολογίσουμε το εμβαδόν ενός τριγώνου (T) όταν γνωρίζουμε τα μήκη των πλευρών του, χρησιμοποιούμε τον τύπο του Ήρωνα [Σχετικά με τη μέθοδο του Ήρωνα ανατρέξτε στο website: [https://en.wikipedia.org/wiki/Heron's formula](https://en.wikipedia.org/wiki/Heron's_formula)]:

$$T = \sqrt{s * (s - a) * (s - b) * (s - c)} \quad [9.1]$$

όπου s είναι η ημιπερίμετρος του τριγώνου:

$$s = (a + b + c) / 2 \quad [9.2]$$

και a , b , c τα μήκη των πλευρών του τριγώνου (Εικόνα 9.2).



Εικόνα 9.2. Χαρακτηριστικά μεγέθη ενός τριγώνου για την εφαρμογή του τύπου του Ήρωνα [Πηγή: <https://el.wikipedia.org/wiki/Τρίγωνο>].

Γράφουμε τις παρακάτω γραμμές κώδικα:

```
# Calculate the area of a triangle: Heron's method
import numpy as np

def triangle_area(a, b, c):
    """Calculate and return the area of a triangle
    from the length of its sides (a, b, c) using Heron's method"""
    s = (a + b + c)/2.
    t = np.sqrt(s*(s - a)*(s - b)*(s - c))
    return t
```

Μόλις δημιουργήσαμε μια δική μας συνάρτηση για να υπολογίζει το εμβαδόν ενός τριγώνου με τον τύπο του Ήρωνα!

Την εντολή:

```
import numpy as np
```

τη γράψαμε στην αρχή του κώδικα, γιατί στη συνέχεια θα χρησιμοποιήσουμε τη συνάρτηση `sqrt()` της βιβλιοθήκης `Numpy`, η οποία υπολογίζει την τετραγωνική ρίζα ενός αριθμού, εντός της συνάρτησης που θα δημιουργήσουμε.

Στη συνέχεια, υπάρχει ένα κενό, γιατί σύμφωνα με το *PEP 8 -- Style Guide for Python Code* πριν και μετά τον ορισμό μιας συνάρτησης το σωστό είναι (κάνει τον κώδικα ευανάγνωστο) να αφήνουμε δύο κενές γραμμές.

Ακολουθεί ο ορισμός της δικής μας συνάρτησης, ο οποίος περιλαμβάνει την *επικεφαλίδα* της συνάρτησης, η οποία ξεκινάει με τη δεσμευμένη λέξη-κλειδί της Python `def` ακολουθούμενη από το όνομα με το οποίο ονοματίζουμε τη συνάρτησή μας και με ένα ζεύγος παρενθέσεων `()`, τελειώνει δε, απαραίτητως, με μια άνω-κάτω τελεία.

Στις συναρτήσεις δίνουμε ονόματα που ακολουθούν τους ίδιους κανόνες ονοματοδοσίας με εκείνους των μεταβλητών και καλό είναι το όνομα να έχει, για δική μας διευκόλυνση, σχέση με το τι κάνει η συνάρτηση.

Έτσι, στη συνάρτηση δώσαμε το όνομα `triangle_area()` γιατί υπολογίζει το εμβαδόν ενός τριγώνου, και εντός των `()` γράψαμε τα ονόματα των παραμέτρων (formal parameters) της συνάρτησης, τα `a`, `b`, `c` που αντιστοιχούν στις πλευρές του τριγώνου, χωρισμένα με κόμματα.

Όταν θέλουμε να υπολογίσουμε, στη συνέχεια του προγράμματός μας, το εμβαδόν ενός τριγώνου, π.χ το `area_triangle_1` με μήκη πλευρών 3, 2 και 4 m, θα γράψουμε τη γραμμή κώδικα:

```
area_triangle_1 = triangle_area(3, 2, 4)
```

δηλαδή θα καλέσουμε τη συνάρτηση `triangle_area()` και εντός των παρενθέσεων θα γράψουμε τις *πραγματικές παραμέτρους* (actual parameters) για τον υπολογισμό του εμβαδού.

Η Python θα αντιστοιχίσει τις πραγματικές παραμέτρους στις παραμέτρους της συνάρτησης, μία προς μία, δηλαδή το 3 → a το 2 → b και το 4 → c, η συνάρτηση θα υπολογίσει το εμβαδόν του τριγώνου και τέλος θα μας επιστρέψει την τιμή του εμβαδού, η οποία θα εκχωρηθεί στη μεταβλητή `area_triangle_1`.

Κάτω από την επικεφαλίδα της συνάρτησης γράφουμε, σε εσοχή, το μπλοκ των εντολών της συνάρτησης.

Το μπλοκ των εντολών αρχίζει με ένα *docstring* που περικλείεται από τριπλά `""" """`. Το docstring είναι μια *συμβολοσειρά τεκμηρίωσης* που περιγράφει σύντομα το τι κάνει η συνάρτηση. Λογικό είναι στην περιγραφή να περιλαμβάνονται οι παράμετροι που δέχεται και το τι μας επιστρέφει.

Στη συνέχεια, γράφονται οι υπόλοιπες εντολές της συνάρτησης, οι οποίες στην περίπτωσή μας είναι οι εντολές που υπολογίζουν την ημιπερίμετρο (s) και το εμβαδόν του τριγώνου (t).

Η τελευταία εντολή του μπλοκ, που αρχίζει με τη δεσμευμένη λέξη-κλειδί της Python `return`:

```
return t
```

είναι αυτή που επιστρέφει την τιμή του εμβαδού (t) στο σημείο του κώδικα από όπου κλήθηκε η συνάρτηση `triangle_area(3, 2, 4)`.

Η πρώτη εντολή που ευθυγραμμίζεται με το αριστερό περιθώριο, δηλαδή είναι αλφαδιασμένη με το `def`, είναι η πρώτη εντολή έξω από το μπλοκ των εντολών της συνάρτησης και δεν συμμετέχει στον ορισμό της.

Στην πλήρη του μορφή, το πρόγραμμα που υπολογίζει το εμβαδόν του τριγώνου είναι:

```
# Calculate the area of a triangle: Heron's method
import numpy as np

def triangle_area(a, b, c):
    """Calculate and return the area of a triangle
    from the length of its sides (a, b, c) using Heron's method"""
    s = (a + b + c)/2.
    t = np.sqrt(s*(s - a)*(s - b)*(s - c))
    return t

area = triangle_area(3, 2, 4)
print('Triangle Area [m2]: {:.3f}'.format(area))
```

και εάν το τρέξουμε στην οθόνη εμφανίζεται το αποτέλεσμα:

```
Triangle Area [m2]: 2.905
```

Αποθηκεύουμε το πρόγραμμα υπολογισμού του εμβαδού ενός τριγώνου από τα μήκη των πλευρών του στο αρχείο `Chapter_9_Triangle_Area.py`

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_9_Triangle_Area.py`

Στη συνέχεια, θα υπολογίσουμε το εμβαδόν του πολυγώνου ABCDEF του σχήματος της Εικόνας 9.1, που αποτελεί π.χ. το οικόπεδο στο οποίο θα πραγματοποιηθεί η γεωτεχνική έρευνα για ένα τεχνικό έργο.

Οι διαστάσεις των πλευρών των τριγώνων, στα οποία έχει χωριστεί το πολύγωνο, δίνονται στον Πίνακα 9.1.

Πίνακας 9.1. Διαστάσεις των τριγώνων του πολυγώνου ABCDEF.

Τρίγωνο	Πλευρά a	Πλευρά b	Πλευρά c
ABF	AF = 70 m	AB = 45 m	BF = 75 m
BCF	BF = 75 m	BC = 50 m	CF = 70 m
CDF	CF = 70 m	CD = 60 m	DF = 80 m
DEF	DF = 80 m	DE = 40 m	EF = 70 m

Ο κώδικας του προγράμματος που υπολογίζει το εμβαδόν του πολυγώνου είναι:

```
# Calculate the area of a polygon,
# adding the area of triangles calculated using Heron's method.
import numpy as np

def triangle_area(a, b, c):
    """Calculate and return the area of a triangle
    from the length of its sides (a, b, c) using Heron's method"""
    s = (a + b + c)/2.
    t = np.sqrt(s*(s - a)*(s - b)*(s - c))
    return t

triangles = [(70, 45, 75), (75, 50, 70), (70, 60, 80), (80, 40, 70)]
polygon_area = 0
for a, b, c in triangles:
    area = triangle_area(a, b, c)
    polygon_area = polygon_area + area
print('Polygon Area [m2]: {:.3f}'.format(polygon_area))
```

Ας εξηγήσουμε τη λογική του προγράμματος. Καταρχάς, γράφουμε τον ορισμό της συνάρτησης `triangle_area(a, b, c)`, την οποία θα χρησιμοποιήσουμε για να υπολογίσουμε το εμβαδόν καθενός τριγώνου.

Με τη γραμμή κώδικα:

```
triangles = [(70, 45, 75), (75, 50, 70), (70, 60, 80), (80, 40, 70)]
```

δημιουργούμε μια λίστα, η οποία έχει για στοιχεία πλειάδες και κάθε πλειάδα έχει ως στοιχεία τα μήκη των πλευρών των τριγώνων του Πίνακα 9.1 (τέσσερα τρίγωνα, τέσσερις πλειάδες). Τη λίστα την εκχωρούμε στη μεταβλητή `triangles`.

Ορίζουμε τη μεταβλητή `polygon_area`, που αντιπροσωπεύει το εμβαδόν του πολυγώνου και την αρχικοποιούμε δίνοντάς της την τιμή 0.

Στη συνέχεια, δημιουργούμε έναν βρόχο `for` ο οποίος:

- Για κάθε στοιχείο (πλειάδα) της μεταβλητής `triangles` αντιστοιχεί τις τιμές-στοιχεία της πλειάδας στις παραμέτρους της συνάρτησης `triangle_area(a, b, c)`. Π.χ. για την πρώτη πλειάδα, το 70 στο `a`, το 45 στο `b` και το 75 στο `c`.
- Στη συνέχεια, καλεί τη συνάρτηση `triangle_area(a, b, c)`, στην οποία τώρα έχουν εκχωρηθεί οι πραγματικές παράμετροι, δηλαδή η συνάρτηση για την πρώτη πλειάδα έχει τη μορφή `triangle_area(70, 45, 75)`, η οποία αφού υπολογίσει το εμβαδόν αυτού του τριγώνου, επιστρέφει την τιμή του και την εκχωρεί στη μεταβλητή `area`.
- Η εντολή `polygon_area = polygon_area + area` προσθέτει διαδοχικά τα εμβαδά των τριγώνων, που υπολογίζονται από τη συνάρτηση `triangle_area(a, b, c)`, και εκχωρεί την τιμή του εμβαδού στη μεταβλητή `polygon_area`.

Παρατηρήστε ότι το πρόγραμμά μας καλεί τέσσερις φορές το ίδιο μπλοκ εντολών, δηλαδή τη συνάρτηση `triangle_area(a, b, c)`, με διαφορετικές τιμές δεδομένων, ενώ εμείς το γράψαμε μια μόνο φορά, υπό παραμετρική μορφή.

Τέλος, η γραμμή κώδικα:

```
print('Polygon Area [m2]: {:.3f}'.format(polygon_area))
```

εμφανίζει το αποτέλεσμα των υπολογισμών μας στην οθόνη:

Polygon Area [m2]: 6667.001

Αποθηκεύουμε ο πρόγραμμα υπολογισμού του εμβαδού ενός πολυγώνου από τα μήκη των πλευρών των τριγώνων, στα οποία το χωρίσαμε, στο αρχείο `Chapter_9_Polygon_Area.py`

Το script του κώδικα, βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_9_Polygon_Area.py`

Σημείωση 1: Η μεταβλητή `triangles`, αντί για λίστα θα μπορούσε να είναι πλειάδα, δηλαδή πλειάδα με μέλη πλειάδες.

Σημείωση 2: Η θέση των ορισμών των συναρτήσεων μπορεί να βρίσκεται οπουδήποτε μέσα στον κώδικα ενός προγράμματος. Πρέπει όμως ο ορισμός μιας συνάρτησης να προηγείται των εντολών που καλούν αυτή τη συνάρτηση.

Εάν, για παράδειγμα, γράψουμε τον ορισμό της συνάρτησης `triangle_area(a, b, c)` στο τέλος του κώδικα και τρέξουμε το πρόγραμμα, θα εμφανιστεί στην οθόνη το μήνυμα λάθους:

`NameError: name 'triangle_area' is not defined`

Μια καλή πρακτική είναι να τοποθετούνται όλοι οι ορισμοί των συναρτήσεων μαζί, στην αρχή του κώδικα, για να είναι εύκολα προσβάσιμοι.

Σημείωση 3: Το docstring μιας συνάρτησης τυγχάνει ειδικού χειρισμού από την Python. Εάν γράψουμε στο πρόγραμμά μας (προσέξτε τα δύο `_` πριν και μετά το `doc`):

```
print(triangle_area.__doc__)
```

στην οθόνη θα εμφανιστεί το κείμενο:

**Calculate and return the area of a triangle
from the lengths of its sides (a, b, c) using Heron's method**

Το γράψιμο του docstring μιας συνάρτησης δεν είναι απαραίτητο, αλλά είναι δείγμα καλής προγραμματιστικής πρακτικής!

Σημείωση 4: Πολλές φορές θα ακούσετε την έκφραση: *το κυρίως πρόγραμμα* (the main program). Στην Python δεν σημαίνει κάτι το ιδιαίτερο, απλά είναι το μέρος του κώδικα ενός προγράμματος που δεν περιλαμβάνει τον κώδικα ορισμού των συναρτήσεων.

09.3 To return

Στη συνάρτηση `triangle_area(a, b, c)` το `return` μας επιστρέφει την τιμή της μεταβλητής `t` η οποία είναι το εμβαδόν του τριγώνου. Θα μπορούσαμε να είχαμε γράψει απευθείας:

```
return np.sqrt(s*(s - a)*(s - b)*(s - c))
```

αλλά ο τρόπος με τον οποίο γράψαμε το `return` στον κώδικά μας είναι πιο κομψός.

Είναι επίσης δυνατόν σε ένα πρόγραμμα να έχουμε μια συνάρτηση που δεν επιστρέφει κάτι. Στην περίπτωση αυτή δεν γράφουμε το `return`.

Για παράδειγμα, η συνάρτηση:

```
def error_message():  
    print("There is an OPEN triangle !!!")
```

δεν επιστρέφει κάτι, αλλά δεν έχει και παραμέτρους! Το μόνο που κάνει, όταν κληθεί, είναι να εμφανίζει στην οθόνη το μήνυμα:

There is an OPEN triangle !!!

Τότε ποια είναι η χρησιμότητά της για να την έχουμε στον κώδικά μας;

Στον τύπο του Ήρωνα, Εξίσωση [9.1], εάν η τιμή της ημιπεριμέτρου του τριγώνου είναι ίση με οποιαδήποτε πλευρά του, τότε το εμβαδόν του τριγώνου είναι μηδέν.

Επίσης, εάν το μήκος μιας πλευράς του τριγώνου είναι μεγαλύτερο από την ημιπερίμετρο, τότε το γινόμενο $(s*(s - a)*(s - b)*(s - c))$ είναι ένας αρνητικός αριθμός και όταν προσπαθήσουμε να υπολογίσουμε την τετραγωνική ρίζα του, με τη συνάρτηση `np.sqrt()` της Numpy, το αποτέλεσμα που θα εμφανιστεί στην οθόνη θα είναι:

Polygon Area [m2]: nan

δηλαδή, το εμβαδόν του πολυγώνου δεν είναι αριθμός (not a number, nan).

Αυτό συμβαίνει γιατί η συνάρτηση `np.sqrt()` της Numpy υπολογίζει τις τετραγωνικές ρίζες μόνο των θετικών αριθμών, όπως αναφέρεται και στην τεκμηρίωση της συνάρτησης: <https://numpy.org/doc/stable/reference/generated/numpy.sqrt.html>

Προκειμένου να αποφύγουμε τέτοιου είδους προβλήματα στο πρόγραμμά μας, π.χ. από λανθασμένη πληκτρολόγηση των μηκών των πλευρών των τριγώνων, δηλαδή να έχουμε τρίγωνα που δεν είναι τρίγωνα, είτε γιατί έχουν εξομοιωθεί με ευθύγραμμα τμήματα είτε γιατί είναι ανοιχτά τρίγωνα, θα τροποποιήσουμε το πρόγραμμα `Chapter_9_Triangle_Area.py` ως ακολούθως:

```
# Calculate the area of a triangle: Heron's method
# Check for data errors
import numpy as np
import sys

def error_message():
    print("This is not a triangle !!!")
    sys.exit()

def triangle_area(a, b, c):
    """Calculate and return the area of a triangle
    from the length of its sides (a, b, c) using Heron's method"""
    s = (a + b + c)/2.
    if s <= a or s <= b or s <= c:
        error_message()
    t = np.sqrt(s*(s - a)*(s - b)*(s - c))
    return t

area = triangle_area(8, 2, 4)
print('Triangle Area [m2]: {:.3f}'.format(area))
```

Καταρχάς, κάνουμε `import` από τις εσωτερικές βιβλιοθήκες της Python τη `sys`, η οποία περιέχει συναρτήσεις και παραμέτρους που σχετίζονται με τον interpreter της Python. Θα χρησιμοποιήσουμε μια συνάρτηση της `sys` στη συνέχεια. Παρατηρήστε ότι δεν δώσαμε nickname στη βιβλιοθήκη `sys`, γιατί το όνομά της είναι σύντομο.

Η συνάρτηση `error_message()`, όταν κληθεί, αφού εμφανίσει στην οθόνη το μήνυμα:

This is not a triangle !!!

δηλαδή, ότι οι τιμές που δώσαμε ως μήκη των πλευρών του τριγώνου δεν σχηματίζουν ένα τρίγωνο, καλεί τη συνάρτηση `exit()` της βιβλιοθήκης `sys`, η οποία δίνει εντολή στον interpreter της Python να τερματίσει την εκτέλεση του προγράμματος σε αυτό το σημείο του κώδικα.

Μέσα στη συνάρτηση `triangle_area(a, b, c)` και μετά την εντολή που υπολογίζει την ημιπερίμετρο του τριγώνου, παρεμβάλλονται οι γραμμές κώδικα:

```
if s <= a or s <= b or s <= c:
    error_message()
else:
    pass
```

με τις οποίες πρώτα ελέγχεται εάν κάποια από τις πλευρές του τριγώνου έχει μήκος μεγαλύτερο ή ίσο με το μήκος της ημιπεριμέτρου. Εάν το αποτέλεσμα του ελέγχου είναι ψευδές (`False`), τότε η συνάρτηση προχωρά στον υπολογισμό του εμβαδού. Εάν το αποτέλεσμα είναι αληθές (`True`), τότε καλείται η συνάρτηση `error_message()`, η οποία, όπως είδαμε παραπάνω, εμφανίζει το μήνυμα λάθους στην οθόνη και τερματίζει την εκτέλεση του προγράμματος.

Αποθηκεύουμε το πρόγραμμα υπολογισμού του εμβαδού ενός τριγώνου από τα μήκη των πλευρών του, με έλεγχο των δεδομένων για τυχόν λάθη, στο αρχείο:

`Chapter_9_Triangle_Area_with_Error_Check.py`

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο:

`Chapter_9_Triangle_Area_with_Error_Check.py`

09.4 Παράμετροι και μεταβλητές συνάρτησης

Στην Python, αλλά και σε άλλες γλώσσες προγραμματισμού, ονομάζουμε *εμβέλεια* (scope) μιας παραμέτρου ή μιας μεταβλητής το τμήμα του κώδικα ενός προγράμματος στο οποίο η παράμετρος ή η μεταβλητή είναι *ορατή*, δηλαδή αναγνωρίσιμη.

Στις συναρτήσεις που ορίζουμε εμείς, όπως είναι η `triangle_area(a, b, c)`, οι παράμετροι `a`, `b`, `c` αλλά και οι μεταβλητές `s` και `t` που περιέχονται μέσα στο μπλοκ της συνάρτησης έχουν εμβέλεια μόνο *μέσα* στη συνάρτηση και ονομάζονται *τοπικές μεταβλητές* (local variables). Αυτό πρακτικά σημαίνει ότι δεν μπορούμε να τις χρησιμοποιήσουμε εκτός της συνάρτησης.

Για παράδειγμα, στον παρακάτω κώδικα:

```
import numpy as np

def triangle_area(a, b, c):
    """Calculate and return the area of a triangle
    from the length of its sides (a, b, c) using Heron's method"""
    s = (a + b + c)/2.
    print("Triangle's Half Perimeter (in) [m] = ", s)
    t = np.sqrt(s*(s - a)*(s - b)*(s - c))
    return t

triangle_area(3, 2, 4)
print("Triangle's Half Perimeter (out) [m] = ", s)
```

μπορούμε να δούμε ποιο είναι το μήκος της ημιπεριμέτρου του τριγώνου από *μέσα* από τη συνάρτηση, αλλά όχι από *έξω*.

Εάν τρέξουμε τον παραπάνω κώδικα, η πρώτη εντολή:

```
print("Triangle's Half Perimeter (in) [m] = ", s)
```

που είναι μέσα στη συνάρτηση, εμφανίζει στην οθόνη το αποτέλεσμα:

```
Triangle's Half Perimeter (in) [m] = 4.5
```

ενώ η δεύτερη εντολή:

```
print("Triangle's Half Perimeter (out) [m] = ", s)
```

που είναι εκτός της συνάρτησης, αντί να μας εμφανίσει την τιμή της ημιπεριμέτρου, μας βγάζει το παρακάτω μήνυμα λάθους:

```
NameError: name 's' is not defined
```

δηλαδή, ότι η μεταβλητή *s* είναι άγνωστη εκτός της συνάρτησης `triangle_area(a, b, c)`.

Οι τοπικές μεταβλητές έχουν *διάρκεια ζωής* (lifetime) όσο εκτελείται η συνάρτηση και καταστρέφονται όταν η εκτέλεση του προγράμματος επιστρέψει στο σημείο του κώδικα απ' όπου έγινε η κλήση της.

Το γεγονός ότι μια συνάρτηση *δεν θυμάται* τις τιμές των μεταβλητών που είχε σε προηγούμενη κλήση της, μας επιτρέπει να τις καλούμε πολλές φορές, όπως κάναμε στην περίπτωση του προγράμματος που υπολόγισε το εμβαδόν του πολυγώνου, γιατί κάθε νέα κλήση μιας συνάρτησης δημιουργεί νέες τοπικές μεταβλητές.

Οι μεταβλητές του κώδικα ενός προγράμματος που δηλώνονται εκτός των μπλοκ των συναρτήσεων ονομάζονται *καθολικές μεταβλητές* (global variables). Οι καθολικές μεταβλητές έχουν εμβέλεια σε κάθε σημείο του προγράμματος, ακόμα και μέσα στις συναρτήσεις!

Δώστε προσοχή στο παρακάτω παράδειγμα κώδικα:

```
z = 10
```

```
def my_function(x, y):  
    w = x + y + z  
    return w
```

```
print('#1 x+y+z = ', my_function(10, 10))  
print('#2 x+y+z = ', my_function(10, 10))
```

όπου έχω μια καθολική μεταβλητή, τη *z*, της οποίας η τιμή προστίθεται στην τιμή των τοπικών μεταβλητών *x* και *y*, όταν κληθεί η συνάρτηση `my_function(10, 10)` για να υπολογίσει το άθροισμα *x + y + z*.

Εάν τρέξουμε το παράδειγμα, το αποτέλεσμα που βλέπουμε στην οθόνη είναι:

```
#1 x+y+z = 30  
#2 x+y+z = 30
```

Εάν τροποποιήσουμε τον κώδικα μόνο στο μπλοκ της συνάρτησης `my_function(x, y)` και γράψουμε:

```
def my_function(x, y):  
    z = z + 10  
    w = x + y + z  
    return w
```

ξανατρέχοντας το παράδειγμα θα δούμε στην οθόνη το παρακάτω μήνυμα λάθους:

```
z = z + 10  
UnboundLocalError: local variable 'z' referenced before assignment
```

Δηλαδή, η Python αναγνωρίζει τη μεταβλητή *z* που είναι μέσα στο μπλοκ της συνάρτησης `my_function(x, y)` ως μια τοπική μεταβλητή, η οποία έχει το ίδιο όνομα με την καθολική μεταβλητή, τη *z*, αλλά κανένα άλλο κοινό χαρακτηριστικό.

Η z , ως τοπική μεταβλητή, για να χρησιμοποιηθεί στην εντολή $z = z + 10$ θα έπρεπε πρώτα να της έχει εκχωρηθεί μια τιμή. Δεν το έχουμε κάνει όμως αυτό στον κώδικά μας και ο interpreter της Python διέγινωσε το λάθος που εμφανίστηκε στην οθόνη.

Σημειώστε ότι η τιμή που εκχωρήσαμε με την εντολή $z = 10$ στην αρχή του κώδικα του παραδείγματος αφορούσε την καθολική μεταβλητή, τη z , και όχι την τοπική μεταβλητή z .

Εάν τροποποιήσουμε ξανά τον κώδικα στο μπλοκ της συνάρτησης `my_function(x, y)` και τη γράψουμε ως:

```
def my_function(x, y):  
    global z  
    z = z + 10  
    w = x + y + z  
    return w
```

και ξανατρέξουμε το παράδειγμα, θα δούμε στην οθόνη το παρακάτω αποτέλεσμα:

```
#1 x+y+z = 40  
#2 x+y+z = 50
```

γιατί τώρα με την εντολή:

```
global z
```

που γράψαμε μέσα στο μπλοκ της συνάρτησης, δηλώσαμε τη μεταβλητή z ως *καθολική* (global) και επομένως η συνάρτηση την χειρίζεται ως τέτοια και μάλιστα μπορεί να μεταβάλει την τιμή της!

Συμπέρασμα: Μέσα στις συναρτήσεις οι τοπικές μεταβλητές είναι οι *κυρίαρχοι του παιχνιδιού*. Οι καθολικές μεταβλητές μπορούν να χρησιμοποιηθούν μέσα στις συναρτήσεις, αλλά δεν μπορούν να μεταβληθούν, εκτός εάν προηγουμένως έχουν δηλωθεί εντός της συνάρτησης ως `global`.

Στα μέχρι τώρα παραδείγματα συναρτήσεων που ορίσαμε, όλες οι παράμετροι των συναρτήσεων περίμεναν να τις εκχωρήσουμε κάποια τιμή, μέσω της κλήσης της συνάρτησης. Οι συναρτήσεις στην Python όμως μπορούν να έχουν και *προεπιλεγμένες τιμές* (default values).

Ας υποθέσουμε ότι θέλουμε να κάνουμε το πρόγραμμα που υπολογίζει την prn μιας ανατίναξης λίγο *φαντεζί* (fancy) και να καλωσορίζει προσωποποιημένα τον χρήστη.

Γράφουμε τις παρακάτω γραμμές κώδικα:

```
def greet(name, msg="Kalimera!"):  
    """A personalised welcome to the user of the program"""  
    print("Hello", name + ', ' + msg)
```

```
user_name = input('Enter your name, please: ')  
greet(user_name)
```

Η συνάρτηση `greet(name, msg="Kalimera!")` που ορίσαμε έχει δύο παραμέτρους, τη `name` που δεν έχει προεπιλεγμένη τιμή και την `msg` που έχει ως προεπιλεγμένη τιμή τη συμβολοσειρά `Kalimera!` Οι προεπιλεγμένες τιμές εκχωρούνται σε μια παράμετρο-όνομα, όπως π.χ. το `msg`, που είναι η συντομογραφία του message, με το σύμβολο `=`. Εκατέρωθεν του `=` δεν υπάρχει κενό διάστημα.

Όταν τρέξουμε τον παραπάνω κώδικα και δώσουμε ως `input` το όνομα George:

```
Enter your name, please: George
```

στην οθόνη θα εμφανιστεί το προσωποποιημένο μήνυμα, γιατί αυτό μόνο κάνει η συνάρτηση `greet(name, msg="Kalimera!")`:

Hello George, Kalimera!

Αυτό συμβαίνει γιατί καλέσαμε τη συνάρτηση ως `greet(user_name)` και η τιμή της μεταβλητής `user_name`, που είχε από την πληκτρολόγηση ως τιμή τη συμβολοσειρά `George`, πέρασε στην παράμετρο `name` της συνάρτησης. Στη θέση της παραμέτρου `msg` εκχωρήθηκε η προεπιλεγμένη τιμή `"Kalimera!"`.

Εάν καλέσουμε τη συνάρτηση `greet(name, msg="Kalimera!")` ως:

```
greet(user_name, "Kalispera!")
```

στην οθόνη θα εμφανιστεί το μήνυμα καλωσορίσματος:

Hello George, Kalispera!

Δηλαδή, όταν κατά την κλήση μιας συνάρτησης δηλώνεται η τιμή μιας παραμέτρου που έχει προεπιλεγμένη τιμή, τότε η προεπιλεγμένη τιμή της αντικαθίσταται, και μόνο για τη συγκεκριμένη κλήση της συνάρτησης, με τη δηλωμένη τιμή.

Συνοψίζοντας:

- Κατά την κλήση μιας συνάρτησης, εάν μια παράμετρός της δεν έχει προεπιλεγμένη τιμή, τότε πρέπει να δηλώνεται υποχρεωτικά η τιμή της.
- Εάν μια παράμετρος συνάρτησης έχει προεπιλεγμένη τιμή και κατά την κλήση της συνάρτησης δηλωθεί τιμή για την παράμετρο αυτή, η δηλωμένη τιμή αντικαθιστά, για τη συγκεκριμένη κλήση, την προεπιλεγμένη τιμή της παραμέτρου.

Οι συναρτήσεις μπορεί να έχουν οποιονδήποτε αριθμό παραμέτρων με προεπιλεγμένες ή μη τιμές. Εκείνο που δεν επιτρέπεται είναι, στην επικεφαλίδα της συνάρτησης, οι παράμετροι με προεπιλεγμένες τιμές να προηγούνται των παραμέτρων με μη-προεπιλεγμένες τιμές.

Εάν την επικεφαλίδα της συνάρτησης, αντί του ορθού:

```
def greet(name, msg="Kalimera!"):
```

τη γράψουμε αλλάζοντας τη σειρά των παραμέτρων:

```
def greet(msg="Kalimera!", name):
```

θα πάρουμε το παρακάτω μήνυμα λάθους:

```
def greet(msg="Kalimera!", name):
```

^

SyntaxError: non-default argument follows default argument

09.5 Συναρτήσεις `lambda`

Οι *συναρτήσεις `lambda`* (*lambda functions*), σε αντίθεση με όσες έχουμε δημιουργήσει μέχρι τώρα, ορίζονται σε *μια γραμμή του κώδικα* (*one-line functions*), μέσα στο κυρίως πρόγραμμα, χωρίς να αφήνουμε πριν και μετά κενές γραμμές.

Για τον ορισμό μιας συνάρτησης `lambda` αντί της λέξης-κλειδί `def` χρησιμοποιούμε τη δεσμευμένη λέξη-κλειδί της Python `lambda`, στην οποία οφείλουν και το όνομά τους. Ονομάζονται και *ανώνυμες συναρτήσεις*.

Μια συνάρτηση `lambda` ορίζεται όπως στο παρακάτω παράδειγμα:

```
equation = lambda a, b, c, x: a*(x**2) + b*x + c
```

Μετά τη λέξη-κλειδί `lambda` γράφονται οι μεταβλητές της συνάρτησης χωρισμένες με κόμματα και τελειώνουν με μια άνω-κάτω τελεία. Αμέσως μετά γράφεται η έκφραση που συνδέει τις μεταβλητές.

Εάν μετά την εντολή-ορισμό της συνάρτησης `lambda` γράψουμε:

```
print('Function lambda = ', equation(3, 8, 16, 4))
```

θα δούμε στην οθόνη το αποτέλεσμα:

```
Function lambda = 96
```

γιατί με την κλήση της συνάρτησης ως `equation(3, 8, 16, 4)` οι μεταβλητές της συνάρτησης `a`, `b`, `c`, `x` έλαβαν τις τιμές 3, 8, 16, 4 αντίστοιχα.

Πάντως, σύμφωνα με το *PEP 8 -- Style Guide for Python Code* της Python, ενθαρρύνεται ο ορισμός συναρτήσεων με τη μέθοδο `def` αντί των συναρτήσεων `lambda`. Έτσι, την ίδια συνάρτηση καλό θα ήταν να τη γράφαμε ως:

```
def equation(a, b, c, x):  
    w = a*(x**2) + b*x + c  
    return w
```

```
print('Function def = ', equation(3, 8, 16, 4))
```

και θα είχαμε το ίδιο βέβαια αποτέλεσμα στην οθόνη:

```
Function def = 96
```

09.6 Η συνάρτηση `main()`

Πολλές γλώσσες προγραμματισμού, όπως οι C, C++, Java, κ.ά. ορίζουν μια ειδική συνάρτηση που ονομάζεται `main()` και την οποία το λειτουργικό σύστημα καλεί αυτόματα όταν εκτελείται ένα πρόγραμμα. Η συνάρτηση αυτή συχνά ονομάζεται και το *σημείο εισόδου* (entry point), γιατί από εκεί ξεκινά η εκτέλεση του προγράμματος.

Αντιθέτως, η Python δεν έχει μια τέτοια ειδική συνάρτηση και η εκτέλεση του προγράμματος ακολουθεί τη σειρά των εντολών του κώδικα του κυρίως προγράμματος.

Στο πλαίσιο όμως του δομημένου προγραμματισμού και των καλών πρακτικών προγραμματισμού, καλό είναι να υπάρχει μια τέτοια συνάρτηση, από την οποία θα ξεκινάει η εκτέλεση του προγράμματος, το entry point. Αυτή η συνάρτηση στην Python μπορεί να έχει οποιοδήποτε όνομα, αλλά επίσης καλό είναι να την ονομάζουμε `main()`.

Η συνάρτηση `main()` θα περιέχει τον κώδικα που θέλουμε να τρέξει όταν ο interpreter της Python εκτελέσει το script του προγράμματός μας.

Η συνάρτηση `main()` σχεδόν πάντα χρησιμοποιείται σε συνδυασμό με την παρακάτω εντολή ελέγχου:

```
def main():  
    print('This is main()')  
  
if __name__ == "__main__":  
    # execute only if run as a script  
    main()
```

Η Python με την εντολή ελέγχου:

```
if __name__ == "__main__":  
    # execute only if run as a script  
    main()
```


που είναι ένας *ιδιωματισμός* (idiom) της Python, τσεκάρει εάν το script που θέλουμε να τρέξουμε είναι ένα πρόγραμμα και όχι ένα module μιας βιβλιοθήκης. Εάν το αποτέλεσμα του ελέγχου είναι αληθές (True), τότε μόνο καλεί τη συνάρτηση `main()`.

Εάν τρέξουμε τον παραπάνω κώδικα, το αποτέλεσμα που θα εμφανιστεί στην οθόνη είναι:

```
This is main()
```

Για περισσότερες πληροφορίες σχετικά με τη χρήση της `main()` επισκεφτείτε την ιστοσελίδα <https://docs.python.org/3/library/main.html>

Στο Κεφάλαιο 10 Αναδιάρθρωση του κώδικα *BLASTppn.py* θα δούμε την εφαρμογή της συνάρτησης `main()` στο πρόγραμμα υπολογισμού της *ppn*.

10 Αναδιάρθρωση του κώδικα BLASTppv.py

10.1 Γενικά

Στο Κεφάλαιο αυτό θα επιχειρήσουμε μια αναδιάρθρωση (restructuring) του κώδικα του προγράμματος BLASTppv.py που υπολογίζει την ppv μιας ανατίναξης, εφαρμόζοντας κυρίως όσα παρουσιάστηκαν στο Κεφάλαιο 09 Συναρτήσεις.

Το διάβασμα αυτού του Κεφαλαίου πρέπει να γίνει με προσοχή, γιατί θα προστεθούν επίσης αρκετά νέα στοιχεία της γλώσσας Python αλλά και τεχνικές τις οποίες χρησιμοποιούμε όταν αναπτύσσουμε *πραγματικά* προγράμματα.

Το πρόγραμμά μας, στην καινούρια του μορφή, θέλουμε να το χρησιμοποιεί ένας ελεγχόμενος αριθμός χρηστών, στους οποίους έχουμε δώσει έναν *κωδικό πρόσβασης* (password), ώστε να το τρέχουν μόνον αυτοί. Κάθε χρήστης θα έχει στη διάθεσή του τρεις (3) προσπάθειες πληκτρολόγησης του σωστού κωδικού πρόσβασης.

Επίσης, θέλουμε να δημιουργήσουμε μια απλή βάση δεδομένων των χρηστών του προγράμματος στο Microsoft Excel. Δηλαδή, σε ένα φύλλο του Excel να καταγράφονται το όνομα του χρήστη, η ημερομηνία και ώρα εισόδου στο πρόγραμμα και εάν έκανε εισαγωγή του σωστού ή όχι κωδικού πρόσβασης.

Όλα αυτά, αλλά και άλλα χρήσιμα στοιχεία προγραμματισμού με την Python, θα παρουσιαστούν στις παραγράφους που ακολουθούν.

Στο τέλος αυτού του Κεφαλαίου, το πρόγραμμα BLASTppv.py, που αρχίσαμε να το γράφουμε στην παράγραφο 04.8 Ο κώδικας στο τέλος του Κεφαλαίου 04 με τις στοιχειώδεις γνώσεις της Python που διαθέταμε τότε και αποθηκεύσαμε στο αρχείο Chapter_4_BLASTppv.py, θα αποκτήσει μια ολοκληρωμένη και πλήρως λειτουργική μορφή.

10.2 Δημιουργία κωδικού πρόσβασης

Έχουμε αποφασίσει ότι ο κωδικός πρόσβασης του χρήστη στο πρόγραμμά μας θα είναι η συμβολοσειρά MMM, δηλαδή τα αρχικά του ονόματος της Σχολής Μηχανικών Μεταλλείων – Μεταλλουργών, με λατινικούς χαρακτήρες.

Με τη βοήθεια του Notepad (Σημειωματάριο) των Windows δημιουργούμε το αρχείο BLASTppv_password_file.txt στο οποίο γράφουμε στην πρώτη γραμμή τη συμβολοσειρά MMM με λατινικούς χαρακτήρες και σώζουμε το αρχείο στο ίδιο directory όπου βρίσκεται και το πρόγραμμά μας.

Γράφουμε τις παρακάτω γραμμές κώδικα:

```
pass_file = open("BLASTppv_password_file.txt", 'r')
password = pass_file.readline()
pass_file.close()
```

Στην πρώτη γραμμή, με τη βοήθεια της συνάρτησης open() *ανοίγουμε* το αρχείο BLASTppv_password_file.txt και με το όρισμα r δηλώνουμε ότι θέλουμε να το *διαβάσουμε* (read). Το ανοιγμένο αρχείο το ονοματίζουμε pass_file και είναι ένα *αντικείμενο* του αρχείου BLASTppv_password_file.txt

Στην επόμενη γραμμή, εφαρμόζοντας τη μέθοδο readline() επί του αντικειμένου του αρχείου, διαβάζουμε την πρώτη γραμμή του περιεχομένου του αρχείου, δηλαδή τη συμβολοσειρά MMM, και την εκχωρούμε στη μεταβλητή password.

Στην τελευταία γραμμή, εφαρμόζοντας τη μέθοδο `close()` επί του αντικειμένου του αρχείου κλείνουμε το ανοιγμένο αρχείο `BLASTppv_password_file.txt`

Εάν το αρχείο `BLASTppv_password_file.txt` δεν είναι ένα τοπικό αρχείο στον υπολογιστή μας, αλλά βρίσκεται αποθηκευμένο σε έναν απομακρυσμένο υπολογιστή, στον οποίο όμως ο δικός μας υπολογιστής έχει πρόσβαση μέσω του Internet, τότε για να υλοποιήσουμε τα παραπάνω, θα γράψουμε τις εξής γραμμές κώδικα:

```
import urllib.request

pass_file = urllib.request.urlopen \
    ('https://www.metal.ntua.gr/BLASTppv_password_file.txt')
password = pass_file.read().decode('utf-8')
pass_file.close()
```

Το αρχείο το οποίο περιέχει το password είναι αποθηκευμένο στον server της Σχολής Μηχανικών Μεταλλείων-Μεταλλουργών ΕΜΠ και η πλήρης διεύθυνση (URL - Uniform Resource Locator) του αρχείου είναι: https://www.metal.ntua.gr/BLASTppv_password_file.txt

Για να αποκτήσει η Python πρόσβαση σε έναν πόρο του Internet μέσω της διεύθυνσης URL, χρησιμοποιεί το ενσωματωμένο module `urllib.request` το οποίο πρέπει να το κάνουμε `import` στο πρόγραμμά μας.

Για περισσότερες πληροφορίες ανατρέξτε στο website:

<https://docs.python.org/3/library/urllib.request.html#module-urllib.request>

Στη συνέχεια, με τη βοήθεια της συνάρτησης `urllib.request.urlopen()` αποκτούμε πρόσβαση και ανοίγουμε το απομακρυσμένο αρχείο:

https://www.metal.ntua.gr/BLASTppv_password_file.txt

Στην επόμενη γραμμή, εφαρμόζοντας τη μέθοδο `read().decode('utf-8')` επί του αντικειμένου του αρχείου διαβάζουμε την πρώτη γραμμή του περιεχομένου του αρχείου, δηλαδή τη συμβολοσειρά `MMM` και την εκχωρούμε στη μεταβλητή `password`.

Η μέθοδος `decode('utf-8')` που χρησιμοποιήσαμε μαζί με τη μέθοδο `read()` είναι μια μέθοδος που χρησιμοποιείται με τις συμβολοσειρές της Python (string method). Αποκωδικοποιεί (decode) χαρακτήρες κειμένου (textual data) σύμφωνα με το format `UTF-8`, που είναι ένα από τα περισσότερο χρησιμοποιούμενα format και default για την Python. `UTF` είναι τα αρχικά των λέξεων *Unicode Transformation Format* και το `8` σημαίνει ότι χρησιμοποιούνται τιμές 8-bit για την κωδικοποίηση των χαρακτήρων κειμένου.

Για περισσότερες πληροφορίες σχετικά με την Python και το format `Unicode` ανατρέξτε στο website: <https://docs.python.org/3/howto/unicode.html>

Στην παράγραφο 12.2 *Καταγραφή δεδομένων σε αρχεία* θα μάθουμε περισσότερα για τους τρόπους με τους οποίους η Python χειρίζεται τα αρχεία.

Τελειώνοντας, θα πρέπει να επισημάνουμε ότι το να δημιουργήσουμε το `password` με τη βοήθεια του Notepad δεν είναι ιδιαίτερα έξυπνο, γιατί ο οποιοσδήποτε μπορεί να το χακάρει πανεύκολα.

Με την εντολή:

```
print(password)
```

στην οθόνη θα εμφανιστεί το `password` του προγράμματος:

```
MMM
```

Τόσο απλά...

10.3 Κρυπτογράφηση κωδικού πρόσβασης

Θα γράψουμε ένα βοηθητικό πρόγραμμα, ανεξάρτητο από το `BLASTppv.py`, με το οποίο θα *κρυπτογραφήσουμε* το password `MMM` και, με την κρυπτογραφημένη του μορφή, θα το αποθηκεύσουμε στο αρχείο `BLASTppv_password_file.txt`

Ο κώδικας του προγράμματος αυτού είναι:

```
from passlib.hash import pbkdf2_sha256

hash_string = pbkdf2_sha256.hash("MMM")
pass_file = open("BLASTppv_password_file.txt", 'w')
pass_file.write(hash_string)
pass_file.close()
```

Για την υλοποίηση της κρυπτογράφησης του password θα χρησιμοποιήσουμε την εξωτερική βιβλιοθήκη της Python `passlib` την οποία πρέπει πρώτα να την εγκαταστήσουμε στον υπολογιστή μας, κατά τα γνωστά:

```
pip install passlib
```

Για την τεκμηρίωση της βιβλιοθήκης ανατρέξτε στο website:

<https://pypi.org/project/passlib/>

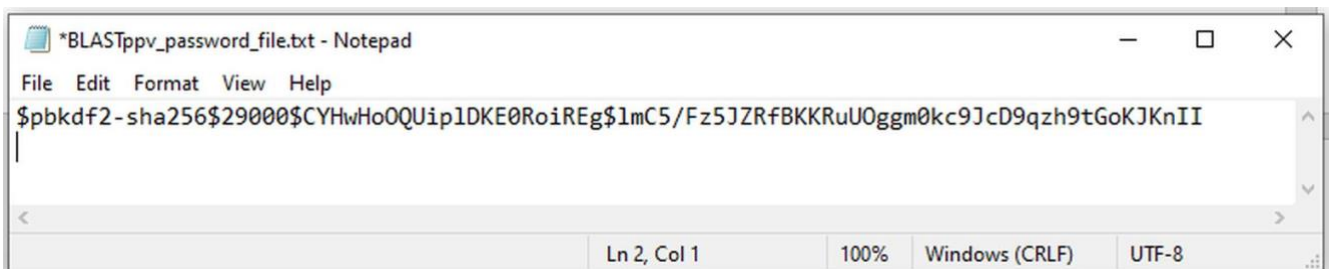
Στην πρώτη γραμμή του κώδικα από το module της βιβλιοθήκης `passlib.hash` κάνουμε `import` τον αλγόριθμο κρυπτογράφησης `pbkdf2_sha256`.

Στη συνέχεια, με τη βοήθεια της *κρυπτογραφικής συνάρτησης κατατεμαχισμού* (hash function) `pbkdf2_sha256.hash()` κρυπτογραφούμε το password `MMM` και την τιμή της κρυπτογραφημένης πλέον μορφής του, την εκχωρούμε στη μεταβλητή `hash_string`.

Ο υπόλοιπος κώδικας του προγράμματος ανοίγει το αρχείο `BLASTppv_password_file.txt`, γράφει σε αυτό την τιμή της μεταβλητής `hash_string` και κλείνει το αρχείο.

Αποθηκεύουμε το script σε αρχείο με όνομα `Chapter_10_Crypto_password.py` και το τρέχουμε.

Εάν στη συνέχεια ανοίξουμε το αρχείο `BLASTppv_password_file.txt` με το Notepad, αυτό που θα δούμε να περιέχει είναι το *κρυπτογράφημα* του password `MMM`, όπως παρουσιάζεται στην Εικόνα 10.1.



Εικόνα 10.1. Το αρχείο `BLASTppv_password_file.txt` που περιέχει το κρυπτογράφημα του password `MMM`, ανοιχμένο στο Notepad.

Εάν τώρα κάποιος προσπαθήσει να χακάρει το password του προγράμματος `BLASTppv.py` με την εντολή `print(password)`, όπως κάναμε στην παράγραφο 10.2 *Δημιουργία κωδικού πρόσβασης*, στην οθόνη θα εμφανιστεί το κρυπτογράφημά του:

```
$pbkdf2-sha256$29000$CYHwHoOQUIp1DKE0RoiREg$1mC5/Fz5JZRfBKkRuU0ggm0kc9JcD9qzh9tGoKJKnII
```

και όχι η πραγματική συμβολοσειρά του password, `MMM`, την οποία πρέπει να γνωρίζει ένας χρήστης για να έχει πρόσβαση στο πρόγραμμά μας.

Σημείωση: Κάθε φορά που γίνεται κλήση της συνάρτησης κατατεμαχισμού `pbkdf2_sha256.hash()` για την κρυπτογράφηση ενός password, π.χ του MMM, δημιουργείται ένα νέο κρυπτογράφημα. Επομένως, εάν τρέξετε το πρόγραμμα `Chapter_10_Crypto_password.py`, θα πάρετε ένα κρυπτογράφημα τελείως διαφορετικό από αυτό της Εικόνας 10.1, το οποίο όμως θα αντιστοιχεί πάντα στη συμβολοσειρά MMM του password.

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_10_Crypto_password.py`

10.4 Είσοδος του χρήστη στο πρόγραμμα

Θα γράψουμε μια συνάρτηση, που θα την ονομάσουμε `check_in()`, με την οποία ο χρήστης θα κάνει την είσοδό του στο πρόγραμμα. Θα πληκτρολογεί το όνομα χρήστη και το password, θα ελέγχουμε εάν έχει πληκτρολογήσει το σωστό password, δίνοντάς του τρεις (3) ευκαιρίες να το κάνει, και θα καταγράφουμε την ημερομηνία και ώρα της εισόδου του στο πρόγραμμα.

Ο κώδικας είναι:

```
from passlib.hash import pbkdf2_sha256
from datetime import datetime
# Password
pass_file = open("BLASTppv_password_file.txt", 'r')
password = pass_file.readline()
pass_file.close()

def check_in():
    """Entrance to the program via user name & password.
    Calls greet() & users_database() """
    print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS')
    while True:
        user_name = input('Enter your name, please:')
        if user_name.isalpha():
            break
        else:
            print('Enter a valid name, please!')
            continue
    now = datetime.now()
    date = now.strftime("%b-%d-%Y")
    time = now.strftime("%H:%M:%S")
    pwd = ' '
    i = 0
    while pbkdf2_sha256.verify(pwd, password) is False:
        pwd = input('Enter your password, please:')
        if pbkdf2_sha256.verify(pwd, password):
            greet(user_name)
            pwd_status = 'OK'
        else:
            i = i + 1
            if i <= 2:
                print('The password is incorrect! Try again')
                print('You can try', 3 - i, 'more times')
                continue
            elif i == 3:
                print('Program Execution Terminated...!')
                pwd_status = 'FAIL'
                break
    users_database(user_name, date, time, pwd_status)
```

Στη συνέχεια, θα εξηγήσουμε τι κάνει η συνάρτηση `check_in()` η οποία, όπως βλέπετε, δεν έχει παραμέτρους (formal parameters):

Μετά την εντολή `print()` που εμφανίζει στην οθόνη τον τίτλο του προγράμματός μας, ακολουθεί ένας βρόχος `while`, εντός του οποίου συμβαίνουν τα εξής:

Ζητούμε από τον χρήστη να πληκτρολογήσει το όνομά του και το εκχωρούμε στη μεταβλητή `user_name`. Σημειώστε ότι η μεταβλητή `user_name` είναι κατηγορίας `str`.

Εάν η μεταβλητή `user_name` αποτελείται μόνο από αλφαβητικούς χαρακτήρες, δηλαδή τα γράμματα του λατινικού αλφάβητου `a-z` και `A-Z`, τότε ενεργοποιείται η εντολή `break` και γίνεται έξοδος από τον βρόχο `while`.

Σε ό,τι αφορά το όνομα του χρήστη, είμαστε πολύ *ελαστικοί*. Έτσι, μπορεί κάποιος να πληκτρολογήσει ως όνομα χρήστη έστω και έναν αλφαβητικό χαρακτήρα π.χ. τα `g`, `G`, `George`, `GeorgePanagiotou`, είναι δεκτά ως ονόματα χρήστη.

Σε αντίθετη περίπτωση, δηλαδή εάν ο χρήστης δεν πληκτρολόγησε για όνομα χρήστη έναν ή περισσότερους συνεχόμενους αλφαβητικούς χαρακτήρες, αλλά το όνομα αποτελείται ή περιλαμβάνει αριθμούς, ειδικούς χαρακτήρες (`! @ # $ % ^ & * ?` κλπ.), κενό διάστημα ή πάτησε μόνο το `ENTER` του πληκτρολογίου, τότε:

εμφανίζεται στην οθόνη το μήνυμα:

```
Enter a valid name, please!
```

ενεργοποιείται η εντολή `continue` και γίνεται εκκίνηση του βρόχου `while` από την αρχή.

Η μέθοδος `isalpha()`, η οποία εφαρμόζεται επί μεταβλητών κατηγορίας `str`, όπως είναι η μεταβλητή `user_name`, λαμβάνει την τιμή `True` όταν η μεταβλητή αποτελείται μόνο από αλφαβητικούς χαρακτήρες (πεζά ή κεφαλαία γράμματα του λατινικού αλφάβητου, χωρίς κενά διαστήματα). Σε αντίθετη περίπτωση, λαμβάνει την τιμή `False`.

Ανάλογης χρήσης είναι η μέθοδος `isnumeric()`, η οποία εφαρμόζεται επί μεταβλητών κατηγορίας `str`, και λαμβάνει την τιμή `True` όταν η μεταβλητή αποτελείται μόνο από αριθμητικούς χαρακτήρες. Σε αντίθετη περίπτωση, λαμβάνει την τιμή `False`.

Τέλος, η μέθοδος `isalnum()`, η οποία εφαρμόζεται επί μεταβλητών κατηγορίας `str`, λαμβάνει την τιμή `True` όταν η μεταβλητή αποτελείται μόνο από αλφαριθμητικούς χαρακτήρες, δηλαδή γράμματα και αριθμούς μόνο. Σε αντίθετη περίπτωση, λαμβάνει την τιμή `False`.

Σημειώστε ότι, έτσι όπως έγινε η υλοποίηση του κώδικα στον βρόχο `while`, έχουμε έναν *ατέρμονα βρόχο*, δηλαδή εάν ο χρήστης αρνείται να πληκτρολογήσει ένα όνομα και επιμένει να πατάει π.χ. συνεχώς το `ENTER`, τότε δεν υπάρχει δυνατότητα εξόδου από τον βρόχο.

Χρονοσήμευση

Με τις εντολές που ακολουθούν κάνουμε *χρονοσήμευση* της στιγμής που ο χρήστης έκανε την είσοδό του στο πρόγραμμα, πληκτρολογώντας το όνομά του:

```
now = datetime.now()
date = now.strftime("%b-%d-%Y")
time = now.strftime("%H:%M:%S")
```

Για τη χρονοσήμευση θα χρησιμοποιήσουμε το ημερολόγιο και ρολόι του υπολογιστή μας, καθώς και την ενσωματωμένη βιβλιοθήκη της Python `datetime` την οποία πρέπει να κάνουμε `import`.

Το module `datetime` της βιβλιοθήκης `datetime` μας επιτρέπει να χειριζόμαστε στην Python ημερομηνίες και χρόνους. Για περισσότερες πληροφορίες ανατρέξτε στην τεκμηρίωση της βιβλιοθήκης: <https://docs.python.org/3/library/datetime.html>

Με την πρώτη γραμμή κώδικα *αποτυπώνουμε* την ημερομηνία και την ώρα της στιγμής που εκτελείται αυτή η γραμμή κώδικα και εκχωρούμε τις τιμές τους στη μεταβλητή `now`.

Με τη δεύτερη γραμμή κώδικα εκχωρούμε στη μεταβλητή `date` την ημερομηνία της στιγμής υπό τη μορφή Μήνας-Ημέρα-Έτος, π.χ. Feb-02-2022 .

Με την τρίτη γραμμή κώδικα εκχωρούμε στη μεταβλητή `time` την ώρα της στιγμής υπό τη μορφή Ώρα:Λεπτά:Δευτερόλεπτα, π.χ. 13:35:38 .

Έλεγχος του κωδικού πρόσβασης χρήστη

Οι υπόλοιπες γραμμές κώδικα της συνάρτησης `check_in()`, που επαναλαμβάνονται χάριν ευκολίας στη συνέχεια, υλοποιούν την εισαγωγή του `password` από τον χρήστη και τον έλεγχο της ορθότητάς του, συγκρίνοντάς το με το κρυπτογραφημένο `password` που περιέχεται στο αρχείο `BLASTppv_password_file.txt`

```
pwd = ' '
i = 0
while pbkdf2_sha256.verify(pwd, password) is False:
    pwd = input('Enter your password, please:')
    if pbkdf2_sha256.verify(pwd, password):
        greet(user_name)
        pwd_status = 'OK'
    else:
        i = i + 1
        if i <= 2:
            print('The password is incorrect! Try again')
            print('You can try', 3 - i, 'more times')
            continue
        elif i == 3:
            print('Program Execution Terminated...!')
            pwd_status = 'FAIL'
            break
users_database(user_name, date, time, pwd_status)
```

Αποφασίσαμε την τιμή του `password` που θα πληκτρολογεί ο χρήστης να την εκχωρούμε σε μια μεταβλητή με όνομα `pwd`, την οποία αρχικοποιούμε στην πρώτη γραμμή του κώδικα, με την τιμή μιας συμβολοσειράς ενός κενού διαστήματος (' '). Επίσης, εισάγουμε τη βοηθητική μεταβλητή *μετρητή* (counter) `i` την οποία αρχικοποιούμε με την ακέραια τιμή 0.

Μετά ακολουθεί στον κώδικα ένας βρόχος `while` στον οποίο συμβαίνουν τα εξής:

Στην επικεφαλίδα του βρόχου `while`, εφαρμόζοντας τη μέθοδο `verify()` επί της συνάρτησης κρυπτογράφησης `pbkdf2_sha256` ελέγχουμε κατά πόσον *επιβεβαιώνεται* ότι η τιμή της μεταβλητής `pwd` *ταυτίζεται* με την τιμή της μεταβλητής `password`, η οποία είναι το `MMM` στην κρυπτογραφημένη του μορφή.

Για να μπορέσουμε να πραγματοποιήσουμε την παραπάνω σύγκριση πρέπει προηγουμένως από το `module` της βιβλιοθήκης `passlib.hash` να έχουμε κάνει `import` τον αλγόριθμο κρυπτογράφησης `pbkdf2_sha256`.

Πρέπει, επίσης, πριν από την κλήση της συνάρτησης `check_in()`, ο κώδικάς μας να έχει διαβάσει από το αρχείο `BLASTppv_password_file.txt` το `password` στην κρυπτογραφημένη του μορφή και να το έχει εκχωρήσει στη μεταβλητή `password`. Αυτό υλοποιείται με τις γραμμές κώδικα που προηγούνται του ορισμού της συνάρτησης `check_in()`, κάτω από τη γραμμή σχολίου `# Password` .

Κατά τον πρώτο έλεγχο της συνθήκης στην επικεφαλίδα του βρόχου `while`, είναι προφανές ότι το αποτέλεσμα της επιβεβαίωσης με τη μέθοδο `verify()` είναι `False` και επομένως η εκτέλεση προχωρά στην πρώτη εντολή του μπλοκ.

Τότε, εμφανίζεται η προτροπή να εισάγουμε το `password` που γνωρίζουμε, το οποίο και πληκτρολογούμε. Η τιμή του εκχωρείται στη μεταβλητή `pwd`, αντικαθιστώντας την τιμή της συμβολοσειράς ενός κενού διαστήματος (' ') που είχε μέχρι τώρα.

Με το μπλοκ εντολών `if-else` που ακολουθεί, γίνεται νέος έλεγχος προκειμένου να γίνει επιβεβαίωση ότι η τιμή του `password` που πληκτρολογήσαμε (`pwd`) είναι ίδια με την τιμή του `password` του προγράμματος (`password`).

Εάν το αποτέλεσμα του ελέγχου είναι `True`, δηλαδή οι τιμές των `pwd` και `password` ταυτίζονται, τότε καλείται η συνάρτηση `greet(name)` με παράμετρο την τιμή της μεταβλητής `user_name`. Στη συνέχεια, στην παράγραφο 10.5 Καλωσόρισμα του χρήστη θα δούμε τι κάνει αυτή η συνάρτηση.

Επίσης, δημιουργούμε μια μεταβλητή με όνομα `pwd_status` και εκχωρούμε σε αυτή την τιμή της συμβολοσειράς `OK`.

Στην περίπτωση επιτυχούς πληκτρολόγησης του `password`, ο κλάδος `else` δεν εκτελείται και όπως έχουμε μάθει στην παράγραφο 05.3 Η εντολή επανάληψης `while`, η λειτουργία του προγράμματος επιστρέφει στην επικεφαλίδα του βρόχου `while`. Εκεί γίνεται ξανά έλεγχος της συνθήκης της επικεφαλίδας και τώρα το αποτέλεσμα είναι `True`. Επομένως, γίνεται έξοδος από τον βρόχο `while`.

Εάν το αποτέλεσμα του ελέγχου με την εντολή:

```
if pbkdf2_sha256.verify(pwd, password):
```

είναι `False`, δηλαδή οι τιμές των `pwd` και `password` δεν ταυτίζονται, τότε μεταφερόμαστε στον κλάδο `else` του μπλοκ `if-else` και δίνουμε στον χρήστη άλλες δύο (2) ευκαιρίες να πληκτρολογήσει το σωστό `password`.

Αυτό το υλοποιούμε με τη βοήθεια της βοηθητικής μεταβλητής-counter `i`, της οποίας η τιμή αυξάνεται κατά μια μονάδα ύστερα από κάθε προσπάθεια του χρήστη, ενώ στην οθόνη εμφανίζονται οι εναπομείνουσες προσπάθειες.

Όταν και η τρίτη προσπάθεια πληκτρολόγησης του σωστού `password` αποτύχει, δημιουργούμε μια μεταβλητή με όνομα `pwd_status` και εκχωρούμε σε αυτή την τιμή της συμβολοσειράς `FAIL`. Τότε, με την εντολή `break` γίνεται έξοδος από τον βρόχο `while`.

Στην περίπτωση τριών αποτυχημένων προσπαθειών πληκτρολόγησης του `password`, η αλληλουχία των μηνυμάτων που εμφανίζονται στην οθόνη είναι:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
Enter your name, please:George
Enter your password, please:1
The password is incorrect! Try again
You can try 2 more times
Enter your password, please:2
The password is incorrect! Try again
You can try 1 more times
Enter your password, please:3
Program Execution Terminated...!
```

Με την τελευταία γραμμή κώδικα της συνάρτησης `check_in()` καλούμε τη συνάρτηση `users_database(name, date, time, pwd)`, με παραμέτρους τις τιμές των μεταβλητών `user_name`, `date`, `time` και `pwd_status`.

Με τη βοήθεια της συνάρτησης αυτής, όπως θα δούμε στη συνέχεια, στην παράγραφο 10.6 Βάση δεδομένων χρηστών του προγράμματος, θα δημιουργήσουμε μια βάση δεδομένων των χρηστών του προγράμματός μας σε ένα φύλλο του Excel.

10.5 Καλωσόρισμα του χρήστη

Με τη συνάρτηση `greet(name)` κάνουμε ένα προσωποποιημένο καλωσόρισμα στον χρήστη του προγράμματος. Ο κώδικας με τον οποίο ορίζεται η συνάρτηση είναι:


```
def greet(name):
    """A personalised welcome to the user of the program
    Calls data_input()"""
    if datetime.now().hour < 12:
        welcome = 'Kalimera'
    else:
        welcome = 'Kalispera'
    print("Hello", name + ', ' + welcome)
    data_input()
```

Η πρώτη γραμμή κώδικα του μπλοκ της συνάρτησης, με τη βοήθεια της μεθόδου `datetime.now().hour` του module `datetime`, ελέγχει εάν η ώρα του ρολογιού του υπολογιστή μας είναι πριν τις 12:00 μεσημβρινή.

Σημειώστε ότι η Python χρησιμοποιεί το 24ωρο ρολόι, δηλαδή 00:00 – 24:00 (24-hour clock ή military time).

Στην περίπτωση μιας προ-μεσημβρινής ώρας, στη μεταβλητή `welcome` εκχωρείται η τιμή `Kalimera`. Εάν η ώρα είναι μετα-μεσημβρινή, τότε στη μεταβλητή `welcome` εκχωρείται η τιμή `Kalispera`.

Η εντολή `print` που ακολουθεί έχει δύο μεταβλητές, τη μεταβλητή `name`, της οποίας η τιμή είναι η τιμή της μεταβλητής `user_name`, και τη μεταβλητή `welcome`, η οποία λαμβάνει τιμή ανάλογα με την ώρα του ρολογιού του υπολογιστή μας.

Σημειώστε ότι η τιμή της μεταβλητής `user_name` πέρασε στη συνάρτηση `greet(name)` όταν την καλέσαμε ως `greet(user_name)` από τη συνάρτηση `check_in()`, και είναι η τιμή-όνομα που πληκτρολόγησε ο χρήστης του προγράμματος.

Έτσι, εάν ο χρήστης `George` τρέξει το πρόγραμμα προ-μεσημβρινές ώρες, θα δει στην οθόνη του το μήνυμα καλωσορίσματος:

```
Hello George, Kalimera
```

ενώ εάν το τρέξει μετα-μεσημβρινές ώρες, το μήνυμα θα είναι:

```
Hello George, Kalispera
```

Η τελευταία γραμμή κώδικα καλεί τη συνάρτηση `data_input()`, με τη βοήθεια της οποίας ο χρήστης θα κάνει την εισαγωγή των δεδομένων του προβλήματος για τον υπολογισμό της ρρν της ανατίναξης και θα τη δούμε στη συνέχεια, στην παράγραφο 10.7 Εισαγωγή δεδομένων του προβλήματος.

10.6 Βάση δεδομένων χρηστών του προγράμματος

Όπως έχουμε αναφέρει στην παράγραφο 10.1 Γενικά, θα δημιουργήσουμε μια απλή βάση δεδομένων, στο Microsoft Excel, όπου θα καταγράφονται το όνομα του χρήστη του προγράμματος, η ημερομηνία και ώρα εισόδου του στο πρόγραμμα και εάν έκανε εισαγωγή του σωστού ή όχι κωδικού πρόσβασης.

Για τον σκοπό αυτό ορίζουμε τη συνάρτηση `users_database(name, date, time, pwd)` και κάνουμε χρήση της εξωτερικής βιβλιοθήκης της Python `Openpyxl`, την οποία πρέπει να εγκαταστήσουμε στον υπολογιστή μας κατά τα γνωστά:

```
pip install openpyxl
```

Για περισσότερες πληροφορίες σχετικά με τη βιβλιοθήκη `Openpyxl` ανατρέξτε στο website: <https://openpyxl.readthedocs.io/en/stable/>

Σημειώστε ότι τη συνάρτηση `users_database(name, date, time, pwd)` την καλούμε από τη συνάρτηση `check_in()` ως `users_database(user_name, date, time, pwd_status)` και επομένως περνάμε σε αυτήν τις τιμές των μεταβλητών `user_name`, `date`, `time` και `pwd_status`, οι οποίες τους έχουν εκχωρηθεί σε προγενέστερες εντολές του κώδικα.

Ανοίγουμε το Excel και δημιουργούμε ένα workbook το οποίο περιέχει έναν τίτλο και τις επικεφαλίδες τεσσάρων στηλών (Εικόνα 10.2). Στην πρώτη στήλη θα καταγράφονται τα ονόματα των χρηστών του προγράμματος, στη δεύτερη και τρίτη στήλη θα καταγράφονται η ημερομηνία και η ώρα εισόδου του χρήστη στο πρόγραμμα και στην τέταρτη στήλη θα καταχωρείται η ένδειξη OK, εάν πληκτρολόγησε το σωστό password, ή η ένδειξη FAIL, εάν πληκτρολόγησε λάθος password και μετά την τρίτη προσπάθεια που έκανε.

Το workbook το αποθηκεύουμε με όνομα `BLASTppv_users_database.xlsx` στο directory του υπολογιστή μας, όπου έχουμε αποθηκευμένο και το πρόγραμμά μας.

	A	B	C	D	E
1	BLASTppv PROGRAM: User's DataBase				
2					
3	USER's NAME	DATE	TIME	PASSWORD	
4					
5					

Εικόνα 10.2. Οθόνη workbook του Microsoft Excel με το περιεχόμενο του αρχείου: `BLASTppv_users_database.xlsx`

Ο κώδικας της συνάρτησης με τη βοήθεια της οποίας θα καταχωρούνται οι χρήστες του προγράμματος στη βάση δεδομένων, δηλαδή στο workbook του Excel, είναι:

```
import openpyxl
def users_database(user, date, time, pwd):
    """Enters logs of the BLASTppv users into the
    user's DataBase"""
    workbook = openpyxl.load_workbook('BLASTppv_users_database.xlsx')
    sheet = workbook.active
    log = [[user, date, time, pwd]]
    for row in log:
        sheet.append(row)
    workbook.save('BLASTppv_users_database.xlsx')
```

Καταρχάς, κάνουμε `import` τη βιβλιοθήκη `openpyxl`. Η βιβλιοθήκη αυτή μας επιτρέπει να διαβάζουμε και να γράφουμε αρχεία του Microsoft Excel 2010 μορφής `.xlsx`, `.xlsm`, `.xltx` και `.xltm` μέσα από τον κώδικα της Python.

Στην πρώτη γραμμή κώδικα στο μπλοκ της συνάρτησης, με τη μέθοδο της βιβλιοθήκης `openpyxl` `openpyxl.load_workbook()` ανοίγουμε το workbook `BLASTppv_users_database.xlsx` και εκχωρούμε το αντικείμενό του στη μεταβλητή `workbook`.

Η επόμενη γραμμή κώδικα καθιστά ενεργό το worksheet του workbook και εκχωρούμε το αντικείμενό του στη μεταβλητή `sheet`. Σημειώστε ότι, όταν δημιουργούμε στο Excel ένα workbook, αυτόματα δημιουργείται και ένα worksheet.

Στη συνέχεια, δημιουργούμε μια λίστα που περιέχει, ως μέλος της, μια άλλη λίστα. Η δεύτερη λίστα έχει ως μέλη τις μεταβλητές `name`, `date`, `time` και `pwd`, οι οποίες λαμβάνουν ως τιμές τις τιμές των μεταβλητών `user_name`, `date`, `time` και `pwd_status` αντίστοιχα, από τη συνάρτηση `check_in()`. Την τιμή της λίστας την εκχωρούμε στη μεταβλητή `log`.

Στον βρόχο `for` με τη μέθοδο `sheet.append(row)` καταχωρούμε-προσθέτουμε στο τέλος του worksheet `sheet` μια γραμμή (`row`) που περιέχει τις τιμές των μεταβλητών `user_name`, `date`, `time` και `pwd_status`.

Οι τιμές αυτές θα καταχωρηθούν στα κελιά των στηλών A, B, C και D αντίστοιχα. Κάθε φορά που ένας χρήστης τρέχει ή προσπαθεί να τρέξει το πρόγραμμά μας, θα καταχωρούνται τα στοιχεία του σε μια καινούρια γραμμή του worksheet της βάσης δεδομένων που έχουμε φτιάξει στο Excel, όπως φαίνεται στην Εικόνα 10.3.

	A	B	C	D	E
1	BLASTppv PROGRAM: User's DataBase				
2					
3	USER's NAME	DATE	TIME	PASSWORD	
4	George	Feb-09-2021	12:35:26	OK	
5	Nikos	Feb-09-2021	12:37:02	FAIL	
6	Lakis	Feb-09-2021	13:51:48	OK	
7					

Εικόνα 10.3. Η βάση δεδομένων χρηστών του προγράμματος BLASTppv .py

Από την ανάγνωση των καταγραφών της βάσης δεδομένων στην Εικόνα 10.3 προκύπτει ότι ο χρήστης Nikos προσπάθησε τρεις φορές ανεπιτυχώς να πληκτρολογήσει το σωστό password, σε αντίθεση με τους χρήστες George και Lakis που πληκτρολόγησαν το σωστό password σε κάποια από τις τρεις προσπάθειες που είχαν στη διάθεσή τους.

Η τελευταία γραμμή του κώδικα σώζει το workbook στο αρχείο BLASTppv_users_database.xlsx

10.7 Εισαγωγή δεδομένων του προβλήματος

Η εισαγωγή των δεδομένων του προβλήματος υπολογισμού της prn γίνεται με τη βοήθεια της συνάρτησης `data_input()`.

Στον κώδικα της συνάρτησης έχουμε ενσωματώσει τεχνικές ελέγχου της συμβατότητας των δεδομένων, που πληκτρολογεί ο χρήστης του προγράμματος, με τις δομές των δεδομένων που απαιτούνται για να τρέξει σωστά το πρόγραμμά μας.

Επίσης, σε ορισμένα δεδομένα θέτουμε περιορισμούς, π.χ. η αναμενόμενη συχνότητα της δόνησης που θα πληκτρολογήσει ο χρήστης πρέπει να έχει τιμές μεταξύ 1 και 120 Hz. Ο κώδικας που θα γράψουμε θα κάνει τους σχετικούς ελέγχους και θα ενημερώνει τον χρήστη εάν η τιμή που πληκτρολόγησε είναι εκτός των επιτρεπτών ορίων.

Ο κώδικας της συνάρτησης `data_input()` είναι:

```
def data_input():
    """Prompts for data input via keyboard.
    Calls usbm_method() & din_method()"""
    print('EISAGOGH DEDOMENON ANATINAXHS:')
    distance_from_blast = float(input('Distance from Blast [m] :'))
    weight_of_explosives = float(input('Weight of Explosives [kg] :'))
    i = 0
    while True:
        try:
            plot_start = float(input('Plot "ppv vs Distance"
                                     ' Start Value (Min = 100) [m] :'))
            if plot_start >= 100.:
                break
            else:
                i = i + 1
                if i == 3:
                    print('Program Execution Terminated...!')
                    sys.exit()
                else:
                    print('Value out of range! Try again')
                    continue
        except ValueError:
            print('You must enter a number !')
            continue
```

```

while True:
    try:
        plot_end = float(input('Plot "ppv vs Distance" '
                                'End Value [m] :'))
        if plot_end > plot_start:
            break
        else:
            print('End Value must be > ', plot_start)
            continue
    except ValueError:
        print('You must enter a number !')
        continue
while True:
    try:
        ppv_method = input('PPV Estimation Method [USBM, DIN] :')
        assert ppv_method in ['USBM', 'DIN']
        break
    except:
        print('Type-in one of: USBM, DIN')
        continue
while True:
    try:
        frequency = float(input('Estimated ppv frequency '
                                '(1 - 120 Hz) [Hz] :'))
        if 1 <= frequency <= 120:
            break
        else:
            print('Frequency must be >= 1 & <= 120 Hz')
            continue
    except ValueError:
        print('You must enter a number !')
        continue
if ppv_method == 'USBM':
    while True:
        try:
            type_of_blast = input('Type of Blast '
                                    '[Mine, Quarry or Construction] :')
            assert type_of_blast in \
                ['Mine', 'Quarry', 'Construction']
            break
        except:
            print('Type-in one of: Mine, Quarry, Construction')
            continue
    usbm_method(distance_from_blast, weight_of_explosives,
                 frequency, type_of_blast, plot_start, plot_end)
elif ppv_method == 'DIN':
    while True:
        try:
            type_of_rock = input('Type of Rock '
                                    '[Sedimentary or Igneous] :')
            assert type_of_rock in ['Sedimentary', 'Igneous']
            break
        except:
            print('Type-in one of: Sedimentary, Igneous')
            continue
    din_method(distance_from_blast, weight_of_explosives,
                frequency, type_of_rock, plot_start, plot_end)

```

Οι τρεις πρώτες γραμμές του κώδικα εμφανίζουν στην οθόνη έναν τίτλο και στη συνέχεια προτρέπουν τον χρήστη να πληκτρολογήσει, κατά τα γνωστά, την απόσταση του σημείου, στο οποίο θέλουμε να υπολογίσουμε

την ρρν από το μέτωπο ανατίναξης, και την ποσότητα της εκρηκτικής ύλης. Οι τιμές αυτές εκχωρούνται στις αντίστοιχες μεταβλητές τους, κατηγορίας `float`.

Επίσης, εισάγουμε τη βοηθητική μεταβλητή-counter `i`, την οποία αρχικοποιούμε με την ακέραια τιμή 0.

Στη συνέχεια του κώδικα, η εισαγωγή των δεδομένων του προβλήματος γίνεται με μια τεχνική η οποία, για κάθε δεδομένο που θα πληκτρολογήσει ο χρήστης, περιλαμβάνει έναν βρόχο `while` εντός του οποίου υπάρχει ένα μπλοκ εντολών `try-except`. Εντός του μπλοκ `try-except`, κατά περίπτωση, μπορεί να υπάρχει και ένα ή περισσότερα μπλοκ εντολών `if-else`.

Το μπλοκ `try-except` μας επιτρέπει να συλλάβουμε και να διαχειριστούμε εξαιρέσεις.

Για να κατανοήσουμε τη χρήση του μπλοκ `try-except`, ας παρακολουθήσουμε πώς λειτουργεί στην περίπτωση του πρώτου βρόχου `while` της συνάρτησης `data_input()`.

Αυτόν τον βρόχο `while` τον γράψαμε για να ελέγξουμε εάν ο χρήστης πληκτρολόγησε σωστά την τιμή της απόστασης που θα είναι η αρχή της καμπύλης του διαγράμματος "`ppv vs Distance`", το οποίο θα κατασκευάζει στη συνέχεια το πρόγραμμά μας.

Στο τμήμα `try` του μπλοκ γράφουμε τις εντολές που εκτιμούμε ότι η εκτέλεση κάποιας ή κάποιων από αυτές μπορεί να εγείρει μια εξαίρεση. Έτσι, γράψαμε την εντολή:

```
plot_start = float(input('Plot "ppv vs Distance"
                        ' Start Value (Min= 100) [m] :'))
```

Εάν ο χρήστης πληκτρολογήσει έναν αριθμό μεγαλύτερο ή ίσο του 100, όπως πρέπει να κάνει σύμφωνα με τις οδηγίες που του δίνουμε, τότε ο έλεγχος με την εντολή `if` που ακολουθεί θα λάβει τιμή `True` και τότε θα εκτελεστεί η εντολή `break`, με την οποία θα γίνει η έξοδος από τον βρόχο `while`.

Εάν ο χρήστης δεν πληκτρολογήσει έναν αριθμό, π.χ. πληκτρολογήσει `!00` (αντί για 100), τότε επειδή ο interpreter της Python αναμένει ως `input` αριθμό προκειμένου να μπορέσει να τον μετατρέψει σε `float`, θα εγερθεί εξαίρεση και η ροή του προγράμματος θα μεταφερθεί άμεσα στο τμήμα `except` του μπλοκ `try-except`.

Εκεί, υπάρχει η εξαίρεση `ValueError`, που αναφέρεται στην κατηγορία των τιμών που διαχειρίζεται ένα πρόγραμμα. Στην Python υπάρχει μια συλλογή από εξαιρέσεις, τις οποίες μπορούμε να χρησιμοποιήσουμε κατά περίπτωση, και περιγράφονται αναλυτικά στο website:

<https://docs.python.org/3/library/exceptions.html>

Στο τμήμα `except` του μπλοκ εκτελούνται οι εντολές που υπάρχουν σε αυτό, δηλαδή εμφανίζεται στην οθόνη το μήνυμα:

You must enter a number !

και με την εντολή `continue`, που ακολουθεί, η ροή του προγράμματος θα μεταφερθεί στην επικεφαλίδα του βρόχου `while` για να επαναληφθεί η διαδικασία πληκτρολόγησης.

Εντός του τμήματος `try` του μπλοκ `try-except`, εάν ο έλεγχος της εντολής `if` λάβει τιμή `False`, δηλαδή εάν ο χρήστης πληκτρολογήσει τιμή μικρότερη του 100, με ένα δεύτερο μπλοκ εντολών `if-else`, του δίνουμε άλλες δύο ευκαιρίες να πληκτρολογήσει τιμή μεγαλύτερη ή ίση του 100, υποδεικνύοντας μάλιστα με το μήνυμα στην οθόνη:

Value out of range! Try again

το σφάλμα που κάνει.

Εάν και η τρίτη προσπάθεια είναι ανεπιτυχής, τότε εμφανίζεται στην οθόνη το μήνυμα:

Program Execution Terminated...!

και καλείται η συνάρτηση `exit ()` της βιβλιοθήκης `sys`, η οποία τερματίζει την εκτέλεση του προγράμματος.

Παρακάτω βλέπουμε τις ενέργειες του χρήστη και τα αποτελέσματα των ενεργειών του, όπως εμφανίζονται στην οθόνη, κατά την εισαγωγή της τιμής που θα εκχωρηθεί στη μεταβλητή `plot_start`.

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
Enter your name, please:George
Enter your password, please:MMM
Hello George, Kalimera
EISAGOGH DEDOMENON ANATINAXHS:
Distance from Blast [m] :100
Weight of Explosives [kg] :200
Plot "ppv vs Distance" Start Value (Min = 100) [m] :s
You must enter a number !
Plot "ppv vs Distance" Start Value (Min = 100) [m] :20
Value out of range! Try again
Plot "ppv vs Distance" Start Value (Min = 100) [m] :30
Value out of range! Try again
Plot "ppv vs Distance" Start Value (Min = 100) [m] :40
Program Execution Terminated...!
```

Σημειώστε ότι κατά την εισαγωγή από το πληκτρολόγιο των τιμών που θα εκχωρηθούν στις μεταβλητές `distance_from_blast` και `weight_of_explosives` δεν χρησιμοποιήσαμε την τεχνική ελέγχου με τη βοήθεια βρόχου `while` και μπλοκ εντολών `try-except`.

Επομένως, εάν ο χρήστης πληκτρολογήσει ως τιμή της απόστασης 150 αντί του ορθού 150 και ENTER, τότε τη διάγνωση του `ValueError` θα την κάνει και πάλι ο interpreter της Python, αλλά τώρα θα εμφανίσει το σχετικό μήνυμα λάθους ο interpreter και θα τερματίσει αμέσως την εκτέλεση του προγράμματος, όπως φαίνεται παρακάτω:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
Enter your name, please:George
Enter your password, please:MMM
Hello George, Kalispera
EISAGOGH DEDOMENON ANATINAXHS:
Distance from Blast [m] :150
Traceback (most recent call last):
in data_input()
    distance_from_blast = float(input('Distance from Blast [m] :'))
ValueError: could not convert string to float: '150'
```

Στον επόμενο βρόχο `while` και μπλοκ `try-except` γίνεται η εισαγωγή της τιμής της απόστασης που θα είναι η τελική τιμή της καμπύλης του διαγράμματος "ppv vs Distance". Εδώ ο λογικός περιορισμός είναι: η τιμή απόστασης του τέλους της καμπύλης να είναι μεγαλύτερη από την τιμή απόστασης της αρχής της και δεν θα υπάρχει η δυνατότητα τριών ευκαιριών σωστής πληκτρολόγησης.

Στη συνέχεια, ακολουθεί ένας βρόχος `while` και μπλοκ `try-except` με τη βοήθεια του οποίου ο χρήστης δηλώνει με ποιο τρόπο επιθυμεί να πραγματοποιηθεί η εκτίμηση της ppv, πληκτρολογώντας USBM ή DIN.

Η εντολή:

```
assert ppv_method in ['USBM', 'DIN']
```

θα ελέγξει εάν έκανε σωστά την πληκτρολόγηση. Δηλαδή, θα γίνει *διαβεβαίωση* (assert) εάν αυτό που πληκτρολόγησε περιλαμβάνεται στη λίστα ['USBM', 'DIN'].

Στην περίπτωση που το αποτέλεσμα αυτού του ελέγχου είναι True, τότε με την εντολή break γίνεται έξοδος από τον βρόχο while.

Εάν το αποτέλεσμα είναι False, τότε εγείρεται εξαίρεση, εμφανίζεται στην οθόνη το μήνυμα:

Type-in one of: USBM, DIN

και με την εντολή continue η ροή του προγράμματος θα μεταφερθεί στην επικεφαλίδα του βρόχου while για να επαναληφθεί η διαδικασία πληκτρολόγησης.

Ο τελευταίος βρόχος while και μπλοκ try-except προτρέπει τον χρήστη να πληκτρολογήσει μια τιμή για την εκτιμώμενη συχνότητα του κυματοπαλμού της prn, η οποία θα εκχωρηθεί στη μεταβλητή frequency. Ο περιορισμός που υπάρχει εδώ είναι η τιμή της μεταβλητής, που θα πληκτρολογήσουμε, να είναι μεταξύ 1 και 120 και να είναι βέβαια αριθμός.

Ο κώδικας της συνάρτησης data_input() τελειώνει με ένα μπλοκ εντολών if-elif

Ο κλάδος if ενεργοποιείται όταν ο χρήστης έχει πληκτρολογήσει ότι επιθυμεί η prn να εκτιμηθεί σύμφωνα με τη μέθοδο του USBM. Στην περίπτωση αυτή, μέσω ενός βρόχου while και μπλοκ εντολών try-except, ο χρήστης πληκτρολογεί τον τύπο της ανατίναξης: Mine, Quarry ή Construction και γίνεται ο σχετικός έλεγχος ορθής πληκτρολόγησης.

Μετά γίνεται κλήση της συνάρτησης:

```
usbm_method(distance_from_blast, weight_of_explosives,  
             frequency, type_of_blast, plot_start, plot_end)
```

με την οποία θα υπολογιστεί η prn κατά τη μέθοδο του USBM, όπως θα δούμε στην παράγραφο 10.8 *Εκτίμηση της prn με τη μέθοδο του USBM*.

Ο κλάδος elif ενεργοποιείται όταν ο χρήστης έχει πληκτρολογήσει ότι επιθυμεί η prn να εκτιμηθεί σύμφωνα με τη μέθοδο του προτύπου DIN. Στην περίπτωση αυτή, μέσω ενός βρόχου while και μπλοκ εντολών try-except, ο χρήστης πληκτρολογεί τον τύπο του πετρώματος: Sedimentary ή Igneous και γίνεται ο σχετικός έλεγχος ορθής πληκτρολόγησης.

Μετά γίνεται κλήση της συνάρτησης:

```
din_method(distance_from_blast, weight_of_explosives,  
            frequency, type_of_rock, plot_start, plot_end)
```

με την οποία θα υπολογιστεί η prn κατά τη μέθοδο του προτύπου DIN, όπως θα δούμε στην παράγραφο 10.9 *Εκτίμηση της prn κατά το πρότυπο DIN*.

Είναι γεγονός ότι η υλοποίηση του κώδικα της εισαγωγής των δεδομένων ενός προβλήματος με την τεχνική που δείξαμε, δηλαδή για κάθε δεδομένο που θα πληκτρολογήσει ο χρήστης πρέπει να υπάρχει ένας βρόχος while εντός του οποίου υπάρχει ένα μπλοκ εντολών try-except, απαιτεί το γράψιμο αρκετών γραμμών κώδικα.

Μας εξασφαλίζει όμως ότι υπάρχει συμβατότητα μεταξύ των δεδομένων που πληκτρολογεί ο χρήστης του προγράμματος με τις δομές των δεδομένων που απαιτούνται, για να τρέξει σωστά το πρόγραμμα, και αποτρέπει έναν άδοξο τερματισμό της εκτέλεσης του προγράμματός μας.

Σίγουρα ο τρόπος με τον οποίο υλοποιήσαμε την εκχώρηση τιμών στις μεταβλητές distance_from_blast και weight_of_explosives είναι πιο απλός και λιγότερο χρονοβόρος, αλλά θα πρέπει να είμαστε προετοιμασμένοι ότι, στην περίπτωση αυτή, είναι πολύ πιθανόν ο interpreter της Python να μας εμφανίσει ένα ενοχλητικό, αλλά ταυτόχρονα ενημερωτικό, κόκκινο μήνυμα λάθους και να τερματίσει βίαια την εκτέλεση του προγράμματός μας.

10.8 Εκτίμηση της ppv με τη μέθοδο του USBM

Για την εκτίμηση της ppv με τη μέθοδο του USBM ορίζουμε τη συνάρτηση:

```
usbm_method(distance_from_blast, weight_of_explosives, frequency,
type_of_blast, plot_start, plot_end)
```

της οποίας ο κώδικας είναι:

```
def usbm_method(distance_from_blast, weight_of_explosives,
                frequency, type_of_blast, plot_start, plot_end):
    """Calculates ppv using the USBM method.
    Calls results_output(), plot() & ppv_compliance_KMLE_plot()"""
    x = np.linspace(plot_start, plot_end)
    ppv_usb = usbm_ppv_Dix[type_of_blast][0]*((distance_from_blast/
        np.sqrt(weight_of_explosives))
        **usbm_ppv_Dix[type_of_blast][1])
    y_ppv_usb = usbm_ppv_Dix[type_of_blast][0]*\
        ((x/np.sqrt(weight_of_explosives))
        **usbm_ppv_Dix[type_of_blast][1])
    results_output(distance_from_blast,
                    weight_of_explosives, frequency,
                    ppv_usb, ppv_method='USBM',
                    type_of_blast=type_of_blast, type_of_rock='-')
    plot(x, y_ppv_usb, ppv_method='USBM')
    ppv_compliance_KMLE_plot(ppv_usb, frequency)
```

Επειδή η συνάρτηση `usbm_method()` έχει κληθεί από τη συνάρτηση `data_input()`, κατά την κλήση της περνούν σε αυτήν οι τιμές των μεταβλητών `distance_from_blast`, `weight_of_explosives`, `frequency`, `type_of_blast`, `plot_start` και `plot_end`, τις οποίες έχει πληκτρολογήσει ο χρήστης και αντιστοιχούν στις παραμέτρους της.

Η πρώτη γραμμή κώδικα του μπλοκ της συνάρτησης ορίζει τη μεταβλητή `x` που αντιπροσωπεύει την array των τιμών του οριζόντιου άξονα `x` του διαγράμματος `Distance vs ppv-USBM`, δηλαδή του άξονα της απόστασης από τη θέση ανατίναξης, με τιμές αρχής και τέλους αυτές που πληκτρολόγησε ο χρήστης.

Η επόμενη γραμμή κώδικα υπολογίζει, κατά τα γνωστά, την ppv σύμφωνα με τη μέθοδο του USBM.

Η τρίτη γραμμή κώδικα ορίζει τη μεταβλητή `y_ppv_usb` που αντιπροσωπεύει τις τιμές του κάθετου άξονα `y` του διαγράμματος `Distance vs ppv-USBM`, δηλαδή του άξονα της ppv, όπως αυτές υπολογίζονται με τη μέθοδο του USBM. Η μεταβλητή `y_ppv_usb` είναι μια array με τιμές τόσες όσες και η μεταβλητή `x`.

Στη συνέχεια, γίνεται κλήση της συνάρτησης:

```
results_output(distance_from_blast,
                weight_of_explosives, frequency,
                ppv_usb, ppv_method='USBM',
                type_of_blast=type_of_blast, type_of_rock='-')
```

με την οποία θα εμφανιστούν τα αποτελέσματα του προγράμματός μας στην οθόνη, δηλαδή τα δεδομένα του προβλήματος και η εκτιμηθείσα ppv κατά τη μέθοδο του USBM (βλέπε παράγραφο 10.10 Εμφάνιση αποτελεσμάτων).

Παρατηρήστε ότι, επειδή η συνάρτηση `results_output()` θα χρησιμοποιηθεί και από τη συνάρτηση `din_method()`, για την εμφάνιση των αντίστοιχων αποτελεσμάτων, στις μεταβλητές `ppv_method`, `type_of_blast` και `type_of_rock` εκχωρούμε τις προεπιλεγμένες τιμές `USBM`, `type_of_blast` και `-` αντίστοιχα, που αφορούν μόνο τον υπολογισμό της ppv με τη μέθοδο του USBM.

Ακολουθεί η κλήση της συνάρτησης:

```
plot(x, y_ppv_usb, ppv_method='USBM')
```


η οποία θα μας κατασκευάσει τη γραφική παράσταση Distance vs ppv-USBM.

Τέλος, γίνεται κλήση της συνάρτησης:

```
ppv_compliance_KMLE_plot(ppv_usb, frequency)
```

με την οποία κατασκευάζεται το *διάγραμμα συμμόρφωσης της ppv-συχνότητας δόνησης* σύμφωνα με τον ΚΜΛΕ, στο οποίο αναφερθήκαμε στην παράγραφο 02.3 Ασφαλή όρια μεγέθους δονήσεων, και τοποθετείται επί του διαγράμματος ένα σημάδι το οποίο αντιστοιχεί στην τιμή της ppv που υπολογίστηκε με τη μέθοδο του USBM (βλέπε παράγραφο 10.12 Διάγραμμα συμμόρφωσης με τον ΚΜΛΕ).

10.9 Εκτίμηση της ppv κατά το πρότυπο DIN

Για την εκτίμηση της ppv κατά το πρότυπο DIN ορίζουμε τη συνάρτηση `din_method(distance_from_blast, weight_of_explosives, frequency, type_of_rock, plot_start, plot_end)`, της οποίας ο κώδικας είναι:

```
def din_method(distance_from_blast, weight_of_explosives,
               frequency, type_of_rock, plot_start, plot_end):
    """Calculates ppv using the DIN method.
    Calls results_output(), plot() & ppv_compliance_KMLE_plot() """
    x = np.linspace(plot_start, plot_end)
    ppv_din = din_ppv_Dix[type_of_rock][0]* \
        (weight_of_explosives**din_ppv_Dix[type_of_rock][1])* \
        (distance_from_blast**din_ppv_Dix[type_of_rock][2])
    y_ppv_din = din_ppv_Dix[type_of_rock][0]* \
        (weight_of_explosives**din_ppv_Dix[type_of_rock][1])* \
        (x**din_ppv_Dix[type_of_rock][2])
    results_output(distance_from_blast,
                  weight_of_explosives, frequency,
                  ppv_din, ppv_method='DIN', type_of_blast='-',
                  type_of_rock=type_of_rock)
    plot(x, y_ppv_din, ppv_method='DIN')
    ppv_compliance_KMLE_plot(ppv_din, frequency)
```

Επειδή η συνάρτηση `din_method()` έχει κληθεί από τη συνάρτηση `data_input()`, κατά την κλήση της περνούν σε αυτήν οι τιμές των μεταβλητών `distance_from_blast`, `weight_of_explosives`, `frequency`, `type_of_rock`, `plot_start` και `plot_end`, τις οποίες έχει πληκτρολογήσει ο χρήστης και αντιστοιχούν στις παραμέτρους της.

Η δομή της συνάρτησης είναι ίδια με εκείνη της συνάρτησης `usbm_method()` και δεν θα επαναληφθεί εδώ.

Παρατηρήστε μόνο ότι στις συναρτήσεις `results_output()` και `plot()`, οι οποίες καλούνται από τη συνάρτηση `din_method()`, στις μεταβλητές `ppv_method`, `type_of_blast` και `type_of_rock` εκχωρούμε τις προεπιλεγμένες τιμές DIN, - και `type_of_rock` αντίστοιχα, οι οποίες αφορούν μόνο τον υπολογισμό της ppv κατά το πρότυπο DIN.

10.10 Εμφάνιση αποτελεσμάτων

Για την εμφάνιση των αποτελεσμάτων του προγράμματος στην οθόνη, ορίζουμε τη συνάρτηση `results_output(distance_from_blast, weight_of_explosives, frequency, ppv, ppv_method, type_of_blast, type_of_rock)`, της οποίας ο κώδικας είναι:

```
def results_output(distance_from_blast,
                  weight_of_explosives, frequency,
                  ppv, ppv_method, type_of_blast, type_of_rock):
    """Displays output on screen"""
    print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS')
```

```

print('INPUT DATA:')
print('Peak Particle Velocity Estimation Method : {:s}'.
      format(ppv_method))
print('Type of Blast           : {:s}'.
      format(type_of_blast))
print('Type of Rock           : {:s}'.
      format(type_of_rock))
print('Distance from Blast [m] : {:.1f}'.
      format(distance_from_blast))
print('Weight of Explosives [kg] : {:.1f}'.
      format(weight_of_explosives))
print('Estimated ppv frequency [Hz] : {:.1f}'.
      format(frequency))
print('OUTPUT:')
print('Peak Particle Velocity ({:s}) [mm/s]      = {:.2f}'.
      format(ppv_method, ppv))

```

Η συνάρτηση `results_output()`, οποία καλείται τόσο από τη συνάρτηση `usbm_method()`, όσο και από τη συνάρτηση `din_method()`, εμφανίζει τα αποτελέσματα της εκτέλεσης του προγράμματος που αντιστοιχούν στη μέθοδο υπολογισμού USBM ή DIN, ανάλογα με την επιλογή που έχει πληκτρολογήσει ο χρήστης.

10.11 Γραφική παράσταση της σχέσης ppv - απόσταση

Για την κατασκευή του διαγράμματος της καμπύλης μεταβολής της ppv συναρτήσει της απόστασης από τη θέση ανατίναξης, ορίζουμε τη συνάρτηση `plot(x, y, ppv_method)`, της οποίας ο κώδικας είναι:

```

def plot(x, y, ppv_method):
    """Plots the ppv-distance curve"""
    plt.plot(x, y, 'b-')
    if ppv_method == 'USBM':
        plt.legend(['ppv-USBM'])
        plt.title('Distance vs ppv-USBM')
    elif ppv_method == 'DIN':
        plt.legend(['ppv-DIN'])
        plt.title('Distance vs ppv-DIN')
    plt.xlabel('Distance from blast [m]')
    plt.ylabel('ppv [mm/s]')
    plt.xticks(np.arange(min(x), max(x) + 1, max(x)/10))
    plt.yticks(np.arange(0, max(y) + 1, 5))
    plt.grid(True)
    if ppv_method == 'USBM':
        plt.savefig('ppv_USBM_vs_distance.png')
    elif ppv_method == 'DIN':
        plt.savefig('ppv_DIN_vs_distance.png')
    plt.show()

```

Η συνάρτηση `plot()`, την οποία καλεί τόσο η συνάρτηση `usbm_method()` όσο και η συνάρτηση `din_method()`, εμφανίζει τη μεταβολή της ppv, που αντιστοιχεί στη μέθοδο υπολογισμού (USBM ή DIN) που έχει επιλέξει ο χρήστης, συναρτήσει της απόστασης από τη θέση ανατίναξης και για το εύρος τιμών απόστασης που έχει ορίσει ο χρήστης.

Παρατηρήστε ότι, ανάλογα με την επιλογή της μεθόδου υπολογισμού της ppv, διαμορφώνονται ο τίτλος και η λεζάντα του διαγράμματος, καθώς και το όνομα του αρχείου στο οποίο αποθηκεύεται το διάγραμμα.

Τέλος, στον οριζόντιο άξονα της απόστασης, τα *σημάδια υποδιαίρεσης* (ticks) και οι αριθμοί τους, επιλέγουμε να απέχουν μεταξύ τους απόσταση ίση με το 1/10 της μέγιστης τιμής της απόστασης, δηλαδή της τιμής της μεταβλητής `plot_end`. Κατ' αυτόν τον τρόπο αποφεύγουμε να υπάρχει επικάλυψη των αριθμών των ticks, όταν ο χρήστης δώσει μια μεγάλη τιμή στη μεταβλητή `plot_end` (αμυντικός προγραμματισμός...).

10.12 Διάγραμμα συμμόρφωσης με τον ΚΜΛΕ

Για την κατασκευή του διαγράμματος συμμόρφωσης της ppp-συχνότητας δόνησης σύμφωνα με τον ΚΜΛΕ (βλέπε παράγραφο 02.3 Ασφαλή όρια μεγέθους δονήσεων) και την τοποθέτηση επί αυτού του σημαδιού το οποίο αντιστοιχεί στην τιμή της ppp που υπολογίστηκε, ορίζουμε τη συνάρτηση `ppv_compliance_KMLE_plot(ppv, frequency)`, της οποίας ο κώδικας είναι:

```
def ppv_compliance_KMLE_plot(ppv, frequency):
    """Plots the KMLE compliance diagram Lines 1,2 & 3
    and the ppv-frequency point on the diagram"""
    fig0 = plt.figure(figsize=(8, 4), dpi=100)
    box = {'facecolor': 'w',
           'edgecolor': 'k',
           'boxstyle': 'round'}

    def line1(freq):
        if freq <= 10.:
            return 20.
        elif 10. < freq <= 50.:
            return 20. + (((freq - 10.)/40.)*20.)
        elif 50. < freq <= 100.:
            return 40. + (((freq - 50.)/50.)*10.)
        elif freq > 100.:
            return 50.
        else:
            return 0.

    freq = np.linspace(0., 120., 121)
    y = np.zeros(len(freq))
    for i in range(len(freq)):
        y[i] = line1(freq[i])
    plt.plot(freq, y, 'g', lw=2)

    def line2(freq):
        if freq <= 10.:
            return 5.
        elif 10. < freq <= 50.:
            return 5. + (((freq - 10.)/40.)*10.)
        elif 50. < freq <= 100.:
            return 15. + (((freq - 50.)/50.)*5.)
        elif freq > 100.:
            return 20.
        else:
            return 0.

    for i in range(len(freq)):
        y[i] = line2(freq[i])
    plt.plot(freq, y, 'b', lw=2)

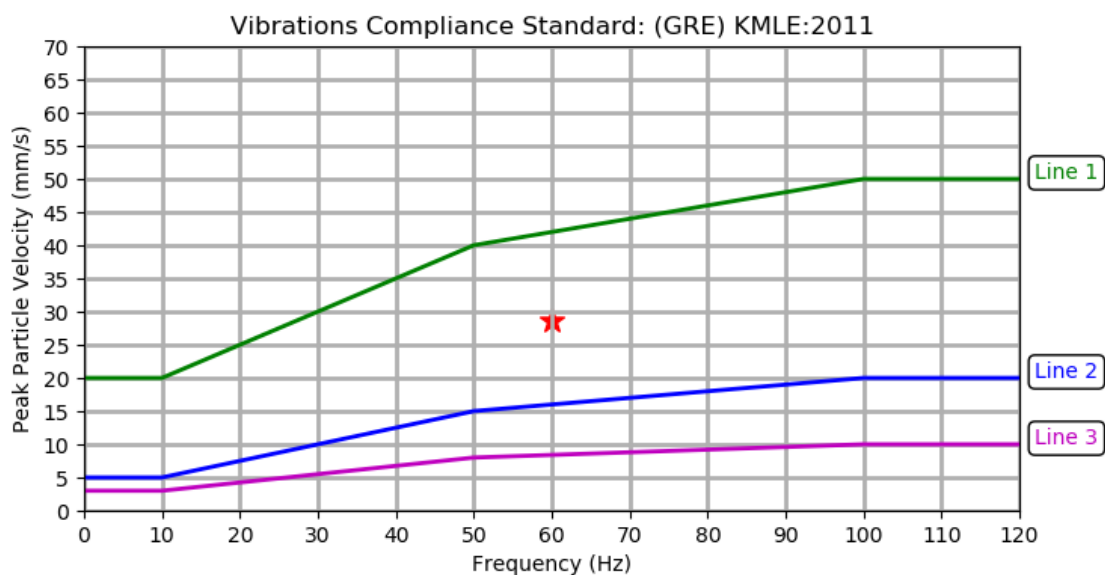
    def line3(freq):
        if freq <= 10.:
            return 3.
        elif 10. < freq <= 50.:
            return 3. + (((freq - 10.)/40.)*5.)
        elif 50. < freq <= 100.:
            return 8. + (((freq - 50.)/50.)*2.)
        elif freq > 100.:
            return 10.
        else:
            return 0.
```

```

for i in range(len(freq)):
    y[i] = line3(freq[i])
plt.plot(freq, y, 'm', lw=2)
plt.title('Vibrations Compliance Standard: (GRE) KMLE:2011')
plt.grid(True, lw=2)
plt.xlabel('Frequency (Hz)')
plt.xlim(0, 120)
plt.ylabel('Peak Particle Velocity (mm/s)')
plt.ylim(0, 70)
plt.xticks(np.arange(0, 120 + 1, 10))
plt.yticks(np.arange(0, 70 + 1, 5))
plt.text(122, 50, 'Line 1', bbox=box).set_color('g')
plt.text(122, 20, 'Line 2', bbox=box).set_color('b')
plt.text(122, 10, 'Line 3', bbox=box).set_color('m')
plt.scatter(frequency, ppv, marker='*', s=140, color='r')
if ppv > 69.:
    plt.text(70, 60, '!!! ppv = {:.2f} mm/s'.format(ppv),
            bbox=box, color='r')
fig0.savefig('KMLE_compliance_plot.png')
fig0.show()

```

Η συνάρτηση `ppv_compliance_KMLE_plot()` κατασκευάζει το διάγραμμα που παρουσιάζεται στην Εικόνα 10.4.



Εικόνα 10.4. Το διάγραμμα συμμόρφωσης της rrr-συχνότητας δόνησης σύμφωνα με τον ΚΜΛΕ, όπως σχεδιάζεται από τη συνάρτηση `ppv_compliance_KMLE_plot()`.

Το διάγραμμα της Εικόνας 10.4, και κατά συνέπεια η συνάρτηση που το κατασκευάζει, αποτελείται από δύο μέρη: ένα στατικό και ένα δυναμικό μέρος.

Το στατικό μέρος διαμορφώνει ένα διάγραμμα ανάλογο αυτού που περιλαμβάνει το κείμενο του άρθρου 88 του ΚΜΛΕ και εμφανίζεται στην Εικόνα 2.1.

Το δυναμικό μέρος τοποθετεί επί του διαγράμματος, με τη μορφή ενός αστεριού κόκκινου χρώματος, το σημείο που αντιστοιχεί στην rrv, την οποία υπολόγισε το πρόγραμμά μας, και στην εκτιμώμενη συχνότητα του κυματοπαλμού της δόνησης, που πληκτρολόγησε ο χρήστης.

Το στατικό μέρος

Με τη γραμμή κώδικα:

```
fig0 = plt.figure(figsize=(8, 4), dpi=100)
```

καλούμε τη συνάρτηση `figure()` του module `pyplot` της βιβλιοθήκης `Matplotlib`, η οποία ορίζει ένα αντικείμενο εικόνας, διαστάσεων πλάτους 8 και ύψους 4 inches και ανάλυσης 100 dots-per-inch (dpi).

Για περισσότερες πληροφορίες σχετικά με τη συνάρτηση `figure()` του module `pyplot` ανατρέξτε στη σελίδα τεκμηρίωσης της συνάρτησης:

https://matplotlib.org/stable/api/as_gen/matplotlib.pyplot.figure.html

Το αντικείμενο της εικόνας το εκχωρούμε στη μεταβλητή `fig0`, την οποία στο τέλος του ορισμού της συνάρτησης `prv_compliance_KMLE_plot()` τη χρησιμοποιούμε για να σώσουμε το διάγραμμα στο αρχείο `KMLE_compliance_plot.png` και στη συνέχεια για να το εμφανίσουμε στην οθόνη.

Με τη γραμμή κώδικα:

```
box = {'facecolor': 'w',  
       'edgecolor': 'k',  
       'boxstyle': 'round'}
```

ορίζουμε τη μεταβλητή `box`, η οποία περιλαμβάνει τα χαρακτηριστικά ενός *κουτιού* (πρόκειται για τα *τρία κουτιά* που εμφανίζονται στο δεξιό περίγραμμα του διαγράμματος), το οποίο έχει λευκό χρώμα (`w`) στην επιφάνειά του, μαύρο περίγραμμα (`k`) και στρογγυλεμένες γωνίες (`round`). Τη μεταβλητή `box` θα τη χρησιμοποιήσουμε στη συνέχεια.

Εντός της συνάρτησης `prv_compliance_KMLE_plot()` ορίζεται η συνάρτηση:

```
def line1(freq):  
    if freq <= 10.:  
        return 20.  
    elif 10. < freq <= 50.:  
        return 20. + (((freq - 10.)/40.)*20.)  
    elif 50. < freq <= 100.:  
        return 40. + (((freq - 50.)/50.)*10.)  
    elif freq > 100.:  
        return 50.  
    else:  
        return 0.
```

η οποία σε συνδυασμό με τον κώδικα που ακολουθεί εκτός του μπλοκ της συνάρτησης:

```
freq = np.linspace(0., 120., 121)  
y = np.zeros(len(freq))  
for i in range(len(freq)):  
    y[i] = line1(freq[i])  
plt.plot(freq, y, 'g', lw=2)
```

σχεδιάζει την άνω (πράσινη) γραμμή του διαγράμματος (Line 1), λαμβάνοντας υπόψη τις τιμές του πίνακα του άρθρου 88 του ΚΜΛΕ που αντιστοιχούν σε *Κτήρια γραφείων και Εργοστάσια* (βλέπε Εικόνα 2.1).

Κατ' ανάλογο τρόπο, η συνάρτηση:

```
def line2(freq):  
    if freq <= 10.:  
        return 5.  
    elif 10. < freq <= 50.:  
        return 5. + (((freq - 10.)/40.)*10.)  
    elif 50. < freq <= 100.:  
        return 15. + (((freq - 50.)/50.)*5.)  
    elif freq > 100.:  
        return 20.  
    else:  
        return 0.
```

σε συνδυασμό με τον κώδικα:

```
for i in range(len(freq)):
    y[i] = line2(freq[i])
plt.plot(freq, y, 'b', lw=2)
```

σχεδιάζει τη μεσαία (μπλε) γραμμή του διαγράμματος (Line 2), λαμβάνοντας υπόψη τις τιμές του πίνακα του άρθρου 88 του ΚΜΛΕ που αντιστοιχούν σε *Συνήθεις κατασκευές οικιών* (βλέπε Εικόνα 2.1).

Τέλος, η συνάρτηση:

```
def line3(freq):
    if freq <= 10.:
        return 3.
    elif 10. < freq <= 50.:
        return 3. + (((freq - 10.)/40.)*5.)
    elif 50. < freq <= 100.:
        return 8. + (((freq - 50.)/50.)*2.)
    elif freq > 100.:
        return 10.
    else:
        return 0.
```

σε συνδυασμό με τον κώδικα:

```
for i in range(len(freq)):
    y[i] = line3(freq[i])
plt.plot(freq, y, 'm', lw=2)
```

σχεδιάζει την κάτω (φούξια) γραμμή του διαγράμματος (Line 3), λαμβάνοντας υπόψη τις τιμές του πίνακα του άρθρου 88 του ΚΜΛΕ που αντιστοιχούν σε *Κατασκευές που λόγω της ιδιαίτερης ευαισθησίας τους στις δονήσεις δεν εντάσσονται στις κατηγορίες 1 και 2 (π.χ. διατηρητέα)* (βλέπε Εικόνα 2.1).

Οι υπόλοιπες εντολές του κώδικα της συνάρτησης `ppv_compliance_KMLE_plot()` μας είναι ήδη γνωστές από το Κεφάλαιο 06 *Γραφικές παραστάσεις* και την παράγραφο 07.5 *Διαγράμματα: πολλές μεταβλητές σε ένα διάγραμμα*, εκτός από τις:

```
plt.text(122, 50, 'Line 1', bbox=box).set_color('g')
plt.text(122, 20, 'Line 2', bbox=box).set_color('b')
plt.text(122, 10, 'Line 3', bbox=box).set_color('m')
```

οι οποίες σχεδιάζουν και τοποθετούν τα κουτιά κειμένου στο δεξιό άκρο των γραμμών του διαγράμματος.

Για το πρώτο κουτί, που αντιστοιχεί στη γραμμή Line 1, η συνάρτηση `text()` του module `pyplot` της βιβλιοθήκης Matplotlib γράφει το κείμενο `Line 1` τοποθετώντας το πρώτο γράμμα του κειμένου (`L`) στη θέση των συντεταγμένων 122, 50.

Το σύστημα συντεταγμένων της αρχής του κειμένου είναι ίδιο με εκείνο του διαγράμματος, δηλαδή το 122 είναι 2 μονάδες δεξιά από το 120 του οριζόντιου άξονα και το 50 είναι 50 μονάδες πάνω από την αρχή (το μηδέν) του κάθετου άξονα.

Το κείμενο βρίσκεται εντός ενός κουτιού, του οποίου τα χαρακτηριστικά λαμβάνονται από τη μεταβλητή `box` και το χρώμα του κειμένου είναι πράσινο, ίδιο με το χρώμα της γραμμής.

Κάνοντας τις κατάλληλες διαφοροποιήσεις στις παραμέτρους της συνάρτησης `text()`, οι υπόλοιπες δύο γραμμές κώδικα σχεδιάζουν και τοποθετούν τα κουτιά κειμένου για τις γραμμές Line 2 και Line 3.

Για περισσότερες πληροφορίες σχετικά με τη συνάρτηση `text()` του module `pyplot` της βιβλιοθήκης Matplotlib ανατρέξτε στη σελίδα τεκμηρίωσης:

https://matplotlib.org/stable/api/as_gen/matplotlib.pyplot.text.html

Το δυναμικό μέρος

Το δυναμικό μέρος αποτελείται από τις εντολές:

```
plt.scatter(frequency, ppv, marker='*', s=140, color='r')
if ppv > 69.:
    plt.text(70, 60, '!!! ppv = {:.2f} mm/s'.format(ppv),
            bbox=box, color='r')
```

Η συνάρτηση `scatter()` του module `pyplot` της βιβλιοθήκης `Matplotlib` τοποθετεί σε ένα σημείο επί του διαγράμματος, με συντεταγμένες τις τιμές των μεταβλητών `frequency` και `ppv`, ένα σημάδι που έχει τη μορφή ενός αστεριού κόκκινου χρώματος. Η παράμετρος `s=140` καθορίζει το μέγεθος του αστεριού (η default τιμή είναι 36).

Για περισσότερες πληροφορίες σχετικά με τη συνάρτηση `text()` του module `pyplot` της βιβλιοθήκης `Matplotlib` ανατρέξτε στη σελίδα τεκμηρίωσης:

https://matplotlib.org/stable/api/as_gen/matplotlib.pyplot.scatter.html

Είναι προφανές ότι, η θέση του αστεριού πάνω στο διάγραμμα εξαρτάται από τις τιμές που λαμβάνουν οι μεταβλητές `frequency` και `ppv`.

Η τιμή της μεταβλητής `frequency` (εκτιμώμενη συχνότητα του κυματοπαλμού της δόνησης) είναι εκείνη που πληκτρολόγησε ο χρήστης κατά την εισαγωγή των δεδομένων του προβλήματος και η συνάρτηση `data_input()` υποχρεώνει τον χρήστη να πληκτρολογήσει μια τιμή μεταξύ 1 και 120.

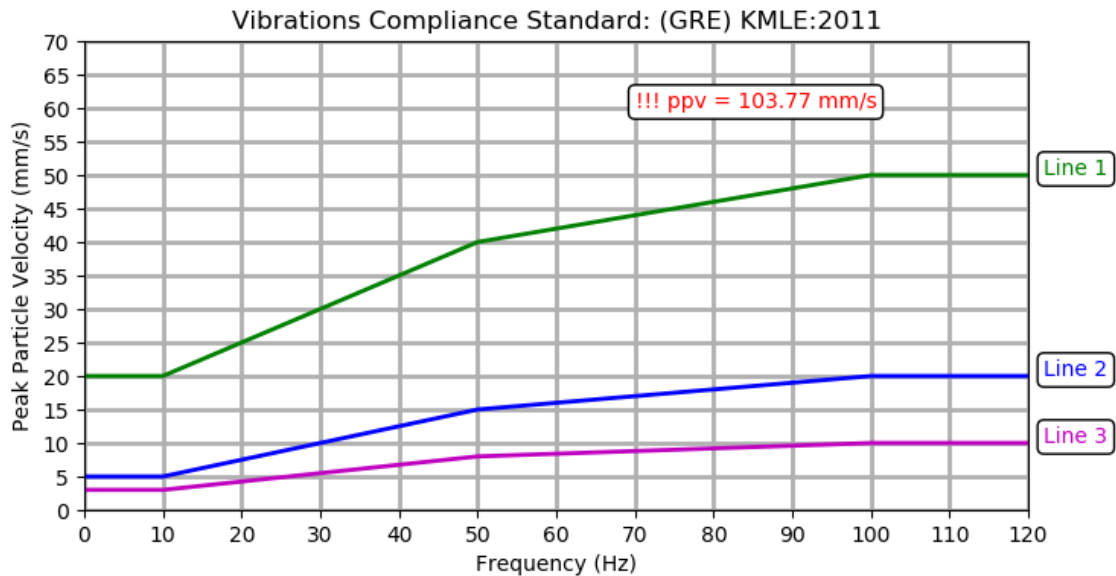
Η τιμή της μεταβλητής `ppv` υπολογίζεται από τη συνάρτηση `usbm_method()` ή τη συνάρτηση `din_method()` και το μέγεθός της εξαρτάται, κυρίως, από τις τιμές των μεταβλητών `distance_from_blast` και `weight_of_explosives`, τις οποίες επίσης πληκτρολογεί ο χρήστης, χωρίς όμως να υφίσταται κανένας περιορισμός ως προς τις μέγιστες τιμές τους.

Κατά συνέπεια, υπάρχει περίπτωση η τιμή της `ppv`, που υπολογίζεται για τα δεδομένα ενός συγκεκριμένου προβλήματος, να είναι μεγαλύτερη των 70 mm/s, που είναι η μέγιστη τιμή της `ppv` που υπάρχει στον κάθετο άξονα του διαγράμματος. Εάν συμβεί αυτό, η συνάρτηση `scatter()` δεν θα μας εμφανίσει το σημείο του αστεριού, γιατί η θέση του θα αντιστοιχεί σε συντεταγμένες εκτός των ορίων του διαγράμματος.

Για να χειριστούμε περιπτώσεις όπως αυτή (αμυντικός προγραμματισμός...), προσθέτουμε στον κώδικά μας ένα μπλοκ `if` με το οποίο, εάν η τιμή της `ppv` που υπολογίζεται από το πρόγραμμα είναι μεγαλύτερη από 69 mm/s, τότε να εμφανίζεται εντός ενός κουτιού στη θέση των συντεταγμένων 70, 60 (δηλαδή εντός του διαγράμματος) κείμενο κόκκινου χρώματος με την τιμή της `ppv`.

Στην Εικόνα 10.5 παρουσιάζεται μια τέτοια περίπτωση, η οποία προκύπτει κατά τον υπολογισμό της `ppv` ενός προβλήματος, τα αποτελέσματα του οποίου εμφανίζονται στο πλαίσιο που ακολουθεί:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
INPUT DATA:
Peak Particle Velocity Estimation Method : USBM
Type of Blast : Mine
Type of Rock : -
Distance from Blast [m] : 100.0
Weight of Explosives [kg] : 500.0
Estimated ppv frequency [Hz] : 50.0
OUTPUT:
Peak Particle Velocity (USBM) [mm/s] = 103.77
```



Εικόνα 10.5. Το διάγραμμα συμμόρφωσης της ppv-συχνότητας με τον ΚΜΛΕ, στην περίπτωση τιμής της ppv που βρίσκεται εκτός των ορίων του διαγράμματος.

10.13 Η συνάρτηση `main()`

Οι τελευταίες γραμμές του κώδικα του προγράμματός μας περιλαμβάνουν:

τον ορισμό της συνάρτησης `main()`, η οποία εκκινεί (πυροδοτεί) την εκτέλεση του προγράμματος, προσδιορίζοντας στον interpreter της Python να αρχίσει την εκτέλεση του script του προγράμματός μας από τη συνάρτηση `check_in()`, την οποία και καλεί:

```
def main():
    check_in()
```

και τον ιδιωματισμό (idiom) της Python:

```
if __name__ == "__main__":
    main()
```

προκειμένου να τσεκάρει ο interpreter της Python εάν το script που θέλουμε να τρέξουμε είναι ένα πρόγραμμα και όχι κάποιο module μιας βιβλιοθήκης.

10.14 Ο κώδικας στο τέλος του Κεφαλαίου 10

Ο κώδικας του προγράμματός μας, όπως αυτός έχει διαμορφωθεί στο τέλος του Κεφαλαίου 10, δίνεται στη συνέχεια:

```
# Το BLASTppv.py είναι ένα πρόγραμμα που
# υπολογίζει την ppv μιας αντανάξης σε metalleia/latomeia/erga
# Import of Packages
import matplotlib.pyplot as plt
import numpy as np
import sys
from datetime import datetime
import openpyxl
from passlib.hash import pbkdf2_sha256
# Dictionaries
usbm_ppv_Dix = {'Mine': [1140., -1.60], 'Quarry': [1090., -1.82],
                'Construction': [1730., -1.60]}
din_ppv_Dix = {'Sedimentary': [969, 0.6, -1.5],
```



```

        'Igneous': [206, 0.8, -1.3]}

# Password
pass_file = open("BLASTppv_password_file.txt", 'r')
password = pass_file.readline()
pass_file.close()

def check_in():
    """Entrance to the program via user name & password.
    Calls greet() & users_database()"""
    print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS')
    while True:
        user_name = input('Enter your name, please:')
        if user_name.isalpha():
            break
        else:
            print('Enter a valid name, please!')
            continue
    now = datetime.now()
    date = now.strftime("%b-%d-%Y")
    time = now.strftime("%H:%M:%S")
    pwd = ' '
    i = 0
    while pbkdf2_sha256.verify(pwd, password) is False:
        pwd = input('Enter your password, please:')
        if pbkdf2_sha256.verify(pwd, password):
            greet(user_name)
            pwd_status = 'OK'
        else:
            i = i + 1
            if i <= 2:
                print('The password is incorrect! Try again')
                print('You can try', 3 - i, 'more times')
                continue
            elif i == 3:
                print('Program Execution Terminated...!')
                pwd_status = 'FAIL'
                break
    users_database(user_name, date, time, pwd_status)

def greet(name):
    """A personalised welcome to the user of the program
    Calls data_input()"""
    if datetime.now().hour < 12:
        welcome = 'Kalimera'
    else:
        welcome = 'Kalispera'
    print("Hello", name + ', ' + welcome)
    data_input()

def users_database(user, date, time, pwd):
    """Enters logs of the BLASTppv users into the
    user's DataBase"""
    workbook = openpyxl.load_workbook('BLASTppv_users_database.xlsx')
    sheet = workbook.active
    log = [[user, date, time, pwd]]
    for row in log:
        sheet.append(row)
    workbook.save('BLASTppv_users_database.xlsx')

```

```

def data_input():
    """Prompts for data input via keyboard.
    Calls usbm_method() & din_method()"""
    print('EISAGOGH DEDOMENON ANATINAXHS:')
    distance_from_blast = float(input('Distance from Blast [m] :'))
    weight_of_explosives = float(input('Weight of Explosives [kg] :'))
    i = 0
    while True:
        try:
            plot_start = float(input('Plot "ppv vs Distance" '
                                     'Start Value (Min = 100) [m] :'))
            if plot_start >= 100.:
                break
            else:
                i = i + 1
                if i == 3:
                    print('Program Execution Terminated...!')
                    sys.exit()
                else:
                    print('Value out of range! Try again')
                    continue
        except ValueError:
            print('You must enter a number !')
            continue
    while True:
        try:
            plot_end = float(input('Plot "ppv vs Distance" '
                                   'End Value [m] :'))
            if plot_end > plot_start:
                break
            else:
                print('End Value must be > ', plot_start)
                continue
        except ValueError:
            print('You must enter a number !')
            continue
    while True:
        try:
            ppv_method = input('PPV Estimation Method [USBM, DIN] :')
            assert ppv_method in ['USBM', 'DIN']
            break
        except:
            print('Type-in one of: USBM, DIN')
            continue
    while True:
        try:
            frequency = float(input('Estimated ppv frequency '
                                    '(1 - 120 Hz) [Hz] :'))
            if 1 <= frequency <= 120:
                break
            else:
                print('Frequency must be >= 1 & <= 120 Hz')
                continue
        except ValueError:
            print('You must enter a number !')
            continue
    if ppv_method == 'USBM':
        while True:
            try:
                type_of_blast = input('Type of Blast '

```

```

        '[Mine, Quarry or Construction] :')
    assert type_of_blast in \
        ['Mine', 'Quarry', 'Construction']
    break
except:
    print('Type-in one of: Mine, Quarry, Construction')
    continue
usbm_method(distance_from_blast, weight_of_explosives,
            frequency, type_of_blast, plot_start, plot_end)
elif ppv_method == 'DIN':
    while True:
        try:
            type_of_rock = input('Type of Rock '
                                '[Sedimentary or Igneous] :')
            assert type_of_rock in ['Sedimentary', 'Igneous']
            break
        except:
            print('Type-in one of: Sedimentary, Igneous')
            continue
    din_method(distance_from_blast, weight_of_explosives,
              frequency, type_of_rock, plot_start, plot_end)

def usbm_method(distance_from_blast, weight_of_explosives,
                frequency, type_of_blast, plot_start, plot_end):
    """Calculates ppv using the USBM method.
    Calls results_output(), plot() & ppv_compliance_KMLE_plot() """
    x = np.linspace(plot_start, plot_end)
    ppv_usb = usbm_ppv_Dix[type_of_blast][0]*((distance_from_blast/
        np.sqrt(weight_of_explosives))
        **usbm_ppv_Dix[type_of_blast][1])
    y_ppv_usb = usbm_ppv_Dix[type_of_blast][0]*\
        ((x/np.sqrt(weight_of_explosives))
        **usbm_ppv_Dix[type_of_blast][1])
    results_output(distance_from_blast,
                  weight_of_explosives, frequency,
                  ppv_usb, ppv_method='USBM',
                  type_of_blast=type_of_blast, type_of_rock='-')
    plot(x, y_ppv_usb, ppv_method='USBM')
    ppv_compliance_KMLE_plot(ppv_usb, frequency)

def din_method(distance_from_blast, weight_of_explosives,
                frequency, type_of_rock, plot_start, plot_end):
    """Calculates ppv using the DIN method.
    Calls results_output(), plot() & ppv_compliance_KMLE_plot() """
    x = np.linspace(plot_start, plot_end)
    ppv_din = din_ppv_Dix[type_of_rock][0]* \
        (weight_of_explosives**din_ppv_Dix[type_of_rock][1])* \
        (distance_from_blast**din_ppv_Dix[type_of_rock][2])
    y_ppv_din = din_ppv_Dix[type_of_rock][0]* \
        (weight_of_explosives**din_ppv_Dix[type_of_rock][1])* \
        (x**din_ppv_Dix[type_of_rock][2])
    results_output(distance_from_blast,
                  weight_of_explosives, frequency,
                  ppv_din, ppv_method='DIN', type_of_blast='-',
                  type_of_rock=type_of_rock)
    plot(x, y_ppv_din, ppv_method='DIN')
    ppv_compliance_KMLE_plot(ppv_din, frequency)

```

```

def results_output(distance_from_blast,
                   weight_of_explosives, frequency,
                   ppv, ppv_method, type_of_blast, type_of_rock):
    """Displays output on screen"""
    print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS')
    print('INPUT DATA:')
    print('Peak Particle Velocity Estimation Method : {:s}'.
          format(ppv_method))
    print('Type of Blast : {:s}'.
          format(type_of_blast))
    print('Type of Rock : {:s}'.
          format(type_of_rock))
    print('Distance from Blast [m] : {:.1f}'.
          format(distance_from_blast))
    print('Weight of Explosives [kg] : {:.1f}'.
          format(weight_of_explosives))
    print('Estimated ppv frequency [Hz] : {:.1f}'.
          format(frequency))
    print('OUTPUT:')
    print('Peak Particle Velocity ({:s}) [mm/s] = {:.2f}'.
          format(ppv_method, ppv))

def plot(x, y, ppv_method):
    """Plots the ppv-distance curve"""
    plt.plot(x, y, 'b-')
    if ppv_method == 'USBM':
        plt.legend(['ppv-USBM'])
        plt.title('Distance vs ppv-USBM')
    elif ppv_method == 'DIN':
        plt.legend(['ppv-DIN'])
        plt.title('Distance vs ppv-DIN')
    plt.xlabel('Distance from blast [m]')
    plt.ylabel('ppv [mm/s]')
    plt.xticks(np.arange(min(x), max(x) + 1, max(x)/10))
    plt.yticks(np.arange(0, max(y) + 1, 5))
    plt.grid(True)
    if ppv_method == 'USBM':
        plt.savefig('ppv_USBM_vs_distance.png')
    elif ppv_method == 'DIN':
        plt.savefig('ppv_DIN_vs_distance.png')
    plt.show()

def ppv_compliance_KMLE_plot(ppv, frequency):
    """Plots the KMLE compliance diagram Lines 1,2 & 3
    and the ppv-frequency point on the diagram"""
    fig0 = plt.figure(figsize=(8, 4), dpi=100)
    box = {'facecolor': 'w',
           'edgecolor': 'k',
           'boxstyle': 'round'}

    def line1(freq):
        if freq <= 10.:
            return 20.
        elif 10. < freq <= 50.:
            return 20. + (((freq - 10.)/40.)*20.)
        elif 50. < freq <= 100.:
            return 40. + (((freq - 50.)/50.)*10.)
        elif freq > 100.:
            return 50.

```

```

        else:
            return 0.

freq = np.linspace(0., 120., 121)
y = np.zeros(len(freq))
for i in range(len(freq)):
    y[i] = line1(freq[i])
plt.plot(freq, y, 'g', lw=2)

def line2(freq):
    if freq <= 10.:
        return 5.
    elif 10. < freq <= 50.:
        return 5. + (((freq - 10.)/40.)*10.)
    elif 50. < freq <= 100.:
        return 15. + (((freq - 50.)/50.)*5.)
    elif freq > 100.:
        return 20.
    else:
        return 0.

for i in range(len(freq)):
    y[i] = line2(freq[i])
plt.plot(freq, y, 'b', lw=2)

def line3(freq):
    if freq <= 10.:
        return 3.
    elif 10. < freq <= 50.:
        return 3. + (((freq - 10.)/40.)*5.)
    elif 50. < freq <= 100.:
        return 8. + (((freq - 50.)/50.)*2.)
    elif freq > 100.:
        return 10.
    else:
        return 0.

for i in range(len(freq)):
    y[i] = line3(freq[i])
plt.plot(freq, y, 'm', lw=2)
plt.title('Vibrations Compliance Standard: (GRE) KMLE:2011')
plt.grid(True, lw=2)
plt.xlabel('Frequency (Hz)')
plt.xlim(0, 120)
plt.ylabel('Peak Particle Velocity (mm/s)')
plt.ylim(0, 70)
plt.xticks(np.arange(0, 120 + 1, 10))
plt.yticks(np.arange(0, 70 + 1, 5))
plt.text(122, 50, 'Line 1', bbox=box).set_color('g')
plt.text(122, 20, 'Line 2', bbox=box).set_color('b')
plt.text(122, 10, 'Line 3', bbox=box).set_color('m')
plt.scatter(frequency, ppv, marker='*', s=140, color='r')
if ppv > 69.:
    plt.text(70, 60, '!!! ppv = {:.2f} mm/s'.format(ppv),
            bbox=box, color='r')
fig0.savefig('KMLE_compliance_plot.png')
fig0.show()

def main():
    check_in()

```

```
if __name__ == "__main__":  
    main()
```

Αποθηκεύουμε το script του προγράμματός μας σε αρχείο με όνομα BLASTppv.py

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο BLASTppv.py μαζί με τα αρχεία BLASTppv_password_file.txt και BLASTppv_users_database.xlsx

Στη συνέχεια, δίνονται δύο εφαρμογές επίδειξης της χρήσης του προγράμματος, με την πρώτη να αναφέρεται στην περίπτωση εκτίμησης της ppv με τη μέθοδο του USBM και τη δεύτερη σύμφωνα με το πρότυπο DIN. Τα αριθμητικά δεδομένα εισόδου είναι κοινά και για τις δύο περιπτώσεις.

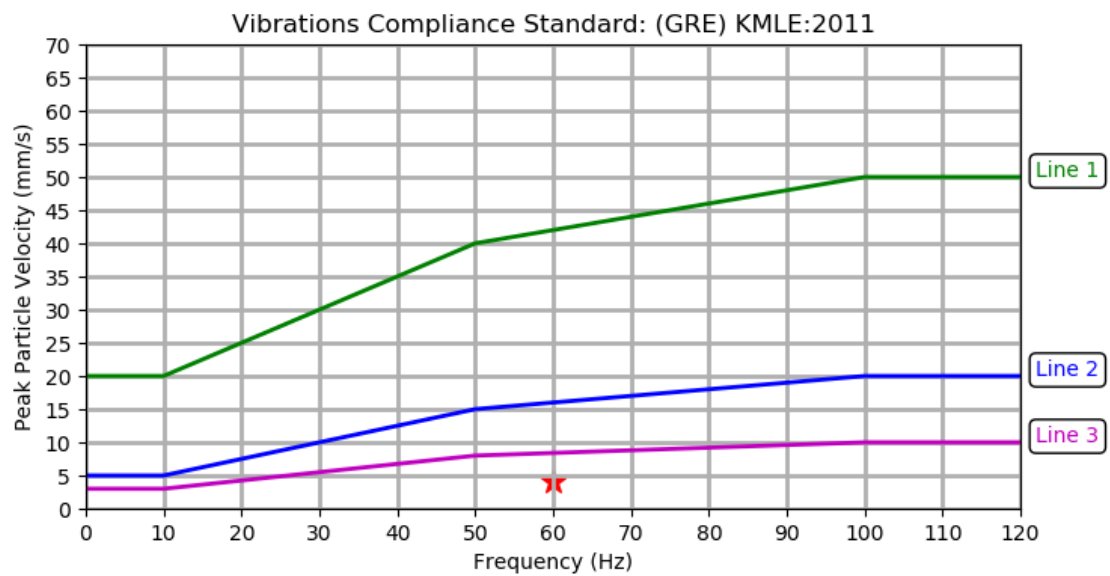
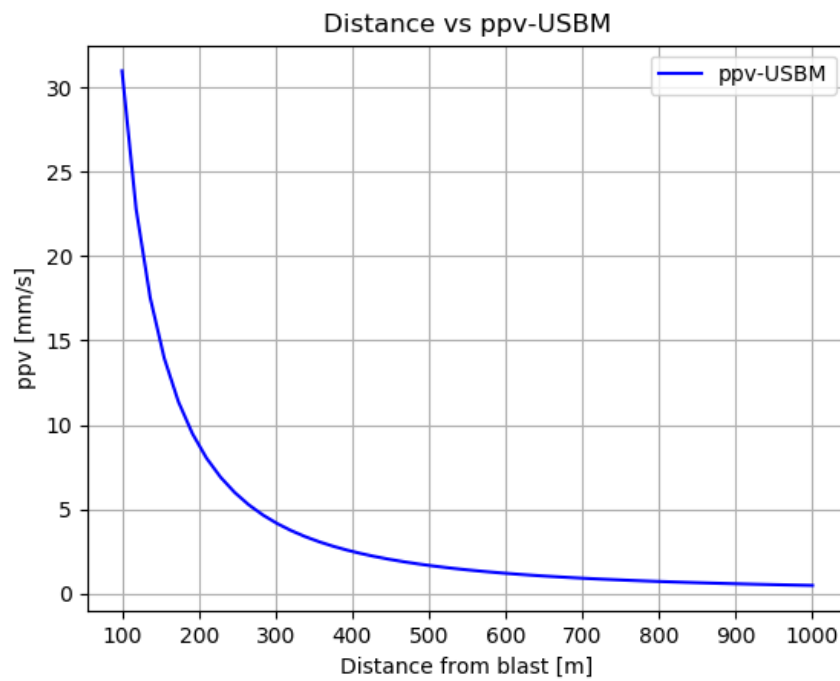
Υπολογισμός με τη μέθοδο του USBM

Δεδομένα εισόδου:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS  
Enter your name, please:George  
Enter your password, please:MMM  
Hello George, Kalispera  
EISAGOGH DEDOMENON ANATINAXHS:  
Distance from Blast [m] :300  
Weight of Explosives [kg] :200  
Plot "ppv vs Distance" Start Value (Min = 100) [m] :100  
Plot "ppv vs Distance" End Value [m] :1000  
PPV Estimation Method [USBM, DIN] :USBM  
Estimated ppv frequency (1 - 120 Hz) [Hz] :60  
Type of Blast [Mine, Quarry or Construction] :Quarry
```

Αποτελέσματα:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS  
INPUT DATA:  
Peak Particle Velocity Estimation Method : USBM  
Type of Blast : Quarry  
Type of Rock : -  
Distance from Blast [m] : 300.0  
Weight of Explosives [kg] : 200.0  
Estimated ppv frequency [Hz] : 60.0  
OUTPUT:  
Peak Particle Velocity (USBM) [mm/s] = 4.20
```



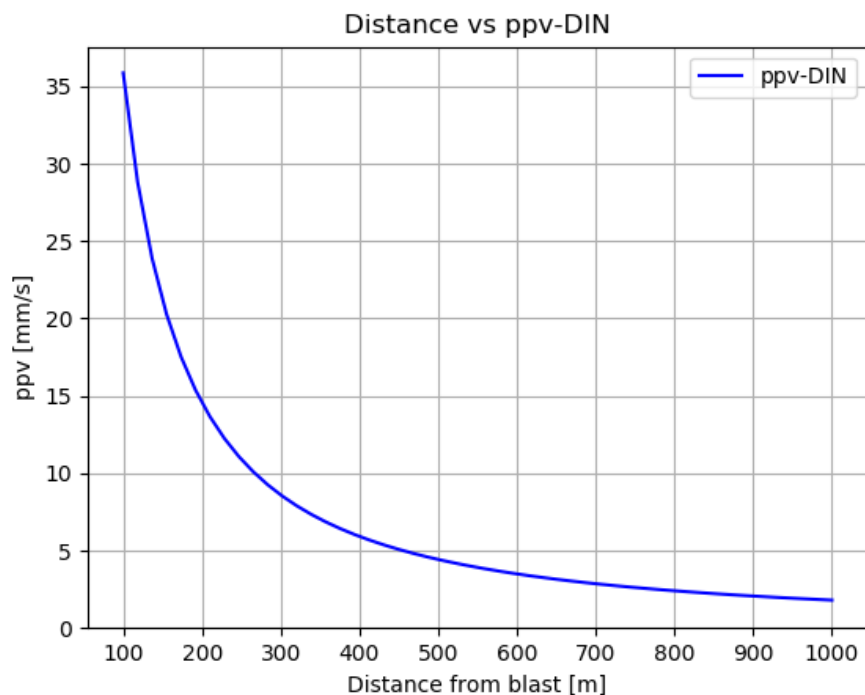
Υπολογισμός με τη μέθοδο του προτύπου DIN

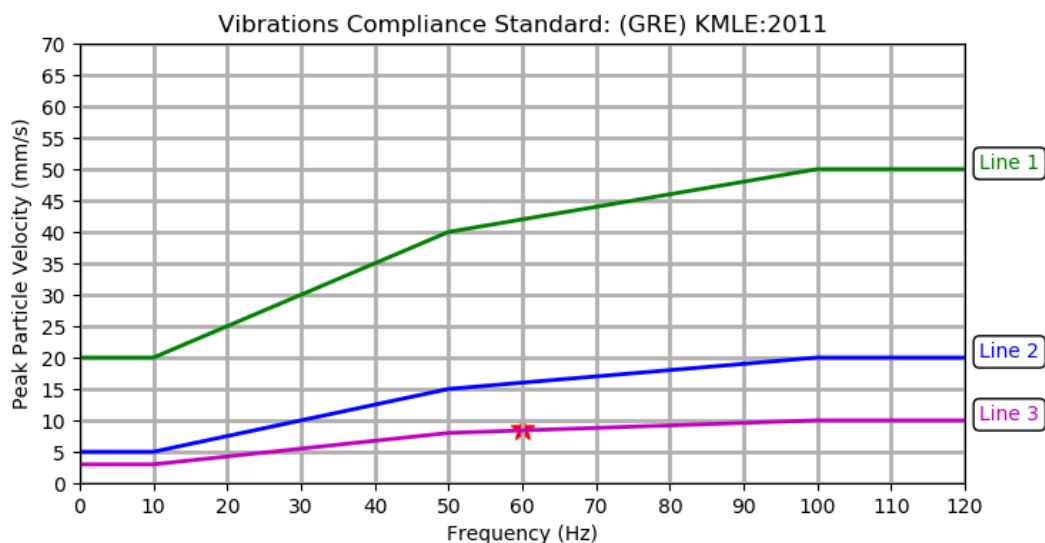
Δεδομένα εισόδου:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
Enter your name, please:George
Enter your password, please:MMM
Hello George, Kalispera
EISAGOGH DEDOMENON ANATINAXHS:
Distance from Blast [m] :300
Weight of Explosives [kg] :200
Plot "ppv vs Distance" Start Value (Min = 100) [m] :100
Plot "ppv vs Distance" End Value [m] :1000
PPV Estimation Method [USBM, DIN] :DIN
Estimated ppv frequency (1 - 120 Hz) [Hz] :60
Type of Rock [Sedimentary or Igneous] :Igneous
```

Αποτελέσματα:

```
PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS
INPUT DATA:
Peak Particle Velocity Estimation Method : DIN
Type of Blast : -
Type of Rock : Igneous
Distance from Blast [m] : 300.0
Weight of Explosives [kg] : 200.0
Estimated ppv frequency [Hz] : 60.0
OUTPUT:
Peak Particle Velocity (DIN) [mm/s] = 8.60
```





10.15 Φορητότητα προγραμμάτων Python

Για να τρέξει ένας χρήστης τον κώδικα του προγράμματος `BLASTppv.py` στον υπολογιστή του πρέπει προηγουμένως να έχει εγκαταστήσει σε αυτόν:

- τη γλώσσα Python,
- τις εξωτερικές (third-party) βιβλιοθήκες:
 - NumPy,
 - Matplotlib,
 - Openpyxl,
 - Passlib.

Επίσης, στο ίδιο directory, μαζί με το αρχείο `BLASTppv.py`, πρέπει να βρίσκονται και τα αρχεία:

`BLASTppv_password_file.txt` και

`BLASTppv_users_database.xlsx`

Από τα παραπάνω είναι προφανές ότι όσοι χρήστες δεν έχουν επαρκείς γνώσεις πληροφορικής, θα αντιμετωπίσουν αντικειμενικές δυσκολίες και δεν θα μπορέσουν να χρησιμοποιήσουν το πρόγραμμά μας.

Επίσης, υπάρχουν δυνητικοί χρήστες του προγράμματός μας, οι οποίοι ενώ διαθέτουν επαρκείς γνώσεις πληροφορικής, δεν είναι διατεθειμένοι, για διάφορους λόγους, να *αναστατώσουν* τον υπολογιστή τους, εγκαθιστώντας επιπλέον λογισμικό για να τρέξουν μια εφαρμογή, όπως είναι αυτή που υλοποιείται από το πρόγραμμα `BLASTppv.py`

Έτσι, το πρόγραμμά μας, διανέμοντάς το στους χρήστες υπό τη μορφή ενός αρχείου-script της Python, ενώ είναι πλήρως λειτουργικό, υπολείπεται σε *φορητότητα* (portability).

Τη λύση στο πρόβλημα αυτό δίνει η χρήση της βιβλιοθήκης PyInstaller.

Με τη βοήθεια αυτής της βιβλιοθήκης:

- Θα δημιουργήσουμε ένα εκτελέσιμο αρχείο του script `BLASTppv.py`, με το όνομα `BLASTppv.exe`.
- Το αρχείο `BLASTppv.exe` μαζί με ό,τι άλλο απαιτείται για να τρέξει το πρόγραμμά μας, δηλαδή τον interpreter και τις βιβλιοθήκες της Python, θα περιλαμβάνονται σε έναν φάκελο (folder) με όνομα `BLASTppv`.

Στη συνέχεια, στον φάκελο BLASTppn θα αντιγράψουμε τα αρχεία:

BLASTppn_password_file.txt και

BLASTppn_users_database.xlsx

Τέλος, με τη βοήθεια ενός προγράμματος συμπίεσης π.χ. του WinRAR των Windows, θα συμπίεσουμε τον φάκελο BLASTppn, για να μειώσουμε το μέγεθός του και θα τον διανεύουμε ως το αρχείο BLASTppn.rar

Ο χρήστης που επιθυμεί να τρέξει το πρόγραμμά μας θα πρέπει:

- Να αποσυμπίεσει το αρχείο BLASTppn.rar
- Να ανοίξει τον φάκελο BLASTppn και
- Να κάνει διπλό-κλικ στο αρχείο BLASTppn.exe

Σημειώστε ότι στον υπολογιστή του χρήστη δεν θα γίνει εγκατάσταση της Python, αλλά ούτε και των εξωτερικών βιβλιοθηκών NumPy, Matplotlib, Openpyxl και Passlib.

Εκτελώντας το αρχείο BLASTppn.exe στο Desktop των Windows, θα εμφανιστεί ένα παράθυρο με τις προτροπές για την εισαγωγή των δεδομένων του προβλήματος υπολογισμού της ppn μιας ανατίναξης.

Όντας εξωτερική βιβλιοθήκη, η PyInstaller πρέπει να εγκατασταθεί στον υπολογιστή μας κατά τα γνωστά:
pip install pyinstaller

Στη συνέχεια, από το παράθυρο εντολών των Windows, το Command Prompt, και όντας στο directory όπου βρίσκεται το αρχείο BLASTppn.py πληκτρολογούμε:

```
pyinstaller BLASTppn.py
```

και η PyInstaller θα δημιουργήσει ένα directory με όνομα dist, εντός του οποίου υπάρχει ο φάκελος BLASTppn για τα περαιτέρω.

Στο website του βιβλίου περιλαμβάνεται το αρχείο BLASTppn.rar

Λεπτομερείς πληροφορίες για τη χρήση της βιβλιοθήκης PyInstaller υπάρχουν στο website:
<https://www.pyinstaller.org>

11 Επίλυση εξισώσεων

11.1 Γενικά

Η επίλυση εξισώσεων αποτελεί καθημερινότητα για τον Μηχανικό.

Στα Κεφάλαια που προηγήθηκαν χρησιμοποιήσαμε μαθηματικές σχέσεις – εξισώσεις για να επιλύσουμε προβλήματα, όπως τον υπολογισμό της ppn , του εμβαδού ενός ακανόνιστου σχήματος ή του μοριακού βάρους μιας χημικής ένωσης.

Αυτό που κάναμε, στα προγράμματα Python που αναπτύξαμε, ήταν να δίνουμε τιμές στις παραμέτρους της μαθηματικής σχέσης, που βρίσκονται δεξιά του $=$, και το αριθμητικό αποτέλεσμα των πράξεων να το εκχωρούμε στη μεταβλητή που βρίσκεται αριστερά του $=$.

Στο Κεφάλαιο αυτό θα δούμε γενικότερα το θέμα των εξισώσεων, παρουσιάζοντας τρόπους επίλυσης εξισώσεων με έναν ή περισσότερους αγνώστους (συστήματα εξισώσεων), διαφορικών εξισώσεων και ολοκληρωμάτων.

Για τον σκοπό αυτό θα χρησιμοποιήσουμε τρεις διαφορετικές προσεγγίσεις: Στις δύο πρώτες θα κάνουμε χρήση συναρτήσεων των εξωτερικών βιβλιοθηκών SymPy και SciPy της Python. Στην τρίτη προσέγγιση θα αναπτύξουμε μια δική μας συνάρτηση επίλυσης εξισώσεων, η οποία βασίζεται στη μαθηματική μέθοδο των *διαδοχικών διχοτομήσεων* (bisection method) και την οποία θα μπορούμε να την καλούμε σε όλα τα προγράμματα Python που γράφουμε.

11.2 Επίλυση εξισώσεων: βιβλιοθήκη SymPy

Η βιβλιοθήκη SymPy ανήκει στην κατηγορία των *Computer Algebra Systems* (CAS), στην οποία ανήκουν και τα εμπορικά πακέτα MATLAB και Mathematica.

Όντας εξωτερική βιβλιοθήκη (third-party), η SymPy πρέπει να εγκατασταθεί στον υπολογιστή μας κατά τα γνωστά:

```
pip install sympy
```

Μαζί με τη SymPy θα εγκατασταθεί και η βιβλιοθήκη mpmath, γιατί υπάρχει εξάρτηση (dependency) της SymPy από την mpmath.

Για περισσότερες πληροφορίες και για την τεκμηρίωση των βιβλιοθηκών ανατρέξτε στα website τους: <https://www.sympy.org> και <https://mpmath.org>

Η SymPy είναι ένα *Symbolic CAS* (SymPy = Symbolic Python) και αυτό σημαίνει ότι οι αριθμοί και οι πράξεις αναπαρίστανται *συμβολικά*, με συνέπεια τα αποτελέσματα των υπολογισμών που γίνονται με τη βοήθεια των συναρτήσεων της να είναι *ακριβή*.

Για παράδειγμα, η μαθηματική έκφραση $\sqrt{2}$ αναπαρίσταται στη SymPy ως `sqrt(2)`, δηλαδή *τετραγωνική ρίζα του 2* και όχι με την προσεγγιστική τιμή του $\sqrt{2}$, δηλαδή 1.4142135623730951.

Εάν γράψουμε και τρέξουμε τον παρακάτω κώδικα:

```
from sympy import *
import numpy as np
sqrt_sympy = sqrt(2)
print('sqrt_sympy = ', sqrt_sympy)
sqrt_numpy = np.sqrt(2)
print('sqrt_numpy = ', sqrt_numpy)
```

το αποτέλεσμα που εμφανίζεται στην οθόνη είναι:

```
sqrt_sympy = sqrt(2)
sqrt_numpy = 1.4142135623730951
```

Στην πρώτη περίπτωση εκχωρήσαμε στη μεταβλητή `sqrt_sympy` την τιμή της τετραγωνικής ρίζας του 2 κάνοντας χρήση της συνάρτησης `sqrt()` της βιβλιοθήκης `SymPy`, γράφοντας `sqrt(2)`, και το αποτέλεσμα είναι: `sqrt(2)`.

Στη δεύτερη περίπτωση εκχωρήσαμε στη μεταβλητή `sqrt_numpy` την τιμή της τετραγωνικής ρίζας του 2 κάνοντας χρήση της συνάρτησης `sqrt()` της βιβλιοθήκης `NumPy`, γράφοντας `np.sqrt(2)`, και το αποτέλεσμα είναι: `1.4142135623730951`.

11.2.1 Εξισώσεις με έναν άγνωστο

Έστω ότι θέλουμε να λύσουμε τη δευτεροβάθμια εξίσωση:

$$x^2 + 2x - 8 = 0 \quad [11.1]$$

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from sympy import *
x = symbols('x')
eq1 = x**2 + 2*x - 8
roots_eq1 = solve(eq1, x)
print('x =', roots_eq1)
```

Στην οθόνη θα εμφανιστεί το παρακάτω αποτέλεσμα:

```
x = [-4, 2]
```

δηλαδή οι δύο ρίζες της εξίσωσης, υπό τη μορφή στοιχείων μιας λίστας.

Με την πρώτη γραμμή κώδικα κάνουμε `import` τη βιβλιοθήκη `sympy` στον κώδικά μας.

Με τη δεύτερη γραμμή κώδικα και με τη βοήθεια της συνάρτησης `symbols()` της `SymPy` ορίζουμε ότι η συμβολοσειρά `x` είναι σύμβολο.

Στην επόμενη σειρά κώδικα γράφουμε την Εξίσωση [11.1] με τον συνηθισμένο τρόπο γραφής στη γλώσσα Python, μιας και η βιβλιοθήκη `SymPy` είναι γραμμένη σε γλώσσα Python, και την εκχωρούμε στη μεταβλητή `eq1`.

Για να επιλύσουμε την εξίσωση, χρησιμοποιούμε τη συνάρτηση `solve()` της `SymPy`, η οποία έχει δύο παραμέτρους: την εξίσωση, η οποία πρέπει να έχει τη μορφή $f(\dots) = 0$, και τον άγνωστο ως προς τον οποίο θέλουμε να λύσουμε την εξίσωση.

Έτσι, στην τέταρτη γραμμή του κώδικα με την εντολή:

```
roots_eq1 = solve(eq1, x)
```

λύνουμε την εξίσωση `eq1` ως προς τον άγνωστο `x` και εκχωρούμε τις τιμές των ριζών στη μεταβλητή `roots_eq1`. Τέλος, εμφανίζουμε τη λύση στην οθόνη.

Έστω ότι θέλουμε τώρα να λύσουμε την τριτοβάθμια εξίσωση (πολυώνυμο) :

$$x^3 + x^2 + x + 1 = 0 \quad [11.2]$$

Στο παραπάνω κώδικα προσθέτουμε τις γραμμές:

```
eq2 = x**3 + x**2 + x + 1
roots_eq2 = solve(eq2, x, dict=True)
print('x =', roots_eq2)
```

Η πρώτη γραμμή εκχωρεί την Εξίσωση [11.2] στη μεταβλητή `eq2`.

Με τη δεύτερη γραμμή κώδικα λύνουμε την εξίσωση eq2 ως προς τον άγνωστο x και εκχωρούμε τις τιμές των ριζών στη μεταβλητή roots_eq2.

Με την παράμετρο dict=True της συνάρτησης solve() ζητάμε οι τιμές των ριζών να εμφανιστούν σε μια λίστα, τα στοιχεία της οποίας να έχουν τη μορφή στοιχείων λεξικού.

Η εντολή print() που ακολουθεί εμφανίζει στην οθόνη το αποτέλεσμα:

```
x = [{x: -1}, {x: -I}, {x: I}]
```

Βλέπουμε ότι η επίλυση της Εξίσωσης [11.2] δίνει τρεις ρίζες, εκ των οποίων η μία είναι πραγματικός αριθμός και οι άλλες δύο είναι μιγαδικός αριθμός και μάλιστα η φανταστική μονάδα: $\pm I$.

Παρατήρηση: Η SymPy εμφανίζει τη φανταστική μονάδα με I κεφαλαίο, ενώ εμείς έχουμε συνηθίσει να τη βλέπουμε-γράφουμε με i πεζό.

Εάν δεν μας ενδιαφέρουν οι μιγαδικές ρίζες μιας εξίσωσης, τότε στη συνάρτηση symbols(), όπου ορίσαμε ότι η συμβολοσειρά x είναι σύμβολο, περιλαμβάνουμε την παράμετρο real=True. Έτσι, εάν γράψουμε:

```
x = symbols('x', real=True)
```

ως αποτέλεσμα της επίλυσης της Εξίσωσης [11.2] θα εμφανιστεί μόνο η πραγματική ρίζα:

```
x = [{x: -1}]
```

Η default επιλογή της SymPy είναι: complex=True

Η SymPy έχει, επίσης, τη δυνατότητα να επιλύει εξισώσεις της μορφής $f(x, y, z, \dots) = 0$ ως προς κάποιον από τους αγνώστους x, y, z, ..., χρησιμοποιώντας αντί για αριθμητικές τιμές σύμβολα.

Έστω, για παράδειγμα, ότι θέλουμε να χρησιμοποιήσουμε την Εξίσωση [2.1] της παραγράφου 02.2 *Μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους*, την οποία για διευκόλυνση ξαναγράφουμε:

$$ppv = h * (d / vw)^b \quad [2.1]$$

για να υπολογίσουμε την ποσότητα w της εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα, έτσι ώστε, σε απόσταση d από το μέτωπο της ανατίναξης, η τιμή της ppv να μην υπερβαίνει το ασφαλές όριο που προβλέπεται από το διάγραμμα του ΚΜΛΕ.

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from sympy import *
ppv, h, d, w, b = symbols('ppv h d w b')
ppv_eq = ppv - h*(d/w**0.5)**b
w_eq = solve(ppv_eq, w)
print('w_eq =', w_eq)
```

Το αποτέλεσμα που εμφανίζεται στην οθόνη είναι:

```
w_eq = [d**2*(ppv/h)**(-2/b)]
```

ή σε μια πιο γνώριμη μαθηματική μορφή:

$$w = d^2 * (ppv / h)^{-2/b} \quad [11.3]$$

Με τη δεύτερη γραμμή κώδικα και με τη συνάρτηση symbols() ορίσαμε όλους τους τελεστές της Εξίσωσης [2.1] ως σύμβολα.

Παρατηρήστε ότι κάναμε μια πολλαπλή δήλωση. Εναλλακτικά, θα μπορούσαμε να είχαμε γράψει για κάθε τελεστή ξεχωριστή εντολή π.χ. ppv = symbols('ppv'), h = symbols('h') κοκ. Επίσης, αντί των

κενών διαστημάτων μεταξύ των συμβόλων στη συνάρτηση `symbols('ppv h d w b')` θα μπορούσαμε να είχαμε βάλει κόμματα, όπως `symbols('ppv, h, d, w, b')`.

Στην επόμενη σειρά κώδικα γράφουμε την Εξίσωση [2.1] σε μορφή $f(x, y, z, \dots) = 0$:

$$ppv - h * (d / \sqrt{w})^b = 0$$

και την εκχωρούμε στη μεταβλητή `ppv_eq`.

Στην τέταρτη γραμμή του κώδικα με την εντολή `w_eq = solve(ppv_eq, w)` λύνουμε την εξίσωση `ppv_eq` ως προς `w` και εκχωρούμε την επίλυση στη μεταβλητή `w_eq`.

Τέλος, εμφανίζουμε την εξίσωση που υπολογίζει την ποσότητα την εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα (`w`).

Εάν τώρα θέλουμε να εφαρμόσουμε την εξίσωση [11.3] για την επίλυση ενός συγκεκριμένου προβλήματος, όπως για παράδειγμα:

Ποια είναι η μέγιστη ποσότητα της εκρηκτικής ύλης που μπορεί να εκραγεί ταυτόχρονα σε ένα μέτωπο ανατίναξης τεχνικού έργου, το οποίο απέχει απόσταση 100 m από ένα σπίτι;

Στον κώδικα με τον οποίο διαμορφώσαμε την εξίσωση [11.3] προσθέτουμε τις παρακάτω γραμμές:

```
weight_expl = N(w_eq[0].subs({'h':1730, 'd':100, 'b':-1.60, 'ppv': 5}))
print(weight_expl)
print('Weight of Explosives (kg) = {:.2f}'.format(weight_expl))
```

Με την πρώτη γραμμή κώδικα κάνουμε ταυτόχρονα τρεις δουλειές:

1. Δίνουμε τις τιμές-δεδομένα του προβλήματος στις παραμέτρους `d` (από τα δεδομένα του προβλήματος), `h` και `b` (από τον Πίνακα 2.1) και `ppv` (από τον Πίνακα και Διάγραμμα της Εικόνας 2.1), με τη βοήθεια της συνάρτησης αντικατάστασης `subs()` της SymPy:

```
w_eq[0].subs({'h':1730, 'd':100, 'b':-1.60, 'ppv': 5})
```

η οποία αντικαθιστά τα σύμβολα `h`, `d`, `b` και `ppv` της `w_eq[0]` με τις αντίστοιχες τιμές τους.

Παρατηρήστε ότι η μεταβλητή `w_eq`, δηλαδή η Εξίσωση [11.3], δίνεται από τη SymPy ως λίστα με ένα (μοναδικό) στοιχείο και επομένως η αναφορά σε αυτήν πρέπει να γίνει με τον δείκτη αναφοράς στο πρώτο στοιχείο λίστας, δηλαδή ως `w_eq[0]`.

2. Με τη συνάρτηση `N()` της SymPy υπολογίζεται η αριθμητική τιμή της Εξίσωσης [11.3] μετά την αντικατάσταση με τις τιμές των δεδομένων του προβλήματος, υπό μορφή δεκαδικού αριθμού.
3. Εκχωρούμε την αριθμητική τιμή στη μεταβλητή `weight_expl`.

Η δεύτερη γραμμή κώδικα εμφανίζει στην οθόνη την τιμή της μεταβλητής `weight_expl` με την ακρίβεια υπολογισμού της SymPy: 6.70123543816216.

Τέλος, η τελευταία γραμμή κώδικα εμφανίζει στην οθόνη την τιμή της μεταβλητής `weight_expl` φορμαρισμένη για ακρίβεια δύο δεκαδικών ψηφίων:

Weight of Explosives (kg) = 6.70

Σημείωση: Με βάση τον πίνακα και το διάγραμμα μέγιστων επιτρεπόμενων τιμών της ταχύτητας δόνησης σωματιδίων του εδάφους σε συνάρτηση με τη συχνότητα, για διάφορα είδη κατασκευών του ΚΜΛΕ (Εικόνα 2.1), η μέγιστη επιτρεπόμενη τιμή της `ppv` για *Συνήθεις κατασκευές οικιών* κυμαίνεται από 5 – 20 mm/s ανάλογα με την τιμή της συχνότητας του κυματοπαλμού της δόνησης.

Τη συχνότητα, σε αντίθεση με την `ppv`, δεν μπορούμε να την εκτιμήσουμε με τη βοήθεια κάποιας σχέσης, αλλά μόνο να τη μετρήσουμε με ειδικά όργανα που ονομάζονται *δονησιογράφοι* (Blast Vibration Monitors).

Όταν σχεδιάζουμε μια ανατίναξη και θέλουμε να εκτιμήσουμε εάν η τιμή της αναμενόμενης p_{rn} δεν υπερβαίνει τα όρια που θέτει ο ΚΜΛΕ, θεωρούμε ότι η τιμή της συχνότητας θα βρίσκεται στη *δυσμενή περιοχή* των 1-10 Hz. Έτσι εξηγείται η τιμή των 5 mm/s που δώσαμε στην παράμετρο p_{rn} της Εξίσωσης [11.3].

11.2.2 Συστήματα εξισώσεων

Έστω ότι θέλουμε να λύσουμε το παρακάτω σύστημα εξισώσεων:

$$\begin{aligned} a + b + c &= 100 \\ 2 * a - 3 * b &= 10 \\ a + c &= 25 \end{aligned} \quad [11.4]$$

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from sympy import *
a, b, c = symbols('a b c')
eq3 = a + b + c - 100
eq4 = 2*a - 3*b - 10
eq5 = a + c - 25
result = solve([eq3, eq4, eq5], [a, b, c])
print('Result =', result)
```

Το αποτέλεσμα που εμφανίζεται στην οθόνη, ως λύση του συστήματος εξισώσεων [11.4], είναι:

Result = {a: 235/2, b: 75, c: -185/2}

Κατά τα γνωστά, γράψαμε στον κώδικα τις εξισώσεις του συστήματος υπό μορφή $f(\dots) = 0$ και στη συνέχεια με τη βοήθεια της συνάρτησης `solve([eq3, eq4, eq5], [a, b, c])` λύσαμε ταυτόχρονα τις τρεις εξισώσεις για τους αγνώστους a , b , c .

Σημειώστε ότι για να επιλυθεί ένα σύστημα εξισώσεων με τη βοήθεια της SymPy, δεν είναι απαραίτητο όλες οι εξισώσεις του να είναι γραμμικές.

Για παράδειγμα, οι λύσεις που δίνει η SymPy κατά την επίλυση του συστήματος εξισώσεων:

$$\begin{aligned} z^2 + w &= 10 \\ z + w^2 &= 100 \end{aligned} \quad [11.5]$$

είναι:

```
result = [{z: 0, w: 10}, {z: -(-40/3 + (-1/2 - sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3) + 400/(9*(-1/2 - sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3)))*(20/3 + (-1/2 - sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3) + 400/(9*(-1/2 - sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3))), w: -10/3 + (-1/2 - sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3) + 400/(9*(-1/2 - sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3))}, {z: -(-40/3 + 400/(9*(-1/2 + sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3)) + (-1/2 + sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3)))*(20/3 + 400/(9*(-1/2 + sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3)) + (-1/2 + sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3)), w: -10/3 + 400/(9*(-1/2 + sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3)) + (-1/2 + sqrt(3)*I/2)*(15973/54 +
sqrt(95919)*I/18)**(1/3))}, {z: -(-40/3 + 400/(9*(15973/54 +
sqrt(95919)*I/18)**(1/3)) + (15973/54 + sqrt(95919)*I/18)**(1/3))*(20/3 +
400/(9*(15973/54 + sqrt(95919)*I/18)**(1/3)) + (15973/54 +
sqrt(95919)*I/18)**(1/3)), w: -10/3 + 400/(9*(15973/54 +
sqrt(95919)*I/18)**(1/3)) + (15973/54 + sqrt(95919)*I/18)**(1/3))}]
```

Σημείωση: Τα σύμβολα των z και w έχουν μαρκιαστεί με κίτρινο χρώμα από τον συγγραφέα για να διακρίνονται εύκολα.

Εάν η SymPy δεν μπορεί να βρει λύση-ρίζα σε μια εξίσωση ή σε ένα σύστημα εξισώσεων, τότε ως αποτέλεσμα δίνει μια κενή λίστα, ήτοι `[]`.

11.2.3 Παράγωγοι συναρτήσεων

Από τη Φυσική γνωρίζουμε ότι στην ευθύγραμμη κίνηση ενός σώματος η μετατόπιση είναι συνάρτηση του χρόνου και δίνεται από τη σχέση:

$$s(t) = v_0 * t + 0.5 * a * t^2 \quad [11.6]$$

όπου:

s : η μετατόπιση

v_0 : η αρχική ταχύτητα

a : η επιτάχυνση

t : ο χρόνος κίνησης

Επίσης, γνωρίζουμε ότι η *πρώτη παράγωγος* της συνάρτησης [11.6] μας δίνει την ταχύτητα και η *δεύτερη παράγωγος* την επιτάχυνση του κινούμενου σώματος.

Για να υπολογίσουμε τις παραγώγους αυτές με τη βοήθεια της SymPy γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from sympy import *
v0, t, a = symbols('v0 t a')
s = v0*t + 0.5*a*t**2
vel = diff(s, t)
acc = diff(s, t, 2)
print('Velocity =', vel)
print('Acceleration =', acc)
```

Το αποτέλεσμα που εμφανίζεται στην οθόνη είναι:

```
Velocity = 1.0*a*t + v0
Acceleration = 1.0*a
```

Στη δεύτερη γραμμή κώδικα και με τη βοήθεια της συνάρτησης `symbols()` ορίζουμε τα σύμβολα που χρησιμοποιούμε για να γράψουμε, στην τρίτη σειρά κώδικα, τη συνάρτηση [11.6], την οποία και εκχωρούμε στη μεταβλητή `s`.

Στην τέταρτη σειρά κώδικα με τη βοήθεια της συνάρτησης `diff()` της SymPy κάνουμε *παραγωγή* (differentiation) της συνάρτησης `s` ως προς `t` και εκχωρούμε την πρώτη παράγωγό της στη μεταβλητή `vel`.

Στην επόμενη σειρά κώδικα, με τον ίδιο τρόπο, υπολογίζουμε τη *δεύτερη παράγωγο* της συνάρτησης `s` ως προς `t` και την εκχωρούμε στη μεταβλητή `acc`.

Παρατηρήστε ότι για να πάρουμε τη δεύτερη παράγωγο στα ορίσματα της συνάρτησης `diff()` έχουμε συμπεριλάβει τον αριθμό 2, ενώ για την πρώτη παράγωγο ο αριθμός 1 παραλείπεται γιατί είναι η default τιμή της συνάρτησης.

Παράδειγμα:

Κατά τη διάρκεια της πρακτικής άσκησης, ένας σπουδαστής χρονομέτρησε την κίνηση ενός φορτωμένου χωματουργικού αυτοκινήτου (ΧΑ) από το σημείο φόρτωσης και κατά μήκος μιας ανηφορικής διαδρομής, και διαπίστωσε ότι η θέση του ΧΑ από το σημείο φόρτωσης, σε m, και για όλο το μήκος της διαδρομής, δίνεται ως συνάρτηση του χρόνου, σε min, από τη συνάρτηση:

$$x(t) = 1.2 * t^3 - 1.3 * t^{2.3} + 2 * t \quad [11.7]$$

Στην εργασία της πρακτικής έγραψε τον παρακάτω κώδικα με τον οποίο:

1. βρήκε τους τύπους που υπολογίζουν την ταχύτητα και την επιτάχυνση του ΧΑ ως συνάρτηση του χρόνου και
2. υπολόγισε τη θέση, την ταχύτητα και την επιτάχυνση 10 min μετά την εκκίνηση του ΧΑ από το σημείο φόρτωσης.

```
from sympy import *
t = symbols('t')
x = 1.2*t**3 - 1.3*t**2.3 + 2*t
vel_t = diff(x, t)
print('vel_t =', vel_t)
acc_t = diff(x, t, 2)
print('acc_t =', acc_t)
x_10 = N(x.subs({'t':10}))
print('Position (m) =', x_10)
vel_10 = N(vel_t.subs({'t':10}))
print('Velocity (m/min) =', vel_10)
acc_10 = N(acc_t.subs({'t':10}))
print('Acceleration (m/min2) =', acc_10)
```

```
vel_t = 3.6*t**2 - 2.99*t**1.3 + 2
acc_t = -3.887*t**0.3 + 7.2*t
Position (m) = 960.615899054046
Velocity (m/min) = 302.341656782431
Acceleration (m/min2) = 64.2444153817160
```

Ο σπουδαστής ήταν ενθουσιασμένος για το επίτευγμά του και περίμενε υψηλό βαθμό. Ο καθηγητής που διάβασε την εργασία έγραψε στις παρατηρήσεις του: «...το να υπολογίζεις-παρουσιάζεις τη θέση, ταχύτητα και επιτάχυνση ενός ΧΑ με ακρίβεια 12 δεκαδικών ψηφίων του μέτρου, δείχνει έλλειψη αντίληψης Μηχανικού!».

Πράγματι: 0.000000000001 m = 0.001 nm (nanometer) = 0.01 Å (Angstrom)...

Ο σπουδαστής κατάλαβε το λάθος του και εκτίμησε τη σημασία που έχει στην παρουσίαση των αποτελεσμάτων των υπολογισμών, που γίνονται με την Python, η χρήση της εντολής `print()` σε συνδυασμό με τη συνάρτηση `format()` (βλέπε παράγραφο 04.4.1 *Φορμαρισμένα αποτελέσματα*).

11.2.4 Ολοκληρώματα συναρτήσεων

Η βιβλιοθήκη SymPy μας δίνει τη δυνατότητα να υπολογίζουμε τα *αόριστα* και *ορισμένα ολοκληρώματα* συναρτήσεων με τη βοήθεια της συνάρτησης `integrate()`.

Στο τέλος του κώδικα του παραδείγματος της πρακτικής άσκησης του σπουδαστή, που είδαμε στην παράγραφο 11.2.3 *Παράγωγοι συναρτήσεων*, προσθέτουμε τον παρακάτω κώδικα:

```
f_t = 3.6*t**2 - 2.99*t**1.3 + 2
f_t_integral = integrate(f_t, t)
print('Integral of f(t) :', f_t_integral)
```

Εδώ, αφού ορίσαμε τη συνάρτηση $f(t) = 3.6 \cdot t^2 - 2.99 \cdot t^{1.3} + 2$, στην επόμενη γραμμή κώδικα ζητήσαμε από τη SymPy, κάνοντας χρήση της συνάρτησης `integrate()`, να κάνει την ολοκλήρωση της `f_t` ως προς `t` και το ολοκλήρωμα να το εκχωρήσει στη μεταβλητή `f_t_integral`.

Το ολοκλήρωμα εμφανίζεται στην οθόνη ως:

Integral of $f(t)$: $1.2*t3 + 2*t - 1.3*t**2.3$**

Παρατήρηση: Επειδή η συνάρτηση $f(t)$ είναι ίδια με τη συνάρτηση vel_t του αποτελέσματος του παραδείγματος της πρακτικής άσκησης που υπολογίζει την ταχύτητα του ΧΑ, έπεται ότι το ολοκλήρωμά της είναι ίδιο με τη Συνάρτηση [11.7] που δίνει τη θέση του ΧΑ, απλά είναι γραμμένο με διαφορετική σειρά.

Έτσι, επιβεβαιώνεται και το θεμελιώδες θεώρημα του *απειροστικού λογισμού* που δηλώνει ότι η ολοκλήρωση και η παραγωγή είναι αντίστροφες πράξεις.

Σημειώστε, επίσης, ότι η συνάρτηση `integrate()` δεν προσθέτει τη σταθερά ολοκλήρωσης στα ολοκληρώματα που υπολογίζει.

Με παρόμοιο τρόπο υπολογίζουμε και το ορισμένο ολοκλήρωμα μιας συνάρτησης.

Συνεχίζοντας, στον κώδικα προσθέτουμε τις γραμμές εντολών:

```
dist = integrate(f_t, (t, 0, 10))
print('Distance (m) = {:.0f}'.format(dist))
```

προκειμένου να υπολογίσουμε το ορισμένο ολοκλήρωμα $\int_0^{10} f(t)$.

Αυτό το επιτυγχάνουμε με την πρώτη γραμμή κώδικα, όπου ζητάμε να υπολογιστεί το ολοκλήρωμα της f_t ως προς t με κάτω όριο το 0 και άνω όριο το 10 και να γίνει εκχώρηση του αποτελέσματος στη μεταβλητή `dist`. Το αποτέλεσμα εμφανίζεται στην οθόνη (με αντίληψη Μηχανικού...) ως:

Distance (m) = 961

Ο καθηγητής της πρακτικής άσκησης εκτίμησε, τώρα, ιδιαίτερα την προσπάθεια του σπουδαστή, αντέγραψε τον κώδικα στον υπολογιστή του και προσέθεσε τις παρακάτω γραμμές:

```
dist_0_1 = integrate(f_t, (t, 0, 1))
print('Distance from 0 - 1 min (m) = {:.0f}'.format(dist_0_1))
dist_9_10 = integrate(f_t, (t, 9, 10))
print('Distance from 9 - 10 min (m) = {:.0f}'.format(dist_9_10))
```

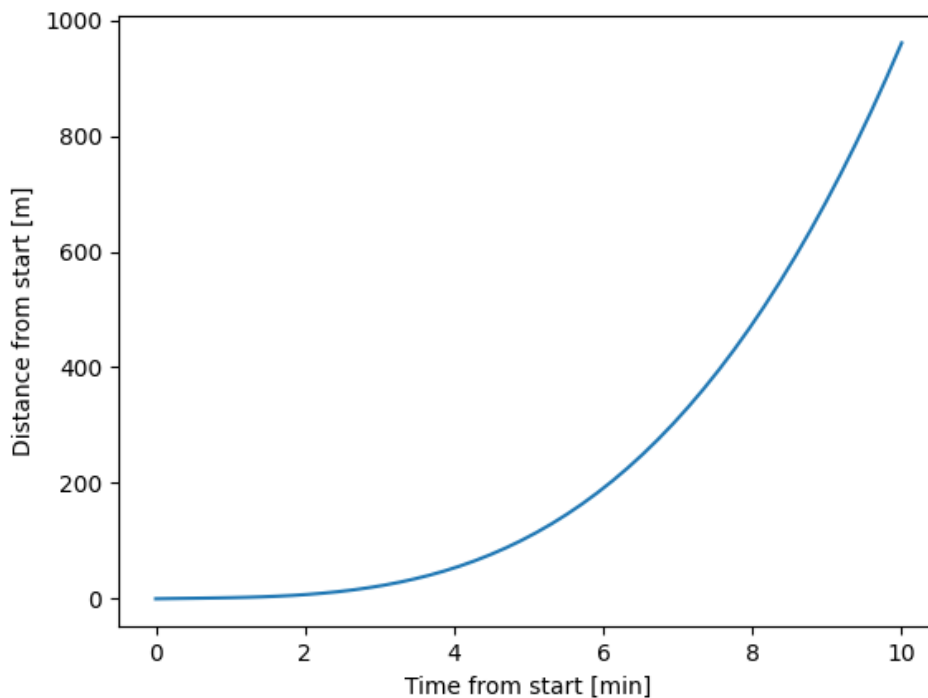
προκειμένου να υπολογίσει την απόσταση που διάνυσε το ΧΑ κατά το πρώτο min της κίνησής του και μεταξύ του 9^{ου} και 10^{ου} min κίνησης. Το αποτέλεσμα που είδε στην οθόνη ήταν:

Distance from 0 - 1 min (m) = 2
Distance from 9 - 10 min (m) = 271

Τα νούμερα του φάνηκαν *περίεργα* και επειδή ως Μηχανικός αντιλαμβανόταν καλύτερα την εξέλιξη των φαινομένων με τη βοήθεια διαγραμμάτων, προσέθεσε τον παρακάτω κώδικα:

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 10)
y = lambdify([t], f_t_integral, "numpy")
plt.plot(x, y(x))
plt.xlabel('Time from start [min]')
plt.ylabel('Distance from start [m]')
plt.savefig('time_vs_distance.png')
plt.show()
```

για να εμφανίσει στην οθόνη το διάγραμμα μεταβολής της θέσης του ΧΑ συναρτήσει του χρόνου, για το διάστημα 0 – 10 min, το οποίο φαίνεται στην Εικόνα 11.1.



Εικόνα 11.1. Μεταβολή της θέσης του χωματουργικού αυτοκινήτου συναρτήσει του χρόνου κίνησης, από τη θέση φόρτωσης.

Ο κώδικας για τη δημιουργία της γραφικής παράστασης είναι παρόμοιος με εκείνον που είδαμε στην παράγραφο 06.3 *Διαμορφώνοντας το διάγραμμα*, με τις παρακάτω διαφοροποιήσεις:

Με την εντολή:

```
y = lambdify([t], f_t_integral, "numpy")
```

και τη βοήθεια της συνάρτησης `lambdify()` της SymPy μετατρέπουμε τη σχέση `f_t_integral`, δηλαδή την $1.2 \cdot t^3 + 2 \cdot t - 1.3 \cdot t^{2.3}$, η οποία είναι γραμμένη από τη SymPy με τη μεταβλητή `t` υπό μορφή συμβόλου, σε ισοδύναμη σχέση κατάλληλη για να κάνουμε αριθμητικούς υπολογισμούς με συναρτήσεις της βιβλιοθήκης NumPy.

Δηλαδή, έχουμε μια νέα ισοδύναμη σχέση που έχει τα χαρακτηριστικά της συνάρτησης `lambda` (βλέπε παράγραφο 09.5 *Συναρτήσεις lambda*):

```
lambda t, f_t_integral: 1.2*t**3 + 2*t - 1.3*t**2.3
```

και με αυτή τη μορφή, που δεν είναι *ορατή* σε εμάς, εκχωρείται στη μεταβλητή `y`.

Στη συνέχεια, η εντολή `plt.plot(x, y(x))` της βιβλιοθήκης Matplotlib κατασκευάζει τη γραφική παράσταση των τιμών του `x` ως προς τις τιμές του `y(x)`, δηλαδή αντικαθιστώντας το `t` με τις τιμές του `x`.

Εάν δεν είχαμε κάνει την παραπάνω μετατροπή, χρησιμοποιώντας τη συνάρτηση `lambdify()` και γράφαμε:

```
y = f_t_integral
plt.plot(x, y)
```

ο interpreter της Python θα σας έβγαζε `ValueError`.

Από τα παραπάνω, δηλαδή τα αποτελέσματα της ολοκλήρωσης για το πρώτο και το τελευταίο λεπτό της κίνησης του ΧΑ καθώς και τη μορφή της καμπύλης μεταβολής της θέσης του ΧΑ συναρτήσει του χρόνου κίνησης, προκύπτει ότι μάλλον ο σπουδαστής έχει κάνει κάποιο λάθος στην επεξεργασία των χρονομετρήσεων του, των οποίων οι τιμές δεν προσαρμόζονται καλά στη Συνάρτηση [11.7].

Το θέμα της προσαρμογής των τιμών μετρήσεων, που κάνουμε στο εργαστήριο ή στο εργοτάξιο, σε καμπύλες συναρτήσεων, θα το παρουσιάσουμε στο Κεφάλαιο 12 *Επεξεργασία δεδομένων* και ιδιαίτερα στην παράγραφο 12.4 *Καμπύλη προσαρμογής*.

Το script του κώδικα της πρακτικής άσκησης του σπουδαστή περιλαμβάνεται στο website του βιβλίου στο αρχείο `Chapter_11_Praktiki_Askisi.py`

11.2.5 Συνήθειες διαφορικές εξισώσεις

Μια διαφορική εξίσωση συσχετίζει μια άγνωστη συνάρτηση $f(x)$ με την παράγωγό της $f'(x)$:

$$f'(x) = f(x) \quad \text{ή} \quad df(x)/dx = f(x) \quad [11.8]$$

Εάν η συνάρτηση f είναι συνάρτηση μιας μεταβλητής, τότε έχουμε μια *συνήθη διαφορική εξίσωση*.

Για την επίλυση των συνήθων διαφορικών εξισώσεων ή και συστημάτων συνήθων διαφορικών εξισώσεων, χρησιμοποιούμε τη συνάρτηση `dsolve()` της βιβλιοθήκης SymPy.

Ο παρακάτω κώδικας επιλύει τη διαφορική Εξίσωση [11.8]:

```
from sympy import *
x = symbols('x')
f = Function('f')
f_x = dsolve(f(x) - diff(f(x), x), f(x))
print('f(x) :', f_x)
```

Στην τρίτη γραμμή κώδικα με τη συνάρτηση `Function()` της SymPy ορίζουμε ότι το σύμβολο f είναι μια μαθηματική συνάρτηση.

Στην επόμενη γραμμή κώδικα εφαρμόζουμε τη συνάρτηση `dsolve()` της βιβλιοθήκης SymPy επί της Εξίσωσης [11.8], την οποία γράφουμε υπό τη μορφή: $f(x) - df(x)/dx = 0$, και τη λύση την εκχωρούμε στη μεταβλητή `f_x` που στη συνέχεια εμφανίζουμε στην οθόνη:

`f(x) : Eq(f(x), C1*exp(x))`

Η λύση δηλαδή της διαφορικής Εξίσωσης [11.8] είναι μια παραμετρική οικογένεια συναρτήσεων (*γενική λύση της διαφορικής εξίσωσης*) της μορφής:

$$f(x) = C1 * e^x \quad [11.9]$$

όπου $C1$ μια σταθερά.

Συχνά στα προβλήματα διαφορικών εξισώσεων έχουμε κάποιες αρχικές συνθήκες και στην περίπτωση αυτή μπορούμε να προσδιορίσουμε τις παραμέτρους-σταθερές της γενικής λύσης.

Έτσι, εάν στην περίπτωση της διαφορικής Εξίσωσης [11.8] η αρχική συνθήκη είναι $f(0) = 1$ και εισάγουμε τη συνθήκη αυτή στον κώδικά μας, προσθέτοντας τις παρακάτω γραμμές κώδικα:

```
f_x = dsolve(f(x) - diff(f(x), x), f(x), ics={f(0): 1})
print('f(x) for f(0) = 1 :', f_x)
```

το αποτέλεσμα που εμφανίζεται στην οθόνη είναι:

`f(x) for f(0) = 1 : Eq(f(x), exp(x))`

δηλαδή:

$$f(x) = e^x \quad [11.10]$$

Η αρχική συνθήκη $f(0) = 1$ εισάγεται στη συνάρτηση `dsolve()` μέσω της παραμέτρου `ics={f(0): 1}` υπό τη μορφή ενός στοιχείου λεξικού.

Για την καλύτερη κατανόηση του τρόπου με τον οποίο μπορούμε να λύσουμε προβλήματα διαφορικών εξισώσεων με τη βοήθεια της SymPy, δίνεται το παρακάτω παράδειγμα.

Παράδειγμα:

Ένας σιδηρουργός πρόκειται να χρησιμοποιήσει τεμάχια χάλυβα, συγκεκριμένων διαστάσεων, για τη διαμόρφωση με τη μέθοδο της *σφυρηλάτησης* (forging) καλλιτεχνικών αντικειμένων. Για να βελτιστοποιήσει την παραγωγή του, ζήτησε από έναν Μεταλλειολόγο να του προσδιορίσει τον χρόνο που θα έχει στη διάθεσή του για να σφυρηλατήσει καθένα αντικείμενο, από τη χρονική στιγμή που θα τα βγάξει πυρακτωμένα από το καμίνι.

Ο Μεταλλειολόγος είδε, από βιβλιογραφικά δεδομένα, ότι για τον συγκεκριμένο τύπο χάλυβα, η σφυρηλάτηση γίνεται ιδανικά μεταξύ των θερμοκρασιών $T_{oven} = 1350\text{ }^{\circ}\text{C}$ και $T_{forge} = 1100\text{ }^{\circ}\text{C}$.

Στη συνέχεια, έκανε μια δοκιμή. Θέρμανε ένα τεμάχιο χάλυβα στη θερμοκρασία των $1350\text{ }^{\circ}\text{C}$ και το άφησε να ψυχθεί στον χώρο του σιδηρουργείου. Μετά από χρόνο $t_{test} = 2\text{ min}$, μέτρησε τη θερμοκρασία του τεμαχίου με ένα οπτικό πυρόμετρο και διαπίστωσε ότι ήταν $T_{test} = 1280\text{ }^{\circ}\text{C}$. Επίσης, σημείωσε και τη θερμοκρασία περιβάλλοντος στον χώρο του σιδηρουργείου, που ήταν $T_{room} = 25\text{ }^{\circ}\text{C}$.

Σύμφωνα με τον *Νόμο Ψύξης του Newton* (Newton's Law of Cooling) [Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Newton%27s_law_of_cooling] ισχύει ότι:

$$dT(t)/dt = k * (T(t) - T_{room}) \quad [11.11]$$

όπου k είναι ο συντελεστής μεταφοράς της θερμότητας.

Για την επίλυση της διαφορικής Εξίσωσης [11.11] με τα δεδομένα του παραδείγματος έγραψε τον παρακάτω κώδικα:

```
from sympy import *
# Data
T_oven = 1350 # oC
T_forge = 1100 # oC
T_room = 25 # oC
t_test = 2 # min
T_test = 1280 # oC
t, k = symbols('t, k')
T = Function('T')
# Newton's Cooling Equation
diff_eq = diff(T(t), t) - k*(T(t) - T_room)
T_eq = dsolve(diff_eq, T(t), ics={T(0): T_oven})
print('T_equation :', T_eq)
```

και στην οθόνη εμφανίστηκε η λύση της:

```
T_equation : Eq(T(t), 1325*exp(k*t) + 25)
```

δηλαδή:

$$T(t) = 1325 * e^{k \cdot t} + 25 \quad [11.12]$$

Για να υπολογίσει τον συντελεστή μεταφοράς της θερμότητας k , με βάση τις μετρήσεις της δοκιμής που έκανε, προσέθεσε τον παρακάτω κώδικα:

```
# Calculation of constant k
k_eq = T_eq.rhs.subs(t, t_test) - T_test
k_solution = solve(k_eq, k)
print('k_solution =', k_solution)
```

Με την πρώτη γραμμή κώδικα απέδωσε στη μεταβλητή του χρόνου t την τιμή της δοκιμής t_{test} , στο δεξιό μέρος (rhs : right-hand-side) της Εξίσωσης T_{eq} [11.12], και επίσης αντικατέστησε το $T(t)$ με την τιμή της δοκιμής T_{test} .

Έτσι, η εξίσωση [11.12] πήρε τώρα τη μορφή:

$$0 = 1325 * e^{k \cdot 2} + 25 - 1280 \quad [11.13]$$

που είναι κατάλληλη για την επίλυσή της, ως προς τον άγνωστο k , από την επόμενη γραμμή κώδικα.

Στην οθόνη εμφανίζονται οι λύσεις-ρίζες της εξίσωσης:

```
k_solution = [-log(265) + log(66515)/2,
              -log(265) + log(66515)/2 + I*pi]
```

Η δεύτερη ρίζα, επειδή είναι μιγαδικός αριθμός, απορρίπτεται και η τιμή του k δίνεται από το πρώτο στοιχείο της λίστας των ριζών:

```
k_solution[0] = -log(265) + log(66515)/2
```

Τώρα πλέον ο Μεταλλειολόγος μπορεί να δώσει την απάντηση στο ερώτημα του σιδηρουργού, προσθέτοντας τον παρακάτω κώδικα:

```
# Equation of cooling for the piece of steel
T_steel_eq = T_eq.rhs.subs(k, k_solution[0])
print('T_steel_eq:', T_steel_eq)
forge_time = solve(T_steel_eq - T_forge, t)
print('Time for Forging (min) = {:.2f}'.format(forge_time[0].evalf()))
```

Στην οθόνη εμφανίζονται η εξίσωση ψύξης του συγκεκριμένου τεμαχίου χάλυβα και ο χρόνος που έχει στη διάθεσή του ο σιδηρουργός για να το σφυρηλατήσει:

```
T_steel_eq : 1325*exp(t*(-log(265) + log(66515)/2)) + 25
Time for Forging (min) = 7.70
```

Στην τελευταία γραμμή κώδικα και εντός του `format()` χρησιμοποιήσαμε τη συνάρτηση `evalf()` της SymPy, η οποία μετατρέπει την ακριβή τιμή της ρίζας της εξίσωσης σε έναν κατά προσέγγιση δεκαδικό αριθμό. Είναι, δηλαδή, ανάλογη της συνάρτησης `N()` της SymPy, που είδαμε στην παράγραφο 11.2.1 Εξισώσεις με έναν άγνωστο.

Εάν αντικαταστήσετε την τελευταία γραμμή κώδικα με τη γραμμή:

```
print('Time for Forging (min) =', forge_time[0])
```

η SymPy θα σας δώσει την ακριβή ρίζα-λύση του προβλήματος:

```
Time for Forging (min) = -log((43/53)**(2/log(265/251)))
```

την οποία ο σιδηρουργός σίγουρα δεν πρόκειται να εκτιμήσει...

Σημείωση: Στην πράξη ο σιδηρουργός θα έχει στη διάθεσή του για σφυρηλάτηση χρόνο μεγαλύτερο των 7.70 min, γιατί η ψύξη του τεμαχίου χάλυβα κατά την εργασία της σφυρηλάτησης θα είναι πιο αργή. Αυτό οφείλεται στη θερμότητα που αναπτύσσεται από την τριβή του τεμαχίου μεταξύ σφύρας και άκμονος.

Το script του κώδικα του παραδείγματος περιλαμβάνεται στο website του βιβλίου στο αρχείο: `Chapter_11_Newtons_Cooling_Law_Forging_sympy.py`

11.3 Επίλυση εξισώσεων: βιβλιοθήκη SciPy

Η SciPy (Scientific Python) είναι μια εξωτερική βιβλιοθήκη της Python, η οποία μαζί με τις βιβλιοθήκες NumPy και Matplotlib, που ήδη έχουμε γνωρίσει, και την Pandas, που θα δούμε στην παράγραφο 12.2 Η βιβλιοθήκη Pandas, ανήκουν στην οικογένεια βιβλιοθηκών του SciPy ecosystem (<https://scipy.org>).

Όντας εξωτερική βιβλιοθήκη, πρέπει να την εγκαταστήσουμε στον υπολογιστή μας κατά τα γνωστά:

```
pip install scipy
```

Η SciPy περιλαμβάνει μια συλλογή από συναρτήσεις που είναι χρήσιμες για την επίλυση προβλημάτων που αντιμετωπίζουν επιστήμονες και μηχανικοί.

Η επίλυση των εξισώσεων με τη χρήση των συναρτήσεων της SciPy γίνεται με την εφαρμογή αλγορίθμων αριθμητικών μεθόδων και αριθμητικών προσεγγίσεων.

Στη συνέχεια, θα επιλύσουμε με τη βοήθεια της βιβλιοθήκης SciPy τις ίδιες εξισώσεις που επιλύσαμε στην παράγραφο 11.2 *Επίλυση εξισώσεων: βιβλιοθήκη SymPy* και για τον λόγο αυτό θα διατηρήσουμε κοινή την αρίθμηση των εξισώσεων. Κατ' αυτόν τον τρόπο, θα έχετε την ευκαιρία να δείτε και να συγκρίνετε τα *συν* και *πλην* των δύο βιβλιοθηκών.

11.3.1 Εξισώσεις με έναν άγνωστο

Έστω ότι θέλουμε να λύσουμε τη δευτεροβάθμια εξίσωση:

$$x^2 + 2x - 8 = 0 \quad [11.1]$$

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from scipy.optimize import fsolve
eq1 = lambda x: x**2 + 2*x - 8
solution_eq1 = fsolve(eq1, [-100, 100])
print('x =', solution_eq1)
```

Στην οθόνη θα εμφανιστεί το παρακάτω αποτέλεσμα:

```
x = [-4.  2.]
```

δηλαδή, οι δύο ρίζες της εξίσωσης, γραμμένες υπό τη μορφή array της NumPy (ndarray).

Ένα *ndarray* της NumPy είναι ένας μαθηματικός πίνακας (μητρώο, matrix) μίας ή περισσότερων διαστάσεων, ο οποίος περιέχει στοιχεία ίδιας κατηγορίας, π.χ. float, σε αντίθεση με μια λίστα η οποία μπορεί να περιέχει στοιχεία διαφόρων κατηγοριών, π.χ. float και string. Για περισσότερες πληροφορίες ανατρέξτε στην τεκμηρίωση στο:

<https://numpy.org/doc/stable/reference/arrays.ndarray.html#arrays-ndarray>

Με την πρώτη γραμμή κώδικα κάνουμε `import`, από την οικογένεια συναρτήσεων `optimize` της βιβλιοθήκης `scipy`, τη συνάρτηση `fsolve` στον κώδικά μας.

Στην επόμενη σειρά κώδικα γράφουμε την Εξίσωση [11.1] υπό τη μορφή μιας συνάρτησης `lambda` της Python και την εκχωρούμε στη μεταβλητή `eq1`.

Εναλλακτικά, αντί να χρησιμοποιήσουμε τη μορφή συνάρτησης `lambda` θα μπορούσαμε να ορίσουμε τη συνάρτηση `eq1` με τον κλασικό τρόπο ορισμού συναρτήσεων της Python, δηλαδή ως:

```
def eq1(x):
    return x**2 + 2*x - 8
```

Για να επιλύσουμε την εξίσωση χρησιμοποιούμε τη συνάρτηση `fsolve()` της SciPy, η οποία έχει δύο υποχρεωτικές παραμέτρους: την *εξίσωση* που πρέπει να είναι υπό τη μορφή μιας συνάρτησης $f(\dots) = 0$, και μια *αρχική εκτίμηση της ρίζας-λύσης* της εξίσωσης υπό τη μορφή μιας ndarray.

Είναι προφανές ότι, εάν η εξίσωση έχει περισσότερες της μίας ρίζες, τότε πρέπει να δοθούν εκτιμήσεις για όλες τις ρίζες της.

Η συνάρτηση `fsolve()` δέχεται και μια σειρά *προαιρετικών παραμέτρων* (optional parameters), τις οποίες δεν θα χρησιμοποιήσουμε εδώ, αλλά θα δεχθούμε τις default τιμές τους, και οι οποίες παρουσιάζονται στην τεκμηρίωση της συνάρτησης στο:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.fsolve.html>

Έτσι, στην τρίτη γραμμή κώδικα με την εντολή:

```
solution_eq1 = fsolve(eq1, [-100, 100])
```

λύνουμε την εξίσωση eq1 δίνοντας ως αρχική εκτίμηση για τις δύο ρίζες της δευτεροβάθμιας εξίσωσης τις τιμές -100 και 100, και εκχωρούμε τη λύση στη μεταβλητή solution_eq1.

Έστω ότι θέλουμε τώρα να λύσουμε την τριτοβάθμια εξίσωση (πολυώνυμο) :

$$x^3 + x^2 + x + 1 = 0 \quad [11.2]$$

Στον παραπάνω κώδικα προσθέτουμε τις γραμμές:

```
eq2 = lambda x: x**3 + x**2 + x + 1
solution_eq2 = fsolve(eq2, [-100, 1, 100])
print('x =', solution_eq2)
```

Η πρώτη γραμμή εκχωρεί την Εξίσωση [11.2], υπό τη μορφή μιας συνάρτησης lambda της Python, στη μεταβλητή eq2. Με τη δεύτερη γραμμή κώδικα λύνουμε την εξίσωση eq2 δίνοντας ως αρχική εκτίμηση για τις τρεις ρίζες της εξίσωσης τις τιμές [-100, 1, 100] και εκχωρούμε τις τιμές των ριζών στη μεταβλητή solution_eq2.

Η εντολή print() που ακολουθεί εμφανίζει στην οθόνη το αποτέλεσμα:

```
x = [-1. -1. -1.]
```

Παρατηρούμε ότι η συνάρτηση fsolve() της SciPy μας δίνει και για τις τρεις ρίζες ως τιμή τον πραγματικό αριθμό -1., ενώ εμείς γνωρίζουμε, από τα μαθηματικά αλλά και από την επίλυση της εξίσωσης [11.2], που κάναμε με τη βοήθεια της SymPy στην παράγραφο 11.2.1 Εξισώσεις με έναν άγνωστο, ότι υπάρχουν και άλλες δύο ρίζες που είναι η φανταστική μονάδα: $\pm i$.

Ως συμπέρασμα, η συνάρτηση fsolve() της SciPy μας δίνει πάντα πραγματικές τιμές κατά την επίλυση των εξισώσεων, άρα είναι πιο κοντά στην *αντίληψη του Μηχανικού*.

Στη συνέχεια, θα υπολογίσουμε την ποσότητα w της εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα, έτσι ώστε σε απόσταση d από το μέτωπο της ανατίναξης η τιμή της ppv να μην υπερβαίνει το ασφαλές όριο που προβλέπεται από το διάγραμμα του ΚΜΛΕ, χρησιμοποιώντας την εξίσωση:

$$ppv = h * (d / \sqrt{w})^b \quad [2.1]$$

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from scipy.optimize import fsolve
h = 1730
d = 100
b = -1.60
ppv = 5
w_eq = lambda w: h*(d/w**0.5)**b - ppv
solution_w_eq = fsolve(w_eq, [10])
print('Weight of Explosives (kg) =', solution_w_eq)
print('Weight of Explosives (kg) = {:.2f}'.format(solution_w_eq[0]))
```

Το αποτέλεσμα που εμφανίζεται στην οθόνη, σε αφορμάριστη και φορμαρισμένη μορφή είναι:

```
Weight of Explosives (kg) = [6.70123544]
Weight of Explosives (kg) = 6.70
```

Καταρχάς, αποδώσαμε στις μεταβλητές h, d, b, και ppv τις ίδιες αριθμητικές τιμές που χρησιμοποιήσαμε όταν επιλύσαμε την εξίσωση [2.1] με τη βοήθεια της βιβλιοθήκης SymPy.

Στη συνέχεια, γράψαμε την εξίσωση [2.1] ως:

$$0 = h * (d / \sqrt{w})^b - ppv$$

υπό τη μορφή μιας συνάρτησης `lambda`, με άγνωστη μεταβλητή της συνάρτησης την `w`, και την εκχωρήσαμε στη μεταβλητή `w_eq`.

Με τη γραμμή κώδικα `solution_w_eq = fsolve(w_eq, [10])` λύνουμε την εξίσωση δίνοντας ως αρχική εκτίμηση της λύσης την τιμή 10 και εκχωρούμε την επίλυση στη μεταβλητή `solution_w_eq`.

Εδώ θα πρέπει να σημειώσουμε ότι η συνάρτηση `fsolve()` έχει μια *ευαισθησία* σε ό,τι αφορά την επιλογή της αρχικής εκτίμησης της λύσης μιας εξίσωσης.

Εάν, για παράδειγμα, αντί της τιμής αρχικής εκτίμησης 10, δώσουμε την τιμή 100, δηλαδή γράψουμε:

```
solution_w_eq = fsolve(w_eq, [100])
```

στην οθόνη θα εμφανιστεί το παρακάτω μήνυμα από τη SciPy:

```
C:\Users\George N Panagiotou\venv\lib\site-  
packages\scipy\optimize\minpack.py:175:  
RuntimeWarning: The iteration is not making good progress, as measured by the  
improvement from the last ten iterations.  
warnings.warn(msg, RuntimeWarning)
```

και ως αποτέλεσμα της επίλυσης της εξίσωσης θα πάρουμε την τιμή της αρχικής εκτίμησης της λύσης που εμείς δώσαμε:

```
Weight of Explosives (kg) = [100.]  
Weight of Explosives (kg) = 100.00
```

που προφανώς είναι μια λάθος λύση!

Επομένως, χρειάζεται προσοχή στην επιλογή της τιμής αρχικής εκτίμησης της λύσης και βέβαια δεν πρέπει να ξεχνάμε ότι η *μέθοδος της δοκιμής-σφάλματος* είναι πάντα χρήσιμη!

11.3.2 Συστήματα εξισώσεων

Έστω ότι θέλουμε να λύσουμε το παρακάτω σύστημα εξισώσεων:

$$\begin{aligned} a + b + c &= 100 \\ 2 * a - 3 * b &= 10 \\ a + c &= 25 \end{aligned} \quad [11.4]$$

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from scipy.linalg import solve  
import numpy as np  
A = np.array([[1,1,1], [2,-3,0], [1,0,1]])  
B = np.array([100,10,25])  
result = solve(A, B)  
print('Result = ', result)
```

Το αποτέλεσμα που εμφανίζεται στην οθόνη είναι:

```
Result = [117.5  75. -92.5]
```

Για την επίλυση του συστήματος εξισώσεων θα χρησιμοποιήσουμε από την οικογένεια συναρτήσεων `linalg` της βιβλιοθήκης `scipy`, τη συνάρτηση `solve` την οποία και κάνουμε `import` στον κώδικά μας.

Επίσης, επειδή θα λύσουμε το σύστημά μας με τη *μέθοδο των οριζουσών*, τους πίνακες των οποίων θα πρέπει να εισάγουμε στον κώδικά μας υπό τη μορφή `array` της NumPy (`ndarray`), κάνουμε `import` και τη βιβλιοθήκη NumPy.

Η τρίτη γραμμή κώδικα εκχωρεί στη μεταβλητή `A` τις τιμές του πίνακα:

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 2 & -3 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

Η τέταρτη γραμμή κώδικα εκχωρεί στη μεταβλητή B τις τιμές του πίνακα:

$$B = \begin{pmatrix} 100 \\ 10 \\ 25 \end{pmatrix}$$

Δηλαδή, το προς επίλυση σύστημα έχει τώρα τη μορφή $A * x = B$, όπου:

$$x = \begin{pmatrix} a \\ b \\ c \end{pmatrix}$$

Τη λύση του συστήματος εξισώσεων επιτυγχάνει η πέμπτη γραμμή κώδικα με τη συνάρτηση `solve()` της SciPy, η οποία έχει ως παραμέτρους τις `array A` και `B`.

Εναλλακτικά, αντί της συνάρτησης `solve()` της SciPy θα μπορούσαμε να είχαμε χρησιμοποιήσει την αντίστοιχη συνάρτηση `solve()` της NumPy. Έτσι, εάν αντικαταστήσετε την πέμπτη γραμμή κώδικα με αυτή:

```
result = np.linalg.solve(A, B)
```

θα πάρετε το ίδιο αποτέλεσμα, ήτοι: `a=117.5 b=75. c=-92.5`

Η συνάρτηση `solve()` δέχεται και μια σειρά προαιρετικών παραμέτρων (optional parameters), οι οποίες παρουσιάζονται στην τεκμηρίωση της συνάρτησης στο:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.solve.html>

11.3.3 Παράγωγοι συναρτήσεων

Για να υπολογίσουμε τις παραγώγους μιας μαθηματικής συνάρτησης με τη βοήθεια της βιβλιοθήκης SciPy, θα χρησιμοποιήσουμε από την οικογένεια συναρτήσεων `misc` της βιβλιοθήκης `scipy`, τη συνάρτηση `derivative` την οποία και κάνουμε `import` στον κώδικά μας.

Ο αριθμητικός προσδιορισμός της παραγώγου γίνεται με τη μέθοδο της πεπερασμένης διαφοράς.

Για να δούμε τη χρήση της συνάρτησης `derivative()` θα πάρουμε το παράδειγμα με την κίνηση του χωματοургικού αυτοκινήτου της πρακτικής άσκησης του σπουδαστή, που επιλύσαμε προηγουμένως με χρήση της βιβλιοθήκης `SymPy` στο παράδειγμα της παραγράφου 11.2.3 *Παράγωγοι συναρτήσεων*, και θα υπολογίσουμε την ταχύτητα (1^η παράγωγος) και την επιτάχυνση (2^η παράγωγος) στον χρόνο 10 min μετά την εκκίνηση του ΧΑ από το σημείο φόρτωσης.

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from scipy.misc import derivative
x = lambda t: 1.2*t**3 - 1.3*t**2.3 + 2*t
vel_10 = derivative(x, 10., n=1, dx=1e-6)
print('Velocity (m/min) = {:.2f}'.format(vel_10))
acc_10 = derivative(x, 10., n=2, dx=1e-6)
print('Acceleration (m/min2) = {:.2f}'.format(acc_10))
```

Στη δεύτερη γραμμή κώδικα γράψαμε τη συνάρτηση [11.7] από το παράδειγμα της πρακτικής άσκησης, που δίνει τη θέση του ΧΑ σε m από το σημείο φόρτωσης, ως συνάρτηση του χρόνου t σε min, υπό τη μορφή μιας συνάρτησης `lambda`, με άγνωστη μεταβλητή το t, και την εκχωρήσαμε στη μεταβλητή x.

Στην επόμενη γραμμή κώδικα η συνάρτηση `derivative()` της SciPy υπολογίζει την πρώτη παράγωγο [n=1] της x στο σημείο 10., δηλαδή στη χρονική στιγμή 10 min, και για απόσταση από την κεντρικά διαφορά 10^{-6} [dx=1e-6].

Το αποτέλεσμα της παραγώγισης, που εμφανίζεται φορμαρισμένο στην οθόνη, είναι:

Velocity (m/min) = 302.34

Στην προτελευταία γραμμή κώδικα η συνάρτηση `derivative()` υπολογίζει τη *δεύτερη παράγωγο* [$n=2$] της x στο σημείο 10., δηλαδή στη χρονική στιγμή 10 min, και για απόσταση από την *κεντρώα διαφορά* 10^{-6} [$dx=1e-6$].

Το αποτέλεσμα της νέας παραγώγισης, που εμφανίζεται φορμαρισμένο στην οθόνη, είναι:

Acceleration (m/min²) = 64.12

Η τεκμηρίωση της συνάρτησης `derivative()` της SciPy δίνεται στο website:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.misc.derivative.html>

11.3.4 Ολοκληρώματα συναρτήσεων

Για να υπολογίσουμε το ορισμένο ολοκλήρωμα μιας μαθηματικής συνάρτησης με τη βοήθεια της βιβλιοθήκης SciPy, θα χρησιμοποιήσουμε από την οικογένεια συναρτήσεων `integrate` της βιβλιοθήκης `scipy`, τη συνάρτηση `quad` την οποία και κάνουμε `import` στον κώδικά μας.

Ο αριθμητικός προσδιορισμός του ολοκληρώματος γίνεται με τη βοήθεια μιας τεχνικής της βιβλιοθήκης QUADPACK της γλώσσας Fortran.

Για να δούμε τη χρήση της συνάρτησης `quad()`, θα συνεχίσουμε το παράδειγμα με την κίνηση του χωματουργικού αυτοκινήτου της πρακτικής άσκησης του σπουδαστή και θα ολοκληρώσουμε τη συνάρτηση $f(t) = 3.6 * t^2 - 2.99 * t^{1.3} + 2$, που υπολογίζει την ταχύτητα του ΧΑ.

Γράφουμε και τρέχουμε τον παρακάτω κώδικα, προκειμένου να υπολογίσουμε το ορισμένο ολοκλήρωμα $\int_{10}^{10} f(t)$, δηλαδή να βρούμε τη θέση του ΧΑ σε χρόνο 10 min μετά την εκκίνησή του από το σημείο φόρτωσης:

```
from scipy.integrate import quad
f_t = lambda t: 3.6*t**2 - 2.99*t**1.3 + 2
dist = quad(f_t, 0, 10)
print(dist)
print('Distance (m) = {:.0f}'.format(dist[0]))
```

Εδώ, αφού ορίσαμε τη συνάρτηση $f(t)$ υπό τη μορφή μιας συνάρτησης `lambda`, με άγνωστη μεταβλητή το t , και την εκχωρήσαμε στη μεταβλητή `f_t`, στην επόμενη γραμμή κώδικα ζητήσαμε από τη SciPy, κάνοντας χρήση της συνάρτησης `quad()` να κάνει την ολοκλήρωση της `f_t` με κάτω όριο το 0 και άνω όριο το 10 και να γίνει εκχώρηση του αποτελέσματος στη μεταβλητή `dist`.

Το αποτέλεσμα της ολοκλήρωσης που εμφανίζεται στην οθόνη με την εντολή `print(dist)` είναι:

(960.6158990540457, 1.0664978889507968e-12)

Παρατηρούμε ότι το αποτέλεσμα δίνεται υπό τη μορφή μιας πλειάδας, όπου ο πρώτος αριθμός αντιστοιχεί στο αριθμητικό αποτέλεσμα του ορισμένου ολοκληρώματος $\int_{10}^{10} f(t)$, και ο δεύτερος αριθμός δίνει μια εκτίμηση της *απόλυτης τιμής του λάθους της ολοκλήρωσης*.

Η τελευταία γραμμή κώδικα μάς εμφανίζει φορμαρισμένο το αποτέλεσμα της ολοκλήρωσης στην οθόνη:

Distance (m) = 961

Η τεκμηρίωση της συνάρτησης `quad()` της SciPy δίνεται στο website:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.quad.html>

11.3.4.1 Υπολογισμός όγκου εκσκαφών

Για τον υπολογισμό του όγκου των εκσκαφών σε ένα υπαίθριο ορυχείο, θα χρησιμοποιήσουμε από την οικογένεια συναρτήσεων `integrate` της βιβλιοθήκης `scipy`, τη συνάρτηση `trapezoid()` και για λόγους σύγκρισης τη συνάρτηση `simpson()`.

Και οι δύο αυτές συναρτήσεις μας επιτρέπουν να βρούμε το ορισμένο ολοκλήρωμα μιας συνάρτησης όταν γνωρίζουμε τις τιμές της σε ορισμένα σημεία της μόνο, σε αντίθεση με την περίπτωση της συνάρτησης `quad()`, την οποία χρησιμοποιούμε όταν γνωρίζουμε αυτή την ίδια τη συνάρτηση.

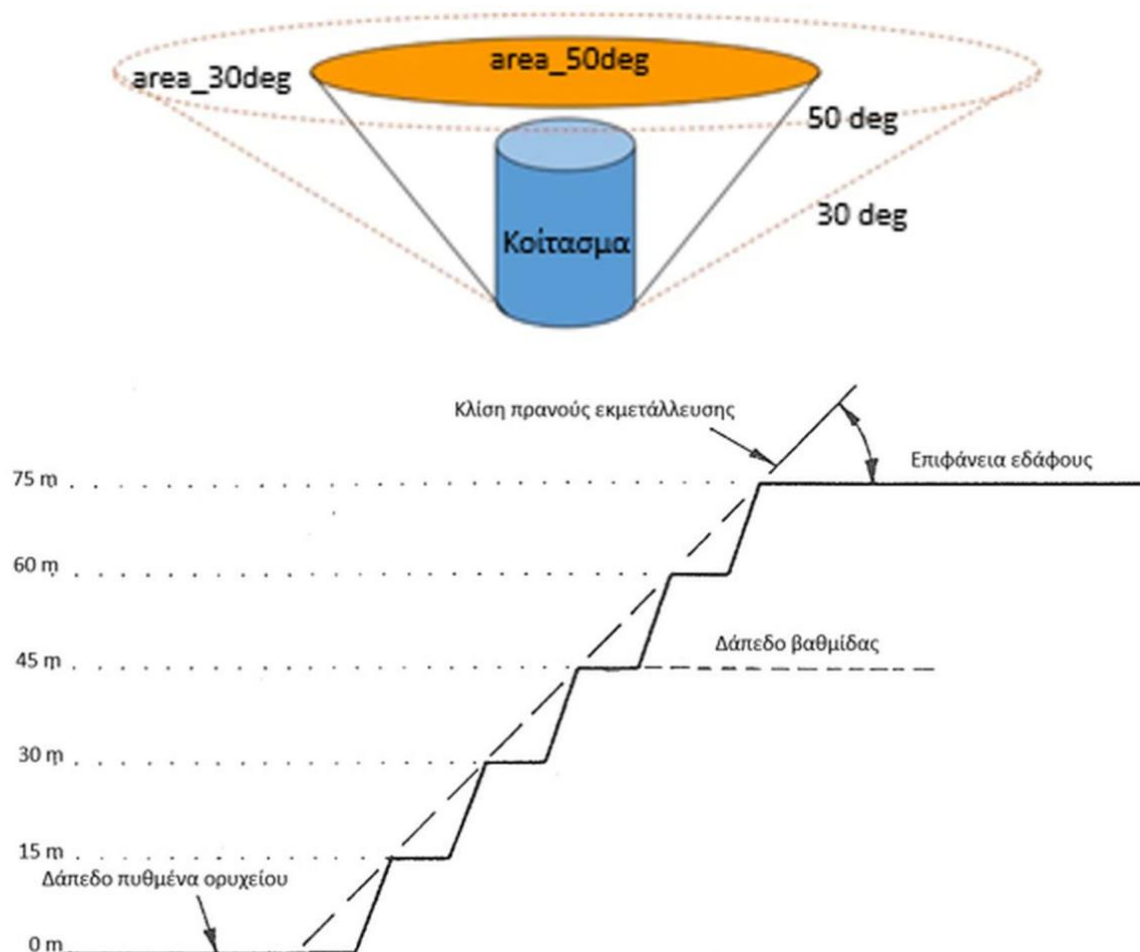
Η συνάρτηση `trapezoid()` εφαρμόζει τον κανόνα του τραπεζοειδούς (Trapezoidal rule), βλέπε: https://en.wikipedia.org/wiki/Trapezoidal_rule

Η συνάρτηση `simpson()` εφαρμόζει τον κανόνα του Simpson (Simpson's rule), βλέπε: https://en.wikipedia.org/wiki/Simpson%27s_rule

Η εκμετάλλευση ενός κοιτάσματος πρόκειται να γίνει με την ανάπτυξη ενός υπαίθριου ορυχείου που θα αποτελείται από πέντε (5) βαθμίδες ύψους 15 m η καθεμία (Εικόνα 11.2).

Ο αρχικός σχεδιασμός προέβλεπε κλίση πρανούς εκμετάλλευσης 50° , αλλά μετά την έναρξη των εργασιών εξόρυξης διαπιστώθηκε ότι οι γεωτεχνικές συνθήκες των πρανών επέβαλαν τη μείωση της κλίσης σε 30° .

Το ζητούμενο είναι να υπολογιστεί ο όγκος των εκσκαφών που πρέπει να γίνουν επιπλέον, λόγω της μείωσης της κλίσης από 50° σε 30° .



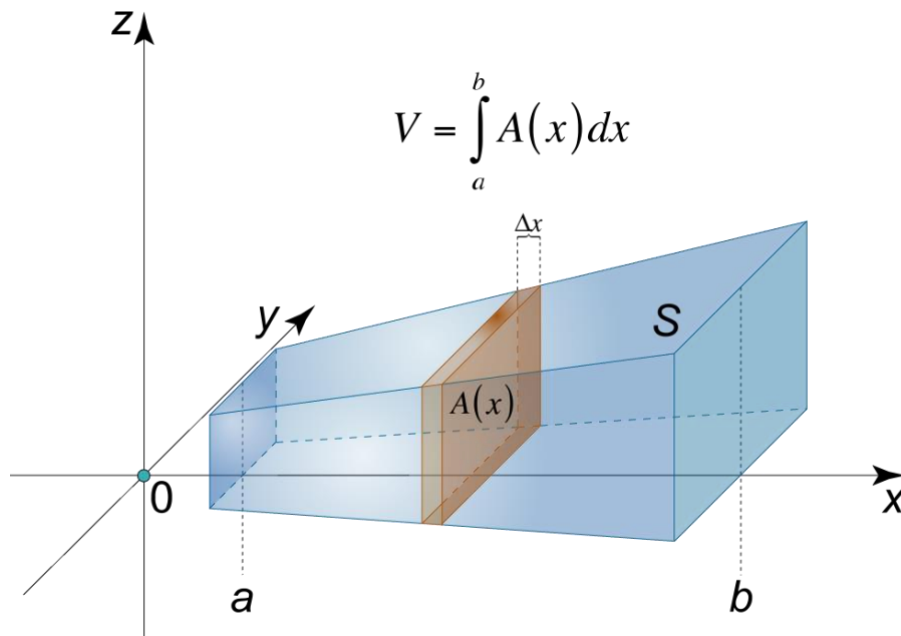
Εικόνα 11.2. Σκαρίφημα εκμετάλλευσης και τομή βαθμίδων υπαίθριου ορυχείου.

Καταρχάς, μετράμε το εμβαδόν των εκσκαφών στο δάπεδο του πυθμένα, στο υψόμετρο του δαπέδου κάθε βαθμίδας και στην επιφάνεια του εδάφους, για κλίση πρανούς εκμετάλλευσης 30° και 50° . Οι τιμές των μετρήσεων, στρογγυλεμένες, δίνονται στον Πίνακα 11.1.

Πίνακας 11.1. Επιφάνειες εκσκαφών εκμετάλλευσης ορυχείου.

Υψόμετρο Δαπέδου (m)	Επιφάνεια Εκσκαφών (m ²) Κλίση Πρανούς 30°	Επιφάνεια Εκσκαφών (m ²) Κλίση Πρανούς 50°
0	105,000	105,000
15	200,000	165,000
30	315,000	250,000
45	508,000	354,000
60	611,000	461,000
75	808,000	595,000

Ο όγκος ενός στερεού S , του οποίου η επιφάνεια της διατομής του, κάθετα στον άξονα x , είναι $A(x)$ για $a \leq x \leq b$ και που ορίζεται από δύο κάθετες επιφάνειες στα σημεία a και b , ισούται με ορισμένο ολοκλήρωμα $\int_a^b A(x).dx$ (Εικόνα 11.3).



Εικόνα 11.3. Υπολογισμός του όγκου στερεού που ορίζεται από δύο επιφάνειες και είναι γνωστό το εμβαδόν διατομής του [Πηγή: <https://math24.net/volume-solid-with-known-cross-section.html>].

Στη συνέχεια, γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
# Calculation of pit excavation volume
# using SciPy Trapezoidal & Simpson's Rules
import numpy as np
from scipy import integrate
# Pit data
bench_level = np.array([0, 15, 30, 45, 60, 75])
bench_area_30deg = np.array([105000, 200000, 315000, 508000, 611000,
                              808000])
bench_area_50deg = np.array([105000, 165000, 250000, 354000, 461000,
                              595000])
# Volume Calculations: SciPy, Trapezoid rule
volume_30deg_trapz = integrate.trapezoid(bench_area_30deg, bench_level)
volume_50deg_trapz = integrate.trapezoid(bench_area_50deg, bench_level)
volume_diff_trapz = volume_30deg_trapz - volume_50deg_trapz
print('Volume Calculations: SciPy, Trapezoid rule:')
```

```
print('Volume Total 30deg-Pit (Trapezoid) [m3] = {:.0f}'
      .format(volume_30deg_trapz))
print('Volume Total 50deg-Pit (Trapezoid) [m3] = {:.0f}'
      .format(volume_50deg_trapz))
print('Volume Difference 30deg vs 50deg Pit (Trapezoid) [m3] = {:.0f}'
      .format(volume_dif_trapz))
```

Κάνουμε import στον κώδικά μας τη βιβλιοθήκη `numpy` και από τη βιβλιοθήκη `scipy` όλη την οικογένεια συναρτήσεων `integrate`.

Κάτω από τη γραμμή σχολίου `# Pit data`, οι μεταβλητές `bench_level`, `bench_area_30deg` και `bench_area_50deg` περιέχουν, υπό μορφή array της NumPy (`ndarray`), τις τιμές των δεδομένων μας, όπως αυτές παρουσιάζονται στον Πίνακα 11.1.

Με τη γραμμή κώδικα:

```
volume_30deg_trapz = integrate.trapezoid(bench_area_30deg, bench_level)
```

η συνάρτηση `trapezoid()` υπολογίζει τον όγκο της χοάνης των εκσκαφών στην περίπτωση που η κλίση του πρανούς εκμετάλλευσης είναι 30°.

Κατ' ανάλογο τρόπο υπολογίζεται ο όγκος της χοάνης των εκσκαφών στην περίπτωση που η κλίση του πρανούς εκμετάλλευσης είναι 50°.

Για την τεκμηρίωση της συνάρτησης `trapezoid()` ανατρέξτε στο website:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.trapezoid.html>

Το αποτέλεσμα των υπολογισμών που εμφανίζεται στην οθόνη είναι:

```
Volume Calculations: SciPy, Trapezoid rule:
Volume Total 30deg-Pit (Trapezoid) [m3] = 31357500
Volume Total 50deg-Pit (Trapezoid) [m3] = 23700000
Volume Difference 30deg vs 50deg Pit (Trapezoid) [m3] = 7657500
```

Παρατηρούμε ότι ο επιπλέον όγκος πετρώματος που πρέπει να εξορυχθεί, εάν η γωνία του πρανούς εκμετάλλευσης από 50° γίνει 30° είναι 7,657,500 m³.

Ο κώδικας που ακολουθεί, υπολογίζει τους όγκους των εκσκαφών με τη βοήθεια της συνάρτησης `simpson()` εφαρμόζοντας τον κανόνα του Simpson:

```
# Volume Calculations: SciPy, Simpson's rule
volume_30deg_simps = integrate.simpson(bench_area_30deg, bench_level)
volume_50deg_simps = integrate.simpson(bench_area_50deg, bench_level)
volume_dif_simps = volume_30deg_simps - volume_50deg_simps
print('')
print("Volume Calculations: SciPy, Simpson's rule:")
print("Volume Total 30deg-Pit (Simpson's) = {:.0f}"
      .format(volume_30deg_simps))
print("Volume Total 50deg-Pit (Simpson's) = {:.0f}"
      .format(volume_50deg_simps))
print("Volume Difference 30deg vs 50deg Pit (Simpson's) [m3] = {:.0f}"
      .format(volume_dif_simps))
```

και εμφανίζει το αποτέλεσμα:

```
Volume Calculations: SciPy, Simpson's rule:
Volume Total 30deg-Pit (Simpson's) = 31230000
Volume Total 50deg-Pit (Simpson's) = 23607500
Volume Difference 30deg vs 50deg Pit (Simpson's) [m3] = 7622500
```

Παρατηρούμε ότι ο επιπλέον όγκος πετρώματος που πρέπει να εξορυχθεί είναι μικρότερος στην περίπτωση του υπολογισμού με τον κανόνα του Simpson.

Αυτό είναι λογικό, γιατί με τον κανόνα του Simpson είναι πιο ακριβής ο υπολογισμός του ολοκληρώματος, σε σύγκριση με τον κανόνα του τραπεζοειδούς.

Πάντως, η διαφορά μεταξύ των δύο τρόπων υπολογισμού δεν είναι ιδιαίτερα μεγάλη, όπως προκύπτει από το τμήμα του κώδικα, που προσθέτουμε:

```
# Compare Trapezoid vs Simpson's volume calculation
dif_trapz_vs_simps = volume_dif_trapz - volume_dif_simps
dif_trapz_vs_simps_pc = (dif_trapz_vs_simps*100)/volume_dif_trapz
print('')
print("Compare Trapezoid vs Simpson's volume calculations")
print("Volume Difference Trapezoid vs Simpson's [m3] | % = {:.0f} | {:.2f}"
      .format(dif_trapz_vs_simps, dif_trapz_vs_simps_pc))
```

Compare Trapezoid vs Simpson's volume calculations Volume Difference Trapezoid vs Simpson's [m3] % = 35000 0.46
--

Για την τεκμηρίωση της συνάρτησης `simpson()` ανατρέξτε στο website: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.simps.html> και παρατηρήστε το πώς αυτή χειρίζεται τον αριθμό των σημείων (N) της προς ολοκλήρωση συνάρτησης με τα οποία την τροφοδοτούμε, μέσω της παραμέτρου `even`!

Το script του κώδικα του παραδείγματος υπολογισμού του όγκου των εκσκαφών περιλαμβάνεται στο website του βιβλίου στο αρχείο `Chapter_11_Volume_Calculation_scipy.py`

11.3.5 Συνήθεις διαφορικές εξισώσεις

Για την επίλυση συνήθων διαφορικών εξισώσεων ή και συστημάτων συνήθων διαφορικών εξισώσεων με τη βοήθεια της βιβλιοθήκης SciPy, θα χρησιμοποιήσουμε από την οικογένεια συναρτήσεων `integrate` της βιβλιοθήκης `scipy`, τη συνάρτηση `odeint`, την οποία και κάνουμε `import` στον κώδικά μας.

Ο αριθμητικός προσδιορισμός της λύσης γίνεται με τη βοήθεια της ρουτίνας `lsoda` της βιβλιοθήκης ODEPACK της γλώσσας Fortran.

Για να δούμε τη χρήση της συνάρτησης `odeint()` θα πάρουμε το παράδειγμα της ψύξης του τεμαχίου χάλυβα στο σιδηρουργείο, που παρουσιάσαμε και επιλύσαμε με τη χρήση της βιβλιοθήκης `SymPy` στην παράγραφο 11.2.5 Συνήθεις διαφορικές εξισώσεις.

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from scipy.integrate import odeint
import numpy as np
# Data
T_oven = 1350 # oC
T_forge = 1100 # oC
T_room = 25 # oC
k = -np.log(265) + np.log(66515) / 2

def T_t(T, t, k, T_room):
    return k*(T - T_room)

# Solving equation T_t by integration
t = np.linspace(0, 20, 100) # min
T_steel = odeint(T_t, T_oven, t, args=(k, T_room))
print('Temperature (oC) =', T_steel)
```

Η διαφορική εξίσωση που πρέπει να επιλύσουμε είναι η:

$$dT(t)/dt = k * (T(t) - T_{room}) \quad [11.11]$$

όπου k είναι ο συντελεστής μεταφοράς της θερμότητας.

Τον συντελεστή μεταφοράς της θερμότητας k τον υπολογίσαμε προηγουμένως, μέσω της SymPy.

Το δεξιό μέρος της εξίσωσης [11.11] το ορίζουμε στον κώδικά μας, ως συνάρτηση $T_t()$, με χρήση της κλασικής μεθόδου ορισμού συναρτήσεων της Python.

Επειδή η επίλυση των διαφορικών εξισώσεων με χρήση της συνάρτησης $odeint()$ της SciPy γίνεται με αριθμητική ολοκλήρωση, πρέπει να ορίσουμε το διάστημα τιμών της μεταβλητής ολοκλήρωσης, που στην περίπτωση μας είναι ο χρόνος t .

Για τον λόγο αυτό, στη γραμμή κώδικα:

```
t = np.linspace(0, 20, 100) # min
```

δημιουργούμε μία array (ndarray), που έχει 100 τιμές από το 0 έως και τα 20 min και αντιστοιχεί στον χρόνο απομάκρυνσης του τεμαχίου χάλυβα από το καμίνι, την οποία και εκχωρούμε στη μεταβλητή t .

Στην επόμενη γραμμή κώδικα η συνάρτηση $odeint()$ της SciPy ολοκληρώνει τη συνάρτηση $T_t()$, η οποία έχει αρχική τιμή ίση με T_oven , δηλαδή $T_t(0) = T_oven$, στα σημεία που δίνονται από την array t , ενώ οι επιπλέον παράμετροι (args) της συνάρτησης $T_t()$ είναι οι τιμές των μεταβλητών k και T_room .

Το αποτέλεσμα που εμφανίζει στην οθόνη η εντολή $print()$ είναι μία array, η πρώτη τιμή της οποίας είναι η τιμή της $T_t(0)$, δηλαδή το T_oven :

```
Temperature (oC) = [[1350.      ] [1342.75553698] [1335.55068194]
[1328.38523862] [1321.25898643] [1314.17167976] [1307.12311671]
[1300.11309268] [1293.14139669] [1286.20781826] [1279.31214953]
[1272.45418286] [1265.63371212] [1258.85053222] [1252.10443941]
[1245.39523075] [1238.72270463] [1232.08666071] [1225.48689903]
[1218.9232214 ] [1212.39543099] [1205.90333186] [1199.44672837]
[1193.02542641] [1186.63923302] [1180.28795625] [1173.97140513]
[1167.68938982] [1161.44172152] [1155.22821246] [1149.04867579]
[1142.9029258 ] [1136.7907778 ] [1130.71204793] [1124.66655326]
[1118.65411208] [1112.6745437 ] [1106.72766841] [1100.81330752]
[1094.93128331] [1089.08141907] [1083.26353906] [1077.4774685 ]
[1071.7230336 ] [1066.00006151] [1060.30838035] [1054.64781908]
[1049.01820734] [1043.41937589] [1037.85115644] [1032.31338162]
[1026.80588494] [1021.32850082] [1015.88106455] [1010.4634123 ]
[1005.07538112] [ 999.71680889] [ 994.3875344 ] [ 989.08739723]
[ 983.81623787] [ 978.57389839] [ 973.36022156] [ 968.17505066]
[ 963.01822984] [ 957.88960408] [ 952.78901923] [ 947.71632194]
[ 942.67135973] [ 937.65398092] [ 932.66403465] [ 927.7013709 ]
[ 922.76584044] [ 917.85729484] [ 912.9755866 ] [ 908.1205692 ]
[ 903.29209671] [ 898.490024 ] [ 893.71420674] [ 888.96450136]
[ 884.24076509] [ 879.54285594] [ 874.87063269] [ 870.22395487]
[ 865.6026828 ] [ 861.00667753] [ 856.4358009 ] [ 851.88991547]
[ 847.36888473] [ 842.87257284] [ 838.40084463] [ 833.9535657 ]
[ 829.53060237] [ 825.13182169] [ 820.75709142] [ 816.40628007]
[ 812.07925684] [ 807.77589164] [ 803.4960551 ] [ 799.23961855]
[ 795.00645419]]]
```

Εάν στο τέλος του κώδικα προσθέσουμε τη γραμμή:

```
print('Time (min) =', t)
```


Θα πάρουμε και την αντίστοιχη array των τιμών του χρόνου t για τους οποίους υπολογίστηκε η θερμοκρασία του τεμαχίου χάλυβα:

Time (min)=	[0.	0.2020202	0.4040404	0.60606061	0.80808081
1.01010101	1.21212121	1.41414141	1.61616162	1.81818182	2.02020202
2.22222222	2.42424242	2.62626263	2.82828283	3.03030303	3.23232323
3.43434343	3.63636364	3.83838384	4.04040404	4.24242424	4.44444444
4.64646465	4.84848485	5.05050505	5.25252525	5.45454545	5.65656566
5.85858586	6.06060606	6.26262626	6.46464646	6.66666667	6.86868687
7.07070707	7.27272727	7.47474747	7.67676768	7.87878788	8.08080808
8.28282828	8.48484848	8.68686869	8.88888889	9.09090909	9.29292929
9.49494949	9.6969697	9.8989899	10.1010101	10.3030303	10.50505051
10.70707071	10.90909091	11.11111111	11.31313131	11.51515152	11.71717172
11.91919192	12.12121212	12.32323232	12.52525253	12.72727273	12.92929293
13.13131313	13.33333333	13.53535354	13.73737374	13.93939394	14.14141414
14.34343434	14.54545455	14.74747475	14.94949495	15.15151515	15.35353535
15.55555556	15.75757576	15.95959596	16.16161616	16.36363636	16.56565657
16.76767677	16.96969697	17.17171717	17.37373737	17.57575758	17.77777778
17.97979798	18.18181818	18.38383838	18.58585859	18.78787879	18.98989899
19.19191919	19.39393939	19.5959596	19.7979798	20.]	

Εάν αντιστοιχίσουμε τις τιμές των array της θερμοκρασίας Temperature (oC) και του χρόνου Time (min), θα διαπιστώσουμε ότι η πλησιέστερη θερμοκρασία στη θερμοκρασία πέρατος της σφυρηλάτησης, $T_{\text{forge}} = 1100$, δίνεται από το 39^ο στοιχείο της array των θερμοκρασιών, ενώ το αντίστοιχο 39^ο στοιχείο της array των χρόνων έχει την τιμή 7.67676768, δηλαδή ο χρόνος που έχει στη διάθεσή του ο σιδηρουργός για να το σφυρηλατήσει είναι 7.68 min.

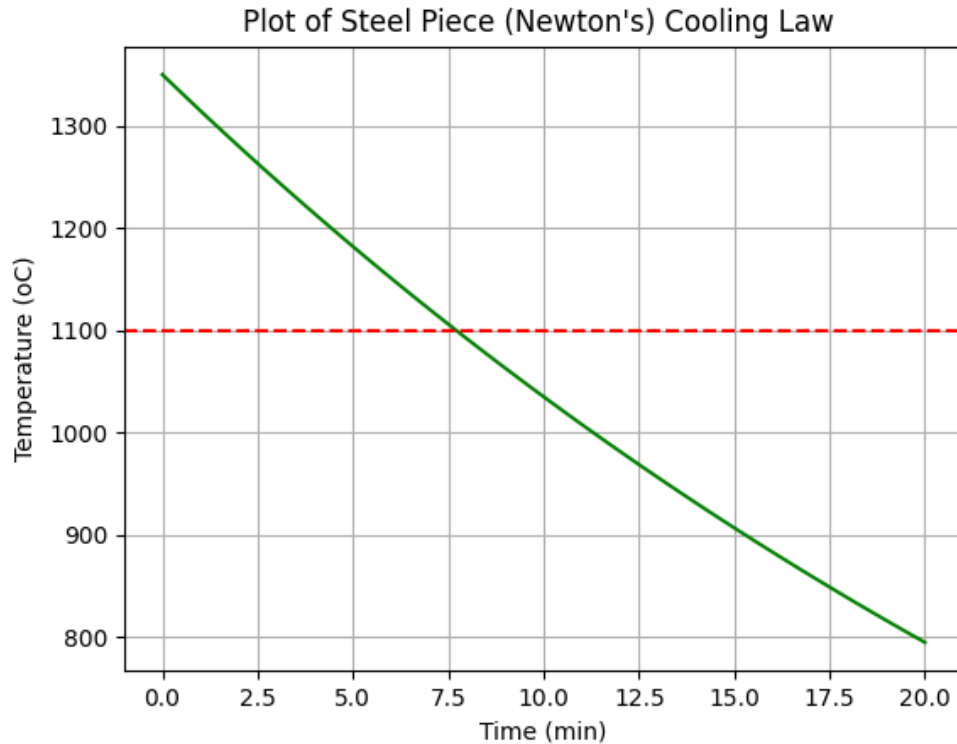
Σημείωση: Για διευκόλυνση του αναγνώστη, οι τιμές του 39^ο στοιχείου των array της θερμοκρασιών και του χρόνου έχουν μαρκαριστεί με κίτρινο χρώμα.

Ο προσδιορισμός του χρόνου σφυρηλάτησης μπορεί να γίνει και γραφικά.

Για τον σκοπό αυτό προσθέτουμε τις παρακάτω γραμμές κώδικα και ξανατρέχουμε το πρόγραμμα:

```
# Plot T_steel vs t
import matplotlib.pyplot as plt
plt.xlabel("Time (min)")
plt.ylabel("Temperature (oC)")
plt.title("Plot of Steel Piece (Newton's) Cooling Law ")
plt.grid(True)
line = plt.plot(t, T_steel, color='green')
plt.axhline(y=T_forge, color='red', linestyle='--')
plt.savefig('Steel_temperature_vs_time.png')
plt.show()
```

Στην οθόνη εμφανίζεται η Εικόνα 11.4, στην οποία βλέπουμε, με πράσινο χρώμα, την καμπύλη μεταβολής της θερμοκρασίας του τεμαχίου χάλυβα συναρτήσει του χρόνου σφυρηλάτησης.



Εικόνα 11.4. Η καμπύλη ψύξης του τεμαχίου χάλυβα συναρτήσει του χρόνου σφυρηλάτησης.

Για την κατασκευή του διαγράμματος της Εικόνας 11.4 χρησιμοποιήσαμε τις ήδη γνωστές μας συναρτήσεις από τη βιβλιοθήκη Matplotlib. Το νέο στοιχείο που προσθέσαμε είναι η γραμμή κώδικα:

```
plt.axhline(y=T_forge, color='red', linestyle='--')
```

προκειμένου να χαράξουμε τη διακεκομμένη κόκκινη οριζόντια γραμμή από το σημείο του άξονα y που αντιστοιχεί στη θερμοκρασία πέρατος της σφυρηλάτησης, $T_{\text{forge}} = 1100$.

Το σημείο τομής της κόκκινης γραμμής με την πράσινη καμπύλη αντιστοιχεί στον χρόνο που έχει στη διάθεσή του ο σιδηρουργός για σφυρηλάτηση.

Όσοι αναγνώστες δεν ικανοποιούνται από την ακρίβεια του χρόνου που δίνει η ανάγνωση του διαγράμματος της Εικόνας 11.4, μπορούν να προσθέσουν στον κώδικα τις παρακάτω γραμμές:

```
import shapely.geometry as sg
line_T_steel = sg.LineString(list(zip(t, T_steel)))
line_T_forge = sg.LineString([min(t), T_forge], (max(t), T_forge))
coords = np.array(line_T_steel.intersection(line_T_forge))
print('Coordinates = ', coords)
```

οι οποίες προσδιορίζουν τις συντεταγμένες του σημείου τομής της κόκκινης γραμμής με την πράσινη καμπύλη, με τη βοήθεια συναρτήσεων της εξωτερικής βιβλιοθήκης Shapely.

Τη βιβλιοθήκη πρέπει να την εγκαταστήσουμε στον υπολογιστή μας κατά τα γνωστά:

```
pip install Shapely
```

και στη συνέχεια να κάνουμε `import` την οικογένεια συναρτήσεων `geometry` της βιβλιοθήκης `shapely`.

Στη δεύτερη γραμμή κώδικα, με τη βοήθεια της συνάρτησης `LineString()` δημιουργούμε τη γραμμή `line_T_steel` χρησιμοποιώντας τις τιμές των `array` του χρόνου `t` και της θερμοκρασίας του τεμαχίου χάλυβα `T_steel` (η πράσινη καμπύλη της Εικόνας 11.4).

Στην επόμενη γραμμή κώδικα, κατ' ανάλογο τρόπο δημιουργούμε τη γραμμή `line_T_forge` (η κόκκινη γραμμή της Εικόνας 11.4).

Η τέταρτη γραμμή κώδικα βρίσκει τις συντεταγμένες της τομής (intersection) των γραμμών `line_T_steel` και `line_T_forge`, οι τιμές των οποίων εμφανίζονται στη συνέχεια στην οθόνη:

```
Coordinates = [7.70470101 1100.]
```

Επομένως, ο χρόνος που έχει στη διάθεσή του ο σιδηρουργός για σφυρηλάτηση, δηλαδή μέχρι η θερμοκρασία του τεμαχίου χάλυβα να γίνει 1100 °C, είναι 7.70 min.

Το script του κώδικα του παραδείγματος περιλαμβάνεται στο website του βιβλίου στο αρχείο: `Chapter_11_Newtons_Cooling_Law_Forging_scipy.py`

Η τεκμηρίωση της συνάρτησης `odeint()` της SciPy υπάρχει στο website:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.odeint.html>

Για την τεκμηρίωση της βιβλιοθήκης Shapely ανατρέξτε στο website:

<https://pypi.org/project/Shapely/>

Σημείωση: Η οικογένεια συναρτήσεων `integrate` της βιβλιοθήκης SciPy περιλαμβάνει και άλλες τεχνικές για τον υπολογισμό ολοκληρωμάτων. Για περισσότερες πληροφορίες ανατρέξτε στο website: <https://docs.scipy.org/doc/scipy/tutorial/integrate.html>

11.4 Επίλυση εξισώσεων: Μέθοδος της διχοτόμησης

Στην παράγραφο αυτή θα παρουσιάσουμε μια τεχνική επίλυσης εξισώσεων με έναν άγνωστο, η οποία χρησιμοποιεί την προσεγγιστική μαθηματική μέθοδο της διχοτόμησης (bisection method). Η μέθοδος αυτή, γνωστή στους αναγνώστες από τα Μαθηματικά, καθώς και η εφαρμογή της παρουσιάζεται στο website: https://en.wikipedia.org/wiki/Bisection_method

Η μέθοδος της διχοτόμησης εφαρμόζεται για την επίλυση εξισώσεων της μορφής $f(x) = 0$, εφόσον ισχύουν οι ακόλουθες προϋποθέσεις:

1. η μεταβλητή x είναι πραγματική,
2. η συνάρτηση $f(x)$ είναι συνεχής στο κλειστό διάστημα $[a, b]$ και
3. οι τιμές των $f(a)$ και $f(b)$ έχουν αντίθετα πρόσημα.

Εάν συντρέχουν οι παραπάνω συνθήκες, βάσει του *θεωρήματος Bolzano*, η συνάρτηση $f(x)$ πρέπει να έχει κατ' ελάχιστο μια ρίζα μεταξύ του a και του b .

Με βάση τα παραπάνω, χρησιμοποιώντας εντολές της γλώσσας Python που ήδη γνωρίζουμε και χωρίς τη χρήση βιβλιοθηκών, μπορούμε να γράψουμε τη δική μας συνάρτηση επίλυσης εξισώσεων, αντί να χρησιμοποιούμε τις συναρτήσεις-κονσέρβες των βιβλιοθηκών SymPy και SciPy.

Τον κώδικα της συνάρτησης που θα γράψουμε για την επίλυση εξισώσεων με τη μέθοδο της διχοτόμησης, θα τον αποθηκεύσουμε σε ένα αρχείο `.py` και θα αποτελεί ένα module της Python.

Το module αυτό θα έχουμε τη δυνατότητα να το χρησιμοποιούμε, αφού προηγουμένως το κάνουμε `import`, όπως κάνουμε με τις βιβλιοθήκες της Python, σε οποιοδήποτε πρόγραμμα γράφουμε στο μέλλον, κάνοντας μόνο μια απλή αναφορά σε αυτό.

Ο συγγραφέας του βιβλίου έχει γράψει το δικό του module για την επίλυση εξισώσεων με τη μέθοδο της διχοτόμησης, στο οποίο έχει δώσει το όνομα `gnp_bisection` και το έχει σώσει στο αρχείο `gnp_bisection.py`

Το script του module είναι:

```
"""
Module name: gnp_bisection
Description:
```

gnp_bisec() is a root finder function for $f(x) = 0$ type equations. It finds, using the bisection method, a value x between $[a, b]$ for which $f(x) = 0$, within a given tolerance (tol).

USE: `gnp_bisec(f, a, b, tol=1.0e-15)`

Input for the method are: a continuous function f and an interval $[a, b]$ for which the values $f(a)$ and $f(b)$ are of opposite sign. There is at least one zero crossing (root) within this interval (Bolzano's theorem).

If convergence is satisfactory, that is $|f(x)| \leq tol$, return x as a root of $f(x)$.

tol : float, optional. Absolute error tolerance. Default value: $1.0e-12$

Author: G.N. Panagiotou

Python: 3.7

Last modified: May 11, 2021

"""

```
def gnp_bisec(f, a, b, tol=1.0e-12):
    if (f(a) * f(b)) > 0:
        raise Exception('The values of a and b do not bound a root')
    dx = abs(b - a)
    while dx > tol:
        x = (a + b)/2.
        if (f(a) * f(x)) < 0:
            b = x
        else:
            a = x
        dx = abs(b - a)
    return x
```

Στη συνέχεια, θα χρησιμοποιήσουμε το module `gnp_bisection` για να υπολογίσουμε την ποσότητα w της εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα έτσι ώστε σε απόσταση d από το μέτωπο της ανατίναξης η τιμή της ppv να μην υπερβαίνει ένα όριο που προβλέπεται από τον ΚΜΛΕ, χρησιμοποιώντας την Εξίσωση [2.1] της παραγράφου 11.2.1 Εξισώσεις με έναν άγνωστο που υπολογίζει την ppv .

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from gnp_bisection import gnp_bisec
h = 1730
d = 100
b = -1.60
ppv = 5
w_eq = lambda w: h*(d/w**0.5)**b - ppv
solution_w_eq = gnp_bisec(w_eq, 0.1, 1000)
print('Weight of Explosives (kg) =', solution_w_eq)
print('Weight of Explosives (kg) = {:.2f}'.format(solution_w_eq))
```

Το αποτέλεσμα που εμφανίζεται στην οθόνη, σε αφορμάριστη και φορμαρισμένη μορφή είναι:

```
Weight of Explosives (kg) = 6.701235438162847
Weight of Explosives (kg) = 6.70
```

Καταρχάς, κάναμε `import` από το module `gnp_bisection` τη συνάρτηση `gnp_bisec`.

Το module πρέπει να είναι αποθηκευμένο σε ένα directory του υπολογιστή μας που να μπορεί να το βλέπει ο interpreter της Python. Το απλούστερο είναι να έχετε το module και το script του προγράμματός σας στο ίδιο directory.

Στη συνέχεια, αποδώσαμε στις μεταβλητές h , d , b , και ppv τις ίδιες αριθμητικές τιμές που χρησιμοποιήσαμε όταν επιλύσαμε την εξίσωση [2.1] με τη βοήθεια συναρτήσεων των βιβλιοθηκών SymPy και SciPy.

Στη συνέχεια, γράψαμε την εξίσωση [2.1] σε μορφή $f(x)=0$:

$$0 = h * (d / \sqrt{w})^b - ppv$$

ως μια συνάρτηση `lambda`, με άγνωστη μεταβλητή της συνάρτησης την `w`, και την εκχωρήσαμε στη μεταβλητή `w_eq`.

Με τη γραμμή κώδικα `solution_w_eq = gnp_bisec(w_eq, 0.1, 1000)` λύνουμε την εξίσωση `w_eq` εκτιμώντας ότι η λύση-ρίζα θα είναι μεταξύ των τιμών 0.1 και 1000, και εκχωρούμε τη λύση στη μεταβλητή `solution_w_eq`.

Εδώ θα πρέπει να σημειώσουμε ότι η μέθοδος της διχοτόμησης και κατ' επέκταση η συνάρτηση `gnp_bisec()` έχει μια *ευαισθησία* σε ό,τι αφορά την επιλογή των τιμών του διαστήματος `[a, b]` εντός του οποίου θα βρίσκεται η ρίζα-λύση.

Εάν, για παράδειγμα, στο πρόβλημά μας επιλέξετε το διάστημα `[0, 1000]` αντί του `[0.1, 1000]`, δηλαδή γράψετε:

```
solution_w_eq = gnp_bisec(w_eq, 0, 1000)
```

και τρέξετε τον κώδικα, θα πάρετε το μήνυμα λάθους:

```
w_eq = lambda w: h * (d/w**0.5)**b - ppv
ZeroDivisionError: float division by zero
```

γιατί το `w` στην εξίσωση `w_eq` θα πάρει, κατά την εφαρμογή της μεθόδου διχοτόμησης, την τιμή 0 και *διαίρεση διά του μηδενός δεν γίνεται!*

Επίσης, εάν επιλέξετε τιμές του διαστήματος `[a, b]` εντός του οποίου δεν υπάρχει ρίζα-λύση, π.χ. γράψετε:

```
solution_w_eq = gnp_bisec(w_eq, 0.1, 5)
```

η συνάρτηση `gnp_bisec()` θα εγείρει μια εξαίρεση (`Exception`) και θα εμφανίσει το μήνυμα:

```
Exception: The values of a and b do not bound a root
```

Συμπέρασμα: Η χρήση της της μεθόδου διχοτόμησης για την επίλυση εξισώσεων θέλει *προσοχή* και *κοινή λογική*.

Το script του κώδικα του παραδείγματος περιλαμβάνεται στο website του βιβλίου στο αρχείο: `Chapter_11_Equation_Solver_bisection.py`

11.5 Περί ανοχής λάθους στις αριθμητικές μεθόδους

Στις παραγράφους αυτού του Κεφαλαίου είδαμε διάφορους τρόπους επίλυσης εξισώσεων ή συστημάτων εξισώσεων.

Οι τρόποι επίλυσης των εξισώσεων με τη βοήθεια συναρτήσεων της βιβλιοθήκης `SymPy` έχουν το χαρακτηριστικό ότι δίνουν ακριβείς ρίζες-λύσεις.

Έτσι, στο πρόβλημα του υπολογισμού του διαθέσιμου χρόνου για σφυρηλάτηση που έχει ο σιδηρουργός, η `SymPy` μας έδωσε την ακριβή λύση:

```
Time for Forging (min) = -log((43/53)**(2/log(265/251)))
```

την οποία, για να έχει χρηστική αξία, τη μετατρέψαμε με τη βοήθεια της συνάρτησης `evalf()` της `SymPy`, στον εύκολα κατανοητό δεκαδικό αριθμό 7.70 (min), δηλαδή σε μια *κατά προσέγγιση*, υπό μαθηματικούς όρους, ρίζα-λύση της εξίσωσης [11.11], αλλά προφανώς *σωστή* λύση για έναν Μηχανικό.

Στα προβλήματα που καλείται να επιλύσει ένας Μηχανικός είναι σύνηθες να γίνεται αποδεκτό ένα λάθος (error) στον υπολογισμό ενός μεγέθους, αρκεί αυτό να είναι μικρότερο από ένα *όριο λάθους* ή *ανοχή* (tolerance). Η ανοχή προσδιορίζεται συνήθως από τη φύση του προβλήματος και του μεγέθους που υπολογίζεται.

Για την επίλυση εξισώσεων με τη βοήθεια συναρτήσεων της βιβλιοθήκης SciPy εφαρμόζονται αλγόριθμοι αριθμητικών μεθόδων και αριθμητικών προσεγγίσεων, που δίνουν λύσεις *κατά προσέγγιση*.

Λέμε ότι ένας κώδικας επίλυσης μιας εξίσωσης *συγκλίνει* σε μια ρίζα-λύση, όταν βρήκε μια λύση της οποίας η απόκλιση από την *ακριβή τιμή*, δηλαδή το λάθος που εμπεριέχει η *κατά προσέγγιση* λύση, είναι μικρότερο από την *ανοχή*.

Για παράδειγμα, η συνάρτηση `fsolve()` της βιβλιοθήκης SciPy, που χρησιμοποιήσαμε για την επίλυση των εξισώσεων, βρίσκει τη λύση της εξίσωσης με ανοχή `xtol=1.49012e-08` (default τιμή), δηλαδή σταματάει να κάνει υπολογισμούς όταν το σχετικό λάθος μεταξύ δύο διαδοχικών αριθμητικών επαναλήψεων – προσεγγίσεων της λύσης είναι μικρότερο ή ίσο του `xtol`. (βλέπε σχετικά στην τεκμηρίωση: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.fsolve.html>)

Στην περίπτωση της χρήσης της μεθόδου διχοτόμησης, η συνάρτηση `gnp_bisec()`, την οποία έχουμε γράψει εμείς, έχει default τιμή ανοχής `tol=1.0e-12`.

Αυτό σημαίνει, όπως φαίνεται και στον ορισμό της `gnp_bisec()`, ότι η συνάρτηση σταματάει να κάνει περαιτέρω υπολογισμούς για την εύρεση της ρίζας-λύσης μιας εξίσωσης όταν η τιμή της `dx = abs(b - a)` γίνει μικρότερη ή ίση του `tol`, δηλαδή όταν η απόλυτη τιμή της διαφοράς των τιμών που έχουν λάβει οι μεταβλητές `a` και `b` κατά την αμέσως προηγούμενη επανάληψη του βρόχου `while` είναι μικρότερη ή ίση από $1.0 * 10^{-12}$.

Για να διαπιστώσουμε την επίδραση της τιμής της ανοχής λάθους στους υπολογισμούς προσδιορισμού της λύσης μιας εξίσωσης, στο τέλος του κώδικα της παραγράφου 11.4 *Επίλυση εξισώσεων: Μέθοδος της διχοτόμησης*, με τον οποίο κάναμε τον υπολογισμό του βάρους της εκρηκτικής ύλης, προσθέτουμε τις παρακάτω γραμμές κώδικα:

```
for i in [0.100, 0.010, 0.001, 0.0001, 0.00001, 0.000001]:
    solution_w_eq = gnp_bisec(w_eq, 0.1, 1000, tol=i)
    print('Weight of Explosives (kg) = {:.f} [for tol= {:.f}]'.
          .format(solution_w_eq, i))
```

οι οποίες υπολογίζουν τις ρίζες-λύσεις της εξίσωσης `w_eq` δίνοντας διαφορετικές τιμές ανοχής λάθους στην παράμετρο `tol` της συνάρτησης `gnp_bisec()`, οι οποίες λαμβάνονται από τις τιμές της λίστας `[0.100, 0.010, 0.001, 0.0001, 0.00001, 0.000001]`.

Από τα αποτελέσματα που εμφανίζονται στην οθόνη:

```
Weight of Explosives (kg) = 6.752167 [for tol= 0.100000]
Weight of Explosives (kg) = 6.698766 [for tol= 0.010000]
Weight of Explosives (kg) = 6.701627 [for tol= 0.001000]
Weight of Explosives (kg) = 6.701210 [for tol= 0.000100]
Weight of Explosives (kg) = 6.701232 [for tol= 0.000010]
Weight of Explosives (kg) = 6.701235 [for tol= 0.000001]
```

είναι προφανές ότι για τα μεγέθη του συγκεκριμένου προβλήματος, η διαφορά των τιμών του βάρους της εκρηκτικής ύλης, που δίνονται ως λύσεις για διαφορετικά όρια λάθους, είναι από ένα σημείο και μετά μικρότερα του γραμμαρίου.

12 Επεξεργασία δεδομένων

12.1 Γενικά

Ένας Μηχανικός κατά τη διάρκεια της δουλειάς του συγκεντρώνει *δεδομένα* (data) από διάφορες πηγές.

Δεδομένα μπορεί να είναι τα αποτελέσματα μετρήσεων στο εργαστήριο ή στο εργοτάξιο, υπολογισμών στο χαρτί ή στον υπολογιστή, καταγραφές, αυτόματες ή μη, των τιμών παραμέτρων λειτουργίας μηχανημάτων, συσκευών παρακολούθησης της εξέλιξης γεωτεχνικών φαινομένων, περιβαλλοντικών ρύπων κ.ά.

Τα δεδομένα αποκτούν αξία όταν μπορούμε να συνάγουμε συμπεράσματα που βασίζονται σε αυτά.

Σε αυτό το Κεφάλαιο θα δούμε πώς μπορούμε να επεξεργαστούμε δεδομένα που έχουμε στη διάθεσή μας, με τη βοήθεια συναρτήσεων από εξωτερικές βιβλιοθήκες της Python.

12.2 Καταγραφή δεδομένων σε αρχεία

Για να επεξεργαστούμε δεδομένα πρέπει πρώτα να τα έχουμε καταγράψει σε ψηφιακή μορφή και να τα έχουμε αποθηκεύσει σε αρχείο στον υπολογιστή μας.

Δύο είναι οι πιο συνήθεις και εύκολοι τρόποι για την καταγραφή δεδομένων, της μορφής και του όγκου, που συνήθως χειρίζεται ένας μηχανικός στην καθημερινότητά του.

Ο πρώτος τρόπος είναι να καταγράψει και να οργανώσει τα δεδομένα υπό τη μορφή ενός φύλλου του προγράμματος Excel της Microsoft, οπότε θα τα σώσει σε ένα αρχείο τύπου `.xlsx`

Ο δεύτερος τρόπος είναι να τα καταγράψει με τη βοήθεια ενός απλού text editor, όπως το Σημειωματάριο (Notepad) των Windows, και να τα σώσει σε αρχείο τύπου `.csv`

Οι σύγχρονες συσκευές μετρήσεων και καταγραφής δεδομένων έχουν τη δυνατότητα να μας παρέχουν τα αρχεία των δεδομένων που έχουν αποθηκεύσει στη μνήμη τους, υπό τη μορφή `.xlsx` ή `.csv`

12.1.1 Αρχεία `.xlsx`

Μέχρι τώρα, το πρόβλημα των δονήσεων των ανατινάξεων το αντιμετωπίσαμε *προληπτικά*, δηλαδή η προσπάθειά μας επικεντρώθηκε στο να εκτιμήσουμε το μέγεθος της μέγιστης ταχύτητας δόνησης σωματιδίων του εδάφους (ppv) πριν γίνει η ανατίναξη.

Στην παράγραφο 02.2 *Μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους* αναφέραμε ότι η ppv μπορεί να μετρηθεί κατά τη διάρκεια μιας ανατίναξης, σε σημεία του εδάφους όπου μας ενδιαφέρει να γνωρίζουμε την τιμή της, με την εγκατάσταση στα σημεία αυτά συσκευών μέτρησης που ονομάζονται *δονησιογράφοι*.

Στην Εικόνα 12.1 παρουσιάζονται τα αποτελέσματα τριάντα μίας (31) μετρήσεων που καταγράφηκαν κατά τη διάρκεια ενός *δονημετρικού ελέγχου* των ανατινάξεων ενός λατομείου. Οι μετρήσεις καταγράφηκαν και οργανώθηκαν σε ένα φύλλο Excel, με τέσσερις στήλες και αποθηκεύτηκαν στο αρχείο `ppv_measurements.xlsx`

Οι στήλες A, B, C και D περιέχουν τα *πεδία* (fields) των μετρήσεων, δηλαδή τον αύξοντα αριθμό της μέτρησης, την απόσταση του δονησιογράφου από το μέτωπο ανατίναξης (Distance), την ποσότητα της εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα (Charge) και την ppv που μετρήθηκε, αντίστοιχα.

	A	B	C	D	E
1	No	Distance	Charge	ppv	
2	1	67	70	27.94	
3	2	244	288	9.14	
4	3	305	34	2.61	
5	4	67	171	35.56	
6	5	280	210	8.64	
7	6	305	128	7.11	
8	7	430	120	2.94	
9	8	365	150	3.51	
10	9	244	78	4.32	
11	10	430	145	3.29	
12	11	244	280	21.84	
13	12	330	130	2.95	
14	13	365	190	7.11	
15	14	280	110	4.57	
16	15	244	76	4.19	
17	16	140	110	21.34	
18	17	244	350	12.19	
19	18	400	125	2.96	
20	19	365	65	2.52	
21	20	140	65	9.65	
22	21	67	160	27.94	
23	22	140	195	12.7	
24	23	280	130	5.33	
25	24	244	227	8.38	
26	25	140	295	24.89	
27	26	140	330	26.88	
28	27	400	87	2.68	
29	28	244	290	8.67	
30	29	280	79	3.13	
31	30	244	395	12.95	
32	31	255	353	10.83	

Εικόνα 12.1. Δεδομένα δονημετρικού ελέγχου ανατινάξεων σε λατομείο. Απόσπασμα φύλλου Excel, αρχείο `ppv_measurements.xlsx`

Στην παράγραφο 10.6 *Βάση δεδομένων χρηστών του προγράμματος* παρουσιάσαμε τη βιβλιοθήκη `Opepyxl`, η οποία μας επιτρέπει να διαβάζουμε και να γράφουμε αρχεία του Microsoft Excel 2010 μορφής `.xlsx`, `.xlsm`, `.xltx` και `.xltm` μέσα από τον κώδικα της Python και δεν θα επεκταθούμε εδώ.

12.1.2 Αρχεία `.csv`

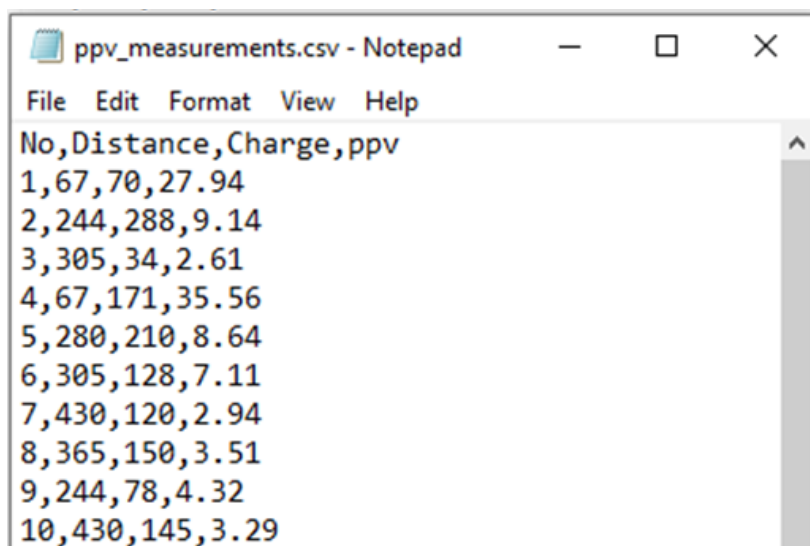
CSV είναι τα αρχικά των λέξεων Comma Separated Values και η ορολογία αυτή υποδηλώνει ότι οι τιμές των δεδομένων μας είναι χωρισμένες με ένα *κόμμα*.

Κάθε μέτρηση (record) γράφεται στη δική της γραμμή κειμένου.

Σε δική της γραμμή γράφεται και η επικεφαλίδα που χαρακτηρίζει το τι αντιπροσωπεύει κάθε στοιχείο του record, δηλαδή το *πεδίο* (field).

Για περισσότερες πληροφορίες σχετικά με τα αρχεία csv ανατρέξτε στο website: https://en.wikipedia.org/wiki/Comma-separated_values

Στην Εικόνα 12.2 παρουσιάζονται τα αποτελέσματα των δέκα (10) πρώτων μετρήσεων (για οικονομία χώρου στο βιβλίο), από τις 31 που καταγράφηκαν κατά τη διάρκεια του δονημετρικού ελέγχου των ανατινάξεων και παρουσιάστηκαν στην Εικόνα 12.1., με τη διαφορά ότι τώρα οι μετρήσεις καταγράφηκαν και οργανώθηκαν με τη βοήθεια του Notepad των Windows και αποθηκεύτηκαν στο αρχείο `ppv_measurements.csv`



Εικόνα 12.2. Δεδομένα δονημετρικού ελέγχου ανατινάξεων σε λατομείο. Απόσπασμα οθόνης Notepad των Windows, αρχείο `ppv_measurements.csv`

Σημειώστε ότι τα `.csv` αρχεία μπορούμε να τα ανοίξουμε και να τα επεξεργαστούμε με το Microsoft Excel, το οποίο επίσης μπορεί να μετατρέψει/αποθηκεύσει τα αρχεία `.xlsx` σε αρχεία `.csv` και αντιστρόφως.

Επίσης, είναι σημαντικό να σημειώσετε ότι, ανάλογα με τις επιλογές που έχουμε κάνει στα *Regional and Language Options* των Windows 10, είναι πιθανόν το Excel κατά τη μετατροπή των αρχείων `.xlsx` σε αρχεία `.csv` να αντικαταστήσει το κόμμα (,) με semicolon (;), δηλαδή το ελληνικό ερωτηματικό.

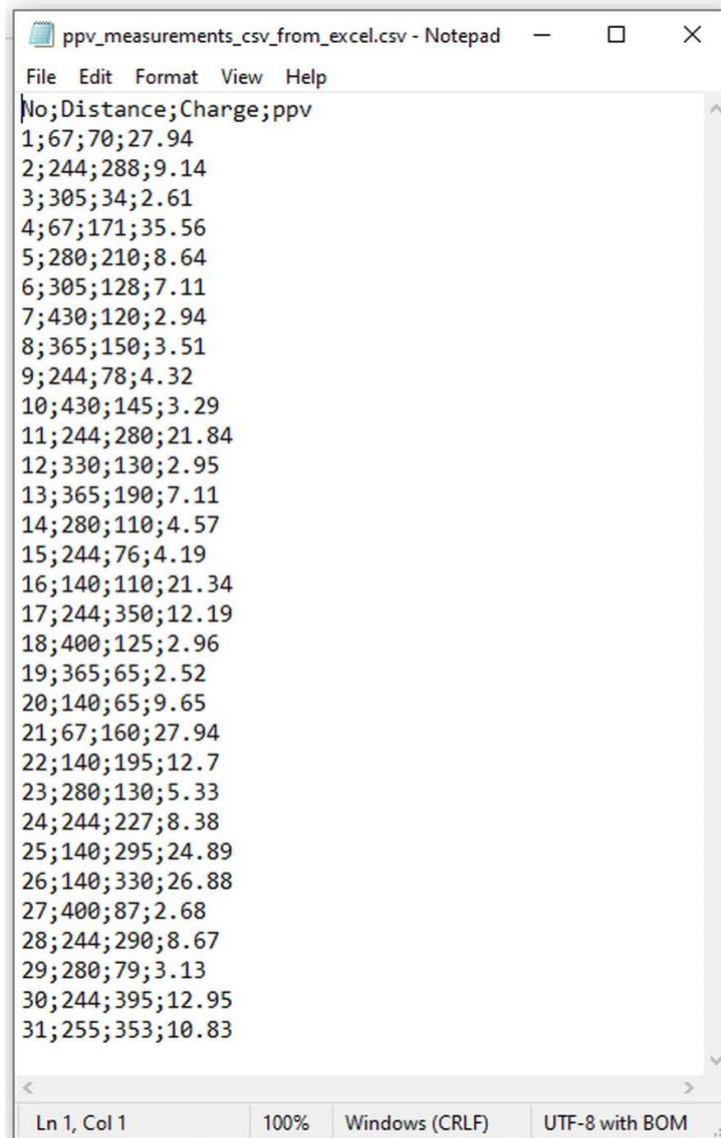
Αυτό συμβαίνει προκειμένου να διατηρηθεί η επιλογή στο Excel του χρήστη να θέτει το κόμμα αντί της τελείας για να χωρίζει το δεκαδικό μέρος ενός αριθμού ή το κόμμα για να χωρίζει σε χιλιάδες έναν αριθμό. Και σε αυτή την περίπτωση, δηλαδή, να χωρίζονται τα records με ;, το αρχείο θεωρείται ότι είναι αρχείο `.csv`

Στην Εικόνα 12.3 παρουσιάζονται τα αποτελέσματα των μετρήσεων του δονημετρικού ελέγχου, όπως αυτά προέκυψαν από τη μετατροπή/αποθήκευση του αρχείου `ppv_measurements.xlsx` στο αρχείο `ppv_measurements_csv_from_excel.csv` από το Excel που είναι εγκατεστημένο στον υπολογιστή του συγγραφέα.

Επίσης, σημειώστε ότι σε ένα αρχείο `.csv` οι τιμές, αντί να είναι χωρισμένες με κόμμα (,) ή semicolon (;), μπορεί να είναι οριοθετημένες με τον χαρακτήρα tab, δηλαδή με κενά, χρησιμοποιώντας το πλήκτρο Tab του πληκτρολογίου (tab-delimited).

Τα αρχεία `.csv` είναι πολύ διαδεδομένα στην πληροφορική, καθώς είναι ο πλέον συνηθισμένος τύπος αρχείων που δημιουργούνται από βάσεις δεδομένων, συμπεριλαμβανομένων αυτών που είναι διαθέσιμες στο Διαδίκτυο, ενώ ταυτόχρονα διαβάζονται από άλλα εμπορικά λογισμικά ή προγράμματα γραμμένα σε διάφορες γλώσσες προγραμματισμού, συμπεριλαμβανομένης της Python.

Η Python χειρίζεται τα αρχεία `.csv` με τη βοήθεια της βιβλιοθήκης CSV, η οποία είναι ένα εσωτερικό module-βιβλιοθήκη της Python και επομένως δεν χρειάζεται εγκατάσταση στον υπολογιστή, αλλά μόνο `import` στον κώδικα της Python.



Εικόνα 12.3. Δεδομένα δονημετρικού ελέγχου ανατινάξεων σε λατομείο. Οθόνη Notepad των Windows, αρχείο `ppv_measurements_csv_from_excel.csv`

Οι συναρτήσεις της βιβλιοθήκης CSV μας επιτρέπουν να διαβάζουμε το περιεχόμενο αρχείων `.csv` μέσα από τον κώδικά μας και να γράφουμε τα αποτελέσματα των υπολογισμών μας σε αρχεία `.csv`

Για πληροφορίες και τεκμηρίωση σχετικά με τη βιβλιοθήκη CSV ανατρέξτε στο website: <https://docs.python.org/3/library/csv.html>

Η συνάρτηση `csv.reader()`

Στη συνέχεια, θα δούμε πώς μπορούμε να διαβάσουμε τα δεδομένα των μετρήσεων του δονημετρικού ελέγχου που έχουμε αποθηκεύσει στο αρχείο `ppv_measurements.csv` μέσα από ένα πρόγραμμα Python.

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
import csv
fields = []
measurements = []
with open('ppv_measurements.csv', mode='r') as csv_file:
    # create a csv.reader object
    ppv_data = csv.reader(csv_file, delimiter=',')
    # extract field names from first row
    fields = next(ppv_data)
```

```

# extract data from each row one by one
for row in ppv_data:
    measurements.append(row)
# check total number of rows extracted
print('Total number of rows: {:d}'.format(ppv_data.line_num))
# printing the field names
print('Field names are: ', ", ".join(field for field in fields))
# printing first 5 rows
print('First 5 measurements are:')
for row in measurements[:5]:
    print('At distance {:s} m, ppv is {:s} mm/s for a charge of {:s} kg'
          .format(row[1], row[3], row[2]))

```

Καταρχάς, δημιουργούμε δύο μεταβλητές μορφής κενής λίστας, τη `fields`, στην οποία θα εκχωρήσουμε τις τιμές της πρώτης γραμμής των δεδομένων μας, δηλαδή τα πεδία των μετρήσεων και τη `measurements`, στην οποία θα εκχωρήσουμε τις τιμές των μετρήσεων.

Στη συνέχεια, ακολουθεί ένα μπλοκ εντολών `with`.

Το μπλοκ εντολών `with` μας επιτρέπει να ανοίγουμε αρχεία, να κάνουμε εργασίες επί του περιεχομένου των αρχείων και στη συνέχεια, όταν βγαίνουμε από το μπλοκ, το αρχείο κλείνει αυτόματα, χωρίς να χρειάζεται να γράψουμε μια εντολή `.close()`.

Έτσι, με τη γραμμή κώδικα-επικεφαλίδα του μπλοκ:

```
with open('ppv_measurements.csv', mode='r') as csv_file:
```

ανοίγουμε το αρχείο `ppv_measurements.csv` σε λειτουργία ανάγνωσης (`mode='r'`) και δημιουργούμε ένα αντικείμενο (object) του αρχείου που το ονομάζουμε `csv_file`, προκειμένου να δουλέψουμε στη συνέχεια με το αντικείμενο του αρχείου.

Με την πρώτη γραμμή κώδικα του μπλοκ εντολών του `with` καλούμε τη συνάρτηση `csv.reader()` και μετατρέπουμε το `csv_file`, του οποίου οι τιμές των δεδομένων είναι διαχωρισμένες με κόμματα (`delimiter=','`), σε ένα διατρήσιμο (iterable) αντικείμενο `csv.reader`, το οποίο ονοματίζουμε `ppv_data`.

Όντας πλέον διατρήσιμο το `ppv_data`, η συνάρτηση `next()` της Python στη γραμμή κώδικα:

```
fields = next(ppv_data)
```

διαβάζει-εξάγει το επόμενο στοιχείο-γραμμή του αντικείμενου-αρχείου `ppv_data`, που στην περίπτωση μας είναι η πρώτη γραμμή δεδομένων.

Η πρώτη γραμμή δεδομένων του αρχείου μας αποτελείται από τα ονόματα των πεδίων (`fields`) των μετρήσεων. Τα ονόματα αυτά μεταφέρονται και σώζονται στη, μέχρι τότε κενή, λίστα `fields`.

Εάν εμφανίσουμε στην οθόνη το περιεχόμενο της λίστας `fields` με την εντολή `print(fields)`:

```
['No', 'Distance', 'Charge', 'ppv']
```

θα διαπιστώσουμε ότι αυτή αποτελείται από στοιχεία κατηγορίας `str` που είναι τα ονόματα των πεδίων των μετρήσεων.

Για να διαβάσουμε-εξάγουμε τις υπόλοιπες σειρές του αντικείμενου-αρχείου `ppv_data`, που αποτελούν τις τιμές των μετρήσεων, χρησιμοποιούμε ένα μπλοκ βρόχου `for`:

```
for row in ppv_data:
    measurements.append(row)
```

με το οποίο διατρέχουμε τις υπόλοιπες σειρές του αρχείου και προσαρτούμε διαδοχικά τις τιμές καθεμιάς σειράς στη λίστα `measurements`.

Εάν εμφανίσουμε στην οθόνη το περιεχόμενο των πέντε (5) πρώτων στοιχείων της λίστας `measurements`, με την εντολή `print (measurements[:5])` :

```
[['1', '67', '70', '27.94'], ['2', '244', '288', '9.14'],  
 ['3', '305', '34', '2.61'], ['4', '67', '171', '35.56'],  
 ['5', '280', '210', '8.64']]
```

θα διαπιστώσουμε ότι κάθε γραμμή του αρχείου των μετρήσεων είναι και αυτή μια λίστα, η οποία αποτελείται από στοιχεία κατηγορίας `str` και τα οποία είναι οι τιμές των πεδίων της μέτρησης.

Στο σημείο αυτό του κώδικα έχουμε τελειώσει με την εξαγωγή και αποθήκευση των δεδομένων του αρχείου `ppv_measurements.csv`

Οι υπόλοιπες γραμμές κώδικα έχουν ως σκοπό να χρησιμοποιήσουν τα δεδομένα μας, που είναι τώρα ενσωματωμένα στις μεταβλητές-λίστες `fields` και `measurements`, για να εμφανίσουν χρηστικές πληροφορίες.

Για να εμφανίσουμε τον συνολικό αριθμό των γραμμών του αρχείου `ppv_measurements.csv` που διαβάστηκαν και εξήχθησαν από τη συνάρτηση `csv.reader()`, εφαρμόζουμε τη μέθοδο `.line_num` επί του αντικείμενου-αρχείου `ppv_data`, στην εντολή `print()` εντός του `format()`.

Για να εμφανίσουμε τα ονόματα των πεδίων, στην αντίστοιχη εντολή `print()` εφαρμόζουμε τη μέθοδο `.join()` επί κάθε στοιχείου της λίστας `fields`, έτσι ώστε να εμφανιστούν τα ονόματα διαχωριζόμενα μεταξύ τους με ένα κόμμα και ένα κενό διάστημα ("`,` ").

Για να εμφανίσουμε φορμαρισμένα τα αποτελέσματα των πέντε (5) πρώτων μετρήσεων, εφαρμόζουμε εντός του βρόχου `for` την εντολή `print()` :

```
print('At distance {:s} m, ppv is {:s} mm/s for a charge of {:s} kg'  
      .format(row[1], row[3], row[2]))
```

η οποία επιλέγει τις τιμές των μετρήσεων που θα εμφανιστούν εντός του κειμένου των αποτελεσμάτων:

```
Total number of rows: 32  
Field names are: No, Distance, Charge, ppv  
First 5 measurements are:  
At distance 67 m, ppv is 27.94 mm/s for a charge of 70 kg  
At distance 244 m, ppv is 9.14 mm/s for a charge of 288 kg  
At distance 305 m, ppv is 2.61 mm/s for a charge of 34 kg  
At distance 67 m, ppv is 35.56 mm/s for a charge of 171 kg  
At distance 280 m, ppv is 8.64 mm/s for a charge of 210 kg
```

Παρατήρηση 1: Εάν αντί του αρχείου `ppv_measurements.csv`, θέλουμε να χρησιμοποιήσουμε το αρχείο `ppv_measurements_csv_from_excel.csv`, το οποίο δημιουργήθηκε μέσω του Excel και έχει αντί για κόμμα το `;` ως διαχωριστικό των τιμών των πεδίων, τότε στη συνάρτηση `csv.reader()` θα αντικαταστήσουμε την παράμετρο `delimiter=','` με την `delimiter=';'`.

Στην περίπτωση αυτή, η δεύτερη γραμμή του output στην οθόνη θα έχει τη μορφή:

```
Field names are: i»¿No, Distance, Charge, ppv
```

δηλαδή εμφανίζεται η αλλόκοτη συμβολοσειρά `i»¿` μπροστά από τα ονόματα των πεδίων.

Αυτό οφείλεται στο ότι το αρχείο `ppv_measurements_csv_from_excel.csv` σώθηκε από το Excel κατά το πρότυπο UTF-8 with BOM (Byte Order Mark), όπως φαίνεται και στο κάτω δεξιά κουτάκι του αποσπάσματος της οθόνης του Notepad στην Εικόνα 12.3.

Παρατήρηση 2: Στα αρχεία `csv`, η ύπαρξη κενών γραμμών μετά την τελευταία γραμμή δεδομένων, δημιουργεί προβλήματα κατά την ανάγνωσή τους και πρέπει να αποφεύγονται.

Η συνάρτηση `csv.writer()`

Στη συνέχεια, θα δούμε πώς μπορούμε να γράψουμε και να αποθηκεύσουμε δεδομένα, τα οποία προκύπτουν μέσα από τον κώδικα ενός προγράμματος Python, σε ένα αρχείο `.csv`

Στον Πίνακα 12.1 περιλαμβάνονται πέντε (5) γνωστές εκρηκτικές ύλες, οι RDX (Cyclonite), NG (Nitroglycerine), HMX (Octogen), PETN (Pentaerithrytol Tetranitrate) και TNT (Trinitrotoluene), καθώς και ο αριθμός των ατόμων C, H, N και O που υπάρχουν στον χημικό τύπο ενός μορίου καθεμιάς από αυτές τις εκρηκτικές ύλες.

Πίνακας 12.1. Εκρηκτικές ύλες και αριθμός ατόμων στον χημικό τύπο τους.

Explosive	C_atoms	H_atoms	N_atoms	O_atoms
RDX	3	6	6	6
NG	3	5	3	9
HMX	4	8	8	8
PETN	5	8	4	12
TNT	7	5	3	6

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
import csv
fields = ['Explosive', 'C_atoms', 'H_atoms', 'N_atoms', 'O_atoms']
rows = [['RDX', 3, 6, 6, 6], ['NG', 3, 5, 3, 9],
        ['HMX', 4, 8, 8, 8], ['PETN', 5, 8, 4, 12],
        ['TNT', 7, 5, 3, 6]]
# name of csv file
filename = 'explosives_CHNO_atoms.csv'
# writing to csv file
with open(filename, mode='w', newline='') as csvfile:
    # create a csv.writer object
    explosives_data = csv.writer(csvfile)
    # write the fields
    explosives_data.writerow(fields)
    # write the data rows
    explosives_data.writerows(rows)
```

Καταρχάς, εκχωρούμε στη μεταβλητή `fields` τα ονόματα της πρώτης γραμμής του Πίνακα 12.1, που είναι και τα πεδία των δεδομένων μας, υπό μορφή λίστας.

Στη συνέχεια, εκχωρούμε στη μεταβλητή `rows`, υπό μορφή λίστας, τα περιεχόμενα των υπόλοιπων γραμμών του Πίνακα 12.1, της καθεμιάς γραμμής τα περιεχόμενα ως λίστα επίσης, δηλαδή λίστες μέσα σε λίστα.

Εκχωρούμε στη μεταβλητή `filename` το όνομα του αρχείου `.csv` στο οποίο θα γράψουμε και θα αποθηκεύσουμε τα δεδομένα του Πίνακα 12.1, δηλαδή το `explosives_CHNO_atoms.csv`

Στη συνέχεια, ακολουθεί ένα μπλοκ εντολών `with` όπου ανοίγουμε το αρχείο που μόλις ονομάτισαμε σε λειτουργία εγγραφής (`mode='w'`), ενώ με την παράμετρο `newline=''` δηλώνουμε ότι δεν θέλουμε να παρεμβάλλεται κενή γραμμή μετά την εγγραφή κάθε γραμμής δεδομένων. Το default είναι να παρεμβάλλεται μια κενή γραμμή. Το αντικείμενο (object) του αρχείου το ονομάζουμε `csvfile`, προκειμένου να δουλέψουμε στη συνέχεια με το αντικείμενο του αρχείου.

Με την επόμενη γραμμή κώδικα καλούμε τη συνάρτηση `csv.writer()` και μετατρέπουμε το `csvfile` σε ένα διατρέξιμο (iterable) αντικείμενο `csv.writer` και το ονοματίζουμε `explosives_data`.

Με τη γραμμή κώδικα:

```
explosives_data.writerow(fields)
```

εγγράφουμε το περιεχόμενο της μεταβλητής `fields`, δηλαδή την πρώτη γραμμή (πεδία τιμών) του Πίνακα 12.1.

Με τη γραμμή κώδικα:

```
explosives_data.writerow(rows)
```

εγγράφουμε το περιεχόμενο της μεταβλητής `rows`, δηλαδή μεμιάς, όλες τις υπόλοιπες γραμμές δεδομένων του Πίνακα 12.1.

Βγαίνοντας από το μπλοκ εντολών `with`, το αρχείο `explosives_CHNO_atoms.csv` κλείνει αυτόματα και αποθηκεύει τα δεδομένα του Πίνακα 12.1 σε αρχείο μορφής `.csv`

Σημειώστε ότι, παρόλο που εμείς γράψαμε στη μεταβλητή `rows` τον αριθμό των ατόμων C, H, N και O ως ακέραιους αριθμούς, εντός του αρχείου `explosives_CHNO_atoms.csv` έχουν αποθηκευτεί υπό μορφή συμβολοσειράς.

Εάν διαβάσουμε τα δεδομένα που έχουμε αποθηκεύσει στο αρχείο `explosives_CHNO_atoms.csv` με τη βοήθεια της συνάρτησης `csv.reader()` τα μεν `fields` θα εμφανιστούν στην οθόνη ως:

```
['Explosive', 'C_atoms', 'H_atoms', 'N_atoms', 'O_atoms']
```

τα δε `rows` ως:

```
[['RDX', '3', '6', '6', '6'], ['NG', '3', '5', '3', '9'], ['HMX', '4', '8', '8', '8'], ['PENT', '5', '8', '4', '12'], ['TNT', '7', '5', '3', '6']]
```

12.2 Η βιβλιοθήκη Pandas

Η βιβλιοθήκη Pandas (Python Data Analysis Library) είναι το πιο δημοφιλές πακέτο για τον χειρισμό δεδομένων σε προγράμματα Python. Η Pandas είναι ιδιαίτερα διαδεδομένη στην κοινότητα των προγραμματιστών που ασχολούνται με το data science και το data mining.

Όντας εξωτερική βιβλιοθήκη, πρέπει να την εγκαταστήσουμε στον υπολογιστή μας κατά τα γνωστά:

```
pip install pandas
```

Στο website της βιβλιοθήκης: <https://pandas.pydata.org> υπάρχει ένας εξαιρετικά λεπτομερής οδηγός χρήσης.

Η Pandas συνεργάζεται άψογα με τις άλλες δύο σημαντικές, και ήδη γνωστές σε μας, βιβλιοθήκες της Python, τη NumPy και τη Matplotlib.

Σε αυτή την παράγραφο θα χρησιμοποιήσουμε μερικές μόνο από τις συναρτήσεις της Pandas, για να επεξεργαστούμε τα δεδομένα των μετρήσεων του δονημετρικού ελέγχου, τα οποία είναι αποθηκευμένα στο αρχείο `ppv_measurements.csv`

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
import pandas as pd
ppv_data = pd.read_csv('ppv_measurements.csv')
print(ppv_data)
```

Με την πρώτη γραμμή κώδικα κάνουμε `import` τη βιβλιοθήκη `pandas` στον κώδικά μας και στη συνέχεια θα τη χρησιμοποιούμε με το παρατσούκλι `pd`.

Η δεύτερη γραμμή κώδικα καλεί τη συνάρτηση `read_csv()` της Pandas, η οποία διαβάζει δεδομένα από αρχεία τύπου `.csv`, προκειμένου να διαβάσει τα δεδομένα του αρχείου `ppv_measurements.csv` και να τα αποθηκεύσει σε ένα `DataFrame` που το ονομάζουμε `ppv_data`. Η εντολή `print()` που ακολουθεί εμφανίζει στην οθόνη το `DataFrame`:

	No	Distance	Charge	ppv
0	1	67	70	27.94
1	2	244	288	9.14
2	3	305	34	2.61
3	4	67	171	35.56
4	5	280	210	8.64
5	6	305	128	7.11
6	7	430	120	2.94
7	8	365	150	3.51
8	9	244	78	4.32
9	10	430	145	3.29
10	11	244	280	21.84
11	12	330	130	2.95
12	13	365	190	7.11
13	14	280	110	4.57
14	15	244	76	4.19
15	16	140	110	21.34
16	17	244	350	12.19
17	18	400	125	2.96
18	19	365	65	2.52
19	20	140	65	9.65
20	21	67	160	27.94
21	22	140	195	12.70
22	23	280	130	5.33
23	24	244	227	8.38
24	25	140	295	24.89
25	26	140	330	26.88
26	27	400	87	2.68
27	28	244	290	8.67
28	29	280	79	3.13
29	30	244	395	12.95
30	31	255	353	10.83

Βλέπουμε ότι η Pandas μας δημιούργησε έναν στοιχισμένο πίνακα (Frame) των δεδομένων μας (Data).

Επίσης, η Pandas είναι αρκετά *έξυπνη* ώστε να αντιληφθεί ότι η πρώτη γραμμή του αρχείου `ppv_measurements.csv` περιείχε τα πεδία των δεδομένων, ενώ οι υπόλοιπες γραμμές τις τιμές τους.

Τέλος, δημιούργησε έναν δείκτη αρίθμησης (index) μόνο για τις μετρήσεις, ο οποίος τοποθετήθηκε στην αριστερή πλευρά του Frame και ο οποίος, κατά τη λογική της Python, αρχίζει την αρίθμηση των μετρήσεων από το 0.

Η συνάρτηση `pd.read_csv()` παίρνει πολλές παραμέτρους. Εδώ εμείς τη χρησιμοποιήσαμε δεχόμενοι τις default τιμές αυτών των παραμέτρων, όπως για παράδειγμα ότι οι τιμές στο αρχείο που διαβάζει χωρίζονται με κόμματα (`sep=', '`) και ότι θα χρησιμοποιηθούν ως ονόματα των στηλών τα ονόματα των πεδίων που υπάρχουν στην πρώτη γραμμή του αρχείου που διαβάζει (`header=0`).

Προσθέτουμε τις παρακάτω γραμμές και τρέχουμε ξανά τον κώδικα:

```
ppv = pd.Series(ppv_data['ppv'])
print('The first 10 values of ppv are:\n', ppv[:10])
print('Max ppv = {:.2f} \nMin ppv = {:.2f} \nMedian ppv = {:.2f}'
      .format(ppv.max(), ppv.min(), ppv.median()))
print(ppv.describe())
```


Η πρώτη γραμμή κώδικα καλεί τη συνάρτηση της Pandas `Series()`, η οποία διαβάζει το περιεχόμενο της στήλης του DataFrame `ppv_data` με όνομα πεδίου `ppv` και δημιουργεί με τις τιμές της στήλης μια array του NumPy (`ndarray`), την οποία ονομάζουμε `ppv`.

Η επόμενη γραμμή κώδικα εμφανίζει στην οθόνη τις πρώτες δέκα (10) τιμές της `ppv`:

```
The first 10 values of ppv are:
```

```
0    27.94
1     9.14
2     2.61
3    35.56
4     8.64
5     7.11
6     2.94
7     3.51
8     4.32
9     3.29
```

```
Name: ppv, dtype: float64
```

Παρατηρούμε ότι οι τιμές μιας `Series` έχουν και αυτές ένα `index`, με τη βοήθεια του οποίου μπορούμε να καλέσουμε την αντίστοιχη τιμή του δεδομένου, όπως κάναμε γράφοντας `ppv[:10]` για να εμφανιστούν οι 10 πρώτες τιμές, δηλαδή οι τιμές με `index` από 0 έως 9.

Στο τέλος του πίνακα των τιμών εμφανίζεται το όνομα του `Series` και ο τύπος/κατηγορία των δεδομένων, που στην περίπτωσή μας είναι `float`, με ακρίβεια 64-bit `float` της NumPy.

Επίσης, πάνω σε μια `Series`, όπως είναι η `ppv`, μπορούμε να εφαρμόσουμε διάφορες μεθόδους.

Για παράδειγμα, η τρίτη γραμμή κώδικα εμφανίζει στην οθόνη τη μέγιστη (`max`), ελάχιστη (`min`) και διάμεση (`median`) τιμή των 31 μετρήσεων της `ppv` κατά τον δονημετρικό έλεγχο:

```
Max ppv = 35.56
```

```
Min ppv = 2.52
```

```
Median ppv = 8.38
```

Τέλος, η τελευταία εντολή `print(ppv.describe())` μας δίνει τη σύντομη *στατιστική ταυτότητα* (*descriptive statistics*) των 31 μετρήσεων:

```
count    31.000000
mean     10.927742
std       9.409108
min       2.520000
25%       3.400000
50%       8.380000
75%      12.825000
max      35.560000
```

```
Name: ppv, dtype: float64
```

Εάν θέλουμε να μάθουμε τα στοιχεία μιας μέτρησης που έχει κάποιο συγκεκριμένο χαρακτηριστικό, π.χ. τη μέτρηση με τη μεγαλύτερη τιμή της `ppv` (`ppv.max()`), εφαρμόζουμε επί του DataFrame τη μέθοδο `loc[]`

Με τη γραμμή κώδικα:

```
print('Max ppv measurement:\n', ppv_data.loc[ppv_data['ppv'] == ppv.max()])
```

θα εμφανιστεί στην οθόνη η μέτρηση που παρουσίασε τη μεγαλύτερη τιμή της `ppv`:

Max ppv measurement:				
	No	Distance	Charge	ppv
3	4	67	171	35.56

η οποία είναι η μέτρηση No 4.

Τα DataFrame και τα Series είναι δομές δεδομένων στην Pandas και υπό την έννοια αυτή είναι ανάλογα των δομών δεδομένων της Python που γνωρίσαμε στο Κεφάλαιο 08 Δομές δεδομένων, δηλαδή των λιστών, λεξικών και πλειάδων.

Ανηγμένη απόσταση

Στη μελέτη των μετρήσεων ενός δονημετρικού ελέγχου ανατινάξεων, ιδιαίτερη σημασία έχουν οι τιμές του λόγου $d / \sqrt{w}$ της Εξίσωσης [2.1], δηλαδή ο λόγος της απόστασης από το μέτωπο ανατίναξης στο σημείο της μέτρησης της ppv, προς την τετραγωνική ρίζα της μέγιστης ποσότητας της εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα. Ο λόγος αυτός ονομάζεται *Ανηγμένη Απόσταση* (Scaled Distance, SD) και έχει μονάδες $m / kg^{0.5}$.

Προσθέτουμε τις παρακάτω γραμμές και τρέχουμε ξανά τον κώδικα:

```
distance = pd.Series(ppv_data['Distance'])
charge = pd.Series(ppv_data['Charge'])
sd = (distance / (charge**0.5))
ppv_data['SD'] = sd
print(ppv_data[:10])
ppv_data.to_csv('Compiled_ppv_measurements.csv', index=False)
```

Με τις δύο πρώτες γραμμές κώδικα και με τη βοήθεια της συνάρτησης `Series()`, δημιουργούμε τα arrays `distance` και `charge` που περιέχουν τις τιμές των δεδομένων των πεδίων `Distance` και `Charge` αντίστοιχα.

Στην επόμενη γραμμή κώδικα υπολογίζουμε την ανηγμένη απόσταση και εκχωρούμε την τιμή της στη μεταβλητή `sd`, η οποία, προφανώς, θα είναι και αυτή της μορφής array.

Με τη γραμμή κώδικα:

```
ppv_data['SD'] = sd
```

δημιουργούμε ένα καινούριο πεδίο τιμών, στο υπάρχον DataFrame `ppv_data`, το όνομα του οποίου είναι `SD` και σε αυτό εκχωρούμε τις τιμές του array `sd`.

Με την εντολή `print()` εμφανίζουμε στην οθόνη τις πρώτες δέκα (10) τιμές του DataFrame `ppv_data` μετά την προσθήκη του νέου πεδίου `SD`:

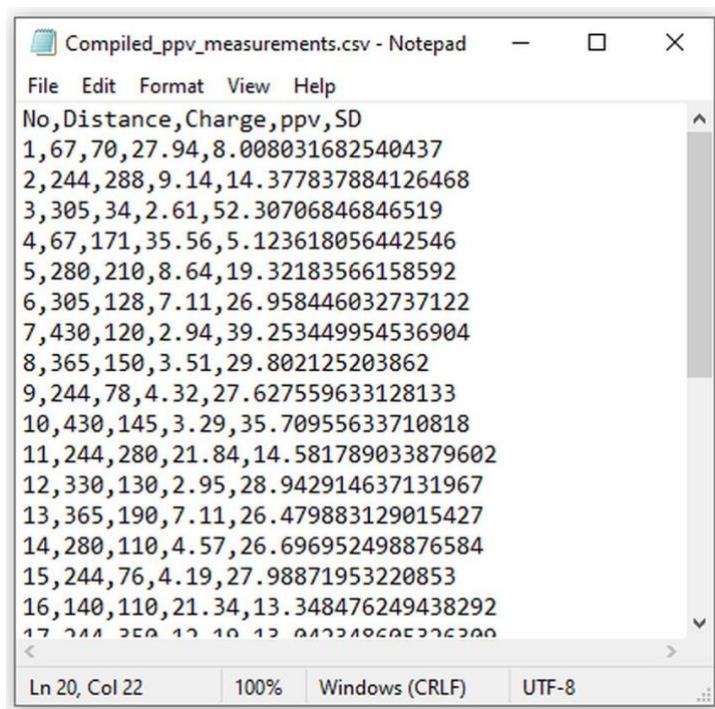
	No	Distance	Charge	ppv	SD
0	1	67	70	27.94	8.008032
1	2	244	288	9.14	14.377838
2	3	305	34	2.61	52.307068
3	4	67	171	35.56	5.123618
4	5	280	210	8.64	19.321836
5	6	305	128	7.11	26.958446
6	7	430	120	2.94	39.253450
7	8	365	150	3.51	29.802125
8	9	244	78	4.32	27.627560
9	10	430	145	3.29	35.709556

Στο τέλος του κώδικα, με την εντολή:

```
ppv_data.to_csv('Compiled_ppv_measurements.csv', index=False)
```

εφαρμόζουμε τη μέθοδο `to_csv()` πάνω στο DataFrame `ppv_data` με την οποία ζητάμε από την Pandas να δημιουργήσει ένα αρχείο τύπου `.csv` με όνομα `Compiled_ppv_measurements.csv` στο οποίο να γίνει εγγραφή των δεδομένων του DataFrame `ppv_data`, εκτός της στήλης του `index` (`index=False`). Το αρχείο θα σωθεί αυτόματα μετά την εγγραφή των δεδομένων.

Εάν ανοίξουμε το αρχείο `Compiled_ppv_measurements.csv` στο Notepad, αυτό θα εμφανιστεί όπως στην Εικόνα 12.4.



Εικόνα 12.4. Εμφάνιση του αρχείου `Compiled_ppv_measurements.csv` στο Notepad, μερική άποψη.

12.3 Οπτικοποίηση δεδομένων

Η *οπτικοποίηση των δεδομένων* (Data Visualization) των μετρήσεων μάς επιτρέπει να ανακαλύψουμε πιθανές τάσεις και διασυνδέσεις που υπάρχουν μεταξύ των τιμών των διαφόρων πεδίων των δεδομένων. Αυτό γενικά είναι δύσκολο ή ακατόρθωτο να συμβεί παρατηρώντας τα δεδομένα μας όταν είναι μορφοποιημένα υπό μορφή πινάκων.

Υπάρχουν πολλές και διάφορες τεχνικές για την οπτικοποίηση δεδομένων [Σχετικά βλέπε στο: https://en.wikipedia.org/wiki/Data_visualization]. Η επιλογή που θα κάνει ο χρήστης εξαρτάται από τη φύση των δεδομένων του και το επιδιωκόμενο με την οπτικοποίηση αποτέλεσμα, περιορίζεται δε από τη φαντασία του.

Στη συνέχεια, θα δουλέψουμε με τα δεδομένα των μετρήσεων του δονημετρικού ελέγχου που έχουμε αποθηκεύσει στο αρχείο `Compiled_ppv_measurements.csv`

Επίσης, θα χρησιμοποιήσουμε συναρτήσεις των βιβλιοθηκών Pandas και Matplotlib, τις οποίες πρέπει να τις κάνουμε `import` στον κώδικά μας.

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
import pandas as pd
import matplotlib.pyplot as plt
ppv_data = pd.read_csv('Compiled_ppv_measurements.csv')
ppv = pd.Series(ppv_data['ppv'])
distance = pd.Series(ppv_data['Distance'])
plt.scatter(distance, ppv)
```

```
plt.xlabel('Distance from blast [m]')
plt.ylabel('ppv [mm/s]')
plt.show()
```

Η συνάρτηση `read_csv()` θα διαβάσει τα δεδομένα του αρχείου `Compiled_ppv_measurements.csv` και θα τα αποθηκεύσει στο `DataFrame` `ppv_data`.

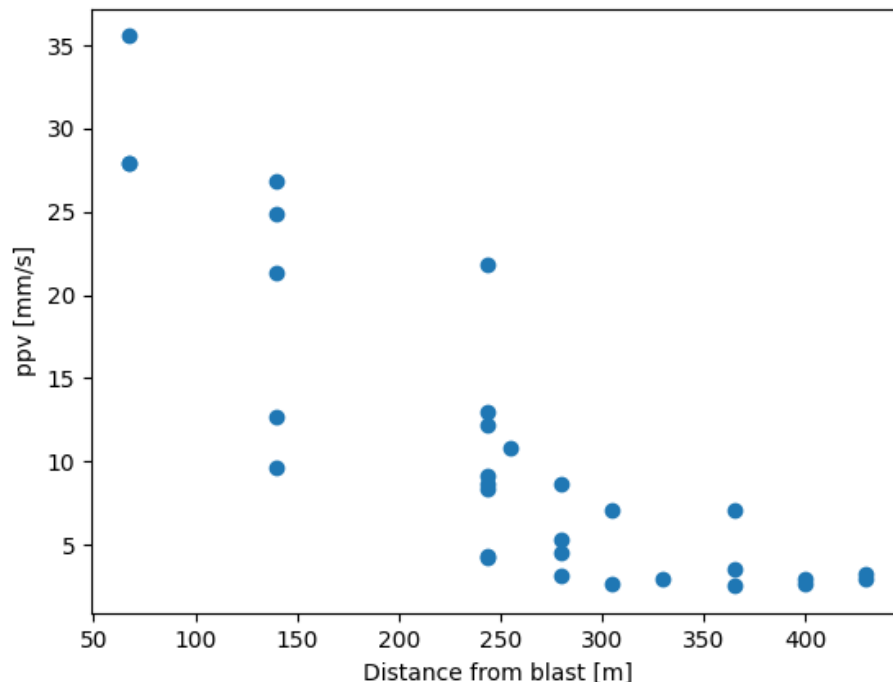
Στη συνέχεια, με τη βοήθεια της συνάρτησης `Series()`, δημιουργούμε τα arrays `ppv` και `distance` τα οποία περιέχουν τις τιμές των δεδομένων των πεδίων `ppv` και `Distance` αντίστοιχα.

Με τη βοήθεια της συνάρτησης `scatter()` δημιουργούμε το διάγραμμα της Εικόνας 12.5, δηλαδή ένα *διάγραμμα διασποράς* των τιμών των ζευγών (`distance`, `ppv`).

Παρατηρώντας το διάγραμμα της Εικόνας 12.5 διαπιστώνουμε ότι υπάρχει η γενική τάση να μειώνεται η `ppv` όσο μεγαλώνει η απόσταση από τη θέση ανατίναξης.

Παρατηρούμε, επίσης, ότι για την ίδια απόσταση από τη θέση ανατίναξης έχουμε περισσότερες της μίας τιμές της `ppv`. Το τελευταίο σημαίνει ότι η `ppv` δεν εξαρτάται μόνο από την απόσταση, αλλά και από κάποιο άλλο μέγεθος.

Από την Εξίσωση [2.1] της παραγράφου 02.2 *Μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους*, γνωρίζουμε ότι το μέγεθος αυτό είναι η μέγιστη ποσότητα εκρηκτικής ύλης που εκρήγνυται ταυτόχρονα.

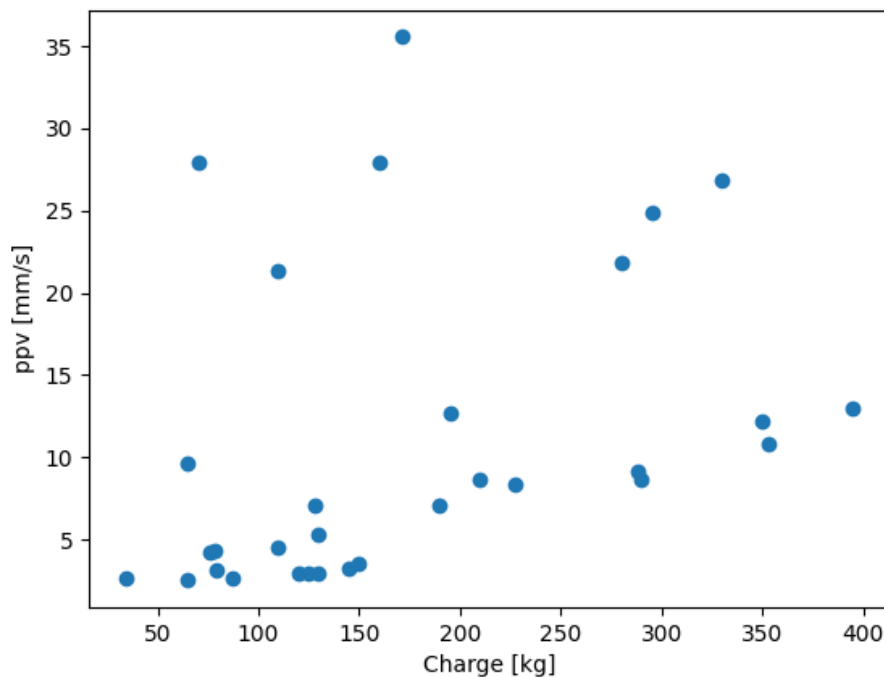


Εικόνα 12.5. Το διάγραμμα διασποράς των τιμών δεδομένων των πεδίων `Distance` και `ppv`.

Προσθέτουμε τις παρακάτω γραμμές και τρέχουμε ξανά τον κώδικα:

```
charge = pd.Series(ppv_data['Charge'])
plt.scatter(charge, ppv)
plt.xlabel('Charge [kg]')
plt.ylabel('ppv [mm/s]')
plt.show()
```

Στην οθόνη εμφανίζεται το διάγραμμα της Εικόνας 12.6, δηλαδή ένα διάγραμμα διασποράς των τιμών των ζευγών (`charge`, `ppv`), το οποίο, κατά την αντίληψη του συγγραφέα, δεν προσφέρεται για τη διατύπωση κάποιου προφανούς συσχετισμού.



Εικόνα 12.6. Το διάγραμμα διασποράς των τιμών δεδομένων των πεδίων Charge και ppv.

Συνεχίζοντας τους πειραματισμούς, προσθέτουμε τις παρακάτω γραμμές και τρέχουμε ξανά τον κώδικα:

```
sd = pd.Series(ppv_data['SD'])
plt.scatter(sd, ppv)
plt.xlabel('Scaled Distance [m/kg**0.5]')
plt.ylabel('ppv [mm/s]')
#plt.xscale('log')
#plt.yscale('log')
#plt.xlim(sd.min() - 1, 10**2 )
#plt.ylim(ppv.min() - 1, 10**2)
plt.show()
```

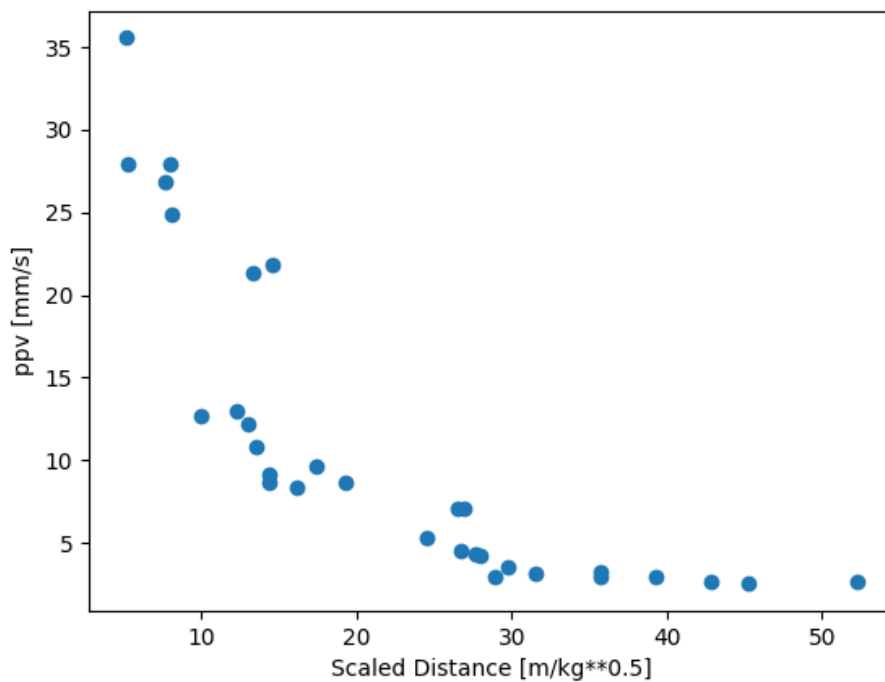
Στην οθόνη εμφανίζεται το διάγραμμα της Εικόνας 12.7, δηλαδή ένα διάγραμμα διασποράς των τιμών των ζευγών (sd, ppv), το οποίο πράγματι δείχνει μια τάση στη μεταβολή της ppv συναρτήσει της ανηγμένης απόστασης.

Η μεταβολή αυτή γίνεται πιο εναργής, εάν το διάγραμμα της Εικόνας 12.7 το ξανασχεδιάσουμε με *λογαριθμικούς άξονες*. Αυτό επιτυγχάνεται με τις εντολές:

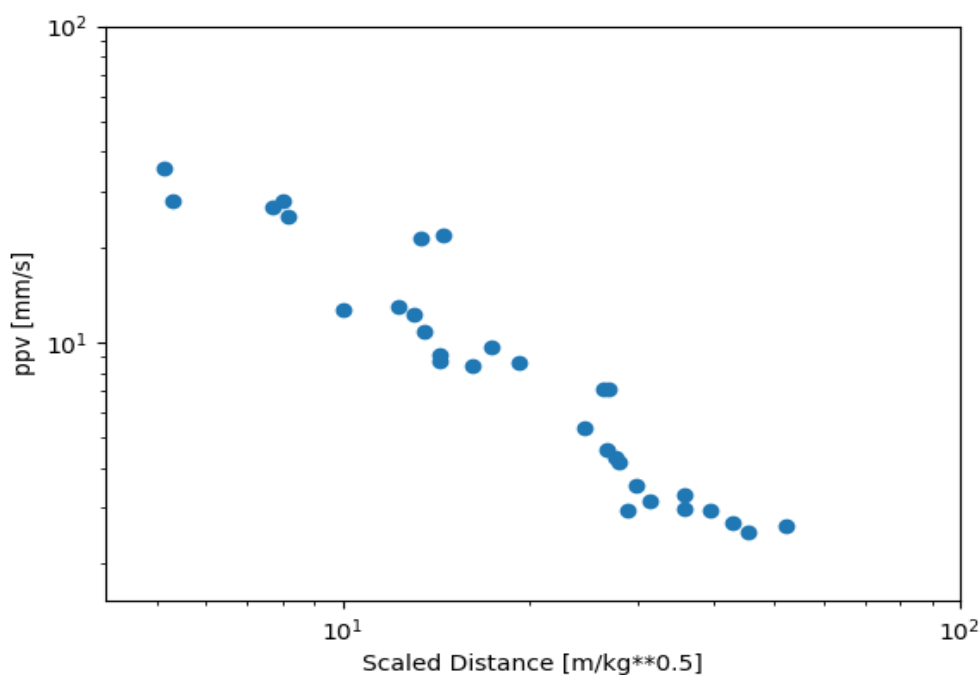
```
plt.xscale('log')
plt.yscale('log')
plt.xlim(sd.min() - 1, 10**2 )
plt.ylim(ppv.min() - 1, 10**2)
```

Σημείωση: Οι παραπάνω γραμμές εντολών περιλαμβάνονται στον κώδικα που σχεδίασε το διάγραμμα της Εικόνας 12.7 υπό μορφή γραμμών σχολίων. Μετατρέψτε τις γραμμές αυτές από σχόλια σε ενεργές εντολές της Python διαγράφοντας τα #. Οι δύο τελευταίες εντολές διαμορφώνουν τις κλίμακες των λογαριθμικών αξόνων του διαγράμματος.

Το νέο διάγραμμα που εμφανίζεται τώρα στην οθόνη παρουσιάζεται στην Εικόνα 12.8.



Εικόνα 12.7. Διάγραμμα διασποράς των τιμών δεδομένων των πεδίων SD και ppv.



Εικόνα 12.8. Το διάγραμμα διασποράς των τιμών δεδομένων των πεδίων SD και ppv με λογαριθμικούς άξονες.

Το διάγραμμα μεταβολής της ppv συναρτήσει της ανηγμένης απόστασης είναι ένα από τα βασικά διαγράμματα που δημιουργούμε κατά την επεξεργασία των μετρήσεων ενός δονημετρικού ελέγχου ανατινάξεων.

Το λογισμικό επεξεργασίας των μετρήσεων, που συνήθως παρέχουν οι κατασκευαστές μαζί με τους δονησιογράφους, έχει πάντα τη δυνατότητα κατασκευής ενός τέτοιου διαγράμματος, από τα δεδομένα των μετρήσεων που καταγράφονται σε αυτούς.

12.4 Καμπύλη προσαρμογής

Παρατηρώντας το διάγραμμα της Εικόνας 12.8 διαπιστώνουμε ότι τα σημεία των τιμών των ζευγών (*sd*, *ppv*) βρίσκονται συγκεντρωμένα σε μία ζώνη που περικλείεται από δύο ευθείες γραμμές, δηλαδή μία γραμμή άνω όριο και μία γραμμή κάτω όριο.

Επομένως, θα υπάρχει και μια γραμμή, ενδιάμεση, η οποία θα προκύπτει από μία εξίσωση που θα περιγράφει τη σχέση μεταξύ των τιμών των *sd* και *ppv*, βασισμένη στα πραγματικά δεδομένα των μετρήσεων του δονημετρικού ελέγχου, έστω κατά προσέγγιση.

Τη γραμμή αυτή ονομάζουμε καμπύλη προσαρμογής (curve fitting) [Σχετικά ανατρέξτε στο website: https://en.wikipedia.org/wiki/Curve_fitting].

12.4.1 Ο συντελεστής συσχέτισης Pearson *r*

Παρόλο που από το διάγραμμα της Εικόνας 12.8 είναι προφανές ότι υφίσταται μια συσχέτιση μεταξύ των τιμών των μεταβλητών *sd* και *ppv*, καλό είναι να εκτιμήσουμε τη συσχέτιση αυτή και ποσοτικά.

Αυτό επιτυγχάνεται με τη βοήθεια του *συντελεστή συσχέτισης Pearson *r** [Σχετικά βλέπε στο: https://en.wikipedia.org/wiki/Pearson_correlation_coefficient].

Ο συντελεστής συσχέτισης Pearson *r*, είναι ένας αδιάστατος δείκτης με τιμές από -1.0 έως 1.0, των τιμών αυτών συμπεριλαμβανομένων, ο οποίος προσδιορίζει τον βαθμό της γραμμικής συσχέτισης μεταξύ δύο συνόλων δεδομένων.

Είναι γενικά αποδεκτό ότι:

- | | |
|---|-----------------------------------|
| • Εάν το <i>r</i> είναι μικρότερο του ± 0.10 | Δεν υπάρχει συσχέτιση |
| • Εάν το <i>r</i> κυμαίνεται μεταξύ ± 0.10 έως ± 0.40 | Υπάρχει χαμηλή συσχέτιση |
| • Εάν το <i>r</i> κυμαίνεται μεταξύ ± 0.40 έως ± 0.70 | Υπάρχει μέτρια συσχέτιση |
| • Εάν το <i>r</i> κυμαίνεται μεταξύ ± 0.70 έως ± 0.99 | Υπάρχει υψηλή συσχέτιση |
| • Εάν το <i>r</i> είναι ± 1 | Υπάρχει τέλεια γραμμική συσχέτιση |

Σημειώστε ότι ο βαθμός γραμμικής συσχέτισης καθορίζεται από την απόλυτη τιμή του συντελεστή *r* και όχι από το πρόσημό του. Το πρόσημο του συντελεστή *r* καθορίζει μόνο το είδος της συσχέτισης, δηλαδή εάν είναι θετικό σημαίνει ότι αύξηση της τιμής μιας μεταβλητής οδηγεί σε αύξηση της άλλης και εάν είναι αρνητικό σημαίνει ότι αύξηση της τιμής μιας μεταβλητής οδηγεί σε μείωση της άλλης.

Στον κώδικα της παραγράφου 12.3 *Οπτικοποίηση δεδομένων* προσθέτουμε τις παρακάτω γραμμές εντολών:

```
from scipy.stats import pearsonr
r, p = pearsonr(sd, ppv)
print('Pearson correlation coefficient: r = {:.6f} p = {:.6f}'
      .format(r, p))
```

Με την πρώτη γραμμή κώδικα κάνουμε `import`, από την οικογένεια συναρτήσεων `stats` της βιβλιοθήκης `scipy`, τη συνάρτηση `pearsonr` στον κώδικά μας.

Για να υπολογίσουμε τον συντελεστή συσχέτισης Pearson *r* χρησιμοποιούμε τη συνάρτηση `pearsonr()` της `SciPy`, η οποία έχει δύο υποχρεωτικές παραμέτρους που είναι οι τιμές των μεταβλητών που θέλουμε να συσχετίσουμε, υπό μορφή `array`, και καμία προαιρετική.

Για την τεκμηρίωση της συνάρτησης `pearsonr()` ανατρέξτε στο website:
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html>

Έτσι, στη δεύτερη γραμμή κώδικα με την εντολή:

```
r, p = pearsonr(sd, prn)
```

υπολογίζουμε τον συντελεστή συσχέτισης r και τον συντελεστή p -value για τις τιμές της ανηγμένης απόστασης (sd) και prn που προέκυψαν από τις μετρήσεις του δονημετρικού ελέγχου.

Τον συντελεστή p -value εμείς δεν θα τον χρησιμοποιήσουμε, αλλά υποχρεωτικά τον υπολογίζει η συνάρτηση `pearsonr()`.

Αλλά θα αναφέρουμε ότι, ο συντελεστής p -value χρησιμοποιείται στον *στατιστικό έλεγχο υποθέσεων* (hypothesis testing) και ότι στην περίπτωση μας αντιστοιχεί στην πιθανότητα ο συσχετισμός μεταξύ των τιμών των μεταβλητών sd και prn να είναι τυχαίος.

Τέλος, εμφανίζουμε τους συντελεστές στην οθόνη:

```
Pearson correlation coefficients: r = -0.799026 p = 0.000000
```

Επομένως, τα αποτελέσματα της συσχέτισης των τιμών της ανηγμένης απόστασης και της prn , με τη βοήθεια της συνάρτησης `pearsonr()` της SciPy, δείχνουν ότι υπάρχει μια *υψηλή αρνητική συσχέτιση* και ότι ο συσχετισμός αυτός *δεν είναι τυχαίος*.

12.4.2 Χάραξη της καμπύλης προσαρμογής

Για την εύρεση των παραμέτρων της καμπύλης προσαρμογής, στις τιμές της ανηγμένης απόστασης (sd) και prn που προέκυψαν από τις μετρήσεις του δονημετρικού ελέγχου, θα χρησιμοποιήσουμε τη συνάρτηση `curve_fit()` της SciPy.

Η συνάρτηση `curve_fit()` ανήκει στην οικογένεια συναρτήσεων `optimize` της βιβλιοθήκης `scipy`, και θα πρέπει να την κάνουμε `import` στον κώδικά μας.

Στην απλή της χρήση, η συνάρτηση `curve_fit()` έχει τη μορφή:

```
port, pcov = curve_fit(f, xdata, ydata)
```

όπου:

$xdata$ είναι η ανεξάρτητη μεταβλητή που αντιστοιχεί στα μετρήσιμα δεδομένα. Στην περίπτωση μας είναι η ανηγμένη απόσταση (sd), η οποία ως γνωστόν προκύπτει από τα μετρήσιμα δεδομένα της απόστασης (*Distance*) και της ποσότητας της εκρηκτικής ύλης (*Charge*).

$ydata$ είναι η εξαρτημένη μεταβλητή των μετρήσιμων δεδομένων. Στην περίπτωση μας είναι η prn , η οποία ως γνωστόν προκύπτει από τα δεδομένα της απόστασης (*Distance*) και της ποσότητας της εκρηκτικής ύλης (*Charge*) σύμφωνα με την Εξίσωση [2.1], δηλαδή την ανηγμένη απόσταση (sd).

f είναι η *συνάρτηση-μοντέλο προσαρμογής* (model function), $f(x, \dots)$, που έχει ως πρώτη παράμετρο την ανεξάρτητη μεταβλητή και ως υπόλοιπες παραμέτρους τις παραμέτρους προσαρμογής. Τη συνάρτηση αυτή πρέπει να την ορίσουμε εμείς.

Εμείς όμως γνωρίζουμε ήδη ότι η Εξίσωση [2.1] συνδέει τα μετρήσιμα μεγέθη prn και sd με τη βοήθεια των παραμέτρων h -factor και b -factor.

Αλλά και να μην το γνωρίζαμε, από τη μορφή της τάσης μεταβολής των μεγεθών των δεδομένων prn και sd στην Εικόνα 12.7, προκύπτει ότι αυτά ακολουθούν μια καμπύλη μορφής $y = a \cdot x^b$.

Επομένως, μπορούμε να ορίσουμε τη συνάρτηση-μοντέλο ως:

```
def f(sd, h_factor, b_factor):
    return h_factor*(sd**b_factor)
```

Η συνάρτηση `curve_fit()` χρησιμοποιεί τη μέθοδο των *μη-γραμμικών ελαχίστων τετραγώνων* (non-linear least squares) για να προσαρμόσει τη συνάρτηση `f` στα δεδομένα μας.

Επίσης, δέχεται και μια σειρά παραμέτρων (optional parameters), τις οποίες δεν θα χρησιμοποιήσουμε, αλλά θα δεχθούμε τις default τιμές τους. Για την τεκμηρίωση της συνάρτησης ανατρέξτε στο website: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html

Καλώντας τη συνάρτηση `curve_fit()` για την εύρεση των παραμέτρων της καμπύλης προσαρμογής στις τιμές της ανηγμένης απόστασης (`sd`) και `ppv`:

```
popt, pcov = curve_fit(f, sd, ppv)
```

παίρνουμε ως αποτέλεσμα τα:

`popt` ένα array που περιέχει τις βέλτιστες τιμές των παραμέτρων προσαρμογής της συνάρτησης `f`, που στην περίπτωση μας είναι τα `h_factor` και `b_factor`.

`pcov` ένα 2-D array που είναι η εκτιμώμενη *συνδιακύμανση* του `popt`. Από αυτήν μπορούμε να υπολογίσουμε την τυπική απόκλιση του λάθους των βέλτιστων τιμών των παραμέτρων προσαρμογής (`perr`) με τη γραμμή κώδικα:

```
perr = np.sqrt(np.diag(pcov))
```

όπου `diag()` είναι η συνάρτηση της βιβλιοθήκης NumPy που εξαγεί τη διαγώνιο μιας array.

Συνοψίζοντας, ο κώδικας για την εύρεση των παραμέτρων της καμπύλης προσαρμογής στις τιμές της ανηγμένης απόστασης (`sd`) και `ppv` που προέκυψαν από τις μετρήσεις του δονημετρικού ελέγχου, είναι:

```
from scipy.optimize import curve_fit
import numpy as np
```

```
def f(sd, h_factor, b_factor):
    return h_factor * (sd**b_factor)
```

```
popt, pcov = curve_fit(f, sd, ppv)
print('popt =', popt)
print('h-factor = {:.2f}  b-factor = {:.2f}'.format(popt[0], popt[1]))
print('pcov =', pcov)
perr = np.sqrt(np.diag(pcov))
print('perr=', perr)
```

τον οποίο, εάν τον τρέξουμε, θα εμφανιστούν στην οθόνη τα αποτελέσματα:

```
popt = [202.47187665  -1.0658278 ]
h-factor = 202.47  b-factor = -1.07
pcov = [[ 1.49724100e+03 -3.26661346e+00]
        [-3.26661346e+00  7.55333523e-03]]
perr= [38.6941985   0.08690993]
```

Σημειώστε ότι στο array `popt` το πρώτο στοιχείο αντιστοιχεί στην παράμετρο προσαρμογής `h_factor` και το δεύτερο στην παράμετρο προσαρμογής `b_factor`, δηλαδή η αντιστοίχιση γίνεται με τη σειρά που έχουμε γράψει τις παραμέτρους προσαρμογής στον ορισμό της συνάρτησης `f`.

Στη συνέχεια, θα χαράξουμε την καμπύλη προσαρμογής πάνω στο διάγραμμα διασποράς των τιμών δεδομένων των πεδίων `SD` και `ppv` σε λογαριθμικούς άξονες, το οποίο ήδη έχουμε παρουσιάσει στην Εικόνα 12.8.

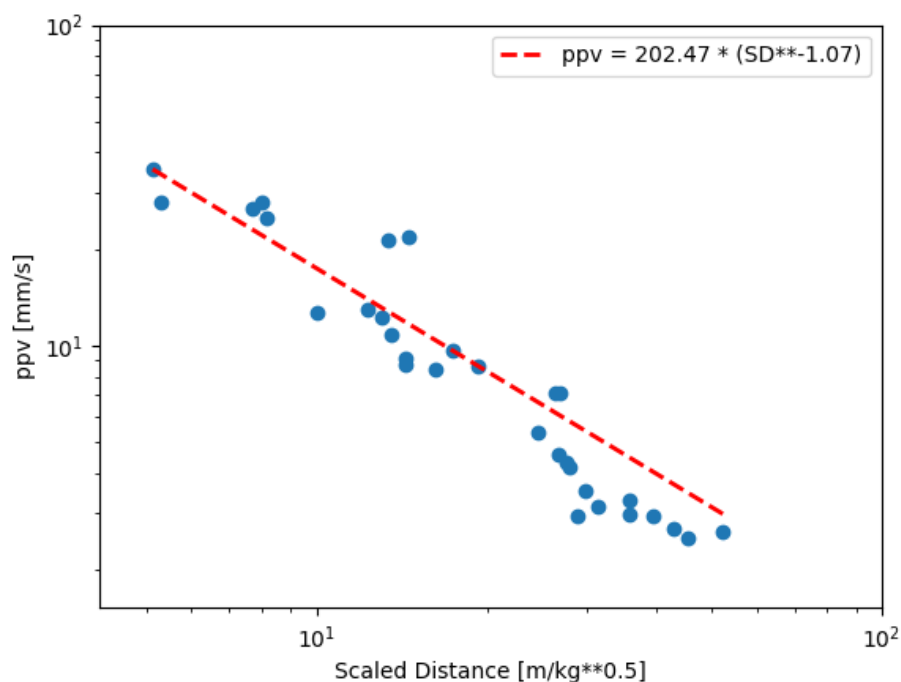
Προσθέτουμε τις παρακάτω γραμμές κώδικα:

```
# define a sequence of inputs between the smallest and largest known inputs
x_line = np.linspace(min(sd), max(sd), 100)
# calculate the output for the range
y_line = f(x_line, popt[0], popt[1])
# create a line plot for the f function
plt.plot(x_line, y_line, '--', lw=2, color='red',
         label='ppv = {:.2f} * (SD**{:.2f})'.format(popt[0], popt[1]))
plt.legend()
plt.show()
```

Με την πρώτη γραμμή κώδικα και με τη βοήθεια της συνάρτησης της NumPy `linspace()` δημιουργούμε μία array με 100 τιμές που ισαπέχουν, μεταξύ της μικρότερης και της μεγαλύτερης τιμής της ανηγμένης απόστασης (`sd`) και την εκχωρούμε στη μεταβλητή `x_line`.

Στη δεύτερη γραμμή κώδικα καλούμε τη συνάρτηση `f` με παραμέτρους την array `x_line` και τις τιμές των παραμέτρων προσαρμογής `popt[0]` και `popt[1]` που προέκυψαν από την εφαρμογή της συνάρτησης `curve_fit()` επί των δεδομένων των μετρήσεων του δονημετρικού ελέγχου. Δημιουργείται έτσι μία array, ίδιου μεγέθους με την `x_line`, που την ονοματίζουμε `y_line`.

Στην επόμενη γραμμή κώδικα, με τη συνάρτηση `plot()` της βιβλιοθήκης Matplotlib χαράζουμε την καμπύλη προσαρμογής, η οποία είναι μια κόκκινη, πάχους 2 points, διακεκομμένη γραμμή, και δημιουργούμε και μια λεζάντα στο διάγραμμα με την εξίσωση της καμπύλης προσαρμογής. Στην οθόνη εμφανίζεται το διάγραμμα της Εικόνας 12.9.



Εικόνα 12.9. Χάραξη της καμπύλης προσαρμογής των τιμών δεδομένων των πεδίων SD και ppv σε διάγραμμα με λογαριθμικούς άξονες.

12.4.3 Καμπύλη προσαρμογής και επίπεδο εμπιστοσύνης

Η εξίσωση της καμπύλης προσαρμογής που υπολογίσαμε στην παράγραφο 12.4.2 *Χάραξη της καμπύλης προσαρμογής* είναι μια μαθηματική σχέση η οποία μας επιτρέπει να εκτιμούμε, όπως και η Εξίσωση [2.1], τη μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (ppv), την οποία θα πρέπει να αναμένουμε σε ένα σημείο, που απέχει από το μέτωπο ανατίναξης απόσταση (`d`), όταν εκρήγνυται ταυτόχρονα ποσότητα εκρηκτικής ύλης (`w`):

$$ppv = 202.47 * (d / \sqrt{w})^{-1.07} \quad [12.1]$$

Η μεγάλη διαφορά μεταξύ της Εξίσωσης [2.1] και της Εξίσωσης [12.1] είναι ότι η δεύτερη δεν χρησιμοποιεί τιμές για τους τελεστές h-factor και b-factor, ανάλογα με τον τύπο της ανατίναξης, οι οποίες προέρχονται από τη βιβλιογραφία (Πίνακας 2.1), αλλά τιμές που προκύπτουν από μετρήσεις δονημετρικού ελέγχου στο πεδίο.

Επομένως, οι τιμές των τελεστών h-factor και b-factor της Εξίσωσης [12.1] *ενσωματώνουν* τις τοπικές συνθήκες του γεωλογικού υποβάθρου που περιβάλλει το μέτωπο ανατίναξης και τα χαρακτηριστικά του σχεδίου ανατίναξης που εφαρμόζεται στη συγκεκριμένη εκμετάλλευση.

Επίσης, είναι προφανές από το διάγραμμα της Εικόνας 12.9 ότι εφαρμόζοντας την Εξίσωση [12.1] για την εκτίμηση της ppv , άλλες φορές θα παίρνουμε τιμές της ppv που είναι μικρότερες από αυτές που μετρήθηκαν και άλλες φορές μεγαλύτερες.

Επειδή θέλουμε να είμαστε *συντηρητικοί* στην εκτίμηση της έντασης των δονήσεων, όταν εκπονούμε τη μελέτη ενός σχεδίου ανατίναξης, έχει επικρατήσει να υπολογίζουμε την καμπύλη προσαρμογής των τιμών της ανηγμένης απόστασης και της ppv , που βασίζεται στις μετρήσεις δονημετρικού ελέγχου, έτσι ώστε να ικανοποιείται ένα *επίπεδο ή συντελεστής εμπιστοσύνης* (confidence level).

Το σύνηθες επίπεδο εμπιστοσύνης που χρησιμοποιούμε είναι το 95%, που πρακτικά σημαίνει ότι η πιθανότητα, η τιμή της ppv που θα μετρηθεί με δονησιογράφο σε μια μελλοντική ανατίναξη να είναι μεγαλύτερη από την τιμή που θα υπολογιστεί από την εξίσωση της καμπύλης προσαρμογής, να είναι μικρότερη από 5%.

Στη συνέχεια, θα υπολογίσουμε τις παραμέτρους της εξίσωσης της καμπύλης προσαρμογής για επίπεδο εμπιστοσύνης 95% και θα τη χαράξουμε πάνω στο διάγραμμα της Εικόνας 12.9.

Όσα θα αναφέρουμε στη συνέχεια προαπαιτούν γνώσεις στατιστικής και πιθανοτήτων και αναφέρονται σε μέγεθος δείγματος, δηλαδή μετρήσεις ppv στην περίπτωση μας, μεγαλύτερο του τριάντα (30). Οι μετρήσεις μας είναι 31 και επομένως ικανοποιείται η συνθήκη.

Εάν το μέγεθος του δείγματος είναι μεγαλύτερο του 30, το *κεντρικό οριακό θεώρημα* λέει ότι για ένα μεγάλο τυχαίο δείγμα μεγέθους n , με μέσο μ και τυπική απόκλιση σ , ο δειγματικός μέσος $\bar{x}$ είναι κανονικά κατανοημένος με μέσο μ και τυπική απόκλιση $\sigma/\sqrt{n}$.

Από τον πίνακα της κανονικής κατανομής, προκύπτει ότι η τυπική κανονική τυχαία μεταβλητή z έχει 95% πιθανότητα να βρίσκεται εντός του εύρους -1.96 έως $+1.96$ [Σχετικά επί αυτού ανατρέξτε στο: <https://en.wikipedia.org/wiki/1.96>].

Συνεπώς, υπάρχει 95% πιθανότητα το $\bar{x}$ να πέσει εντός του διαστήματος:

$$\mu \pm 1.96 \sigma/\sqrt{n} \quad [12.2]$$

Για την υλοποίηση των παραπάνω συλλογισμών και γενικεύοντας για την περίπτωση που ο χρήστης επιθυμεί να χρησιμοποιήσει διαφορετικό επίπεδο εμπιστοσύνης, αντί του 95%, γράφουμε τον παρακάτω κώδικα:

```
from scipy.stats import norm
conf_level = 0.95
z = (1 + conf_level)/2
z_score = norm.ppf(z)
print('Z-score =', z_score)
```

Κάνουμε `import` στον κώδικά μας τη συνάρτηση `norm()` που ανήκει στην οικογένεια συναρτήσεων `stats` της βιβλιοθήκης `scipy`, στην πρώτη γραμμή κώδικα.

Στη συνέχεια, εκχωρούμε στη μεταβλητή `conf_level` το επιθυμητό επίπεδο εμπιστοσύνης.

Εφαρμόζοντας τη μέθοδο `norm.ppf(z)` επί της μεταβλητής z υπολογίζουμε την *εκατοστιαία σημειακή συνάρτηση* (percent point function), η οποία είναι η αντίστροφη της *αθροιστικής συνάρτησης κατανομής* (cumulative distribution function) και η τιμή της οποίας ονομάζεται *z-score*.

Για επίπεδο εμπιστοσύνης 95% στην οθόνη εμφανίζεται:

```
Z-score = 1.959963984540054
```

Σημειώστε ότι οι τιμές του `Z-score` για επίπεδα εμπιστοσύνης 80%, 90% και 99% είναι 1.28, 1.645 και 2.575 αντίστοιχα.

Στην παράγραφο 12.4.2 *Χάραξη της καμπύλης προσαρμογής*, με τη βοήθεια της συνάρτησης `curve_fit()` υπολογίσαμε και την τυπική απόκλιση του λάθους των βέλτιστων τιμών των παραμέτρων της καμπύλης προσαρμογής (`perr`).

Επομένως, με τις γραμμές κώδικα, που προσθέτουμε:

```
popt_95 = popt + z_score * perr  
print('h-factor(95%) = {:.2f}  b-factor(95%) = {:.2f}'  
      .format(popt_95[0], popt_95[1]))
```

εφαρμόζουμε τη Σχέση [12.2], ως προς την άνω τιμή της, στις τιμές των βέλτιστων παραμέτρων προσαρμογής της συνάρτησης `f` που έχουμε ήδη υπολογίσει και εμφανίζουμε στην οθόνη τις νέες τιμές των παραμέτρων της καμπύλης προσαρμογής, για επίπεδο εμπιστοσύνης 95%, δηλαδή των `h_factor` και `b_factor` για επίπεδο εμπιστοσύνης 95%:

```
h-factor(95%) = 278.31  b-factor(95%) = -0.90
```

Τέλος, προσθέτοντας τις γραμμές κώδικα:

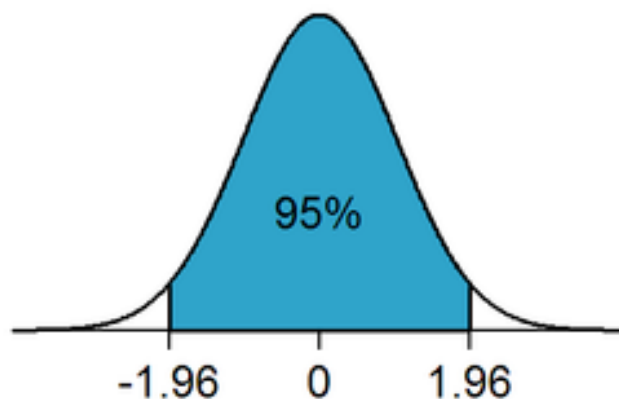
```
y_line_95 = f(x_line, popt_95[0], popt_95[1])  
plt.plot(x_line, y_line_95, '-', lw=2, color='red',  
         label='ppv = {:.2f} * (SD*{:.2f})'.format(popt_95[0], popt_95[1]))  
plt.legend()  
plt.show()
```

εμφανίζουμε στην οθόνη το διάγραμμα της Εικόνας 12.10, το οποίο περιέχει και την καμπύλη προσαρμογής για επίπεδο εμπιστοσύνης 95%.

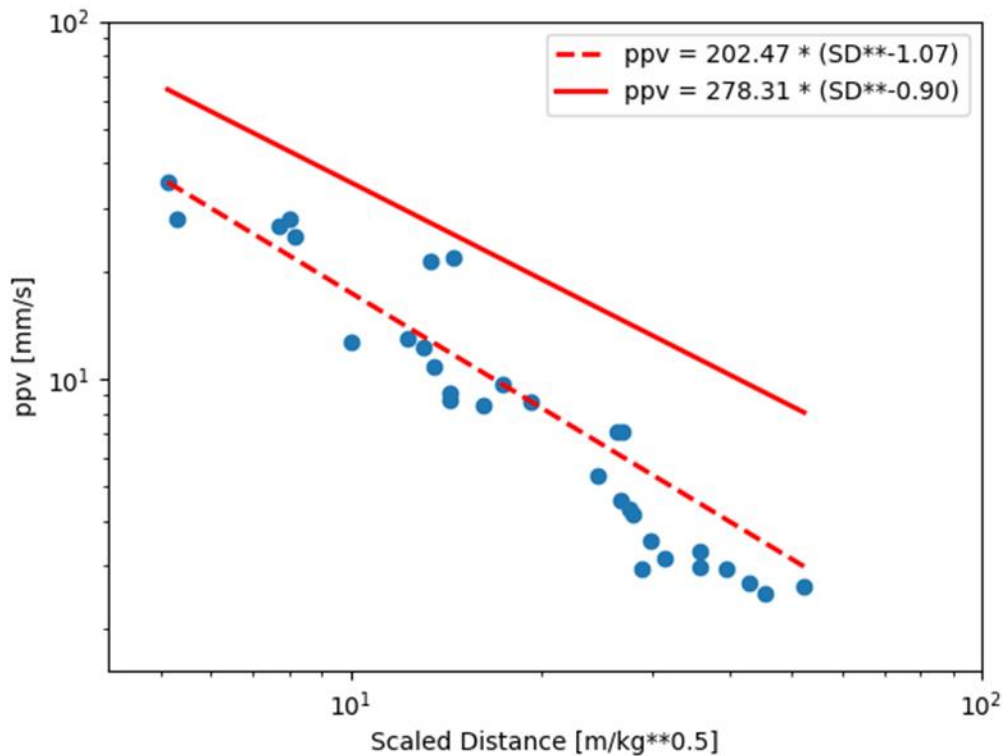
Ύστερα από όσα αναφέραμε, εάν επιθυμούμε οι υπολογισμοί μας να μας *δίνουν τη σιγουριά* ότι η `ppv` που θα εκτιμήσουμε θα πέφτει μέσα στο 95% των μετρήσεων του δονημετρικού ελέγχου, θα πρέπει να χρησιμοποιήσουμε την Εξίσωση [12.3], η οποία αντιστοιχεί στη συνεχή γραμμή του διαγράμματος της Εικόνας 12.10.

$$ppv = 278.32 * (d / \sqrt{w})^{-0.90} \quad [12.3]$$

Σημείωση: Γραφική ερμηνεία του 1.96.



Εικόνα 12.11. Το 95% της επιφάνειας κάτω από την κανονική κατανομή απέχει 1.96 φορές την τυπική απόκλιση από τον μέσο της κατανομής. [Πηγή: <https://en.wikipedia.org/wiki/1.96>].



Εικόνα 12.10. Χάραξη της καμπύλης προσαρμογής για επίπεδο εμπιστοσύνης 95% (συμπαγής κόκκινη γραμμή).

12.5 Ο κώδικας στο τέλος του Κεφαλαίου 12

Στο Κεφάλαιο αυτό αναπτύξαμε έναν κώδικα που είναι εξαιρετικά χρήσιμος για την επεξεργασία των μετρήσεων δονήσεων που έχουν καταγραφεί σε δονησιογράφους.

Ο ίδιος κώδικας, με κατάλληλη, κατά περίπτωση, τροποποίηση μπορεί να χρησιμοποιηθεί γενικότερα για την επεξεργασία μετρήσεων που έχουν πραγματοποιηθεί στο εργοτάξιο ή στο εργαστήριο.

Αφού διαγράψουμε από τον κώδικα τις γραμμές με τις οποίες πειραματιστήκαμε εξερευνώντας διάφορα θέματα για την κατανόηση του τρόπου λειτουργίας των βιβλιοθηκών της Python, τον παραθέτουμε στη συνέχεια ολοκληρωμένο:

```
""" Data Processing
This script reads data from a .csv file and then
processes the data using pandas, scipy and matplotlib functions.
Calculates:
1. Descriptive statistics
2. Pearson r correlation coefficients
3. Curve fitting parameters
4. Curve fitting parameters for a confidence level
Plots data & curve-fitting lines
The script, as listed, processes blast monitoring data, but
with proper changes may process any numeric data. """
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import pearsonr, norm
from scipy.optimize import curve_fit
import numpy as np
# Read blast monitoring data and process them with pandas
ppv_data = pd.read_csv('ppv_measurements.csv', sep=',')
distance = pd.Series(ppv_data['Distance'])
```

```

charge = pd.Series(ppv_data['Charge'])
ppv = pd.Series(ppv_data['ppv'])
sd = (distance/(charge**0.5))
ppv_data['SD'] = sd
sd = pd.Series(ppv_data['SD'])
ppv_data.to_csv('Compiled_ppv_measurements.csv', index=False)
print(ppv_data)
# Descriptive statistics for ppv
print('Descriptive statistics for ppv:')
print(ppv.describe())
# Pearson r for sd & ppv
r, p = pearsonr(sd, ppv)
print('Pearson correlation coefficients for (sd, ppv):\n'
      'r = {:.6f} p = {:.6f}'.format(r, p))
# Curve fitting

def f(sd, h_factor, b_factor):
    return h_factor*(sd**b_factor)

popt, pcov = curve_fit(f, sd, ppv)
print('Curve fitting parameters:')
print('h-factor = {:.2f} b-factor = {:.2f}'.format(popt[0], popt[1]))
perr = np.sqrt(np.diag(pcov))
# Curve fitting for a 95% confidence level
conf_level = 0.95
z = (1 + conf_level)/2
z_score = norm.ppf(z)
popt_conf = popt + z_score * perr
print('Curve fitting parameters for {}% conf. level:'
      .format(conf_level*100))
print('h-factor({}% ) = {:.2f} b-factor({}% ) = {:.2f}'
      .format(conf_level*100, popt_conf[0],
              conf_level*100, popt_conf[1]))
# Plot data & fitting curves
plt.scatter(sd, ppv)
plt.xlabel('Scaled Distance [m/kg**0.5]')
plt.ylabel('ppv [mm/s]')
plt.xscale('log', )
plt.yscale('log')
plt.xlim(sd.min() - 1, 10**2)
plt.ylim(ppv.min() - 1, 10**2)
# Define a sequence of inputs between the min/max sd inputs
x_line = np.linspace(min(sd), max(sd), 100)
# Calculate the output for the sequence
y_line = f(x_line, popt[0], popt[1])
# Line plot of the curve-fitted f function
plt.plot(x_line, y_line, '--', lw=2, color='red',
         label='ppv = {:.2f} * (SD**{:.2f})'.format(popt[0], popt[1]))
# Line plot of the curve-fitted f function
# with a confidence level
y_line_conf = f(x_line, popt_conf[0], popt_conf[1])
plt.plot(x_line, y_line_conf, '-', lw=2, color='red',
         label='ppv = {:.2f} * (SD**{:.2f})'.format(popt_conf[0],
                                                    popt_conf[1]))

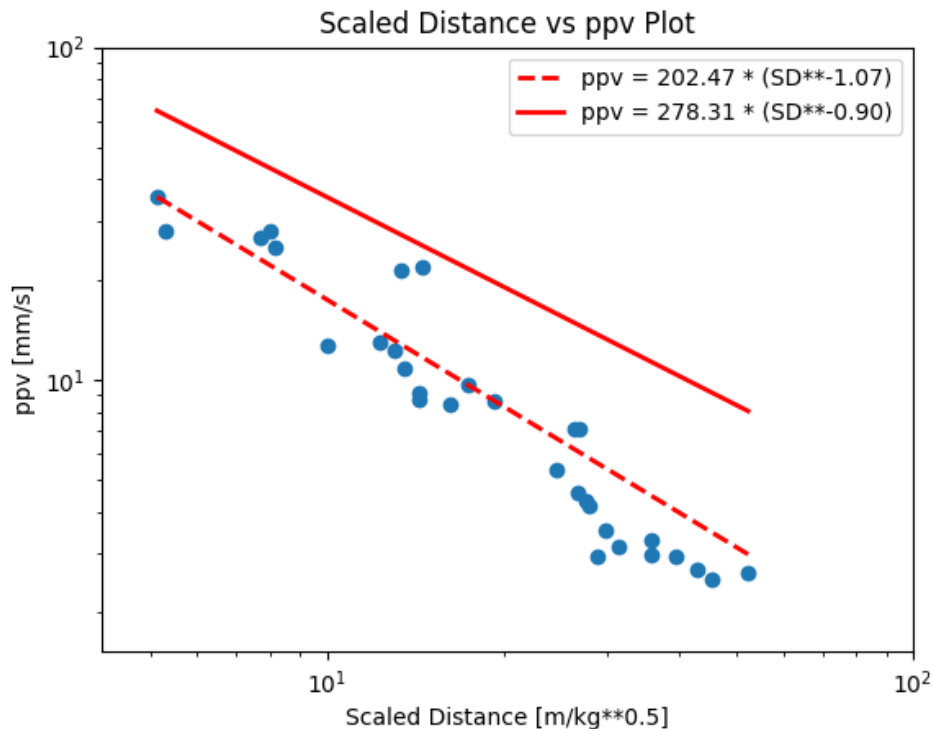
plt.title('Scaled Distance vs ppv Plot')
plt.legend()
plt.savefig('scaled_distance_vs_ppv_plot.png')
plt.show()

```

Εάν τρέξετε το παραπάνω script, στην οθόνη θα εμφανιστούν τα ακόλουθα αποτελέσματα:

	No	Distance	Charge	ppv	SD
0	1	67	70	27.94	8.008032
1	2	244	288	9.14	14.377838
2	3	305	34	2.61	52.307068
3	4	67	171	35.56	5.123618
4	5	280	210	8.64	19.321836
5	6	305	128	7.11	26.958446
6	7	430	120	2.94	39.253450
7	8	365	150	3.51	29.802125
8	9	244	78	4.32	27.627560
9	10	430	145	3.29	35.709556
10	11	244	280	21.84	14.581789
11	12	330	130	2.95	28.942915
12	13	365	190	7.11	26.479883
13	14	280	110	4.57	26.696952
14	15	244	76	4.19	27.988720
15	16	140	110	21.34	13.348476
16	17	244	350	12.19	13.042349
17	18	400	125	2.96	35.777088
18	19	365	65	2.52	45.272678
19	20	140	65	9.65	17.364863
20	21	67	160	27.94	5.296815
21	22	140	195	12.70	10.025608
22	23	280	130	5.33	24.557625
23	24	244	227	8.38	16.194849
24	25	140	295	24.89	8.151115
25	26	140	330	26.88	7.706746
26	27	400	87	2.68	42.884501
27	28	244	290	8.67	14.328173
28	29	280	79	3.13	31.502461
29	30	244	395	12.95	12.276972
30	31	255	353	10.83	13.572281

```
Descriptive statistics for ppv:
count    31.000000
mean     10.927742
std       9.409108
min       2.520000
25%       3.400000
50%       8.380000
75%      12.825000
max      35.560000
Name: ppv, dtype: float64
Pearson correlation coefficients for (sd, ppv):
r = -0.799026  p = 0.000000
Curve fitting parameters:
h-factor = 202.47  b-factor = -1.07
Curve fitting parameters for 95.0% conf. level:
h-factor(95.0%) = 278.31  b-factor(95.0%) = -0.90
```



Παρατήρηση 1: Το αρχείο δεδομένων (το `ppv_measurements.csv` στην περίπτωση μας) πρέπει να βρίσκεται στο ίδιο directory με το script της Python. Σε διαφορετική περίπτωση γράφουμε το όνομα του αρχείου με το πλήρες path που έχει στον σκληρό δίσκο του υπολογιστή μας.

Παρατήρηση 2: Εάν τα data στο .csv αρχείο δεδομένων χωρίζονται με ; αντικαταστήστε την παράμετρο `sep=' , '` με τη `sep=' ; '`.

Παρατήρηση 3: Εάν επιθυμούμε να υπολογίσουμε και να χαράξουμε την καμπύλη προσαρμογής με διαφορετικό επίπεδο εμπιστοσύνης από το 95%, τροποποιούμε κατάλληλα την εντολή `conf_level = 0.95` γράφοντας το επιθυμητό επίπεδο εμπιστοσύνης σε δεκαδική μορφή.

Παρατήρηση 4: Εάν δεν επιθυμούμε να σχεδιάσουμε το διάγραμμα των δεδομένων μας και τις καμπύλες προσαρμογής σε λογαριθμικούς άξονες, μετατρέπουμε σε σχόλια τις γραμμές κώδικα:

```
plt.xscale('log')
plt.yscale('log')
```

και προσαρμόζουμε το εύρος των αξόνων του διαγράμματος στα δεδομένα μας, τροποποιώντας κατάλληλα τις εντολές:

```
plt.xlim(sd.min() - 1, 10**2)
plt.ylim(ppv.min() - 1, 10**2)
```

Το script του κώδικα επεξεργασίας δεδομένων βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_12_Data_Processing_Blast_Monitor_Measurements.py` μαζί με το αρχείο `ppv_measurements.csv`

13 Βελτιστοποίηση

13.1 Γενικά

Με τον όρο *μαθηματική βελτιστοποίηση* (mathematical optimization) ή απλά *βελτιστοποίηση* (optimization) γενικά εννοούμε την επιλογή της κατάλληλης τιμής μίας ή περισσότερων μεταβλητών, με βάση κάποιο κριτήριο ή κριτήρια, από ένα σύνολο εναλλακτικών επιλογών [Σχετικά με τη βελτιστοποίηση ανατρέξτε στο website: https://en.wikipedia.org/wiki/Mathematical_optimization].

Στην πράξη αυτό ισοδυναμεί με την εύρεση των τιμών των μεταβλητών $x_1, \dots, x_n$ που *ελαχιστοποιούν* (minimize) ή *μεγιστοποιούν* (maximize) την τιμή της συνάρτησης $f(x_1, \dots, x_n)$.

Επειδή είναι πιο εύκολη η επίλυση προβλημάτων ελαχιστοποίησης, συνήθως τα προβλήματα μεγιστοποίησης ανάγονται σε προβλήματα ελαχιστοποίησης, λαμβάνοντας υπόψη ότι:

$$\max[f(x_1, \dots, x_n)] = \min[-f(x_1, \dots, x_n)] \quad [13.1]$$

Το αντικείμενο της βελτιστοποίησης είναι αρκετά ευρύ και το πεδίο εφαρμογής καλύπτει πολλούς κλάδους της επιστήμης και της τεχνολογίας. Στο Κεφάλαιο αυτό θα παρουσιάζουμε ορισμένες μόνο περιπτώσεις βελτιστοποίησης, με την έμφαση να δίνεται στον *Γραμμικό Προγραμματισμό*.

13.2 Ελαχιστοποίηση συνάρτησης μορφής $f(x)$

Στην παράγραφο 11.2.1 *Εξισώσεις με έναν άγνωστο* λύσαμε τη δευτεροβάθμια εξίσωση:

$$x^2 + 2x - 8 = 0 \quad [11.1]$$

και βρήκαμε ότι οι ρίζες είναι -4 και 2.

Στη συνέχεια, θα βρούμε την τιμή του x που ελαχιστοποιεί τη συνάρτηση:

$$f(x) = x^2 + 2x - 8 \quad [13.2]$$

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from scipy.optimize import minimize_scalar
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x**2 + 2*x - 8

min_f = minimize_scalar(f)
print('f(x) minimum, x = {:.4f}'.format(min_f.x))
print('f(x) minimum value = {:.4f}'.format(min_f.fun))
# Plot function & min point
x = np.linspace(-10, 10, 100)
plt.plot(x, f(x), label='f(x) = x**2 + 2*x - 8')
plt.scatter(min_f.x, f(min_f.x), color='red', marker='o',
            label='min f(x), x = {:.4f}'.format(min_f.x))
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.show()
```

Με την πρώτη γραμμή κώδικα κάνουμε import, από την οικογένεια συναρτήσεων optimize της βιβλιοθήκης scipy, τη συνάρτηση minimize_scalar στον κώδικά μας.

Η `minimize_scalar()` ελαχιστοποιεί μια βαθμωτή (scalar) συνάρτηση μιας μεταβλητής εφαρμόζοντας τη μέθοδο του *Brent*, που είναι η default method της συνάρτησης, για να βρει το τοπικό ελάχιστο.

Για την τεκμηρίωση της συνάρτησης `minimize_scalar()` ανατρέξτε στο website: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize_scalar.html

Στη συνέχεια, ορίζουμε τη συνάρτηση f κατά τα γνωστά, και με τη γραμμή κώδικα:

```
min_f = minimize_scalar(f)
```

καλούμε τη `minimize_scalar()` προκειμένου να βρει το ελάχιστο της f .

Η τιμή της μεταβλητής x , που καθιστά τη συνάρτηση f ελάχιστη (`min_f.x`), και η ελάχιστη τιμή της συνάρτησης f (`min_f.fun`), που εμφανίζονται στην οθόνη με τις εντολές `print()`, είναι:

```
f(x) minimum, x = -1.0000  
f(x) minimum value = -9.0000
```

Οι υπόλοιπες εντολές του κώδικα σχεδιάζουν, κατά τα γνωστά, την καμπύλη της συνάρτησης και τοποθετούν επί αυτής το σύμβολο του σημείου του ελαχίστου, όπως φαίνεται στην Εικόνα 13.1.

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο: `Chapter_13_Function_Minimum.py`

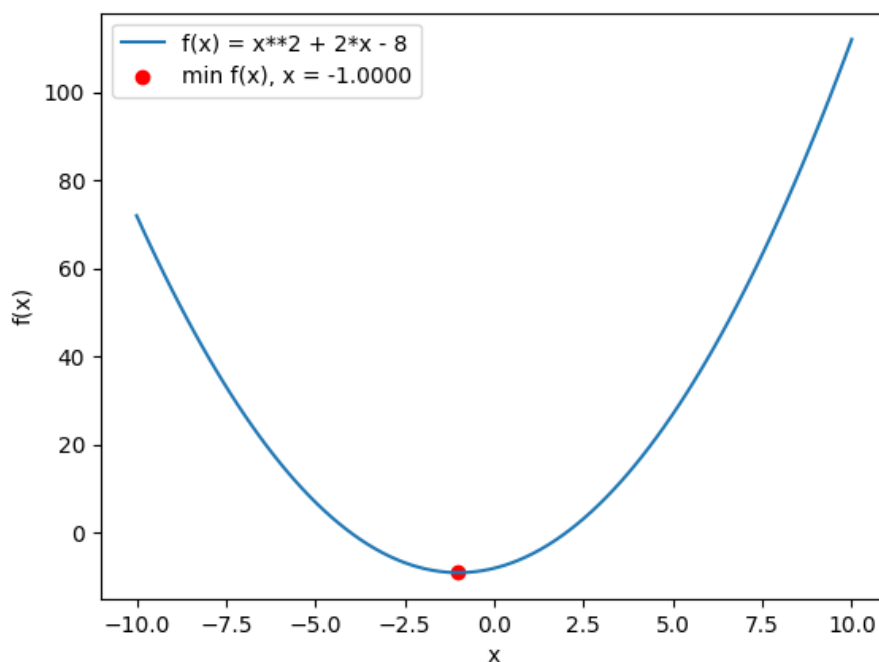
Η συνάρτηση [13.2], όπως φαίνεται και από την καμπύλη της, παρουσιάζει ένα και μοναδικό ελάχιστο, το οποίο είναι ταυτόχρονα τοπικό (local) και καθολικό (global) ελάχιστο.

Υπάρχουν, όμως, συναρτήσεις, όπως για παράδειγμα η:

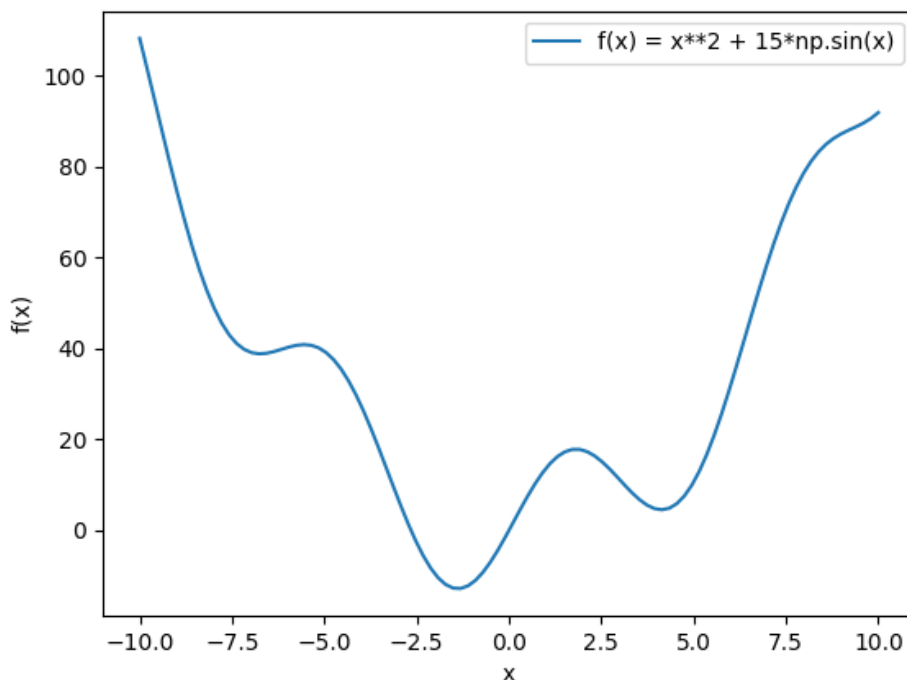
$$f(x) = x^2 + 15 * \sin(x) \quad [13.3]$$

οι οποίες εκτός από το καθολικό ελάχιστο, έχουν ένα ή και περισσότερα του ενός τοπικά ελάχιστα.

Πράγματι, από το σχήμα της καμπύλης της Συνάρτησης [13.3], που παρουσιάζεται στην Εικόνα 13.2, διαπιστώνουμε ότι εκατέρωθεν του καθολικού ελαχίστου υπάρχουν δύο τοπικά ελάχιστα.



Εικόνα 13.1. Η καμπύλη και το σημείο ελαχίστου της συνάρτησης $f(x) = x^2 + 2x - 8$.



Εικόνα 13.2. Η καμπύλη της συνάρτησης $f(x) = x^2 + 15 * \sin(x)$.

Εάν εφαρμόσουμε τη συνάρτηση `minimize_scalar()` της Scipy για την εύρεση του ελαχίστου της $f(x)$, τότε μόνο κατά τύχη αυτή θα μας προσδιορίσει το καθολικό ελάχιστο της $f(x)$, που αντιστοιχεί σε μια τιμή του x κάπου μεταξύ -2.5 και 0 .

Για να βρούμε το καθολικό ελάχιστο της $f(x)$ θα χρησιμοποιήσουμε τη συνάρτηση `basinhopping()` της Scipy, που εφαρμόζει τον αλγόριθμο *basin-hopping* επί της $f(x)$, και την οποία πρέπει να κάνουμε `import` στον κώδικά μας.

Για την τεκμηρίωση της συνάρτησης `basinhopping()` ανατρέξτε στο website:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.basinhopping.html>

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from scipy.optimize import basinhopping, minimize_scalar
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x**2 + 15*np.sin(x)

# Plot f(x)
x = np.linspace(-10, 10, 100)
plt.plot(x, f(x), label="f(x) = x**2 + 15*np.sin(x)")
# Global optimization
min_f_global = basinhopping(f, 0)
print('f(x) global minimum, x = {:.4f}'.format(min_f_global.x[0]))
print('f(x) value @ global minimum = {:.4f}'.format(min_f_global.fun))
plt.scatter(min_f_global.x, f(min_f_global.x), color='red', marker='o',
            label='Global min f(x), x = {:.4f}'.format(min_f_global.x[0]))
# Local optimization #1
min_f_local_1 = minimize_scalar(f, method='bounded', bounds=(-10, -2.5))
print('f(x) local minimum #1, x = {:.4f}'.format(min_f_local_1.x))
print('f(x) value @ local minimum #1 = {:.4f}'.format(min_f_local_1.fun))
```

```
plt.scatter(min_f_local_1.x, min_f_local_1.fun, color='black', marker='P',
            label='Local min f(x) #1, x = {:.4f}'.format(min_f_local_1.x))
# Local optimization #2
min_f_local_2 = minimize_scalar(f, method='bounded', bounds=(0, 7))
print('f(x) local minimum #2, x = {:.4f}'.format(min_f_local_2.x))
print('f(x) value @ local minimum #2 = {:.4f}'.format(min_f_local_2.fun))
plt.scatter(min_f_local_2.x, min_f_local_2.fun, color='black', marker='D',
            label='Local min f(x) #2, x = {:.4f}'.format(min_f_local_2.x))
# Plot decoration
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.show()
```

Καταρχάς, ορίζουμε τη συνάρτηση $f(x)$ και στη συνέχεια, επειδή αυτό είναι μια καλή πρακτική, σχεδιάζουμε την καμπύλη της συνάρτησης για να δούμε πού περίπου βρίσκονται το καθολικό και τα τοπικά ελάχιστα (Εικόνα 13.2).

Στη συνέχεια, με τη γραμμή κώδικα:

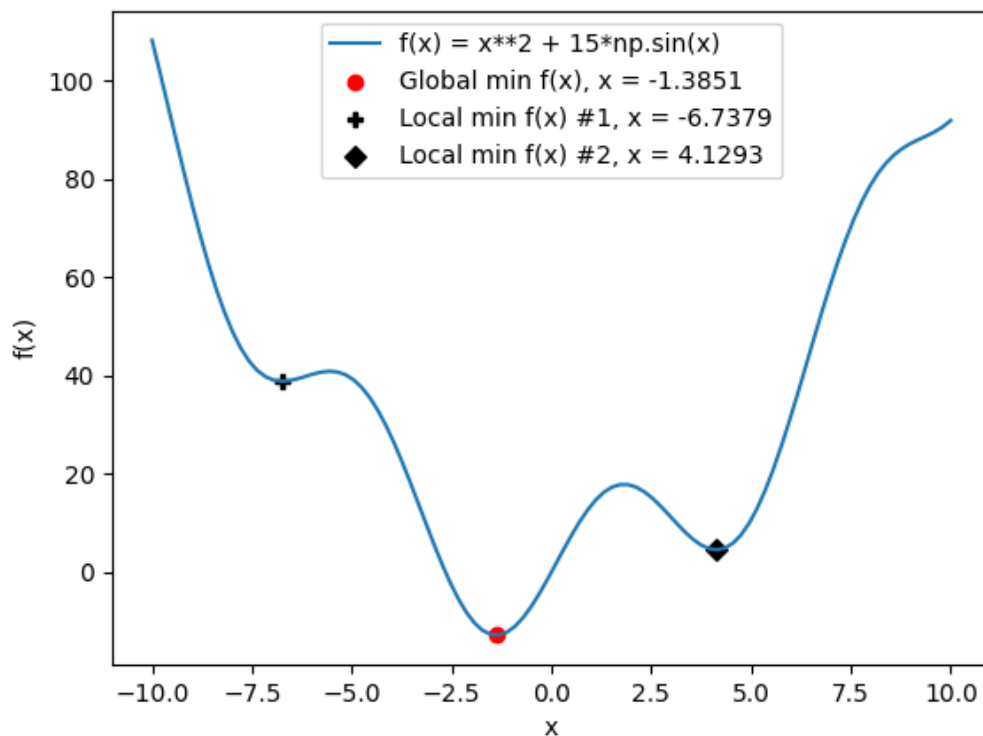
```
min_f_global = basinhopping(f, 0)
```

καλούμε την `basinhopping()` προκειμένου να μας προσδιορίσει το καθολικό ελάχιστο της συνάρτησης f , δίνοντας μια *αρχική εκτίμηση* (initial guess) ότι αυτό θα συμβαίνει για την τιμή $x = 0$.

Οι εντολές που ακολουθούν στον κώδικα, εμφανίζουν στην οθόνη την τιμή του καθολικού ελαχίστου (`min_f_global.x[0]`) και την τιμή της συνάρτησης στο καθολικό ελάχιστο (`min_f_global.fun`):

```
f(x) global minimum, x = -1.3851
f(x) value @ global minimum = -12.8236
```

και τοποθετούν το σημείο του καθολικού ελαχίστου, με μια κόκκινη κουκίδα, επί της καμπύλης της συνάρτησης (Εικόνα 13.3).



Εικόνα 13.3. Το καθολικό και τα τοπικά ελάχιστα της συνάρτησης $f(x) = x^2 + 15 * \sin(x)$.

Για να προσδιορίσουμε το τοπικό ελάχιστο που βρίσκεται στα αριστερά του καθολικού ελαχίστου, για μια τιμή του x κάπου μεταξύ -10 και -2.5 , εφαρμόζουμε τη συνάρτηση `minimize_scalar()`, που τη χρησιμοποιήσαμε ήδη για να βρούμε το ελάχιστο της Συνάρτησης [13.2], αλλά τώρα με τη μέθοδο `bounded` και ορίζοντας τα όρια (`bounds`) του x εντός των οποίων θα ψάξει η `minimize_scalar()` να βρει το τοπικό ελάχιστο, δηλαδή μεταξύ των τιμών του x από -10 έως -2.5 :

```
min_f_local_1 = minimize_scalar(f, method='bounded', bounds=(-10, -2.5))
```

Στην οθόνη εμφανίζονται η τιμή του τοπικού ελαχίστου (`min_f_local_1.x`) και η τιμή της συνάρτησης στο τοπικό ελάχιστο (`min_f_local.fun`):

```
f(x) local minimum #1, x = -6.7379
f(x) value @ local minimum #1 = 38.8112
```

και τοποθετείται το σημείο του τοπικού ελαχίστου, με έναν μαύρο σταυρό, επί της καμπύλης της συνάρτησης (Εικόνα 13.3).

Κατ' ανάλογο τρόπο προσδιορίζεται και το τοπικό ελάχιστο που βρίσκεται στα δεξιά του καθολικού ελαχίστου, για μια τιμή του x κάπου μεταξύ 0 και 7 :

```
min_f_local_2 = minimize_scalar(f, method='bounded', bounds=(0, 7))
```

```
f(x) local minimum #2, x = 4.1293
f(x) value @ local minimum #2 = 4.5296
```

με το σημείο αυτού του τοπικού ελαχίστου να εμφανίζεται με έναν μαύρο ρόμβο επί της καμπύλης της συνάρτησης (Εικόνα 13.3).

Σημείωση: Εάν θέλετε να επιβεβαιώσετε τα αποτελέσματα των ελαχίστων που προέκυψαν από τον παραπάνω κώδικα επί της Συνάρτησης [13.3] κάνοντας τις πράξεις με την αριθμομηχανή σας, θυμηθείτε ότι οι τριγωνομετρικές συναρτήσεις στην Python δέχονται τις τιμές των γωνιών σε ακτίνια (rad) και όχι σε μοίρες...

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο:
`Chapter_13_Function_Global_Local_Minima.py`

13.3 Βελτιστοποίηση συνάρτησης πολλών μεταβλητών υπό συνθήκες

Ιδιαίτερο ενδιαφέρον για τον Μηχανικό παρουσιάζουν τα προβλήματα που απαιτούν τη βελτιστοποίηση μιας συνάρτησης υπό συνθήκες και συνήθως αυτά τα προβλήματα αποτελούν αντικείμενο μελέτης της *Επιχειρησιακής Έρευνας* (Operations Research).

Ο *Γραμμικός Προγραμματισμός* (Linear Programming) είναι μία από τις μεθοδολογίες της Επιχειρησιακής Έρευνας. Κατ' ουσία, πρόκειται για μια μαθηματική τεχνική με την εφαρμογή της οποίας προσδιορίζουμε τις βέλτιστες λύσεις σε προβλήματα τα οποία μπορούν να εκφραστούν με τη βοήθεια γραμμικών εξισώσεων και ανισοτήτων [Σχετικά βλέπε στο: https://en.wikipedia.org/wiki/Linear_programming].

Στην πράξη, αυτό ισοδυναμεί με την εύρεση των τιμών των μεταβλητών $x_1, \dots, x_n$ που βελτιστοποιούν, δηλαδή ελαχιστοποιούν (minimize) ή μεγιστοποιούν (maximize), την τιμή της συνάρτησης $f(x_1, \dots, x_n)$.

Η συνάρτηση $f(x_1, \dots, x_n)$ ονομάζεται *αντικειμενική συνάρτηση* και οι μεταβλητές $x_1, \dots, x_n$, που αποτελούν τις παραμέτρους του προς βελτιστοποίηση προβλήματος, ονομάζονται *μεταβλητές εισόδου*.

Οι μεταβλητές εισόδου υπόκεινται σε *περιορισμούς* (constraints), οι οποίοι εκφράζονται υπό τη μορφή ανισοτήτων, αλλά και ισοτήτων.

Τέλος, ένας συνήθης περιορισμός στα προβλήματα βελτιστοποίησης, τα οποία καλείται να αντιμετωπίσει ένας Μηχανικός, είναι ότι οι τιμές των μεταβλητών $x_1, \dots, x_n$ που βελτιστοποιούν την αντικειμενική συνάρτηση πρέπει να είναι μεγαλύτερες ή ίσες του μηδενός.

Επομένως, το πρόβλημα της βελτιστοποίησης καταστρώνεται, στην απλή μορφή ενός παραδείγματος με δύο μεταβλητές, ως:

$$\text{Αντικειμενική συνάρτηση: } f(x_1, x_2) = c_1 \cdot x_1 + c_2 \cdot x_2$$

$$\text{Περιορισμοί: } a_1 \cdot x_1 + a_2 \cdot x_2 \leq a_3$$

$$b_1 \cdot x_1 + b_2 \cdot x_2 \geq b_3$$

$$\text{Θετικές μεταβλητές: } x_1 \geq 0$$

$$x_2 \geq 0$$

όπου: $c_1, c_2, a_1, a_2, a_3, b_1, b_2, b_3$ πραγματικοί αριθμοί

Η βασική μέθοδος που χρησιμοποιείται για την επίλυση προβλημάτων γραμμικού προγραμματισμού είναι η μέθοδος *Simplex* και οι παραλλαγές αυτής. Σχετικά με τη μέθοδο Simplex ανατρέξτε στο website: https://en.wikipedia.org/wiki/Simplex_algorithm.

Στη συνέχεια θα δούμε πώς, με τη βοήθεια της βιβλιοθήκης Scipy της Python, μπορούμε να εφαρμόσουμε τη μέθοδο Simplex στην επίλυση προβλημάτων γραμμικού προγραμματισμού.

13.3.1 Το προς βελτιστοποίηση πρόβλημα

Μια μεταλλουργία αλουμινίου πρόκειται να συνάψει σύμβαση για την αγορά βωξίτη συνολικής ποσότητας 1,000,000 t/y, με τις κάτωθι προδιαγραφές ποιότητας, σε ό,τι αφορά τις περιεκτικότητες των οξειδίων Al_2O_3 , SiO_2 και CaO στον βωξίτη που θα επεξεργαστεί:

$$Al_2O_3 \geq 48 \%$$

$$SiO_2 \leq 6.50 \%$$

$$CaO \leq 0.80 \%$$

Τον βωξίτη θα τον προμηθευτεί από τρία (3) μεταλλεία (M1, M2, M3) τα οποία παράγουν βωξίτη, του οποίου οι προδιαγραφές ποιότητας, το κόστος (συμπεριλαμβανομένης της μεταφοράς στο εργοστάσιο αλουμινίου, τιμή CIF) και οι διαθέσιμες ποσότητες δίνονται στον Πίνακα 13.1.

Πίνακας 13.1. Προδιαγραφές ποιότητας, κόστος και διαθέσιμες ποσότητες βωξίτη.

Μεταλλείο	M1	M2	M3
Al_2O_3 %	48.80	54.50	45.40
SiO_2 %	7.43	3.79	6.81
CaO %	0.10	0.15	1.00
Κόστος, €/t	33	38	32
Διαθέσιμη ποσότητα, t/y	600,000	300,000	700,000

Οι ποσότητες βωξίτη μετά την παραλαβή τους από το εργοστάσιο και πριν τη μεταλλουργική επεξεργασία τους, θα αναμειχθούν έτσι ώστε να ικανοποιούν τις προδιαγραφές ποιότητας της μεταλλουργίας.

Το ζητούμενο είναι, με βάση τα παραπάνω δεδομένα, να προσδιοριστούν οι ποσότητες του βωξίτη που θα προμηθευτεί η μεταλλουργία αλουμινίου από καθένα μεταλλείο, έτσι ώστε να έχει το ελάχιστο συνολικό κόστος αγοράς για τα 1,000,000 t/y βωξίτη.

13.3.2 Κατάστρωση του σχεδίου επίλυσης

Έστω m_1 , m_2 και m_3 οι ποσότητες του βωξίτη που θα αγοραστούν από τα μεταλλεία M1, M2 και M3 αντίστοιχα, οι οποίες αποτελούν και τις μεταβλητές εισόδου του προβλήματος.

Με βάση τις τιμές των δεδομένων ποιότητας, κόστους και διαθέσιμης ποσότητας του Πίνακα 13.1, μπορούμε να καταστρώσουμε:

την αντικειμενική συνάρτηση που πρέπει να ελαχιστοποιηθεί:

$$\text{Minimize } z = 33*m_1 + 38*m_2 + 32*m_3 \quad [\text{ΑΣ}]$$

Παραδοσιακά, στον γραμμικό προγραμματισμό η αντικειμενική συνάρτηση συμβολίζεται με το γράμμα z και η τιμή της, όταν ελαχιστοποιηθεί, θα είναι το συνολικό ποσό που θα πληρώσει η μεταλλουργία για την αγορά 1,000,000 t/y βωξίτη και από τα τρία μεταλλεία.

Το μείγμα των ποσοτήτων m_1 , m_2 και m_3 που θα αγοραστούν πρέπει να ικανοποιεί τους εξής περιορισμούς, σύμφωνα με τα δεδομένα του προβλήματος, τα οποία παρουσιάστηκαν στην παράγραφο 13.3.1 *Το προς βελτιστοποίηση πρόβλημα*:

$$48.80*m_1 + 54.50*m_2 + 45.40*m_3 \geq 1,000,000 * 48 \quad [\text{Π1}]$$

$$7.43*m_1 + 3.79*m_2 + 6.81*m_3 \leq 1,000,000 * 6.50 \quad [\text{Π2}]$$

$$0.10*m_1 + 0.15*m_2 + 1.00*m_3 \leq 1,000,000 * 0.80 \quad [\text{Π3}]$$

και μετά την εκτέλεση των πράξεων:

$$48.80*m_1 + 54.50*m_2 + 45.40*m_3 \geq 48,000,000 \quad [\text{Π1}]$$

$$7.43*m_1 + 3.79*m_2 + 6.81*m_3 \leq 6,500,000 \quad [\text{Π2}]$$

$$0.10*m_1 + 0.15*m_2 + 1.00*m_3 \leq 800,000 \quad [\text{Π3}]$$

Επίσης:

$$m_1 + m_2 + m_3 = 1,000,000 \quad [\text{Π4}]$$

$$0 \leq m_1 \leq 600,000 \quad [\text{Π5}]$$

$$0 \leq m_2 \leq 300,000 \quad [\text{Π6}]$$

$$0 \leq m_3 \leq 700,000 \quad [\text{Π7}]$$

13.3.3 Η μέθοδος Simplex: Scipy

Για την εύρεση του ελαχίστου της αντικειμενικής συνάρτησης z με τη μέθοδο Simplex θα χρησιμοποιήσουμε τη συνάρτηση `linprog` της SciPy.

Για την τεκμηρίωση της συνάρτησης `linprog()` ανατρέξτε στο website:

<https://docs.scipy.org/doc/scipy/reference/optimize.linprog-simplex.html>

Η συνάρτηση `linprog` ανήκει στην οικογένεια συναρτήσεων `optimize` της βιβλιοθήκης `scipy` και θα πρέπει να την κάνουμε `import` στον κώδικά μας.

Η χρήση της `linprog()` είναι γενικά απλή, αλλά έχει κάποιες ιδιαιτερότητες που πρέπει να προσέξουμε:

1. Μπορεί να κάνει μόνο ελαχιστοποίηση (minimization) της αντικειμενικής συνάρτησης και όχι μεγιστοποίηση (maximization).

Εάν το πρόβλημά μας απαιτεί τη μεγιστοποίηση της αντικειμενικής συνάρτησης, τότε το μετατρέπουμε σε πρόβλημα ελαχιστοποίησης, πολλαπλασιάζοντας το δεξιό μέλος της αντικειμενικής συνάρτησης επί -1, οπότε το αριστερό μέλος αντιστρέφεται από Maximize z σε Minimize z .

2. Όλες οι ανισότητες των περιορισμών πρέπει να είναι της μορφής $\leq$ (μικρότερο ή ίσο) και όχι $\geq$ (μεγαλύτερο ή ίσο).

Εάν ένας περιορισμός είναι της μορφής $\geq$, όπως ο [Π1] του προβλήματός μας, τότε τον μετατρέπουμε σε περιορισμό μορφής $\leq$ πολλαπλασιάζοντας και τα δύο μέλη της ανισότητας επί -1.

Κατόπιν των ανωτέρω, για να προχωρήσουμε στην εφαρμογή της μεθόδου Simplex με χρήση της `linprog()` για την επίλυση του προβλήματός μας, θα μετατρέψουμε τον περιορισμό [Π1] στο ισοδύναμό του:

$$-48.80 \cdot m_1 - 54.50 \cdot m_2 - 45.40 \cdot m_3 \leq -48,000,000 \quad [\Pi 1]$$

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from scipy.optimize import linprog
# Objective function
z = [33, 38, 32]
# Inequalities
lhs_ineq = [[-48.80, -54.50, -45],
             [7.43, 3.79, 6.81],
             [0.10, 0.15, 1.00]]
rhs_ineq = [48000000,
            6500000,
            800000]
# Equalities
lhs_eq = [[1, 1, 1]]
rhs_eq = [1000000]
# Bounds
bnds = [(0, 600000),
        (0, 300000),
        (0, 700000)]
# Optimization
opt = linprog(c=z, A_ub=lhs_ineq, b_ub=rhs_ineq,
              A_eq=lhs_eq, b_eq=rhs_eq, bounds=bnds,
              method='simplex', options={'tol': 1e-12})
# Results
print('Bauxite from M1 mine (t) = {:.0f}'.format(opt.x[0]))
print('Bauxite from M2 mine (t) = {:.0f}'.format(opt.x[1]))
print('Bauxite from M3 mine (t) = {:.0f}'.format(opt.x[2]))
print('Total cost for 1,000,000 t of bauxite (EUR) ='
      '{:.0f}'.format(opt.fun))
print('Average cost of 1 t of bauxite (EUR/t) ='
      '{:.3f}'.format(opt.fun/1000000))
```

Καταρχάς, πρέπει να δημιουργήσουμε μεταβλητές στις οποίες θα εκχωρήσουμε, υπό μορφή array, τους αριθμητικούς συντελεστές των m_1 , m_2 και m_3 στην αντικειμενική συνάρτηση και τους περιορισμούς.

- Στη μεταβλητή `z` εκχωρούμε τους συντελεστές στην αντικειμενική συνάρτηση [ΑΣ].
- Στη μεταβλητή `lhs_ineq` εκχωρούμε τους συντελεστές του αριστερού σκέλους (left hand side) των ανισοτήτων στους περιορισμούς [Π1], [Π2] και [Π3].
- Στη μεταβλητή `rhs_ineq` εκχωρούμε τους συντελεστές του δεξιού σκέλους (right hand side) των ανισοτήτων στους περιορισμούς [Π1], [Π2] και [Π3].
- Στη μεταβλητή `lhs_eq` εκχωρούμε τους συντελεστές του αριστερού σκέλους της ισότητας στον περιορισμό [Π4].
- Στη μεταβλητή `rhs_eq` εκχωρούμε τον συντελεστή του δεξιού σκέλους της ισότητας στον περιορισμό [Π4].

- Στη μεταβλητή `bnds` εκχωρούμε, υπό τη μορφή πλειάδας, την ανώτερη και κατώτερη τιμή (`bounds`) που μπορεί να έχει καθεμία από τις μεταβλητές `m1`, `m2` και `m3`, σύμφωνα με τους περιορισμούς [Π5], [Π6] και [Π7] αντίστοιχα.

Η βελτιστοποίηση, δηλαδή η ελαχιστοποίηση της αντικειμενικής συνάρτησης $[A\bar{z}]$, γίνεται με την εντολή:

```
opt = linprog(c=z, A_ub=lhs_ineq, b_ub=rhs_ineq,
             A_eq=lhs_eq, b_eq=rhs_eq, bounds=bnds,
             method='simplex', options={'tol': 1e-12})
```

όπου:

- Στις παραμέτρους της συνάρτησης `c`, `A_ub`, `b_ub`, `A_eq`, `b_eq` και `bounds` δίνονται οι τιμές των μεταβλητών που δημιουργήσαμε στον κώδικά μας και με τη σειρά που παρουσιάστηκαν παραπάνω.
- Στην παράμετρο `method` με την τιμή `simplex` που της δώσαμε δηλώνουμε ότι στη βελτιστοποίηση της αντικειμενικής συνάρτησης θα χρησιμοποιηθεί η κλασική μέθοδος Simplex.

Μπορούν να χρησιμοποιηθούν και άλλες μέθοδοι, μεταξύ των οποίων είναι και η *Revised Simplex Method* (`revised simplex`), η οποία είναι ταχύτερη της κλασικής μεθόδου Simplex. Για περισσότερες πληροφορίες ανατρέξτε στην τεκμηρίωση της συνάρτησης `linprog()`.

- Στην παράμετρο `options` με την τιμή `'tol': 1e-12` που της δώσαμε, ορίσαμε την *ανοχή* (*tolerance*) με την οποία δεχόμαστε το μηδέν στην αριθμητική μέθοδο της ελαχιστοποίησης να είναι κοντά στο *πραγματικό μηδέν* (*0*), δηλαδή δεχόμαστε ότι η τιμή $1 \cdot 10^{-12}$ είναι πρακτικά 0 (βλέπε και παράγραφο 11.5 *Περί ανοχής λάθους στις αριθμητικές μεθόδους*).

Σημειώστε ότι η default τιμή της ανοχής στη `linprog()` είναι `'tol': 1e-12` και υπό αυτή την έννοια δεν είναι απαραίτητο να συμπεριλάβετε την παράμετρο `options={'tol': 1e-12}`. Εμείς το κάνουμε όμως για διδακτικούς λόγους.

Και αυτό γιατί υπάρχουν περιπτώσεις εφαρμογής της μεθόδου Simplex, όπου δεν είναι δυνατή η βελτιστοποίηση της αντικειμενικής συνάρτησης με μια τόσο μικρή τιμή ανοχής ($1e-12$).

Στην περίπτωση αυτή ο interpreter της Python θα μας βγάλει ένα μήνυμα λάθους, όπως για παράδειγμα το παρακάτω:

ValueError: Phase 1 of the simplex method failed to find a feasible solution. The pseudo-objective function evaluates to 3.8e-09 which exceeds the required tolerance of 1e-12 for a solution to be considered 'close enough' to zero to be a basic solution. Consider increasing the tolerance to be greater than 3.8e-09. If this tolerance is unacceptably large the problem may be infeasible

Για να προχωρήσουμε τότε στη βελτιστοποίηση του προβλήματος, θα πρέπει να αλλάξουμε την default τιμή της ανοχής με την προτεινόμενη στο μήνυμα λάθους, δηλαδή θα γράψουμε π.χ. `options={'tol': 4e-09}` στη συγκεκριμένη περίπτωση.

Οι εντολές `print()` που ακολουθούν μας εμφανίζουν στην οθόνη τις τιμές των μεταβλητών `m1`, `m2` και `m3` που βελτιστοποιούν – ελαχιστοποιούν την αντικειμενική συνάρτηση $[A\bar{z}]$ (`opt.x[0]`, `opt.x[1]`, `opt.x[2]`) και την τιμή της ελαχιστοποιημένης αντικειμενικής συνάρτησης (`opt.fun`):


```
Bauxite from M1 mine (t) = 163736
Bauxite from M2 mine (t) = 136264
Bauxite from M3 mine (t) = 700000
Total cost for 1,000,000 t of bauxite (EUR) = 32981319
Average cost of 1 t of bauxite (EUR/t) = 32.981
```

Σημείωση 1: Οι δυνατότητες της συνάρτησης `linprog` της SciPy είναι περιορισμένες και κατάλληλες για την επίλυση προβλημάτων βελτιστοποίησης με μικρό αριθμό μεταβλητών και περιορισμών. Για μεγαλύτερα και πιο σύνθετα προβλήματα γραμμικού προγραμματισμού, άλλες πιο εξειδικευμένες βιβλιοθήκες της Python, όπως η PuLP, είναι καταλληλότερες.

Σημείωση 2: Πέραν των ιδιοτήτων που αναφέραμε παραπάνω, η `linprog()` δεν μπορεί να χειριστεί ακέραιες μεταβλητές. Οι μεταβλητές `m1`, `m2` και `m3` που αντιπροσωπεύουν τις ποσότητες του βωξίτη που θα αγοραστούν, σε τόνους, λαμβάνουν δεκαδικές τιμές και εμείς με τις φορμαρισμένες εντολές `print()` δεν εμφανίσαμε το δεκαδικό μέρος (`{ : .0f }`).

Εάν αντί για την ποσότητα του βωξίτη σε τόνους θέλαμε να υπολογίσουμε τα φορτία βωξίτη (π.χ. φορτία πλοίων των 50,000 t το καθένα) που θα αγοράζαμε από κάθε μεταλλείο, η χρήση της συνάρτησης `linprog()` δεν θα μπορούσε να δώσει λύση.

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο:
`Chapter_13_Simplex_Bauxite_Contract_Scipy.py`

Στη συνέχεια, θα δούμε πώς με τη χρήση της βιβλιοθήκης PuLP της Python επιλύουμε προβλήματα γραμμικού προγραμματισμού όπου οι μεταβλητές είναι ακέραιοι αριθμοί (Integer Programs).

13.4 Βελτιστοποίηση συνάρτησης ακέραιων μεταβλητών υπό συνθήκες: PuLP

Η βιβλιοθήκη PuLP ως μια εξωτερική βιβλιοθήκη της Python πρέπει να εγκατασταθεί στο υπολογιστή μας κατά τα γνωστά:

```
pip install PuLP
```

Για πληροφορίες και τεκμηρίωση ανατρέξτε στο website της βιβλιοθήκης:
<https://pypi.org/project/PuLP/>

Η PuLP υποστηρίζει την επίλυση προβλημάτων γραμμικού προγραμματισμού με τη βοήθεια διαφόρων εξελιγμένων αλγορίθμων (linear programming solvers), εκ των οποίων ορισμένοι παρέχονται δωρεάν (open-source), ενώ άλλοι διατίθενται επί πληρωμή για εμπορική χρήση. Καθένας από αυτούς τους αλγορίθμους πρέπει να εγκατασταθεί ξεχωριστά.

Ωστόσο, στην εγκατάσταση της PuLP συμπεριλαμβάνεται, ως default solver, ο CBC (Coin-or Branch and Cut), που είναι κατάλληλος για την επίλυση προβλημάτων των οποίων οι μεταβλητές λαμβάνουν δεκαδικές ή ακέραιες τιμές ή μείγμα αυτών (Mixed-Integer Programs - MIP). Για περισσότερες πληροφορίες, ανατρέξτε στο website: <https://github.com/coin-or/Cbc>

13.4.1 Το προς βελτιστοποίηση πρόβλημα

Μία από τις συνηθισμένες κατηγορίες προβλημάτων ακέραιου γραμμικού προγραμματισμού είναι τα *προβλήματα μεταφοράς* (transportation problems).

Στα προβλήματα αυτά το ζητούμενο είναι να υπολογίσουμε τον αριθμό των φορτίων που θα μεταφέρονται από τα σημεία προέλευσης $x_1, \dots, x_n$ στα σημεία προορισμού $y_1, \dots, y_k$ έτσι ώστε να ελαχιστοποιείται το συνολικό κόστος μεταφοράς, ενώ παράλληλα θα ικανοποιούνται οι περιορισμοί προσφοράς και ζήτησης. Κατ' αυτόν τον τρόπο υπολογίζουμε το βέλτιστο σχέδιο μεταφοράς.

Ένα τέτοιο πρόβλημα μεταφοράς διατυπώνεται στη συνέχεια.

Μια λατομική επιχείρηση διαθέτει δύο (2) λατομεία ασβεστολίθου (Q1 και Q2) που παράγουν αδρανή προϊόντα και τροφοδοτούν τρία (3) εργοστάσια παρασκευής έτοιμου σκυροδέματος (P1, P2 και P3).

Για τη μεταφορά των αδρανών, η επιχείρηση χρησιμοποιεί φορτηγά αυτοκίνητα δημοσίας χρήσης και το κόστος μεταφοράς, που είναι 0.10 €/t.km, επιβαρύνει τη λατομική επιχείρηση. Κάθε φορτηγό μεταφέρει 30 t αδρανών (φορτίο).

Οι αποστάσεις μεταφοράς από τα λατομεία στα εργοστάσια έτοιμου σκυροδέματος, σε km, δίνονται στον Πίνακα 13.2.

Καθένα από τα εργοστάσια έτοιμου σκυροδέματος έχει τις εξής απαιτήσεις σε φορτία αδρανών τον μήνα:

P1: 600 P2: 1,250 P3: 1,050

Πίνακας 13.2. Αποστάσεις μεταφοράς, σε km, από τα λατομεία (Q1, Q2) στα εργοστάσια έτοιμου σκυροδέματος (P1, P2, P3).

	P1	P2	P3
Q1	25	30	25
Q2	30	15	25

Καθένα από τα λατομεία έχει τη δυνατότητα να τροφοδοτήσει οποιοδήποτε εργοστάσιο, αλλά μπορεί να παραδώσει τα παρακάτω συνολικά φορτία τον μήνα:

Q1: 900 Q2: 2,000

Το ζητούμενο είναι, με βάση τα παραπάνω δεδομένα, να προσδιοριστούν τα φορτία αδρανών που θα μεταφερθούν από κάθε λατομείο σε κάθε εργοστάσιο, έτσι ώστε η λατομική επιχείρηση να έχει το ελάχιστο κόστος μεταφοράς.

13.4.2 Κατάστρωση του σχεδίου επίλυσης

Με βάση τα δεδομένα του προβλήματος και αφού κάνουμε τους υπολογισμούς του μεταφορικού κόστους κάθε φορτίου, από λατομείο σε εργοστάσιο συναρτήσει της μεταξύ τους απόστασης, προκύπτει ο Πίνακας 13.3 που περιλαμβάνει τις παραμέτρους του προβλήματος.

Πίνακας 13.3. Κόστος μεταφοράς, σε €, φορτίων αδρανών από τα λατομεία (Q1, Q2) στα εργοστάσια έτοιμου σκυροδέματος (P1, P2, P3), απαιτούμενα και διαθέσιμα φορτία.

	P1	P2	P3	Φορτία Διαθέσιμα
Q1	$30 \cdot 25 \cdot 0.1 = 75$	$30 \cdot 30 \cdot 0.1 = 90$	$30 \cdot 25 \cdot 0.1 = 75$	900
Q2	$30 \cdot 30 \cdot 0.1 = 90$	$30 \cdot 15 \cdot 0.1 = 45$	$30 \cdot 25 \cdot 0.1 = 75$	2,000
Φορτία Απαιτούμενα	600	1,250	1,050	2,900

Από την άθροιση των τιμών της τελευταίας γραμμής του Πίνακα 13.3 προκύπτει ότι τα συνολικά απαιτούμενα φορτία αδρανών από όλα τα εργοστάσια είναι 2,900. Επίσης, από την άθροιση των τιμών της τελευταίας στήλης του Πίνακα 13.3 προκύπτει ότι τα συνολικά διαθέσιμα φορτία αδρανών από όλα τα λατομεία είναι 2,900. Στην περίπτωση αυτή λέμε ότι έχουμε ένα *ισοζυγισμένο* ή *ισορροπημένο* (balanced) πρόβλημα μεταφοράς.

Έστω $q1_p1$ είναι ο αριθμός των φορτίων που θα μεταφερθούν από το λατομείο Q1 στο εργοστάσιο P1, $q1_p2$ είναι ο αριθμός των φορτίων που θα μεταφερθούν από το λατομείο Q1 στο εργοστάσιο P2, κ.ο.κ., όπως παρουσιάζονται στον Πίνακα 13.4.

Πίνακας 13.4. Μεταβλητές προς βελτιστοποίηση, απαιτούμενα και διαθέσιμα φορτία.

	P1	P2	P3	Φορτία Διαθέσιμα
Q1	$q1_p1$	$q1_p2$	$q1_p3$	900

Q2	q2_p1	q2_p2	q2_p3	2,000
Φορτία Απαιτούμενα	600	1,250	1,050	2,900

Οι μεταβλητές αυτές, δηλαδή τα μεταφερόμενα φορτία, είναι οι μεταβλητές εισόδου και πρέπει να βελτιστοποιηθούν. Επειδή τα λατομεία είναι 2 και τα εργοστάσια 3, υπάρχουν $2 \times 3 = 6$ μεταβλητές προς βελτιστοποίηση.

Με βάση τις τιμές των δεδομένων κόστους μεταφοράς, διαθέσιμων και απαιτούμενων φορτίων (Πίνακας 13.3 & Πίνακας 13.4), μπορούμε να καταστρώσουμε:

την αντικειμενική συνάρτηση, η οποία πρέπει να ελαχιστοποιηθεί:

$$\text{Minimize } z = 75 \cdot q1_p1 + 90 \cdot q1_p2 + 75 \cdot q1_p3 + 90 \cdot q2_p1 + 45 \cdot q2_p2 + 75 \cdot q2_p3 \quad [A\Sigma]$$

τους περιορισμούς για τα απαιτούμενα φορτία:

$$q1_p1 + q2_p1 \geq 600 \quad [A1]$$

$$q1_p2 + q2_p2 \geq 1250 \quad [A2]$$

$$q1_p3 + q2_p3 \geq 1050 \quad [A3]$$

τους περιορισμούς για τα διαθέσιμα φορτία:

$$q1_p1 + q1_p2 + q1_p3 \leq 900 \quad [\Delta1]$$

$$q2_p1 + q2_p2 + q2_p3 \leq 2,000 \quad [\Delta2]$$

Επίσης:

Οι μεταβλητές: $q1_p1, q1_p2, q1_p3, q2_p1, q2_p2, q2_p3 \geq 0$ και ακέραιες.

13.4.3 Βελτιστοποίηση με τον αλγόριθμο CBC: PuLP

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from pulp import *
# Create the model
model = LpProblem(name='Minimize_Transportation_Cost',
                  sense=LpMinimize)
# Initialize the decision variables
q1_p1 = LpVariable(name='Q1_to_P1', lowBound=0, cat='Integer')
q1_p2 = LpVariable(name='Q1_to_P2', lowBound=0, cat='Integer')
q1_p3 = LpVariable(name='Q1_to_P3', lowBound=0, cat='Integer')
q2_p1 = LpVariable(name='Q2_to_P1', lowBound=0, cat='Integer')
q2_p2 = LpVariable(name='Q2_to_P2', lowBound=0, cat='Integer')
q2_p3 = LpVariable(name='Q2_to_P3', lowBound=0, cat='Integer')
# Add the constraints to the model
model += (q1_p1 + q2_p1 >= 600, "A1_constraint")
model += (q1_p2 + q2_p2 >= 1250, "A2_constraint")
model += (q1_p3 + q2_p3 >= 1050, "A3_constraint")
model += (q1_p1 + q1_p2 + q1_p3 <= 900, "D1_constraint")
model += (q2_p1 + q2_p2 + q2_p3 <= 2000, "D2_constraint")
# Add the objective function to the model
model += 75*q1_p1 + 90*q1_p2 + 75*q1_p3 + \
        90*q2_p1 + 45*q2_p2 + 75*q2_p3
# Solve the problem
model.solve()
#model.solve(PULP_CBC_CMD(msg=0))
print(model.name)
print('Status of solution: {}'.format(LpStatus[model.status]))
print('Transportation plan (Route: Truck-loads)')
for var in model.variables():
    print('{} : {:.0f}'.format(var.name, var.value()))
```

```
print('Transportation cost - minimum (EUR) = {:.2f}'
      .format(model.objective.value()))
```

Καταρχάς, κάνουμε `import` τη βιβλιοθήκη `pulp` στον κώδικά μας.

Στη συνέχεια, με τη βοήθεια της συνάρτησης `LpProblem()` της PuLP ορίζουμε το μοντέλο του προβλήματός μας, στο οποίο δίνουμε το όνομα - χαρακτηρισμό `Minimize_Transportation_Cost` (δεν επιτρέπονται κενά διαστήματα μεταξύ των λέξεων) και στην παράμετρο `sense` την τιμή `LpMinimize`, δηλαδή ότι πρόκειται περί *προβλήματος ελαχιστοποίησης*. Εάν είχαμε *πρόβλημα μεγιστοποίησης*, θα δίναμε την τιμή `LpMaximize`. Το μοντέλο του προβλήματος το εκχωρούμε στη μεταβλητή `model`.

Ακολούθως ορίζουμε τις 6 μεταβλητές του προβλήματος οι οποίες θα βελτιστοποιηθούν με τη βοήθεια της συνάρτησης `LpVariable()` της PuLP:

```
q1_p1 = LpVariable(name='Q1_to_P1', lowBound=0, cat='Integer')
```

δηλαδή, η μεταβλητή `q1_p1` έχει όνομα - χαρακτηρισμό `Q1_to_P1` (από το λατομείο Q1 στο εργοστάσιο P1), η μικρότερη τιμή που μπορεί να λάβει (`lowBound`) είναι 0 (Σημείωση: η ανώτερη default τιμή της είναι $+\infty$) και είναι κατηγορίας (`cat`) ακέραια (`Integer`)).

Αναλόγως ορίζονται και οι υπόλοιπες πέντε (5) μεταβλητές του προβλήματος.

Με τις επόμενες γραμμές κώδικα προσθέτουμε στο `model` τους περιορισμούς του προβλήματος [A1], [A2], [A3], [Δ1], [Δ2] και την αντικειμενική συνάρτηση [ΑΣ], που έχουμε παρουσιάσει στην κατάστρωση του σχεδίου επίλυσης, στην παράγραφο 13.4.2 *Κατάστρωση του σχεδίου επίλυσης*, και με τη συντακτική μορφή που δίνεται στον κώδικα.

Η γραμμή κώδικα που ακολουθεί:

```
model.solve()
```

εφαρμόζει επί του `model` τη μέθοδο `solve()` της PuLP για την ελαχιστοποίηση της αντικειμενικής συνάρτησης του προβλήματός μας, χρησιμοποιώντας τον ενσωματωμένο αλγόριθμο (default solver) CBC.

Στην περίπτωση αυτή θα εμφανιστεί στην οθόνη μια πολύ λεπτομερής αναφορά της διαδικασίας βελτιστοποίησης του προβλήματος. Εάν αυτό δεν είναι επιθυμητό, αντικαταστήστε την εντολή `model.solve()` με την:

```
model.solve(PULP_CBC_CMD(msg=0))
```

η οποία στον κώδικα είναι απενεργοποιημένη, γιατί εμφανίζεται ως σχόλιο.

Σημειώστε ότι το `PULP_CBC_CMD` είναι το συμβολικό όνομα με το οποίο η `solve()` αντιλαμβάνεται ότι θα χρησιμοποιήσει τον αλγόριθμο CBC.

Οι εντολές `print()` που ακολουθούν στον κώδικα εμφανίζουν στην οθόνη:

- Το όνομα - χαρακτηρισμό του μοντέλου του προβλήματος, `model.name`.
- Την κατάσταση (status) της λύσης, `LpStatus[model.status]`.

Οι τιμές που μπορεί να εμφανιστούν είναι: *Optimal* (βέλτιστη), *Not Solved* (άλυτο πρόβλημα), *Infeasible* (αδύνατη), *Unbounded* (χωρίς όρια), *Undefined* (απροσδιόριστη).

- Το όνομα - χαρακτηρισμό των μεταβλητών, `var.name`, και τις βέλτιστες τιμές τους, `var.value()`. Τα στοιχεία αυτά βρίσκονται στο `model.variables()`.
- Την τιμή της βελτιστοποιημένης (ελαχιστοποιημένης στην περίπτωση του προβλήματός μας) αντικειμενικής συνάρτησης, `model.objective.value()`.

Τα αποτελέσματα της βελτιστοποίησης εμφανίζονται στην οθόνη ως:

```
Minimize_Transportation_Cost
Status of solution: Optimal
Transportation plan (Route: Truck-loads)
Q1_to_P1 : 600
Q1_to_P2 : 0
Q1_to_P3 : 300
Q2_to_P1 : 0
Q2_to_P2 : 1250
Q2_to_P3 : 750
Transportation cost - minimum (EUR) = 180000.00
```

Επομένως, για να ελαχιστοποιηθεί το κόστος μεταφοράς των φορτίων αδρανών από τα λατομεία Q1 και Q2 στα εργοστάσια έτοιμου σκυροδέματος P1, P2 και P3 πρέπει:

- το λατομείο Q1 να τροφοδοτήσει το εργοστάσιο P1 με 600 φορτία, το εργοστάσιο P3 με 300 φορτία και με κανένα φορτίο το εργοστάσιο P2,
- το λατομείο Q2 να τροφοδοτήσει το εργοστάσιο P2 με 1250 φορτία, το εργοστάσιο P3 με 750 φορτία και με κανένα φορτίο το εργοστάσιο P1.

Το ελάχιστο συνολικό κόστος μεταφοράς που μπορεί να επιτευχθεί είναι 180,000 EUR.

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο:

Chapter_13_Transportation_Problem_Aggregates_Loads_PuLP.py

14 Διαχείριση έργου

14.1 Γενικά

Ο Μηχανικός μελετά και εκτελεί έργα.

Ένα έργο μπορεί να είναι *άυλο* (π.χ. η τεχνική μελέτη εκμετάλλευσης ενός μεταλλείου) ή *κατασκευαστικό* (π.χ. η διάνοιξη μιας σήραγγας).

Ανεξάρτητα από τη φύση του, το έργο πρέπει να ολοκληρωθεί στον *βέλτιστο χρόνο*, με το μικρότερο δυνατό κόστος και η ποιότητά του να είναι του επιπέδου που προκαθορίστηκε.

Για την επίτευξη των παραπάνω στόχων, αποτελούν απαραίτητες προϋποθέσεις ο λεπτομερής σχεδιασμός του έργου, ο ορθολογικός προγραμματισμός των σταδίων της εκτέλεσής του, η αποτελεσματική διοίκηση και η καλή οργάνωση του έργου.

Σε αυτό το Κεφάλαιο θα αναπτύξουμε τρία προγράμματα τα οποία ο Μηχανικός μπορεί να τα χρησιμοποιεί στη *διαχείριση του χρόνου* (χρονικός προγραμματισμός) των έργων που αναλαμβάνει.

Το πρώτο πρόγραμμα εφαρμόζει την τεχνική του *δικτυωτού προγραμματισμού έργων CPM*, για την ανάπτυξη του λεπτομερούς χρονοδιαγράμματος των δραστηριοτήτων ενός υπό διαχείριση έργου, καθώς και την εύρεση των κρίσιμων δραστηριοτήτων του.

Στη μέθοδο CPM, οι χρόνοι εκτέλεσης των επιμέρους εργασιών ενός έργου εκτιμούμε ότι θα είναι σταθεροί και ότι η όποια αβεβαιότητα ως προς την επίτευξή τους είναι μη σημαντική.

Το δεύτερο πρόγραμμα εφαρμόζει την τεχνική του *δικτυωτού προγραμματισμού έργων PERT*, για την ανάπτυξη του λεπτομερούς χρονοδιαγράμματος των δραστηριοτήτων ενός υπό διαχείριση έργου, καθώς και την εύρεση των κρίσιμων δραστηριοτήτων του.

Στη μέθοδο PERT οι χρόνοι εκτέλεσης των επιμέρους εργασιών ενός έργου δεν είναι σταθεροί, αλλά εκτιμούμε ότι θα κυμαίνονται στοχαστικά μεταξύ μιας *αισιόδοξης*, μιας *πιθανής* και μιας *απαισιόδοξης* χρονικής διάρκειας.

Το τρίτο πρόγραμμα κατασκευάζει το *διάγραμμα Gantt* ενός υπό διαχείριση έργου, με το οποίο είναι δυνατός ο χρονικός προγραμματισμός του, αλλά κυρίως η παρακολούθηση της εξέλιξης της πορείας εκτέλεσής του.

Ως παράδειγμα για την ανάπτυξη και εφαρμογή αυτών των προγραμμάτων θα χρησιμοποιηθεί το έργο της εγκατάστασης ενός θραυστήρα μεταλλεύματος σε υπόγειο μεταλλείο.

Η περιγραφή του έργου, που ακολουθεί, βασίζεται στο παράδειγμα το οποίο περιλαμβάνεται στην παράγραφο 30.6 *Project Control Techniques* του Mining Engineering Handbook, Society of Mining Engineers (SME-AIME), 1st edition, 1973, το οποίο έχει τροποποιηθεί κατάλληλα προκειμένου να εξυπηρετήσει τις ανάγκες αυτού του Κεφαλαίου.

Σημείωση: Οι νεότερες εκδόσεις του Mining Engineering Handbook, 2nd edition (1992) και η τρέχουσα 3rd edition (2011) δεν περιλαμβάνουν στη θεματολογία τους το αντικείμενο του *Project Control Techniques*.

14.2 Το προς διαχείριση έργο

Η διεύθυνση ενός υπόγειου μεταλλείου αποφάσισε να εγκαταστήσει έναν θραυστήρα μεταλλεύματος εντός ενός θαλάμου, στο επίπεδο του πυθμένα του φρέατος ανέλκυσης.

Ο θραυστήρας θα τροφοδοτεί με θραυσμένο μέταλλευμα το skip ανέλκυσης με τη βοήθεια μεταφορικής ταινίας. Η μεταφορική ταινία θα κατασκευαστεί από το μηχανολογικό-ηλεκτρολογικό τμήμα του μεταλλείου, χρησιμοποιώντας υπάρχοντα εξαρτήματα και μηχανισμούς.

Η εταιρεία προμήθειας του θραυστήρα θα αναλάβει τις εργασίες σχεδίασης και κατασκευής του θραυστήρα, της αποσυναρμολόγησης και μεταφοράς του στη θέση εγκατάστασης στο μεταλλείο, της συναρμολόγησης στον υπόγειο θάλαμο και των τελικών δοκιμών καλής λειτουργίας.

Το τμήμα εκμετάλλευσης του μεταλλείου θα αναλάβει την όρυξη του υπόγειου θαλάμου εγκατάστασης του θραυστήρα, την κατασκευή από σκυρόδεμα του δαπέδου και της βάσης της μεταφορικής ταινίας και την κατασκευή της βάσης του θραυστήρα επίσης από σκυρόδεμα.

Το μηχανολογικό-ηλεκτρολογικό τμήμα του μεταλλείου θα αναλάβει τις εργασίες εγκατάστασης της μεταφορικής ταινίας και της παροχής ηλεκτρικού ρεύματος στο συγκρότημα του θραυστήρα και της μεταφορικής ταινίας.

Οι μηχανικοί του μεταλλείου, σε συνεργασία με τον προμηθευτή του θραυστήρα, κατάρτησαν τη λίστα των επιμέρους εργασιών του έργου και την εκτιμώμενη χρονική διάρκεια καθεμίας εργασίας, σε εβδομάδες, η οποία παρουσιάζεται στον Πίνακα 14.1.

Πίνακας 14.1. Λίστα των επιμέρους εργασιών του έργου της εγκατάστασης θραυστήρα και μεταφορικής ταινίας στο υπόγειο μεταλλείο.

Αριθμός Εργασίας	Περιγραφή Εργασίας	Χρονική Διάρκεια (Εβδομάδες)
1	Σχεδίαση του θραυστήρα (DESIGN CRUSHER)	2
2	Κατασκευή του θραυστήρα (BUILD CRUSHER)	4
3	Αποσυναρμολόγηση θραυστήρα και μεταφορά στη θέση εγκατάστασης (DISASSEMBLE AND MOVE CRUSHER TO STATION)	5
4	Συναρμολόγηση του θραυστήρα (ASSEMBLE CRUSHER)	4
5	Τελικές δοκιμές (FINAL TEST)	1
6	Όρυξη υπόγειου θαλάμου εγκατάστασης του θραυστήρα (CUT CRUSHER STATION)	3
7	Κατασκευή δαπέδου και βάσης της μεταφορικής ταινίας από σκυρόδεμα (POUR FLOOR AND CONVEYOR BASE)	1
8	Κατασκευή της βάσης του θραυστήρα από σκυρόδεμα (POUR CRUSHER BASE)	2
9	Εγκατάσταση της μεταφορικής ταινίας (INSTALL CONVEYOR)	6
10	Εργασίες παροχής ηλεκτρικού ρεύματος (INSTALL WIRING)	6
11	Ωρίμανση σκυροδέματος βάσης θραυστήρα (CURE CRUSHER BASE)	2

Από τις έντεκα συνολικά εργασίες του έργου που περιγράφονται στον Πίνακα 14.1:

- Οι εργασίες 1, 2, 3, 4 και 5 θα εκτελεστούν από την εταιρεία η οποία θα προμηθεύσει τον θραυστήρα (Crusher Ltd).
- Οι εργασίες 6, 7 και 8 θα εκτελεστούν από το τμήμα εκμετάλλευσης του μεταλλείου (Mining Dpt).
- Οι εργασίες 9 και 10 θα εκτελεστούν από το μηχανολογικό-ηλεκτρολογικό τμήμα του μεταλλείου (Mechanical/Electrical Dpt).
- Η εργασία 11 θα γίνει από μόνη της...

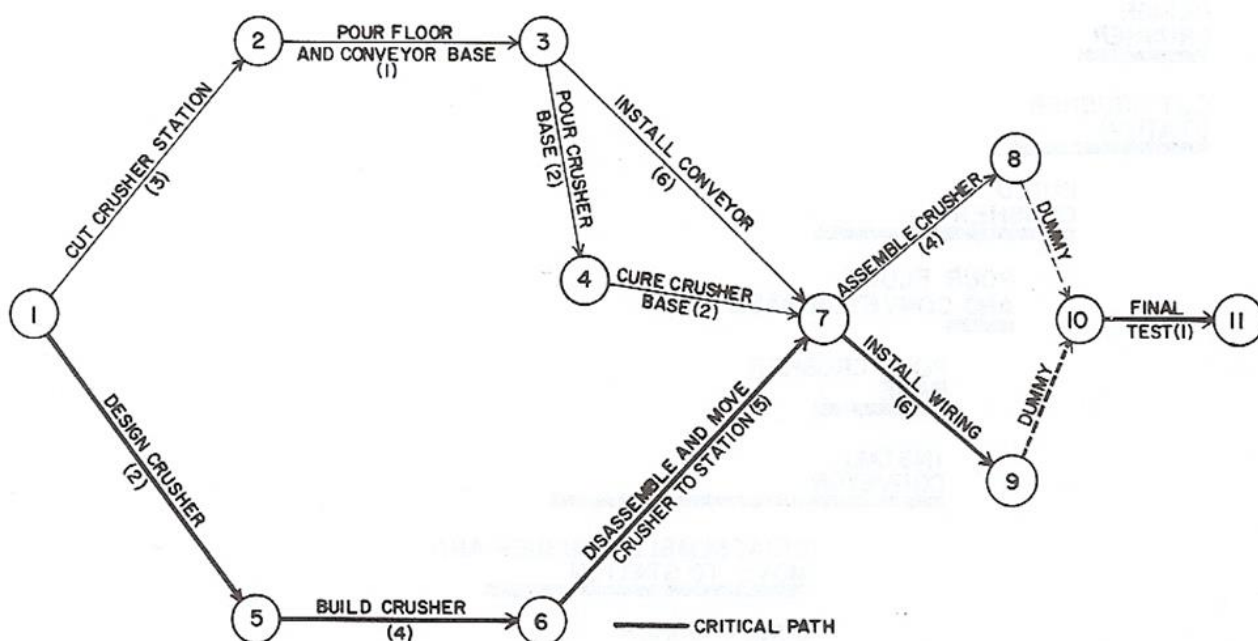
14.3 Δικτυωτός χρονικός προγραμματισμός έργων: CPM

Ο δικτυωτός προγραμματισμός έργων ή δικτυωτή ανάλυση (network analysis) χρησιμοποιείται στη διαχείριση έργων με την εφαρμογή της τεχνικής προγραμματισμού έργων CPM (Critical Path Method). Σχετικά βλέπε στο: https://en.wikipedia.org/wiki/Critical_path_method

Η μέθοδος CPM είναι κατάλληλη για τη διαχείριση έργων που συγκροτούνται από συγκεκριμένες δραστηριότητες, οι οποίες είναι δυνατόν να εκτελεστούν εντός σαφών χρονικών διαστημάτων και βρίσκονται υπό τον έλεγχο ενός φορέα διοίκησης.

Το δικτυωτό διάγραμμα της μεθόδου CPM, με το οποίο περιγράφεται λεπτομερώς η πορεία εκτέλεσης του έργου της εγκατάστασης του θραυστήρα στο υπόγειο μεταλλείο, και του οποίου οι επιμέρους εργασίες παρουσιάζονται στον Πίνακα 14.1, παρουσιάζεται στην Εικόνα 14.1.

Από το δικτυωτό διάγραμμα της Εικόνας 14.1 προκύπτει ότι το έργο μας αποτελείται από 11 γεγονότα – κόμβους (event – node) που συμβολίζονται με κύκλους και αριθμούνται από το 1 έως το 11. Κάθε κόμβος σηματοδοτεί την έναρξη ή το πέρας μιας δραστηριότητας (activity). Ο κόμβος με τον αριθμό 1 σηματοδοτεί την έναρξη του έργου και ο κόμβος με τον αριθμό 11 σηματοδοτεί το πέρας του έργου.



Εικόνα 14.1. Το δικτυωτό διάγραμμα της μεθόδου CPM για το έργο της εγκατάστασης θραυστήρα και μεταφορικής ταινίας σε υπόγειο μεταλλείο [Πηγή: Mining Engineering Handbook, SME-AIME, 1st edition, 1973 (paragraph 30.6 Project Control Techniques)].

Κάθε δραστηριότητα συμβολίζεται με ένα βέλος που ενώνει τον κόμβο έναρξης της δραστηριότητας με τον κόμβο πέρατος της δραστηριότητας (κατά τη φορά του βέλους). Ο αριθμός του κόμβου έναρξης (i) πρέπει να είναι μικρότερος του αριθμού του κόμβου πέρατος (j). Στο βέλος σημειώνεται το όνομα της δραστηριότητας και, εντός παρενθέσεων, η χρονική της διάρκεια, η οποία δεν είναι απαραίτητο να είναι ανάλογη του μήκους του βέλους.

Σε ορισμένα δικτυωτά διαγράμματα CPM, όπως στην περίπτωση του έργου μας, είναι απαραίτητο να εντάξουμε ορισμένες πλασματικές (dummy) δραστηριότητες για να έχουμε μια σωστή απεικόνιση της λογικής αλληλουχίας ή της αλληλεξάρτησης των πραγματικών δραστηριοτήτων. Οι πλασματικές δραστηριότητες έχουν μηδενικό χρόνο εκτέλεσης, δεν απαιτούν μέσα παραγωγής και συνήθως αναπαρίστανται στο διάγραμμα με βέλος που έχει διακεκομμένη γραμμή.

Με βάση τα παραπάνω, συντάσσουμε τον Πίνακα 14.2 στον οποίο φαίνονται η αλληλουχία των δραστηριοτήτων του έργου και η χρονική τους διάρκεια. Ο συνολικός αριθμός των δραστηριοτήτων του έργου είναι 13.

Ο αντικειμενικός σκοπός της εφαρμογής της μεθόδου CPM στη διαχείριση του έργου του παραδείγματός μας είναι:

Πίνακας 14.2. Κόμβοι, δραστηριότητες και χρονική διάρκεια αυτών. Διάγραμμα CPM του έργου της εγκατάστασης θραυστήρα και μεταφορικής ταινίας στο υπόγειο μεταλλείο.

Κόμβος Έναρξης (i)	Κόμβος Πέρατος (j)	Περιγραφή Εργασίας	Χρονική Διάρκεια (Εβδομάδες)
1	2	Όρυξη υπόγειου θαλάμου εγκατάστασης του θραυστήρα (CUT CRUSHER STATION)	3
2	3	Κατασκευή δαπέδου και βάσης της μεταφορικής ταινίας από σκυρόδεμα (POUR FLOOR AND CONVEYOR BASE)	1
3	4	Κατασκευή της βάσης του θραυστήρα από σκυρόδεμα (POUR CRUSHER BASE)	2
4	7	Ωρίμανση σκυροδέματος βάσης θραυστήρα (CURE CRUSHER BASE)	2
3	7	Εγκατάσταση της μεταφορικής ταινίας (INSTALL CONVEYOR)	6
1	5	Σχεδίαση του θραυστήρα (DESIGN CRUSHER)	2
5	6	Κατασκευή του θραυστήρα (BUILD CRUSHER)	4
6	7	Αποσυναρμολόγηση θραυστήρα και μεταφορά στη θέση εγκατάστασης (DISASSEMBLE AND MOVE CRUSHER TO STATION)	5
7	8	Συναρμολόγηση του θραυστήρα (ASSEMBLE CRUSHER)	4
7	9	Εργασίες παροχής ηλεκτρικού ρεύματος (INSTALL WIRING)	6
8	10	Πλασματική (DUMMY)	0
9	10	Πλασματική (DUMMY)	0
10	11	Τελικές δοκιμές (FINAL TEST)	1

Με βάση το δικτυωτό διάγραμμα της Εικόνας 14.1 καθώς και τις τιμές του Πίνακα 14.2, και λαμβάνοντας υπόψη τις αλληλουχίες - αλληλεξαρτήσεις των δραστηριοτήτων του έργου, θα προσδιορίσουμε, ύστερα από κατάλληλους υπολογισμούς:

- τον νωρίτερα δυνατό χρόνο έναρξης (Earliest Start Time) κάθε δραστηριότητας,
- τον νωρίτερα δυνατό χρόνο πέρατος (Earliest Finish Time) κάθε δραστηριότητας,
- τον αργότερο δυνατό χρόνο έναρξης (Latest Start Time) κάθε δραστηριότητας, χωρίς να καθυστερήσει η έναρξη των εργασιών που ακολουθούν,
- τον αργότερο δυνατό χρόνο πέρατος (Latest Finish Time) κάθε δραστηριότητας, χωρίς να καθυστερήσει η έναρξη των εργασιών που ακολουθούν,
- το ολικό χρονικό περιθώριο (Total Float Time ή Slack Time) κάθε δραστηριότητας, που ορίζεται ως το χρονικό διάστημα κατά το οποίο μπορεί να υπάρξει υπέρβαση του αρχικού χρόνου εκτέλεσης μιας δραστηριότητας, χωρίς να υπάρξει υπέρβαση του συνολικού χρόνου εκτέλεσης του έργου,
- τις κρίσιμες δραστηριότητες που συγκροτούν την *κρίσιμη διαδρομή* (Critical Path): είναι αυτές που παρουσιάζουν *μηδενικό ολικό χρονικό περιθώριο*,
- τον αναμενόμενο χρόνο εκτέλεσης του έργου (Expected Project Duration), ο οποίος είναι το άθροισμα των χρόνων εκτέλεσης των δραστηριοτήτων της κρίσιμης διαδρομής.

14.3.1 Το πρόγραμμα `cpm.exe`

Το πρόγραμμα `cpm.exe` είναι το εκτελέσιμο αρχείο ενός προγράμματος που κάνει όλους τους υπολογισμούς που απαιτούνται κατά την εφαρμογή της μεθόδου CPM στη διαχείριση ενός έργου. Το έργο μπορεί να αποτελείται το μέγιστο από 100 δραστηριότητες και 100 κόμβους.

Το έγραψε ο συγγραφέας του βιβλίου στις αρχές της δεκαετίας του 1990, όταν εισήγαγε και πρωτοδίδαξε το μάθημα της Διαχείρισης Έργου στη Σχολή ΜΜΜ. Σκοπός του προγράμματος ήταν να καλύψει τις τότε διδακτικές ανάγκες του μαθήματος και γράφτηκε στη γλώσσα προγραμματισμού FORTRAN 77 για προσωπικούς υπολογιστές που έτρεχαν με το λειτουργικό σύστημα MS-DOS.

Την εποχή εκείνη η FORTRAN ήταν η κυρίαρχη γλώσσα προγραμματισμού στην επιστημονική κοινότητα [Σχετικά με τη γλώσσα FORTRAN βλέπε: <https://en.wikipedia.org/wiki/Fortran>].

Το πρόγραμμα αποδείχθηκε αξιόπιστο και χρήσιμο. Οι καιροί όμως άλλαξαν και για να χρησιμοποιηθεί σήμερα σε υπολογιστές που τρέχουν στα Windows 10, αλλά και για να εξυπηρετήσει τους σκοπούς αυτού του Κεφαλαίου, έπρεπε να γίνουν τροποποιήσεις στον αρχικό κώδικά του προκειμένου:

- να διαβάζει τα δεδομένα εισόδου από ένα αρχείο τύπου txt, το όνομα του οποίου είναι `cpm.in` και
- να γράφει τα αποτελέσματα των υπολογισμών που κάνει σε ένα αρχείο τύπου txt, το όνομα του οποίου είναι `cpm.out`

Οι τροποποιήσεις έγιναν χρησιμοποιώντας εντολές της Fortran 95, που είναι έκδοση νεότερη και διαφορετική της FORTRAN 77, αλλά χάρη στην backward compatibility του compiler της Fortran 95, διατηρήθηκαν αρκετές original εντολές της FORTRAN 77, συμπεριλαμβανομένων των εντολών μορφοποίησης των αποτελεσμάτων του προγράμματος.

Για τις παραπάνω εργασίες χρησιμοποιήθηκε το Integrated Development Environment (IDE) της NetBeans Ver. 8.2, που είναι κατάλληλο για την ανάπτυξη προγραμμάτων στις γλώσσες Fortran, C, C++ και διατίθεται δωρεάν από το website: <https://netbeans.apache.org/>

Για την εκσφαλμάτωση και τη δημιουργία του εκτελέσιμου αρχείου χρησιμοποιήθηκε ο compiler της GNU Fortran, που διατίθεται δωρεάν από το website: <https://gcc.gnu.org/fortran/>

Το πρόγραμμα `cpm.exe` είναι διαθέσιμο στο website του βιβλίου.

Ιστορικό Σημείωμα:

Στην παρακάτω εντολή `format` της Fortran 95, παρμένη από τον κώδικα του προγράμματος `cpm.exe`:

```
520    format (10X, 36HCPM TIME ESTIMATES AND CRITICAL PATH)
```

με την οποία μορφοποιείται το λεκτικό της επικεφαλίδας των αποτελεσμάτων του προγράμματος, υπάρχει ο τελεστής 36H (μαρκαρισμένος με κίτρινο χρώμα για εύκολη διάκριση) ο οποίος λέει στον compiler της Fortran ότι η συμβολοσειρά CPM TIME ESTIMATES AND CRITICAL PATH που ακολουθεί αποτελείται από 36 σύμβολα (χαρακτήρες) συμπεριλαμβανομένων των κενών διαστημάτων (Hollerith string).

Ο τελεστής ονομάστηκε H προς τιμή του Herman Hollerith, Μηχανικού Μεταλλείων και εφευρέτη της μηχανής διάτρησης καρτών και της μηχανής πινακοποίησης διάτρητων καρτών.

Αυτές οι εφευρέσεις έθεσαν τις βάσεις για τη σύγχρονη βιομηχανία επεξεργασίας της πληροφορίας.

Η εταιρεία του Hollerith συνενώθηκε μαζί με άλλες τρεις εταιρείες το 1911 και δημιούργησαν την CTR, η οποία το 1924 μετονομάστηκε σε IBM [Σχετικά με τον Herman Hollerith και τις εφευρέσεις του ανατρέξτε στο: https://en.wikipedia.org/wiki/Herman_Hollerith].

Στις σύγχρονες εκδόσεις της Fortran, η χρήση του τελεστή H δεν συνιστάται, γιατί έχει αντικατασταθεί με την απλούστερη ισοδύναμη μορφή, 'string':

```
520    format (10X, 'CPM TIME ESTIMATES AND CRITICAL PATH')
```

14.3.2 Το πρόγραμμα CPM.py

Εκμεταλλευόμενοι το γεγονός ότι έχουμε ένα έτοιμο πρόγραμμα, το `cpm.exe`, σε εκτελέσιμη μορφή (binary file), με το οποίο μπορούμε να κάνουμε τη δουλειά μας, δηλαδή να κάνουμε τους υπολογισμούς κατά την

εφαρμογή της μεθόδου CPM στη διαχείριση ενός έργου, θα γράψουμε τον κώδικα ενός προγράμματος στην Python, με το οποίο ο χρήστης θα κάνει τα εξής:

1. Θα εισάγει τα δεδομένα του έργου που απαιτούνται για την εφαρμογή της μεθόδου CPM, από το πληκτρολόγιο, και αυτά θα αποθηκεύονται στο αρχείο με όνομα `cpm.in`
2. Θα δίνει εντολή στο λειτουργικό σύστημα του υπολογιστή (τα Windows 10) να τρέξει το πρόγραμμα/εκτελέσιμο αρχείο `cpm.exe` χρησιμοποιώντας ως δεδομένα εισόδου αυτά που περιέχονται στο αρχείο `cpm.in`
3. Θα διαβάζει το περιεχόμενο του αρχείου `cpm.out` στο οποίο το πρόγραμμα `cpm.exe` αποθήκευσε τα αποτελέσματα της εκτέλεσής του και θα τα εμφανίζει στην οθόνη του υπολογιστή.

Γράφουμε, αποθηκεύουμε ως `CPM.py` και τρέχουμε τον παρακάτω κώδικα:

```
import subprocess
# Create input file 'cpm.in' to serve as the data file
# for the Fortran program 'cpm.exe'
f = open('cpm.in', 'w')
f.truncate() # erase all data
print('PROJECT MANAGEMENT')
print('CPM TIME ESTIMATES AND CRITICAL PATH')
print('INPUT DATA:')
nodes = int(input('Number of Events (nodes) [integer] :'))
print('{:d}'.format(nodes), file=f)
activities = int(input('Number of Activities [integer] :'))
print('{:d}'.format(activities), file=f)
for i in range(1, activities + 1):
    print('Activity Number : {:d}'.format(i))
    act = int(input('Number of Starting Event [integer] :'))
    next_act = int(input('Number of Ending Event [integer] :'))
    duration = float(input('Estimated Duration Time [real, time units] :'))
    print('{:d}, {:d}, {:.3f}'.format(act, next_act, duration), file=f)
f.close()
# Run 'cpm.exe' to read input data file 'cpm.in' & do CPM calculations
programName = 'cpm.exe'
fileName = 'cpm.in'
p = subprocess.run([programName, fileName])
if p.returncode != 0:
    raise RuntimeError('subprocess did not run right! \n {}'.format(p.stderr))
# Open output file 'cpm.out', read 'cpm.exe' results & print them
cpm_output = open('cpm.out', 'r')
output_text = cpm_output.read()
print(output_text)
cpm_output.close()
```

Τα αρχεία `CPM.py`, `cpm.in` και `cpm.exe` πρέπει να βρίσκονται στο ίδιο directory, στο οποίο θα αποθηκευτεί επίσης και το αρχείο `cpm.out`

Με την πρώτη γραμμή κώδικα κάνουμε `import` το module `subprocess` που υπάρχει στη standard βιβλιοθήκη της Python και δεν απαιτείται εγκατάσταση. Για την τεκμηρίωση του module ανατρέξτε στο website: <https://docs.python.org/3/library/subprocess.html>

Με τις επόμενες γραμμές του κώδικα:

Δημιουργούμε ένα αντικείμενο αρχείου, το `f`, επί του οποίου μπορούμε να εφαρμόσουμε μεθόδους. Η συνάρτηση `open()` δέχεται δύο ορίσματα: το πρώτο όρισμα είναι το όνομα του αρχείου που δημιουργούμε και ανοίγουμε και το δεύτερο όρισμα είναι ο τρόπος ανοίγματος (mode), που στην περίπτωσή μας είναι `'w'`, δηλαδή ανοίγουμε το αρχείο για γράψιμο (write).

Το όνομα του αρχείου πρέπει να είναι `cpm.in`, γιατί αρχείο δεδομένων με αυτό το όνομα μόνο μπορεί να διαβάσει το πρόγραμμα `cpm.exe`

Εάν δεν υπάρχει ήδη το αρχείο `cpm.in`, π.χ. από προηγούμενο τρέξιμο του `cpm.py`, τότε δημιουργείται. Εάν υπάρχει το αρχείο `cpm.in` τότε η εντολή:

```
f.truncate() # erase all data
```

σβήνει όλα τα δεδομένα που ήταν αποθηκευμένα σε αυτό, προκειμένου να δεχθεί καινούρια.

Στην οθόνη εμφανίζεται προτροπή για να πληκτρολογήσουμε:

- τον αριθμό των γεγονότων-κόμβων,
- τον αριθμό των δραστηριοτήτων,
- για κάθε δραστηριότητα, το γεγονός-κόμβο έναρξης, το γεγονός-κόμβο λήξης και τη χρονική διάρκεια της δραστηριότητας.

Τα παραπάνω δεδομένα αποθηκεύονται στο αρχείο και με την εντολή `f.close()` γίνεται το κλείσιμο του αρχείου `cpm.in`

Στη συνέχεια, παρουσιάζεται ένα απόσπασμα της οθόνης εισαγωγής δεδομένων του έργου μας, με βάση τα στοιχεία που προκύπτουν από τον Πίνακα 14.2:

```
PROJECT MANAGEMENT
CPM TIME ESTIMATES AND CRITICAL PATH
INPUT DATA:
Number of Events (nodes) [integer] :11
Number of Activities [integer] :13
Activity Number : 1
Number of Starting Event [integer] :1
Number of Ending Event [integer] :2
Estimated Duration Time [real, time units] :3
Activity Number : 2
Number of Starting Event [integer] :2
Number of Ending Event [integer] :3
Estimated Duration Time [real, time units] :1
.....
Activity Number : 13
Number of Starting Event [integer] :10
Number of Ending Event [integer] :11
Estimated Duration Time [real, time units] :1
```

Εάν ανοίξουμε το αρχείο `cpm.in` με το Notepad (Εικόνα 14.2), θα παρατηρήσουμε τη μορφή με την οποία αποθηκεύονται τα δεδομένα εισόδου προκειμένου να διαβαστούν σωστά από τον πρόγραμμα `cpm.exe` που είναι γραμμένο στη Fortran.

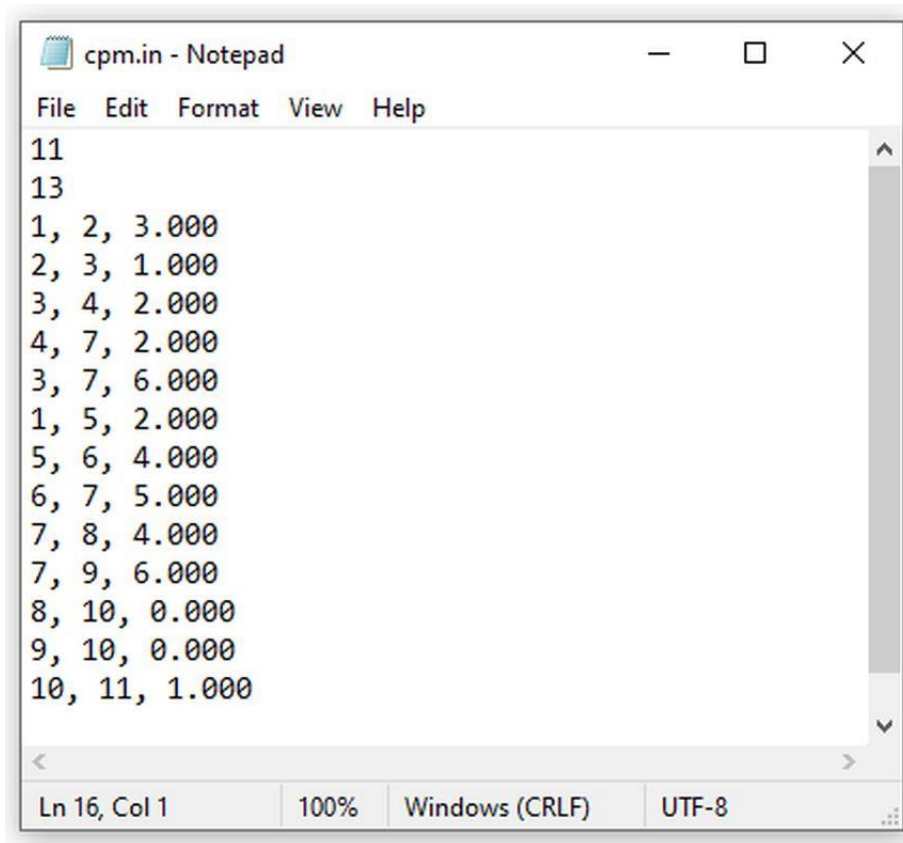
Η γραμμή κώδικα:

```
p = subprocess.run([programName, fileName])
```

εφαρμόζει τη μέθοδο `run()` του module `subprocess` με τις παραμέτρους:

`programName` στην οποία έχουμε εκχωρήσει προηγουμένως το όνομα το εκτελέσιμου αρχείου `cpm.exe` και την

`fileName` στην οποία έχουμε εκχωρήσει προηγουμένως το όνομα του αρχείου των δεδομένων `cpm.in`



Εικόνα 14.2. Το περιεχόμενο του αρχείου `cpm.in` που περιλαμβάνει τα δεδομένα εισόδου του προγράμματος `cpm.exe` όπως εμφανίζεται στο Notepad.

και δίνει εντολή στο λειτουργικό σύστημα του υπολογιστή μας να τρέξει το πρόγραμμα `cpm.exe` με δεδομένα εισόδου τα δεδομένα που έχουμε αποθηκεύσει στο αρχείο `cpm.in`

Εάν όλα πάνε καλά, τότε η παράμετρος `p.returncode` της μεθόδου `subprocess.run()` θα λάβει εσωτερικά, χωρίς να εμφανιστεί σε εμάς, την τιμή 0, και ο interpreter της Python θα προχωρήσει στην εκτέλεση των επόμενων γραμμών του κώδικα.

Το μπλοκ `if` που ακολουθεί:

```
if p.returncode != 0:
    raise RuntimeError('subprocess did not run right! \n {}'.format(p.stderr))
```

ενεργοποιείται μόνο στην περίπτωση που κάτι πάει λάθος με την εκτέλεση της μεθόδου `subprocess.run()`, οπότε η τιμή της `p.returncode` δεν θα είναι ίση με 0.

Στην περίπτωση αυτή εγείρεται ένα Run Time Error και στην οθόνη εμφανίζεται ο κωδικός του λάθους, ο οποίος θα μας διευκολύνει στην επίλυση του προβλήματος που παρουσιάστηκε.

Το πρόγραμμα `cpm.exe` αποθηκεύει, κάθε φορά που το τρέχουμε, τα αποτελέσματα των υπολογισμών της εφαρμογής της μεθόδου CPM στο αρχείο `cpm.out`

Εάν ανοίξετε το αρχείο `cpm.out` με το Notepad (Εικόνα 14.3) θα παρατηρήσετε τη μορφή με την οποία αποθηκεύονται τα αποτελέσματα, όντας φορμαρισμένα με εντολές της Fortran.

```
cpm.out - Notepad
File Edit Format View Help

-----
          CPM TIME ESTIMATES AND CRITICAL PATH
-----
NUMBER OF EVENTS      = 11
NUMBER OF ACTIVITIES = 13
-----

ACTIVITY TIME   EARLIEST   LATEST   SLACK
 I   J         START FINISH START FINISH  TIME
-----
 1   2   3.0   0.0   3.0   1.0   4.0   1.0
 2   3   1.0   3.0   4.0   4.0   5.0   1.0
 3   4   2.0   4.0   6.0   7.0   9.0   3.0
 4   7   2.0   6.0   8.0   9.0  11.0   3.0
 3   7   6.0   4.0  10.0   5.0  11.0   1.0
 1   5   2.0   0.0   2.0   0.0   2.0   0.0
 5   6   4.0   2.0   6.0   2.0   6.0   0.0
 6   7   5.0   6.0  11.0   6.0  11.0   0.0
 7   8   4.0  11.0  15.0  13.0  17.0   2.0
 7   9   6.0  11.0  17.0  11.0  17.0   0.0
 8  10   0.0  15.0  15.0  17.0  17.0   2.0
 9  10   0.0  17.0  17.0  17.0  17.0   0.0
10  11   1.0  17.0  18.0  17.0  18.0   0.0
-----

CRITICAL PATH ACTIVITIES
      1 - 5
      5 - 6
      6 - 7
      7 - 9
      9 -10
      10 -11
-----

EXPECTED PROJECT DURATION (TIME UNITS) = 18.00
-----

Ln 1, Col 1    100%    Windows (CRLF)    UTF-8
```

Εικόνα 14.3. Το περιεχόμενο του αρχείου cpm.out που περιλαμβάνει τα αποτελέσματα του προγράμματος cpm.exe όπως εμφανίζεται στο Notepad.

Ο κώδικας τελειώνει με τις εντολές οι οποίες ανοίγουν το αρχείο cpm.out, σε mode ανάγνωσης (read), διαβάζουν το περιεχόμενό του και μας το εμφανίζουν στην οθόνη, όπως ακριβώς αυτό έχει φορμαριστεί από το πρόγραμμα cpm.exe της Fortran:

CPM TIME ESTIMATES AND CRITICAL PATH

NUMBER OF EVENTS = 11

NUMBER OF ACTIVITIES = 13

ACTIVITY TIME			EARLIEST		LATEST		SLACK
I	J		START	FINISH	START	FINISH	TIME
1	2	3.0	0.0	3.0	1.0	4.0	1.0
2	3	1.0	3.0	4.0	4.0	5.0	1.0
3	4	2.0	4.0	6.0	7.0	9.0	3.0
4	7	2.0	6.0	8.0	9.0	11.0	3.0
3	7	6.0	4.0	10.0	5.0	11.0	1.0
1	5	2.0	0.0	2.0	0.0	2.0	0.0
5	6	4.0	2.0	6.0	2.0	6.0	0.0
6	7	5.0	6.0	11.0	6.0	11.0	0.0
7	8	4.0	11.0	15.0	13.0	17.0	2.0
7	9	6.0	11.0	17.0	11.0	17.0	0.0
8	10	0.0	15.0	15.0	17.0	17.0	2.0
9	10	0.0	17.0	17.0	17.0	17.0	0.0
10	11	1.0	17.0	18.0	17.0	18.0	0.0

CRITICAL PATH ACTIVITIES

1 - 5
5 - 6
6 - 7
7 - 9
9 - 10
10 - 11

EXPECTED PROJECT DURATION (TIME UNITS) = 18.00

Η κρίσιμη διαδρομή που συνδέει τους κόμβους των κρίσιμων δραστηριοτήτων και η οποία προέκυψε από τους υπολογισμούς του προγράμματος CPM. py, έχει σχεδιαστεί στο δικτυωτό διάγραμμα της μεθόδου CPM για το έργο της εγκατάστασης θραυστήρα και μεταφορικής ταινίας στο υπόγειο μεταλλείο, στην Εικόνα 14.1, με έντονες (bold) γραμμές και χαρακτήρες.

Οι πληροφορίες που μας δίνουν τα αποτελέσματα του προγράμματός μας είναι εξαιρετικά σημαντικές για τον χρονικό προγραμματισμό του έργου και την εν γένει διαχείρισή του.

Η πλήρης ερμηνεία όμως των αποτελεσμάτων της εφαρμογής της μεθόδου CPM στη διαχείριση ενός έργου, είναι πέραν του σκοπού αυτού του βιβλίου.

Για παράδειγμα, θα αναφέρουμε μόνο ότι από την ανάγνωση της πρώτης γραμμής του πίνακα των δραστηριοτήτων στα αποτελέσματα, που είναι μαρκαρισμένη με κίτρινο χρώμα από τον συγγραφέα του βιβλίου, συμπεραίνουμε ότι για τη δραστηριότητα *Όρυξη υπόγειου θαλάμου εγκατάστασης του θραυστήρα (CUT CRUSHER STATION)*, ACTIVITY 1 – 2, στην οποία έχει δοθεί χρονική διάρκεια 3 εβδομάδων, υπάρχει ολικό χρονικό περιθώριο (Slack Time) μίας (1) εβδομάδας.

Αυτό πρακτικά σημαίνει ότι η δραστηριότητα αυτή είτε μπορεί να αρχίσει μια εβδομάδα αργότερα από την προγραμματισμένη ημερομηνία έναρξης είτε να αρχίσει στην προγραμματισμένη ημερομηνία αλλά αντί να τελειώσει εντός των προγραμματισμένων 3 εβδομάδων, να καθυστερήσει μία επιπλέον εβδομάδα, χωρίς να υπάρξει καμία καθυστέρηση στον συνολικό χρόνο εκτέλεσης του όλου έργου.

Εν κατακλείδι, θα πρέπει να επισημάνουμε την πρακτική αξία που έχει το module subprocess της Python, καθώς μας επιτρέπει να αξιοποιούμε χρήσιμους κώδικες, τους οποίους έχουμε αναπτύξει στο παρελθόν σε άλλες γλώσσες (π.χ. C, C++, Fortran κλπ.), και να ενσωματώνουμε τη λειτουργία (τρέξιμο) των εκτελέσιμων αρχείων τους σε προγράμματα της Python.

Το script του κώδικα CPM.py βρίσκεται στο website του βιβλίου στο αρχείο: Chapter_14_CPM.py

14.4 Δικτυωτός χρονικός προγραμματισμός έργων: PERT

Ο δικτυωτός προγραμματισμός έργων με την τεχνική προγραμματισμού έργων PERT (Program Evaluation and Review Technique) χρησιμοποιείται όταν υπάρχει αδυναμία εκτίμησης της ακριβούς διάρκειας του χρόνου εκτέλεσης των επιμέρους εργασιών ενός έργου και κρίνεται σκόπιμο να εισάγουμε μια στοχαστική εκτίμηση αυτής της διάρκειας.

Σχετικά με τη μέθοδο PERT βλέπε στο website:

https://en.wikipedia.org/wiki/Program_evaluation_and_review_technique

Στην εφαρμογή της, η μέθοδος PERT ακολουθεί την ίδια λογική με τη μέθοδο CPM, με τη διαφορά ότι στην PERT, για κάθε επιμέρους εργασία ενός έργου, πρέπει να δώσουμε τρεις τιμές του εκτιμώμενου χρόνου εκτέλεσης:

- τον αισιόδοξο χρόνο εκτέλεσης [A] (Optimistic Time),
- τον πιθανότερο χρόνο εκτέλεσης [M] (Most Likely Time),
- τον απαισιόδοξο χρόνο εκτέλεσης [B] (Pessimistic Time).

Προκειμένου να εφαρμόσουμε τη μέθοδο PERT στο έργο της εγκατάστασης θραυστήρα και μεταφορικής ταινίας στο υπόγειο μεταλλείο που περιγράψαμε στην παράγραφο 14.2 *Το προς διαχείριση έργο*, θα τροποποιήσουμε τον Πίνακα 14.2 ώστε να περιλαμβάνει τους αισιόδοξους, πιθανούς και απαισιόδοξους χρόνους εκτέλεσης κάθε εργασίας, και ο οποίος θα λάβει τη μορφή του Πίνακα 14.3.

Πίνακας 14.3. Κόμβοι, δραστηριότητες και χρονική διάρκεια αυτών για την εφαρμογή της μεθόδου PERT στο έργο της εγκατάστασης θραυστήρα και μεταφορικής ταινίας στο υπόγειο μεταλλείο. Α: Αισιόδοξος χρόνος εκτέλεσης, Μ: Πιθανός χρόνος εκτέλεσης, Β: Απαισιόδοξος χρόνος εκτέλεσης. Όλοι οι χρόνοι είναι σε εβδομάδες.

Κόμβος Έναρξης (i)	Κόμβος Πέρατος (j)	Περιγραφή Εργασίας	A	M	B
1	2	Όρυξη υπόγειου θαλάμου εγκατάστασης του θραυστήρα (CUT CRUSHER STATION)	2.5	3	3.5
2	3	Κατασκευή δαπέδου και βάσης της μεταφορικής ταινίας από σκυρόδεμα (POUR FLOOR AND CONVEYOR BASE)	1	1.5	2
3	4	Κατασκευή της βάσης του θραυστήρα από σκυρόδεμα (POUR CRUSHER BASE)	1	2	2.5
4	7	Ωρίμανση σκυροδέματος βάσης θραυστήρα (CURE CRUSHER BASE)	2	2	2
3	7	Εγκατάσταση της μεταφορικής ταινίας (INSTALL CONVEYOR)	5.5	6	7
1	5	Σχεδίαση του θραυστήρα (DESIGN CRUSHER)	2.5	3	3.5
5	6	Κατασκευή του θραυστήρα (BUILD CRUSHER)	3.5	4.5	6
6	7	Αποσυναρμολόγηση θραυστήρα και μεταφορά στη θέση εγκατάστασης (DISASSEMBLE AND MOVE CRUSHER TO STATION)	4	5	6.5
7	8	Συναρμολόγηση του θραυστήρα (ASSEMBLE CRUSHER)	3.5	5	5.5
7	9	Εργασίες παροχής ηλεκτρικού ρεύματος (INSTALL WIRING)	3	4.5	6
8	10	Πλασματική (DUMMY)	0	0	0
9	10	Πλασματική (DUMMY)	0	0	0

10	11	Τελικές δοκιμές (FINAL TEST)	1	1.5	2
----	----	------------------------------	---	-----	---

Η τεχνική PERT, προκειμένου να υπολογίσει τον αναμενόμενο χρόνο εκτέλεσης καθεμίας εργασίας και τη μεταβλητότητα (διασπορά) των τιμών, χρησιμοποιεί μια τροποποιημένη μορφή της *στατιστικής κατανομής Beta*, γνωστή και ως *κατανομή PERT*. Σχετικά με την κατανομή PERT ανατρέξτε στο website: https://en.wikipedia.org/wiki/PERT_distribution

Με βάση την κατανομή PERT, ο αναμενόμενος χρόνος εκτέλεσης (TIME) μιας δραστηριότητας δίνεται από τη σχέση:

$$\text{TIME} = (A + (4 * M) + B) / 6 \quad [14.1]$$

και η μεταβλητότητα (VARIANCE) από τη σχέση:

$$\text{VARIANCE} = ((B - A) / 6)^2 \quad [14.2]$$

όπου:

A : ο αισιόδοξος χρόνος εκτέλεσης

M : ο πιθανότερος χρόνος εκτέλεσης

B : ο απαισιόδοξος χρόνος εκτέλεσης

14.4.1 Το πρόγραμμα `pert.exe`

Το πρόγραμμα `pert.exe` είναι το εκτελέσιμο αρχείο ενός προγράμματος που κάνει τους υπολογισμούς κατά την εφαρμογή της μεθόδου PERT στη διαχείριση ενός έργου. Το έργο μπορεί να αποτελείται το μέγιστο από 100 δραστηριότητες και 100 κόμβους.

Το έγραψε ο συγγραφέας του βιβλίου στις αρχές της δεκαετίας του 1990, σε γλώσσα FORTRAN 77, όπως και το πρόγραμμα `crp.exe`, στο πλαίσιο της διδασκαλίας του μαθήματος *Διαχείριση Έργου*.

Το πρόγραμμα τροποποιήθηκε, χρησιμοποιώντας εντολές της Fortran 95, για να τρέχει σε υπολογιστές με το λειτουργικό σύστημα Windows 10 και για να εξυπηρετήσει του σκοπούς του βιβλίου, προκειμένου:

- να διαβάζει τα δεδομένα εισόδου από ένα αρχείο τύπου `txt`, το όνομα του οποίου είναι `pert.in` και
- να γράφει τα αποτελέσματα των υπολογισμών που κάνει σε ένα αρχείο τύπου `txt`, το όνομα του οποίου είναι `pert.out`

Το πρόγραμμα `pert.exe` είναι διαθέσιμο στο website του βιβλίου.

14.4.2 Το πρόγραμμα `PERT.py`

Με δεδομένο ότι έχουμε ένα έτοιμο πρόγραμμα, το `pert.exe`, σε εκτελέσιμη μορφή (binary file), με το οποίο μπορούμε να κάνουμε τους υπολογισμούς κατά την εφαρμογή της μεθόδου PERT στη διαχείριση ενός έργου, θα γράψουμε τον κώδικα ενός προγράμματος Python, με το οποίο ο χρήστης θα κάνει τα εξής:

1. Θα εισάγει τα δεδομένα του έργου που απαιτούνται για την εφαρμογή της μεθόδου PERT, από το πληκτρολόγιο, και αυτά θα αποθηκεύονται στο αρχείο με όνομα `pert.in`
2. Θα δίνει εντολή στο λειτουργικό σύστημα του υπολογιστή μας να τρέξει το πρόγραμμα/εκτελέσιμο αρχείο `pert.exe` χρησιμοποιώντας ως δεδομένα εισόδου αυτά που περιέχονται στο αρχείο `pert.in`
3. Θα διαβάζει το περιεχόμενο του αρχείου `pert.out` στο οποίο το πρόγραμμα `pert.exe` αποθήκευσε τα αποτελέσματα της εκτέλεσής του, και θα τα εμφανίζει στην οθόνη του υπολογιστή.

Γράφουμε, αποθηκεύουμε ως PERT.py και τρέχουμε τον παρακάτω κώδικα:

```
import subprocess
# Create input file 'pert.in' to serve as the data file
# for the Fortran program 'pert.exe'
f = open('pert.in', 'w')
f.truncate() # erase all data
print('PROJECT MANAGEMENT')
print('PERT TIME ESTIMATES AND CRITICAL PATH')
print('INPUT DATA:')
nodes = int(input('Number of Events (nodes) [integer] :'))
print('{:d}'.format(nodes), file=f)
activities = int(input('Number of Activities [integer] :'))
print('{:d}'.format(activities), file=f)
for i in range(1, activities + 1):
    print('Activity Number : {:d}'.format(i))
    act = int(input('Number of Starting Event [integer] :'))
    next_act = int(input('Number of Ending Event [integer] :'))
    a = float(input('Estimated Optimistic Duration Time '
                    '[real, time units] :'))
    m = float(input('Estimated Most-Likely Duration Time '
                    '[real, time units] :'))
    b = float(input('Estimated Pessimistic Duration Time '
                    '[real, time units] :'))
    print('{:d}, {:d}, {:.3f}, {:.3f}, {:.3f}'
          .format(act, next_act, a, m, b), file=f)
f.close()
# Run 'pert.exe' to read input data file 'pert.in' &
# do PERT calculations
programName = 'pert.exe'
fileName = 'pert.in'
p = subprocess.run([programName, fileName])
if p.returncode != 0:
    raise RuntimeError('subprocess did not run right! \n {}'.
                       .format(p.stderr))
# Open output file 'pert.out', read 'pert.exe' results & print them
pert_output = open('pert.out', 'r')
output_text = pert_output.read()
print(output_text)
pert_output.close()
```

Τα αρχεία PERT.py, pert.in και pert.exe πρέπει να βρίσκονται στο ίδιο directory, στο οποίο θα αποθηκευτεί επίσης και το αρχείο pert.out

Η δομή του κώδικα είναι παρόμοια με εκείνη του προγράμματος CPM.py, το οποίο αναλύσαμε στην παράγραφο 14.3.2 Το πρόγραμμα CPM.py, με τη διαφορά ότι τώρα στην οθόνη εμφανίζεται η προτροπή για να πληκτρολογήσουμε για κάθε δραστηριότητα: το γεγονός-κόμβο έναρξης, το γεγονός-κόμβο λήξης, την αισιόδοξη χρονική διάρκεια της δραστηριότητας, την πιθανή χρονική διάρκεια της δραστηριότητας και την απαισιόδοξη χρονική διάρκεια της δραστηριότητας.

Στη συνέχεια, παρουσιάζεται ένα απόσπασμα της οθόνης εισαγωγής δεδομένων του έργου, με βάση τα στοιχεία που προκύπτουν από τον Πίνακα 14.3:

```

PROJECT MANAGEMENT
PERT TIME ESTIMATES AND CRITICAL PATH
INPUT DATA:
Number of Events (nodes) [integer] :11
Number of Activities [integer] :13
Activity Number : 1
Number of Starting Event [integer] :1
Number of Ending Event [integer] :2
Estimated Optimistic Duration Time [real, time units] :2.5
Estimated Most-Likely Duration Time [real, time units] :3
Estimated Pessimistic Duration Time [real, time units] :3.5
Activity Number : 2
Number of Starting Event [integer] :2
Number of Ending Event [integer] :3
Estimated Optimistic Duration Time [real, time units] :1
Estimated Most-Likely Duration Time [real, time units] :1.5
Estimated Pessimistic Duration Time [real, time units] :2
.....
Activity Number : 13
Number of Starting Event [integer] :10
Number of Ending Event [integer] :11
Estimated Optimistic Duration Time [real, time units] :1
Estimated Most-Likely Duration Time [real, time units] :1.5
Estimated Pessimistic Duration Time [real, time units] :2

```

Εάν ανοίξετε το αρχείο των αποτελεσμάτων `pert.out` του προγράμματος με το Notepad (Εικόνα 14.4), θα παρατηρήσετε τη μορφή με την οποία αποθηκεύονται τα αποτελέσματα, όντας φορμαρισμένα με εντολές της Fortran.

*pert.out - Notepad

File Edit Format View Help

PERT TIME ESTIMATES AND CRITICAL PATH

NUMBER OF EVENTS = 11
NUMBER OF ACTIVITIES = 13

ACTIVITY		TIME ESTIMATES			TIME	VARIANCE
I	J	A	M	B		
1	2	2.5	3.0	3.5	3.0	0.03
2	3	1.0	1.5	2.0	1.5	0.03
3	4	1.0	2.0	2.5	1.9	0.06
4	7	2.0	2.0	2.0	2.0	0.00
3	7	5.5	6.0	7.0	6.1	0.06
1	5	2.5	3.0	3.5	3.0	0.03
5	6	3.5	4.5	6.0	4.6	0.17
6	7	4.0	5.0	6.5	5.1	0.17
7	8	3.5	5.0	5.5	4.8	0.11
7	9	3.0	4.5	6.0	4.5	0.25
8	10	0.0	0.0	0.0	0.0	0.00
9	10	0.0	0.0	0.0	0.0	0.00
10	11	1.0	1.5	2.0	1.5	0.03

ACTIVITY		TIME	EARLIEST		LATEST		SLACK
I	J		START	FINISH	START	FINISH	TIME
1	2	3.0	0.0	3.0	2.1	5.1	2.1
2	3	1.5	3.0	4.5	5.1	6.6	2.1
3	4	1.9	4.5	6.4	8.7	10.7	4.2
4	7	2.0	6.4	8.4	10.7	12.7	4.2
3	7	6.1	4.5	10.6	6.6	12.7	2.1
1	5	3.0	0.0	3.0	-0.0	3.0	-0.0
5	6	4.6	3.0	7.6	3.0	7.6	-0.0
6	7	5.1	7.6	12.7	7.6	12.7	-0.0
7	8	4.8	12.7	17.5	12.7	17.5	-0.0
7	9	4.5	12.7	17.2	13.0	17.5	0.3
8	10	0.0	17.5	17.5	17.5	17.5	0.0
9	10	0.0	17.2	17.2	17.5	17.5	0.3
10	11	1.5	17.5	19.0	17.5	19.0	0.0

CRITICAL PATH ACTIVITIES

1 - 5
5 - 6
6 - 7
7 - 8
8 - 10
10 - 11

EXPECTED PROJECT DURATION (TIME UNITS) = 19.00
SUM OF VARIANCES OF ACTIVITIES ON CRITICAL PATH = 0.51

Ln 51, Col 56 100% Windows (CRLF) UTF-8

Εικόνα 14.4. Το περιεχόμενο του αρχείου `pert.out` που περιλαμβάνει τα αποτελέσματα του προγράμματος `pert.exe` όπως εμφανίζεται στο Notepad.

Με την ίδια μορφή форматіςματος εμφανίζονται και στην οθόνη του υπολογιστή μας τα αποτελέσματα της εκτέλεσης του προγράμματος `PERT.py`, καθόσον οι τελευταίες εντολές του κώδικα αυτό που κάνουν είναι να ανοίξουν το αρχείο `pert.out`, να διαβάζουν το περιεχόμενό του και να μας το εμφανίζουν αυτούσιο στην οθόνη.

Η ερμηνεία των αποτελεσμάτων της εφαρμογής της μεθόδου PERT στη διαχείριση ενός έργου είναι πέραν του σκοπού αυτού του Κεφαλαίου. Οι πληροφορίες όμως που μας δίνουν τα αποτελέσματα είναι εξαιρετικά σημαντικές για τον χρονικό προγραμματισμό του έργου και την εν γένει διαχείρισή του.

Επειδή η τεχνική PERT, προκειμένου να υπολογίσει τον αναμενόμενο χρόνο εκτέλεσης καθεμίας εργασίας, χρησιμοποιεί μια στατιστική κατανομή, είναι προφανές ότι και η κρίσιμη διαδρομή που προκύπτει από τους υπολογισμούς έχει και αυτή στοχαστικό χαρακτήρα.

Για τον project manager έχει ιδιαίτερο ενδιαφέρον να μπορεί να εκτιμήσει την πιθανότητα που υπάρχει να εκτελεστεί ένα έργο εντός ενός συγκεκριμένου χρονικού διαστήματος (DD – Due Date).

Εάν υποθέσουμε ότι η κατανομή των συχνοτήτων εμφάνισης του συνολικού χρόνου εκτέλεσης των δραστηριοτήτων είναι κανονική, με αναμενόμενο χρόνο εκτέλεσης T , δηλαδή τις 19 εβδομάδες, οι οποίες αντιστοιχούν στον χρόνο εκτέλεσης των δραστηριοτήτων της κρίσιμης διαδρομής του έργου μας, και μεταβλητότητα v , δηλαδή το άθροισμα των μεταβλητοτήτων των δραστηριοτήτων της κρίσιμης διαδρομής του προβλήματος (0.51), τότε μπορούμε να υπολογίσουμε την *ανηγμένη απόκλιση Z* από τη σχέση:

$$Z = (DD - T) / \sqrt{v} \quad [14.3]$$

Εφαρμόζοντας τη σχέση [14.3] για τις τιμές του έργου μας που υπολογίσαμε με τη μέθοδο PERT και για επιθυμητό χρόνο εκτέλεσης του έργου της εγκατάστασης θραυστήρα και μεταφορικής ταινίας στο υπόγειο μεταλλείο (DD) τις 18 εβδομάδες, που υπολογίσαμε με τη μέθοδο CPM στην παράγραφο 14.3.2 *Το πρόγραμμα CPM.py*, λαμβάνουμε:

$$Z = (18 - 19) / \sqrt{0.51} = -1.4 \quad [14.4]$$

Με βάση την τιμή της ανηγμένης απόκλισης Z που προέκυψε από την Εξίσωση [14.4], εκτιμούμε ότι η πιθανότητα εκτέλεσης του έργου σε 18 εβδομάδες είναι 0.91924 δηλαδή 92%.

Η τιμή της πιθανότητας εκτέλεσης του έργου προέκυψε από πίνακες που συσχετίζουν την τιμή της Z με την πιθανότητα για την κανονική κατανομή και υπάρχουν στο website:
https://en.wikipedia.org/wiki/Standard_normal_table

Το script του κώδικα `PERT.py` βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_14_PERT.py`

14.5 Το διάγραμμα παρακολούθησης εκτέλεσης έργου Gantt

Το *διάγραμμα Gantt* είναι ένα οριζόντιο ραβδόγραμμα (bar chart), το οποίο απεικονίζει τις δραστηριότητες ενός έργου και τις αντίστοιχες διάρκειές τους.

Η βασικότερη χρήση του είναι για την παρακολούθηση της προόδου εκτέλεσης ενός έργου. Αναπτύχθηκε ως εργαλείο ελέγχου της παραγωγής το 1917 από τον Αμερικανό Μηχανολόγο Μηχανικό Henry Gantt, στον οποίο οφείλει και το όνομα [βλέπε: https://en.wikipedia.org/wiki/Gantt_chart].

Στην Εικόνα 14.5 παρουσιάζεται το διάγραμμα Gantt του έργου της εγκατάστασης του θραυστήρα και μεταφορικής ταινίας στο υπόγειο μεταλλείο, το οποίο περιγράψαμε στην παράγραφο 14.2 *Το προς διαχείριση έργο*.

Στον οριζόντιο άξονα του διαγράμματος τοποθετείται ο χρόνος, με κατάλληλες υποδιαιρέσεις που ταιριάζουν στις ανάγκες μας και στη χρονική διάρκεια του έργου, ενώ στον κατακόρυφο άξονα ή σπάνια επάνω στις ράβδους, τοποθετούνται οι τίτλοι των δραστηριοτήτων του έργου.

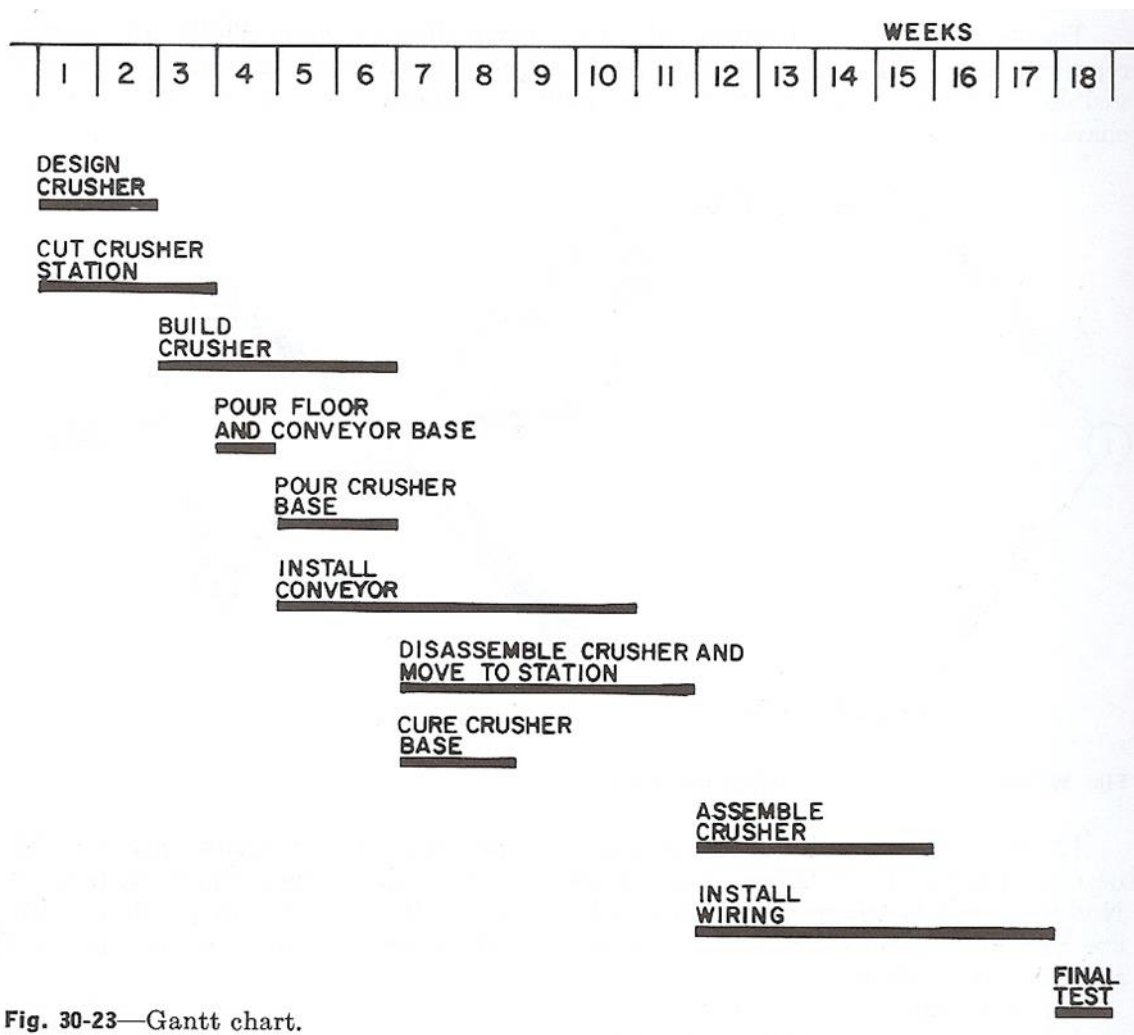


Fig. 30-23—Gantt chart.

Εικόνα 14.5. Το διάγραμμα Gantt του έργου της εγκατάστασης θραυστήρα και μεταφορικής ταινίας στο υπόγειο μεταλλείο [Πηγή: Mining Engineering Handbook, SME-AIME, 1st edition, 1973 (paragraph 30.6 *Project Control Techniques*)].

Για να σχεδιαστεί ένα διάγραμμα Gantt, πρέπει να καταγραφούν όλες οι δραστηριότητες του έργου και οι αντίστοιχες διάρκειές τους.

Στη συνέχεια, γίνεται η χάραξη της διάρκειας κάθε δραστηριότητας, υπό τη μορφή μιας ράβδου, το μήκος της οποίας είναι ανάλογο με τη χρονική διάρκεια που απαιτείται για την ολοκλήρωσή της υπόψη δραστηριότητας.

Κάθε ράβδος αρχίζει από το σημείο που στον οριζόντιο άξονα αντιστοιχεί στο χρονικό σημείο έναρξης της συγκεκριμένης δραστηριότητας και τελειώνει στο σημείο που αντιστοιχεί στο πέρας της.

Στη συνέχεια, θα αναπτύξουμε ένα πρόγραμμα στην Python προκειμένου να κατασκευάζει το διάγραμμα Gantt της Εικόνας 14.5, εμπλουτίζοντάς το με πρόσθετα χαρακτηριστικά.

Μέχρι τώρα, για την κατασκευή γραφημάτων χρησιμοποιήσαμε συναρτήσεις της βιβλιοθήκης Matplotlib που είναι η *de facto* βιβλιοθήκη γραφικών της Python. Ωστόσο, για την κατασκευή του διαγράμματος Gantt θα χρησιμοποιήσουμε μια άλλη βιβλιοθήκη γραφικών της Python, την Plotly, η οποία διαθέτει συναρτήσεις που είναι καταλληλότερες για τη δημιουργία ενός διαγράμματος Gantt.

Η βιβλιοθήκη Plotly είναι μια *διαδραστική* (interactive) βιβλιοθήκη γραφικών ανοικτού κώδικα, που υποστηρίζει περισσότερους από 40 διαφορετικούς τύπους γραφημάτων κατάλληλων για χρήση στην επιστήμη, τη στατιστική, τα οικονομικά κ.ά.

Για την τεκμηρίωση της βιβλιοθήκης ανατρέξετε στο website: <https://plotly.com/python/>

Ως εξωτερική βιβλιοθήκη πρέπει να εγκατασταθεί στον υπολογιστή μας, μέσω της γνωστής διαδικασίας:

```
pip install plotly==5.6.0
```

όπου 5.6.0 είναι η τρέχουσα (Μάρτιος 2022) έκδοση (version) της βιβλιοθήκης.

Από την Plotly, σε αυτό το Κεφάλαιο θα χρησιμοποιήσουμε το module Plotly Express (plotly.express) που είναι ένα λιτό υποσύνολο της βιβλιοθήκης, επαρκές όμως για να κάνουμε τη δουλειά μας και όχι μόνο, όπως θα διαπιστώσετε από την τεκμηρίωσή του στο website: <https://plotly.com/python/plotly-express/#overview>

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
import plotly.express as px
import pandas as pd
from datetime import datetime

df = pd.DataFrame([
    dict(Task="Design Crusher", Start='2021-06-21',
          Finish='2021-07-02', Resource='Crusher Ltd'),
    dict(Task="Cut Crusher Station", Start='2021-06-21',
          Finish='2021-07-09', Resource='Mining Dpt'),
    dict(Task="Build Crusher", Start='2021-07-05',
          Finish='2021-07-30', Resource='Crusher Ltd'),
    dict(Task="Pour Floor & Conveyor Base", Start='2021-07-12',
          Finish='2021-07-16', Resource='Mining Dpt'),
    dict(Task="Pour Crusher Base", Start='2021-07-19',
          Finish='2021-07-30', Resource='Mining Dpt'),
    dict(Task="Install Conveyor", Start='2021-07-19',
          Finish='2021-08-27', Resource='Mechanical/Electrical Dpt'),
    dict(Task="Disassemble Crusher & Move to Station", Start='2021-08-02',
          Finish='2021-09-03', Resource='Crusher Ltd'),
    dict(Task="Cure Crusher Base", Start='2021-07-31',
          Finish='2021-08-06', Resource='None'),
    dict(Task="Assemble Crusher", Start='2021-09-06',
          Finish='2021-10-01', Resource='Crusher Ltd'),
    dict(Task="Install Wiring", Start='2021-09-06',
          Finish='2021-10-15', Resource='Mechanical/Electrical Dpt'),
    dict(Task="Final Test", Start='2021-10-18',
          Finish='2021-10-22', Resource='Crusher Ltd')])

fig = px.timeline(df, title='Gantt Chart - Project: `
                  `Underground Crusher Station', x_start="Start",
                  x_end="Finish", y="Task", color="Resource")
fig.update_yaxes(autorange="reversed")
# Add a vertical line indicating a specific date (e.g. today)
now = datetime.now()
today = now.strftime('%Y-%m-%d')
fig.add_shape(type='line',
              x0=today,
              y0=0,
              x1=today,
              y1=1,
              line=dict(color='black', width=2),
              xref='x',
```



```
yref='paper')
fig.show()
```

Με την πρώτη γραμμή κώδικα κάνουμε `import` το module `plotly.express` και του δίνουμε το παρατσούκλι `px`.

Με τη δεύτερη γραμμή κώδικα κάνουμε `import` τη βιβλιοθήκη `pandas` και της δίνουμε το παρατσούκλι `pd`. Τη βιβλιοθήκη `pandas` την γνωρίσαμε στην παράγραφο 12.2 *Η βιβλιοθήκη Pandas*.

Με την τρίτη γραμμή κώδικα κάνουμε `import` το module `datetime` που μας επιτρέπει να χειριζόμαστε στην Python ημερομηνίες και χρόνους. Το module `datetime` το γνωρίσαμε στην παράγραφο 10.4 *Είσοδος του χρήστη στο πρόγραμμα*.

Στη συνέχεια, με τη βοήθεια της `pandas` δημιουργούμε ένα `DataFrame`, το `df`, στο οποίο εκχωρούμε τα δεδομένα του έργου μας υπό τη μορφή ενός dictionary. Το dictionary έχει ως πεδία-κλειδιά τα:

Task : Η περιγραφή της δραστηριότητας.

Start : Η ημερομηνία έναρξης της δραστηριότητας με τη μορφή YEAR-MM-DD (έτος-μήνας-ημέρα). Αρχή εβδομάδας είναι η ημέρα Δευτέρα.

Finish : Η ημερομηνία πέρατος της δραστηριότητας με τη μορφή YEAR-MM-DD (έτος-μήνας-ημέρα). Τέλος εβδομάδας είναι η ημέρα Παρασκευή.

Resource : Στην περίπτωση μας, ο φορέας που είναι υπεύθυνος για την εκτέλεση της δραστηριότητας του έργου μας. Θα μπορούσε να είναι το όνομα του υπεύθυνου για την παρακολούθηση ή την εκτέλεση της δραστηριότητας ή ακόμα έναν ποιοτικό χαρακτηρισμό που χαρακτηρίζει τη δραστηριότητα π.χ. 'Complete', 'Incomplete', 'Not Started' (Περατώθηκε, Σε εξέλιξη, Δεν ξεκίνησε).

Η γραμμή κώδικα:

```
fig = px.timeline(df, title='Gantt Chart - Project: `
                    `Underground Crusher Station', x_start="Start",
                    x_end="Finish", y="Task", color="Resource")
```

διαμορφώνει το διάγραμμα Gantt, εφαρμόζοντας τη συνάρτηση `timeline()` του `plotly.express` επί των δεδομένων του `DataFrame df`.

Οι παράμετροι της συνάρτησης `timeline()` είναι:

title : Ο τίτλος του διαγράμματος Gantt.

x_start : Η ημερομηνία έναρξης της δραστηριότητας που θα τη διαβάσει από το πεδίο `Start` του `DataFrame df` (οριζόντιος άξονας του διαγράμματος, `x`).

x_end : Η ημερομηνία πέρατος της δραστηριότητας που θα τη διαβάσει από το πεδίο `Finish` του `DataFrame df` (οριζόντιος άξονας του διαγράμματος, `x`).

y : Η περιγραφή της δραστηριότητας που θα τη διαβάσει από το πεδίο `Task` του `DataFrame df` (κάθετος άξονας του διαγράμματος, `y`).

color : Ο χρωματικός κώδικας του ραβδογράμματος, ο οποίος διαμορφώνεται αυτόματα από το `plotly.express`, ανάλογα με την τιμή του πεδίου `Resource` του `DataFrame df`.

Η γραμμή κώδικα:

```
fig.update_yaxes(autorange="reversed")
```

μας διαμορφώνει το διάγραμμα Gantt κατά τρόπο τέτοιο ώστε οι δραστηριότητες να παρουσιάζονται τοποθετώντας την πρώτη χρονικά δραστηριότητα στο άνω μέρος του διαγράμματος. Εάν παραλειφθεί αυτή η εντολή, τότε η πρώτη χρονικά δραστηριότητα θα εμφανιστεί στο κάτω μέρος του διαγράμματος, κάτι ασυνήθιστο για τα διαγράμματα Gantt.

Οι υπόλοιπες γραμμές κώδικα λαμβάνουν από το ρολόι του υπολογιστή μας την τρέχουσα ημερομηνία και από την τιμή της ημερομηνίας αυτής, στον άξονα x , χαράσσουν μια κάθετη χοντρή μαύρη γραμμή, προκειμένου να έχουμε μια αίσθηση της εξέλιξης των δραστηριοτήτων του έργου σε σχέση την τρέχουσα ημερομηνία, το σήμερα.

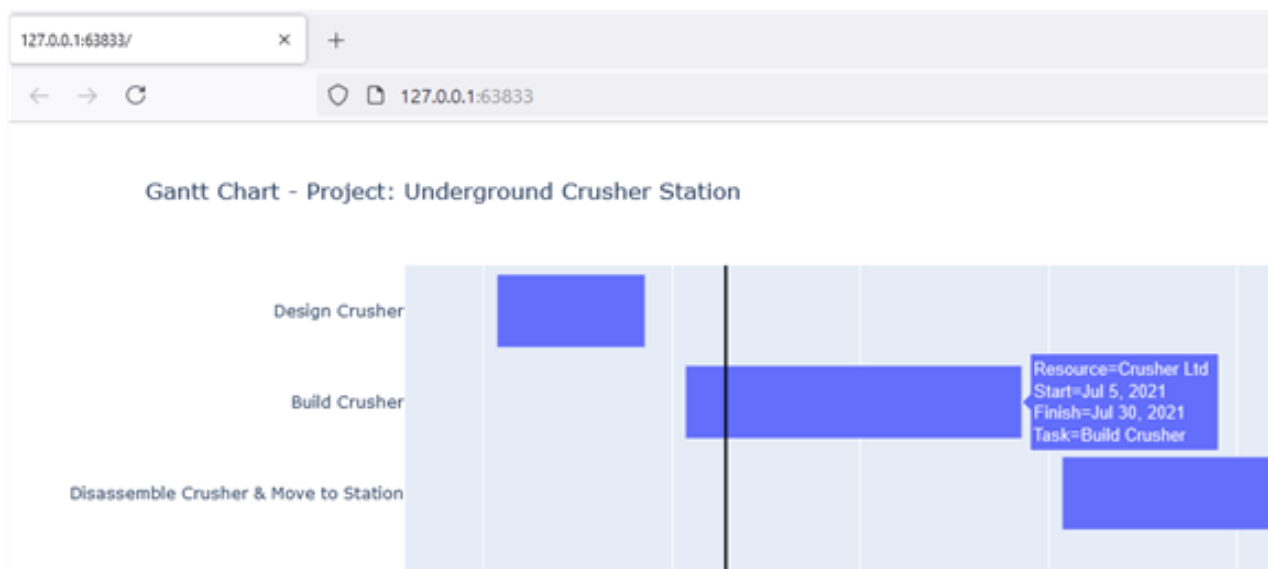
Η τελευταία γραμμή του κώδικα εμφανίζει το διάγραμμα Gantt του έργου μας στην οθόνη (Εικόνα 14.6).

Όπως εύκολα διαπιστώνετε, το διάγραμμα Gantt εμφανίζεται σχεδιασμένο στον default web browser που διαθέτει ο υπολογιστής σας (ο συγγραφέας χρησιμοποιεί τον Firefox της Mozilla).



Εικόνα 14.6. Το διάγραμμα Gantt του έργου μας που εμφανίζει το `module plotly.express` της βιβλιοθήκης Plotly στον web browser (1^η εκδοχή).

Αυτό καθιστά τις εικόνες που παράγονται από τη βιβλιοθήκη Plotly *διαδραστικές* (interactive) και μας δίνει, μεταξύ άλλων, τη δυνατότητα να βλέπουμε τα δεδομένα μιας δραστηριότητας τοποθετώντας τον δείκτη του ποντικιού πάνω στη ράβδο της δραστηριότητας (Εικόνα 14.7).

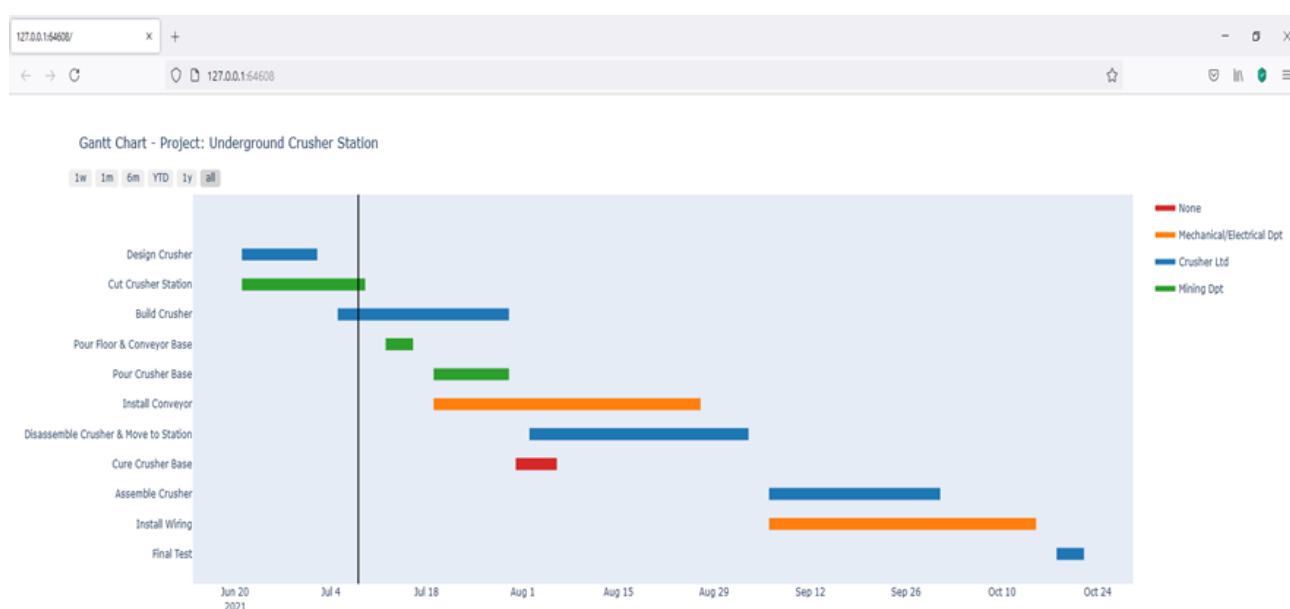


Εικόνα 14.7. Εμφάνιση των δεδομένων της δραστηριότητας Build Crusher του διαγράμματος Gantt.

Στο άνω δεξιό άκρο της οθόνης του του διαγράμματος Gantt, υπάρχουν διάφορα εικονίδια εργαλείων, μεταξύ των οποίων και ένα που μας δίνει τη δυνατότητα να κατεβάσουμε (download) μια στατική εικόνα του διαγράμματος σε αρχείο τύπου .png και στη συνέχεια να την ενσωματώσουμε σε κείμενο.

Ταυτόχρονα ανοίγει στον web browser και ένα δεύτερο παράθυρο το οποίο παρουσιάζει το ίδιο διάγραμμα Gantt, σε μια πιο εκλεπτυσμένη μορφή, και ίσως πιο όμορφη, όπως φαίνεται στην Εικόνα 14.8.

Το script του κώδικα που κατασκευάζει το διάγραμμα Gantt βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_14_GANTT.py`



Εικόνα 14.8. Το διάγραμμα Gantt του έργου μας που εμφανίζει το module `plotly.express` της βιβλιοθήκης Plotly στον web browser (2^η εκδοχή).

15 Χημικοί τύποι ενώσεων

15.1 Γενικά

Στην παράγραφο 08.4.1 Άντληση δεδομένων από λεξικά, υπολογίσαμε το μοριακό βάρος του μπετονίτη γράφοντας τον κώδικα ενός προγράμματος που είχε ως σκοπό να αναδείξει τον τρόπο άντλησης δεδομένων από λεξικά, υποσχεθήκαμε δε ότι θα παρουσιάσουμε σε επόμενο Κεφάλαιο έναν αυτοματοποιημένο τρόπο υπολογισμού του μοριακού βάρους μιας χημικής ένωσης.

Στο Κεφάλαιο αυτό θα παρουσιάσουμε δύο μεθόδους με τις οποίες μπορούμε να χειριστούμε χημικούς τύπους, δηλαδή να διαβάζουμε τα χημικά στοιχεία που περιέχουν και τον αριθμό των ατόμων του καθενός στοιχείου, και στη συνέχεια να κάνουμε τον υπολογισμό του μοριακού βάρους.

15.2 Γραμματική ανάλυση

Με τον όρο *γραμματική ανάλυση* (parsing) εννοούμε να χωρίσουμε μία λέξη ή και μία πρόταση σε τμήματα σύμφωνα με κάποιους κανόνες. Από τη σκοπιά της γλώσσας Python, η λέξη ή η πρόταση δεν είναι τίποτε άλλο από μια συμβολοσειρά (string).

Έτσι, για παράδειγμα, εάν εφαρμόσουμε τεχνικές parsing πάνω στη συμβολοσειρά του χημικού τύπου του νερού H₂O, ο κώδικας Python θα διαπίστωνε ότι το νερό αποτελείται από 2 άτομα H και 1 άτομο O.

15.2.1 Το module: PyParsing

Το module PyParsing είναι μια καθιερωμένη βιβλιοθήκη συναρτήσεων της Python για τη γραμματική ανάλυση συμβολοσειρών που στηρίζεται στην τεχνική *Parsing Expression Grammars - PEGs*.

Ως εξωτερική βιβλιοθήκη πρέπει να εγκατασταθεί στον υπολογιστή μας, μέσω της γνωστής διαδικασίας:

```
pip install pyparsing
```

Για την τεκμηρίωση της βιβλιοθήκης ανατρέξτε στο website:

<https://pypi.org/project/pyparsing>

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
# Enter formula
formula = input('Enter the Chemical Formula :')
# 1st Method: pyparsing module
from pyparsing import *
caps = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
lowers = caps.lower()
digits = "0123456789."
element = Word(caps, lowers)
atom_number = Word(digits)
elementRef = Group(element + Optional(atom_number, default="1"))
chemicalFormula = OneOrMore(elementRef)
formulaData = chemicalFormula.parseString(formula)
print(formula, "->", formulaData)
```

Με την πρώτη εντολή του κώδικα κάνουμε input, μέσω του πληκτρολογίου, τον χημικό τύπο της ένωσης που μας ενδιαφέρει να κάνουμε γραμματική ανάλυση και τον εκχωρούμε στη μεταβλητή formula.

Ο εισαγόμενος χημικός τύπος πρέπει να έχει τη μορφή:

C3.7H2.5O4C15Fe5.8Na

δηλαδή να περιλαμβάνει το σύμβολο του χημικού στοιχείου ακολουθούμενο από τον αριθμό των ατόμων του στοιχείου που συμμετέχουν στη σύσταση της χημικής ένωσης. Ο αριθμός των ατόμων μπορεί να περιλαμβάνει και δεκαδικό μέρος, ενώ στην περίπτωση που παραλείπεται λαμβάνεται ίσος με τη μονάδα. Δεν επιτρέπονται παρενθέσεις ούτε κενά μεταξύ των μελών του χημικού τύπου.

Στη συνέχεια, αφού πρώτα κάνουμε `import` το module `pyarsing`, ορίζουμε τις μεταβλητές:

`caps` : Τα κεφαλαία γράμματα του λατινικού αλφάβητου.

`lowsers` : Τα αντίστοιχα των κεφαλαίων, πεζά γράμματα του λατινικού αλφάβητου.

`digits` : Οι αριθμοί 0 έως και 9 καθώς και το διαχωριστικό του δεκαδικού μέρους δηλαδή η τελεία.

`element` : Αντιστοιχεί στον χημικό τύπο ενός στοιχείου και ορίζεται μέσω της κλάσης `Word` του module `pyarsing`. Είναι ένα `string` που έχει ως χαρακτήρες ένα από τα κεφαλαία γράμματα της μεταβλητής `caps` ακολουθούμενο από ένα από τα πεζά γράμματα της μεταβλητής `lowsers`.

`atom_number` : Αντιστοιχεί στον αριθμό των ατόμων ενός στοιχείου και ορίζεται μέσω της κλάσης `Word` του module `pyarsing`. Είναι ένα `string` που έχει ως χαρακτήρες έναν ή περισσότερους από τους αριθμούς της μεταβλητής `digits`, συμπεριλαμβανομένης και της τελείας στην περίπτωση δεκαδικών αριθμών.

`elementRef` : Αντιστοιχεί στα άτομα ενός χημικού στοιχείου, ήτοι τον χημικό τύπο και τον αριθμό των ατόμων του στοιχείου και ορίζεται μέσω της κλάσης `Group` του module `pyarsing`. Είναι ένα `string` που έχει ως χαρακτήρες την τιμή της μεταβλητής `element` και την τιμή της μεταβλητής `atom_number`. Εάν δεν έχει τιμή η μεταβλητή `atom_number`, τότε λαμβάνει την `default` τιμή 1.

`chemicalFormula` : Αντιστοιχεί στη δομή του εισαγόμενου χημικού τύπου και ορίζεται μέσω της κλάσης `OneOrMore` του module `pyarsing`. Είναι ένα `string` μίας ή περισσότερων επαναλήψεων χαρακτήρων της μορφής `elementRef`.

Με τη γραμμή κώδικα:

```
formulaData = chemicalFormula.parseString(formula)
```

γίνεται η γραμματική ανάλυση του χημικού τύπου της ένωσης, που έχουμε πληκτρολογήσει και εκχωρήσει στη μεταβλητή `formula`, σύμφωνα με τα χαρακτηριστικά δομής που έχουν εκχωρηθεί στη μεταβλητή `chemicalFormula` και τα αποτελέσματα εκχωρούνται στη μεταβλητή `formulaData`.

Εάν τρέξουμε τον παραπάνω κώδικα πληκτρολογώντας τον χημικό τύπο `C3.7H2.5O4C15Fe5.8Na`

Enter the Chemical Formula : C3.7H2.5O4C15Fe5.8Na

στην οθόνη θα εμφανιστούν τα:

```
C3.7H2.5O4C15Fe5.8Na -> [['C', '3.7'], ['H', '2.5'], ['O', '4'], ['C1', '5'],  
['Fe', '5.8'], ['Na', '1']]
```

Λαμβάνουμε, δηλαδή, ως αποτέλεσμα της γραμματικής ανάλυσης του χημικού τύπου, μια λίστα, η οποία αποτελείται από τόσες λίστες όσα είναι τα χημικά στοιχεία της ένωσης. Καθεμία από τις λίστες αυτές έχει δύο μέλη: τον χημικό τύπο του στοιχείου και τον αριθμό των ατόμων του στη χημική ένωση.

Αφού γνωρίζουμε τα στοιχεία που συμμετέχουν στη χημική ένωση και τον αριθμό των ατόμων καθενός από αυτά, μπορούμε να υπολογίσουμε το μοριακό βάρος της χημικής ένωσης (MW), προσθέτοντας στον κώδικά μας το λεξικό των ατομικών βαρών `elements_AW_Dix` που παρουσιάσαμε στην παράγραφο 08.4 *Λεξικά* και τις παρακάτω γραμμές κώδικα:

```
# Molecular weight calculation  
mw = sum([elements_AW_Dix[sym] * float(qty) for sym, qty in formulaData])  
print('MW of {:s} [g] = {:.4f}'.format(formula, mw))
```

Στην οθόνη θα εμφανιστεί το μοριακό βάρος της χημικής ένωσης:

MW of C3.7H2.5O4C15Fe5.8Na [g] = 635.1255

15.2.2 To module: Chemparse

Σχετικά πρόσφατα (2020) δημοσιεύτηκε το module Chemparse, το οποίο είναι εξειδικευμένο για να κάνει parsing χημικών τύπων.

Ως εξωτερική βιβλιοθήκη πρέπει να εγκατασταθεί στον υπολογιστή μας, μέσω της γνωστής διαδικασίας:

```
pip install chemparse
```

Για την τεκμηρίωση του module ανατρέξτε στο website: <https://pypi.org/project/chemparse>

Δύο χαρακτηριστικά που διακρίνουν το module Chemparse, σε σχέση με το module PyParsing (όπως το χρησιμοποιήσαμε στην παράγραφο 15.2.1 To module: PyParsing), είναι ότι πρώτον μπορεί να χειριστεί χημικούς τύπους που περιλαμβάνουν παρενθέσεις π.χ. (CH3)2(CH2)4 και δεύτερον τα αποτελέσματα της γραμματικής ανάλυσης του χημικού τύπου δίνονται υπό μορφή λεξικού και όχι λίστας.

Στον κώδικα της παραγράφου 15.2.1 To module: PyParsing προσθέτουμε τις παρακάτω γραμμές:

```
# 2nd Method: chemparse module
import chemparse
formulaData = chemparse.parse_formula(formula)
print(formula, "->", formulaData)
```

Με τη γραμμή κώδικα:

```
formulaData = chemparse.parse_formula(formula)
```

γίνεται η γραμματική ανάλυση του χημικού τύπου της ένωσης, που έχουμε πληκτρολογήσει και εκχωρήσει στη μεταβλητή formula και τα αποτελέσματα εκχωρούνται στη μεταβλητή formulaData.

Εάν τρέξουμε τον κώδικα και πληκτρολογήσουμε τον χημικό τύπο C3.7H2.5O4C15Fe5.8Na, στην περίπτωση της εφαρμογής του module Chemparse, το αποτέλεσμα της γραμματικής ανάλυσης που θα εμφανιστεί στην οθόνη είναι:

```
C3.7H2.5O4C15Fe5.8Na -> {'C': 3.7, 'H': 2.5, 'O': 4.0, 'Cl': 5.0, 'Fe': 5.8, 'Na': 1.0}
```

Προσθέτοντας και τις γραμμές κώδικα:

```
# Molecular weight calculation
mw = 0
for element, atom_number in formulaData.items():
    mw += atom_number * elements_AW_Dix[element]
print('MW of {:s} [g] = {:.4f}'.format(formula, mw))
```

υπολογίζουμε και εμφανίζουμε στην οθόνη το μοριακό βάρος της χημικής ένωσης.

MW of C3.7H2.5O4C15Fe5.8Na [g] = 635.1255

15.3 Ο κώδικας στο τέλος του Κεφαλαίου 15

Ο πλήρης κώδικας με τον οποίο κάνουμε parsing των χημικών τύπων ενώσεων με τα modules PyParsing και Chemparse και στη συνέχεια υπολογίζουμε το μοριακό βάρος της χημικής ένωσης είναι:

```
# Chemical formula parsing & molecular weight calculation
#
# Atomic Weights of Chemical Elements
# ['symbol': relative atomic mass (in g/atom)]
# (Source: NIST-JANAF Thermochemical Tables, 4th Edition, Jan1998)
```

```

elements_AW_Dix = {
    'Ac': 227.0278, 'Ag': 107.8682, 'Al': 26.98154, 'Ar': 39.948,
    'As': 74.9216, 'Au': 196.9665, 'B': 10.81, 'Ba': 137.33,
    'Be': 9.01218, 'Bi': 208.9804, 'Br': 79.904, 'C': 12.011,
    'Ca': 40.08, 'Cd': 112.418, 'Ce': 140.12, 'Cl': 35.453,
    'Co': 58.9312, 'Cr': 51.996, 'Cs': 132.9054, 'Cu': 63.546,
    'D': 2.014102, 'Dy': 162.50, 'Er': 167.26, 'Eu': 151.96,
    'F': 18.998403, 'Fe': 55.847, 'Ga': 69.72, 'Gd': 157.25,
    'Ge': 72.59, 'H': 1.00794, 'He': 4.00260, 'Hf': 178.49,
    'Hg': 200.59, 'Ho': 164.9304, 'I': 126.9045, 'In': 114.82,
    'Ir': 192.22, 'K': 39.0983, 'Kr': 83.80, 'La': 138.9055,
    'Li': 6.941, 'Lu': 174.967, 'Mg': 24.305, 'Mn': 54.9380,
    'Mo': 95.94, 'N': 14.0067, 'Na': 22.98977, 'Nb': 92.9064,
    'Nd': 144.24, 'Ne': 20.179, 'Ni': 58.69, 'Np': 237.0482,
    'O': 15.9994, 'Os': 190.2, 'P': 30.97376, 'Pa': 231.0359,
    'Pb': 207.2, 'Pr': 140.9077, 'Pt': 195.0, 'Ra': 226.0254,
    'Rb': 85.4678, 'Re': 186.207, 'Rh': 102.9055, 'Ru': 101.07,
    'S': 32.06, 'Sb': 121.75, 'Sc': 44.9559, 'Se': 78.96,
    'Si': 28.0855, 'Sm': 150.36, 'Sn': 118.69, 'Sr': 87.62,
    'Ta': 180.9479, 'Tb': 158.9254, 'Te': 127.60, 'Th': 232.0381,
    'Ti': 47.88, 'Tl': 204.383, 'Tm': 168.9342, 'U': 238.0289,
    'V': 50.9415, 'W': 183.85, 'Xe': 131.29, 'Y': 88.9059,
    'Yb': 173.04, 'Zn': 65.38, 'Zr': 91.22}

# Enter formula
formula = input('Enter the Chemical Formula :')
# 1st Method: pyparsing module
from pyparsing import *
caps = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
lower = caps.lower()
digits = "0123456789."
element = Word(caps, lower)
atom_number = Word(digits)
elementRef = Group(element + Optional(atom_number, default="1"))
chemicalFormula = OneOrMore(elementRef)
formulaData = chemicalFormula.parseString(formula)
print(formula, "->", formulaData)
# Molecular weight calculation
mw = sum([elements_AW_Dix[sym] * float(qty) for sym, qty in formulaData])
print('MW of {:s} [g] = {:.4f}'.format(formula, mw))
# 2nd Method: chemparse module
import chemparse
formulaData = chemparse.parse_formula(formula)
print(formula, "->", formulaData)
# Molecular weight calculation
mw = 0
for element, atom_number in formulaData.items():
    mw += atom_number * elements_AW_Dix[element]
print('MW of {:s} [g] = {:.4f}'.format(formula, mw))

```

To script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο:
Chapter_15_Chemical_Formula_Parsing_Molecular_Weight.py

16 Αντικειμενοστραφής προγραμματισμός

16.1 Γενικά

Στο Κεφάλαιο αυτό δεν πρόκειται να παρουσιαστούν οι τεχνικές του αντικειμενοστραφούς προγραμματισμού.

Το Κεφάλαιο θα πρέπει να θεωρηθεί ως ένα βοηθητικό κείμενο, προκειμένου ο αναγνώστης να μυηθεί στις έννοιες και στο συντακτικό που χρησιμοποιούνται όταν εφαρμόζουμε τεχνικές αντικειμενοστραφούς προγραμματισμού σε κώδικα Python.

Αυτό γίνεται προκειμένου να μπορέσει ο αναγνώστης να παρακολουθήσει με ευχέρεια την παρουσίαση των Κεφαλαίων 17 *Γραφική διεπαφή χρήστη* και 18 *Προσομοίωση συστημάτων*, τα οποία ακολουθούν.

Στα προγράμματα που γράψαμε μέχρι τώρα εφαρμόσαμε την αρχή: *τα δεδομένα να είναι ξεχωριστά από τους αλγόριθμους*, σύμφωνα με τα προβλεπόμενα από τον *Διαδικαστικό Προγραμματισμό* (Procedural Programming), ο οποίος είναι υποκατηγορία του *Δομημένου Προγραμματισμού* (Structured Programming).

Όμως, η ανάγκη ανάπτυξης κώδικα προγραμμάτων για την επίλυση σύνθετων, πραγματικών (real-world) προβλημάτων, οδήγησε στην επινόηση του *Αντικειμενοστραφούς Προγραμματισμού* (Object-Oriented Programming, OOP), ο οποίος βασίζεται στην έννοια των *αντικειμένων* (objects), τα οποία μπορεί να περιλαμβάνουν ταυτόχρονα *δεδομένα* (data) και *αλγόριθμους* (procedures).

Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Object-oriented_programming

Η Python είναι ένα χαρακτηριστικό παράδειγμα γλώσσας προγραμματισμού που είναι κατάλληλη για την ανάπτυξη προγραμμάτων OOP. Άλλωστε, πολλά από τα δομικά στοιχεία της γλώσσας είναι αντικείμενα και υπό αυτή τη μορφή τους τα έχουμε χρησιμοποιήσει μέχρι τώρα. Π.χ. αντικείμενα είναι οι λίστες, τα λεξικά και βέβαια οι συναρτήσεις των βιβλιοθηκών που κάναμε import στα προγράμματά μας.

Επειδή ο προσανατολισμός του βιβλίου είναι στη χρήση αντικειμένων της Python και όχι στη γραφή του κώδικα μορφοποίησης αυτών των αντικειμένων, όποιος ενδιαφέρεται να εξερευνήσει τις δυνατότητες της Python στην ανάπτυξη προγραμμάτων OOP θα πρέπει να ανατρέξει στη σχετική βιβλιογραφία που περιλαμβάνεται στο website: <https://wiki.python.org/moin/PythonBooks>

16.2 Κλάσεις και αντικείμενα

Η βασική δομή δεδομένων, που δημιουργεί ο χρήστης της Python όταν αναπτύσσει κώδικα OOP, ονομάζεται *κλάση* (class).

Η κλάση λειτουργεί σαν *πρότυπο* (καλούπι) για τη δημιουργία αντικειμένων.

Τις κλάσεις τις έχουμε συναντήσει ήδη στα πρώτα Κεφάλαια του βιβλίου, π.χ. στην παράγραφο 04.3.1 *Περί μεταβλητών* είδαμε ότι οι μεταβλητές λαμβάνουν τιμές που ανήκουν στις κλάσεις: `int`, `float`, `str`.

Γράφουμε τον παρακάτω κώδικα:

```
class Cyclos:
    """Calculations based on circle radius"""
    pi = 3.14

    def __init__(self, radius=0):
        self.radius = radius

    def periphery(self):
        return 2 * Cyclos.pi * self.radius
```

Μόλις δημιουργήσαμε μια κλάση με το όνομα `Cyclos`, η οποία κάνει γεωμετρικούς υπολογισμούς σε σχήματα που δημιουργούνται με παράμετρο την ακτίνα του κύκλου.

Οι κλάσεις ονοματίζονται ακολουθώντας τους ίδιους κανόνες ονοματοδοσίας με εκείνους των μεταβλητών και συναρτήσεων, με τη διαφορά ότι, συμβατικά, το πρώτο γράμμα της κλάσης είναι κεφαλαίο.

Οι εντολές που αποτελούν το σώμα μιας κλάσης γράφονται εντός εσοχής, ενώ το τέλος της εσοχής σηματοδοτεί και το πέρας των εντολών της κλάσης.

Την επικεφαλίδα της κλάσης, συνήθως, ακολουθεί κείμενο, εντός τριπλών εισαγωγικών `""" ... """`, το οποίο τεκμηριώνει την κλάση.

Εάν γράψουμε, εκτός της κλάσης, την εντολή:

```
print('Class Cyclos documentation:', Cyclos.__doc__)
```

στην οθόνη θα εμφανιστεί το κείμενο:

```
Class Cyclos documentation: Calculations based on circle radius
```

Στη συνέχεια, δημιουργούμε τη μεταβλητή `pi` στην οποία δίνουμε την τιμή 3.14 (είναι δηλαδή το π). Η μεταβλητή αυτή θα είναι διαθέσιμη στα αντικείμενα της κλάσης και ονομάζεται *μεταβλητή κλάσης*.

Εάν γράψουμε, εκτός της κλάσης, την εντολή:

```
print('pi =', Cyclos.pi)
```

στην οθόνη θα εμφανιστεί:

```
pi = 3.14
```

Ο υπόλοιπος κώδικας της κλάσης αποτελείται από λειτουργίες και αλγόριθμους πάνω στα αντικείμενα, οι οποίες ορίζονται υπό τη μορφή συναρτήσεων και ονομάζονται *μέθοδοι* (methods).

Μεθόδους κλάσεων έχουμε ήδη χρησιμοποιήσει αρκετές φορές μέχρι τώρα στο βιβλίο, π.χ. στην παράγραφο 08.3.1 *Μέθοδοι στις λίστες*.

Η πρώτη μέθοδος είναι η ειδική μέθοδος *αρχικοποίησης τιμών* (constructor) `__init__` η οποία καλείται κάθε φορά που δημιουργείται στη μνήμη ένα *στιγμιότυπο* (instance) της κλάσης, δηλαδή ένα αντικείμενο.

Η πρώτη παράμετρος της μεθόδου είναι το `self`, η οποία είναι μια μεταβλητή που αναφέρεται στο *ίδιο το αντικείμενο* (τον εαυτό του) και με τη βοήθειά της διακρίνονται μεταξύ τους τα αντικείμενα που δημιουργεί μια κλάση.

Η `self` πρέπει να είναι η πρώτη παράμετρος και στις υπόλοιπες μεθόδους της κλάσης, πολλές φορές είναι και η μοναδική. Σημειώστε ότι το όνομα `self` δεν είναι δεσμευτικό στην Python, αλλά το χρησιμοποιούμε κατά σύμβαση.

Η δεύτερη παράμετρος της μεθόδου `__init__`, στην περίπτωση της κλάσης `Cyclos`, είναι η μεταβλητή `radius` στην οποία έχουμε δώσει αρχική τιμή 0.

Στο σώμα της μεθόδου ορίζουμε ότι:

```
self.radius = radius
```

που σημαίνει ότι κάθε νέο αντικείμενο που δημιουργείται από την κλάση `Cyclos` θα έχει το δικό του αντίγραφο της μεταβλητής `radius`.

Εάν γράψουμε, εκτός της κλάσης, τις εντολές:

```
cyclos_1 = Cyclos(1)
print('Radius of cyclos_1 =', cyclos_1.radius)
```


και τρέξουμε τον κώδικα, στην οθόνη θα εμφανιστεί:

```
Radius of cyclos_1 = 1
```

Και εάν γράψουμε, εκτός της κλάσης, τις εντολές:

```
cyclos_2 = Cyclos(2)
print('Radius of cyclos_2 =', cyclos_2.radius)
```

και τρέξουμε τον κώδικα, στην οθόνη θα εμφανιστεί:

```
Radius of cyclos_2 = 2
```

Η επόμενη μέθοδος που περιλαμβάνεται στον κώδικά μας είναι η `periphery`, η οποία υπολογίζει το μήκος της περιφέρειας ενός κύκλου όταν δώσουμε ως input το μήκος της ακτίνας του κύκλου.

Εάν καλέσουμε, εκτός της κλάσης, τη μέθοδο `periphery` για το αντικείμενο `cyclos_2`:

```
print('Circle Periphery = ', cyclos_2.periphery())
```

και τρέξουμε τον κώδικα, στην οθόνη θα εμφανιστεί:

```
Circle Periphery = 12.56
```

Η κλάση `Cyclos` θα μπορούσε να εμπλουτιστεί και με άλλες μεθόδους, όπως τις:

```
def area(self):
    return Cyclos.pi * (self.radius ** 2)

def sphere_volume(self):
    return (4/3) * Cyclos.pi * (self.radius ** 3)

def sphere_area(self):
    return 4 * Cyclos.pi * (self.radius ** 2)
```

οι οποίες υπολογίζουν, κατά σειρά, την επιφάνεια του κύκλου, τον όγκο της σφαίρας που δημιουργείται εκ της περιστροφής του κύκλου και την επιφάνεια της σφαίρας.

Εάν γράψουμε, εκτός της κλάσης, τις εντολές:

```
print('Circle Area = ', cyclos_2.area())
print('Sphere Volume = ', cyclos_2.sphere_volume())
print('Sphere Area = ', cyclos_2.sphere_area())
```

και τρέξουμε τον κώδικα, στην οθόνη θα εμφανιστούν τα:

```
Circle Area = 12.56
Sphere Volume = 33.49333333333333
Sphere Area = 50.24
```

16.3 Κληρονομικότητα

Η *κληρονομικότητα* (inheritance) είναι το πιο σημαντικό χαρακτηριστικό του αντικειμενοστραφούς προγραμματισμού, καθώς επιτρέπει σε μια *κλάση-παιδί* (child class) να αποκτήσει-κληρονομήσει τις ιδιότητες και τις μεθόδους μιας άλλης *κλάσης-γονιός* (parent class).

Η κληρονομικότητα διευκολύνει την ανάπτυξη μεγάλων και περίπλοκων προγραμμάτων, γιατί επιτρέπει στην ουσία την επαναχρησιμοποίηση κώδικα με ή χωρίς αλλαγές.

Στον κώδικα που αναπτύξαμε στην παράγραφο 16.2 *Κλάσεις* προσθέτουμε τις παρακάτω γραμμές:

```
class Cylinder(Cyclos):
    def __init__(self, radius=0, height=0):
        self.radius = radius
        self.height = height

    def cylinder_volume(self):
        return Cyclos.area(self) * self.height
```

Η κλάση `Cylinder`, που μόλις δημιουργήσαμε, κληρονομεί την κλάση `Cyclos` και βέβαια τις μεθόδους της, και για τον λόγο αυτό η κληρονομούμενη κλάση γράφεται εντός των παρενθέσεων.

Πέραν της μεθόδου `__init__` η κλάση `Cylinder` διαθέτει και τη μέθοδο `cylinder_volume`, η οποία υπολογίζει τον όγκο ενός κυλίνδρου όταν γνωρίζουμε την ακτίνα της βάσης και το ύψος του, από τη σχέση: Όγκος Κυλίνδρου = Επιφάνεια Βάσης * Ύψος.

Την επιφάνεια της βάσης την υπολογίζουμε μέσω της κληρονομούμενης μεθόδου `area` της κλάσης `Cyclos`, κάνοντας την κλήση `Cyclos.area(self)`.

Έτσι, εάν γράψουμε, εκτός της κλάσης, την εντολή:

```
print('Cylinder Volume = ', Cylinder(2,4).cylinder_volume())
```

η μέθοδος `cylinder_volume` της κλάσης `Cylinder`, η οποία έχει κληρονομήσει τη μέθοδο `area` της κλάσης `Cyclos`, θα μας υπολογίσει τον όγκο ενός κυλίνδρου με μήκος ακτίνας βάσης 2 και ύψος 4 μονάδες μήκους.

Εάν τρέξουμε τον κώδικα, στην οθόνη θα εμφανιστεί το αποτέλεσμα:

Cylinder Volume = 50.24

Θα μπορούσαν να γραφτούν πολύ περισσότερα για τον αντικειμενοστραφή προγραμματισμό με τη χρήση της `Python`, αλλά αυτό είναι πέραν του σκοπού του βιβλίου. Για όσους ενδιαφέρονται, η βιβλιογραφία επί του αντικειμένου είναι πραγματικά μεγάλη.

16.4 Ο κώδικας στο τέλος του Κεφαλαίου 16

Ο πλήρης κώδικας με τον οποίο παρουσιάσαμε τα βασικά στοιχεία του τρόπου ανάπτυξης εφαρμογών αντικειμενοστραφούς προγραμματισμού είναι:

```
# Demonstration of Object-Oriented Programming
```

```
class Cyclos:
    """Calculations based on circle radius"""
    pi = 3.14

    def __init__(self, radius=0):
        self.radius = radius

    def periphery(self):
        return 2 * Cyclos.pi * self.radius

    def area(self):
        return Cyclos.pi * (self.radius ** 2)

    def sphere_volume(self):
        return (4 / 3) * Cyclos.pi * (self.radius ** 3)

    def sphere_area(self):
        return 4 * Cyclos.pi * (self.radius ** 2)

print('Class Cyclos documentation:', Cyclos.__doc__)
```

```

print('pi =', Cyclos.pi)
cyclos_1 = Cyclos(1)
print('Radius of cyclos_1 =', cyclos_1.radius)
cyclos_2 = Cyclos(2)
print('Radius of cyclos_2 =', cyclos_2.radius)
print('Circle Periphery = ', cyclos_2.periphery())
print('Circle Area = ', cyclos_2.area())
print('Sphere Volume = ', cyclos_2.sphere_volume())
print('Sphere Area = ', cyclos_2.sphere_area())

class Cylinder(Cyclos):
    def __init__(self, radius=0, height=0):
        self.radius = radius
        self.height = height

    def cylinder_volume(self):
        return Cyclos.area(self) * self.height

print('Cylinder Volume = ', Cylinder(2,4).cylinder_volume())

```

Το script του κώδικα βρίσκεται στο website του βιβλίου στο αρχείο:
Chapter_16_Object_Oriented_Programming.py

17 Γραφική διεπαφή χρήστη

17.1 Γενικά

Το γραφικό περιβάλλον χρήστη ή πιο γνωστή ως η *γραφική διεπαφή χρήστη* (Graphical User Interface, GUI) επιτρέπει τη χρήση εικονιδίων και άλλων οπτικών στοιχείων της οθόνης του υπολογιστή, προκειμένου να εκτελεστούν εντολές, να εισαχθούν δεδομένα ή να γίνουν επιλογές, κατά την εκτέλεση ενός προγράμματος.

Στα μέχρι τώρα προγράμματα Python που έχουμε αναπτύξει, η αλληλοεπίδραση του χρήστη με το πρόγραμμα γίνεται μέσω αριθμών ή κειμένου (συμβολοσειρών) που εισάγουμε με τη βοήθεια του πληκτρολογίου.

Αυτό έρχεται σε αντίθεση με τη σύγχρονη τάση ανάπτυξης εφαρμογών, όπου ο χρήστης επικοινωνεί με το πρόγραμμα μέσω μιας *γραφικής διεπιφάνειας* (GUI), ενώ επίσης δεν συμβαδίζει ούτε και με το γραφικό περιβάλλον του λειτουργικού συστήματος των Windows στο οποίο τρέχει ο υπολογιστής μας.

Για την ανάπτυξη εφαρμογών GUI, η Python διαθέτει μεγάλο αριθμό πακέτων και toolkits, από τα οποία μπορεί να επιλέξει ο προγραμματιστής εκείνο που τον ενδιαφέρει. Σχετικά βλέπε στο website: <https://wiki.python.org/moin/GuiProgramming>

Σε κάθε περίπτωση η ανάπτυξη εφαρμογών GUI είναι χρονοβόρα, απαιτεί το γράψιμο πολλών γραμμών κώδικα και ακολουθεί μια, συνήθως μακρά, διαδικασία δοκιμής και σφάλματος.

Στην Εικόνα 17.1 παρουσιάζεται το παράθυρο μιας εφαρμογής GUI που ανέπτυξε ο συγγραφέας του βιβλίου για το πρόγραμμα EnviroBLASTO, το οποίο υπολογίζει τις επιπτώσεις στον περιβάλλοντα χώρο των ανατινάξεων κατά την εξόρυξη των πετρωμάτων και αποτελείται από 2598 γραμμές κώδικα Python.

Στο Κεφάλαιο αυτό θα αναπτύξουμε μια εφαρμογή GUI που θα αποτελέσει τη διεπιφάνεια επικοινωνίας του χρήστη με το πρόγραμμα `BLASTprn.py` ο κώδικας του οποίου είναι αυτός που διαμορφώθηκε στο τέλος του Κεφαλαίου 10 (βλέπε παράγραφο 10.14 *Ο κώδικας στο τέλος του Κεφαλαίου 10*).

17.2 Η βιβλιοθήκη Tkinter

Η Tkinter είναι μια βιβλιοθήκη ανοικτού κώδικα για την ανάπτυξη εφαρμογών GUI στην Python, η οποία εγκαθίσταται αυτόματα στον υπολογιστή μας μαζί με την Python 3. Σχετικά ανατρέξτε στο website: <https://docs.python.org/3/library/tkinter.html>

Είναι κατάλληλη για την ανάπτυξη απλών αλλά και πιο σύνθετων εφαρμογών GUI.

Για παράδειγμα, η εφαρμογή GUI που παρουσιάζεται στην Εικόνα 17.1 έχει αναπτυχθεί με τη βοήθεια της βιβλιοθήκης Tkinter και της βιβλιοθήκης Pmw.

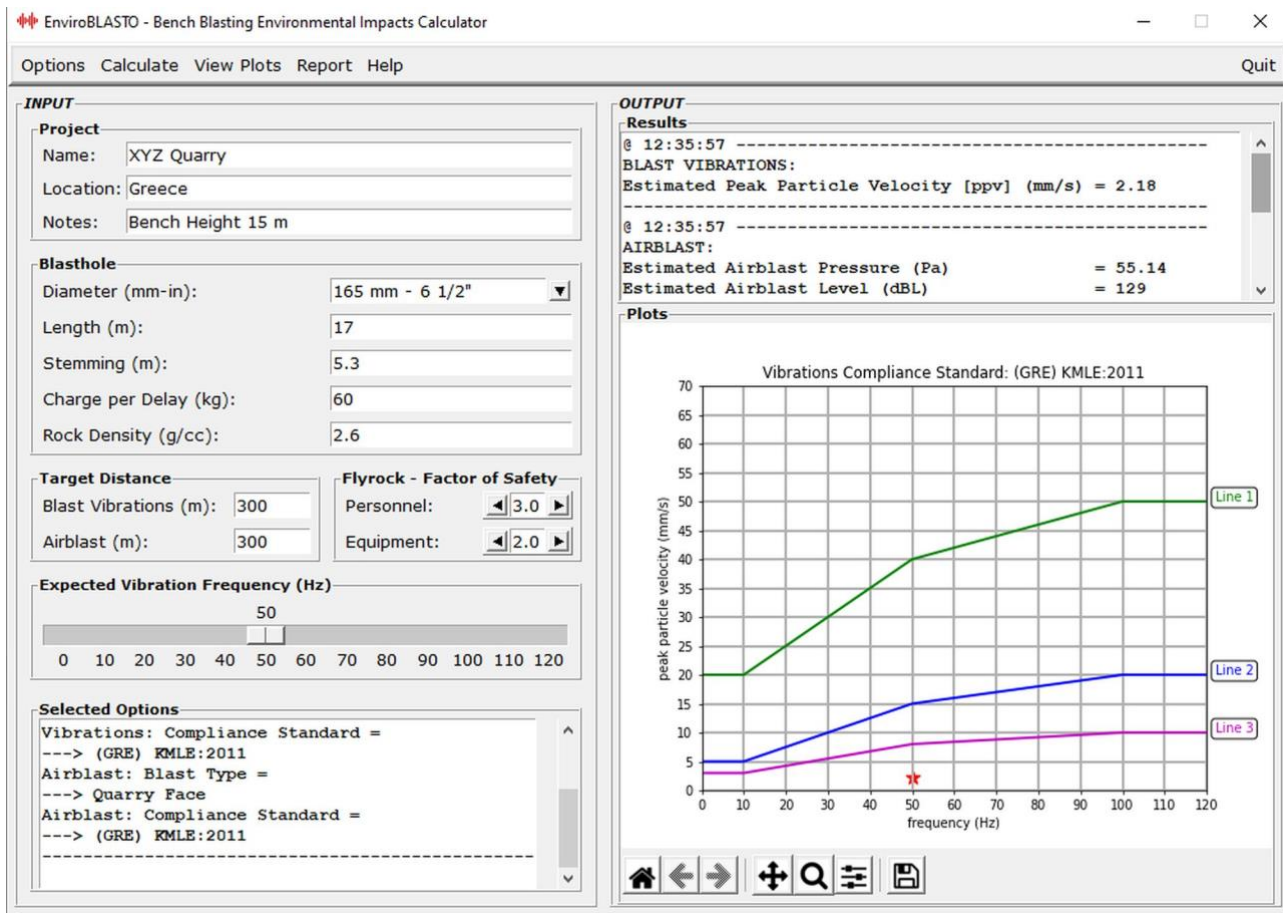
Η βιβλιοθήκη Pmw βασίζεται στην Tkinter την οποία και επεκτείνει με πολύ χρήσιμες κλάσεις. Θα τη γνωρίσουμε στην παράγραφο 17.7 *Τα megawidgets της βιβλιοθήκης Pmw*.

Η Tkinter είναι επίσης πολύ καλά τεκμηριωμένη και διαθέτει πολλά βοηθήματα, βλέπε σχετικά στο: <https://wiki.python.org/moin/TkInter> καθώς και εξαιρετικά online εγχειρίδια εκμάθησης, όπως για παράδειγμα είναι οι εκπαιδευτικές σημειώσεις *Python GUI with Tkinter*, οι οποίες είναι προσπελάσιμες στο website: <https://coderslegacy.com/python/python-gui>

Η Tkinter για να κτίσει μια εφαρμογή GUI χρησιμοποιεί *widgets*, δηλαδή αντικείμενα διεπαφών.

Ανάλογα με το τι GUI θέλουμε να κτίσουμε, επιλέγουμε το κατάλληλο widget, το συνδυάζουμε με τα προηγούμενα και τα επόμενα widgets, το εντάσσουμε στο παράθυρο της εφαρμογής μας, και τέλος, το διασυνδέουμε με τον κώδικα της εφαρμογής, η οποία κάνει τους υπολογισμούς και δίνει τα αποτελέσματα του προγράμματος.

Στις επόμενες παραγράφους θα δούμε τα στάδια ανάπτυξης της GUI εφαρμογής μας.



Εικόνα 17.1. Το παράθυρο GUI του προγράμματος EnviroBLASTO.

17.3 Πλάνο διαμόρφωσης του κώδικα BLASTppnGUI .py

Ο αντικειμενικός σκοπός μας είναι να κατασκευάσουμε ένα παράθυρο GUI, στο περιβάλλον των Windows 10, μέσα από το οποίο ο χρήστης θα τρέχει το πρόγραμμα BLASTppn .py, επαναλαμβανόμενα, μεταβάλλοντας τις τιμές μίας ή περισσότερων εκ των μεταβλητών των δεδομένων εισόδου, χωρίς να τερματίζεται η λειτουργία του προγράμματος μεταξύ των διαδοχικών τρεξιμάτων.

Εάν ανατρέξουμε στον κώδικα του προγράμματος BLASTppn .py, στην παράγραφο 10.14 Ο κώδικας στο τέλος του Κεφαλαίου 10, θα διαπιστώσουμε ότι τα δεδομένα εισόδου του προγράμματος, παραβλέποντας εκείνα που αφορούν τη διαδικασία πιστοποίησης του χρήστη, είναι:

- Distance from Blast [m]
- Weight of Explosives [kg]
- PPV Estimation Method [USBM or DIN]
- Type of Blast [Mine, Quarry or Construction]
- Type of Rock [Sedimentary or Igneous]
- Estimated ppv frequency (1 - 120 Hz) [Hz]
- Plot "ppv vs Distance" Start Value (Min = 100) [m]
- Plot "ppv vs Distance" End Value [m]

Σε αυτά θα προσθέσουμε και άλλο ένα που θα το ονομάσουμε:

- Project Name

και στο οποίο ο χρήστης θα γράφει, προαιρετικά, το όνομα ή πληροφορίες που θα αφορούν ένα συγκεκριμένο project για το οποίο θα τρέξει το πρόγραμμα.

Από την εκτέλεση του `BLASTppv.py` που κάναμε στο Κεφάλαιο 10 γνωρίζουμε επίσης ότι τα αποτελέσματα των υπολογισμών, που δίνονται υπό μορφή κειμένου, είναι:

- Peak Particle Velocity (USBM) [mm/s] ή
- Peak Particle Velocity (DIN) [mm/s]

Επίσης, στα αποτελέσματα περιλαμβάνονται και δύο γραφικές παραστάσεις:

- το διάγραμμα μεταβολής της ppv συναρτήσει της απόστασης,
- το διάγραμμα συμμόρφωσης της ppv /συχνότητας με τον ΚΜΛΕ.

Επομένως, μια λογική προσέγγιση στον σχεδιασμό του παραθύρου της GUI έκδοσης του προγράμματος `BLASTppv.py` είναι να το *διαιρέσουμε* λειτουργικά και οπτικά σε τρεις περιοχές: *Δεδομένα Εισόδου*, *Αποτελέσματα* και *Γραφικές Παραστάσεις*.

Το νέο πρόγραμμα που θα προκύψει θα το ονομάσουμε: `BLASTppvGUI.py`

17.4 Το βασικό GUI παράθυρο

Θα αρχίσουμε την ανάπτυξη του `BLASTppvGUI.py` διαμορφώνοντας το βασικό GUI παράθυρο.

Γράφουμε και τρέχουμε τον παρακάτω κώδικα:

```
from tkinter import *
class MyGUI:
    def __init__(self):
        self.root = Tk()
        self.root.geometry('850x750')
        self.root.title('BLASTppvGUI - PROGRAMMA YPOLOGISMOY THS '
                        'PPV ANATINAXHS')
        self.frame_1 = Frame(self.root)
        self.frame_1.pack(side=TOP)
        label_top = Label(self.frame_1, text='INPUT DATA')
        label_top.pack()
        self.root.mainloop()

if __name__ == "__main__":
    blastppvgui = MyGUI()
```

Με την πρώτη γραμμή κώδικα κάνουμε `import` τη βιβλιοθήκη `tkinter` στο πρόγραμμά μας, με τρόπο ώστε στη συνέχεια να έχουμε πρόσβαση σε μεταβλητές, συναρτήσεις και κλάσεις της βιβλιοθήκης χωρίς να απαιτείται η πρόσθεση στα ονόματά τους του prefix `tkinter`.

Ακολουθεί ο ορισμός της κλάσης `MyGUI` και τέλος το κυρίως πρόγραμμα που αποτελείται από την εντολή:

```
blastppvgui = MyGUI()
```

η οποία δημιουργεί ένα στιγμιότυπο της κλάσης `MyGUI` με όνομα `blastppvgui`.

Στο μπλοκ της μεθόδου constructor `__init__(self)` της κλάσης `MyGUI`:

Με την εντολή `self.root = Tk()` δημιουργούμε ένα στιγμιότυπο της κλάσης `tkinter.Tk()`, το οποίο μας κατασκευάζει το βασικό παράθυρο της εφαρμογής μας (Σημειώστε ότι στον κώδικα το `tkinter` δεν χρειάζεται να το γράψουμε λόγω του τρόπου με τον οποίο κάναμε `import` τη βιβλιοθήκη `tkinter`).

Κατά σύμβαση, στο παράθυρο αυτό δίνουμε το όνομα `self.root` και σε αυτό περιλαμβάνονται και τα συνήθη κουμπιά *Minimize*, *Restore Down/Maximize* και *Close* ενός τυπικού παράθυρου των Windows.

Οι επόμενες δύο εντολές ορίζουν, η μεν πρώτη τη γεωμετρία (`geometry`) του παράθυρου, δηλαδή το Πλάτος x Ύψος σε pixels, η δε δεύτερη τον τίτλο (`title`) που θα εμφανίζεται στο παράθυρο.

Στη συνέχεια, εισάγουμε εντός του παράθυρου που μόλις δημιουργήσαμε, το πρώτο μας widget που είναι ένα αντικείμενο τύπου Πλαίσιο (`Frame`) με όνομα `self.frame_1` χωρίς περίγραμμα.

Η εντολή που ακολουθεί, με τη βοήθεια της μεθόδου `pack()`, αφενός μεν καθορίζει τη θέση του πλαισίου μας εντός του παραθύρου, η οποία θα είναι στην ΑΝΩ (TOP) πλευρά του, αφετέρου δε το κάνει ορατό το πλαίσιο, όταν εμφανιστεί το παράθυρο στην οθόνη του υπολογιστή.

Η μέθοδος `pack()` πρέπει να καλείται και κάθε φορά που δημιουργείται ένα widget, προκειμένου να γίνει αυτό ορατό στο παράθυρο μιας GUI εφαρμογής.

Τα Πλαίσια είναι πολύ χρήσιμα, γιατί λειτουργούν ως containers, εντός των οποίων μπορούμε να περιλάβουμε ένα ή περισσότερα widget και έτσι να οργανώσουμε το γραφικό μας περιβάλλον.

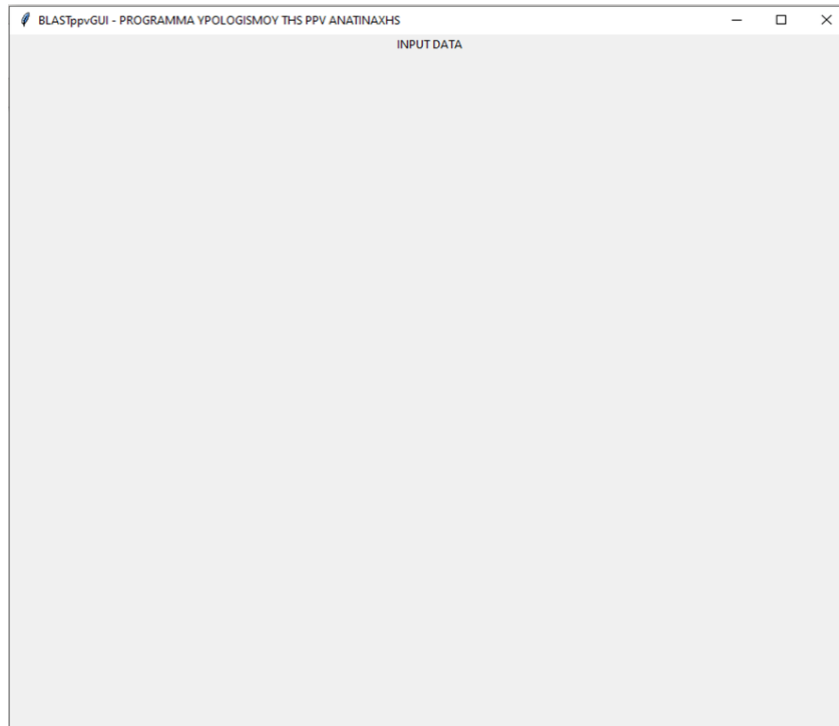
Η επόμενη εντολή δημιουργεί ένα αντικείμενο τύπου *Ετικέτα* (`Label`), εντός του πλαισίου `self.frame_1` που περιέχει το κείμενο `INPUT DATA` και το ονοματίζουμε `label_top`.

Το αντικείμενο αυτό εμφανίζεται κεντραρισμένο στην κορυφή του πλαισίου `self.frame_1` με την εντολή `label_top.pack()`

Η τελευταία εντολή της κλάσης `MyGUI` εφαρμόζει τη μέθοδο `mainloop()` επί του αντικειμένου `self.root`

Αυτό σημαίνει ότι εντός του παράθυρου GUI μπορούμε να εκτελέσουμε κινήσεις και εντολές με το ποντίκι ή το πληκτρολόγιο, να συμβούν γεγονότα, να εφαρμοστούν οι κατάλληλες κατά περίπτωση μέθοδοι για τον χειρισμό των γεγονότων και το παράθυρο να παραμένει ανοιχτό. Το παράθυρο θα κλείσει πατώντας το κουμπί *Close* του παράθυρου GUI.

Στην Εικόνα 17.2 παρουσιάζεται το παράθυρο GUI της εφαρμογής μας, όπως έχει διαμορφωθεί μέχρι τώρα.



Εικόνα 17.2. Η αρχική μορφή του παράθυρου GUI του προγράμματος `BLASTppvGUI.py`

Η Tkinter διαθέτει τρεις διαχειριστές διάταξης (layout managers) των widgets, τις μεθόδους: `pack()`, `grid()` και `place()`.

Στον κώδικα που εμφάνισε το παράθυρο GUI της Εικόνας 17.2 χρησιμοποιήσαμε τον διαχειριστή διάταξης `pack()`, στη συνέχεια όμως, για την ανάπτυξη της εφαρμογής μας, θα χρησιμοποιήσουμε τον διαχειριστή `grid()`, ο οποίος θεωρείται γενικά ότι είναι πιο εύκολα κατανοητός, συγκριτικά με τους άλλους δύο.

Σύμφωνα με τον διαχειριστή διάταξης `grid()`, η επιφάνεια του παράθυρου GUI αποτελείται από μια μήτρα (matrix) με κελιά (cells), η οποία έχει γραμμές (rows) και στήλες (columns), κατ' αναλογία με ένα φύλλο του Excel. Τα widgets καταλαμβάνουν ένα ή περισσότερα κελιά (Εικόνα 17.3).

Μια πολύ καλή παρουσίαση του τρόπου χρήσης του διαχειριστή διάταξης `grid()` μπορείτε να διαβάσετε στο website: <https://www.pythonguis.com/tutorials/create-ui-with-tkinter-grid-layout-manager> από το οποίο προέρχεται και η Εικόνα 17.3.

Σημειώστε ότι όταν χρησιμοποιούμε τον διαχειριστή διάταξης `grid()` δεν χρειάζεται να διαμορφώνουμε Πλαίσιο (Frame) για την ένταξη στην επιφάνειά του των widgets, όπως συμβαίνει με τον διαχειριστή διάταξης `pack()`. Υπάρχει όμως και εδώ η απαίτηση να καλείται η μέθοδος `grid()` κάθε φορά που δημιουργείται ένα widget, προκειμένου να γίνει αυτό ορατό στο παράθυρο της GUI εφαρμογής μας.



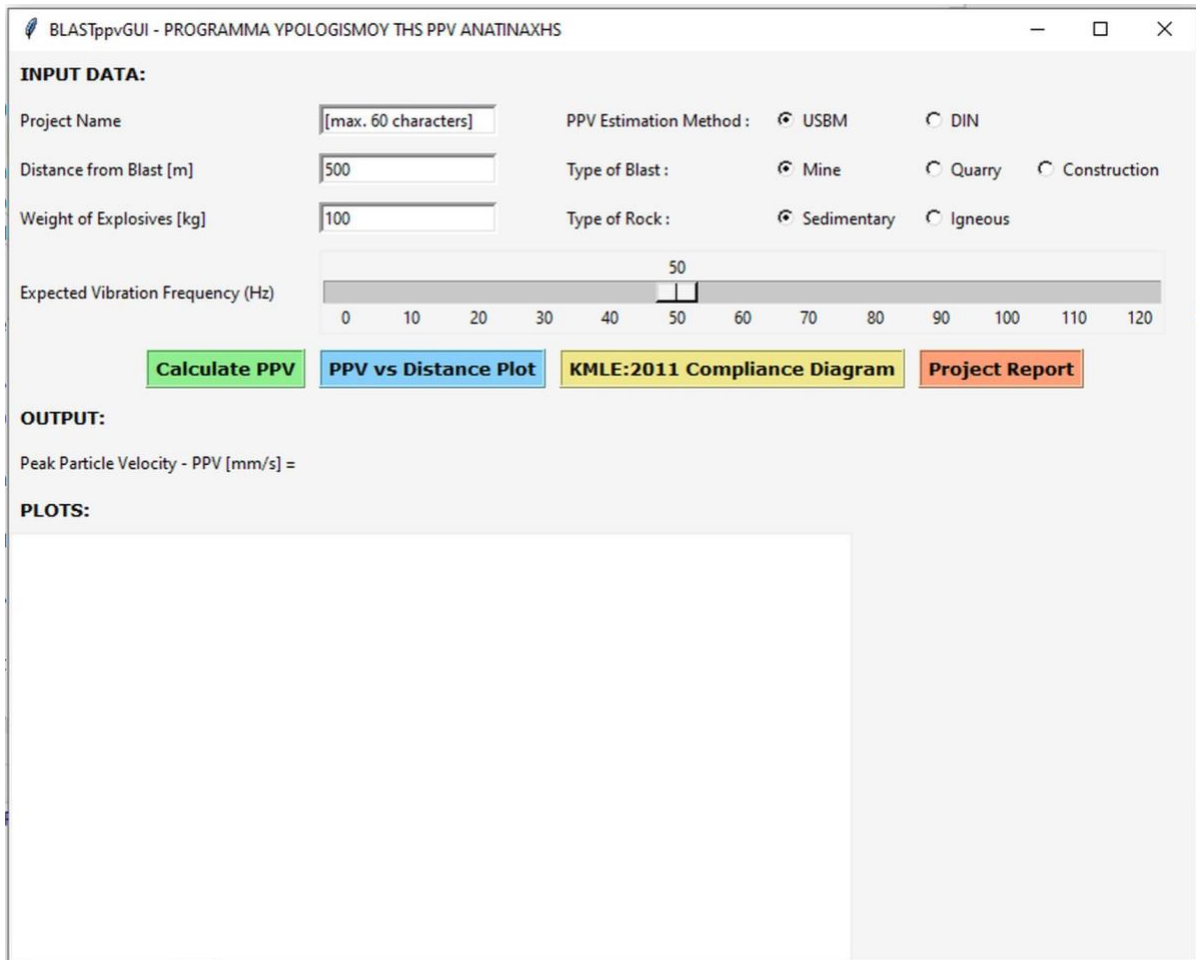
Εικόνα 17.3. Σχηματική διάταξη widgets με βάση τον διαχειριστή διάταξης `grid()` [Πηγή: <https://www.pythonguis.com/tutorials/create-ui-with-tkinter-grid-layout-manager>].

Πέραν του υπολογιστικού μέρους, που αφορά το γράψιμο του κώδικα Python μιας εφαρμογής GUI, είναι προφανές ότι κυρίαρχο είναι και το θέμα της *αισθητικής εικόνας* (διάταξη, χρώματα, γραμματοσειρές, κ.ά.) αλλά και της πρακτικής-λειτουργικής μορφής που θα έχει το παράθυρο μιας εφαρμογής GUI.

Με άλλα λόγια, η εμφάνιση μιας εφαρμογής GUI είναι θέμα προσωπικών επιλογών του ερασιτέχνη προγραμματιστή ή επιλογών του πελάτη ή του οργανισμού για τον οποίο εργάζεται ένας επαγγελματίας προγραμματιστής.

17.5 Το GUI παράθυρο του `BLASTprvGUI.py`

Προσπαθώντας να συνδυάσει τη λειτουργικότητα του παραθύρου GUI, την απλότητα του κώδικα Python που το υλοποιεί και είναι σύμφωνη με τους διδακτικούς σκοπούς αυτού του βιβλίου, αλλά και την προσωπική του αισθητική, ο συγγραφέας κατέληξε στη διαμόρφωση του παραθύρου GUI που παρουσιάζεται στην Εικόνα 17.4.



Εικόνα 17.4. Το παράθυρο GUI του προγράμματος BLASTppvGUI.py

Ο κώδικας που διαμορφώνει το παράθυρο της Εικόνας 17.4 και μετατρέπει το πρόγραμμα BLASTppv.py σε μια εφαρμογή GUI, δηλαδή στο πρόγραμμα BLASTppvGUI.py, δίνεται στη συνέχεια:

```
# To BLASTppvGUI.py einai ena programma poy
# ypologizei tin ppv mias anatinaxis se metalleia/latomeia/erga
# mesa apo ena Graphical User Interface (GUI)

# Import of Packages
from tkinter import *
from tkinter import simpledialog
from tkinter import messagebox
from fpdf import FPDF
import matplotlib.pyplot as plt
import numpy as np
import os

# Dictionaries
usbm_ppv_Dix = {'Mine': [1140., -1.60], 'Quarry': [1090., -1.82],
                 'Construction': [1730., -1.60]}
din_ppv_Dix = {'Sedimentary': [969, 0.6, -1.5],
                'Igneous': [206, 0.8, -1.3]}

class MyGUI:
    def __init__(self):
        self.root = Tk()
        self.root.geometry('850x750')
        self.root.maxsize(850, 750) # width x height
```

```

self.root.config(bg="whitesmoke")
self.root.title('BLASTppvGUI - PROGRAMMA YPOLOGISMOY THS '
                'PPV ANATINAXHS')
self.remove_files()
# Data Input Section
self.label_input = Label(self.root, text='INPUT DATA:',
                          bg="whitesmoke", font='verdana 10 bold')
self.label_input.grid(row=0, column=0, columnspan=1,
                      padx=5, pady=5, sticky=W)
# Project Name
self.label_project = Label(self.root, text='Project Name',
                           bg="whitesmoke")
self.label_project.grid(row=1, column=0, columnspan=1,
                       padx=5, pady=5, sticky=W)
self.project_name = StringVar()
self.project_name.set('[max. 60 characters]')
self.project_name = Entry(self.root, bd=3,
                          textvariable=self.project_name)
self.project_name.grid(row=1, column=1, columnspan=1,
                      padx=5, pady=5, sticky=W)
# Distance from Blast
self.label_distance = Label(self.root,
                             text='Distance from Blast [m]', bg="whitesmoke")
self.label_distance.grid(row=2, column=0, columnspan=1,
                        padx=5, pady=5, sticky=W)
self.distance_from_blast = StringVar()
self.distance_from_blast.set('500')
self.distance_from_blast = Entry(self.root, bd=3,
                                 textvariable=self.distance_from_blast)
self.distance_from_blast.grid(row=2, column=1, padx=5,
                              pady=5, sticky=W)
# Weight of Explosives
self.label_weight = Label(self.root,
                           text='Weight of Explosives [kg]', bg="whitesmoke")
self.label_weight.grid(row=3, column=0, columnspan=1,
                      padx=5, pady=5, sticky=W)
self.weight_of_explosives = StringVar()
self.weight_of_explosives.set('100')
self.weight_of_explosives = Entry(self.root, bd=3,
                                 textvariable=self.weight_of_explosives)
self.weight_of_explosives.grid(row=3, column=1, padx=5,
                              pady=5, sticky=W)
# PPV Estimation Method
self.ppv_method = StringVar()
self.ppv_method.set('USBM')
self.label_ppv_method = Label(self.root,
                              text='PPV Estimation Method :', bg="whitesmoke")
self.label_ppv_method.grid(row=1, column=3, columnspan=1,
                          padx=5, pady=5, sticky=W)
k = -1
for i in ['USBM', 'DIN']:
    self.ppv_method_list = Radiobutton(self.root, text=i,
                                       variable=self.ppv_method, value=i, bg="whitesmoke")
    k = k + 1
    self.ppv_method_list.grid(row=1, column=4+k, padx=5,
                             pady=5, sticky=W)
# USBM Method options (Type of Blast)
self.type_of_blast = StringVar()
self.type_of_blast.set('Mine')
self.label_type_of_blast = Label(self.root,
                                 text="Type of Blast :", bg="whitesmoke")

```

```

self.label_type_of_blast.grid(row=2, column=3, padx=5,
                              pady=5, sticky=W)

k = -1
for i in ['Mine', 'Quarry', 'Construction'] :
    self.type_of_blast_list = Radiobutton(self.root, text=i,
        variable=self.type_of_blast, value=i, bg="whitesmoke")
    k = k + 1
    self.type_of_blast_list.grid(row=2, column=4+k, padx=5,
        pady=5, sticky=W)

# DIN Method options (Type of Rock)
self.type_of_rock = StringVar()
self.type_of_rock.set('Sedimentary')
self.label_type_of_rock = Label(self.root,
    text="Type of Rock :", bg="whitesmoke")
self.label_type_of_rock.grid(row=3, column=3, padx=5,
    pady=5, sticky=W)

k = -1
for i in ['Sedimentary', 'Igneous']:
    self.type_of_rock_list = Radiobutton(self.root, text=i,
        variable=self.type_of_rock, value=i, bg="whitesmoke")
    k = k + 1
    self.type_of_rock_list.grid(row=3, column=4 + k, padx=5,
        pady=5, sticky=W)

# Frequency
self.label_frequency = Label(self.root,
    text='Expected Vibration Frequency (Hz)', bg="whitesmoke")
self.label_frequency.grid(row=4, column=0, columnspan=1,
    padx=5, pady=5, sticky=W)

self.frequency_estimated = Scale(self.root,
    from_=0, to=120., orient=HORIZONTAL, tickinterval=10.,
    bg="whitesmoke")
self.frequency_estimated.grid(row=4, column=1, columnspan=7,
    padx=5, pady=5, sticky=W+E)

self.frequency_estimated.set(50)

# Calculate PPV Button
self.calculate_ppv_button = Button(self.root,
    text='Calculate PPV', relief=RIDGE, bg="lightgreen",
    font='verdana 10 bold', command=self.calculate_ppv)
self.calculate_ppv_button.grid(row=6, column=0,
    columnspan=1, padx=5, pady=5, sticky=E)

# PPV vs Distance Plot Button
self.ppv_vs_distance_button = Button(self.root,
    text='PPV vs Distance Plot', relief=RIDGE, bg="lightskyblue",
    font='verdana 10 bold', command=self.ppv_vs_distance_plot)
self.ppv_vs_distance_button.grid(row=6, column=1, columnspan=1,
    padx=5, pady=5, sticky=W)

# KMLE:2011 Compliance Diagram Button
self.kmle_diagram_button = Button(self.root,
    text='KMLE:2011 Compliance Diagram', relief=RIDGE,
    bg="khaki", font='verdana 10 bold',
    command=self.ppv_compliance_KMLE_plot)
self.kmle_diagram_button.grid(row=6, column=2,
    columnspan=3, padx=5, pady=5, sticky=W)

# Project Report Button
self.project_report_button = Button(self.root,
    text='Project Report', relief=RIDGE,
    bg="lightsalmon", font='verdana 10 bold',
    command=self.project_report)
self.project_report_button.grid(row=6, column=5,
    columnspan=2, padx=5, pady=5, sticky=W)

# Output Section

```

```

self.label_output = Label(self.root, text='OUTPUT:',
                           bg="whitesmoke", font='verdana 10 bold')
self.label_output.grid(row=7, column=0, columnspan=1,
                       padx=5, pady=5, sticky=W)
self.label_ppv = Label(self.root,
                       text='Peak Particle Velocity - PPV [mm/s] = ',
                       bg="whitesmoke")
self.label_ppv.grid(row=8, column=0, columnspan=1, padx=5,
                   pady=5, sticky=W)
self.ppv_out = StringVar()
self.display_output = Label(self.root,
                            textvariable=self.ppv_out, bg="whitesmoke",
                            font='verdana 12 bold')
self.display_output.grid(row=8, column=1, columnspan=1,
                        padx=5, pady=5, sticky=W)

# Plots Section
self.label_plots = Label(self.root, text='PLOTS:',
                          bg="whitesmoke", font='verdana 10 bold')
self.label_plots.grid(row=9, column=0, columnspan=1,
                      padx=5, pady=5, sticky=W)
self.canvas_area = Canvas(self.root, bg='white',
                          width=600,height=450)
self.canvas_area.grid(row=10, column=0, rowspan=10,
                      columnspan=11, sticky=W)

self.root.mainloop()

def calculate_ppv(self):
    """Calculates ppv"""
    if self.ppv_method.get() == 'USBM':
        self.ppv = usbm_ppv_Dix[self.type_of_blast.get()][0] * \
            (float(self.distance_from_blast.get())/
             np.sqrt(float(self.weight_of_explosives.get())))) \
            ** usbm_ppv_Dix[self.type_of_blast.get()][1]
    elif self.ppv_method.get() == 'DIN':
        self.ppv = din_ppv_Dix[self.type_of_rock.get()][0] * \
            (float(self.weight_of_explosives.get()) **
             din_ppv_Dix[self.type_of_rock.get()][1]) * \
            (float(self.distance_from_blast.get()) **
             din_ppv_Dix[self.type_of_rock.get()][2])
    self.ppv_out.set('{:.2f}'.format(self.ppv))

def ppv_vs_distance_plot(self):
    """Plots the ppv-distance curve"""
    self.end_value = simpldialog.askfloat("PPV vs Distance Plot "
                                         "Values", "Maximum Plot Distance [m]",
                                         minvalue=200.0, maxvalue=10000.0, parent=self.root)
    if self.end_value is int or float:
        plot_start = 100
        plot_end = self.end_value
    if self.end_value is None:
        return
    fig1 = plt.figure(figsize=(6, 4), dpi=100)
    x = np.linspace(plot_start, plot_end)
    if self.ppv_method.get() == 'USBM':
        y = usbm_ppv_Dix[self.type_of_blast.get()][0]*\
            ((x/np.sqrt(float(self.weight_of_explosives.get()))))
            **usbm_ppv_Dix[self.type_of_blast.get()][1]
        plt.title('ppv-USBM vs Distance')
    elif self.ppv_method.get() == 'DIN':
        y = din_ppv_Dix[self.type_of_rock.get()][0] * \
            (float(self.weight_of_explosives.get()) **

```

```

        din_ppv_Dix[self.type_of_rock.get()][1]) * \
        (x ** din_ppv_Dix[self.type_of_rock.get()][2])
    plt.title('ppv-DIN vs Distance')
    plt.plot(x, y, 'b-')
    plt.xlabel('Distance from blast [m]')
    plt.ylabel('ppv [mm/s]')
    plt.xticks(np.arange(min(x), max(x)*1.1, max(x)/10))
    plt.yticks(np.arange(0, max(y) + 1, 5))
    plt.grid(True)
    self.root.update()
    if self.ppv_method.get() == 'USBM':
        fig1.savefig('ppv_USBM_vs_distance_GUI.png')
        self.ppv_vs_distance_plot = PhotoImage(file=
            'ppv_USBM_vs_distance_GUI.png')
        self.canvas_area.create_image(0, 0,
            image=self.ppv_vs_distance_plot, anchor=NW)
    elif self.ppv_method.get() == 'DIN':
        fig1.savefig('ppv_DIN_vs_distance_GUI.png')
        self.ppv_vs_distance_plot = PhotoImage(file=
            'ppv_DIN_vs_distance_GUI.png')
        self.canvas_area.create_image(0, 0,
            image=self.ppv_vs_distance_plot, anchor=NW)
    fig1.clear()

def ppv_compliance_KMLE_plot(self):
    """Plots the KMLE compliance diagram Lines 1,2 & 3
    and the (ppv, frequency) point on the diagram"""
    self.root.update()
    self.calculate_ppv()
    fig0 = plt.figure(figsize=(6, 4), dpi=100)
    box = {'facecolor': 'w', 'edgecolor': 'k',
        'boxstyle': 'round'}

    def line1(freq):
        if freq <= 10.:
            return 20.
        elif 10. < freq <= 50.:
            return 20. + (((freq - 10.) / 40.) * 20.)
        elif 50. < freq <= 100.:
            return 40. + (((freq - 50.) / 50.) * 10.)
        elif freq > 100.:
            return 50.
        else:
            return 0.
    freq = np.linspace(0., 120., 121)
    y = np.zeros(len(freq))
    for i in range(len(freq)):
        y[i] = line1(freq[i])
    plt.plot(freq, y, 'g', lw=2)

    def line2(freq):
        if freq <= 10.:
            return 5.
        elif 10. < freq <= 50.:
            return 5. + (((freq - 10.) / 40.) * 10.)
        elif 50. < freq <= 100.:
            return 15. + (((freq - 50.) / 50.) * 5.)
        elif freq > 100.:
            return 20.
        else:
            return 0.

```

```

for i in range(len(freq)):
    y[i] = line2(freq[i])
plt.plot(freq, y, 'b', lw=2)

def line3(freq):
    if freq <= 10.:
        return 3.
    elif 10. < freq <= 50.:
        return 3. + (((freq - 10.) / 40.) * 5.)
    elif 50. < freq <= 100.:
        return 8. + (((freq - 50.) / 50.) * 2.)
    elif freq > 100.:
        return 10.
    else:
        return 0.

for i in range(len(freq)):
    y[i] = line3(freq[i])

plt.plot(freq, y, 'm', lw=2)
plt.title('Vibrations Compliance Standard: (GRE) KMLE:2011')
plt.grid(True, lw=2)
plt.xlabel('Frequency (Hz)')
plt.xlim(0, 120)
plt.ylabel('Peak Particle Velocity (mm/s)')
plt.ylim(0, 70)
plt.xticks(np.arange(0, 120 + 1, 10))
plt.yticks(np.arange(0, 70 + 1, 5))
plt.text(122, 50, 'Line 1', bbox=box).set_color('g')
plt.text(122, 20, 'Line 2', bbox=box).set_color('b')
plt.text(122, 10, 'Line 3', bbox=box).set_color('m')
plt.scatter(float(self.frequency_estimated.get()),
            self.ppv, marker='*', s=140, color='r')
if self.ppv > 69.:
    plt.text(70, 60, '!!! ppv = {:.2f} mm/s'.format(self.ppv),
            bbox=box, color='r')
fig0.savefig('KMLE_compliance_plot_GUI.png')
self.kmle_plot = PhotoImage(file='KMLE_compliance_plot_GUI.png')
self.canvas_area.create_image(0,0, image=self.kmle_plot,
                              anchor=NW)

fig0.clear()

def project_report(self):
    f = open('project_report_GUI.txt', 'w')
    print('PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS', file=f)
    print('Project Name : {:s}'.format(self.project_name.get()),
          file=f)
    print('INPUT DATA:', file=f)
    print('Peak Particle Velocity Estimation Method : {:s}'.
          format(self.ppv_method.get()), file=f)
    if self.ppv_method.get() == 'USBM':
        print('Type of Blast : {:s}'.
              format(self.type_of_blast.get()), file=f)
    elif self.ppv_method.get() == 'DIN':
        print('Type of Rock : {:s}'.
              format(self.type_of_rock.get()), file=f)
    print('Distance from Blast [m] : {:.1f}'.
          format(float(self.distance_from_blast.get()))),
          file=f)
    print('Weight of Explosives [kg] : {:.1f}'.

```

```

        format(float(self.weight_of_explosives.get()),
               file=f)
print('Estimated ppv frequency [Hz] : {:.1f}'.
      format(float(self.frequency_estimated.get()),
             file=f)
print('OUTPUT:', file=f)
print('Peak Particle Velocity ({:s}) [mm/s] = {:.2f}'.
      format(self.ppv_method.get(), self.ppv),
      file=f)
f.close()
# Output from txt to pdf
if os.path.isfile('project_report_GUI.pdf'):
    try:
        os.remove('project_report_GUI.pdf')
    except OSError:
        messagebox.showerror("ERROR", "File "
                              "'project_report_GUI.pdf' is OPEN in Acrobat "
                              "Reader! CLOSE or SAVE it with a different name!")
report = FPDF() # = FPDF('P', 'mm', 'A4')
# Add a page
report.add_page()
report.set_font("Courier", "", size=11)
# open the text file in read mode
f = open("project_report_GUI.txt", "r")
# insert lines of text in pdf
for x in f:
    report.cell(200, 7, txt=x, ln=1, align='L')
report.ln(h='10')
# insert images
if self.ppv_method.get() == 'USBM' and \
    os.path.isfile('ppv_USBM_vs_distance_GUI.png'):
    report.image('ppv_USBM_vs_distance_GUI.png',
                w=120, h=75, type='PNG')
elif self.ppv_method.get() == 'DIN' and \
    os.path.isfile('ppv_DIN_vs_distance_GUI.png'):
    report.image('ppv_DIN_vs_distance_GUI.png',
                w=120, h=75, type='PNG')
report.ln(h='')
if os.path.isfile('KMLE_compliance_plot_GUI.png'):
    report.image('KMLE_compliance_plot_GUI.png',
                w=120, h=75, type='PNG')
# save the pdf with name .pdf
report.output("project_report_GUI.pdf").encode('latin-1')
# read .pdf with default reader
try:
    os.startfile("project_report_GUI.pdf")
except OSError:
    pass

def remove_files(self):
    """Removes all files created during previous session"""
    if os.path.isfile('ppv_USBM_vs_distance_GUI.png'):
        os.remove('ppv_USBM_vs_distance_GUI.png')
    if os.path.isfile('ppv_DIN_vs_distance_GUI.png'):
        os.remove('ppv_DIN_vs_distance_GUI.png')
    if os.path.isfile('KMLE_compliance_plot_GUI.png'):
        os.remove('KMLE_compliance_plot_GUI.png')
    if os.path.isfile('project_report_GUI.txt'):
        os.remove('project_report_GUI.txt')
    if os.path.isfile('project_report_GUI.pdf'):
        try:

```



```

        os.remove('project_report_GUI.pdf')
    except OSError:
        messagebox.showerror("ERROR", "File "
            "'project_report_GUI.pdf' is OPEN in "
            "Acrobat Reader! CLOSE or SAVE it with a "
            "different name!")

if __name__ == "__main__":
    blastppvgui = MyGUI()

```

Καταρχάς, δεν θα τρομάξουμε από το μέγεθος του κώδικα! Είναι γεγονός ότι ο κώδικας εφαρμογών GUI έχει μεγάλη έκταση.

Το script του προγράμματος `BLASTppvGUI.py` βρίσκεται στο website του βιβλίου.

Παρατηρώντας πιο *ψύχραιμα* τον κώδικα διαπιστώνουμε ότι έχει την ίδια δομή με τον κώδικα που διαμόρφωσε το βασικό παράθυρο GUI και παρουσιάσαμε-αναλύσαμε στην παράγραφο 17.4 *Το βασικό GUI παράθυρο*.

Η διαφορά είναι ότι τώρα στο σώμα της μεθόδου constructor `__init__(self)` της κλάσης `MyGUI` περιλαμβάνονται οι ορισμοί πολύ περισσότερων widgets, τόσων, όσων απαιτούνται για να έχουμε το αποτέλεσμα της Εικόνας 17.4.

Γενικά, τα widgets διακρίνονται σε *στατικά* και *δυναμικά*.

Τα στατικά widgets είναι συνήθως αντικείμενα τύπου *Ετικέτας* (`Label`).

Έτσι, χρησιμοποιώντας αντικείμενα Ετικέτας έχουμε χωρίσει το παράθυρο σε τρεις λειτουργικές περιοχές: `INPUT DATA:`, `OUTPUT:` και `PLOTS:` όπου θα εντάξουμε τα widgets των δεδομένων εισόδου, του αποτελέσματος των υπολογισμών και των γραφικών παραστάσεων, αντίστοιχα.

Τα δυναμικά widgets είναι συνήθως αντικείμενα τύπου *Εισόδου* (`Entry`) ή *Επιλογών* (μοναδικών ή πολλαπλών).

Για παράδειγμα, χρησιμοποιώντας αντικείμενα Εισόδου δίνουμε τιμές στις μεταβλητές *Distance from Blast* (Απόσταση από την Ανατίναξη) και *Weight of Explosives* (Βάρος Εκρηκτικών).

Αλλά ας δούμε τον κώδικα από την αρχή του script, στην κλάση `MyGUI`.

Η ομάδα εντολών:

```

self.root = Tk()
self.root.geometry('850x750')
self.root.maxsize(850, 750) # width x height
self.root.config(bg="whitesmoke")
self.root.title('BLASTppvGUI - PROGRAMMA ΥΠΟΛΟΓΙΣΜΟΥ ΤΗΣ '
                'PPV ANATINAXHS')

```

διαμορφώνει το βασικό παράθυρο της εφαρμογής μας, στο οποίο δίνουμε το όνομα `self.root` και έχει γεωμετρία (`geometry`) με διαστάσεις πλάτους 850 pixels και ύψος 750 pixels. Εξ ορισμού έχει ενσωματωμένα τα κουμπιά *Minimize*, *Restore Down/Maximize* και *Close* ενός τυπικού παραθύρου των Windows.

Επειδή δεν θέλουμε το παράθυρο να αλλάζει διαστάσεις όταν πατηθεί το κουμπί *Restore Down/Maximize* αλλά να διατηρεί τις αρχικές τιμές πλάτους-ύψους, εισάγουμε την εντολή:

```

self.root.maxsize(850, 750) # width x height

```

Η εντολή:

```

self.root.config(bg="whitesmoke")

```

καθορίζει το χρώμα του *υπόβαθρου* (background, bg) του παράθυρου να είναι μια *απόχρωση του γκρι* (whitesmoke).

Στη δημιουργία του GUI για την επιλογή χρωμάτων θα χρησιμοποιήσουμε το χρωματολόγιο της Εικόνας 7.4 από την παλέτα χρωμάτων της βιβλιοθήκης Matplotlib, αλλά υπάρχουν και πολλές άλλες επιλογές χρωμάτων (βλέπε σχετικά στην παράγραφο 17.6 *Επιλογή χρωμάτων με τη βοήθεια της tkinter*).

Η τελευταία εντολή γράφει στην επικεφαλίδα του παράθυρου τον τίτλο που επιθυμούμε.

17.5.1 To widget Label

Οι εντολές του κώδικα που ακολουθούν εκείνες της διαμόρφωσης του παράθυρου, δημιουργούν και εντάσσουν τα widgets της εφαρμογής μας στις τρεις λειτουργικές περιοχές (sections) του παράθυρου: INPUT DATA:, OUTPUT: και PLOTS:

Έτσι οι εντολές:

```
self.label_input = Label(self.root, text='INPUT DATA:',  
                          bg="whitesmoke", font='verdana 10 bold')  
self.label_input.grid(row=0, column=0, columnspan=1,  
                      padx=5, pady=5, sticky=W)
```

δημιουργούν ένα widget τύπου Ετικέτα (Label), στο παράθυρο self.root, που έχει το κείμενο (text) INPUT DATA:, σε χρώμα υπόβαθρου (bg) whitesmoke και γραμματοσειρά (font) verdana 10 bold

Αυτό το widget-αντικείμενο, που το ονοματίζουμε self.label_input, με τη βοήθεια του διαχειριστή διάταξης grid() εμφανίζεται στη γραμμή (row) 0 και στη στήλη (column) 0 και εκτείνεται (columnspan) καταλαμβάνοντας τον χώρο μιας (1) στήλης.

Σημειώστε ότι η αρίθμηση των γραμμών/στηλών, όπως και πολλών άλλων στοιχείων της Python, αρχίζει από το μηδέν. Η παράμετρος rowspan, κατ' αναλογία της columnspan, επιτρέπει την κατάληψη από ένα widget περισσότερων της μίας γραμμών.

Από τις υπόλοιπες παραμέτρους του διαχειριστή διάταξης grid(), οι μεν padx και pady ορίζουν τον κενό χώρο σε pixels του widget από την κάτω αριστερή γωνία (αρχή των αξόνων x & y) του κελιού, που στην περίπτωσή μας είναι 5 pixels, η δε παράμετρος sticky επιτρέπει τη στοίχιση του widget μέσα στο κελί του.

Η στοίχιση του widget γίνεται με τη χρήση των γραμμάτων: N, E, S, W τα οποία αντιστοιχούν στα τέσσερα σημεία του ορίζοντα North, East, South, West, καθώς και με τον συνδυασμό αυτών: NE, SE, SW, NW.

Η default στοίχιση-θέση του widget είναι στο κέντρο του κελιού όπου έχει τοποθετηθεί. Στην περίπτωσή μας, η επιλογή W κάνει στοίχιση του κειμένου INPUT DATA: στη δυτική (αριστερή) πλευρά του κελιού.

17.5.2 To widget Entry

Ακολουθούν οι γραμμές κώδικα που δημιουργούν τα widgets, με τα οποία ο χρήστης εισάγει δεδομένα με τη βοήθεια του πληκτρολογίου.

Αυτά αποτελούνται από δύο επιμέρους widgets το καθένα, ήτοι ένα widget που εμφανίζει το λεκτικό του δεδομένου εισόδου και ένα άλλο widget που δημιουργεί χώρο-πλαίσιο για την πληκτρολόγηση της τιμής του δεδομένου.

Για παράδειγμα, στον κώδικα, κάτω από τη γραμμή σχολίου # Distance from Blast, οι εντολές:

```
self.label_distance = Label(self.root,  
                             text='Distance from Blast [m]', bg="whitesmoke")  
self.label_distance.grid(row=2, column=0, columnspan=1,  
                         padx=5, pady=5, sticky=W)
```

δημιουργούν και εμφανίζουν στο παράθυρο GUI το λεκτικό του δεδομένου εισόδου που είναι η Απόσταση από την Ανατίναξη (Distance from Blast [m]), ενώ οι εντολές που ακολουθούν:

```
self.distance_from_blast = StringVar()
self.distance_from_blast.set('500')
self.distance_from_blast = Entry(self.root, bd=3,
                                textvariable=self.distance_from_blast)
self.distance_from_blast.grid(row=2, column=1, padx=5,
                              pady=5, sticky=W)
```

κατά σειρά:

Δημιουργούν τη μεταβλητή `self.distance_from_blast` στην οποία εκχωρούνται τα χαρακτηριστικά της κλάσης `StringVar()` της Tkinter. Αυτό μας επιτρέπει να παρακολουθούμε τις μεταβολές που συμβαίνουν στη μεταβλητή `self.distance_from_blast` λόγω εισαγωγής μια νέας τιμής για αυτήν από τον χρήστη.

Δίνουμε την αρχική τιμή 500 στη μεταβλητή `self.distance_from_blast` με τη βοήθεια της μεθόδου `set()`. Σημειώστε ότι όλες οι μεταβλητές εισόδου που δημιουργούνται με το widget `Entry` είναι τύπου συμβολοσειράς (`str`).

Στη μεταβλητή `self.distance_from_blast` εκχωρούμε ένα αντικείμενο της Tkinter τύπου `Entry()`, το οποίο δημιουργεί ένα ορθογώνιο πλαίσιο εντός του οποίου ο χρήστης πληκτρολογεί αλφαριθμητικούς χαρακτήρες. Η παράμετρος `bd=3` ορίζει το πάχος του πλαισίου να είναι 3 pixels (το default είναι 2 pixels) και η παράμετρος `textvariable=self.distance_from_blast` μας επιτρέπει να ανακτούμε την τρέχουσα τιμή που εμφανίζεται στο ορθογώνιο πλαίσιο, στη συνέχεια του κώδικα, ως τιμή της μεταβλητής `self.distance_from_blast`.

Με ανάλογο τρόπο γράφεται ο κώδικας για την εισαγωγή του ονόματος του project (Project Name) και του Βάρους της Εκρηκτικής Ύλης (Weight of Explosives [kg]) ως δεδομένων εισόδου του προγράμματος που απαιτούν πληκτρολόγηση.

Σημειώστε ότι η προτροπή στο όνομα του project [max. 60 characters] για περιορισμό των χαρακτήρων που θα πληκτρολογήσουμε σε 60, δεν έχει σχέση με περιορισμό που θέτει το widget `Entry`, αλλά σχετίζεται με την εμφάνιση των αποτελεσμάτων του προγράμματός μας σε αρχείο τύπου pdf (βλέπε παράγραφο 17.5.8 Η βιβλιοθήκη `fpdf`).

17.5.3 Το widget Radiobutton

Το επόμενο σετ των δεδομένων μας είναι του τύπου της μιας επιλογής από συγκεκριμένη λίστα δυνητικών επιλογών. Για παράδειγμα, προκειμένου να επιλεγεί η μέθοδος υπολογισμού της *prn*, δηλαδή εάν θα είναι σύμφωνα με τη μέθοδο του USBM ή του προτύπου DIN, ο χρήστης θα επιλέξει μόνο μία εκ των δύο επιλογών: USBM ή DIN.

Στην περίπτωση αυτή, θα χρησιμοποιήσουμε ένα αντικείμενο της Tkinter τύπου `Radiobutton()`, δηλαδή *Κουμπί Επιλογής*, το οποίο εμφανίζεται σε ομάδες δύο ή περισσότερων κουμπιών-επιλογών.

Συνήθως, κάποιο από τα κουμπιά-επιλογές είναι προεπιλεγμένο και η επιλογή αυτή καταργείται με την επιλογή ενός άλλου κουμπιού. Όλα τα κουμπιά της ομάδας επιλογών συσχετίζονται με μια κοινή μεταβλητή, η οποία λαμβάνει την τιμή του κουμπιού που επιλέγεται.

Για παράδειγμα, στον κώδικα κάτω από τη γραμμή σχολίου # PPV Estimation Method, οι εντολές:

```
self.ppv_method = StringVar()
self.ppv_method.set('USBM')
self.label_ppv_method = Label(self.root,
                              text='PPV Estimation Method :', bg="whitesmoke")
self.label_ppv_method.grid(row=1, column=3, columnspan=1,
                           padx=5, pady=5, sticky=W)
k = -1
```

```

for i in ['USBM', 'DIN']:
    self.ppv_method_list = Radiobutton(self.root, text=i,
        variable=self.ppv_method, value=i, bg="whitesmoke")
    k = k + 1
    self.ppv_method_list.grid(row=1, column=4+k, padx=5,
        pady=5, sticky=W)

```

κατά σειρά:

Δημιουργούν τη μεταβλητή `self.ppv_method` στην οποία εκχωρούμε τα χαρακτηριστικά της κλάσης `StringVar()` και της δίνουμε την αρχική τιμή `USBM`.

Δημιουργούν και εμφανίζουν στο παράθυρο GUI το λεκτικό του δεδομένου εισόδου, που είναι η Μέθοδος Εκτίμησης της `ppv` (`PPV Estimation Method` :), στο κελί το οποίο αντιστοιχεί στη γραμμή 1 και στη στήλη 3.

Στη συνέχεια, με τη βοήθεια ενός βρόχου `for` και της μεταβλητής `i`, η οποία παίρνει τιμές από τη λίστα `['USBM', 'DIN']`, καθώς και του αντικειμένου της Tkinter τύπου `Radiobutton()` δημιουργούμε τα κουμπιά επιλογής `USBM` και `DIN`, τα οποία συσχετίζονται με τη μεταβλητή `self.ppv_method`.

Η βοηθητική μεταβλητή `k` χρησιμοποιείται για τη στοίχιση των κουμπιών από τον διαχειριστή διάταξης `grid()`.

Κατά την εκτέλεση του προγράμματος, η επιλογή της τιμής ενός `Radiobutton` γίνεται με τη βοήθεια του ποντικιού (απλό κλικ) στο *μάτι της επιλογής*, το οποίο αποκτά τότε μια μαύρη βούλα.

Με ανάλογο τρόπο γράφεται ο κώδικας για την επιλογή του είδους της Ανατίναξης, όταν επιλεγεί η μέθοδος `USBM`, ο οποίος παρουσιάζεται μετά τη γραμμή σχολίου `# USBM Method options (Type of Blast)` ή ο κώδικας για την επιλογή του είδους του πετρώματος, όταν επιλεγεί η μέθοδος `DIN`, ο οποίος παρουσιάζεται μετά τη γραμμή σχολίου `# DIN Method options (Type of Rock)`.

17.5.4 To widget Scale

Για την εισαγωγή του δεδομένου της Αναμενόμενης Συχνότητας Δόνησης (`Expected Vibration Frequency (Hz)`) επιλέχθηκε η χρήση ενός αντικειμένου της Tkinter τύπου `Scale()`, δηλαδή μιας *Κλίμακας με κινητό δρομέα επιλογής τιμής*.

Ο κώδικας υλοποίησης βρίσκεται κάτω από τη γραμμή σχολίου `# Frequency`.

Με τις γραμμές κώδικα:

```

self.frequency_estimated = Scale(self.root,
    from_=0, to=120., orient=HORIZONTAL, tickinterval=10.,
    bg="whitesmoke")
self.frequency_estimated.grid(row=4, column=1, columnspan=7,
    padx=5, pady=5, sticky=W+E)
self.frequency_estimated.set(50)

```

στη μεταβλητή `self.frequency_estimated` εκχωρούμε το αντικείμενο της Tkinter τύπου `Scale()`, το οποίο δημιουργεί μια Κλίμακα με δρομέα από (`from_`) 0 έως (`to`) 120, οριζόντιας διάταξης (`orient=HORIZONTAL`), με σημάδια υποδιαίρεσης (`ticks`) και αριθμούς ανά διάστημα (`tickinterval`) 10 μονάδων.

Στη συνέχεια, με τον διαχειριστή διάταξης `grid()` επιλέγουμε η κλίμακα να καταλάβει τον χώρο (`columnspan`) 7 στηλών από άκρη σε άκρη (`sticky=W+E`).

Τέλος, με τη μέθοδο `set()` τοποθετούμε τον κινητό δρομέα της κλίμακας στην αρχική θέση-τιμή των 50 Hz.

Ο δρομέας μετακινείται κατά μήκος της κλίμακας με το ποντίκι (`click & drag`) και η μεταβλητή `self.frequency_estimated` παίρνει την τιμή της θέσης που θα σταθεί ο δρομέας.

17.5.5 To widget Button

Οι επόμενες γραμμές του κώδικα δημιουργούν τέσσερα έγχρωμα παραλληλόγραμμα πλαίσια-κουμπιά, διατεταγμένα σε μια σειρά (βλέπε Εικόνα 17.4).

Το πράσινο κουμπί με επιγραφή `Calculate PPV`, όταν το επιλέξει ο χρήστης με το ποντίκι (μονό κλικ), υπολογίζει τη μέγιστη ταχύτητα δόνησης σωματιδίων του εδάφους (`ppv`).

Για τον υπολογισμό της χρησιμοποιούνται ως τιμές των δεδομένων εισόδου είτε οι προεπιλεγμένες τιμές του παράθυρου GUI είτε οι τιμές που πληκτρολόγησε/επέλεξε ο χρήστης.

Το αποτέλεσμα των υπολογισμών θα εμφανιστεί στην περιοχή (section) του παράθυρου GUI κάτω από την επιγραφή `OUTPUT`:

Οι γραμμές κώδικα:

```
# Calculate PPV Button
self.calculate_ppv_button = Button(self.root,
                                   text='Calculate PPV', relief=RIDGE, bg="lightgreen",
                                   font='verdana 10 bold', command=self.calculate_ppv)

self.calculate_ppv_button.grid(row=6, column=0,
                               columnspan=1, padx=5, pady=5, sticky=E)
```

υλοποιούν την εμφάνιση και λειτουργία του πράσινου κουμπιού, με τη χρήση ενός αντικειμένου της Tkinter τύπου `Κουμπί Button()`.

Στο κουμπί, με τη βοήθεια παραμέτρων, έχουμε δώσει τα εξής χαρακτηριστικά: το κείμενο-όνομα (`text`) `Calculate PPV` που θα έχει το κουμπί, το στυλ του περιγράμματος του κουμπιού (`relief`) `RIDGE` (με ράχη), το χρώμα του υπόβαθρου (`bg`) `lightgreen`, τη γραμματοσειρά (`font`) `verdana 10 bold` και τέλος τη συνάρτηση που θα εκτελεστεί μετά από ένα γεγονός (`command`), δηλαδή το κλικ του ποντικιού πάνω στο παραλλόγραμμα του κουμπιού, `self.calculate_ppv`

Ο ορισμός της συνάρτησης που θα εκτελεστεί, δηλαδή το μπλοκ εντολών που έχει επικεφαλίδα:

```
def calculate_ppv(self):
```

πρέπει να βρίσκεται εντός του μπλοκ ορισμού της `class MyGUI`, αλλά εκτός του μπλοκ ορισμού της συνάρτησης- constructor `def __init__(self)`

Το αποτέλεσμα των υπολογισμών που θα προκύψει από την κλήση της συνάρτησης `self.calculate_ppv` θα εκχωρηθεί στη μεταβλητή `self.ppv`, η τιμή της οποίας θα είναι ανάλογη της μεθόδου υπολογισμού της `ppv` που έχει επιλεγεί (`USBM` ή `DIN`), όπως φαίνεται στον κώδικα ορισμού της συνάρτησης.

Η τελευταία γραμμή του κώδικα ορισμού της συνάρτησης `calculate_ppv(self)`:

```
self.ppv_out.set('{:.2f}'.format(self.ppv))
```

με τη βοήθεια της μεθόδου `set()`, αποδίδει την τιμή της `self.ppv` που υπολογίστηκε, και μάλιστα μορφοποιημένη σε δεκαδικό αριθμό με 2 δεκαδικά ψηφία, στη μεταβλητή `self.ppv_out`

Ο κώδικας κάτω από τη γραμμή σχολίου `# Output Section`, αφού δημιουργήσει και εμφανίσει τις ετικέτες `OUTPUT`: και `Peak Particle Velocity - PPV [mm/s]` = στη συνέχεια με τις γραμμές:

```
self.ppv_out = StringVar()
self.display_output = Label(self.root,
                             textvariable=self.ppv_out, bg="whitesmoke",
                             font='verdana 12 bold')
self.display_output.grid(row=8, column=1, columnspan=1,
                        padx=5, pady=5, sticky=W)
```

δημιουργεί τη μεταβλητή `self.ppv_out` στην οποία εκχωρούνται τα χαρακτηριστικά της κλάσης `StringVar()` της Tkinter, και στη συνέχεια η μεταβλητή αυτή αποδίδεται στην παράμετρο `textvariable` του widget τύπου `Label` με όνομα `self.display_output`

Συνοψίζοντας, όταν κλικάρουμε το πράσινο κουμπί 'Calculate PPV' θα κληθεί η συνάρτηση `self.calculate_ppv`, η οποία θα υπολογίσει την `ppv` με βάση τα δεδομένα εισόδου που έχουμε καταχωρήσει στο παράθυρο GUI του προγράμματός μας και θα εκχωρήσει την τιμή της στη μεταβλητή `self.ppv`. Στη συνέχεια, η τιμή της `self.ppv`, με την εφαρμογή της μεθόδου `set()` επί της μεταβλητής `self.ppv_out` κλάσης `StringVar()`, θα εμφανιστεί στη θέση του κελιού που αντιστοιχεί στη γραμμή 8 και στήλη 1, με γραμματοσειρά `verdana 12 bold`, όπως φαίνεται στην Εικόνα 17.5.

The screenshot shows a Tkinter window titled "BLASTppvGUI - PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS". It contains several input fields and buttons. The "INPUT DATA:" section includes fields for "Project Name" (Latomeio XYZ), "Distance from Blast [m]" (300), "Weight of Explosives [kg]" (100), and "Expected Vibration Frequency (Hz)" (49). There are also radio buttons for "PPV Estimation Method" (USBM, DIN), "Type of Blast" (Mine, Quarry, Construction), and "Type of Rock" (Sedimentary, Igneous). Below these are four buttons: "Calculate PPV" (green), "PPV vs Distance Plot" (blue), "KMLE:2011 Compliance Diagram" (yellow), and "Project Report" (red). The "OUTPUT:" section shows "Peak Particle Velocity - PPV [mm/s] = 2.23". The "PLOTS:" section is currently empty.

Εικόνα 17.5. Εμφάνιση του αποτελέσματος υπολογισμού της `ppv` στο παράθυρο GUI.

17.5.6 Τα widgets Canvas & PhotoImage

Το γαλάζιο κουμπί (`bg="lightskyblue"`) με επιγραφή `PPV vs Distance Plot`, το οποίο όταν το επιλέξει ο χρήστης με το ποντίκι σχεδιάζει το διάγραμμα μεταβολής της `ppv` συναρτήσει της απόστασης από τη θέση ανατίναξης, υλοποιείται από τον κώδικα που βρίσκεται κάτω από τη γραμμή σχολίου `# PPV vs Distance Plot Button`.

Η συνάρτηση που καλείται για να σχεδιάσει το διάγραμμα είναι η αναφερόμενη στην παράμετρο `command=self.ppv_vs_distance_plot` και το διάγραμμα εμφανίζεται στον χώρο που έχει προβλεφθεί, με υπόβαθρο λευκού χρώματος, κάτω από την ετικέτα `PLOTS`:

Οι παρακάτω γραμμές κώδικα:

```
self.canvas_area = Canvas(self.root, bg='white',
                           width=600,height=450)
self.canvas_area.grid(row=10, column=0, rowspan=10,
                      colspan=11, sticky=W)
```

υλοποιούν τη δημιουργία ενός *Καμβά* (Canvas), δηλαδή μιας ορθογώνιας περιοχής στο παράθυρο GUI εντός της οποίας μπορούμε να εκτελέσουμε λειτουργίες σχεδίασης.

Αυτό επιτυγχάνεται με τη χρήση ενός αντικειμένου της Tkinter τύπου Canvas (), στο οποίο με τη βοήθεια παραμέτρων έχουμε δώσει τα εξής χαρακτηριστικά: χρώμα υπόβαθρου λευκό (bg='white'), πλάτος 600 pixels (width=600) και ύψος 450 pixels (height=450).

Ο κώδικας της συνάρτησης self.ppv_vs_distance_plot που σχεδιάζει το διάγραμμα είναι παρόμοιος με εκείνον που αναπτύξαμε στο πρόγραμμα BLASTrrn.py και παρουσιάστηκε στην παράγραφο 10.14 Ο κώδικας στο τέλος του Κεφαλαίου 10, περιλαμβάνεται δε στο μπλοκ εντολών που έχει επικεφαλίδα:

```
def ppv_vs_distance_plot(self):
```

Οι τελευταίες γραμμές κώδικα του μπλοκ, που παρατίθενται για διευκόλυνση στη συνέχεια, εμφανίζουν το διάγραμμα στην περιοχή του παραθύρου GUI που προσδιορίστηκε από το αντικείμενο Canvas () :

```
self.root.update()
if self.ppv_method.get() == 'USBM':
    fig1.savefig('ppv_USBM_vs_distance_GUI.png')
    self.ppv_vs_distance_plot = PhotoImage(file=
                                             'ppv_USBM_vs_distance_GUI.png')
    self.canvas_area.create_image(0, 0,
                                  image=self.ppv_vs_distance_plot, anchor=NW)
elif self.ppv_method.get() == 'DIN':
    fig1.savefig('ppv_DIN_vs_distance_GUI.png')
    self.ppv_vs_distance_plot = PhotoImage(file=
                                             'ppv_DIN_vs_distance_GUI.png')
    self.canvas_area.create_image(0, 0,
                                  image=self.ppv_vs_distance_plot, anchor=NW)
fig1.clear()
```

Με την κλήση, καταρχάς, της μεθόδου update () ανανεώνεται η γραφική παράσταση που ενδεχομένως να υπάρχει στον καμβά, από προηγούμενη κλήση της συνάρτησης self.ppv_vs_distance_plot ή της συνάρτησης self.ppv_compliance_KMLE_plot, η οποία σχεδιάζει το διάγραμμα συμμόρφωσης με τον ΚΜΛΕ (βλέπε στη συνέχεια αυτής της παραγράφου) και χρησιμοποιεί τον ίδιο καμβά για να εμφανίσει το διάγραμμα αυτό στο παράθυρο GUI.

Στη συνέχεια, το διάγραμμα, το οποίο κατασκευάστηκε για τη μεταβολή της ppv, που υπολογίστηκε σύμφωνα με τη μέθοδο του USBM, συναρτήσει της απόστασης, αποθηκεύεται σε ένα αρχείο με όνομα ppv_USBM_vs_distance_GUI.png με την εντολή:

```
fig1.savefig('ppv_USBM_vs_distance_GUI.png')
```

Η εντολή:

```
self.ppv_vs_distance_plot = PhotoImage(file=
                                         'ppv_USBM_vs_distance_GUI.png')
```

με τη χρήση ενός αντικειμένου της Tkinter τύπου PhotoImage (), δημιουργεί ένα αντικείμενο *Φωτογραφίας* του αρχείου ppv_USBM_vs_distance_GUI.png

Στη συνέχεια, εμφανίζουμε τη φωτογραφία, με την εντολή:

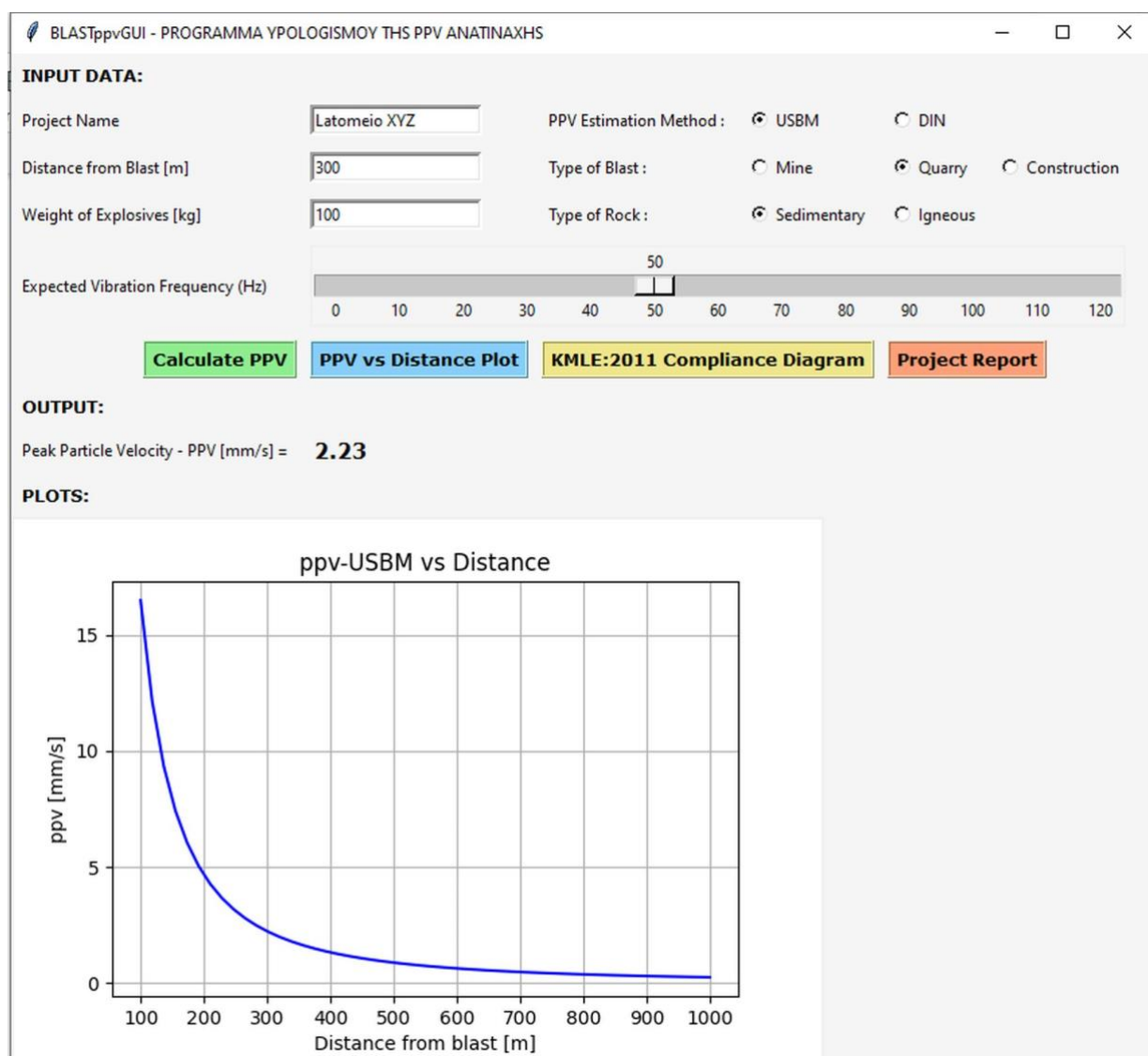
```
self.canvas_area.create_image(0, 0,
                              image=self.ppv_vs_distance_plot, anchor=NW)
```

εφαρμόζοντας τη μέθοδο `create_image()` επί του αντικειμένου καμβά `self.canvas_area` που δημιουργήσαμε προηγούμενα, με τις εξής παραμέτρους: (x,y) 0,0 δηλαδή η φωτογραφία θα τοποθετηθεί στην αρχή των αξόνων του καμβά, η εικόνα που θα εμφανιστεί (`image`) θα είναι εκείνη που έχει εκχωρηθεί στο αντικείμενο φωτογραφίας `self.ppv_vs_distance_plot` και θα *αγκυροβολήσει* στην περιοχή του καμβά έτσι ώστε το σημείο (x, y) να βρίσκεται στο βορειοδυτικό άκρο του καμβά (`anchor=NW`).

Το αποτέλεσμα εμφανίζεται το παράθυρο GUI της Εικόνας 17.6.

Κατ' ανάλογο τρόπο σχεδιάζεται και παρουσιάζεται στο παράθυρο GUI το αντίστοιχο διάγραμμα μεταβολής της *ppv*, που υπολογίζεται σύμφωνα με το πρότυπο DIN, συναρτήσει της απόστασης, το οποίο αποθηκεύεται σε ένα αρχείο με όνομα `ppv_DIN_vs_distance_GUI.png`

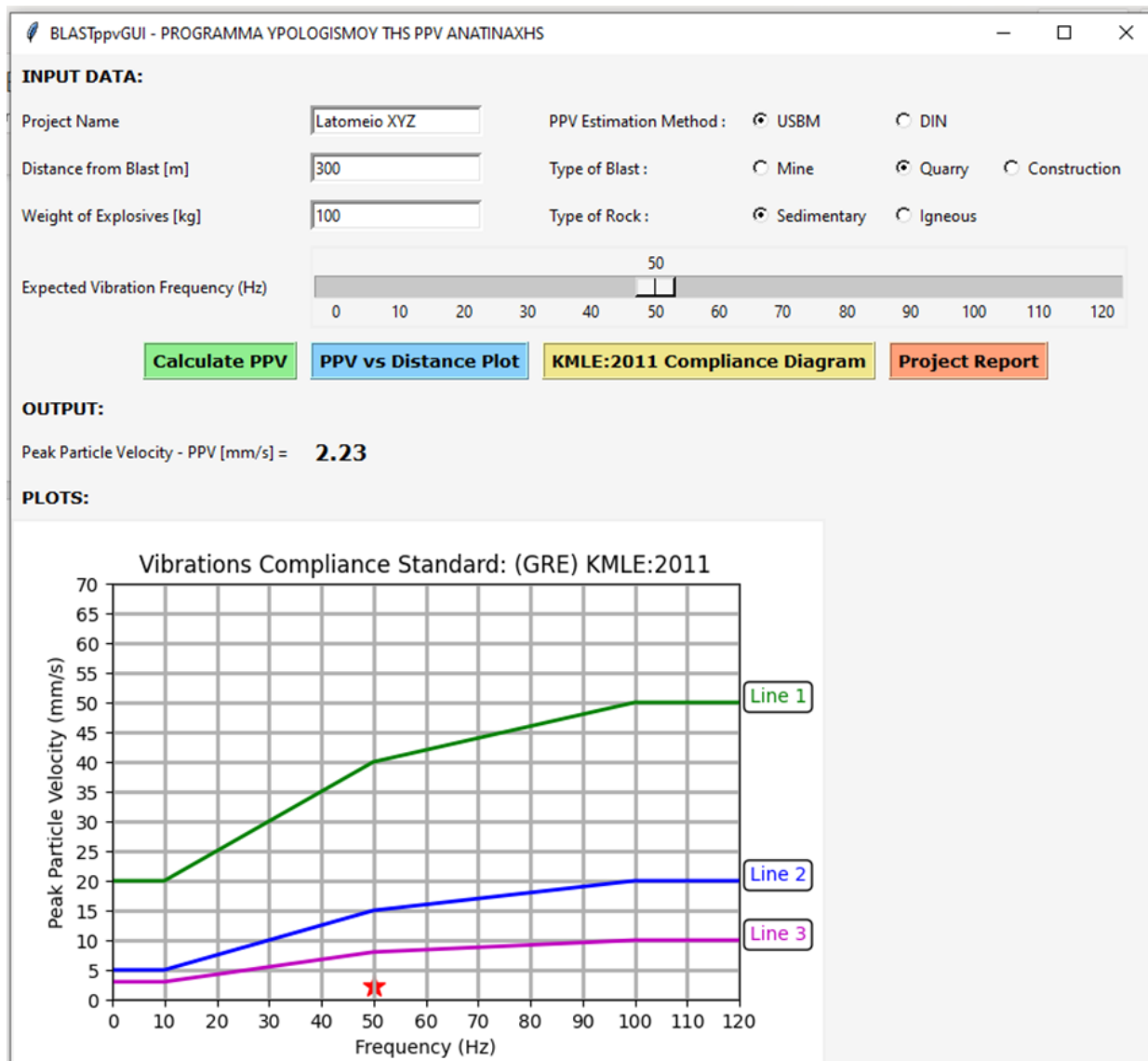
Η τελευταία εντολή `fig1.clear()` *αδειάζει* το περιεχόμενο της μνήμης της μεταβλητής `fig1` η οποία περιέχει τα στοιχεία του γραφήματος που δημιουργήθηκε με τη βοήθεια συναρτήσεων και μεθόδων της βιβλιοθήκης Matplotlib.



Εικόνα 17.6. Εμφάνιση του διαγράμματος μεταβολής της *ppv* συναρτήσει της απόστασης στο παράθυρο GUI.

Το *κίτρινο κουμπί* (`bg="khaki"`) με επιγραφή `KMLE:2011 Compliance Diagram`, το οποίο όταν το επιλέξει ο χρήστης με το ποντίκι σχεδιάζει το διάγραμμα συμμόρφωσης της *ppv*/συχνότητας με τα προβλεπόμενα από τον ΚΜΛΕ, υλοποιείται από τον κώδικα που βρίσκεται κάτω από τη γραμμή σχολίου `# KMLE:2011 Compliance Diagram Button`.

Η συνάρτηση που καλείται για να σχεδιάσει το διάγραμμα είναι η αναφερόμενη στην παράμετρο `command=self.ppv_compliance_KMLE_plot` και το διάγραμμα, αφού αποθηκευτεί στο αρχείο `KMLE_compliance_plot_GUI.png`, εμφανίζεται στην περιοχή των PLOTS (Εικόνα 17.7).



Εικόνα 17.7. Εμφάνιση του διαγράμματος συμμόρφωσης της ppv/συχνότητας με τον ΚΜΛΕ στο παράθυρο GUI.

Ο κώδικας της συνάρτησης `self.ppv_compliance_KMLE_plot` είναι παρόμοιος με εκείνον που αναπτύξαμε για τον ίδιο σκοπό στο πρόγραμμα `BLASTppv.py` και παρουσιάστηκε στην παράγραφο 10.14 Ο κώδικας στο τέλος του Κεφαλαίου 10.

Συνοψίζοντας, παρατηρούμε ότι τρία (3) διαφορετικά διαγράμματα τα οποία δημιουργούνται από το πρόγραμμά μας και αποθηκεύονται στα αρχεία εικόνας:

`ppv_USBm_vs_distance_GUI.png`

`ppv_DIN_vs_distance_GUI.png`

`KMLE_compliance_plot_GUI.png`

εμφανίζονται στον ίδιο καμβά-περιοχή του παραθύρου GUI εναλλάξ, ανάλογα με το ποιο κουμπί (γαλάζιο ή κίτρινο) και ποια επιλογή (USBM ή DIN) έχουμε κλικάρει με το ποντίκι.

17.5.7 To widget `simpdialog.askfloat`

Εάν ανατρέξουμε στον κώδικα του προγράμματος `BLASTppv.py` στην παράγραφο 10.14 Ο κώδικας στο τέλος του Κεφαλαίου 10, θα διαπιστώσουμε ότι στις εντολές `input()`, με τις οποίες εισάγουμε από το πληκτρολόγιο τα δεδομένα εισόδου, περιλαμβάνονται και οι τιμές των μεταβλητών `plot_start` και `plot_end` που αντιστοιχούν στην αρχική και τελική τιμή της απόστασης στον οριζόντιο άξονα του διαγράμματος μεταβολής της `ppv` συναρτήσεως της απόστασης.

Στον κώδικα του προγράμματος `BLASTppvGUI.py` οι παρακάτω εντολές, που βρίσκονται στην αρχή του μπλοκ ορισμού της συνάρτησης `ppv_vs_distance_plot()`:

```
self.end_value = simpdialog.askfloat("PPV vs Distance Plot Values",
                                     "Maximum Plot Distance [m]",
                                     minvalue=200.0, maxvalue=10000.0, parent=self.root)
if self.end_value is int or float:
    plot_start = 100
    plot_end = self.end_value
if self.end_value is None:
    return
```

υλοποιούν την εισαγωγή από το πληκτρολόγιο της τελικής τιμής της απόστασης (`self.plot_end`) με τη βοήθεια της συνάρτησης `askfloat()` του module `simpdialog` της βιβλιοθήκης Tkinter.

Για τον λόγο αυτό, στην αρχή του προγράμματός μας πρέπει να κάνουμε `import` το module με τη γραμμή κώδικα:

```
from tkinter import simpdialog
```

Για την τεκμηρίωση του module των διαλόγων της Tkinter ανατρέξτε στο website: <https://docs.python.org/3/library/dialog.html>

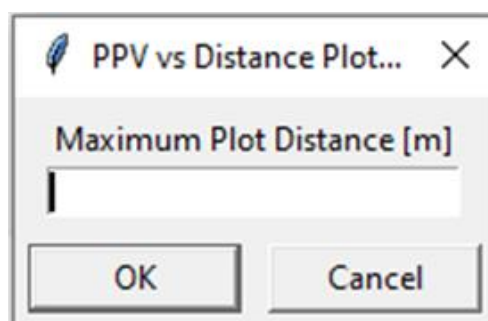
Η κλήση της συνάρτησης `askfloat()` γίνεται με τη γραμμή κώδικα:

```
self.end_value = simpdialog.askfloat("PPV vs Distance Plot Values",
                                     "Maximum Plot Distance [m]",
                                     minvalue=200.0, maxvalue=10000.0, parent=self.root)
```

η οποία εμφανίζει ένα τυποποιημένο αναδυόμενο (pop-up) μικρο-παράθυρο εντός του παράθυρου GUI του προγράμματός μας (`parent=self.root`), το οποίο έχει τίτλο `PPV vs Distance Plot Values` και ενσωματωμένα τα κουμπιά `OK` και `Cancel` (Εικόνα 17.8).

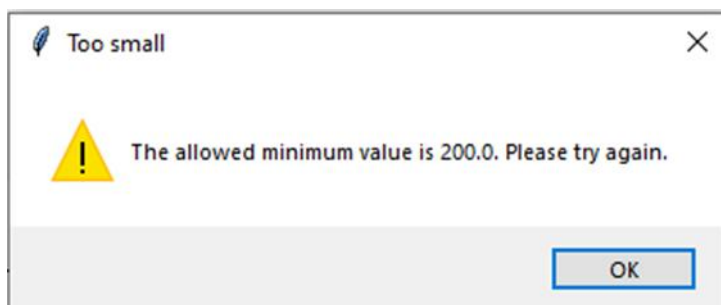
Στο σώμα του παράθυρου υπάρχει η ετικέτα `Maximum Plot Distance [m]` και κάτω από αυτήν το πλαίσιο εντός widget τύπου `Entry`, έτοιμο (ο cursor του ποντικιού είναι ήδη τοποθετημένος στην αρχή του πλαισίου) για να δεχθεί την τιμή που θα πληκτρολογήσουμε, η οποία πρέπει να είναι δεκαδικής μορφής (float).

Στις παραμέτρους της συνάρτησης έχουμε ορίσει ως ελάχιστη τιμή τα 200 m (`minvalue=200.0`) και μέγιστη τιμή τα 10000 m (`maxvalue=10000.0`).



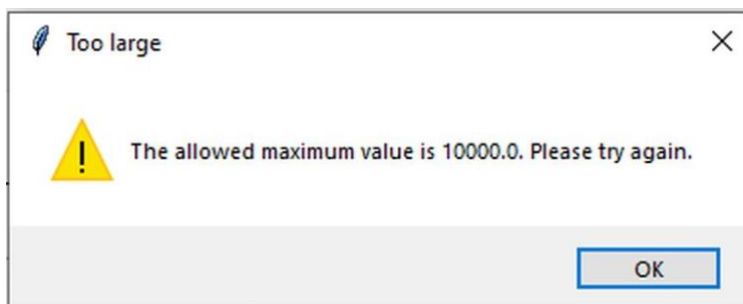
Εικόνα 17.8. Εμφάνιση, εντός του παράθυρου GUI του προγράμματος, του αναδυόμενου παράθυρου για την πληκτρολόγηση της τιμής της μεταβλητής `self.plot_end`

Εάν ο χρήστης πληκτρολογήσει τιμή μικρότερη του 200 και πατήσει το κουμπί OK, τότε θα εμφανιστεί το αναδυόμενο παράθυρο με τίτλο: *Too small* και το μήνυμα της Εικόνας 17.9, προτρέποντας τον χρήστη να προσπαθήσει ξανά να πληκτρολογήσει τιμή μεγαλύτερη ή ίση του 200.



Εικόνα 17.9. Αναδυόμενο παράθυρο προτροπής πληκτρολόγησης τιμής μεγαλύτερης του 200.

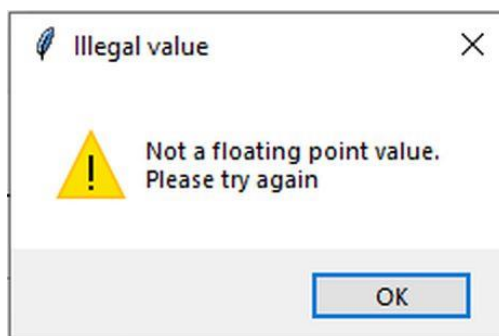
Εάν ο χρήστης πληκτρολογήσει τιμή μεγαλύτερη του 10000 και πατήσει το κουμπί OK, τότε θα εμφανιστεί το αναδυόμενο παράθυρο με τίτλο *Too large* και το μήνυμα της Εικόνας 17.10, προτρέποντας τον χρήστη να προσπαθήσει ξανά να πληκτρολογήσει τιμή μικρότερη ή ίση του 10000.



Εικόνα 17.10. Αναδυόμενο παράθυρο προτροπής πληκτρολόγησης τιμής μικρότερης του 10000.

Σημειώστε ότι η συνάρτηση `askfloat()` δέχεται την πληκτρολόγηση και ακέραιων τιμών π.χ. 300 αντί 300.0 τις οποίες στη συνέχεια μετατρέπει σε δεκαδική μορφή.

Τέλος, εάν ο χρήστης πληκτρολογήσει, από απροσεξία, αντί για αριθμό μια συμβολοσειρά π.χ. !000 αντί για 1000 και πατήσει το κουμπί OK, τότε θα εμφανιστεί το αναδυόμενο παράθυρο με τίτλο *Illegal value* και το μήνυμα της Εικόνας 17.11, προτρέποντας τον χρήστη να προσπαθήσει ξανά να πληκτρολογήσει μια αποδεκτή τιμή.



Εικόνα 17.11. Αναδυόμενο παράθυρο προτροπής πληκτρολόγησης μιας δεκαδικής τιμής.

Σε κάθε περίπτωση, το αρχικό παράθυρο που θα δεχθεί την τιμή της μεταβλητής `self.plot_end` (Εικόνα 17.8) παραμένει ανοικτό στο παρασκήνιο και καθίσταται πάλι ενεργό μόλις πατηθεί το κουμπί OK ή το κουμπί Close του παραθύρου προτροπής για τη διόρθωση του λάθους.

Επίσης, σημειώστε ότι στο παράθυρο της Εικόνας 17.8, το πάτημα του κουμπιού OK είναι ισοδύναμο με το πάτημα του ENTER του πληκτρολογίου και το πάτημα του κουμπιού Cancel είναι ισοδύναμο με το πάτημα του κουμπιού Close του παραθύρου.

Εάν ο χρήστης πατήσει το κουμπί Cancel, τότε η μεταβλητή `self.plot_end` λαμβάνει την τιμή `None` και σύμφωνα με τον κώδικά μας, η λειτουργία της συνάρτησης `self.ppv_vs_distance_plot` με την εντολή `return` τερματίζεται.

Το module `simpdialog` της βιβλιοθήκης Tkinter, πέραν της συνάρτησης `askfloat()`, που χρησιμοποιήσαμε στον κώδικά μας, περιλαμβάνει και τις συναρτήσεις:

`simpdialog.askinteger()` η οποία δημιουργεί ένα αναδυόμενο παράθυρο διαλόγου για την πληκτρολόγηση της τιμής μιας ακεραίας μεταβλητής και

`simpdialog.askstring()` η οποία δημιουργεί ένα αναδυόμενο παράθυρο διαλόγου για την πληκτρολόγηση της τιμής μιας μεταβλητής τύπου συμβολοσειράς,

οι οποίες έχουν την ίδια δομή με εκείνη της `simpdialog.askfloat()`

17.5.8 Η βιβλιοθήκη `fpdf`

Το *σομόν κουμπί* (`bg="lightsalmon"`) με επιγραφή `Project Report`, το οποίο όταν το επιλέξει ο χρήστης με το ποντίκι δημιουργεί μια *αναφορά* (report) που περιλαμβάνει τα δεδομένα εισόδου, το αποτέλεσμα και τα διαγράμματα μιας εφαρμογής (τρεξίματος) του προγράμματός μας, υλοποιείται από τον κώδικα που βρίσκεται κάτω από τη γραμμή σχολίου `# Project Report Button`.

Η συνάρτηση που καλείται για να δημιουργήσει την αναφορά είναι η αναφερόμενη στην παράμετρο `command=self.project_report` και ο κώδικας ορισμού της παρατίθεται στο μπλοκ εντολών που έχει επικεφαλίδα:

```
def project_report(self):
```

Το πρώτο μέρος των εντολών του μπλοκ, με μια σειρά εντολών `print()` δημιουργεί το περιεχόμενο της αναφοράς και το αποθηκεύει σε ένα αρχείο κειμένου (`.txt`) με όνομα `project_report_GUI.txt` κατά τρόπο ανάλογο με εκείνον που είδαμε στο πρόγραμμα `Chapter_6_BLASTppv.py` και παρουσιάσαμε στην παράγραφο 06.5 Ο κώδικας στο τέλος του κεφαλαίου 06.

Το δεύτερο μέρος των εντολών του μπλοκ μετατρέπει το αρχείο `.txt` σε αρχείο τύπου `.pdf` και επιπλέον προσθέτει σε αυτό τα διαγράμματα που τυχόν επέλεξε ο χρήστης του προγράμματος να δημιουργήσει. Τέλος, ανοίγει το αρχείο `.pdf`, με τον default pdf reader που έχει εγκατεστημένο στον υπολογιστή του ο χρήστης, για ανάγνωση και αρχειοθέτηση.

Οι γραμμές των εντολών του δεύτερου μέρους του κώδικα ορισμού της συνάρτησης `self.project_report` είναι:

```
report = FPDF() # FPDF('P', 'mm', 'A4')
# Add a page
report.add_page()
report.set_font("Courier", "", size=11)
# open the text file in read mode
f = open("project_report_GUI.txt", "r")
# insert lines of text in pdf
for x in f:
    report.cell(200, 7, txt=x, ln=1, align='L')
report.ln(h='10')
# insert images
if self.ppv_method.get() == 'USBM' and \
    os.path.isfile('ppv_USBm_vs_distance_GUI.png'):
    report.image('ppv_USBm_vs_distance_GUI.png',
                w=120, h=75, type='PNG')
elif self.ppv_method.get() == 'DIN' and \
    os.path.isfile('ppv_DIN_vs_distance_GUI.png'):
    report.image('ppv_DIN_vs_distance_GUI.png',
                w=120, h=75, type='PNG')
```

```

report.ln(h='')
if os.path.isfile('KMLE_compliance_plot_GUI.png'):
    report.image('KMLE_compliance_plot_GUI.png',
                 w=120, h=75, type='PNG')
# save the pdf with name .pdf
report.output("project_report_GUI.pdf")
# read .pdf with default reader
try:
    os.startfile("project_report_GUI.pdf")
except OSError:
    pass

```

Για να δημιουργήσουμε αρχεία pdf μέσα από τον κώδικα Python, θα χρησιμοποιήσουμε την εξωτερική βιβλιοθήκη της fpdf, την οποία και πρέπει να εγκαταστήσουμε στον υπολογιστή μας κατά τα γνωστά:

```
pip install fpdf
```

και στη συνέχεια να την κάνουμε import με τη γραμμή κώδικα:

```
from fpdf import FPDF
```

Για την τεκμηρίωση της βιβλιοθήκης fpdf ανατρέξτε στο website:

<https://pypi.org/project/fpdf>

Με την πρώτη γραμμή του κώδικα καλούμε τον constructor FPDF της βιβλιοθήκης fpdf για να δημιουργήσει ένα αντικείμενο με όνομα report, δηλαδή ένα έγγραφο pdf με τις default τιμές του constructor που είναι: οι σελίδες του εγγράφου να έχουν προσανατολισμό P (Portrait), ως μονάδα μέτρησης να χρησιμοποιηθούν τα mm και το μέγεθος της σελίδας να είναι A4.

Τις τιμές των παραπάνω παραμέτρων θα μπορούσαμε να τις ορίσουμε και ρητά, αντικαθιστώντας την πρώτη γραμμή του κώδικα με την αντίστοιχη της:

```
report = FPDF('P', 'mm', 'A4')
```

ή και να τις τροποποιήσουμε, π.χ. επιλέγοντας προσανατολισμό L (Landscape), μονάδα μέτρησης τις in (inches) και μέγεθος σελίδας A3.

Η επόμενη εντολή προσθέτει στο αντικείμενο-έγγραφο μια σελίδα με τα χαρακτηριστικά που έχουμε επιλέξει, ενώ η αρχή του κειμένου στη σελίδα θα τοποθετηθεί στην άνω-αριστερή γωνία και στην default θέση 10 mm από τα όρια της σελίδας A4.

Η εντολή που ακολουθεί, και πρέπει υποχρεωτικά να προηγείται της προσθήκης κειμένου στη σελίδα του εγγράφου:

```
report.set_font("Courier", "", size=11)
```

δηλώνει τη γραμματοσειρά που θα χρησιμοποιήσουμε, η οποία θα είναι Courier, κανονικής γραφής (""), και μεγέθους 11 points (pts). Σημειώστε ότι ανεξάρτητα από τη μονάδα μέτρησης που έχει επιλεγεί για το έγγραφο, το μέγεθος της γραμματοσειράς μετριέται πάντα σε pts.

Ο χρήστης πέρα από τη γραμματοσειρά της αρεσκείας του, π.χ. Arial ή Times, μπορεί να επιλέξει γραφή **Bold** ("B"), *Italics* ("I") ή Underlined ("U").

Επειδή το κείμενο που θα ενσωματώσουμε στο αρχείο pdf θα το αντλήσουμε από το αρχείο project_report_GUI.txt, πρέπει να το ανοίξουμε σε κατάσταση ανάγνωσης με την εντολή:

```
f = open("project_report_GUI.txt", "r")
```

Στη συνέχεια, με τη βοήθεια ενός βρόχου for, που εφαρμόζεται σε καθεμία γραμμή x του αρχείου project_report_GUI.txt, εκτελούμε την εντολή:

```
report.cell(200, 7, txt=x, ln=1, align='L')
```

με την οποία δημιουργούμε ένα κελί (`cell`), που είναι μια παραλληλόγραμμη περιοχή υποδοχής κειμένου, διαστάσεων: πλάτους 200 mm και ύψους 7 mm, στην οποία θα ενταχθεί το κείμενο της γραμμής `x` του αρχείου (`txt=x`).

Η θέση για την ένταξη της επόμενης γραμμής `x` του αρχείου, θα είναι η αρχή της επόμενης γραμμής του κελιού (`ln=1`), ενώ η στοίχιση του κειμένου θα είναι το αριστερό όριο του κελιού (`align='L'`).

Μετά την *ένταξη-εκτύπωση* στο κελί όλων των γραμμών του αρχείου-κειμένου της αναφοράς των αποτελεσμάτων του προγράμματος, με την εντολή:

```
report.ln(h='10')
```

εισάγουμε ένα κενό (line break) ύψους 10 mm. Η default τιμή (`h=' '`) είναι ίδια με το ύψος του τελευταίου κελιού που εκτυπώθηκε, δηλαδή στην περίπτωσή μας τα 7 mm.

Εάν έχουμε κλικάρει με το ποντίκι τα κουμπιά που δημιουργούν τα διαγράμματα του προγράμματός μας και υπάρχουν στο directory που βρίσκεται το αρχείο `BLASTppvGUI.py` ένα ή περισσότερα εκ των αρχείων:

```
ppv_USBM_vs_distance_GUI.png
```

```
ppv_DIN_vs_distance_GUI.png
```

```
KMLE_compliance_plot_GUI.png
```

η γραμμή κώδικα:

```
report.image('ppv_USBM_vs_distance_GUI.png',  
             w=120, h=75, type='PNG')
```

θα ενσωματώσει στο έγγραφο `report` την εικόνα που υπάρχει στο αρχείο `ppv_USBM_vs_distance_GUI.png`, η οποία θα έχει διαστάσεις: πλάτος 120 mm (`w=120`) και ύψος 75 mm (`h=75`), δηλώνουμε δε ότι το format της εικόνας είναι `.png` (`type='PNG'`).

Ανάλογες είναι οι γραμμές κώδικα για την εμφάνιση και των άλλων αρχείων φωτογραφίας, εάν έχουν δημιουργηθεί από τον χρήστη του προγράμματος.

Στη συνέχεια, οι γραμμές κώδικα:

```
report.output("project_report_GUI.pdf").encode('latin-1')  
# read .pdf with default reader  
try:  
    os.startfile("project_report_GUI.pdf")  
except OSError:  
    pass
```

η μεν πρώτη στέλνει-δημοσιεύει το έγγραφο pdf που δημιουργήσαμε στο αρχείο με όνομα `project_report_GUI.pdf`, ο δε βρόχος `try-except` που ακολουθεί, ανοίγει το pdf αρχείο με τον default pdf reader του υπολογιστή μας, π.χ. τον Adobe Acrobat Reader DC (Εικόνα 17.12).

Σημειώστε ότι η μέθοδος `report.output()` αυτόματα καλεί τη μέθοδο `report.close()` για να κλείσει-τερματίσει τη διαδικασία δημιουργίας του αρχείου pdf, πριν τη δημοσίευσή του.

Παρατήρηση: Όταν χρησιμοποιούμε τη βιβλιοθήκη `fpdf`, για να διαβαστεί σωστά από τον pdf reader του υπολογιστή μας το αρχείο pdf που δημιουργήσαμε, πρέπει μαζί με τη μέθοδο `report.output()` να εφαρμοστεί και η μέθοδος `encode('latin-1')`.

Σύντομες, αλλά περιεκτικές, οδηγίες χρήσης της βιβλιοθήκης `fpdf` θα βρείτε στο website: <https://pyfpdf.readthedocs.io/en/latest/Tutorial/index.html>

PROGRAMMA YPOLOGISMOY THS PPV ANATINAXHS

Project Name : Latomeio XYZ

INPUT DATA:

Peak Particle Velocity Estimation Method : USBM

Type of Blast : Quarry

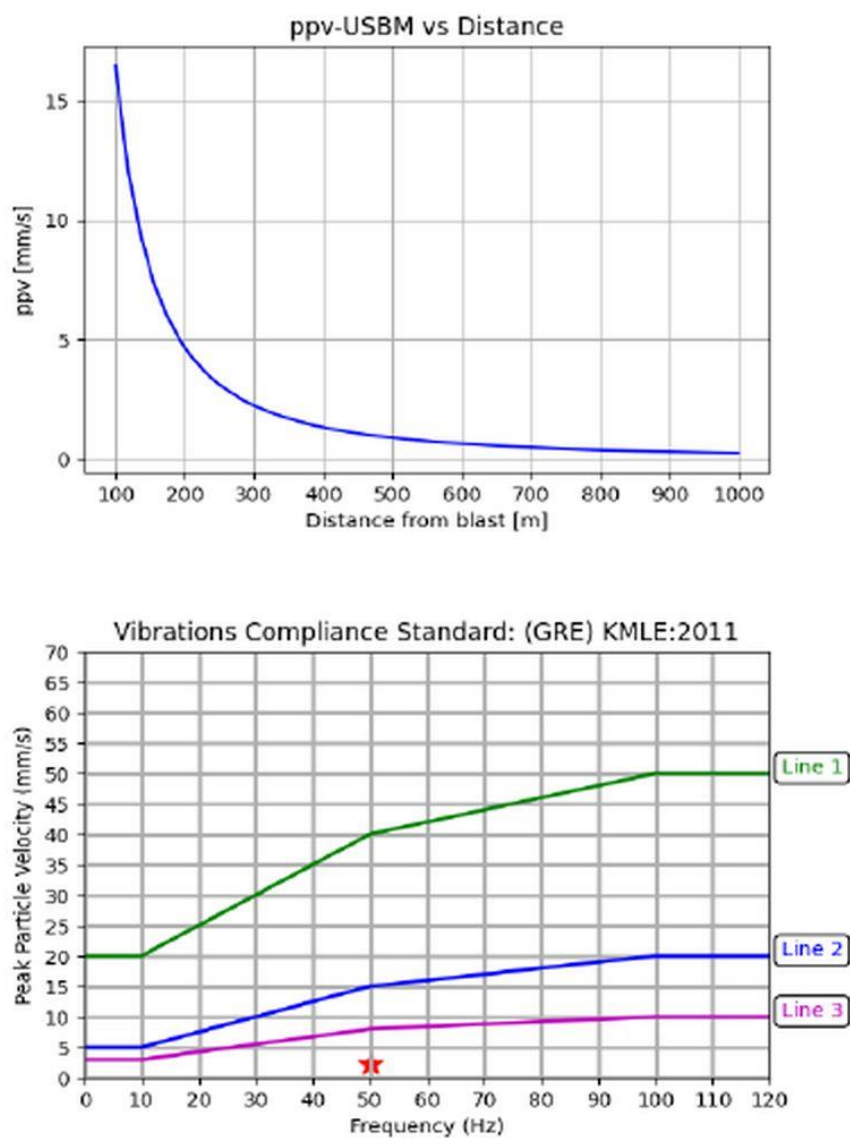
Distance from Blast [m] : 300.0

Weight of Explosives [kg] : 100.0

Estimated ppv frequency [Hz] : 50.0

OUTPUT:

Peak Particle Velocity (USBM) [mm/s] = 2.23



Εικόνα 17.12. Εμφάνιση στον Adobe Acrobat Reader DC του εγγράφου των αποτελεσμάτων του προγράμματος BLASTppvGUI.py (αρχείο project_report_GUI.pdf).

17.5.9 Η βιβλιοθήκη `os`

Η βιβλιοθήκη `os` (Miscellaneous operating system interfaces) της Python προσφέρει συναρτήσεις και μεθόδους που σχετίζονται με το λειτουργικό σύστημα του υπολογιστή μας.

Ως εσωτερική βιβλιοθήκη της Python δεν χρειάζεται εγκατάσταση, αλλά πρέπει να την κάνουμε `import` στον κώδικά μας:

```
import os
```

Για την τεκμηρίωση της βιβλιοθήκης ανατρέξτε στο `website`:

<https://docs.python.org/3/library/os.html>

Στην προηγούμενη παράγραφο 17.5.8 *Η βιβλιοθήκη `fpdf`* και μέσα στον βρόχο `try-except` χρησιμοποιήσαμε τη μέθοδο της βιβλιοθήκης `os`:

```
os.startfile("project_report_GUI.pdf")
```

με την οποία ανοίγουμε το αρχείο `project_report_GUI.pdf` με το πρόγραμμα του υπολογιστή μας με το οποίο *συνδέεται λειτουργικά* το αρχείο, λόγω της επέκτασης του ονόματός του.

Για παράδειγμα, στον υπολογιστή του συγγραφέα του βιβλίου, η μέθοδος `os.startfile()` ανοίγει τα αρχεία που έχουν επέκταση `.pdf` με το εγκατεστημένο πρόγραμμα Adobe Acrobat Reader DC.

Εάν δεν υπάρχει διαθέσιμος pdf reader στον υπολογιστή μας (μάλλον απίθανο...), τότε θα εγερθεί το `exception OSError` και δεν θα ανοίξει το αρχείο.

Στο πρόγραμμα `BLASTppvGUI.py` χειριζόμαστε μια σειρά από αρχεία με επεκτάσεις `.png`, `.txt` και `.pdf`, το περιεχόμενο των οποίων ανανεώνεται κάθε φορά που αλλάζουμε τις τιμές ή τις επιλογές μας στο παράθυρο GUI του προγράμματος.

Στην αρχή του μπλοκ των εντολών της κλάσης `MyGUI` με τη γραμμή κώδικα:

```
self.remove_files()
```

καλούμε τη συνάρτηση `self.remove_files`, ο κώδικας της οποίας περιλαμβάνεται στο μπλοκ εντολών που έχει επικεφαλίδα:

```
def remove_files(self):
```

Σκοπός της συνάρτησης αυτής είναι να ελέγξει εάν υπάρχουν αποθηκευμένα στον υπολογιστή μας συγκεκριμένα αρχεία, που δημιουργήθηκαν από προηγούμενη χρήση του προγράμματός μας, και εάν ναι, να τα διαγράψει.

Τα αρχεία που ελέγχονται είναι τα:

```
ppv_USBM_vs_distance_GUI.png
```

```
ppv_DIN_vs_distance_GUI.png
```

```
KMLE_compliance_plot_GUI.png
```

```
project_report_GUI.txt
```

```
project_report_GUI.pdf
```

Έτσι με τις γραμμές κώδικα:

```
if os.path.isfile('ppv_USBM_vs_distance_GUI.png') :  
    os.remove('ppv_USBM_vs_distance_GUI.png')
```

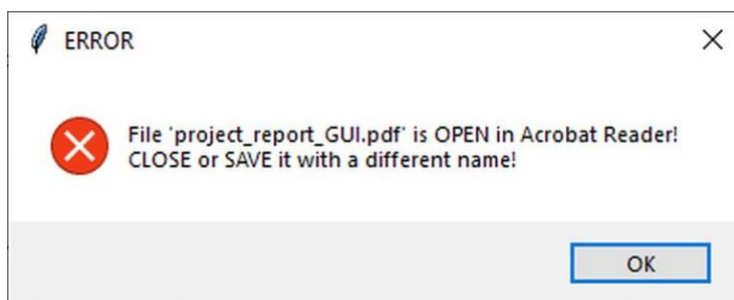

η μέθοδος `os.path.isfile()` ελέγχει εάν το αρχείο `ppv_USBm_vs_distance_GUI.png` υπάρχει στο τρέχον directory, όπου βρίσκεται και το πρόγραμμα `BLASTppvGUI.py` και εάν το αποτέλεσμα του ελέγχου είναι `True`, τότε η μέθοδος `os.remove()` διαγράφει αυτό το αρχείο από το directory.

Ειδικά για το αρχείο `project_report_GUI.pdf`, ο έλεγχος και η διαγραφή του γίνονται με τις παρακάτω γραμμές κώδικα:

```
if os.path.isfile('project_report_GUI.pdf'):
    try:
        os.remove('project_report_GUI.pdf')
    except OSError:
        messagebox.showerror("ERROR", "File "
                               "'project_report_GUI.pdf' is OPEN in "
                               "Acrobat Reader! CLOSE or SAVE it with a "
                               "different name!")
```

επειδή είναι πιθανόν να έχουμε ξεχάσει ανοικτό το αρχείο στον pdf reader, από προηγούμενο άνοιγμα ή τρέξιμο του προγράμματος `BLASTppvGUI.py`

Εάν συμβαίνει αυτό, τότε εμφανίζεται το μήνυμα της Εικόνας 17.13 σε ένα αναδυόμενο παράθυρο, όπου μας δίνονται οι οδηγίες να *Κλείσουμε* (CLOSE) ή να *Σώσουμε* (SAVE) το αρχείο με διαφορετικό όνομα.



Εικόνα 17.13. Εμφάνιση μηνύματος λάθους, επειδή το αρχείο `project_report_GUI.pdf` είναι ανοικτό στον Acrobat Reader.

Οι ίδιες γραμμές κώδικα, όπως παραπάνω, υπάρχουν και στο μπλοκ εντολών ορισμού της συνάρτησης `project_report(self)` γιατί, εάν ο χρήστης έχει ανοικτό το αρχείο `project_report_GUI.pdf` και ξανακάνει, μετά από αλλαγή δεδομένων ή επιλογών, κλικ στο σομόν κουμπί με επιγραφή `Project Report`, τότε το πρόγραμμα θα *κρασάρει* επειδή η μέθοδος:

```
report.output("project_report_GUI.pdf").encode('latin-1')
```

δεν μπορεί να στείλει-δημοσιεύει το έγγραφο pdf που δημιουργήσαμε με τα νέα δεδομένα, σε ένα αρχείο με το ίδιο όνομα που είναι ήδη ανοιγμένο από τον pdf reader.

Σημειώστε ότι όλες οι συναρτήσεις της βιβλιοθήκης `os` εγείρουν `OSError`, εάν τα ονόματα των αρχείων δεν είναι σωστά ή τα αρχεία δεν είναι προσβάσιμα ή δεν γίνονται δεκτά από το λειτουργικό σύστημα του υπολογιστή μας.

17.5.10 To widget `messagebox.showerror`

Το widget `messagebox.showerror()` περιλαμβάνεται στις συναρτήσεις του module `messagebox` της βιβλιοθήκης `Tkinter`, το οποίο και πρέπει να κάνουμε `import` με τη γραμμή κώδικα

```
from tkinter import messagebox
```

Για την τεκμηρίωση του module ανατρέξτε στο website:

<https://docs.python.org/3/library/tkinter.messagebox.html>

Η κλήση του widget γίνεται με τη γραμμή κώδικα:

```
messagebox.showerror("ERROR", "File "
    "'project_report_GUI.pdf' is OPEN in "
    "Acrobat Reader! CLOSE or SAVE it with a "
    "different name!")
```

όπου η πρώτη παράμετρος είναι η συμβολοσειρά του τίτλου του αναδυόμενου παράθυρου και η δεύτερη είναι η συμβολοσειρά του μηνύματος που θέλουμε να εμφανιστεί (Εικόνα 17.13).

Το module εκτός από το `showerror()` περιλαμβάνει και άλλα τυποποιημένα αναδυόμενα παράθυρα μηνυμάτων, μεταξύ των οποίων είναι τα `showwarning()`, `showinfo()`, `askquestion()` κ.ά.

17.6 Επιλογή χρωμάτων με τη βοήθεια της Tkinter

Στα προγράμματα Python που έχουμε παρουσιάσει μέχρι τώρα στο βιβλίο, όπου χρειάστηκε να χρωματίσουμε γραμμές, επιφάνειες, συμβολοσειρές κ.ά. χρησιμοποιήσαμε τον λεκτικό χαρακτηρισμό του χρώματος, π.χ. `"r"` ή `"red"` για το κόκκινο χρώμα ή `"whitesmoke"` για μια απόχρωση του γρι, ακολουθώντας την ονοματολογία των χρωμάτων που περιλαμβάνονται στα χρωματολόγια των Εικόνων 7.3 & 7.4 της παραγράφου 07.5.1 *Χρώματα*.

Η Python, όμως, δεν περιορίζει τις χρωματικές μας επιλογές σε αυτά μόνο τα χρωματολόγια, αλλά μας δίνει τη δυνατότητα να επιλέξουμε και να χρησιμοποιήσουμε χρώματα και αποχρώσεις αυτών από μια πρακτικά ατελείωτη βεντάλια χρωμάτων.

Επειδή αυτές τις χρωματικές επιλογές δεν είναι πρακτικά δυνατό να τις δηλώνουμε με όνομα, χρησιμοποιούμε εναλλακτικά είτε τον κωδικό RGB (RGB code) [Σχετικά με χρωματικό σύστημα RGB βλέπε: https://en.wikipedia.org/wiki/RGB_color_model] ή τον Δεκαεξαδικό κωδικό (Hexadecimal color code) [βλέπε: https://en.wikipedia.org/wiki/Web_colors], οι οποίοι είναι μοναδικοί για κάθε χρώμα-απόχρωση.

Η βιβλιοθήκη Tkinter διαθέτει το module `colorchooser` το οποίο επιτρέπει στον χρήστη να επιλέξει το χρώμα που επιθυμεί από μια εργαλειοθήκη επιλογής χρωμάτων.

Για να διευκολυνθεί ο αναγνώστης στην επιλογή των χρωμάτων που θα χρησιμοποιήσει στα προγράμματα Python που θα γράψει, αλλά και για πιο πρακτικές εφαρμογές (π.χ. να βάψει το δωμάτιό του), γράφτηκε ο παρακάτω κώδικας:

```
from tkinter import *
from tkinter import colorchooser

class MyColorSelector:
    def __init__(self):
        self.root = Tk()
        self.root.geometry('650x200')
        self.root.maxsize(650, 250)
        self.root.config(bg="whitesmoke")
        self.root.title('My Python Color Selector')
        self.my_color_is_label = Label(self.root,
                                       text='The Selected Color is:',
                                       bg="whitesmoke", font='verdana 12')
        self.my_color_is_label.grid(row=0, column=0, columnspan=1,
                                    padx=5, pady=5, sticky=W)
        self.my_color = Canvas(self.root, bg="whitesmoke",
                               width=100, height=20)
        self.my_color.grid(row=0, column=1, columnspan=1,
                           padx=5, pady=5, sticky=W)
        self.my_color_RGB_label = Label(self.root, bg="whitesmoke",
                                       text='RGB code:', font='verdana 12')
        self.my_color_RGB_label.grid(row=1, column=0, columnspan=2,
                                      padx=5, pady=5, sticky=W)
```

```

self.RGB_out = StringVar()
self.RGB_output = Label(self.root, textvariable=self.RGB_out,
                        bg="whitesmoke", font='verdana 12')
self.RGB_output.grid(row=1, column=1, columnspan=1,
                    padx=5, pady=5, sticky=W)
self.my_color_HEX_label = Label(self.root, bg="whitesmoke",
                                text='Hexadecimal code:', font='verdana 12')
self.my_color_HEX_label.grid(row=2, column=0, columnspan=2,
                            padx=5, pady=5, sticky=W)

self.HEX_out = StringVar()
self.HEX_output = Label(self.root, textvariable=self.HEX_out,
                        bg="whitesmoke", font='verdana 12')
self.HEX_output.grid(row=2, column=1, columnspan=1,
                    padx=5, pady=5, sticky=W)
self.choose_button = Button(self.root, text="Select Color",
                            command=self.select_color, relief=RIDGE,
                            font='verdana 10 bold', bg='#b9cbe6')
self.choose_button.grid(row=3, column=1, columnspan=1,
                        padx=5, pady=5, sticky=W)

self.root.mainloop()

def select_color(self):
    color = colorchooser.askcolor(title="Tkinter Color Palette")
    if str(color[1]) == 'None':
        self.my_color.configure(bg="whitesmoke")
    else:
        self.my_color.configure(bg=color[1])
    self.RGB_out.set(color[0])
    self.HEX_out.set(color[1])

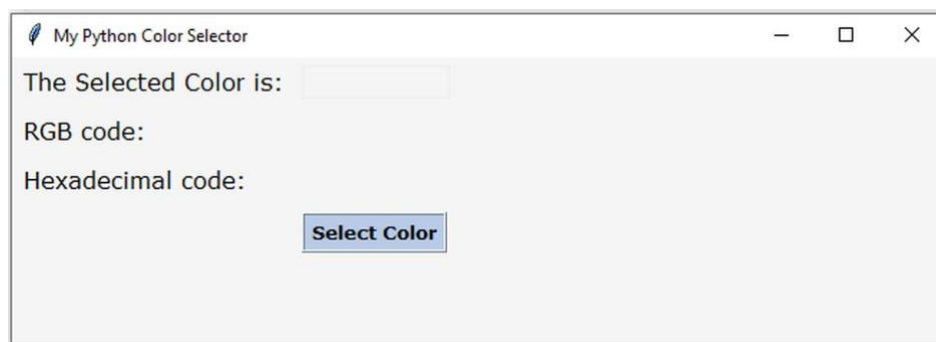
```

```

if __name__ == "__main__":
    mycolor_palette = MyColorSelector()

```

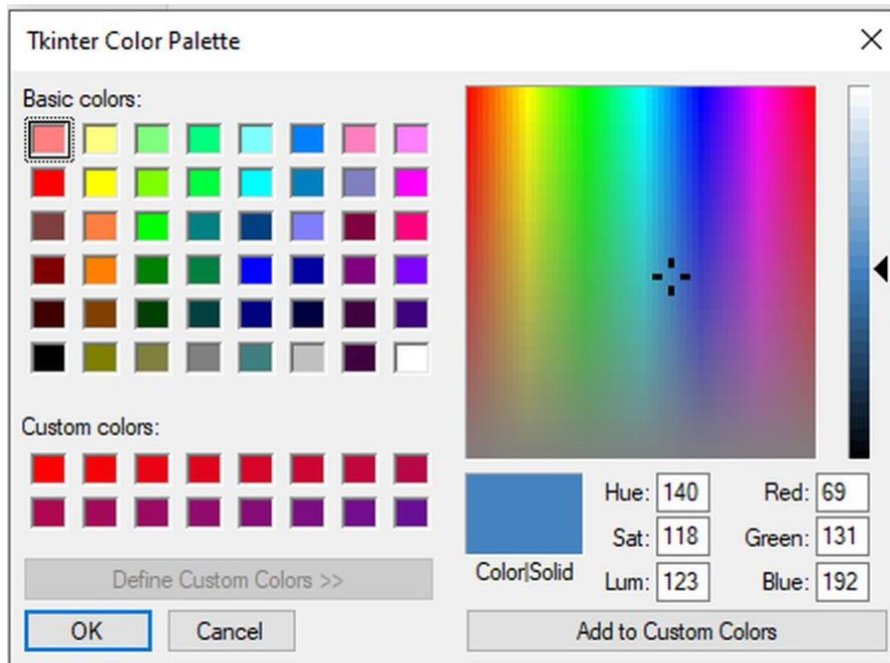
Εάν τρέξουμε το παραπάνω script του προγράμματος `MyColorSelector.py`, στην οθόνη θα εμφανιστεί το παράθυρο GUI της Εικόνας 17.14.



Εικόνα 17.14. Το παράθυρο GUI του προγράμματος επιλογής χρωμάτων `MyColorSelector.py`

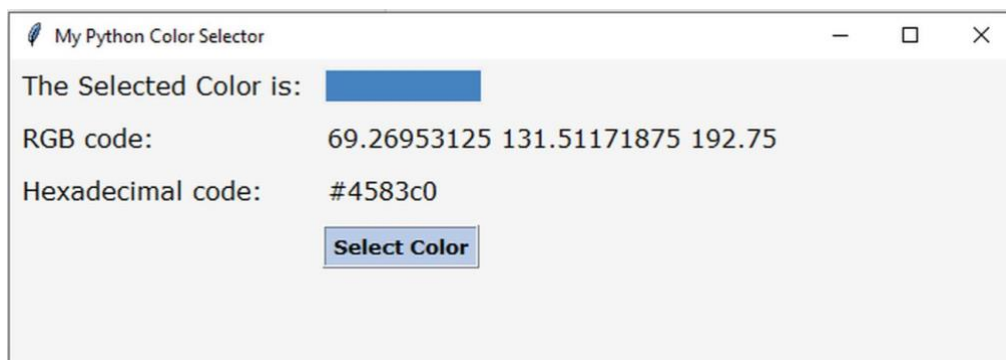
Πατώντας το κουμπί `Select Color`, εμφανίζεται το αναδυόμενο παράθυρο της Εικόνας 17.15, από το οποίο ο χρήστης μπορεί να επιλέξει ένα χρώμα από τα *Βασικά* (Basic) ή τα *Συννηθισμένα* για τις επιλογές του χρήστη (Custom) χρώματα ή ακόμα και να συνθέσει το δικό του χρώμα από την παλέτα του χρωματολογίου.

Εάν, για παράδειγμα, επιλέξουμε, κάνοντας κλικ με το ποντίκι, το χρώμα που ορίζεται από τη θέση του σταυρονήματος επί της επιφάνειας της παλέτας του χρωματολογίου, βλέπε Εικόνα 17.15, στη συνέχεια την απόχρωσή του, μετακινώντας τον δείκτη-βέλος στα δεξιά της παλέτας, και τέλος πατήσουμε το κουμπί `OK`, το αναδυόμενο παράθυρο θα κλείσει, και στο παράθυρο GUI του `MyColorSelector.py` θα εμφανιστούν κατά σειρά:



Εικόνα 17.15. Το αναδυόμενο (pop-up) παράθυρο επιλογής χρώματος/απόχρωσης.

Το δείγμα, ο κωδικός RGB και ο αντίστοιχος Δεκαεξαδικός κωδικός του χρώματος που επιλέξαμε ή συνθέσαμε, όπως φαίνεται στην Εικόνα 17.16.



Εικόνα 17.16. Τα στοιχεία του χρώματος-απόχρωσης που επιλέχθηκαν από το αναδυόμενο παράθυρο της Εικόνας 17.15.

Η δομή του κώδικα είναι εύκολα κατανοητή μετά τα όσα αναφέρθηκαν στην παράγραφο 17.5 *To GUI παράθυρο του BLASTprvGUI.py*

Πρακτικά, όλη τη δουλειά την κάνει η συνάρτηση `select_color()`, η οποία μέσω της μεθόδου `askcolor()` του module `colorchooser` της βιβλιοθήκης Tkinter, εκχωρεί την τιμή του επιλεγέντος χρώματος στη μεταβλητή `color`, όταν πατηθεί το κουμπί OK του παράθυρου της Εικόνας 17.15.

Η τιμή του χρώματος δίνεται υπό τη μορφή μιας πλειάδας που έχει δύο στοιχεία: το στοιχείο `color[0]` παρέχει την τιμή του χρώματος με τον κωδικό RGB, ενώ το δεύτερο στοιχείο `color[1]` παρέχει την τιμή του χρώματος με τον Δεκαεξαδικό κωδικό του.

Εάν πατηθεί το κουμπί Cancel, τότε η μεταβλητή `color` λαμβάνει την τιμή `None`.

Για την τεκμηρίωση του module `colorchooser` ανατρέξτε στο website:

<https://docs.python.org/3/library/tkinter.colorchooser.html>

Το script του προγράμματος επιλογής χρωμάτων βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_17_MyColorSelector.py`

Σημείωση: Τα χρώματα που επιλέγουμε με τη βοήθεια του προγράμματος `MyColorSelector.py` είναι χρώματα του πραγματικού κόσμου. Εάν σας αρέσει το χρώμα που επιλέξαμε σε αυτή την παράγραφο, με κωδικό RGB 69 131 192, και θέλετε να βάψετε κάτι με αυτό, ψάχνοντας στο Google, θα διαπιστώσετε ότι τα πλησιέστερα σε αυτό πλαστικά χρώματα που κυκλοφορούν στο εμπόριο είναι το Athens Blue 797 της Benjamin Moore και το Mykonos S39H6 της Dulux.

17.7 Τα megawidgets της βιβλιοθήκης Pmw

Όπως αναφέραμε και στην παράγραφο 17.1 Γενικά αυτού του Κεφαλαίου, η Python διαθέτει έναν μεγάλο αριθμό πακέτων-toolkits, από τα οποία μπορεί να επιλέξει ο προγραμματιστής κάποιο που θα χρησιμοποιεί για την ανάπτυξη εφαρμογών που θα διαθέτουν Graphical User Interface (GUI).

Στο website: <https://wiki.python.org/moin/GuiProgramming> δίνεται μια μακριά λίστα τέτοιων πακέτων για να επιλέξει ο ενδιαφερόμενος.

Πολλά από αυτά παρέχουν σχεδιαστικές ευκολίες και αυτοματισμούς για τη δημιουργία σύνθετων και υπολογιστικά απαιτητικών GUI, αλλά καλό είναι να γνωρίζουμε ότι θα πρέπει να αφιερώσουμε αρκετό χρόνο και κόπο για την εκμάθηση ενός τέτοιου πακέτου.

Στις προηγούμενες παραγράφους του Κεφαλαίου είδαμε πώς μπορούμε να δημιουργήσουμε το παράθυρο GUI μιας εφαρμογής χρησιμοποιώντας τη βιβλιοθήκη Tkinter, που είναι το de-facto standard πακέτο GUI της Python και για τον λόγο αυτό εγκαθίσταται στον υπολογιστή μας μαζί με την Python.

Ο συγγραφέας του βιβλίου έχει την άποψη ότι για τις ανάγκες ανάπτυξης ενός GUI σε προγράμματα που αναπτύσσουν σπουδαστές ή και Μηχανικοί και προορίζονται για προσωπική ή μη εμπορική χρήση, η Tkinter έχει τη δυνατότητα να αναπτύξει αξιοπρεπή παράθυρα GUI.

Μάλιστα, εάν η Tkinter συνδυαστεί με τη βιβλιοθήκη Pmw, που είναι μια επέκταση της Tkinter, τότε το αποτέλεσμα πλησιάζει το επαγγελματικό επίπεδο.

Η βιβλιοθήκη Pmw αποτελείται από megawidgets, δηλαδή από σύνθετα widgets που βασίζονται στα widgets της Tkinter και διευκολύνουν τον προγραμματισμό εφαρμογών GUI.

Στο website <http://pmw.sourceforge.net/> θα βρείτε πληροφορίες για την εγκατάσταση της Pmw, τεκμηρίωση και παραδείγματα.

Η παρουσίαση των megawidgets της Pmw είναι πέραν του σκοπού αυτού του βιβλίου και θα περιοριστούμε μόνο σε ένα δείγμα γραφής.

Στον κώδικα του προγράμματος `BLASTppnGUI.py`, για να δημιουργήσουμε το τμήμα του παραθύρου GUI με το οποίο εισάγουμε την τιμή της ποσότητας της εκρηκτικής ύλης, γράψαμε τις παρακάτω γραμμές κώδικα:

```
# Weight of Explosives
self.label_weight = Label(self.root,
                           text='Weight of Explosives [kg]', bg="whitesmoke")
self.label_weight.grid(row=3, column=0, columnspan=1,
                       padx=5, pady=5, sticky=W)
self.weight_of_explosives = StringVar()
self.weight_of_explosives.set('100')
self.weight_of_explosives = Entry(self.root, bd=3,
                                  textvariable=self.weight_of_explosives)
self.weight_of_explosives.grid(row=3, column=1, padx=5,
                               pady=5, sticky=W)
```

Χρησιμοποιώντας το ανάλογο για την περίπτωση megawidget της Pmw, το `Pmw.EntryField`, οι αντίστοιχες γραμμές κώδικα είναι:

```
# Weight of Explosives --> Pmw
import Pmw
self.weight_of_explosives = Pmw.EntryField(self.root,
    labelpos=W, label_text='Weight of Explosives [kg]',
    labelmargin=70, validate={'validator': 'real',
        'min': 5.0, 'max': 5000.0, 'minstrict': 0,
        'maxstrict': 0}, errorbackground='red', value=100)
self.weight_of_explosives.grid(row=3, column=0, columnspan=2, padx=5,
    pady=5, sticky=W)
```

Καταρχάς, κάνουμε import την Pmw.

Στη συνέχεια, για να εισάγουμε την τιμή της ποσότητας της εκρηκτικής ύλης, χρησιμοποιούμε το megawidget EntryField, το οποίο συνδυάζει τα widgets Label () και Entry () της Tkinter σε ένα widget, το οποίο μάλιστα έχει επιπλέον δυνατότητες-ευκολίες.

Για παράδειγμα, μέσω της παραμέτρου validate γίνεται άμεσα έλεγχος της τιμής που πληκτρολογούμε, η οποία πρέπει να είναι ένας πραγματικός αριθμός μεταξύ 5.0 και 5000.0 και εάν δεν είναι, τότε το ορθογώνιο πλαίσιο που δέχεται την τιμή θα χρωματιστεί κόκκινο (errorbackground='red').

Τέλος, χρησιμοποιούμε την ίδια εντολή του διαχειριστή γεωμετρίας grid(), με τη διαφορά ότι τώρα προστίθεται η παράμετρος columnspan=2, γιατί το megawidget EntryField() εκτείνεται σε 2 στήλες, μία για την ετικέτα και άλλη μία για το πεδίο πληκτρολόγησης της τιμής.

Εάν αντικαταστήσουμε στο πρόγραμμα BLASTprnGUI.py το τμήμα του κώδικα, που αφορά την εισαγωγή της τιμής της ποσότητας της εκρηκτικής ύλης και υλοποιήθηκε χρησιμοποιώντας τα widgets του Tkinter, με εκείνο που υλοποιήθηκε χρησιμοποιώντας το megawidget της Pmw, το παράθυρο GUI που θα προκύψει θα έχει την ίδια ακριβώς εμφάνιση στην οθόνη του υπολογιστή.

18 Προσομοίωση συστημάτων

18.1 Γενικά

Το Κεφάλαιο αυτό περιλαμβάνεται στο βιβλίο για τους εξής λόγους:

1. Στο πρόγραμμα σπουδών της Σχολής Μηχανικών Μεταλλείων – Μεταλλουργών του ΕΜΠ υπάρχει μάθημα με τίτλο *Προσομοίωση Μεταλλευτικών Συστημάτων*, το οποίο εισήγαγε και πρωτοδίδαξε ο συγγραφέας του βιβλίου.
2. Το Εργαστήριο Εξόρυξης Πετρωμάτων του ΕΜΠ έχει μακρόχρονη εμπλοκή στην ανάπτυξη μοντέλων προσομοίωσης μεταλλευτικών συστημάτων, είτε με κώδικα γραμμένο σε γλώσσες προγραμματισμού (Fortran, GPSS/H, Python) είτε με το ειδικό για προσομοίωση λογισμικό Arena® της Rockwell Automation [<https://www.arenasimulation.com>].

Μάλιστα η Μονάδα Προσομοίωσης Μεταλλευτικών Συστημάτων (MSSLab), που το 2000 ενσωματώθηκε στο Εργαστήριο, διοργάνωσε το *1st International Symposium on Mine Simulation via the Internet*, (MineSim'96), Dec. 2-13, 1996. Δηλαδή, εν έτι 1996 διοργανώθηκε ένα συνέδριο μέσω του Internet, κάτι που σήμερα, 26 χρόνια μετά, θεωρείται πολύ συνηθισμένο.

3. Η προσομοίωση έχει καθιερωθεί ως το κύριο υπολογιστικό εργαλείο για τον σχεδιασμό και τη μελέτη σύνθετων συστημάτων σε πολλούς βιομηχανικούς κλάδους, συμπεριλαμβανομένης της μεταλλευτικής βιομηχανίας.
4. Σχετικά πρόσφατα παρουσιάστηκε και είναι υπό συνεχή ανάπτυξη, η βιβλιοθήκη Salabim, η οποία διευκολύνει εξαιρετικά την ανάπτυξη μοντέλων προσομοίωσης στην Python, δίνοντας επιπλέον τη δυνατότητα στον προγραμματιστή να δημιουργήσει, σε πραγματικό χρόνο, το animation του μοντέλου προσομοίωσης.

18.2 Περί προσομοίωσης

Προσομοίωση (simulation) είναι η μίμηση της λειτουργίας ενός *συστήματος* (system), δηλαδή ενός συνόλου αντικειμένων και διεργασιών, το οποίο μπορεί να είναι υπαρκτό στον πραγματικό κόσμο που ζούμε ή υποθετικό, κατά τη διάρκεια μιας ορισμένης χρονικής περιόδου.

Για να προσομοιώσουμε ένα σύστημα θα πρέπει να δημιουργήσουμε πρώτα ένα *μοντέλο* (model).

Το μοντέλο θα αναπαριστά τα αντιπροσωπευτικά χαρακτηριστικά και τις συμπεριφορές του προς προσομοίωση συστήματος, ενώ η προσομοίωση θα αναπαριστά τη μετεξέλιξη του μοντέλου κατά τη διάρκεια της χρονικής περιόδου που αυτό προσομοιώνεται. Γενικά περί προσομοίωσης μπορείτε να διαβάσετε στο website: <https://en.wikipedia.org/wiki/Simulation>

Τα μοντέλα μπορεί να είναι *φυσικά*, να έχουν δηλαδή υλική υπόσταση, όπως π.χ. είναι ο *σεισμικός προσομοιωτήρας* του Εργαστηρίου Αντισεισμικής Τεχνολογίας της Σχολής Πολιτικών Μηχανικών του Εθνικού Μετσόβιου Πολυτεχνείου [<http://lee.civil.ntua.gr/trapeza.html>] ή *υπολογιστικά*, όπως είναι τα προγράμματα προσομοίωσης της πτήσεως αεροσκαφών (flight simulators).

Στη συνέχεια θα ασχοληθούμε με την προσομοίωση συστημάτων με χρήση υπολογιστικών μοντέλων (computer simulations), όπου τα χαρακτηριστικά και οι συμπεριφορές του μοντέλου περιγράφονται με αλγόριθμους και δεδομένα.

Στα υπολογιστικά μοντέλα προσομοίωσης, ο υπολογιστής τρέχει το μοντέλο και μελετούμε τη συμπεριφορά του συστήματος, το οποίο αντιπροσωπεύει, κατά τη διάρκεια μιας χρονικής περιόδου. Αλλάζοντας τις τιμές των μεταβλητών του μοντέλου, μπορούμε να κάνουμε εκτιμήσεις για τη συμπεριφορά του συστήματος υπό διαφορετικές συνθήκες λειτουργίας.

Η προσομοίωση, όταν ξεκίνησε να χρησιμοποιείται ως εργαλείο μελέτης και σχεδιασμού βιομηχανικών συστημάτων στη δεκαετία του 1960, ήταν μια ακριβή μεθοδολογία λόγω του υψηλού ωριαίου κόστους χρήσης των τότε υπολογιστών (mainframe computers). Στα τέλη της δεκαετίας του 1970 και ιδίως από τα μέσα της δεκαετίας του 1980 και μετά, με τη διάδοση της χρήσης των προσωπικών υπολογιστών, η προσομοίωση γνώρισε μεγάλη εξάπλωση στην επιστήμη και την τεχνολογία και έγινε ένα εξαιρετικά χρήσιμο εργαλείο για τον Μηχανικό και ιδιαίτερα για τον Μεταλλειολόγο.

Ο συγγραφέας του βιβλίου ευτύχησε να γράψει το πρώτο του πρόγραμμα προσομοίωσης κατά τη διάρκεια των μεταπτυχιακών του σπουδών στο University of Newcastle upon Tyne της Αγγλίας, το 1978-79.

Επρόκειτο για το πρόγραμμα SIMKAR που προσομοίωνε τις εργασίες εξόρυξης-μεταφοράς-απόθεσης του Λιγνιτωρυχείου Καρδιάς Πτολεμαΐδας, γραμμένο στη γλώσσα προγραμματισμού FORTRAN IV που έτρεχε με το λειτουργικό σύστημα MTS σε έναν υπολογιστή mainframe IBM 370/168. Επειδή είναι η πρώτη εφαρμογή της μεθόδου της προσομοίωσης στην ελληνική μεταλλευτική βιομηχανία, ο κώδικας του SIMKAR, στη φυσική μορφή 3,300 διάτρητων καρτών – εντολών της FORTRAN, αποτελεί έκθεμα του Ορυκτολογικού Μουσείου γαιο-ΟΡΑΜΑ της Σχολής Μηχανικών Μεταλλείων – Μεταλλουργών του ΕΜΠ (Εικόνα 18.1).

Το 2018, 40 χρόνια μετά, ξαναέγραψε το SIMKAR στην Python χρησιμοποιώντας τη βιβλιοθήκη Salabim. Το πρόγραμμα τώρα έχει περίπου 5100 γραμμές κώδικα, αλλά διαθέτει παράθυρο GUI και τρέχει άνετα κάτω από τα Windows 10 σε ένα κοινό notebook.

Με δεδομένη τη διαθεσιμότητα φθηνής υπολογιστικής ισχύος και δωρεάν λογισμού, όπως η γλώσσα Python και η βιβλιοθήκη Salabim, η προσομοίωση είναι σήμερα ένα ανεκτίμητο εργαλείο στα χέρια του Μηχανικού και του ερευνητή, γιατί του επιτρέπει να σχεδιάσει και να δοκιμάσει τη λειτουργία ενός καινούριου συστήματος πριν το κατασκευάσει. Επίσης, μπορεί να αξιολογήσει τις επιπτώσεις των αλλαγών που σχεδιάζει να πραγματοποιήσει σε ένα υφιστάμενο σύστημα, π.χ. στο σύστημα μεταφοράς σε ένα μεταλλείο, χωρίς να διαταράξει τη λειτουργία του, να προμηθευτεί νέο εξοπλισμό ή να πάρει ρίσκα.



Εικόνα 18.1. Ο κώδικας του προγράμματος προσομοίωσης SIMKAR σε μορφή διάτρητων καρτών. B: Τα αποτελέσματα της προσομοίωσης σε μηχανογραφικό χαρτί (μόνον...). C: Η διάταξη του εξοπλισμού του Λιγνιτωρυχείου Καρδιάς που προσομοιώθηκε [Πηγή: Ορυκτολογικό Μουσείο γαιο-ΟΡΑΜΑ της Σχολής Μηχανικών Μεταλλείων – Μεταλλουργών του ΕΜΠ].

18.3 Στοχαστική προσομοίωση διακριτών γεγονότων

Ένα σύστημα, και κατ' επέκταση το μοντέλο που το προσομοιώνει, ονομάζεται *συνεχές* (continuous) όταν, κατά τη διάρκεια μιας χρονικής περιόδου, οι αλλαγές στην κατάσταση του συμβαίνουν κατά *συνεχή τρόπο*.

Κλασικό παράδειγμα συνεχούς συστήματος είναι η μεταβολή της στάθμης της τεχνητής λίμνης ενός υδροηλεκτρικού σταθμού. Είναι προφανές ότι η στάθμη μεταβάλλεται συνεχώς, γιατί ποσότητες νερού αφαιρούνται από τη λίμνη προκειμένου να τροφοδοτήσουν τους υδροστροβίλους, αλλά και λόγω της εξάτμισης από την επιφάνεια της λίμνης, ενώ άλλες ποσότητες νερού προστίθενται στη λίμνη από ποτάμια και ρυάκια που εκβάλλουν σε αυτήν αλλά και από τις βροχοπτώσεις/χιονοπτώσεις στην περιοχή της λίμνης.

Ένα σύστημα, και κατ' επέκταση το μοντέλο που το προσομοιώνει, ονομάζεται *διακριτών γεγονότων* (discrete event) όταν, κατά τη διάρκεια μιας χρονικής περιόδου, οι αλλαγές στην κατάσταση του συμβαίνουν σε *ξεχωριστές-διακριτές χρονικές στιγμές*.

Τυπικό παράδειγμα είναι η αλλαγή καταστάσεων ενός χωματουργικού αυτοκινήτου σε ένα μεταλλείο. Κατά τη διάρκεια της βάρδιας, κάποια στιγμή αρχίζει να φορτώνει μέταλλευμα, κάποια άλλη στιγμή έχοντας τελειώσει τη φόρτωση, ξεκινάει τη μεταφορά του μεταλλεύματος, μια άλλη χρονική στιγμή φτάνει στον θραυστήρα και αδειάζει το μέταλλευμα και κάποια άλλη στιγμή, όντας άδειο, παίρνει τον δρόμο της επιστροφής στη θέση φόρτωσης.

Εάν μελετούσαμε το μοντέλο προσομοίωσης ενός αναλογικού ρολογιού ακριβείας, τότε θα δεχόμασταν, χωρίς να αποκλίνουμε από την πραγματικότητα, ότι ο χρόνος που απαιτείται για να κάνει μια πλήρη περιστροφή ο δείκτης των δευτερολέπτων είναι πάντα σταθερός και διαρκεί 1 min.

Στο μοντέλο προσομοίωσης του χωματουργικού αυτοκινήτου που περιγράφηκε προηγουμένως, είναι ουτοπικό να δεχθούμε ότι η χρονική διάρκεια του κύκλου φόρτωσης-μεταφοράς-εκκένωσης-επιστροφής είναι πάντα, κατά τη διάρκεια μιας 8ωρης βάρδιας, ακριβώς 11 min. Το λογικό είναι να δεχθούμε, βασιζόμενοι σε παρατηρήσεις του πραγματικού συστήματος, ότι ο χρόνος του κύκλου παρουσιάζει διακυμάνσεις και η διάρκειά του κάθε φορά μεταβάλλεται, προκύπτει δε ότι συντομότερος χρόνος είναι 9 min, ο αργότερος 13 min και ο συνηθισμένος 10.5 min.

Βλέπουμε, δηλαδή, ότι στην περίπτωση του ρολογιού ο χρόνος μεταβάλλεται κατά σταθερά διαστήματα και τότε έχουμε *ντετερμινιστική* (deterministic) προσομοίωση, ενώ στην περίπτωση του χωματουργικού αυτοκινήτου ο χρόνος μεταβάλλεται τυχαία εντός ορίων και τότε έχουμε *στοχαστική* (stochastic) προσομοίωση.

Η στοχαστική προσομοίωση ονομάστηκε και προσομοίωση Monte Carlo, λόγω της χρήσης κατανομών πιθανοτήτων και τυχαίων αριθμών, που παρέπεμπαν στο Καζίνο του Μόντε Κάρλο στο Πριγκιπάτο του Μονακό [βλέπε: https://en.wikipedia.org/wiki/Monte_Carlo_method].

Στη μεταλλευτική βιομηχανία και έρευνα έχει ιδιαίτερο ενδιαφέρον η *στοχαστική προσομοίωση διακριτών γεγονότων* (Discrete Event Stochastic Simulation) των μεταλλευτικών συστημάτων και με αυτό τον τύπο της προσομοίωσης θα ασχοληθούμε στη συνέχεια του Κεφαλαίου.

Για την ιστορία, πέρσι (2021) συμπληρώθηκαν 60 χρόνια από την πρώτη δημοσίευση εργασίας προσομοίωσης μεταλλευτικού συστήματος από τον K. Rist με τίτλο *"The solution of a transportation problem by use of a Monte Carlo technique"*, Proceedings of the 1st APCOM Conference, University of Arizona, Tucson, 1961.

18.4 Το πείραμα της βελόνας του Buffon

Έναν αιώνα περίπου πριν από την ίδρυση του Καζίνο του Μόντε Κάρλο, ο Γάλλος φυσιοδίφης και μαθηματικός Georges-Louis Leclerc, περισσότερο γνωστός ως Comte de Buffon [Σχετικά με τον Comte de Buffon βλέπε: https://en.wikipedia.org/wiki/Georges-Louis_Leclerc,_Comte_de_Buffon]

πειραματίστηκε με την τυχαιότητα και την προσομοίωση ενός φυσικού μοντέλου, προκειμένου να υπολογίσει την τιμή του π , δηλαδή το 3.14159...

Ο Κόμης Buffon διαπίστωσε ότι ρίχνοντας βελόνες μήκους L στο πάτωμα, όπου είχε χαράξει παράλληλες γραμμές που απέχουν μεταξύ τους απόσταση W και ήταν $W \geq L$, βλέπε Εικόνα 18.2, η πιθανότητα P να έρθει σε επαφή μια βελόνα με κάποια γραμμή στο πάτωμα είναι:

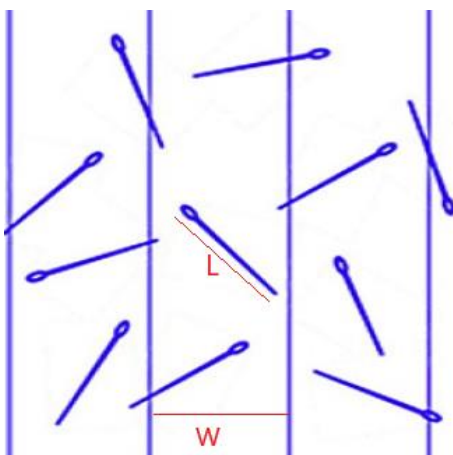
$$P = (2 * L) / (\pi * W) \quad [18.1]$$

και τότε η τιμή του π δίνεται από τη σχέση:

$$\pi = (2 * L) / (P * W) \quad [18.2]$$

Εξήγηση για το πώς προκύπτει η Εξίσωση [18.1] θα βρείτε στα websites:

https://en.wikipedia.org/wiki/Buffon%27s_needle_problem και <https://mste.illinois.edu/activity/buffon>



Εικόνα 18.2. Το πείραμα της βελόνας του Buffon [Πηγή: <https://www.pepijnvanderp.nl/2018/09/lazzarini-buffons-needle-pi>].

Ο κώδικας που ακολουθεί μετατρέπει το φυσικό μοντέλο του Buffon σε ένα μαθηματικό μοντέλο, γραμμένο στην Python και το προσομοιώνει:

```
# The Buffon's needle simulation model
from random import *
import numpy as np
needle_length = 7
lines_space = 10
number_of_trials = 1000000
hits_counter = 0
seed(12345)
for i in range(number_of_trials):
    needle_center_position = uniform(0, lines_space/2)
    needle_angle = uniform(0, np.pi)
    needle_tip_position = (needle_length/2)*np.sin(needle_angle)
    if needle_center_position <= needle_tip_position:
        hits_counter = hits_counter + 1
    else:
        pass
hit_probability = hits_counter/number_of_trials
pi_estimate = (2*needle_length)/(hit_probability*lines_space)
print("The Buffon's needle simulation model")
print('Hits probability = {:.5f}'.format(hit_probability))
print('Pi estimation     = {:.5f}'.format(pi_estimate))
```

Εάν τρέξουμε τον παραπάνω προσομοιωτή, στην οθόνη θα εμφανιστεί το αποτέλεσμα της προσομοίωσης:

The Buffon's needle simulation model

Hits probability = 0.44587

Pi estimation = 3.13992

Δηλαδή, εάν ρίξουμε 1,000,000 φορές μια βελόνα μήκους $L=7$ cm στο πάτωμα, όπου έχουμε χαράξει παράλληλες γραμμές που απέχουν μεταξύ τους απόσταση $W=10$ cm, η πιθανότητα να έρθει σε επαφή η βελόνα με κάποια γραμμή είναι $P=44.587\%$ και η εκτιμώμενη τιμή του π είναι 3.13992, δηλαδή αρκετά κοντά στο 3.14

Καταρχάς, στον κώδικα κάνουμε `import` το module `random` που εμπεριέχεται στην εγκατάσταση της Python και μας επιτρέπει να καλούμε γεννήτριες *ψευδο-τυχαίων αριθμών* (pseudo-random number generators) και στατιστικές κατανομές.

Για την τεκμηρίωση του module ανατρέξτε στο website:

<https://docs.python.org/3/library/random.html>

Επίσης, κάνουμε `import` και τη βιβλιοθήκη `numpy`.

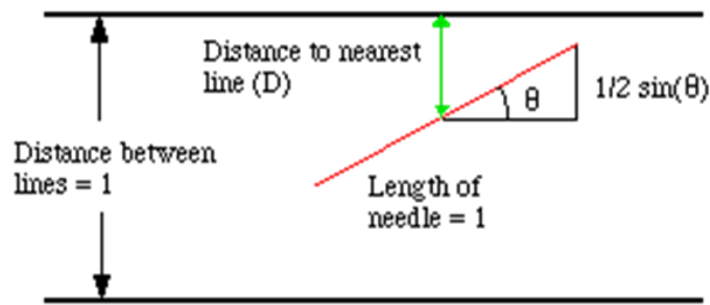
Στη συνέχεια δίνουμε τιμές στις μεταβλητές του προβλήματος και κάνουμε αρχικοποίηση στον μετρητή `hits_counter`, στον οποίο θα καταγράφουμε πόσες φορές η βελόνα έρχεται σε επαφή με μια γραμμή.

Η συνάρτηση `seed()` του module `random` δέχεται ως παράμετρο έναν ακέραιο αριθμό (τον 12345 στην περίπτωση μας) και *αρχικοποιεί* τη γεννήτρια των ψευδο-τυχαίων αριθμών.

Στο μπλοκ `for` που ακολουθεί, για κάθε ρίψη της βελόνας, και επειδή το εάν αυτή θα έρθει σε επαφή με μια γραμμή εξαρτάται, από τη θέση του κέντρου της βελόνας καθώς και από τη γωνία που σχηματίζει ο άξονάς της με τον άξονα των παράλληλων γραμμών στο πάτωμα, σχετικά με αυτό ανατρέξτε στο website <https://mste.illinois.edu/activity/buffon> και την Εικόνα 18.3:

1. Καλούμε τη συνάρτηση `uniform()` του module `random` η οποία, με τη βοήθεια της γεννήτριας των ψευδο-τυχαίων αριθμών, μας επιστρέφει έναν τυχαίο δεκαδικό αριθμό D έτσι ώστε $0 \leq D \leq W/2$ και ο οποίος αντιστοιχεί στη θέση του κέντρου της βελόνας από την πλησιέστερη γραμμή. Η τιμή του D εκχωρείται στη μεταβλητή `needle_center_position`.
2. Ομοίως, καλούμε τη συνάρτηση `uniform()`, η οποία τώρα μας επιστρέφει έναν τυχαίο δεκαδικό αριθμό θ έτσι ώστε $0 \leq \theta \leq \pi$ που αντιστοιχεί στη γωνία που σχηματίζει ο άξονας της βελόνας με τον άξονα των παράλληλων γραμμών. Η τιμή του θ εκχωρείται στη μεταβλητή `needle_angle`.
3. Με τη βοήθεια της τριγωνομετρίας υπολογίζουμε τη θέση του άκρου της βελόνας, δηλαδή την απόστασή του από το κέντρο της βελόνας, και την εκχωρούμε στη μεταβλητή `needle_tip_position`.
4. Εάν η απόσταση του κέντρου της βελόνας είναι μικρότερη ή ίση από την απόσταση του άκρου της, τότε η βελόνα έρχεται σε επαφή με τη γραμμή και ο μετρητής `hits_counter` αυξάνει την τιμή του κατά 1.

Η πιθανότητα να έρθει σε επαφή η βελόνα με κάποια γραμμή του πατώματος υπολογίζεται από το πηλίκο της τελικής τιμής του μετρητή `hits_counter` διά τον αριθμό των ρίψεων και το π από την Εξίσωση [18.2].



Εικόνα 18.3. Υπολογισμός της θέσης του άκρου της βελόνας στο πείραμα του Buffon [Πηγή: <https://mste.illinois.edu/activity/buffon>].

Από την προσομοίωση του πειράματος της βελόνας του Buffon διαπιστώνουμε τα ακόλουθα:

- Με τη μέθοδο της προσομοίωσης μπορούμε να *εκτιμήσουμε* την τιμή μεγεθών που γενικά *δύσκολα υπολογίζονται*.
- Η τιμή ενός μεγέθους, το οποίο εκτιμούμε, εμπεριέχει ένα ποσοστό λάθους.
- Όσες περισσότερες επαναλήψεις, δηλαδή ρίψεις της βελόνας, κάνουμε τόσο μικρότερο θα είναι το ποσοστό του λάθους και κατά συνέπεια ακριβέστερη η εκτίμηση του μεγέθους.
- Το πόσες επαναλήψεις πρέπει να κάνουμε εξαρτάται από το πόσο ικανοποιημένοι είμαστε από την ακρίβεια του αποτελέσματος της εκτίμησης.
- Επειδή ορισμένα γεγονότα που διέπουν τα πραγματικά συστήματα που προσομοιώνουμε μεταβάλλονται στοχαστικά, η προσομοίωσή τους με προγράμματα υπολογιστών καθιστούν αναγκαία τη χρήση γεννητριών ψευδο-τυχαίων αριθμών και στατιστικών κατανομών των τιμών των μεταβλητών που αποτελούν τα δεδομένα εισόδου.

Στον Πίνακα 18.1 παρουσιάζονται τα αποτελέσματα της προσομοίωσης του πειράματος του Buffon για διαφορετικό αριθμό ρίψεων της βελόνας.

Πίνακας 18.1. Επίδραση του αριθμού των ρίψεων της βελόνας στο αποτέλεσμα της προσομοίωσης του πειράματος του Buffon (τιμή του π).

Αριθμός Ρίψεων Βελόνας	Πιθανότητα Επαφής με Γραμμή	Τιμή του π
100	0.41000	3.41463
1000	0.44700	3.13199
1,000,000	0.44587	3.13992
10,000,000	0.44572	3.14099

Το script του κώδικα που προσομοιώνει το πείραμα του Buffon βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_18_Buffon.py`

18.5 Γεννήτριες ψευδο-τυχαίων αριθμών

Όλες οι κύριες γλώσσες προγραμματισμού διαθέτουν *γεννήτριες τυχαίων αριθμών*. Στην πράξη πρόκειται για αλγόριθμους που παράγουν μια σειρά από τυχαίους αριθμούς, η οποία σειρά κάποια στιγμή επαναλαμβάνεται. Για τον λόγο αυτό, ονομάζονται *γεννήτριες ψευδο-τυχαίων αριθμών* [Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Random_number_generation].

Η Python διαθέτει τη συνάρτηση `random()` του module `random` η οποία παράγει ψευδο-τυχαίους δεκαδικούς αριθμούς στο ημίκλειστο διάστημα $[0.0, 1.0)$ με βάση τη γεννήτρια Mersenne Twister [Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Mersenne_Twister].

Οι δεκαδικοί ψευδο-τυχαίοι αριθμοί που παράγονται από τη συνάρτηση `random()` έχουν ακρίβεια 53-bit, κατανέμονται ομοιόμορφα μεταξύ του 0.0 και του 1.0, όπου υπάρχει πιθανότητα να παραχθεί το 0.0 αλλά όχι το 1.0, και έχουν περίοδο επανάληψης $2^{19937}-1$.

Για να εκκινήσει την παραγωγή ψευδο-τυχαίων η γεννήτρια πρέπει να τροφοδοτηθεί με μια *τιμή-σπόρο* (seed value), ο οποίος είναι ένας ακέραιος αριθμός που εισάγεται στη γεννήτρια μέσω της συνάρτησης `seed()` του module `random`.

Για παράδειγμα, εάν τρέξουμε τις παρακάτω γραμμές κώδικα:

```
from random import *
seed(12345)
for i in range(10):
    random_number = random()
    print('Random number {:d} = {:.9f}'.format(i, random_number))
```

εμφανίζονται στην οθόνη 10 ψευδο-τυχαίοι δεκαδικοί αριθμοί, κατανεμημένοι ομοιόμορφα στο ημίκλειστο διάστημα [0.0, 1.0), οι οποίοι παρήχθησαν από τη συνάρτηση `random()` με την τιμή-σπόρο 12345:

```
Random number 0 = 0.416619873
Random number 1 = 0.010169169
Random number 2 = 0.825206509
Random number 3 = 0.298639855
Random number 4 = 0.368411689
Random number 5 = 0.193661349
Random number 6 = 0.566008169
Random number 7 = 0.161687824
Random number 8 = 0.124266884
Random number 9 = 0.432936268
```

Εάν ξανατρέξουμε τον κώδικα, θα πάρουμε ακριβώς τους ίδιους ψευδο-τυχαίους αριθμούς.

Εάν αντικαταστήσουμε τη γραμμή κώδικα `seed(12345)` με τη `seed(123)` και τρέξουμε τον κώδικα, στην οθόνη θα εμφανιστεί μια τελείως διαφορετική σειρά ψευδο-τυχαίων αριθμών:

```
Random number 0 = 0.052363599
Random number 1 = 0.087186678
Random number 2 = 0.407241764
Random number 3 = 0.107700235
Random number 4 = 0.901198878
Random number 5 = 0.038153666
Random number 6 = 0.536202040
Random number 7 = 0.332197699
Random number 8 = 0.852086619
Random number 9 = 0.159662397
```

Αυτό είναι πολύ χρήσιμο, γιατί μας επιτρέπει να χρησιμοποιούμε διαφορετικές *σειρές ψευδο-τυχαίων αριθμών* (random number streams), προκειμένου να λαμβάνουμε τιμές μεταβλητών από στατιστικές κατανομές, όπως θα δούμε στη συνέχεια.

Εάν δεν δώσουμε τιμή-σπόρο μέσω της συνάρτησης `seed()`, δηλαδή την παραλείψουμε, τότε η συνάρτηση `random()` θα πάρει ως default τιμή-σπόρο την ώρα του ρολογιού του υπολογιστή μας (system time) σε milliseconds. Στην περίπτωση αυτή είναι προφανές ότι, κάθε φορά που τρέχουμε μια προσομοίωση, θα χρησιμοποιούμε και μια διαφορετική σειρά ψευδο-τυχαίων αριθμών.

Είναι σημαντικό να αντιληφθούμε ότι, εάν επιθυμούμε να μελετήσουμε την επίδραση της μεταβολής των τιμών μιας παραμέτρου στα αποτελέσματα της στοχαστικής προσομοίωσης ενός συστήματος, όπως κάναμε στην παράγραφο 18.4 *Το πείραμα της βελόνας του Buffon*, όπου μεταβάλλαμε τον αριθμό των ρίψεων της βελόνας για τη δημιουργία του Πίνακα 18.1, τότε η σειρά των τυχαίων αριθμών θα πρέπει να παραμένει η ίδια.

18.6 Στατιστικές κατανομές

Το module `random` της Python διαθέτει συναρτήσεις που περιγράφουν έναν περιορισμένο αριθμό στατιστικών κατανομών. Ήδη στον κώδικα της παραγράφου 18.4 *Το πείραμα της βελόνας του Buffon*, χρησιμοποιήσαμε από το module αυτό τη συνάρτηση της *ομοιόμορφης κατανομής* `uniform(a, b)`, όπου `a` και `b` αριθμοί.

Η βιβλιοθήκη `Scipy` της Python, όμως, διαθέτει συναρτήσεις που περιγράφουν μια πληθώρα στατιστικών κατανομών, περισσότερες από 80 για *συνεχείς μεταβλητές* (continuous random variables) και 10 για *διακριτές μεταβλητές* (discrete random variables).

Για τις στατιστικές κατανομές της βιβλιοθήκης `Scipy` ανατρέξτε στο website: <https://docs.scipy.org/doc/scipy/reference/stats.html>

Επίσης, η βιβλιοθήκη `Salabim`, την οποία θα χρησιμοποιήσουμε στη συνέχεια για την προσομοίωση μεταλλευτικών συστημάτων, διαθέτει μια συλλογή με τις στατιστικές κατανομές που συνήθως χρησιμοποιούμε για την προσομοίωση συστημάτων διακριτών γεγονότων [Σχετικά με τις στατιστικές κατανομές της `Salabim` βλέπε στο website: <https://www.salabim.org/manual/Distributions.html>].

18.7 Προσαρμογή δεδομένων σε κατανομή: `fitter`

Όταν μελετάμε με τη μέθοδο της στοχαστικής προσομοίωσης διακριτών γεγονότων ένα πραγματικό σύστημα, όπως για παράδειγμα το σύστημα φόρτωσης-μεταφοράς με φορτωτές ή εκσκαφείς και χωματουργικά αυτοκίνητα (ΧΑ) σε ένα μεταλλείο, πρέπει να κάνουμε μετρήσεις για να συλλέξουμε δεδομένα, που αφορούν τις μεταβλητές λειτουργίας του πραγματικού συστήματος.

Οι μεταβλητές αυτές περιλαμβάνουν:

- τον χρόνο φόρτωσης του ΧΑ,
- τον χρόνο μετάβασης του ΧΑ στη θέση απόθεσης των αγόνων ή στον θραυστήρα του μεταλλεύματος,
- τον χρόνο εκκένωσης του περιεχομένου του ΧΑ,
- τον χρόνο επιστροφής του κενού ΧΑ στη θέση φόρτωσης,
- τον αριθμό των βλαβών στα ΧΑ, τα μηχανήματα φόρτωσης και τον θραυστήρα, καθώς και τη χρονική διάρκεια καθεμίας βλάβης,
- την ποσότητα του μεταλλεύματος ή των αγόνων που μεταφέρουν κάθε φορά τα ΧΑ.

Δεδομένα όπως τα παραπάνω, για να χρησιμοποιηθούν στον κώδικα ενός προγράμματος προσομοίωσης, πρέπει να αναλυθούν και να προκύψει η στατιστική κατανομή την οποία ακολουθούν, με τον καλύτερο βαθμό προσαρμογής.

Η βιβλιοθήκη `fitter` της Python είναι το εργαλείο που θα χρησιμοποιήσουμε για αυτή τη δουλειά.

Η κλάση `Fitter()` της βιβλιοθήκης αυτής προσδιορίζει την κατανομή στην οποία προσαρμόζεται καλύτερα ένα δείγμα των μετρήσεών μας, αναζητώντας τη βέλτιστη ανάμεσα στις 80 στατιστικές κατανομές συνεχών μεταβλητών της `Scipy`. Επίσης, μας δίνει τις παραμέτρους αυτής της βέλτιστης κατανομής και το διάγραμμα του ιστογράμματος των μετρήσεων μαζί με τις καμπύλες των κατανομών που προσαρμόστηκαν σε αυτές.

Για την τεκμηρίωση της βιβλιοθήκης ανατρέξτε στο website: <https://pypi.org/project/fitter>

Ως εξωτερική βιβλιοθήκη της Python πρέπει να εγκατασταθεί με τη γνωστή διαδικασία:

```
pip install fitter
```


Στη συνέχεια, θα δούμε μια εφαρμογή της βιβλιοθήκης `fitter` στην εύρεση της κατανομής που ακολουθούν οι χρόνοι του κύκλου των ΧΑ σε ένα σύστημα φόρτωσης-μεταφοράς μεταλλεύματος.

Συγκεκριμένα, έγιναν χρονομετρήσεις του χρόνου φόρτωσης (`Load_Time_Data`), του χρόνου μετάβασης στον θραυστήρα (`Haul_Time_Data`), του χρόνου εκκένωσης στον θραυστήρα (`Dump_Time_Data`) και του χρόνου επιστροφής στη θέση φόρτωσης (`Return_Time_Data`) των ΧΑ.

Τις μετρήσεις των χρόνων σε min, 1000 τον αριθμό, τις περάσαμε σε ένα φύλλο Excel και το αποθηκεύσαμε σε αρχείο με όνομα `time_data_study.xlsx` (βλέπε απόσπασμα στην Εικόνα 18.4). Το πλήρες αρχείο των μετρήσεων `time_data_study.xlsx` υπάρχει στο website του βιβλίου.

	A	B	C	D	E	F
1		Load_Time_Data	Haul_Time_Data	Dump_Time_Data	Return_Time_Data	
2	0	3.92	5.45	2.3	5.93	
3	1	3.03	4.72	2.12	5.59	
4	2	2.82	7.25	2.27	5.27	
5	3	2.89	7.45	2.16	5.17	
6	4	5.3	7.36	2.14	5.93	
7	5	2.8	6.18	2.28	5.59	
8	6	4.95	7.2	2.11	5.62	
9	7	2.75	7.24	2.2	5.12	
10	8	3.07	6.29	2.25	5.66	
11	9	3.69	7.12	2.17	5.61	
12	10	4.56	8.4	2.15	5.87	
13	11	3.01	7	2.27	5.33	
14	12	5.28	7.44	2.13	5.87	
15	13	3.02	6.58	2.25	5.56	
16	14	4.62	7.96	2.23	5.32	
17	15	3.21	8.3	2.2	5.14	
18	16	3.55	8.34	2.26	5.71	
19	17	2.81	8.64	2.28	5.55	
20	18	2.76	6.39	2.18	5.32	
21	19	2.85	7.77	2.28	5.12	
22	20	3.62	5.67	2.24	5.62	
23	21	3.83	6.7	2.12	5.67	

Εικόνα 18.4. Οι μετρήσεις των χρόνων του κύκλου των χωματουργικών αυτοκινήτων, σε min, που περιέχονται στο αρχείο `time_data_study.xlsx` Εμφανίζονται οι πρώτες 21 από τις συνολικά 1000 μετρήσεις.

Γράφουμε τον παρακάτω κώδικα:

```
from fitter import *
import pandas as pd
import pylab
print('Fitter, available distributions: \n', get_distributions())
print('Fitter, set of common distributions: \n', get_common_distributions())
time_study_data = pd.read_excel('time_data_study.xlsx', index_col=0)
#print(time_study_data.head())
load_time_data = time_study_data['Load_Time_Data'].to_numpy()
f = Fitter(load_time_data, distributions=['norm', 'expon', 'uniform', 'triang'])
f.fit()
print('Distribution fitting: ', time_study_data['Load_Time_Data'].name)
print(f.summary())
print('Best fitting distribution:')
print(f.get_best())
```

```
print('-----')
pylab.savefig('Load_Time_Data.png')
pylab.show()
```

Καταρχάς, κάνουμε import τις βιβλιοθήκες `fitter` και `pandas`, καθώς και το module `pylab`.

Το `PyLab` είναι ένα *βολικό* (convenience) module της Python, το οποίο κάνει ταυτόχρονα import το module `pyplot` της βιβλιοθήκης `Matplotlib`, για τον σχεδιασμό γραφικών παραστάσεων και τη βιβλιοθήκη `NumPy`, για τους μαθηματικούς υπολογισμούς και τη χρήση `arrays`.

Εάν έχετε κάνει ήδη την εγκατάσταση των βιβλιοθηκών `Matplotlib` και `NumPy` στον υπολογιστή σας, τότε μπορείτε να έχετε και τις υπηρεσίες του module `PyLab` στον κώδικά σας.

Παρόλο που η χρήση του `PyLab` δεν ενδείκνυται (disapproved) [βλέπε σχετικά στο website: <https://matplotlib.org/stable/api/index.html>], θα το χρησιμοποιήσουμε στον κώδικα για λόγους ομοιομορφίας, επειδή χρησιμοποιείται εσωτερικά στη βιβλιοθήκη `fitter`.

Η γραμμή κώδικα:

```
print('Fitter, available distributions: \n', get_distributions())
```

εμφανίζει στην οθόνη μια λίστα με τα ονόματα των 80 στατιστικών κατανομών της βιβλιοθήκης `Scipy` στις οποίες η `fitter` θα προσπαθήσει να προσαρμόσει τις μετρήσεις μας:

```
Fitter, available distributions:
['alpha', 'anglit', 'arcsine', 'argus', 'beta', 'betaprime', 'bradford', 'burr',
'burrr12', 'cauchy', 'chi', 'chi2', 'cosine', 'crystalball', 'dgamma',
'dweibull', 'erlang', 'expon', 'exponnorm', 'exponpow', 'exponweib', 'f',
'fatiguelife', 'fisk', 'foldcauchy', 'foldnorm', 'gamma', 'gausshyper',
'genexpon', 'genextreme', 'gengamma', 'genhalflogistic', 'geninvgauss',
'genlogistic', 'gennorm', 'genpareto', 'gilbrat', 'gompertz', 'gumbel_1',
'gumbel_r', 'halfcauchy', 'halfgennorm', 'halflogistic', 'halfnorm',
'hypsecant', 'invgamma', 'invgauss', 'invweibull', 'johnsonsb', 'johnsonsu',
'kappa3', 'kappa4', 'ksone', 'kstwo', 'kstwobign', 'laplace',
'laplace_asymmetric', 'levy', 'levy_1', 'levy_stable', 'loggamma', 'logistic',
'loglaplace', 'lognorm', 'loguniform', 'lomax', 'maxwell', 'mielke', 'moyal',
'nakagami', 'ncf', 'nct', 'ncx2', 'norm', 'norminvgauss', 'pareto', 'pearson3',
'powerlaw', 'powerlognorm', 'powernorm', 'rayleigh', 'rdist', 'recipinvgauss',
'reciprocal', 'rice', 'rv_continuous', 'rv_histogram', 'semicircular',
'skewnorm', 't', 'trapezoid', 'trapz', 'triang', 'truncexpon', 'truncnorm',
'tukeylambda', 'uniform', 'vonmises', 'vonmises_line', 'wald', 'weibull_max',
'weibull_min', 'wrapcauchy']
```

ενώ η γραμμή κώδικα:

```
print('Fitter, set of common distributions: \n', get_common_distributions())
```

εμφανίζει στην οθόνη τα ονόματα ενός υποσυνόλου των συνηθέστερων στατιστικών κατανομών:

```
Fitter, set of common distributions:
['cauchy', 'chi2', 'expon', 'exponpow', 'gamma', 'lognorm', 'norm', 'powerlaw',
'rayleigh', 'uniform']
```

Εάν παρέλθει χρόνος μεγαλύτερος των 30 sec και η `fitter` δεν έχει κατορθώσει να προσαρμόσει ικανοποιητικά στις μετρήσεις του δείγματός μας κάποια από τις κατανομές, τότε προσπαθεί να προσαρμόσει την επόμενη κατανομή και στην οθόνη εμφανίζεται ένα ή περισσότερα μηνύματα `WARNING`, όπως τα παρακάτω:

```
WARNING:root:SKIPPED kstwo distribution (taking more than 30 seconds)
WARNING:root:SKIPPED johnsonsu distribution (taking more than 30 seconds)
WARNING:root:SKIPPED kappa4 distribution (taking more than 30 seconds)
WARNING:root:SKIPPED ksone distribution (taking more than 30 seconds)
```

Η γραμμή κώδικα:

```
time_study_data = pd.read_excel('time_data_study.xlsx', index_col=0)
```

καλεί τη συνάρτηση της Pandas `read_excel()`, η οποία διαβάζει δεδομένα από αρχεία τύπου `.xlsx`, προκειμένου να διαβάσει τις μετρήσεις-δεδομένα του αρχείου `time_data_study.xlsx` και να τα αποθηκεύσει σε ένα `DataFrame` το οποίο ονομάζουμε `time_study_data`

Εάν επιθυμείτε να δείτε τη μορφή που έχει το `DataFrame` `time_study_data` μετατρέψτε το σχόλιο:

```
#print(time_study_data.head())
```

σε ενεργή γραμμή κώδικα και στην οθόνη θα εμφανιστούν οι πρώτες 5 γραμμές του `DataFrame`:

	Load_Time_Data	Haul_Time_Data	Dump_Time_Data	Return_Time_Data
0	3.06	7.10	2.25	5.39
1	2.95	5.38	2.22	5.46
2	4.15	5.66	2.28	5.12
3	5.58	5.27	2.19	4.87
4	4.54	7.57	2.26	5.16

Η γραμμή κώδικα:

```
load_time_data = time_study_data['Load_Time_Data'].to_numpy()
```

μετατρέπει τις μετρήσεις του `DataFrame` που είναι κάτω από την επικεφαλίδα `Load_Time_Data` (δηλαδή της μετρήσεις του χρόνου φόρτωσης του ΧΑ) σε `array` της `NumPy` [Σχετικά βλέπε στο website: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_numpy.html].

Αυτό είναι απαραίτητο να γίνει προκειμένου να είναι δυνατή η επεξεργασία των μετρήσεων από την κλάση `Fitter` της βιβλιοθήκης `fitter`, την οποία καλούμε στη συνέχεια με τη γραμμή κώδικα:

```
f = Fitter(load_time_data, distributions=['norm', 'expon', 'uniform', 'triang'])
```

Η κλάση `Fitter` προσαρμόζει τις μετρήσεις, που τώρα είναι υπό τη μορφή του `array` `load_time_data`, στις κατανομές οι οποίες δίνονται στην παράμετρο `distributions` της κλάσης.

Οι τιμές της παραμέτρου αυτής μπορεί να είναι:

- `distributions=get_distributions()` Στην περίπτωση που θέλουμε να ελεγχθεί η προσαρμογή των μετρήσεων σε όλες τις 80 κατανομές (το πρόγραμμα θα τρέχει για αρκετή ώρα...). Αυτή είναι και η `default` τιμή στην περίπτωση που παραλείψουμε να δηλώσουμε την παράμετρο.
- `distributions= get_common_distributions()` Στην περίπτωση που θέλουμε να ελεγχθεί η προσαρμογή των μετρήσεων στο υποσύνολο των συνηθέστερων στατιστικών κατανομών.
- `distributions=['norm', 'expon', 'uniform', 'triang']` Στην περίπτωση που θέλουμε να ελεγχθεί η προσαρμογή των μετρήσεων σε συγκεκριμένες στατιστικές κατανομές, δίνουμε τη λίστα των ονομάτων αυτών των κατανομών. Την επιλογή αυτή κάναμε στον κώδικά μας.

Η γραμμή κώδικα:

```
f.fit()
```

εφαρμόζει τη μέθοδο `fit()` επί του αντικειμένου `f` της κλάσης `Fitter` και προσδιορίζει τις παραμέτρους καθεμίας κατανομής που προσαρμόστηκε καλύτερα στις μετρήσεις.

Η εντολή `print()` που ακολουθεί εμφανίζει στην οθόνη μια επικεφαλίδα που περιλαμβάνει το όνομα των μετρήσεων που προσαρμόζουμε στις κατανομές, όπως αυτό αναφέρεται στο `DataFrame` τους.

Η γραμμή κώδικα:

```
print(f.summary())
```

αφού εφαρμόσει τη μέθοδο `summary()` επί του αντικειμένου `f` της κλάσης `Fitter`, εμφανίζει στην οθόνη τον πίνακα με τις 5 κατανομές που προσαρμόστηκαν καλύτερα, ή όλες εάν επιλέξαμε λιγότερες από 5, μαζί με τα κριτήρια κατάταξης και σχεδιάζει το ιστόγραμμα των μετρήσεων και τις καμπύλες προσαρμογής.

Εάν τρέξουμε τον παραπάνω κώδικα, θα εμφανιστεί στην οθόνη ο ακόλουθος πίνακας αποτελεσμάτων:

Distribution fitting: Load_Time_Data				
	sumsquare_error	aic	bic	kl_div
expon	0.262500	662.875521	-8231.443489	inf
norm	2.305319	1091.074478	-6058.720681	inf
triang	2.872304	445.567975	-5831.917526	inf
uniform	5.225210	384.122775	-5240.444871	inf

Η default κατάταξη των κατανομών γίνεται με βάση το κριτήριο X^2 (sums square error - `sumsquare_error`), σύμφωνα με το οποίο όσο μικρότερη είναι η τιμή του κριτηρίου, τόσο καλύτερη είναι η προσαρμογή των μετρήσεων στην υπόψη στατιστική κατανομή [Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Residual_sum_of_squares].

Παράλληλα δίνονται και οι τιμές που προκύπτουν από την εφαρμογή των κάτωθι κριτηρίων:

- Akaike information criterion (aic) [Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Akaike_information_criterion].
- Bayesian information criterion (bic) [Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Bayesian_information_criterion].
- Kullback–Leibler divergence (kl_div) [Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Kullback-Leibler_divergence].

Η γραμμή κώδικα:

```
print(f.get_best())
```

αφού εφαρμόσει τη μέθοδο `get_best()` επί του αντικειμένου `f` της κλάσης `Fitter`, εμφανίζει στην οθόνη τις παραμέτρους της κατανομής που προσαρμόστηκε καλύτερα στις μετρήσεις. Η ονοματολογία των παραμέτρων των κατανομών ακολουθεί εκείνη που εφαρμόζεται στη βιβλιοθήκη `SciPy`:

```
Best fitting distribution:
{'expon': {'loc': 2.7, 'scale': 1.02217}}
```

Η γραμμή κώδικα:

```
pylab.savefig('Load_Time_Data.png')
```

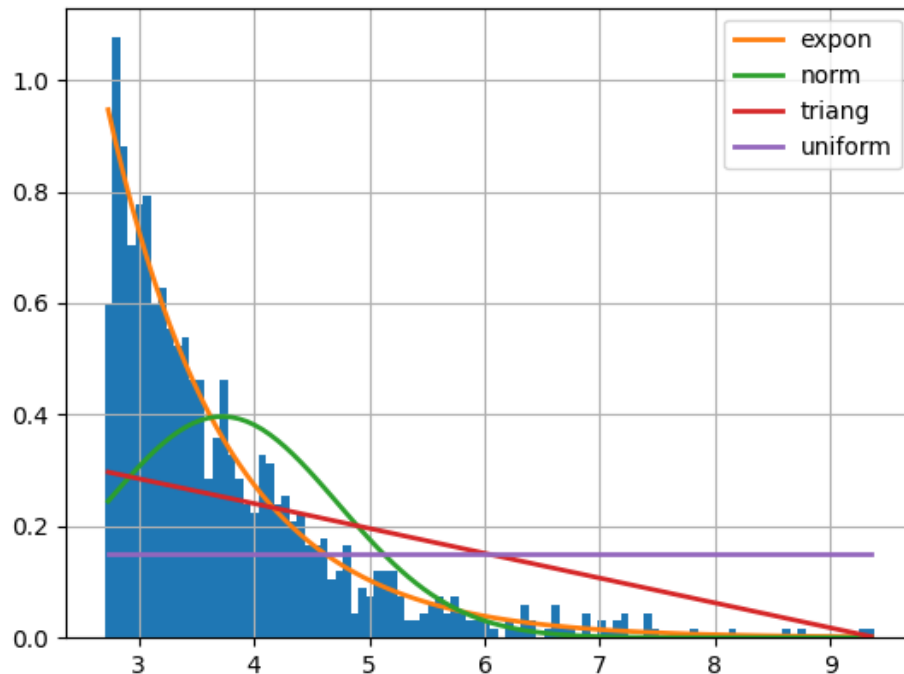
αποθηκεύει τη γραφική παράσταση με το ιστόγραμμα των μετρήσεων και τις καμπύλες προσαρμογής, η οποία δημιουργήθηκε προηγουμένως με τη μέθοδο `summary()` στο αρχείο `Load_Time_Data.png`, εφαρμόζοντας τη συνάρτηση `savefig()` του module `pylab`, που είναι η ομώνυμη συνάρτηση του module `pyplot` της βιβλιοθήκης `Matplotlib`.

Τέλος, η γραμμή κώδικα:

```
pylab.show()
```

εμφανίζει στην οθόνη τη γραφική παράσταση που παρουσιάζεται στην Εικόνα 18.5.

Το script του κώδικα που προσαρμόζει ένα σετ δεδομένων σε στατιστικές κατανομές βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_18_Fitter_One_Data_Set.py`



Εικόνα 18.5. Το ιστόγραμμα των μετρήσεων `Load_Time_Data` και οι καμπύλες προσαρμογής των στατιστικών κατανομών.

Για λόγους πληρότητας αυτής της παραγράφου, αλλά και *επίδειξης της δυνατότητας* που έχει η Python να δημιουργεί *συμπαγή (compact)* κώδικα, ο κώδικας που ακολουθεί κάνει προσαρμογή σε στατιστικές κατανομές των μετρήσεων όλων των χρόνων του κύκλου των ΧΑ που περιέχονται στο αρχείο `time_data_study.xlsx`

```
from fitter import *
import pandas as pd
import pylab
time_study_data = pd.read_excel('time_data_study.xlsx',
                                index_col=0)
data_for_fitting = ['Load_Time_Data', 'Haul_Time_Data',
                    'Dump_Time_Data', 'Return_Time_Data']
for i in data_for_fitting:
    data = time_study_data[i].values
    f = Fitter(data, distributions=['norm', 'expon', 'uniform', 'triang'])
    f.fit()
    print('Distribution fitting: ', time_study_data[i].name)
    print(f.summary())
    print('Best fitting distribution:')
    print(f.get_best())
    print('-----')
    pylab.savefig(time_study_data[i].name+'.png')
    pylab.show()
```

Εάν τρέξουμε τον παραπάνω κώδικα στην οθόνη, θα εμφανιστούν τα αποτελέσματα (παραλείπονται οι γραφικές παραστάσεις):

```

Distribution fitting: Load_Time_Data
      sumsquare_error      aic      bic  kl_div
expon          0.262500    662.875521 -8231.443489    inf
norm           2.305319   1091.074478 -6058.720681    inf
triang         2.872304    445.567975 -5831.917526    inf
uniform        5.225210    384.122775 -5240.444871    inf
Best fitting distribution:
{'expon': {'loc': 2.7, 'scale': 1.02217}}
-----
Distribution fitting: Haul_Time_Data
      sumsquare_error      aic      bic  kl_div
norm           0.153646   501.005820 -8767.043743    inf
triang         0.318129   442.153829 -8032.329685    inf
uniform        1.578266   384.719790 -6437.613080    inf
expon          2.471573   446.162675 -5989.085092    inf
Best fitting distribution:
{'norm': {'loc': 6.77637, 'scale': 1.1469140870614503}}
-----
Distribution fitting: Dump_Time_Data
      sumsquare_error      aic      bic  kl_div
uniform       10065.500000 -317.887582  2322.929246    inf
triang        10324.817998 -290.138636  2355.273777    inf
norm          10339.314019 -282.422672  2349.769035    inf
expon         10810.833623 -256.483621  2394.364255    inf
Best fitting distribution:
{'uniform': {'loc': 2.1, 'scale': 0.19999999999999973}}
-----
Distribution fitting: Return_Time_Data
      sumsquare_error      aic      bic  kl_div
norm           3.755062   312.737702 -5570.834848    inf
triang         6.555320   217.356217 -5006.755115    inf
uniform        20.314723   158.945434 -3882.593865    inf
expon          30.609893   221.015587 -3472.616526    inf
Best fitting distribution:
{'norm': {'loc': 5.41105, 'scale': 0.34471814211033336}}
-----

```

Το script του κώδικα που προσαρμόζει πολλά σετ δεδομένων σε στατιστικές κατανομές βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_18_Fitter_Many_Data_Sets.py`

18.8 Η βιβλιοθήκη Salabim

Η βιβλιοθήκη Salabim περιλαμβάνει κλάσεις και συναρτήσεις που προσομοιώνουν συστήματα διακριτών γεγονότων στη γλώσσα Python.

Είναι μια πλήρης βιβλιοθήκη, με κλάσεις που προσομοιώνουν τη λειτουργία *μονάδων ενδιαφέροντος* (components), π.χ. χωματουργικά αυτοκίνητα, πλοία, αντικείμενα που συναρμολογούνται σε μια γραμμή παραγωγής κ.ά., *μονάδων που προσφέρουν υπηρεσίες* (resources), π.χ. φορτωτές, λιμάνια, μηχανήματα κ.ά., *ουρές αναμονής* (queues), ενώ διαθέτει τη δική της συλλογή στατιστικών κατανομών για τη δειγματοληψία στοχαστικών μεταβλητών, π.χ. των χρόνων του κύκλου ενός χωματουργικού αυτοκινήτου.

Μια εξαιρετική δυνατότητα της Salabim είναι ότι επιτρέπει το animation, 2D και 3D, του συστήματος που προσομοιώνεται σε *πραγματικό χρόνο* (real time animation).

Η βιβλιοθήκη Salabim έχει αναπτυχθεί και συντηρείται από τον Ruud van der Ham, ο οποίος έχει μακρά ιστορία τόσο στην ανάπτυξη εφαρμογών όσο και εργαλείων προσομοίωσης. Στο επίσημο website της Salabim: <https://www.salabim.org> εκτός από την τεκμηρίωση της βιβλιοθήκης, υπάρχει και ένα λεπτομερές εγχειρίδιο χρήστη.

Ως εξωτερική βιβλιοθήκη της Python πρέπει να εγκατασταθεί στον υπολογιστή μας κατά τα γνωστά:

```
pip install salabim
```

Επιπρόσθετα, για τη δημιουργία 2D animation των προσομοιωτών, πρέπει να εγκατασταθεί στον υπολογιστή η βιβλιοθήκη Pillow (εάν δεν υπάρχει ήδη στον υπολογιστή σας από την εγκατάσταση της Python):

```
pip install Pillow
```

Για την εγγραφή του animation του προσομοιωτή σε video, πρέπει να εγκατασταθούν και οι βιβλιοθήκες Opencv και NumPy:

```
pip install opencv-python  
pip install numpy
```

Η προετοιμασία του υπολογιστή μας για να εκτελεί 3D animations, με τα οποία δεν θα ασχοληθούμε σε αυτό το βιβλίο, απαιτεί την εγκατάσταση επιπλέον βιβλιοθηκών. Λεπτομερείς πληροφορίες δίνονται στο website: <https://www.salabim.org/manual/Introduction.html>

Στις επόμενες παραγράφους θα δούμε τον τρόπο χρήσης των κλάσεων και συναρτήσεων της Salabim με τη βοήθεια παραδειγμάτων προσομοίωσης μεταλλευτικών συστημάτων.

18.9 Προσομοίωση συστήματος φόρτωσης

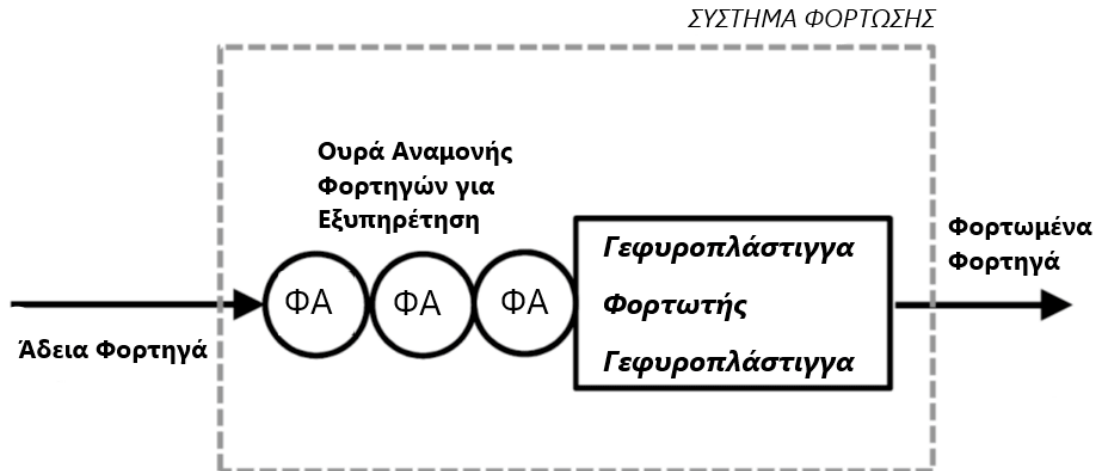
Ένα λατομείο ασβεστόλιθου παράγει αδρανή προϊόντα και τροφοδοτεί με αυτά το έργο της κατασκευής ενός αυτοκινητόδρομου στην περιοχή του. Τα φορτηγά αυτοκίνητα (ΦΑ) δημοσίας χρήσης, που μεταφέρουν τα αδρανή, φορτώνονται με έναν ελαστικοφόρο φορτωτή από σωρούς έτοιμων προϊόντων.

Από μετρήσεις προέκυψαν τα ακόλουθα:

- Τα ΦΑ φθάνουν στον χώρο φόρτωσης με ρυθμό που ακολουθεί μια ομοιόμορφη κατανομή που έχει ελάχιστη τιμή 10 min και μέγιστη 15 min.
- Ο χρόνος φόρτωσης κάθε ΦΑ, οποίος περιλαμβάνει τη ζύγισή του κενό, τους ελιγμούς για να βρεθεί στη θέση φόρτωσης, τη φόρτωσή του από τον φορτωτή και τη ζύγισή του φορτωμένο, ακολουθεί μια τριγωνική κατανομή με ελάχιστη τιμή 8 min, μέγιστη τιμή 15 min και συνήθη τιμή 12 min.
- Από το αρχείο ζυγίσεων της γεφυροπλάστιγγας προκύπτει ότι το φορτίο κάθε ΦΑ ακολουθεί μια κανονική τιμή με μέση τιμή 30 t και τυπική απόκλιση 1 t.

Λόγω της διαρρύθμισης του χώρου στην πλατεία έτοιμων προϊόντων του λατομείου, τα ΦΑ σταθμεύουν και σχηματίζουν μια ουρά αναμονής μπροστά από τη γεφυροπλάστιγγα, όταν το ΦΑ που προηγείται δεν έχει ακόμα ολοκληρώσει τον κύκλο ζύγισης-φόρτωσης-ζύγισης.

Στην Εικόνα 18.6 παρουσιάζεται σχηματικά το παραπάνω σύστημα φόρτωσης στο λατομείο.



Εικόνα 18.6. Σχηματική παρουσίαση του συστήματος φόρτωσης φορτηγών αυτοκινήτων με αδρανή στο λατομείο (ΦΑ: φορτηγό αυτοκίνητο).

18.9.1 Προσομοίωση του φορτωτή ως component

Προκειμένου να μελετήσουμε τη λειτουργία του παραπάνω συστήματος φόρτωσης με τη μέθοδο της προσομοίωσης διακριτών γεγονότων και με τη βοήθεια της βιβλιοθήκης Salabim, γράφουμε τον παρακάτω κώδικα:

```
import salabim as sim
number_of_shifts = 10
sim_duration = 480*number_of_shifts

class Truck(sim.Component):
    def process(self):
        self.enter(loading_queue)
        if loader.ispassive():
            loader.activate()
        yield self.passivate()

class Loader(sim.Component):
    def process(self):
        while True:
            while len(loading_queue) == 0:
                yield self.passivate()
            self.truck = loading_queue.pop()
            yield self.hold(sim.Triangular(8, 15, 12).sample())
            env.trucks_loaded += 1
            truck_load = sim.Normal(30, 1).sample()
            env.aggregates_tonnage += truck_load
            self.truck.activate()

# Main body
env = sim.Environment(trace=True)
sim.ComponentGenerator(Truck, lat=sim.Uniform(10, 15))
loader = Loader()
loading_queue = sim.Queue('Queue for loading')
env.trucks_loaded = 0
env.aggregates_tonnage = 0
env.run(duration=sim_duration)
# Simulation output
```

```

print('SIMULATION REPORT')
print('Simulation time [8-hour shifts] = {:d}'.format(number_of_shifts))
print('Number of trucks arrived for loading = {:d}'
      .format(loading_queue.number_of_arrivals))
print('Number of loaded trucks = {:d}'.format(env.trucks_loaded))
print('Loaded aggregates [t] = {:.2f}'.format(env.aggregates_tonnage))
utilization = (1 -
               (loader.status.value_duration("passive")/sim_duration))*100
print('Loader utilization [%] = {:.2f}'.format(utilization))
print('Queue for loading:')
print(' Length [trucks] minimum = {:.0f}, median = {:.0f}, maximum = {:.0f}'
      .format(loading_queue.length.minimum(ex0=True),
              loading_queue.length.median(ex0=True),
              loading_queue.length.maximum(ex0=True)))
print(' Length of stay [min] minimum = {:.2f}, median = {:.2f},'
      ' maximum = {:.2f}'.
      format(loading_queue.length_of_stay.minimum(ex0=True),
              loading_queue.length_of_stay.median(ex0=True),
              loading_queue.length_of_stay.maximum(ex0=True)))

```

Καταρχάς, κάνουμε `import` τη βιβλιοθήκη `salabim`, τις κλάσεις και συναρτήσεις της οποίας θα καλούμε χρησιμοποιώντας το παρατσούκλι `sim`.

Οι γραμμές κώδικα:

```

number_of_shifts = 10
sim_duration = 480*number_of_shifts

```

ορίζουν τον αριθμό των βαρδιών (shifts) που θα προσομοιωθούν και τη διάρκεια του χρόνου προσομοίωσης του συστήματος φόρτωσης σε min (μία 8ωρη βάρδια = 480 min).

Η `Salabim` δεν έχει συγκεκριμένη μονάδα μέτρησης του χρόνου. Εξαρτάται από τον χρήστη να αποφασίσει ποια μονάδα χρόνου θα χρησιμοποιήσει, π.χ. έτη, ημέρες, ώρες, min, sec κλπ., αρκεί να χρησιμοποιεί την ίδια μονάδα χρόνου στο σύνολο του κώδικα.

Σημείωση: Στις νεότερες εκδόσεις της βιβλιοθήκης προστέθηκε η δυνατότητα *ανάμειξης μονάδων χρόνου* στον κώδικα της `Salabim`, η οποία όμως δεν θα χρησιμοποιηθεί στο Κεφάλαιο αυτό [Σχετικά βλέπε στην παράγραφο *Time Units* του website: <https://www.salabim.org/manual/Miscellaneous.html>].

Στον κώδικά μας όλοι οι χρόνοι είναι σε min.

Κάτω από τη γραμμή σχολίου `# Main body` με τις εντολές:

```

env = sim.Environment(trace=True)

```

δημιουργούμε ένα αντικείμενο της κλάσης `Environment` της `Salabim` που το ονομάζουμε `env`

Η παράμετρος `trace=True` δημιουργεί μια λίστα-καταγραφή των μεταβολών του συστήματος που προσομοιώνεται, κατά τους *διακριτούς χρόνους της προσομοίωσης* και δίνεται ως `output` στο τέλος της προσομοίωσης (βλέπε παρακάτω). Αυτό είναι εξαιρετικά χρήσιμο για τον έλεγχο της ορθότητας της λογικής του μοντέλου προσομοίωσης που αναπτύσσουμε. Μπορεί να παραλειφθεί, δίνοντας στην παράμετρο την τιμή `False`.

```

sim.ComponentGenerator(Truck, iat=sim.Uniform(10, 15))

```

δημιουργούμε ένα και μοναδικό *στιγμιότυπο* (instance) της κλάσης `ComponentGenerator` της `Salabim`, το οποίο ονομάζεται αυτόματα `Truck.generator.0`

Με την κλάση αυτή *προκαλούμε τις αφίξεις* των ΦΑ (`truck`) στον χώρο φόρτωσης, με *χρόνο μεταξύ των αφίξεων* (inter-arrival time, `iat`) που ακολουθεί μια ομοιόμορφη κατανομή, η οποία έχει ελάχιστη τιμή 10 min και μέγιστη 15 min.

Τα ΦΑ κατά την άφιξή τους ονοματίζονται, αυτόματα, `truck.0`, `truck.1`, `truck.2` κ.ο.κ., δηλαδή έχουν το λεκτικό της κλάσης `Truck` αντικαθιστώντας το κεφαλαίο `T` με πεζό `t` ακολουθούμενο από μια τελεία και τον αύξοντα αριθμό του ΦΑ. Εάν επιθυμούμε να ονοματίσουμε διαφορετικά τα ΦΑ, π.χ. με τη μάρκα τους `MAN_TGS`, τότε χρησιμοποιούμε την παράμετρο `name='MAN_TGS.'` και η εντολή γράφεται:

```
sim.ComponentGenerator(Truck, iat=sim.Uniform(10, 15), name='MAN_TGS.')
```

Η βιβλιοθήκη `Salabim` διαθέτει μια συλλογή στατιστικών κατανομών, μεταξύ των οποίων περιλαμβάνεται και η ομοιόμορφη κατανομή, την οποία στην περίπτωση μας καλούμε ως `sim.Uniform(10, 15)`.

```
loader = Loader()
```

δημιουργούμε ένα και μοναδικό στιγμιότυπο (instance) της κλάσης `Loader` που ονοματίζεται αυτόματα `loader.0` (βλέπε παρακάτω).

```
loading_queue = sim.Queue('Queue for loading')
```

δημιουργούμε ένα αντικείμενο της κλάσης `Queue` της `Salabim`, που το ονομάζουμε `loading_queue`, αντιπροσωπεύει την ουρά αναμονής των άδειων ΦΑ που περιμένουν για φόρτωση και ονομάζουμε την ουρά αναμονής με το περιγραφικό όνομα `Queue for loading`.

```
env.trucks_loaded = 0
```

δημιουργούμε και αρχικοποιούμε μια μεταβλητή-μετρητή (counter) στο περιβάλλον του μοντέλου της προσομοίωσης, στην οποία θα καταγράφεται ο αριθμός των ΦΑ που φορτώθηκαν.

```
env.aggregates_tonnage = 0
```

δημιουργούμε και αρχικοποιούμε μια μεταβλητή-μετρητή στο περιβάλλον του μοντέλου της προσομοίωσης, στην οποία θα προστίθεται το βάρος των αδρανών που προμηθεύεται από το λατομείο το έργο, κάθε φορά που φορτώνεται ένα ΦΑ.

```
env.run(duration=sim_duration)
```

δίνουμε την εντολή να εκκινήσει η προσομοίωση, η οποία θα διαρκέσει χρόνο ίσο με την τιμή της μεταβλητής `sim_duration`.

Στο τέλος αυτού του χρόνου, δηλαδή μετά από $480 \cdot 10 = 4800$ min, η `Salabim` θα καλέσει *κρυφά* (under the hood) τη συνάρτηση `main` για να τερματίσει την προσομοίωση.

Τα components

Τα *components* (δομικά στοιχεία) είναι βασικά συστατικά εντός μοντέλου προσομοίωσης που δημιουργείται με τη βοήθεια της `Salabim`. Ένα *component* είναι είτε *ενεργό* (active) είτε *μη-ενεργό* (data).

Ένα ενεργό *component* έχει μία ή περισσότερες *διεργασίες* (processes), οι οποίες περιγράφονται από μια *γεννήτρια συνάρτηση* (generator function), ενεργοποιείται δε κάποια στιγμή κατά τη διάρκεια του χρόνου της προσομοίωσης.

Η γεννήτρια συνάρτηση περιλαμβάνει τουλάχιστον μια εντολή `yield`.

Ένα ενεργό *component* γίνεται μη-ενεργό με την εντολή `cancel` ή όταν φθάσει στο τέλος της γεννήτριας συνάρτησης.

Ένα μη-ενεργό *component* γίνεται ενεργό με την εντολή `activate`.

Στο μοντέλο του συστήματος φόρτωσης που προσομοιώνουμε, είναι προφανές ότι τα ενεργά *components* είναι τα ΦΑ και ο φορτωτής.

Για τα *components* αυτά, στο σώμα του κώδικά μας έχουμε δημιουργήσει δύο δικές μας κλάσεις, την `Truck` και τη `Loader`, οι οποίες *κληρονομούν* (inherit) τις ιδιότητες και μεθόδους της κλάσης `Component`

της Salabim. Οι δύο αυτές κλάσεις μας *περιγράφουν* την κίνηση των φορτηγών αυτοκινήτων και τη λειτουργία του φορτωτή, αντίστοιχα, κατά τη διάρκεια της προσομοίωσης του μοντέλου.

Για τη διευκόλυνση του αναγνώστη παραθέτουμε ξανά τον κώδικα αυτών των δύο κλάσεων:

```
class Truck(sim.Component):
    def process(self):
        self.enter(loading_queue)
        if loader.ispassive():
            loader.activate()
        yield self.passivate()

class Loader(sim.Component):
    def process(self):
        while True:
            while len(loading_queue) == 0:
                yield self.passivate()
            self.truck = loading_queue.pop()
            yield self.hold(sim.Triangular(8, 15, 12).sample())
            env.trucks_loaded += 1
            truck_load = sim.Normal(30, 1).sample()
            env.aggregates_tonnage += truck_load
            self.truck.activate()
```

Η προσομοίωση έχει την παρακάτω ροή:

Μετά την έναρξη της προσομοίωσης (και της βάρδιας), το `truck.0` φτάνει για φόρτωση στον χρόνο 14.816 min. Ο χρόνος αυτός προκύπτει από τη δειγματοληψία της κατανομής `sim.Uniform(10, 15)` στη γραμμή κώδικα:

```
sim.ComponentGenerator(Truck, iat=sim.Uniform(10, 15))
```

Οι χρόνοι διακριτών γεγονότων στους οποίους λαμβάνουν χώρα μεταβολές στο σύστημα φόρτωσης (timeline) κατά την προσομοίωσή του, εμφανίζονται στην Εικόνα 18.7, όπου παρουσιάζονται οι δύο πρώτες σελίδες των καταγραφών του trace log.

Το πλήρες trace log, έκτασης 211 σελίδων, της προσομοίωσης του συστήματος φόρτωσης για 10 8ωρες βάρδιες (4800 min) υπάρχει στο website του του βιβλίου, στο αρχείο:

Chapter_18_TRs_LO_as_Component_Trace_log.pdf

Αμέσως, το `truck.0` εισέρχεται στο τέλος της ουράς αναμονής με την εντολή:

```
self.enter(loading_queue)
```

και ελέγχει εάν ο φορτωτής (`loader`) είναι διαθέσιμος (`ispassive`) για φόρτωση με το μπλοκ `if`:

```
if loader.ispassive():
    loader.activate()
```

εάν ναι, τότε το `truck.0` ενεργοποιεί-δεσμεύει (`activate`) τον `loader` για φόρτωση.

Εάν ο φορτωτής δεν είναι διαθέσιμος για κάποιο λόγο, π.χ. φορτώνει ένα προπορευόμενο ΦΑ, τότε ένα `truck` με την εντολή:

```
yield self.passivate()
```

μπαίνει σε κατάσταση αδράνειας.

Ο φορτωτής, με την εντολή:

```
self.truck = loading_queue.pop()
```

βγάζει από την ουρά αναμονής το `truck.0` ή το εκάστοτε πρώτο φορτηγό αυτοκίνητο, εάν περιμένουν περισσότερα στην ουρά αναμονής, στη λογική του FIFO (First In, First Out), και με την εντολή:

```
yield self.hold(sim.Triangular(8, 15, 12).sample())
```

το κρατάει για χρονικό διάστημα 12.035 min, που αντιστοιχεί στον χρόνο φόρτωσης. Ο χρόνος φόρτωσης προκύπτει ύστερα από τη δειγματοληψία της τριγωνικής κατανομής `sim.Triangular(8, 15, 12)`

Όταν παρέλθει αυτός ο χρόνος, δηλαδή σε χρόνο $14.816 + 12.035 = 26.851$ min μετά την έναρξη της προσομοίωσης, τότε:

Η μεταβλητή `env.trucks_loaded` αυξάνει την τιμή της κατά μία μονάδα.

Η μεταβλητή `env. env.aggregates_tonnage` αυξάνει την τιμή της κατά την ποσότητα αδρανών, που προκύπτει ύστερα από τη δειγματοληψία της κανονικής κατανομής `sim.Normal(30, 1)`.

Ενεργοποιείται ξανά το `truck.0` με την εντολή:

```
self.truck.activate()
```

και φορτωμένο με αδρανή φεύγει από το λατομείο, ενώ παράλληλα τερματίζεται και η συμμετοχή του στην προσομοίωση.

Εν τω μεταξύ, σε χρόνο 11.179 min μετά την άφιξη του `truck.0`, δηλαδή σε χρόνο $14.816 + 11.179 = 25.994$ min μετά την έναρξη της προσομοίωσης, φτάνει στο λατομείο για φόρτωση με την εντολή:

line#	time	current	component	action	information

				line numbers refers to	Chapter18_TRs-LO_as_Component.py
27				default environment initialize	
27				main create	
27	0.000	main		current	
28				Truck.generator.0 create	
28				Truck.generator.0 activate +14.816	scheduled for 14.816 process=do_iat
29				loader.0 create	
29				loader.0 activate	scheduled for 0.000 @ 14 process=process
30				Queue for loading create	
33				main run +4800.000	scheduled for 4800.000 @ 33+
14	0.000	loader.0		current	
17				loader.0 passivate	
28	14.816	Truck.generator.0		current	
33				truck.0 create	
33				truck.0 activate	scheduled for 14.816 @ 6 process=process
33				Truck.generator.0 hold +11.179	scheduled for 25.994
6	14.816	truck.0		current	
7				truck.0	enter Queue for loading
9				loader.0 activate	scheduled for 14.816 @ 17+
10				truck.0 passivate	
17+	14.816	loader.0		current	
18				truck.0	leave Queue for loading
19				loader.0 hold +12.035	scheduled for 26.851 @ 19+
28	25.994	Truck.generator.0		current	
33				truck.1 create	
33				truck.1 activate	scheduled for 25.994 @ 6 process=process
33				Truck.generator.0 hold +11.134	scheduled for 37.128
6	25.994	truck.1		current	
7				truck.1	enter Queue for loading
10				truck.1 passivate	
19+	26.851	loader.0		current	
23				truck.0 activate	scheduled for 26.851 @ 10+
18				truck.1	leave Queue for loading
19				loader.0 hold +12.117	scheduled for 38.968 @ 19+
10+	26.851	truck.0		current	
10+				truck.0 ended	
28	37.128	Truck.generator.0		current	
33				truck.2 create	
33				truck.2 activate	scheduled for 37.128 @ 6 process=process
33				Truck.generator.0 hold +13.925	scheduled for 51.053
6	37.128	truck.2		current	
7				truck.2	enter Queue for loading
10				truck.2 passivate	
19+	38.968	loader.0		current	
23				truck.1 activate	scheduled for 38.968 @ 10+
18				truck.2	leave Queue for loading
19				loader.0 hold +13.966	scheduled for 52.934 @ 19+
10+	38.968	truck.1		current	
10+				truck.1 ended	
28	51.053	Truck.generator.0		current	

Εικόνα 18.7. Απόσπασμα του αρχείου Chapter_18_TRs_LO_as_Component_Trace_log.pdf

```
sim.ComponentGenerator(Truck, iat=sim.Uniform(10, 15))
```

και εισέρχεται στο τέλος της ουράς αναμονής, το δεύτερο ΦΑ, το `truck.1`, το οποίο, επειδή ο φορτωτής είναι δεσμευμένος φορτώνοντας το `truck.0`, αναμένει στην ουρά τόσο χρόνο, όσος χρειάζεται για να τελειώσει η φόρτωση του προπορευόμενου `truck.0`, δηλαδή μέχρις ο χρόνος να γίνει 26.851 min.

Επομένως, ο χρόνος αναμονής του `truck.1` θα είναι $26.851 - 25.994 = 0.857$ min.

Μετά την παρέλευση του χρόνου αναμονής, το `truck.1` θα ακολουθήσει την ανάλογη διαδικασία που ακολούθησε το `truck.0`

Σημείωση: Οι τιμές των χρόνων που παρατέθηκαν παραπάνω συμβαδίζουν με τις τιμές χρόνου στο timeline του trace log της Salabim. Τυχόν διαφορές στο τελευταίο δεκαδικό ψηφίο (π.χ. 25.994 αντί του *ορθού* 25.995) οφείλονται σε στρογγυλοποιήσεις των δεκαδικών ψηφίων του ρολογιού της προσομοίωσης.

Επίσης παρατηρήστε στον κώδικα της κλάσης `Loader (sim.Component)` τις εντολές:

```
while len(loading_queue) == 0:  
    yield self.passivate()
```

οι οποίες βάζουν τον φορτωτή σε κατάσταση αδράνειας, όταν το μήκος της ουράς αναμονής είναι 0, δηλαδή όταν δεν υπάρχει κανένα ΦΑ διαθέσιμο για φόρτωση.

Κάτω από τη γραμμή σχολίου `# Simulation output` με τις συνήθεις εντολές `print()` εμφανίζουμε στην οθόνη τα αποτελέσματα της προσομοίωσης:

SIMULATION REPORT

Simulation time [8-hour shifts] = 10

Number of trucks arrived for loading = 388

Number of loaded trucks = 387

Loaded aggregates [t] = 11612.58

Loader utilization [%] = 94.26

Queue for loading:

Length [trucks] minimum = 1, median = 1, maximum = 1

Length of stay [min] minimum = 0.00, median = 0.84, maximum = 11.32

Το τμήμα του κώδικα που δίνει τα αποτελέσματα της προσομοίωσης είναι εύκολα κατανοητό. Θα σχολιάσουμε μόνο τη γραμμή εντολής:

```
utilization = (1 -  
                (loader.status.value_duration("passive")/sim_duration))*100
```

η οποία υπολογίζει το ποσοστό του χρόνου προσομοίωσης κατά τον οποίο ο φορτωτής είναι απασχολημένος φορτώνοντας τα ΦΑ (utilization).

Η μέθοδος `loader.status.value_duration("passive")` μας δίνει τον συνολικό χρόνο κατά τον οποίο ο φορτωτής ήταν αδρανής (*passive*) κατά τη διάρκεια της προσομοίωσης.

Επίσης, στην παρουσίαση των στατιστικών στοιχείων (ελάχιστο, διάμεσος, μέγιστο) του αριθμού των ΦΑ που βρίσκονται στην ουρά αναμονής (`loading_queue.length`) και του χρόνου παραμονής σε αυτήν (`loading_queue.length_of_stay`) χρησιμοποιήσαμε την παράμετρο `ex0=True`, η οποία δεν λαμβάνει υπόψη στον υπολογισμό των στατιστικών στοιχείων τα ΦΑ που είχαν μηδενικό χρόνο παραμονής.

Τέλος, παρατηρήστε στον πίνακα των αποτελεσμάτων της προσομοίωσης ότι όταν τέλειωσε ο χρόνος προσομοίωσης, στην ουρά αναμονής υπήρχε ένα ΦΑ που περίμενε να φορτωθεί.

Το script του κώδικα που κάνει την προσομοίωση του συστήματος φόρτωσης των ΦΑ, θεωρώντας τον φορτωτή ως `component`, βρίσκεται στο website του βιβλίου στο αρχείο:

`Chapter_18_TRs_LO_as_Component.py`

18.9.2 Προσομοίωση του φορτωτή ως resource

Στο μοντέλο προσομοίωσης του συστήματος φόρτωσης των φορτηγών αυτοκινήτων στο λατομείο, στην παράγραφο 18.9.1 *Προσομοίωση του φορτωτή ως Component*, θεωρήσαμε ότι ο φορτωτής είναι ένα component. Μπορούμε όμως να αναπτύξουμε το μοντέλο μας και να είναι ίσως πιο εύκολα κατανοητό, θεωρώντας ότι ο φορτωτής είναι ένα resource.

Στη βιβλιοθήκη Salabim τα *resources* είναι *εξυπηρετητές* (servers), οι οποίοι έχουν περιορισμένη *χωρητικότητα* (capacity) και μπορούν να εξυπηρετήσουν ταυτόχρονα τόσες μονάδες, οι οποίες τους ζητούν εξυπηρέτηση (components), όσες αντιστοιχούν στη χωρητικότητά τους.

Δηλαδή, εάν η θέση φόρτωσης στο λατομείο έχει έναν φορτωτή, τότε η χωρητικότητα του resource-φορτωτή είναι 1 και μπορεί να εξυπηρετήσει ένα (1) ΦΑ, ενώ εάν στη θέση φόρτωσης υπάρχουν δύο φορτωτές, τότε η χωρητικότητα του resource είναι 2 και μπορεί να εξυπηρετήσει δύο (2) ΦΑ ταυτόχρονα.

Γράφουμε τις παρακάτω γραμμές κώδικα:

```
import salabim as sim
number_of_shifts = 10
sim_duration = 480*number_of_shifts

class Truck(sim.Component):
    def process(self):
        self.enter(loading_queue)
        yield self.request(loader)
        self.leave(loading_queue)
        yield self.hold(sim.Triangular(8, 15, 12).sample())
        env.trucks_loaded += 1
        truck_load = sim.Normal(30, 1).sample()
        env.loaded_aggregates += truck_load
        self.release(loader)

# Main body
env = sim.Environment(trace=False)
sim.ComponentGenerator(Truck, lat=sim.Uniform(10, 15))
loading_queue = sim.Queue('Queue for loading')
loader = sim.Resource('Loader', capacity=1)
env.trucks_loaded = 0
env.loaded_aggregates = 0
env.run(duration=sim_duration)

# Simulation output
print('SIMULATION REPORT')
print('Simulation time [8-hour shifts] = {:d}'.format(number_of_shifts))
print('Number of trucks arrived for loading = {:d}'
      .format(loading_queue.number_of_arrivals))
print('Number of loaded trucks = {:d}'.format(env.trucks_loaded))
print('Loaded aggregates [t] = {:.2f}'.format(env.loaded_aggregates))
print('Loader utilization [%] = {:.2f}'.format(loader.occupancy.mean()*100))
print('Queue for loading:')
print(' Length [trucks] minimum = {:.0f}, median = {:.0f}, maximum = {:.0f}'
      .format(loading_queue.length.minimum(ex0=True),
              loading_queue.length.median(ex0=True),
              loading_queue.length.maximum(ex0=True)))
print(' Length of stay [min] minimum = {:.2f}, median = {:.2f}, '
      ' maximum = {:.2f}'.
      format(loading_queue.length_of_stay.minimum(ex0=True),
              loading_queue.length_of_stay.median(ex0=True),
              loading_queue.length_of_stay.maximum(ex0=True)))
```

Ο κώδικας έχει πολλές ομοιότητες με εκείνον της παραγράφου 18.9.1 *Προσομοίωση του φορτωτή ως Component* και για τον λόγο αυτό στη συνέχεια θα σχολιάσουμε τις διαφοροποιήσεις που κάναμε.

Καταρχάς, στον κώδικα κάτω από τη γραμμή σχολίου `# Main body` αντικαταστήσαμε την εντολή:

```
loader = Loader()
```

με την εντολή:

```
loader = sim.Resource('Loader', capacity=1)
```

δηλαδή, δηλώσαμε ότι ο φορτωτής τώρα είναι ένα αντικείμενο της κλάσης `Resource` της `Salabim`, που το ονομάζουμε `loader` και έχει χωρητικότητα 1 γιατί έχουμε έναν φορτωτή στο λατομείο.

Παράλληλα, διαγράφουμε και τον κώδικα της κλάσης `Loader(sim.Component)` γιατί δεν έχει πλέον λόγο ύπαρξης, αφού ο φορτωτής θα μοντελοποιηθεί ως `resource` και όχι ως `component`.

Η κλάση `Truck(sim.Component)` επίσης, έχει τώρα διαφορετική δομή:

```
class Truck(sim.Component):
    def process(self):
        self.enter(loading_queue)
        yield self.request(loader)
        self.leave(loading_queue)
        yield self.hold(sim.Triangular(8, 15, 12).sample())
        env.trucks_loaded += 1
        truck_load = sim.Normal(30, 1).sample()
        env.loaded_aggregates += truck_load
        self.release(loader)
```

Η προσομοίωση ακολουθεί την παρακάτω ροή:

Το `truck.0` φτάνει για φόρτωση σε χρόνο που προκύπτει από την εντολή:

```
sim.ComponentGenerator(Truck, lat=sim.Uniform(10, 15))
```

και αμέσως εισέρχεται στο τέλος της ουράς αναμονής με την εντολή:

```
self.enter(loading_queue)
```

Στη συνέχεια, με την εντολή:

```
yield self.request(loader)
```

αιτείται (request) μια θέση φόρτωσης στον φορτωτή.

Εάν ο φορτωτής δεν είναι διαθέσιμος, δηλαδή φορτώνει ένα άλλο ΦΑ (κάτι που βέβαια δεν ισχύει στην περίπτωση του πρώτου ΦΑ `truck.0`), τότε περιμένει στην ουρά αναμονής μέχρις ότου ο φορτωτής γίνει διαθέσιμος.

Όταν ο φορτωτής γίνει διαθέσιμος, το φορτηγό αυτοκίνητο τον δεσμεύει και εγκαταλείπει την ουρά αναμονής με την εντολή:

```
self.leave(loading_queue)
```

Με την εντολή:

```
yield self.hold(sim.Triangular(8, 15, 12).sample())
```

ο φορτωτής κρατάει το ΦΑ για χρονικό διάστημα που αντιστοιχεί στον χρόνο φόρτωσης, ο οποίος προκύπτει από τη δειγματοληψία της τριγωνικής κατανομής `sim.Triangular(8, 15, 12)`

Στη συνέχεια, ενημερώνονται με τις ανάλογες τιμές οι μεταβλητές `env.trucks_loaded` και `env.aggregates_tonnage` και τέλος, με την εντολή:

```
self.release(loader)
```

αποδεσμεύεται ο φορτωτής από το `truck.0` και είναι πλέον διαθέσιμος να δεχθεί το επόμενο ΦΑ.

Κάτω από τη γραμμή σχολίου `# Simulation output`, η μόνη αλλαγή που κάναμε είναι στον υπολογισμό του ποσοστού απασχόλησης του φορτωτή (utilization), ο οποίος στην περίπτωση των Resources υπολογίζεται με τη βοήθεια της μεθόδου `loader.occupancy.mean()`.

Εάν τρέξουμε τον κώδικα, στην οθόνη εμφανίζονται τα αποτελέσματα της προσομοίωσης, τα οποία είναι ακριβώς τα ίδια με τα αποτελέσματα του κώδικα που παρουσιάστηκε στην παράγραφο 18.9.1 *Προσομοίωση του φορτωτή ως Component*.

Το πλήρες trace log, έκτασης 234 σελίδων, της προσομοίωσης του συστήματος φόρτωσης για 10 8ωρες βάρδιες (4800 min) υπάρχει στο website του του βιβλίου, στο αρχείο: `Chapter_18_TRs_LO_as_Resource_Trace_log.pdf`

Εάν στο τέλος του κώδικα προσθέσουμε την εντολή:

```
loader.print_statistics()
```

στην οθόνη θα εμφανιστούν τα πλήρη στατιστικά του φορτωτή μετά το πέρας της προσομοίωσης (Εικόνα 18.8).

Εάν στο τέλος του κώδικα προσθέσουμε την εντολή:

```
loading_queue.print_statistics()
```

στην οθόνη θα εμφανιστούν τα πλήρη στατιστικά της ουράς αναμονής των φορτηγών αυτοκινήτων μετά το πέρας της προσομοίωσης (Εικόνα 18.9).

Statistics of Loader at 4800.000		all	excl.zero	zero
-----		-----	-----	-----
Length of requesters of Loader	duration	4800	676.766	4123.234
	mean	0.141	1	
	std.deviation	0.348	0	
	minimum	0	1	
	median	0	1	
	90% percentile	1	1	
	95% percentile	1	1	
	maximum	1	1	
Length of stay in requesters of Loader	entries	388	388	0
	mean	1.744	1.744	
	std.deviation	2.181	2.181	
	minimum	0	0	
	median	0.840	0.840	
	90% percentile	4.775	4.775	
	95% percentile	6.478	6.478	
	maximum	11.321	11.321	

Εικόνα 18.8. Τα στατιστικά του φορτωτή μετά το πέρας της προσομοίωσης.

Length of claimers of Loader	duration	4800	4524.720	275.280
	mean	0.943	1	
	std.deviation	0.233	0	
	minimum	0	1	
	median	1	1	
	90% percentile	1	1	
	95% percentile	1	1	
	maximum	1	1	
Length of stay in claimers of Loader	entries	387	387	0
	mean	11.687	11.687	
	std.deviation	1.446	1.446	
	minimum	8.297	8.297	
	median	11.843	11.843	
	90% percentile	13.527	13.527	
	95% percentile	13.950	13.950	
	maximum	14.667	14.667	
Capacity of Loader	duration	4800	4800	0
	mean	1	1	
	std.deviation	0	0	
	minimum	1	1	
	median	1	1	
	90% percentile	1	1	
	95% percentile	1	1	
	maximum	1	1	
Available quantity of Loader	duration	4800	275.280	4524.720
	mean	0.057	1	
	std.deviation	0.233	0	
	minimum	0	1	
	median	0	1	
	90% percentile	0	1	
	95% percentile	1	1	
	maximum	1	1	

Εικόνα 18.8 (Συνέχεια). Τα στατιστικά του φορτωτή μετά το πέρας της προσομοίωσης.

Claimed quantity of Loader	duration	4800	4524.720	275.280
	mean	0.943	1	
	std.deviation	0.233	0	
	minimum	0	1	
	median	1	1	
	90% percentile	1	1	
	95% percentile	1	1	
	maximum	1	1	
Occupancy of Loader	duration	4800	4524.720	275.280
	mean	0.943	1	
	std.deviation	0.233	0	
	minimum	0	1	
	median	1	1	
	90% percentile	1	1	
	95% percentile	1	1	
	maximum	1	1	

Εικόνα 18.8 (Συνέχεια). Τα στατιστικά του φορτωτή μετά το πέρας της προσομοίωσης.

Statistics of Queue for loading at 4800				
		all	excl.zero	zero
Length of Queue for loading	duration	4800	676.766	4123.234
	mean	0.141	1	
	std.deviation	0.348	0	
	minimum	0	1	
	median	0	1	
	90% percentile	1	1	
	95% percentile	1	1	
	maximum	1	1	
Length of stay in Queue for loading	entries	388	388	0
	mean	1.744	1.744	
	std.deviation	2.181	2.181	
	minimum	0	0	
	median	0.840	0.840	
	90% percentile	4.775	4.775	
	95% percentile	6.478	6.478	
	maximum	11.321	11.321	

Εικόνα 18.9. Τα στατιστικά της ουράς αναμονής των φορτηγών αυτοκινήτων μετά το πέρας της προσομοίωσης.

Το script του κώδικα που κάνει την προσομοίωση του συστήματος φόρτωσης των ΦΑ, θεωρώντας τον φορτωτή ως *resource*, βρίσκεται στο website του βιβλίου στο αρχείο:
`Chapter_18_TRs_LO_as_Resource.py`

18.9.3 Προσομοίωση της φόρτωσης με πολλούς φορτωτές

Ο χειριστής του φορτωτή του λατομείου, που προσομοιώσαμε στην παράγραφο 18.9.2 *Προσομοίωση του φορτωτή ως resource*, διαμαρτύρεται προς τη διεύθυνση του λατομείου ότι οι συνθήκες εργασίας του είναι εξοντωτικές.

Πράγματι, από τα αποτελέσματα της προσομοίωσης προκύπτει ότι το ποσοστό χρησιμοποίησης του φορτωτή (utilization) είναι 94.26%, το οποίο σημαίνει ότι ο χειριστής του στην 8ωρη βάρδια έχει στη διάθεσή του μόνο 27.55 min για να *χαλαρώσει*, γιατί τον υπόλοιπο χρόνο της βάρδιας φορτώνει συνεχώς ΦΑ.

Η διεύθυνση προτείνει στον Μεταλλειολόγο του λατομείου να εξετάσει την περίπτωση χρησιμοποίησης και ενός δεύτερου φορτωτή στη φόρτωση των αυτοκινήτων με αδρανή. Ένας τέτοιος φορτωτής υπάρχει στο λατομείο και επιτυγχάνει τους ίδιους χρόνους φόρτωσης με τον ήδη χρησιμοποιούμενο.

Ο Μεταλλειολόγος αλλάζει στον κώδικα του μοντέλου προσομοίωσης, που παρουσιάστηκε στην παράγραφο 18.9.2 *Προσομοίωση του φορτωτή ως resource*, την εντολή:

```
loader = sim.Resource('Loader', capacity=1)
```

με την:

```
loader = sim.Resource('Loader', capacity=2)
```

ξανατρέχει το πρόγραμμα και διαπιστώνει, από τα αποτελέσματα της προσομοίωσης του νέου μοντέλου, ότι τώρα το ποσοστό χρησιμοποίησης των φορτωτών (utilization) είναι 46.69%:

Loader utilization [%] = 46.69

Η προσθήκη δεύτερου φορτωτή μάλλον δεν φαίνεται να είναι λογική. Ίσως η αντικατάσταση του χειριστή του μοναδικού φορτωτή, κατά τη διάρκεια των διαλειμμάτων ανάπαυσης, από έναν άλλον χειριστή να είναι η καλύτερη λύση.

Στον κώδικά μας, με την εντολή:

```
yield self.request(loader)
```

το ΦΑ αιτήθηκε (requested) μια (1) θέση φόρτωσης στον φορτωτή, επειδή η default τιμή της μεθόδου `request` είναι 1.

Ένα component όμως μπορεί να αιτηθεί ταυτόχρονα περισσότερες της μίας θέσεις εξυπηρέτησης από έναν εξυπηρετητή ή και θέσεις από διαφορετικούς εξυπηρετητές.

Για παράδειγμα, εάν προσομοιώναμε την επισκευή μιας μπουλντόζας στο συνεργείο ενός μεταλλείου, με την παρακάτω εντολή, η μπουλντόζα που μπαίνει στο συνεργείο για επισκευή, αιτείται δύο (2) μηχανικούς και έναν (1) ηλεκτρολόγο:

```
yield self.request((mechanic, 2), electrician)
```

Με την εντολή:

```
self.release(mechanic, electrician)
```

αποδεσμεύουμε όλους τους ασχολούμενους με την επισκευή της μπουλντόζας και με την εντολή:

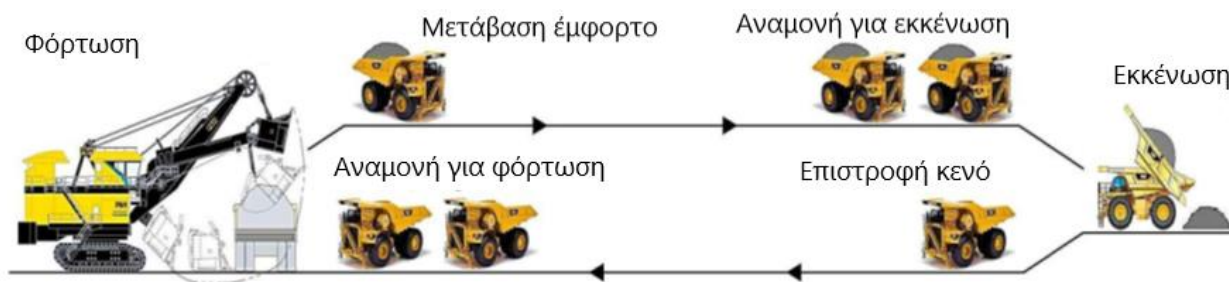
```
self.release((mechanic, 1), electrician)
```

αποδεσμεύουμε μόνο τον έναν μηχανικό και τον ηλεκτρολόγο, ενώ ο άλλος μηχανικός συνεχίζει μόνος του την επισκευή της μπουλντόζας.

18.10 Προσομοίωση συστήματος φόρτωσης-μεταφοράς

Στην παράγραφο αυτή θα προσομοιώσουμε ένα σύνθετο μεταλλευτικό σύστημα φόρτωσης-μεταφοράς, το οποίο αποτελείται από έναν εκσκαφέα και έναν αριθμό χωματουργικών αυτοκινήτων που εργάζονται στη βαθμίδα αποκάλυψης ενός μεταλλείου.

Τα χωματουργικά αυτοκίνητα (ΧΑ) φορτώνονται από τον εκσκαφέα με άγωνα, τα οποία μεταφέρουν στη θέση απόθεσης όπου και εκκενώνουν. Μετά επιστρέφουν κενά στη θέση φόρτωσης. Δεν επιτρέπεται στα ΧΑ να κάνουν προσπεράσεις τόσο κατά τη μετάβαση στη θέση απόθεσης όσο και κατά την επιστροφή τους στη θέση φόρτωσης (Εικόνα 18.10).



Εικόνα 18.10. Σκαρίφημα του συστήματος φόρτωσης-μεταφοράς αγόνων σε μεταλλείο.

Τα δεδομένα του συστήματος φόρτωσης-μεταφοράς που θα προσομοιώσουμε, καθώς και οι μεταβλητές του κώδικα στις οποίες εκχωρούνται τα δεδομένα, δίνονται παρακάτω:

```
# INPUT DATA SECTION
excavator_type = 'CAT 365C' # 4.6 m3
truck_type = 'CAT 773F' # 55 t/35 m3
number_of_trucks = 5
number_of_excavators = 1
number_of_dumps = 2
truck_interarrival_time = 2.5
loading_time = sim.Uniform(3.5, 4.3)
travel_time_excavator_to_dumps = sim.Uniform(5.3, 6.5)
dumping_time = sim.Uniform(1.5, 2.5)
return_time_dumps_to_excavator = sim.Uniform(4.5, 5.5)
truck_load_volume = 30 # average, m3
excavator_mtbtf = sim.Uniform(20, 30) # hours
excavator_mttr = sim.Normal(30, 5) # min
number_of_shifts = 10
sim_duration = 480*number_of_shifts # min
```

Οι παραπάνω γραμμές δεδομένων είναι μέρος του κώδικα του μοντέλου προσομοίωσης (όπως θα δούμε στη συνέχεια), και με αυτόν τον τρόπο γραφής μπορούμε να αλλάζουμε εύκολα τα δεδομένα, χωρίς να επεμβαίνουμε στον κυρίως κώδικα του μοντέλου.

Έτσι μπορούμε να εξετάσουμε διαφορετικά σενάρια διαμόρφωσης του συστήματος φόρτωσης-μεταφοράς, αποφεύγοντας την πιθανότητα λάθους και αλλοίωσης της δομής του κώδικα του μοντέλου.

Εναλλακτικά, θα μπορούσαμε να είχαμε δημιουργήσει ένα παράθυρο γραφικής διεπαφής χρήστη (GUI), ανάλογο εκείνου που παρουσιάστηκε στο Κεφάλαιο 17 *Γραφική διεπαφή χρήστη*.

Το σύστημα φόρτωσης-μεταφοράς, στη διαμόρφωση που θα το προσομοιώσουμε, αποτελείται από έναν (1) εκσκαφέα CAT 365C και πέντε (5) ΧΑ CAT 773F.

Στην απόθεση υπάρχει χώρος, ώστε δύο (2) ΧΑ να μπορούν να εκκενώνουν ταυτόχρονα.

Κατά την έναρξη της προσομοίωσης, δηλαδή την πρώτη φόρτωση των ΧΑ, το πρώτο ΧΑ βρίσκεται ήδη στη θέση φόρτωσης (χρόνος 0) και τα υπόλοιπα τέσσερα φθάνουν διαδοχικά, σε χρόνους που ισαπέχουν 2 . 5 min.

Όλοι οι χρόνοι του κύκλου των ΧΑ, δηλαδή οι χρόνοι φόρτωσης, μετάβασης στην απόθεση, εκκένωσης και επιστροφής από την απόθεση στη θέση φόρτωσης, λαμβάνονται από δειγματοληψίες επί ομοιόμορφων στατιστικών κατανομών, οι παράμετροι των οποίων προέκυψαν από χρονομετρήσεις του κύκλου.

Το κάθε ΧΑ μεταφέρει άγωνα των οποίων ο όγκος είναι κατά μέσον όρο 30 m^3 .

Ο εκσκαφέας παθαίνει βλάβες κάθε 20 με 30 ώρες (είναι λίγο ταλαιπωρημένος...), οι οποίες κατανέμονται ομοιόμορφα στο διάστημα αυτό [mtbf = Mean Time Between Failures].

Η αποκατάσταση καθεμίας βλάβης γίνεται επιτόπου, από κινητό συνεργείο, και διαρκεί χρόνο που λαμβάνεται από τη δειγματοληψία μιας κανονικής κατανομής με μέση τιμή 30 min και τυπική απόκλιση 5 min [mttr = Mean Time To Repair].

Είναι προφανές ότι και τα ΧΑ θα παθαίνουν βλάβες, αλλά δεν τις λαμβάνουμε υπόψη γιατί το μοντέλο μας θα γινόταν περισσότερο περίπλοκο.

Η προσομοίωση θα διαρκέσει 10 8ωρες βάρδιες.

Ακολουθεί ο πλήρης κώδικας του προγράμματος προσομοίωσης του συστήματος φόρτωσης-μεταφοράς:

```
# A Simple Excavator-Trucks Simulation Model in Salabim
import salabim as sim

# INPUT DATA SECTION
excavator_type = 'CAT 365C' # 4.6 m3
truck_type = 'CAT 773F' # 55 t/35 m3
number_of_trucks = 5
number_of_excavators = 1
number_of_dumps = 2
truck_interarrival_time = 2.5
loading_time = sim.Uniform(3.5, 4.3)
travel_time_excavator_to_dumps = sim.Uniform(5.3, 6.5)
dumping_time = sim.Uniform(1.5, 2.5)
return_time_dumps_to_excavator = sim.Uniform(4.5, 5.5)
truck_load_volume = 30 # average, m3
excavator_mtbf = sim.Uniform(20, 30) # hours
excavator_mttr = sim.Normal(30, 5) # min
number_of_shifts = 10
sim_duration = 480*number_of_shifts

class TruckGenerator(sim.Component):
    """
    Generates n trucks, then TruckGenerator "ends" and the n trucks
    circulate within the system """

    def process(self):
        for i in range(number_of_trucks):
            Truck(name="TR,",)
            yield self.hold(truck_interarrival_time)

class Truck(sim.Component):
    """
    Controls trucks into the load-travel-dump-return cycle """

    def process(self):
        while True:
            self.enter(queue_for_loading)
            yield self.request((excavator, 1))
```

```

self.leave(queue_for_loading)
yield self.hold(loading_time.sample())
self.release((excavator, 1))
env.number_of_trucks_loaded += 1
self.enter(travelling_excavator_to_dumps)
yield self.hold(travel_time_excavator_to_dumps.sample())
self.leave(travelling_excavator_to_dumps)
self.enter(queue_for_dumping)
yield self.request((dumps, 1))
self.leave(queue_for_dumping)
yield self.hold(dumping_time.sample())
self.release((dumps, 1))
env.number_of_trucks_dumped += 1
env.dumps_volume += truck_load_volume
self.enter(returning_dumps_to_excavator)
yield self.hold(return_time_dumps_to_excavator.sample())
self.leave(returning_dumps_to_excavator)

```

```

class ExcavatorFailure(sim.Component):
    """ Generates failures for the excavator """

    def process(self):
        while True:
            if excavator_mtbtf == 0:
                break
            else:
                yield self.hold(excavator_mtbtf.sample()*60)
                excavator_status.set('DOWN')
                excavator.set_capacity(0)
                env.excavator_failures_number += 1
                truck_loading = excavator.claimers()[0]
                if truck_loading:
                    truck_loading.interrupt()
                excavator_mttr_sample = sim.Bounded(excavator_mttr,
                                                    lowerbound=0).sample()

                yield self.hold(excavator_mttr_sample)
                excavator_status.set('UP')
                excavator.set_capacity(1)
                env.excavator_failures_time += excavator_mttr_sample
                if truck_loading:
                    truck_loading.resume()

```

```

# Main body
env = sim.Environment(trace=True)
env.number_of_trucks_loaded = 0
env.number_of_trucks_dumped = 0
env.dumps_volume = 0
env.excavator_failures_number = 0
env.excavator_failures_time = 0
queue_for_loading = sim.Queue('Queue for Loading')
queue_for_dumping = sim.Queue('Queue for Dumping')
travelling_excavator_to_dumps = sim.Queue('Travelling from Excavator to Dumps')
returning_dumps_to_excavator = sim.Queue('Returning from Dumps to Excavator')
excavator = sim.Resource('Excavator', capacity=number_of_excavators)
dumps = sim.Resource('Dumps', capacity=number_of_dumps)
excavator_status = sim.State('Excavator Status', value='UP')
TruckGenerator()

```

```

ExcavatorFailure()
env.run(duration=sim_duration)
# Simulation output
print('SIMULATION REPORT')
print('Excavator type: {:s}'.format(excavator_type))
print('Truck type: {:s}'.format(truck_type))
print('Number of Excavators = {:d}, Trucks = {:d}'
      .format(number_of_excavators, number_of_trucks))
print('Simulation time [8-hour shifts] = {:d}'
      .format(number_of_shifts))
print('Number of trucks loaded = {:d} ({:.2f}/h)'
      .format(env.number_of_trucks_loaded,
              (env.number_of_trucks_loaded/sim_duration)*60))
print('Number of trucks dumped = {:d} ({:.2f}/h)'
      .format(env.number_of_trucks_dumped,
              (env.number_of_trucks_loaded/sim_duration)*60))
print('Volume of waste dumped [m3] = {:.2f} ({:.2f} m3/h)'
      .format(env.dumps_volume, (env.dumps_volume/sim_duration)*60))
print('Excavator utilization [%] = {:.2f}'
      .format(excavator.occupancy.mean()*100))
print('Number of Excavator failures = {:d}'
      .format(env.excavator_failures_number))
print('Time of Excavator failures [min] = {:.2f}'
      .format(env.excavator_failures_time))
print('Queue for loading:')
print(' Length [trucks] minimum = {:.0f}, median = {:.0f}, maximum = {:.0f}'
      .format(queue_for_loading.length.minimum(ex0=True),
              queue_for_loading.length.median(ex0=True),
              queue_for_loading.length.maximum(ex0=True)))
print(' Length of stay [min] minimum = {:.2f}, mean = {:.2f},'
      ' maximum = {:.2f}'
      .format(queue_for_loading.length_of_stay.minimum(ex0=True),
              queue_for_loading.length_of_stay.mean(ex0=True),
              queue_for_loading.length_of_stay.maximum(ex0=True)))

```

Όλες οι εντολές που βρίσκονται κάτω από τη γραμμή σχολίου # Main body, είναι ήδη γνωστές, εκτός από την εντολή:

```
excavator_status = sim.State('Excavator Status', value='UP')
```

Η κλάση State της Salabim μας επιτρέπει να ορίζουμε μια κατάσταση, η οποία αποκτά μια συγκεκριμένη τιμή κάποια χρονική στιγμή, κατά τη διάρκεια του χρόνου προσομοίωσης.

Στον κώδικά μας, η κατάσταση αυτή εκχωρείται στη μεταβλητή excavator_status και αντιπροσωπεύει το εάν ο εκσκαφέας μπορεί να εργαστεί (UP) ή αδυνατεί λόγω βλάβης (DOWN). Κατά τον ορισμό της δίνουμε και την αρχική τιμή, που στην περίπτωση μας είναι UP, δηλαδή κατά την έναρξη της προσομοίωσης ο εκσκαφέας είναι σε θέση να εργαστεί.

Με την εντολή:

```
excavator_status.set('DOWN')
```

αλλάζουμε την τιμή της κατάστασης του εκσκαφέα σε DOWN.

Η κλάση TruckGenerator, για κάθε ΧΑ που έχουμε εντάξει στο σύστημα φόρτωσης-μεταφοράς, καλεί την κλάση Truck, περιμένει να περάσουν 2.5 min και μετά ξανακαλεί την κλάση Truck. Με τον τρόπο αυτό εισάγονται διαδοχικά και τα 5 ΧΑ του συστήματός μας στους διακριτούς χρόνους: 0, 2.5, 5, 7.5 και 10 min από την έναρξη της προσομοίωσης.

Ο ιδιωτισμός στην κλήση της κλάσης Truck:

```
Truck(name="TR, ")
```

μας επιτρέπει να ονοματίζουμε τα 5 ΧΑ ως: TR.1, TR.2, TR.3, TR.4 και TR.5 αντί του: truck.0, truck.1, truck.2, truck.3 και truck.4 που θα ονομαζόντουσαν αυτόματα εάν καλούσαμε την κλάση Truck με τον συνήθη τρόπο, δηλαδή ως:

```
Truck()
```

Αυτός ο τρόπος ονοματοδοσίας στα ΧΑ συμβαδίζει με την *ανθρώπινη αντίληψη*, που έχει συνηθίσει να αρχίζει τη μέτρηση των αντικειμένων από το 1, σε αντίθεση με την Python που αρχίζει τη μέτρηση από το 0. Επίσης, θα μας βολέψει στη συνέχεια όταν θα κάνουμε το animation του μοντέλου προσομοίωσης (βλέπε παράγραφο 18.11 Animation συστήματος φόρτωσης-μεταφοράς).

Η κλάση Truck ελέγχει την κίνηση των ΧΑ κατά τον κύκλο τους, δηλαδή: αναμονή για φόρτωση, φόρτωση, μετάβαση στην απόθεση, αναμονή για απόθεση, απόθεση, επιστροφή για φόρτωση.

Για να εξασφαλίσουμε ότι τα ΧΑ δεν θα κάνουν προσπεράσματα κατά τη μετάβαση και κατά την επιστροφή τους, κάνουμε ένα trik.

Δημιουργούμε δύο *εικονικές* ουρές την travelling_excavator_to_dumps και την returning_dumps_to_excavator, οι οποίες αντιπροσωπεύουν τη διαδρομή μετάβασης στην απόθεση και τη διαδρομή επιστροφής στη θέση φόρτωσης, αντίστοιχα.

Ύστερα από όσα έχουμε αναφέρει κατά την παρουσίαση των μοντέλων προσομοίωσης των συστημάτων φόρτωσης, η κατανόηση του κώδικα της κλάσης Truck δεν πρέπει να παρουσιάζει δυσκολίες.

Η κλάση ExcavatorFailure διαχειρίζεται τις βλάβες που συμβαίνουν στον εκσκαφέα. Στην περίπτωση που δεν επιθυμούμε σε ένα τρέξιμο του μοντέλου προσομοίωσης να περιλαμβάνονται βλάβες, τότε στο # INPUT DATA SECTION θα δηλώσουμε:

```
excavator_mtbtf = 0 # hours
excavator_mttr = 0 # min
```

και με τον κλάδο if του μπλοκ if-else:

```
if excavator_mtbtf == 0:
    break
```

τερματίζουμε τη λειτουργία της κλάσης ExcavatorFailure.

Εάν θέλουμε ο εκσκαφέας να παθαίνει βλάβες, τότε ενεργοποιείται ο κλάδος else του μπλοκ:

```
else:
    yield self.hold(excavator_mtbtf.sample()*60)
    excavator_status.set('DOWN')
    excavator.set_capacity(0)
    env.excavator_failures_number += 1
    truck_loading = excavator.claimers()[0]
    if truck_loading:
        truck_loading.interrupt()
    excavator_mttr_sample = sim.Bounded(excavator_mttr,
                                         lowerbound=0).sample()
    yield self.hold(excavator_mttr_sample)
    excavator_status.set('UP')
    excavator.set_capacity(1)
    env.excavator_failures_time += excavator_mttr_sample
    if truck_loading:
        truck_loading.resume()
```

Με την πρώτη γραμμή κώδικα περιμένουμε να παρέλθει ο χρόνος από την αρχή της προσομοίωσης ή, εάν δεν είναι η πρώτη βλάβη, ο χρόνος από την εμφάνιση της προηγούμενης βλάβης, για να συμβεί βλάβη στον εκσκαφέα. Όταν έρθει αυτή η χρονική στιγμή:

- η κατάσταση του εκσκαφέα λαμβάνει την τιμή DOWN,

- η χωρητικότητα του εκσκαφέα λαμβάνει την τιμή 0, δηλαδή δεν είναι πλέον ικανός να φορτώνει ΧΑ,
- η μεταβλητή-μετρητής του αριθμού των βλαβών του εκσκαφέα αυξάνει την τιμή της κατά 1.

Με τη γραμμή κώδικα:

```
truck_loading = excavator.claimers()[0]
```

εκχωρούμε στη μεταβλητή `truck_loading` το όνομα του ΧΑ που βρίσκεται πρώτο στη σειρά των ΧΑ τα οποία *διεκδικούν μια θέση* φόρτωσης από τον εκσκαφέα (`claimers`).

Αυτό το ΧΑ μπορεί να βρίσκεται είτε σε κατάσταση άδειου ΧΑ πριν την έναρξη της φόρτωσης είτε να έχει ξεκινήσει, όταν έπαθε βλάβη ο εκσκαφέας, η φόρτωσή του και να είναι μερικώς φορτωμένο. Στη δεύτερη περίπτωση, οι εντολές:

```
if truck_loading:
    truck_loading.interrupt()
```

διακόπτουν τη φόρτωση αυτού του ΧΑ.

Με τη γραμμή κώδικα:

```
excavator_mttr_sample = sim.Bounded(excavator_mttr,
                                     lowerbound=0).sample()
```

εκχωρούμε στη μεταβλητή `excavator_mttr_sample` τον χρόνο αποκατάστασης της βλάβης και επειδή η δειγματοληψία του χρόνου αυτού γίνεται από μια κανονική τιμή, δεχόμαστε μόνο τις θετικές τιμές (βλέπε παράγραφο 18.12.1 *Η χρήση της κανονικής κατανομής*).

Στη συνέχεια, περιμένουμε να παρέλθει ο χρόνος που απαιτείται για την επισκευή του εκσκαφέα και όταν έρθει αυτή η χρονική στιγμή:

- η κατάσταση του εκσκαφέα λαμβάνει την τιμή UP,
- η χωρητικότητα του εκσκαφέα λαμβάνει την τιμή 1, δηλαδή είναι πλέον σε θέση να φορτώνει ΧΑ,
- στη μεταβλητή-μετρητή του χρόνου των βλαβών του εκσκαφέα προστίθεται η τιμή της μεταβλητής `excavator_mttr_sample`.

Τέλος, με τις εντολές:

```
if truck_loading:
    truck_loading.resume()
```

ξεναρχίζει η φόρτωση του ΧΑ, που είχε διακοπεί λόγω της βλάβης του εκσκαφέα, για το υπόλοιπο του χρόνου φόρτωσής του.

Κάτω από τη γραμμή σχολίου `# Simulation output`, με τις συνήθεις εντολές `print()` εμφανίζουμε στην οθόνη τα αποτελέσματα της προσομοίωσης:

SIMULATION REPORT

Excavator type: CAT 365C

Truck type: CAT 773F

Number of Excavators = 1, Trucks = 5

Simulation time [8-hour shifts] = 10

Number of trucks loaded = 1208 (15.10/h)

Number of trucks dumped = 1205 (15.10/h)

Volume of waste dumped [m³] = 36150.00 (451.88 m³/h)

Excavator utilization [%] = 97.93

Number of Excavator failures = 3

Time of Excavator failures [min] = 99.46

Queue for loading:

Length [trucks] minimum = 1, median = 1, maximum = 4

Length of stay [min] minimum = 0.00, mean = 2.98, maximum = 41.09

Στο website του βιβλίου υπάρχει το αρχείο `Chapter_18_TRs_EX_Mine_Trace_log.pdf` με το πλήρες trace log, έκτασης 128 σελίδων, της προσομοίωσης του συστήματος φόρτωσης-μεταφοράς για τις πρώτες οκτώ (8) ώρες προσομοίωσης (480 min).

Το script του κώδικα που κάνει την προσομοίωση του συστήματος φόρτωσης-μεταφοράς βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_18_TRs_EX_Mine.py`

18.11 Animation συστήματος φόρτωσης-μεταφοράς

Η απεικόνιση της λειτουργίας ενός μοντέλου προσομοίωσης υπό μορφή *κινούμενων σχεδίων* (animation), δεν γίνεται για λόγους εντυπωσιασμού ή επίδειξης προγραμματιστικών ικανοτήτων, αλλά για καθαρά πρακτικούς-χρηστικούς λόγους.

Το animation είναι ένα δυνατό εργαλείο στα χέρια του προγραμματιστή για την εκσφαλμάτωση του κώδικα και τον έλεγχο της ορθής λειτουργίας του μοντέλου προσομοίωσης.

Επίσης, εξυπηρετεί στην παρουσίαση των αποτελεσμάτων διαφορετικών εναλλακτικών σεναρίων διαμόρφωσης ενός συστήματος, το οποίο μελετήθηκε με τη μέθοδο της προσομοίωσης, σε στελέχη της βιομηχανίας τα οποία λαμβάνουν αποφάσεις, αλλά δεν διαθέτουν τεχνικό υπόβαθρο.

Και επειδή, συνήθως, ισχύει το κατά τον Ευριπίδη «*σὺν μυρίοισι τὰ καλὰ γίνεται πόνοις*» (δηλαδή, τα καλά κόποις κτώνται), η ανάπτυξη του κώδικα προσομοίωσης με δυνατότητα animation είναι κοπιώδης και απαιτεί τη συστηματική εφαρμογή της μεθόδου της *δοκιμής και σφάλματος* (trial and error). Επιπλέον, απαιτεί και κάποιες βασικές σχεδιαστικές ικανότητες στην οθόνη του υπολογιστή.

Η βιβλιοθήκη Salabim περιλαμβάνει μια συλλογή κλάσεων και συναρτήσεων (animation engine), που ενσωματώνονται στον κώδικα του μοντέλου προσομοίωσης. Κατ' αυτό τον τρόπο, επιτυγχάνεται ο συγχρονισμός του animation με το *ρολόι της προσομοίωσης* (simulation clock) και έχουμε ένα animation του μοντέλου προσομοίωσης σε *πραγματικό χρόνο* (real time - synchronized).

Στη συνέχεια, ακολουθεί ο κώδικας που προσομοιώνει και ταυτόχρονα κάνει animation το σύστημα φόρτωσης-μεταφοράς που παρουσιάσαμε στην παράγραφο 18.10 *Προσομοίωση συστήματος φόρτωσης-μεταφοράς*:

```
# A Simple Animated Excavator-Trucks Simulation Model in Salabim
import salabim as sim
```

```
# INPUT DATA SECTION
```

```
excavator_type = 'CAT 365C' # 4.6 m3
```

```
truck_type = 'CAT 773F' # 55 t/35 m3
```

```
number_of_trucks = 5
```

```
number_of_excavators = 1
```

```

number_of_dumps = 2
truck_interarrival_time = 2.5
loading_time = sim.Uniform(3.5, 4.3)
travel_time_excavator_to_dumps = sim.Uniform(5.3, 6.5)
dumping_time = sim.Uniform(1.5, 2.5)
return_time_dumps_to_excavator = sim.Uniform(4.5, 5.5)
truck_load_volume = 30 # average, m3
excavator_mtbtf = sim.Uniform(1, 2) # hours
excavator_mttr = sim.Normal(30, 5) # min
number_of_shifts = 10
sim_duration = 480*number_of_shifts

```

```

class TruckGenerator(sim.Component):
    """
    Generates n trucks, then TruckGenerator 'ends' and the n trucks
    circulate within the system """

    def process(self):
        for i in range(number_of_trucks):
            Truck(name='TR,')
            yield self.hold(truck_interarrival_time)

class Truck(sim.Component):
    """
    Controls trucks into the load-travel-dump-return cycle """

    def animation_objects(self, id):
        a0 = sim.AnimateRectangle(spec=(0, 0, 40, 20), text=self.name(),
                                   fillcolor=id)
        return 50, 0, a0

    def process(self):
        while True:
            self.enter(queue_for_loading)
            yield self.request((excavator, 1))
            self.leave(queue_for_loading)
            yield self.hold(loading_time.sample())
            self.release((excavator, 1))
            env.number_of_trucks_loaded += 1
            self.enter(travelling_excavator_to_dumps)
            yield self.hold(travel_time_excavator_to_dumps.sample())
            self.leave(travelling_excavator_to_dumps)
            self.enter(queue_for_dumping)
            yield self.request((dumps, 1))
            self.leave(queue_for_dumping)
            yield self.hold(dumping_time.sample())
            self.release((dumps, 1))
            env.number_of_trucks_dumped += 1
            env.dumps_volume += truck_load_volume
            self.enter(returning_dumps_to_excavator)
            yield self.hold(return_time_dumps_to_excavator.sample())
            self.leave(returning_dumps_to_excavator)

```

```

class ExcavatorFailure(sim.Component):
    """

```


Generates failures for the excavator """

```
def process(self):
    while True:
        if excavator_mtbtf == 0:
            break
        else:
            yield self.hold(excavator_mtbtf.sample()*60)
            excavator_status.set('DOWN')
            excavator.set_capacity(0)
            env.excavator_failures_number += 1
            truck_loading = excavator.claimers()[0]
            if truck_loading:
                truck_loading.interrupt()
            excavator_mttr_sample = sim.Bounded(excavator_mttr,
                                                lowerbound=0).sample()
            yield self.hold(excavator_mttr_sample)
            excavator_status.set('UP')
            excavator.set_capacity(1)
            env.excavator_failures_time += excavator_mttr_sample
            if truck_loading:
                truck_loading.resume()
```

Animation Section

```
def do_animation():
    # Dumpsite
    sim.AnimateImage('waste_dumpsite_110x110.png', x=900, y=260)
    sim.AnimateText(text=lambda: 'Volume (m3): {:.30f}'
                      .format(env.dumps_volume),
                    x=880, y=260, textcolor='blue')
    sim.AnimateQueue(dumps.requesters(), x=850, y=290, direction='w',
                     title='Queueing for Dumping -->', id='blue')
    sim.AnimateQueue(dumps.claimers(), x=975, y=320, title='', id='blue')
    # Excavator
    sim.AnimateImage('shovel_60x60.gif', x=10, y=230)
    sim.AnimateQueue(excavator.claimers(), x=75, y=245, title='',
                     direction='e', id='blue')
    sim.AnimateText(text=lambda: '{:d}'.format(number_of_excavators),
                    x=15, y=280, textcolor='blue', fontsize=24)
    sim.AnimateText(text=lambda: 'Status: {:s}'
                      .format(excavator_status.get()),
                    x=10, y=220, textcolor='red', fontsize=18)
    sim.AnimateText(text=lambda: 'Utilization (%): {:.52f}'
                      .format(excavator.occupancy.mean() * 100), x=10, y=205,
                    textcolor='blue')
    sim.AnimateText(text=lambda: 'Failures (#): {:d}'
                      .format(env.excavator_failures_number), x=10, y=190,
                    textcolor='blue')
    sim.AnimateText(text=lambda: 'Down Time (min): {:.62f}'
                      .format(env.excavator_failures_time), x=10, y=175,
                    textcolor='blue')
    sim.AnimateQueue(queue_for_loading, x=280, y=180, direction='e',
                     title='<-- Queueing for Loading', id='black')
    # Travel & Return routes
    sim.AnimateQueue(travelling_excavator_to_dumps, x=350, y=290,
                     direction='w',
                     title='Travelling to Dumpsite -->', id='blue')
    sim.AnimateQueue(returning_dumps_to_excavator, x=700, y=180,
```

```

        direction='e',
        title='<-- Returning from Dumpsite',
        id='black')
# Caption
sim.AnimateText(text=lambda: 'Number of Excavators: {:d}'
                .format(number_of_excavators), x=10, y=420,
                textcolor='blue', fontsize=16)
sim.AnimateText(text=lambda: 'Excavator Type: {:s}'
                .format(excavator_type),
                x=250, y=420, textcolor='blue', fontsize=16)
sim.AnimateText(text=lambda: 'Number of Trucks: {:d}'
                .format(number_of_trucks),
                x=10, y=390, textcolor='blue', fontsize=16)
sim.AnimateText(text=lambda: 'Truck Type: {:s}'.format(truck_type),
                x=250, y=390, textcolor='blue', fontsize=16)
# Monitors
sim.AnimateMonitor(
    queue_for_loading.length,
    x=30,
    y=570,
    height=75,
    width=(env.width() - 100)/2,
    horizontal_scale=4,
    title=lambda: 'Queue for Loading - Number of Waiting Trucks: {:d}'
                .format(queue_for_loading.length()))
sim.AnimateMonitor(
    queue_for_loading.length_of_stay,
    x=530,
    y=570,
    height=75,
    width=(env.width() - 100)/2,
    horizontal_scale=4,
    vertical_scale=2,
    title=lambda: 'Queue for Loading - Mean Waiting Time (min): '
                '{:5.2f}'
                .format(queue_for_loading.length_of_stay.mean()))
sim.AnimateMonitor(
    queue_for_dumping.length,
    x=30,
    y=470,
    height=75,
    width=(env.width() - 100)/2,
    horizontal_scale=4,
    title=lambda: 'Queue for Dumping - Number of Waiting Trucks: {:d}'
                .format(dumps.requesters().length()))
sim.AnimateMonitor(
    queue_for_dumping.length_of_stay,
    x=530,
    y=470,
    height=75,
    width=(env.width() - 100)/2,
    horizontal_scale=4,
    vertical_scale=2,
    title=lambda: 'Queue for Dumping - Mean Waiting Time (min): '
                '{:5.2f}'
                .format(dumps.requesters().length_of_stay.mean()))
env.animation_parameters(modelname='Excavator & Trucks Simulation '
                           'Model', animate=True, height=780,
                           video='Video_Excavator_and_Trucks_Simulation_Model.mp4')

```

```

# Main body
env = sim.Environment(trace=False)
env.number_of_trucks_loaded = 0
env.number_of_trucks_dumped = 0
env.dumps_volume = 0
env.excavator_failures_number = 0
env.excavator_failures_time = 0
queue_for_loading = sim.Queue('Queue for Loading')
queue_for_dumping = sim.Queue('Queue for Dumping')
travelling_excavator_to_dumps = sim.Queue('Travelling from Excavator to '
                                           'Dumps')
returning_dumps_to_excavator = sim.Queue('Returning from Dumps to '
                                           'Excavator')
excavator = sim.Resource('Excavator', capacity=number_of_excavators)
dumps = sim.Resource('Dumps', capacity=number_of_dumps)
excavator_status = sim.State('Excavator Status', value='UP')
TruckGenerator()
ExcavatorFailure()
do_animation()
env.run(duration=sim_duration)
env.video(False)
# Simulation output
print('SIMULATION REPORT')
print('Excavator type: {:s}'.format(excavator_type))
print('Truck type: {:s}'.format(truck_type))
print('Number of Excavators = {:d}, Trucks = {:d}'
      .format(number_of_excavators, number_of_trucks))
print('Simulation time [8-hour shifts] = {:d}'
      .format(number_of_shifts))
print('Number of trucks loaded = {:d} ({:.2f}/h)'
      .format(env.number_of_trucks_loaded,
              (env.number_of_trucks_loaded/sim_duration)*60))
print('Number of trucks dumped = {:d} ({:.2f}/h)'
      .format(env.number_of_trucks_dumped,
              (env.number_of_trucks_dumped/sim_duration)*60))
print('Volume of waste dumped [m3] = {:.2f} ({:.2f} m3/h)'
      .format(env.dumps_volume, (env.dumps_volume/sim_duration)*60))
print('Excavator utilization [%] = {:.2f}'
      .format(excavator.occupancy.mean()*100))
print('Number of Excavator failures = {:d}'
      .format(env.excavator_failures_number))
print('Time of Excavator failures [min] = {:.2f}'
      .format(env.excavator_failures_time))
print('Queue for loading:')
print(' Length [trucks] minimum = {:.0f}, median = {:.0f}, maximum = {:.0f}'
      .format(queue_for_loading.length.minimum(ex0=True),
              queue_for_loading.length.median(ex0=True),
              queue_for_loading.length.maximum(ex0=True)))
print(' Length of stay [min] minimum = {:.2f}, mean = {:.2f}, '
      ' maximum = {:.2f}'
      .format(queue_for_loading.length_of_stay.minimum(ex0=True),
              queue_for_loading.length_of_stay.mean(ex0=True),
              queue_for_loading.length_of_stay.maximum(ex0=True)))

```

Καταρχάς, παρατηρούμε ότι ο κώδικας περιλαμβάνει αυτούσιες τις εντολές του προγράμματος που παρουσιάσαμε στην παράγραφο 18.10 Προσομοίωση συστήματος φόρτωσης-μεταφοράς και έχει, επιπλέον, τις εντολές που θα κάνουν το animation του μοντέλου μας.

Στη συνέχεια, θα παρουσιάσουμε τις εντολές animation του κώδικα, αλλά η παρουσίαση δεν θα είναι εκτενής, ιδίως σε ό,τι αφορά το εύρος των τιμών των παραμέτρων που δέχονται οι εντολές αυτές.

Για λεπτομέρειες, ο αναγνώστης θα πρέπει να ανατρέξει στην τεκμηρίωση της βιβλιοθήκης Salabim:
<https://www.salabim.org/manual/Animation.html> και
<https://www.salabim.org/manual/Reference.html#animation>.

Εντός της κλάσης `Truck` ορίζουμε τη συνάρτηση:

```
def animation_objects(self, id):  
    a0 = sim.AnimateRectangle(spec=(0, 0, 40, 20), text=self.name(),  
                              fillcolor=id)
```

η οποία, με τη βοήθεια της κλάσης `AnimateRectangle`, εμφανίζει στο παράθυρο του `animation` (βλέπε Εικόνα 18.11) για κάθε ΧΑ που εντάσσεται στο σύστημα φόρτωσης-μεταφοράς, ένα παραλληλόγραμμα, του οποίου οι κορυφές έχουν συντεταγμένες που δίνονται, σε pixels, από την τιμή της παραμέτρου `spec`.

Εντός του παραλληλογράμμου εμφανίζεται με την τιμή της παραμέτρου `text` το όνομα του ΧΑ, δηλαδή `TR.1`, `TR.2` κλπ., ενώ η τιμή της παραμέτρου `fillcolor`, η οποία ορίζει το χρώμα γεμίσματος του παραλληλογράμμου, δίνεται από την τιμή της μεταβλητής `id`.

Η μεταβλητή `id` παίρνει την τιμή `blue`, όταν το ΧΑ είναι φορτωμένο με άγωνα και τότε το παραλληλόγραμμα χρωματίζεται μπλε, και την τιμή `black` όταν το ΧΑ είναι άδειο και τότε το παραλληλόγραμμα χρωματίζεται μαύρο.

Τα παραλληλόγραμμα των ΧΑ αλλάζουν θέσεις και χαρακτηριστικά στο παράθυρο του `animation`, δυναμικά, σύμφωνα με τα προβλεπόμενα από τις εντολές του κώδικα προσομοίωσης.

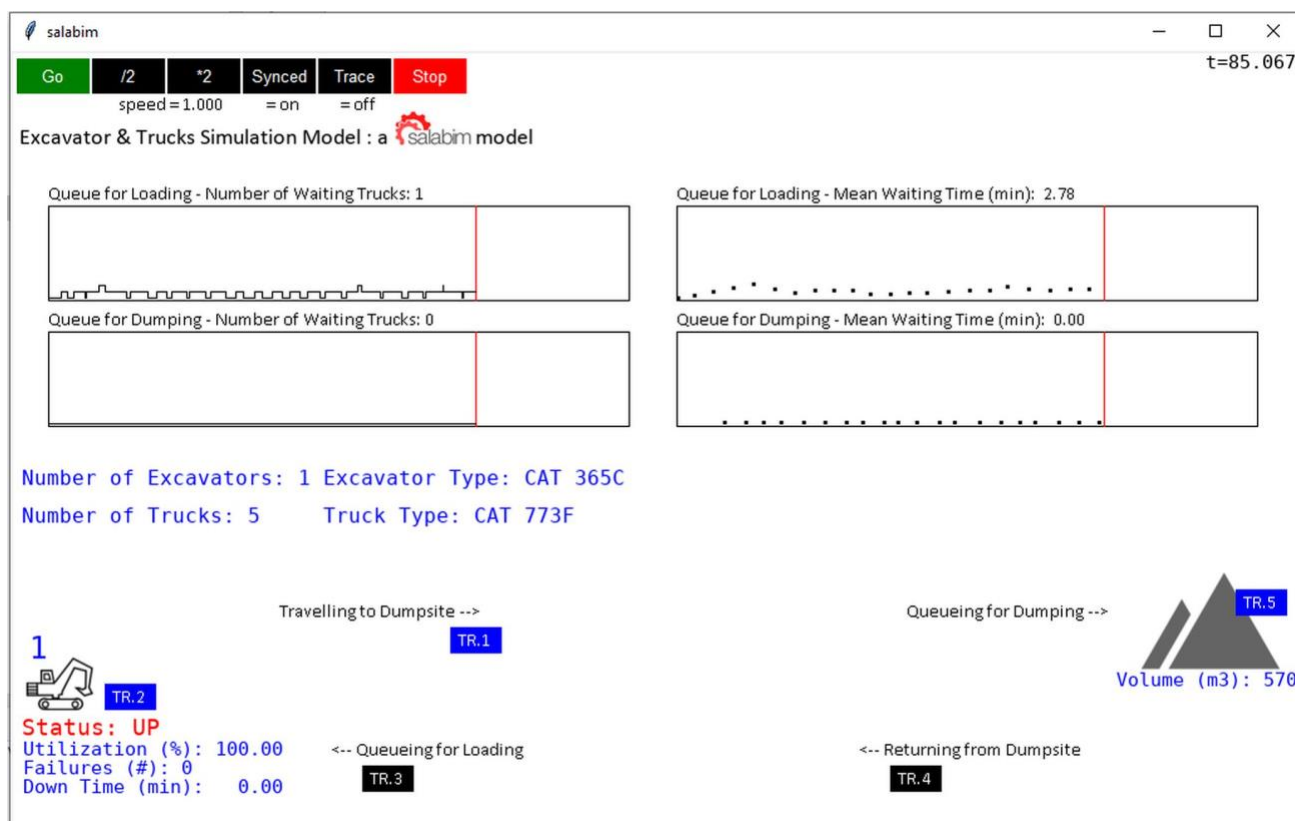
Κάτω από τη γραμμή σχολίου `# Animation Section` στο κυρίως σώμα του κώδικα, ορίζουμε τη συνάρτηση `do_animation()`, η οποία περιλαμβάνει όλες τις υπόλοιπες εντολές για την πραγμάτωση του `animation`. Οι εντολές αυτές παρατίθενται θεματικά.

Έτσι, κάτω από τη γραμμή σχολίου `# Dumpsite`, η εντολή:

```
sim.AnimateImage('waste_dumpsite_110x110.png', x=900, y=260)
```

με τη βοήθεια της κλάσης `AnimateImage` εμφανίζει την εικόνα που υπάρχει στο αρχείο `waste_dumpsite_110x110.png` και αναπαριστά στο παράθυρο τους σωρούς απόθεσης των αγόνων.

Οι συντεταγμένες της κάτω αριστερής γωνίας της εικόνας με τους σωρούς απόθεσης των αγόνων δίνονται με τις παραμέτρους `x` (οριζόντιος άξονας) και `y` (κάθετος άξονας), σε pixels. Η αρχή των αξόνων (0,0) βρίσκεται στην κάτω αριστερή γωνία του παραθύρου `animation`.



Εικόνα 18.11. Το παράθυρο animation του μοντέλου προσομοίωσης του συστήματος φόρτωσης-μεταφοράς κατά τη χρονική στιγμή $t=85.067$ min από την έναρξη της προσομοίωσης.

Τα αρχεία των εικόνων πρέπει να βρίσκονται στο ίδιο directory με το αρχείο του κώδικα του προγράμματός μας.

Η εντολή:

```
sim.AnimateText(text=lambda: 'Volume (m3): {:.3.0f}'.format(env.dumps_volume),
                x=880, y=260, textcolor='blue')
```

με τη βοήθεια της κλάσης `AnimateText` εμφανίζει, σε μπλε χρώμα, το κείμενο `Volume (m3) :` και την τιμή της μεταβλητής `env.dumps_volume`, μορμαρισμένη, η οποία θα μεταβάλλεται κάθε φορά που εκκενώνει ένα ΧΑ στην απόθεση.

Η εντολή:

```
sim.AnimateQueue(dumps.requesters(), x=850, y=290, direction='w',
                title='Queueing for Dumping -->', id='blue')
```

με τη βοήθεια της κλάσης `AnimateQueue` εμφανίζει μια ουρά αναμονής, πάνω από την οποία υπάρχει τίτλος με το κείμενο `Queueing for Dumping -->`, σε μαύρο χρώμα (default), font Calibri ή Arial (default) και μέγεθος 15 pts (default).

Σε αυτή την ουρά αναμονής παραμένουν τα ΧΑ που αιτούνται θέση στην απόθεση για εκκένωση (`dumps.requesters`) και στοιβάζονται προς τα αριστερά της ουράς (w, Δυτικά), ενώ τα παραλληλόγραμμα των ΧΑ έχουν μπλε χρώμα.

Η εντολή:

```
sim.AnimateQueue(dumps.claimers(), x=975, y=320, direction='n', title='',
                id='blue')
```

πρόκειται περί μιας *εικονικής* ουράς, χωρίς τίτλο, στην οποία παραμένουν τα ΧΑ στα οποία έχει δοθεί μια θέση εκκένωσης (`dumps.claimers`) και εκκενώνουν το περιεχόμενο στους σωρούς της απόθεσης. Επειδή αυτά τα ΧΑ είναι ακόμα *μισοάδεια*, τα παραλληλόγραμμά τους έχουν ακόμα το μπλε χρώμα. Οι θέσεις εκκένωσης των ΧΑ είναι 2 και εάν εμφανιστεί και δεύτερο ΧΑ για να εκκενώσει, αυτό θα στοιβαχθεί προς τα πάνω στην ουρά γιατί έχουμε επιλέξει `direction='n'` (n, Βόρεια).

Κάτω από τη γραμμή σχολίου `# Excavator` παρατίθενται οι εντολές που αφορούν τον εκσκαφέα και είναι ανάλογες εκείνων που παρουσιάσαμε για το `animation` της απόθεσης:

Με τη γραμμή κώδικα:

```
sim.AnimateText(text=lambda: '{:d}'.format(number_of_excavators),
                x=15, y=280, textcolor='blue', fontsize=24)
```

εμφανίζουμε πάνω από την εικόνα του εκσκαφέα, με μπλε χρώμα και μέγεθος font 24 pts, τον αριθμό των εκσκαφέων που είναι ενταγμένοι στο σύστημα, δηλαδή την τιμή της μεταβλητής `number_of_excavators`.

Με τη γραμμή κώδικα:

```
sim.AnimateQueue(queue_for_loading, x=280, y=180, direction='e',
                 title='<-- Queueing for Loading', id='black')
```

εμφανίζουμε την ουρά αναμονής των ΧΑ που περιμένουν να τα φορτώσει ο εκσκαφέας. Τα ΧΑ στοιβάζονται προς τα δεξιά της ουράς (e, Ανατολικά) και τα παραλληλόγραμμα των άδειων ΧΑ έχουν μαύρο χρώμα.

Σημειώστε ότι, αντί για την τιμή `queue_for_loading`, η οποία είναι η τιμή της παραμέτρου `name` της κλάσης `AnimateQueue` (το `name=`, ως ευκόλως εννοούμενο, παραλείπεται), θα μπορούσαμε να χρησιμοποιήσουμε την τιμή `excavator.requesters()`, όπως κάναμε στον κώδικα με τον οποίο απεικονίσαμε την ουρά αναμονής των ΧΑ που περιμένουν να εκκενώσουν στην απόθεση.

Αυτό επιτρέπεται γιατί ο `excavator`, όπως και τα `dumps`, προσομοιώνονται ως `Resource`.

Κάτω από τη γραμμή σχολίου `# Monitors` υπάρχουν οι εντολές που διαμορφώνουν τα τέσσερα διαγράμματα μεταβολής των τιμών: *αριθμός ΧΑ που αναμένουν και μέσος χρόνος αναμονής*, στις ουρές αναμονής για φόρτωση και εκκένωση.

Τα `Monitors` (επιτηρητές) στη `Salabim` μας επιτρέπουν να συλλέγουμε δεδομένα (`data`) κατά τη διάρκεια της προσομοίωσης.

Αυτόματα συλλέγονται από τη `Salabim` δεδομένα για τις κλάσεις `Resource`, `Queue` και `State`. Επιπλέον, ο χρήστης μπορεί να ορίσει και δικά του `Monitors`.

Η εντολή:

```
sim.AnimateMonitor(
    queue_for_loading.length,
    x=30,
    y=570,
    height=75,
    width=(env.width() - 100)/2,
    horizontal_scale=4,
    title=lambda: 'Queue for Loading - Number of Waiting Trucks: {:d}'
    .format(queue_for_loading.length()))
```

με τη βοήθεια της κλάσης `AnimateMonitor` εμφανίζει το άνω-αριστερά διάγραμμα της Εικόνας 18.11, στο οποίο παρουσιάζεται η μεταβολή του αριθμού των ΧΑ που βρίσκονται στην ουρά αναμονής για φόρτωση, κατά τη διάρκεια του χρόνου προσομοίωσης.

Η τελευταία εντολή της συνάρτησης `do_animation`:

```
env.animation_parameters(modelname='Excavator & Trucks Simulation '
                             'Model', animate=True,
                             video='Video_Excavator_and_Trucks_Simulation_Model.mp4')
```

ορίζει τις παραμέτρους του animation.

Η παράμετρος `modelname` δίνει τον τίτλο του μοντέλου μας, που θα εμφανιστεί στο άνω μέρος του παράθυρου animation, και η παράμετρος `video` το όνομα του αρχείου, στο οποίο θα καταγραφεί και θα αποθηκευτεί το βίντεο με το animation του μοντέλου προσομοίωσης. Εάν δεν επιθυμούμε το γύρισμα βίντεο, τότε παραλείπουμε την παράμετρο `video`.

Εφόσον επιθυμούμε την παραγωγή βίντεο, εκτός από την παράμετρο `video` πρέπει:

- στο τέλος του μπλοκ των εντολών, που βρίσκονται κάτω από τη γραμμή σχολίου `# Main body`, να γράψουμε, για να σταματήσει η εγγραφή του βίντεο, την εντολή:

```
env.video(False)
```

- να έχουμε εγκαταστήσει στον υπολογιστή μας τις βιβλιοθήκες `Opencv` και `NumPy` [Σχετικά βλέπε στο website: <https://www.salabim.org/manual/Introduction.html>] κατά τα γνωστά:

```
pip install opencv-python
pip install numpy
```

Αποθηκεύουμε το πρόγραμμα σε αρχείο με όνομα `Chapter_18_TRs_EX_Mine_Animated.py` και στο ίδιο directory πρέπει να υπάρχουν τα αρχεία εικόνων `shovel_60x60.gif` και `waste_dumpsite_110x110.png`

Εάν τρέξουμε το πρόγραμμα, θα αναδυθεί στην οθόνη το παράθυρο του animation και θα αρχίσει η ροή των εικόνων ταυτόχρονα με την εκτέλεση της προσομοίωσης.

Με τα κουμπιά του Menu Bar που υπάρχουν στο παράθυρο του animation έχουμε τη δυνατότητα, ανά πάσα στιγμή, να σταματήσουμε το animation, καθώς και να μειώσουμε στο μισό ($/2$) ή να αυξήσουμε στο διπλάσιο ($*2$), και στα υποπολλαπλάσια ή πολλαπλάσια αυτών, την ταχύτητα του animation. Για να συνεχίσουμε τη ροή του animation πατάμε το Go.

Όπως φαίνεται και από το λογότυπο του παραθύρου GUI, το 2D animation της Salabim εκτελείται σε ένα παράθυρο Tkinter!

Η βιβλιοθήκη Salabim έχει τη δυνατότητα να κάνει και τριδιάστατο (3D) animation των μοντέλων προσομοίωσης. Για σχετικές πληροφορίες βλέπε στο εγχειρίδιο της Salabim:

<https://www.salabim.org/manual/3d%20animation.html>

Στο website αυτού του βιβλίου υπάρχουν:

1. το script του κώδικα που εκτελεί ταυτόχρονα την προσομοίωση και το animation του μοντέλου του συστήματος φόρτωσης-μεταφοράς, στο αρχείο `Chapter_18_TRs_EX_Mine_Animated.py`
2. τα αρχεία εικόνων `shovel_60x60.gif` και `waste_dumpsite_110x110.png`
3. το αρχείο `Video_Excavator_and_Trucks_Simulation_Model.mp4` που είναι το βίντεο του animation για τις πρώτες οκτώ (8) ώρες της προσομοίωσης.

18.12 Οι στατιστικές κατανομές της βιβλιοθήκης Salabim

Η βιβλιοθήκη Salabim διαθέτει μια συλλογή με τις πλέον γνωστές και συνήθως χρησιμοποιούμενες στατιστικές κατανομές. Μεταξύ αυτών περιλαμβάνονται οι κατανομές: Beta, Erlang, Exponential, Gamma, Normal, Poisson, Triangular, Uniform, Weibull κ.ά. [Σχετικά με της στατιστικές κατανομές που διαθέτει η βιβλιοθήκη Salabim βλέπε: <https://www.salabim.org/manual/Distributions.html>].

Όταν δημιουργούμε ένα καινούριο περιβάλλον προσομοίωσης στη Salabim, όπως κάναμε στους κώδικες των μοντέλων προσομοίωσης που αναπτύξαμε, με την εντολή:

```
env = sim.Environment(trace=False)
```

η παράμετρος `random_seed` της `sim.Environment()` λαμβάνει την default τιμή `random_seed=None`, που σημαίνει ότι η τιμή-σπόρος (seed value) για την παραγωγή ψευδο-τυχαίων αριθμών, κατά τη δειγματοληψία τιμών από όλες τις στατιστικές κατανομές της Salabim, θα είναι ο ακέραιος αριθμός 1234567.

Εάν το επιθυμούμε, μπορούμε βέβαια να ορίσουμε τη δική μας τιμή-σπόρο π.χ. την 7654321, με την εντολή:

```
env = sim.Environment(trace=False, random_seed=7654321)
```

Εάν δώσουμε την τιμή `random_seed=""` τότε η τιμή-σπόρος θα προκύψει από την ώρα του ρολογιού του υπολογιστή (system time) σε milliseconds.

Στην περίπτωση αυτή, είναι προφανές ότι κάθε φορά που τρέχουμε το μοντέλο προσομοίωσης, τα αποτελέσματα δεν θα είναι αναπαράξιμα και επομένως δεν θα μπορούμε να κάνουμε συγκρίσεις μεταξύ διαφορετικών διαμορφώσεων του συστήματος που προσομοιώνουμε [Σχετικά, βλέπε παράγραφο 18.5 Γεννήτριες ψευδο-τυχαίων αριθμών].

Η λήψη μιας τιμής της κατανομής (δειγματοληψία) π.χ. του χρόνου φόρτωσης γίνεται με την εντολή:

```
yield self.hold(sim.Triangular(8, 15, 12).sample())
```

ή πιο απλά:

```
yield self.hold(sim.Triangular(8, 15, 12)())
```

Επίσης, η τιμή της δειγματοληψίας μπορεί να εκχωρηθεί σε μια μεταβλητή, όπως κάναμε με το βάρος των αδρανών που φορτώνονται σε κάθε ΧΑ:

```
truck_load = sim.Normal(30, 1).sample()
```

Τέλος, εάν θέλουμε να χρησιμοποιήσουμε μια στατιστική κατανομή που δεν περιλαμβάνεται στη συλλογή της βιβλιοθήκη Salabim, π.χ. τη λογαριθμοκανονική κατανομή (lognormal), τότε μπορούμε να την εισάγουμε από το module `numpy.random` της βιβλιοθήκης NumPy κάνοντας χρήση της κλάσης `sim.External()` ως εξής:

```
lognormal_dis = sim.External(numpy.random.lognormal, mu=5, sigma=1)
truck_load = lognormal_dis.sample()
```

Με τον ίδιο τρόπο μπορούμε να χρησιμοποιήσουμε και στατιστικές κατανομές από τα modules `random` και `scipy.stats` της Python και της SciPy, αντίστοιχα. Είναι προφανές ότι πρέπει να κάνουμε `import` στον κώδικά μας τα modules αυτά πριν τα χρησιμοποιήσουμε.

18.12.1 Η χρήση της κανονικής κατανομής στην προσομοίωση

Η χρήση της κανονικής στατιστικής κατανομής σε κώδικες προσομοίωσης θέλει ιδιαίτερη προσοχή. Ο λόγος είναι ότι η κανονική κατανομή ορίζεται από $-\infty$ έως $+\infty$ και αυτό έχει ως συνέπεια, κατά τη δειγματοληψία να υπάρχει η πιθανότητα να προκύψει μια αρνητική τιμή. Εάν η τιμή αυτή αφορά τον χρόνο διάρκειας μιας εργασίας, προφανώς δεν είναι αποδεκτή.

Η βιβλιοθήκη Salabim, για να καλύψει την περίπτωση αυτή, διαθέτει την κλάση `Bound`, η οποία επιτρέπει, κατά τη δειγματοληψία από μια κατανομή, τη λήψη μόνο των τιμών που είναι μεγαλύτερες από ένα όριο.

Εάν, για παράδειγμα, ο χρόνος φόρτωσης των χωματουργικών αυτοκινήτων ακολουθεί μια κανονική τιμή με μέση τιμή 10 min και τυπική απόκλιση 2 min, οι εντολές:

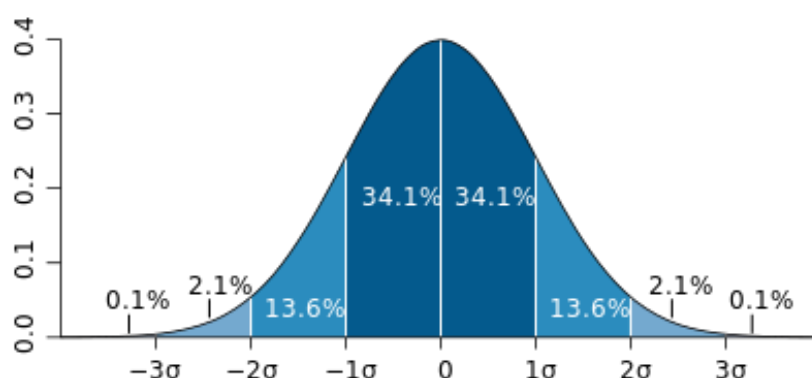
```
loading_time = sim.Normal(10, 2) # mean=10, st.deviation=2
yield self.hold(sim.Bounded(loading_time, lowerbound=5).sample())
```

μας εξασφαλίζουν ότι, από τη δειγματοληψία της κανονικής κατανομής του χρόνου φόρτωσης θα ληφθούν υπόψη στην προσομοίωση μόνο οι θετικές τιμές και μάλιστα όσες είναι μεγαλύτερες ή ίσες των 5 min.

Και εάν αναρωτηθήκατε γιατί δεν εφαρμόσαμε την κλάση Bound στην περίπτωση της δειγματοληψίας από κανονική κατανομή, για τον υπολογισμό του βάρους των αδρανών που φορτώνονται σε κάθε ΧΑ, και στον κώδικά μας γράψαμε:

```
truck_load = sim.Normal(30, 1).sample()
```

η απάντηση βρίσκεται στο: https://en.wikipedia.org/wiki/Normal_distribution και στην Εικόνα 18.12.



Εικόνα 18.12. Στην κανονική κατανομή το 99.7% των παρατηρήσεων (δειγματοληψιών) βρίσκεται εντός της περιοχής που απέχει τρεις τυπικές αποκλίσεις από τη μέση τιμή [Πηγή: https://en.wikipedia.org/wiki/Normal_distribution].

18.13 Τεχνικές προσομοίωσης συστημάτων

Σκοπός των μοντέλων προσομοίωσης συστημάτων είναι να αναπαραστήσουν-μιμηθούν το πραγματικό σύστημα, με τη μεγαλύτερη δυνατή ακρίβεια και λεπτομέρεια.

Όμως, ο βαθμός της ακρίβειας και λεπτομέρειας που πρέπει να επιδιώκουμε εξαρτάται από τις ιδιαιτερότητες του συστήματος που προσομοιώνουμε και τα ζητούμενα που βάζουμε.

Για παράδειγμα, στο σύστημα φόρτωσης-μεταφοράς της παραγράφου 18.10 Προσομοίωση συστήματος φόρτωσης-μεταφοράς δεν λάβαμε υπόψη την ποιότητα του μεταφερόμενου από τα ΧΑ υλικού, και ορθά κάναμε, γιατί ως άγονο πέτρωμα πάει στα μάζα.

Εάν όμως ο εκσκαφέας φορτώνει μέταλλευμα, τότε ενδεχομένως να μας ενδιέφερε η ποιότητα του φορτίου του μεταλλεύματος που μεταφέρει κάθε ΧΑ, γιατί σε ένα μεταλλείο μπορεί το καλής ποιότητας μέταλλευμα να οδηγείται στον θραυστήρα και το κακής ποιότητας στην απόθεση αγόνων ή να αποθηκεύεται σε σωρούς για πιθανή μελλοντική αξιοποίηση (stock). Είναι προφανές ότι στην περίπτωση αυτή το μοντέλο μας θα πρέπει να γίνει πιο λεπτομερές και επομένως πιο σύνθετο από εκείνο της φόρτωσης-μεταφοράς αγόνων.

Η προσομοίωση διακριτών γεγονότων βασίζεται στους χρόνους εκτέλεσης των εργασιών-γεγονότων που προσομοιώνονται. Οι χρόνοι αυτοί προκύπτουν, συνήθως, από χρονομετρήσεις των γεγονότων, οι τιμές των οποίων πρέπει να προσαρμοστούν σε μια στατιστική κατανομή (βλέπε παράγραφο 18.7 Προσαρμογή μετρήσεων σε κατανομή: fitter). Όσο μεγαλύτερο είναι το δείγμα των μετρήσεων, τόσο καλύτερη είναι η προσαρμογή σε κάποια στατιστική κατανομή, αλλά και τόσο ακριβέστερα η στατιστική κατανομή αντιπροσωπεύει την πραγματικότητα.

Μετά την ανάπτυξη του κώδικα του μοντέλου προσομοίωσης ενός συστήματος, αρχίζει η επίπονη διαδικασία της επαλήθευσης και της επιβεβαίωσης του μοντέλου.

Η *επαλήθευση* (validation) αναφέρεται στο εάν κατασκευάσαμε το μοντέλο σωστά. Δηλαδή, εάν οι παράμετροι εισόδου (δεδομένα) και η λογική του μοντέλου αναπαρίστανται σωστά στον κώδικα του προγράμματος.

Η *επιβεβαίωση* (verification) αναφέρεται στο εάν κατασκευάσαμε το σωστό μοντέλο. Δηλαδή, εάν το μοντέλο-κώδικας αναπαριστά με ακρίβεια το πραγματικό σύστημα που προσομοιώνει.

Η προσεκτική εκσφαλμάτωση του κώδικα, η μελέτη του αρχείου καταγραφής των μεταβολών του συστήματος που προσομοιώνεται κατά τους διακριτούς χρόνους της προσομοίωσης (Trace log) και η κατασκευή και θέαση του animation του μοντέλου βοηθούν τα μέγιστα τόσο στην επαλήθευση όσο και στην επιβεβαίωση.

Η προσομοίωση διακριτών γεγονότων βασίζεται στη στοχαστικότητα. Η χρήση διαφορετικών τιμών-σπόρων (seed value), για την παραγωγή ψευδο-τυχαίων αριθμών κατά τη δειγματοληψία τιμών χρόνου από τις στατιστικές κατανομές, αυξάνει την τυχαιότητα-στοχαστικότητα των τιμών χρόνου που χρησιμοποιούνται όταν τρέχει το μοντέλο προσομοίωσης.

Μία άλλη τεχνική που αυξάνει τη στοχαστικότητα είναι η *αναπαραγωγή-επανάληψη* (replication) της προσομοίωσης με διαφορετικές τιμές-σπόρους παραγωγής ψευδο-τυχαίων αριθμών. Όσο περισσότερες επαναλήψεις γίνουν, τόσο μικρότερη είναι η μεταβλητότητα των αποτελεσμάτων της προσομοίωσης.

Ο αριθμός των απαιτούμενων επαναλήψεων προσδιορίζεται με την εφαρμογή στατιστικών μεθόδων (κεντρικό οριακό θεώρημα κ.ά.). Ο μέσος όρος των τιμών των αποτελεσμάτων που προκύπτουν από καθεμία επανάληψη μας δίνει μια στατιστικά *πιο σωστή* τιμή.

Στο μοντέλο προσομοίωσης του συστήματος φόρτωσης-μεταφοράς που τρέξαμε στην παράγραφο 18.10 *Προσομοίωση συστήματος φόρτωσης-μεταφοράς*, ο χρόνος προσομοίωσης ορίστηκε σε δέκα (10) 8ωρες βάρδιες, δηλαδή 4800 min.

Στον Πίνακα 18.2 παρουσιάζονται ορισμένα από τα αποτελέσματα της προσομοίωσης του συστήματος της παραγράφου 18.10 *Προσομοίωση συστήματος φόρτωσης-μεταφοράς*, το οποίο τρέξαμε για διαφορετικούς χρόνους προσομοίωσης (8ωρες βάρδιες).

Πίνακας 18.2. Τα αποτελέσματα προσομοίωσης του μοντέλου φόρτωσης-μεταφοράς για διαφορετικούς χρόνους προσομοίωσης.

Χρόνος προσομοίωσης [8ωρες βάρδιες]	Ποσοστό χρησιμοποίησης εκκαφέα [%]	Όγκος αγόνων που εκκενώνουν τα ΧΑ [m ³ /h]	Μέσος χρόνος αναμονής ΧΑ για φόρτωση [min]
1	100	450.00	2.81
10	97.93	451.88	2.98
100	98.06	452.96	2.99
1,000	98.02	452.55	3.01
10,000	98.03	452.47	3.01
100,000	98.04	452.50	3.01

Παρατηρούμε ότι όσο αυξάνει ο χρόνος προσομοίωσης, το εύρος διακύμανσης των τιμών του ποσοστού χρησιμοποίησης του εκκαφέα (utilization), του όγκου των αγόνων ανά ώρα που εκκενώνουν τα ΧΑ στην απόθεση και του μέσου χρόνου αναμονής τους για φόρτωση μειώνεται. Αυτό είναι λογικό, γιατί αυξάνεται σημαντικά ο αριθμός των δειγματοληψιών που γίνονται από τις στατιστικές κατανομές του μοντέλου.

Είναι προφανές ότι η αύξηση του χρόνου προσομοίωσης έχει ανάλογο *υπολογιστικό κόστος*. Η εκτέλεση της προσομοίωσης του μοντέλου φόρτωσης-μεταφοράς για 100,000 8ωρες βάρδιες (δηλαδή 91 περίπου χρόνια!) διήρκεσε περίπου 33 min σε έναν υπολογιστή με επεξεργαστή Intel® Core™ i7-1065G7 @ 1.50 GHz και 8.00 GB RAM.

Τέλος, για να αποτρέψουμε ή τουλάχιστον να μειώσουμε την επίδραση των αρχικών συνθηκών στα αποτελέσματα της προσομοίωσης, εφαρμόζουμε την τεχνική της *προθέρμανσης* (warmup).

Δηλαδή, τρέχουμε το μοντέλο μας για ένα χρονικό διάστημα, στο τέλος του οποίου διαγράφουμε όλες τις τιμές των στατιστικών που συλλέγονται από τα Monitors και συνεχίζουμε από αυτό το χρονικό σημείο την προσομοίωση για το επιθυμητό χρονικό διάστημα, κατά τη διάρκεια του οποίου συλλέγονται τα στατιστικά που αξιοποιούμε.

Για να γίνει κατανοητός ο τρόπος με τον οποίο δουλεύει η τεχνική της προθέρμανσης και να αναδειχθεί η σημασία της στην προσομοίωση, δίνεται στη συνέχεια μια τροποποιημένη-απλοποιημένη παραλλαγή του κώδικα του συστήματος φόρτωσης-μεταφοράς της παραγράφου 18.10 *Προσομοίωση συστήματος φόρτωσης-μεταφοράς*:

```
# A Simple Excavator-Trucks Simulation Model in Salabim
# with warmup period
import salabim as sim

# INPUT DATA SECTION
excavator_type = 'CAT 365C' # 4.6 m3
truck_type = 'CAT 773F' # 55 t/35 m3
number_of_trucks = 5
number_of_excavators = 1
number_of_dumps = 2
truck_interarrival_time = 0
loading_time = sim.Uniform(3.5, 4.3)
travel_time_excavator_to_dumps = sim.Uniform(5.3, 6.5)
dumping_time = sim.Uniform(1.5, 2.5)
return_time_dumps_to_excavator = sim.Uniform(4.5, 5.5)
truck_load_volume = 30 # average, m3
warmup_time = 120
number_of_shifts = 1
sim_duration = 480*number_of_shifts

class TruckGenerator(sim.Component):
    """
    Generates n trucks, then TruckGenerator "ends" and the n trucks
    circulate within the system """

    def process(self):
        for i in range(number_of_trucks):
            Truck(name="TR," )
            yield self.hold(truck_interarrival_time)

class Truck(sim.Component):
    """
    Controls trucks into the load-travel-dump-return cycle """

    def process(self):
        while True:
            self.enter(queue_for_loading)
            yield self.request((excavator, 1))
            self.leave(queue_for_loading)
            yield self.hold(loading_time.sample())
            self.release((excavator, 1))
            env.number_of_trucks_loaded += 1
            self.enter(travelling_excavator_to_dumps)
            yield self.hold(travel_time_excavator_to_dumps.sample())
            self.leave(travelling_excavator_to_dumps)
            self.enter(queue_for_dumping)
```

```

yield self.request((dumps, 1))
self.leave(queue_for_dumping)
yield self.hold(dumping_time.sample())
self.release((dumps, 1))
env.number_of_trucks_dumped += 1
env.dumps_volume += truck_load_volume
self.enter(returning_dumps_to_excavator)
yield self.hold(return_time_dumps_to_excavator.sample())
self.leave(returning_dumps_to_excavator)

# Main body
env = sim.Environment(trace=False)
env.number_of_trucks_loaded = 0
env.number_of_trucks_dumped = 0
env.dumps_volume = 0
queue_for_loading = sim.Queue('Queue for Loading')
queue_for_dumping = sim.Queue('Queue for Dumping')
travelling_excavator_to_dumps = sim.Queue('Travelling from Excavator to Dumps')
returning_dumps_to_excavator = sim.Queue('Returning from Dumps to Excavator')
excavator = sim.Resource('Excavator', capacity=number_of_excavators)
dumps = sim.Resource('Dumps', capacity=number_of_dumps)
excavator_status = sim.State('Excavator Status', value='UP')
TruckGenerator()
env.run(duration=warmup_time)
queue_for_loading.reset_monitors()
env.run(duration=sim_duration)
# Simulation output
print('SIMULATION REPORT')
print('Warmup time [min] = {:.0f}'.format(warmup_time))
print('Queue for loading:')
print(' Length [trucks] minimum = {:.0f}, median = {:.0f}, maximum = {:.0f}'
      .format(queue_for_loading.length.minimum(ex0=True),
              queue_for_loading.length.median(ex0=True),
              queue_for_loading.length.maximum(ex0=True)))
print(' Length of stay [min] minimum = {:.2f}, mean = {:.2f}, '
      ' maximum = {:.2f}'.
      format(queue_for_loading.length_of_stay.minimum(ex0=True),
              queue_for_loading.length_of_stay.mean(ex0=True),
              queue_for_loading.length_of_stay.maximum(ex0=True)))

```

Όπως παρατηρούμε, στον κώδικα δεν υπάρχουν οι εντολές που μοντελοποιούσαν τις βλάβες του εκσκαφέα και στα αποτελέσματα της προσομοίωσης παρουσιάζονται μόνο τα στατιστικά της ουράς αναμονής των ΧΑ για φόρτωση.

Στα δεδομένα προσθέσαμε τη μεταβλητή `warmup_time`, στην οποία εκχωρούμε τον χρόνο προθέρμανσης, ενώ στη μεταβλητή `truck_interarrival_time` εκχωρήσαμε την τιμή 0. Αυτό σημαίνει ότι, κατά την έναρξη της προσομοίωσης και τα 5 ΧΑ βρίσκονται στην ουρά αναμονής για φόρτωση. Επίσης, μειώσαμε τον χρόνο προσομοίωσης σε μία (1) βάρδια.

Οι τελευταίες γραμμές στο τμήμα του κώδικα, που είναι κάτω από τη γραμμή σχολίου `# Main body`:

```

env.run(duration=warmup_time)
queue_for_loading.reset_monitors()
env.run(duration=sim_duration)

```

κατά σειρά:

- Τρέχουν το μοντέλο για χρόνο προθέρμανσης 120 min.
- Μηδενίζουν τα στατιστικά που συλλέχθηκαν από τα Monitors της ουράς αναμονής των ΧΑ για φόρτωση (`queue_for_loading`).

- Ξανατρέχουν το μοντέλο για χρόνο προσομοίωσης μία 8ωρη βάρδια. Κατά την έναρξη της δεύτερης αυτής προσομοίωσης τα 5 ΧΑ εκκινούν από τις θέσεις στις οποίες βρέθηκαν στο 120ό min, δηλαδή όταν τέλειωσε ο χρόνος προθέρμανσης και όχι από τη θέση τους στην ουρά αναμονής ΧΑ για φόρτωση. Τα στατιστικά που συλλέγονται κατά τη διάρκεια αυτού του χρονικού διαστήματος εμφανίζονται στα αποτελέσματα της προσομοίωσης.

Εάν τρέξουμε τον παραπάνω κώδικα χωρίς προθέρμανση (`warmup_time = 0`), στην οθόνη θα εμφανιστούν τα αποτελέσματα:

```
SIMULATION REPORT
Warmup time [min] = 0
Queue for loading:
Length [trucks] minimum = 1, median = 1, maximum = 4
Length of stay [min] minimum = 0.96, mean = 3.09, maximum = 16.49
```

Εάν τρέξουμε τον κώδικα με προθέρμανση (`warmup_time = 120`), στην οθόνη θα εμφανιστούν τα αποτελέσματα:

```
SIMULATION REPORT
Warmup time [min] = 120
Queue for loading:
Length [trucks] minimum = 1, median = 1, maximum = 2
Length of stay [min] minimum = 0.96, mean = 2.75, maximum = 4.53
```

Οι τεχνικές που παρουσιάστηκαν στην παράγραφο αυτή αναλύονται διεξοδικά, μαζί με πλήθος άλλων αντικειμένων της προσομοίωσης διακριτών γεγονότων, στο εξαιρετικό κλασικό βιβλίο των J. Banks, J.S. Carson, B.L. Nelson και D.M. Nicol *“Discrete-Event System Simulation”* (5th edition, Pearson Education Ltd, 2014), στο οποίο οι αναγνώστες, που ενδιαφέρονται για την προσομοίωση συστημάτων, ενθαρρύνονται να ανατρέξουν.

Το script του κώδικα που περιλαμβάνει την εφαρμογή της τεχνικής της προθέρμανσης βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_18_TRs_EX_Mine_Warmup.py`

19 Δεδομένα από το Web

19.1 Γενικά

Ο Παγκόσμιος Ιστός (World Wide Web, www) ή κοινώς το Web, είναι μια *αστείρευτη* πηγή πληροφορίας για πληθώρα θεμάτων, απλών, καθημερινών, αλλά και πιο εξειδικευμένων.

Στο Κεφάλαιο αυτό θα δούμε πώς μπορούμε να ενσωματώσουμε πληροφορίες που είναι διαθέσιμες στο Web σε ένα πρόγραμμα Python. Θα αναπτύξουμε μια εφαρμογή προκειμένου να λαμβάνουμε το τρέχον δελτίο καιρού σε οποιαδήποτε τοποθεσία της Γης, δίνοντας το όνομα της πόλης που μας ενδιαφέρει ή τις γεωγραφικές συντεταγμένες της τοποθεσίας.

Τα μετεωρολογικά στοιχεία θα τα αντλήσουμε από το Website του Open Weather Map, που είναι προσπελάσιμο στο website: <https://openweathermap.org>

19.2 Άδεια λήψης δεδομένων από το Open Weather Map

Προκειμένου να μας επιτρέψει το Open Weather Map να αντλήσουμε στοιχεία για τις τρέχουσες μετεωρολογικές συνθήκες μιας τοποθεσίας, πρέπει προηγουμένως να μας εκχωρήσει ένα API key.

API είναι τα αρχικά των λέξεων Application Programming Interface (Διεπαφή Προγραμματισμού Εφαρμογών). Ένα API είναι ένα σύνολο κανόνων, οι οποίοι ορίζουν το πώς επικοινωνούν μεταξύ τους ηλεκτρονικοί υπολογιστές ή προγράμματα [Σχετικά βλέπε στο: <https://www.ibm.com/cloud/learn/api>].

Στην περίπτωση μας το API θα είναι ο *μεσάζων* (intermediary layer), ο οποίος θα διαχειρίζεται τη μεταφορά των μετεωρολογικών δεδομένων από τον Web server του Open Weather Map στο πρόγραμμα Python που θα αναπτύξουμε, ανταποκρινόμενος στις *κλήσεις* (API calls ή requests) που θα γίνονται από τις εντολές του προγράμματός μας.

Η πρόσβαση στα δεδομένα του Open Weather Map είναι προσωποποιημένη και επιτυγχάνεται με τη βοήθεια ενός API key που χορηγείται στον χρήστη, κατόπιν αίτησης εγγραφής στο website: <https://openweathermap.org/api>

Η υπηρεσία Current Weather Data προσφέρεται δωρεάν και επιτρέπει 60 API calls/min και 1,000,000 API calls/month.

Το API key αποστέλλεται στο email του χρήστη και έχει τη μορφή μιας συμβολοσειράς, όπως π.χ. είναι η: af86_5001_5942_5c2c_82d2b00faff5

Σημείωση: Επειδή αυτό είναι το προσωπικό API key του συγγραφέα του βιβλίου, για προφανείς λόγους, τέσσερις χαρακτήρες του έχουν αντικατασταθεί με _.

19.3 Ανάπτυξη της εφαρμογής

Γράφουμε τις παρακάτω γραμμές κώδικα:

```
# Current Weather Report
import requests
from datetime import datetime
from pprint import pprint
API_key = "af86_5001_5942_5c2c_82d2b00faff5"
# Data input
while True:
    print('LOCAL WEATHER NOW')
    print("Enter your location:")
    preference = input("Enter '1' for 'Town/Country' OR '2' for "
```



```

        "Coordinates (Lat/Lon)" : ")
if preference == '1':
    location = input('Enter the name of the location: ')
    country = input('Enter the country code (e.g. GR for Greece): ')
    url_part_1 = f'http://api.openweathermap.org/data/2.5/weather?' \
        f'q={location},{country}&units=metric&appid='
    break
elif preference == '2':
    lat = input('Enter the latitude of the location: ')
    lon = input('Enter the longitude of the location: ')
    url_part_1 = f'http://api.openweathermap.org/data/2.5/weather?' \
        f'lat={lat}&lon={lon}&units=metric&appid='
    break
else:
    print("WARNING ! Enter '1' OR '2'")
#
weather_url = url_part_1 + API_key
weather_data = requests.get(weather_url).json()
# Output
pprint(weather_data)

def datefromtimestamp(timestamp):
    date_time = datetime.utcfromtimestamp(timestamp)\
        .strftime('%Y-%m-%d %H:%M:%S')
    return date_time

# Formatted output
print('LOCAL WEATHER REPORT')
while True:
    if weather_data['cod'] == '404':
        print('ERROR! Location: {:s}, {:s}'.format(location, country))
        print('        is not available in the Open Weather Map Database.')
        print("        Instead, try the location's coordinates.")
        break
    elif preference == '1':
        print('Current Weather in {:s}, {:s}'.format(location, country))
    elif preference == '2':
        print('Current Weather at Latitude: {:s} & Longitude: {:s} [{:s}]:'
            .format(lat, lon, weather_data['name']))
    print('Description           : {:s}, {:s}'
        .format(weather_data['weather'][0]['main'],
            weather_data['weather'][0]['description']))
    print('Temperature (oC)         : {:.2f}'
        .format(weather_data['main']['temp']))
    print('Feels like (oC)            : {:.2f}'
        .format(weather_data['main']['feels_like']))
    print('Max. Temperature (oC)      : {:.2f}'
        .format(weather_data['main']['temp_max']))
    print('Min. Temperature (oC)      : {:.2f}'
        .format(weather_data['main']['temp_min']))
    print('Humidity (%)                : {:.0f}'
        .format(weather_data['main']['humidity']))
    print('Atmospheric Pressure (hPa) : {:d}'
        .format(weather_data['main']['pressure']))
    print('Wind:')
    print(' Direction (deg)           : {:d}'
        .format(weather_data['wind']['deg']))
    print(' Speed (m/s)               : {:.2f}'
        .format(weather_data['wind']['speed']))

```

```

try:
    print(' Gust (m/s)           : {:.2f}'
          .format(weather_data['wind']['gust']))
    print('Rain, last hour (mm)   : {:.2f}'
          .format(weather_data['rain']['1h']))
    print('Snow, last hour (mm)   : {:.2f}'
          .format(weather_data['snow']['1h']))
except KeyError:
    pass
print('Sunrise (date & UTC time) : {:s}'
      .format(datefromtimestamp(weather_data['sys']['sunrise'])))
print('Sunset (date & UTC time)  : {:s}'
      .format(datefromtimestamp(weather_data['sys']['sunset'])))
break

```

Καταρχάς, κάνουμε import τη βιβλιοθήκη requests.

Η βιβλιοθήκη requests της Python μας επιτρέπει να στέλνουμε αιτήματα (requests) με βάση το πρωτόκολλο επικοινωνίας HTTP (HyperText Transfer Protocol, Πρωτόκολλο Μεταφοράς Υπερκειμένου), το οποίο είναι το κύριο πρωτόκολλο που χρησιμοποιείται στους Web browsers για να μεταφέρει δεδομένα ανάμεσα σε έναν server και έναν client. Στο website της βιβλιοθήκης: <https://docs.python-requests.org> θα βρείτε πληροφορίες για τη χρήση της.

Ως εξωτερική βιβλιοθήκη της Python πρέπει να εγκατασταθεί στον υπολογιστή μας κατά τα γνωστά:

```
pip install requests
```

Επίσης, θα κάνουμε import το module datetime, που μας επιτρέπει να χειριζόμαστε ημερομηνίες και τον χρόνο και το γνωρίσαμε στην παράγραφο 10.4 *Είσοδος του χρήστη στο πρόγραμμα*, καθώς και το module pprint (Data pretty printer) [βλέπε στο: <https://docs.python.org/3/library/pprint.html>].

Τα modules αυτά ανήκουν σε ενσωματωμένες βιβλιοθήκες της Python και επομένως δεν χρειάζονται εγκατάσταση, αλλά μόνο import.

Στη μεταβλητή API_key έχουμε εκχωρήσει την τιμή (συμβολοσειρά) του API key που μας έχει χορηγήσει το Open Weather Map.

Σημείωση: Για να τρέξετε τον παραπάνω κώδικα πρέπει να αντικαταστήσετε το *πειραγμένο* API key του συγγραφέα με το προσωπικό δικό σας API key.

Εντός του βρόχου while που ακολουθεί, περιλαμβάνονται οι εντολές με τις οποίες γίνεται η εισαγωγή των δεδομένων από το πληκτρολόγιο.

Ο χρήστης έχει δύο επιλογές:

1. να εισάγει το όνομα της τοποθεσίας-πόλης και της χώρας,
2. να εισάγει τις γεωγραφικές συντεταγμένες (σε δεκαδική μορφή) της τοποθεσίας

για την οποία επιθυμεί να λάβει το τρέχον δελτίο καιρού.

Εάν επιλέξει την πρώτη περίπτωση, στη μεταβλητή url_part_1 εκχωρείται η τιμή ενός formatted string, το οποίο μας προτείνει το Open Weather Map, όταν η αναζήτηση των μετεωρολογικών δεδομένων μιας τοποθεσίας γίνεται με το όνομα της πόλης και της χώρας [Σχετικά βλέπε στο website: <https://openweathermap.org/api/geocoding-api>]:

```
url_part_1 = f'http://api.openweathermap.org/data/2.5/weather?' \
             f'q={location},{country}&units=metric&appid='

```

Παρατηρήστε ότι το φορμάρισμα του string έγινε με τη μέθοδο f-string, την οποία παρουσιάσαμε στην παράγραφο 04.4.1 *Φορμαρισμένα αποτελέσματα*, αλλά δεν τη χρησιμοποιήσαμε στα προγράμματα που γράψαμε μέχρι τώρα.

Η παράθεση της χώρας είναι απαραίτητη, γιατί με το ίδιο όνομα πόλης είναι πιθανό να υπάρχουν περισσότερες της μίας τοποθεσίες επί της Γης, αλλά σε διαφορετικές χώρες, π.χ. Athens, GR (η Αθήνα μας) και Athens, US (στην πολιτεία Τζόρτζια των Ηνωμένων Πολιτειών) κ.ά.

Στη δεύτερη περίπτωση, στη μεταβλητή `url_part_1` εκχωρείται η τιμή ενός formatted string, το οποίο επίσης μας προτείνει το Open Weather Map, όταν η αναζήτηση των μετεωρολογικών δεδομένων μιας τοποθεσίας γίνεται με τις γεωγραφικές συντεταγμένες, ήτοι το γεωγραφικό πλάτος (latitude, `lat`) και το γεωγραφικό μήκος (longitude, `lon`) [Σχετικά βλέπε στο: <https://openweathermap.org/current>]:

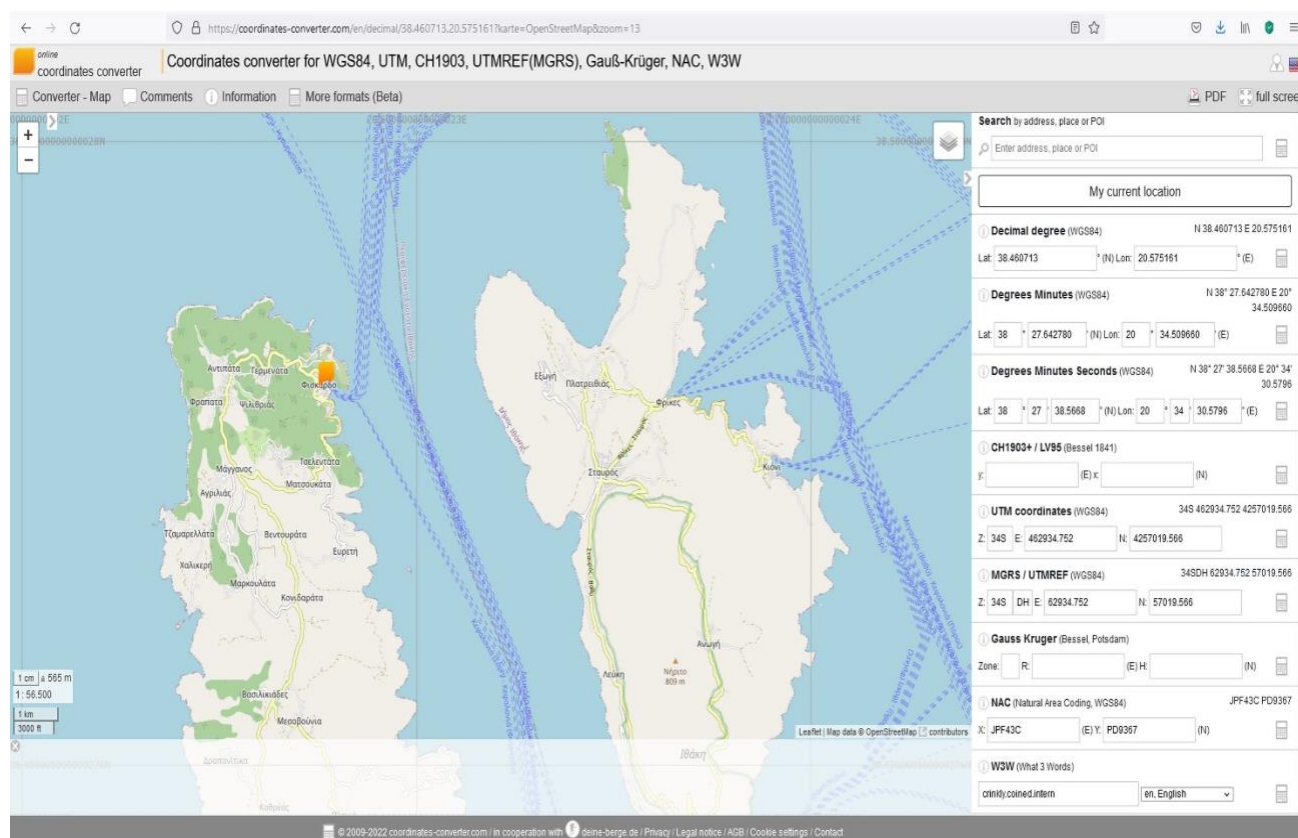
```
url_part_1 = f'http://api.openweathermap.org/data/2.5/weather?' \
            f'lat={lat}&lon={lon}&units=metric&appid='
```

Για να βρει τις γεωγραφικές συντεταγμένες μιας τοποθεσίας, ο συγγραφέας του βιβλίου χρησιμοποιεί τη δωρεάν Web-εφαρμογή Online Coordinates Converter, που είναι διαθέσιμη στο: <https://coordinates-converter.com/en/> η οποία δίνει τις γεωγραφικές συντεταγμένες μιας τοποθεσίας σε διάφορα τοπογραφικά προβολικά συστήματα.

Στην Εικόνα 19.1 παρουσιάζεται η οθόνη του Online Coordinates Converter και οι γεωγραφικές συντεταγμένες του χωριού Φισκάρδο στη Βόρεια Κεφαλονιά.

Το πρόγραμμα Python που αναπτύσσουμε δέχεται τις γεωγραφικές συντεταγμένες σε δεκαδική μορφή σύμφωνα με το Παγκόσμιο προβολικό σύστημα WGS84, το οποίο χρησιμοποιείται και από το δορυφορικό σύστημα εντοπισμού θέσης GPS. Το ελληνικό προβολικό σύστημα ΕΣΓΑ87 είναι απόλυτα συμβατό με το WGS84.

Από την Εικόνα 19.1 προκύπτει ότι οι γεωγραφικές συντεταγμένες του Φισκάρδου είναι: γεωγραφικό πλάτος (`lat`): 38.4607 και γεωγραφικό μήκος (`lon`): 20.5752 .



Εικόνα 19.1. Η οθόνη της Web-εφαρμογής Online Coordinates Converter και οι γεωγραφικές συντεταγμένες του χωριού Φισκάρδο στη Βόρεια Κεφαλονιά.

Με τη γραμμή κώδικα:

```
weather_url = url_part_1 + API_key
```

στο string της μεταβλητής `url_part_1` προσθέτουμε (προσαρτούμε) το string του προσωπικού μας API key, και το νέο string που προκύπτει το εκχωρούμε στη μεταβλητή `weather_url`

Τώρα πλέον έχουμε το URL στην πλήρη μορφή του και μπορούμε να κάνουμε call/request στον Web server του Open Weather Map για να μας αποστείλει τα τρέχοντα μετεωρολογικά στοιχεία της τοποθεσίας, που έχουμε πληκτρολογήσει, με τη βοήθεια της εντολής:

```
weather_data = requests.get(weather_url).json()
```

Ο όρος URL (Uniform Resource Locator, Ενιαίος Εντοπιστής Πόρων) δηλώνει μια διεύθυνση ενός πόρου του Web, είναι δηλαδή η Web address του πόρου [βλέπε: <https://en.wikipedia.org/wiki/URL>].

Το Open Weather Map διαθέτει τα μετεωρολογικά στοιχεία στις format JSON, XML και HTML. Εμείς επιλέξαμε τη format JSON και για τον λόγο αυτό προσθέσαμε την επέκταση (extension) `.json`

Η JSON (JavaScript Object Notation) είναι ένα σύννηθες format για την ανταλλαγή δεδομένων μεταξύ Web εφαρμογών και servers [βλέπε: <https://en.wikipedia.org/wiki/JSON>].

Τα μετεωρολογικά στοιχεία που μας αποστέλλει ο server του Open Weather Map εκχωρούνται στη μεταβλητή `weather_data` υπό τη μορφή ενός λεξικού.

Η εντολή:

```
pprint(weather_data)
```

εμφανίζει στην οθόνη τα μετεωρολογικά στοιχεία (για την πόλη των Αθηνών) ως:

```
{'base': 'stations',
 'clouds': {'all': 0},
 'cod': 200,
 'coord': {'lat': 37.9795, 'lon': 23.7162},
 'dt': 1644571690,
 'id': 264371,
 'main': {'feels_like': 12.41,
          'humidity': 48,
          'pressure': 1024,
          'temp': 13.73,
          'temp_max': 15.23,
          'temp_min': 11.64},
 'name': 'Athens',
 'sys': {'country': 'GR',
        'id': 2043076,
        'sunrise': 1644556808,
        'sunset': 1644595126,
        'type': 2},
 'timezone': 7200,
 'visibility': 10000,
 'weather': [{'description': 'clear sky',
                'icon': '01d',
                'id': 800,
                'main': 'Clear'}],
 'wind': {'deg': 214, 'gust': 2.24, 'speed': 0.89}}
```

που είναι μια πιο *pretty* μορφή εμφάνισης των τιμών του λεξικού από ότι εκείνη που θα παίρναμε εάν χρησιμοποιούσαμε την εντολή:

```
print(weather_data)
```

οπότε οι τιμές του λεξικού θα εμφανίζονταν ως:

```
{'coord': {'lon': 23.7162, 'lat': 37.9795}, 'weather': [{'id': 800, 'main': 'Clear', 'description': 'clear sky', 'icon': '01d'}], 'base': 'stations', 'main': {'temp': 13.73, 'feels_like': 12.41, 'temp_min': 11.64, 'temp_max': 15.23, 'pressure': 1024, 'humidity': 48}, 'visibility': 10000, 'wind': {'speed': 0.89, 'deg': 214, 'gust': 2.24}, 'clouds': {'all': 0}, 'dt': 1644571690, 'sys': {'type': 2, 'id': 2043076, 'country': 'GR', 'sunrise': 1644556808, 'sunset': 1644595126}, 'timezone': 7200, 'id': 264371, 'name': 'Athens', 'cod': 200}
```

Κάτω από τη γραμμή σχολίου # Formatted output παρατίθεται μια σειρά εντολών με τις οποίες εμφανίζονται στην οθόνη τα κυριότερα μετεωρολογικά δεδομένα του λεξικού σε μια εύκολα αναγνώσιμη (ανθρώπινη) μορφή:

```
LOCAL WEATHER REPORT
Current Weather in Athens, GR:
Description           : Clear, clear sky
Temperature (oC)      : 13.73
Feels like (oC)       : 12.41
Max. Temperature (oC) : 15.23
Min. Temperature (oC) : 11.64
Humidity (%)          : 48
Atmospheric Pressure (hPa) : 1024
Wind:
  Direction (deg)     : 214
  Speed (m/s)         : 0.89
  Gust (m/s)          : 2.24
Sunrise (date & UTC time) : 2022-02-11 05:20:08
Sunset (date & UTC time)  : 2022-02-11 15:58:46
```

Σε αυτό το τμήμα του κώδικα άξια προσοχής είναι τα παρακάτω:

Εάν εισάγουμε το όνομα μιας τοποθεσίας που δεν είναι διαθέσιμη στη βάση δεδομένων του Open Weather Map, π.χ. το Φισκάρδο:

```
LOCAL WEATHER NOW
Enter your location:
Enter '1' for 'Town/Country' OR '2' for 'Coordinates (Lat/Lon)' : 1
Enter the name of the location: Fiskardo
Enter the country code (e.g. GR for Greece): GR
```

τότε, το μεν λεξικό θα έχει ως τιμές τις:

```
{'cod': '404', 'message': 'city not found'}
```

και στην οθόνη θα εμφανιστεί το μήνυμα:

```
LOCAL WEATHER REPORT
ERROR! Location: Fiskardo, GR
       is not available in the Open Weather Map Database.
       Instead, try the location's coordinates.
```

Εάν, αντί για το όνομα της τοποθεσίας εισάγουμε τις γεωγραφικές συντεταγμένες του Φισκάρδου:

LOCAL WEATHER NOW

Enter your location:

Enter '1' for 'Town/Country' OR '2' for 'Coordinates (Lat/Lon)' : 2

Enter the latitude of the location: 38.4607

Enter the longitude of the location: 20.5752

στην οθόνη θα εμφανιστούν τα μετεωρολογικά στοιχεία της πλησιέστερης (μετεωρολογικά) τοποθεσίας, που στην περίπτωση του Φισκάρδου είναι η Ιθάκη [Itháki]:

LOCAL WEATHER REPORT

Current Weather at Latitude: 38.4607 & Longitude: 20.5752 [Itháki]:

Description : Clear, clear sky

Temperature (oC) : 14.97

Feels like (oC) : 14.04

Max. Temperature (oC) : 14.97

Min. Temperature (oC) : 13.88

Humidity (%) : 58

Atmospheric Pressure (hPa) : 1025

Wind:

Direction (deg) : 209

Speed (m/s) : 0.48

Gust (m/s) : 0.88

Sunrise (date & UTC time) : 2022-02-11 05:33:27

Sunset (date & UTC time) : 2022-02-11 16:10:35

Επίσης, με τη βοήθεια του βρόχου `try-except` εμφανίζονται τα στοιχεία της ριπής ανέμου (Gust), βροχόπτωσης (Rain) και χιονόπτωσης (Snow) μόνο εάν υφίστανται, δηλαδή εάν υπάρχουν τα αντίστοιχα keys στο λεξικό. Ο όγκος της βροχόπτωσης ή χιονόπτωσης δίνεται σε mm που έπεσαν κατά την τελευταία μία (1) ώρα.

Σε ό,τι αφορά τις μονάδες μέτρησης, στο URL με το οποίο κάνουμε call/request στον Web server του Open Weather Map, με τη βοήθεια της παραμέτρου `units=metric` έχουμε δηλώσει ότι επιθυμούμε το μετρικό σύστημα. Για περισσότερες πληροφορίες σχετικά με τις μονάδες μέτρησης ανατρέξτε στη τεκμηρίωση: <https://openweathermap.org/weather-data>

Ειδικά για τον χρόνο της Ανατολής και Δύσης του Ήλιου σε μια τοποθεσία, αυτός δίνεται σε χρόνο UTC (Coordinated Universal Time, Συντονισμένη Παγκόσμια Ώρα) [Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Coordinated_Universal_Time].

Η χειμερινή ώρα στην Ελλάδα είναι UTC + 02:00 και η θερινή UTC + 03:00.

Ως μονάδα του χρόνου χρησιμοποιείται από το Open Weather Map η *unix time stamp*, που είναι ο αριθμός των δευτερολέπτων που έχουν παρέλθει από το *Unix Epoch* μέχρι σήμερα-τώρα.

Ως Unix Epoch ορίζεται η 1^η Ιανουαρίου 1970 00:00 UTC.

Για παράδειγμα, η τοπική ώρα 12:00 (μεσημβρινή) της 11^{ης} Φεβρουαρίου 2022 στην Ελλάδα είναι: 1644573600 unix time stamp [Σχετικά βλέπε στο website: <https://www.unixtimestamp.com>].

Προκειμένου να μετατρέψουμε τις ώρες της Ανατολής (Sunrise) και Δύσης (Sunset) του Ήλιου από χρόνο unix time stamp σε συμβατική μορφή, γράψαμε τη συνάρτηση:

```
def datefromtimestamp(timestamp):  
    date_time = datetime.utcfromtimestamp(timestamp) \  
        .strftime('%Y-%m-%d %H:%M:%S')  
    return date_time
```

η οποία με τη βοήθεια της συνάρτησης `datetime.utcnow()` του module `datetime` μας δίνει το επιθυμητό αποτέλεσμα.

Έτσι, οι τιμές της ώρας Ανατολής και Δύσης του Ήλιου της 11^{ης} Φεβρουαρίου 2022 στην Αθήνα, που στο λεξικό δίνονται ως:

```
'sunrise': 1644556808, 'sunset': 1644595126
```

στην οθόνη εμφανίζονται ως:

Sunrise (date & UTC time)	: 2022-02-11 05:20:08
Sunset (date & UTC time)	: 2022-02-11 15:58:46

Το script του κώδικα που δίνει το τρέχον δελτίο καιρού μιας τοποθεσίας βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_19_Current_Weather_Report.py`

20 Περί πυθωνικής μουσικής

20.1 Γενικά

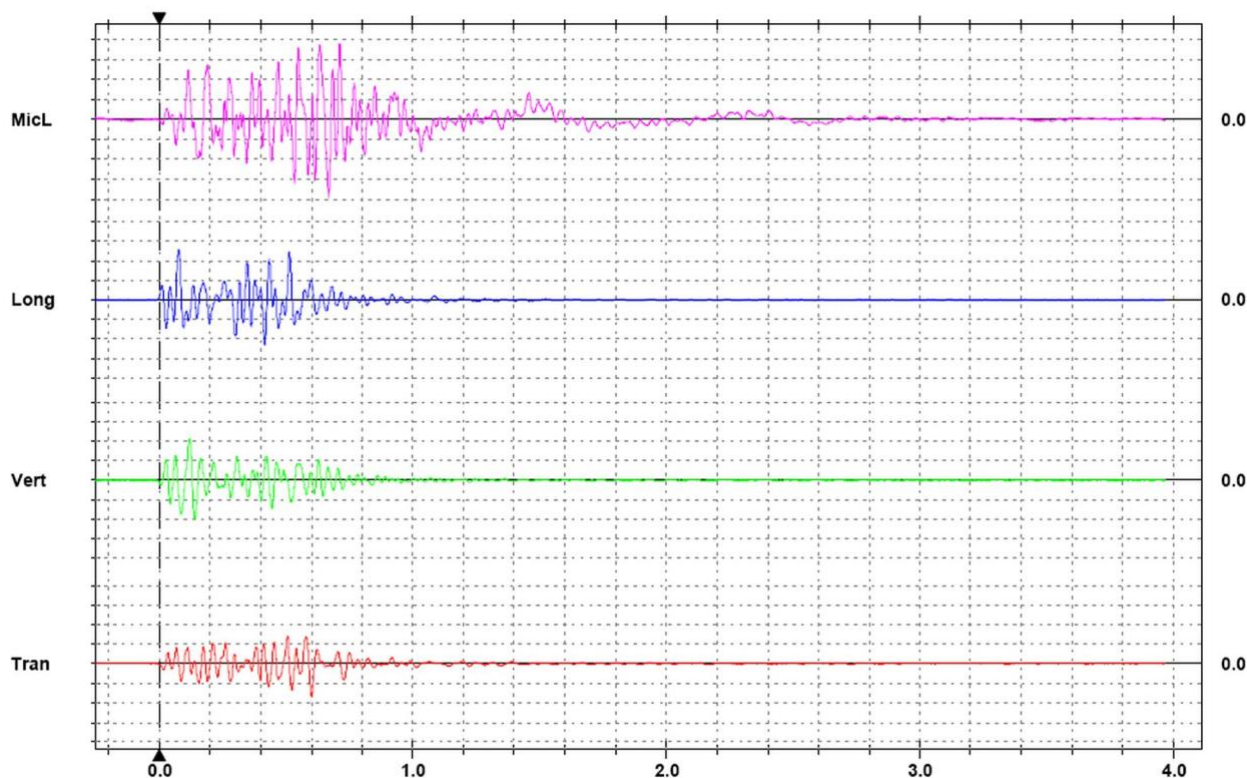
Ο συγγραφέας δεν έχει παρακολουθήσει μαθήματα μουσικής, αλλά του αρέσει να ακούει μουσική και να εργάζεται υπό τους ήχους της.

Αφορμή για τη συγγραφή αυτού του Κεφαλαίου ήταν η ανάγνωση του άρθρου *How to Play Music Using Mathematics in Python*, το οποίο γράφτηκε από τον Nishu Jain και δημοσιεύτηκε το 2020 στο website του Towards Data Science [βλέπε στο: <https://towardsdatascience.com/mathematics-of-music-in-python-b7d838c84f72>].

Από το άρθρο αυτό προέρχεται και ο βασικός κώδικας που περιλαμβάνεται στην παράγραφο 20.3 *Ο ήχος των πλίκτρων ενός πιάνου*, κατάλληλα τροποποιημένος.

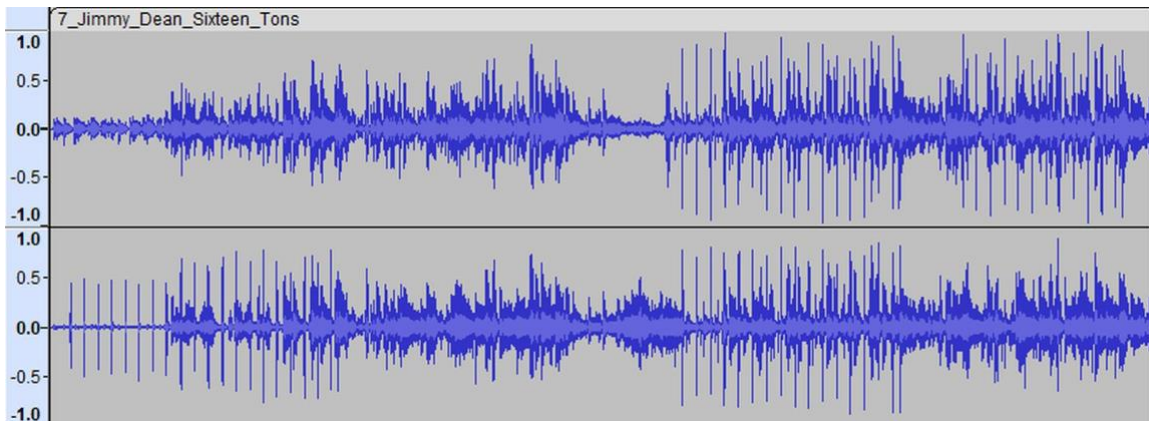
Στην παράγραφο 02.1 *Γενικά* αναφέραμε ότι, κατά την ανατίναξη των πετρωμάτων, εκτός από τις δονήσεις του εδάφους, παράγεται και ένα αέριο ωστικό κύμα.

Στην Εικόνα 20.1 παρουσιάζεται η μορφή του κυματοπαλμού του αέριου ωστικού κύματος (άνω καμπύλη), μαζί με τους κυματοπαλμούς των δονήσεων του εδάφους (κάτω τρεις καμπύλες) μιας ανατίναξης, όπως καταγράφηκαν από δονησιογράφο του Εργαστηρίου Εξόρυξης Πετρωμάτων του ΕΜΠ.



Εικόνα 20.1. Καταγραφή από δονησιογράφο του κυματοπαλμού του αέριου ωστικού κύματος (άνω καμπύλη) και των κυματοπαλμών των δονήσεων του εδάφους (κάτω τρεις καμπύλες) μιας ανατίναξης σε λατομείο ασβεστολιθίου [Πηγή: Εργαστήριο Εξόρυξης Πετρωμάτων, ΕΜΠ].

Στην Εικόνα 20.2 παρουσιάζεται ο κυματοπαλμός του κλασικού μεταλλευτικού τραγουδιού *Sixteen Tons* σε μια εκτέλεσή του από τον πολύ γνωστό Αμερικάνο καλλιτέχνη της country μουσικής Jimmy Dean [https://en.wikipedia.org/wiki/Jimmy_Dean].



Εικόνα 20.2. Ο κυματοπαλμός του τραγουδιού Sixteen Tons στην εκτέλεσή του από τον Jimmy Dean (Stereo, Άνω: Left Chanel – Κάτω: Right Chanel), όπως αναπαράγεται από το πρόγραμμα Audacity.

Ο κυματοπαλμός του τραγουδιού Sixteen Tons προέκυψε με την αναπαραγωγή του αρχείου 7_Jimmy_Dean_Sixteen_Tons.mp3 στο πρόγραμμα Audacity.

Το Audacity είναι ένα πρόγραμμα επεξεργασίας ήχου και ηχογράφησης (audio editor and recorder) το οποίο μπορείτε να εγκαταστήσετε δωρεάν στον υπολογιστή σας από το website: <https://www.audacityteam.org>

Το αρχείο 7_Jimmy_Dean_Sixteen_Tons.mp3 του τραγουδιού Sixteen Tons υπάρχει στο website του βιβλίου.

Από τη σύγκριση των κυματοπαλμών των Εικόνων 20.1 και 20.2 αντιλαμβανόμαστε ότι τόσο το αέριο ωστικό κύμα όσο και η μουσική που ακούμε είναι ταλαντώσεις των σωματιδίων του ατμοσφαιρικού αέρα, αρμονικές στην περίπτωση της μουσικής, θορυβώδεις στην περίπτωση του αέριου ωστικού κύματος.

20.2 Ταλαντώσεις και ήχοι

Από τη Φυσική γνωρίζουμε ότι οι ταλαντώσεις χαρακτηρίζονται από τα μεγέθη: μέγιστο πλάτος (amplitude), περίοδος (period) ή συχνότητα (frequency) και διάρκεια (time) της ταλάντωσης (Εικόνα 20.3).

Η εξίσωση της καμπύλης της απλής αρμονικής ταλάντωσης (ημιτονοειδής κυματοπαλμός) συναρτήσει του χρόνου, δίνεται από τη σχέση [βλέπε: [https://en.wikipedia.org/wiki/Sine wave](https://en.wikipedia.org/wiki/Sine_wave)]:

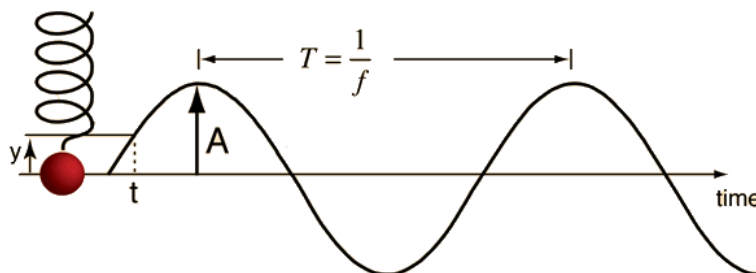
$$y(t) = A * \sin(2 * \pi * f * t) \quad [20.1]$$

όπου:

A : μέγιστο πλάτος ταλάντωσης

f : συχνότητα ταλάντωσης, $T = 1/f$ όπου T η περίοδος της ταλάντωσης

t : χρόνος



Εικόνα 20.3. Γραφική παράσταση και ορισμός μεγεθών της απλής αρμονικής ταλάντωσης [Πηγή: <http://hyperphysics.phy-astr.gsu.edu/hbase/shm.html>].

Επίσης, η βιβλιοθήκη SciPy διαθέτει τη συνάρτηση `scipy.io.wavfile.write()` με την οποία μπορούμε να γράψουμε ένα array της NumPy σε ένα αρχείο ήχου τύπου wav [Σχετικά βλέπε στο website: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.io.wavfile.write.html#scipy.io.wavfile.write>].

Με βάση το παράδειγμα του υπάρχει στην τεκμηρίωση της συνάρτησης, γράφουμε τον παρακάτω κώδικα:

```
import numpy as np
from scipy.io.wavfile import write
fs = 440
amplitude = np.iinfo(np.int16).max
print('amplitude = ', amplitude)
samplerate = 44100
t = np.linspace(0., 1., samplerate)
data = amplitude*np.sin(2.*np.pi*fs*t)
with open('sine_wave_sound.wav', 'wb') as f:
    write(f, samplerate, data.astype(np.int16))
```

τον τρέχουμε και παίρνουμε το αρχείο ήχου `sine_wave_sound.wav`, το οποίο αντιστοιχεί σε ένα ημιτονοειδές κύμα συχνότητας 440 Hz, μέγιστου πλάτους ταλάντωσης 32767, χρόνου 1 sec, από το οποίο πήραμε δείγματα με ρυθμό 44100 Hz, επειδή με την εντολή:

```
t = np.linspace(0., 1., samplerate)
```

χωρίσαμε τον χρόνο του 1 sec σε 44100 τμήματα.

Επίσης, με την εντολή:

```
amplitude = np.iinfo(np.int16).max
```

εκχωρήσαμε στη μεταβλητή `amplitude` τη μέγιστη τιμή, η οποία προκύπτει από την εφαρμογή της κλάσης `numpy.iinfo` επί των 16-bit ακέραιων αριθμών που έχει δυνατότητα να παράγει ο υπολογιστής μας [Σχετικά βλέπε: <https://numpy.org/doc/stable/reference/generated/numpy.iinfo.html>].

Η τιμή αυτή εμφανίζεται στην οθόνη με την εντολή:

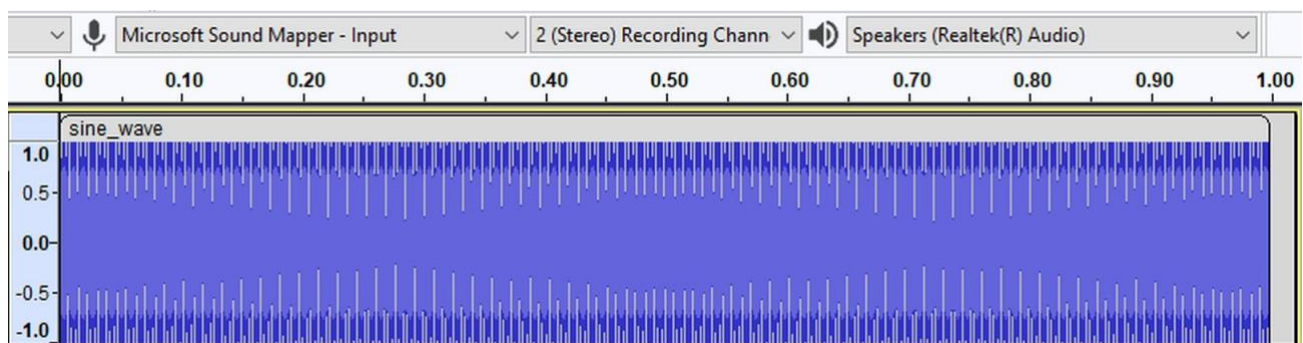
```
print('amplitude = ', amplitude)
```

και είναι:

```
amplitude = 32767
```

Το αρχείο `sine_wave_sound.wav` είναι μορφής 16-bit PCM-Mono, διάρκειας ενός (1) sec και είναι διαθέσιμο στο website του βιβλίου.

Μπορείτε να ακούσετε το αρχείο με το πρόγραμμα αναπαραγωγής ήχου του υπολογιστή σας ενώ, εάν χρησιμοποιήσετε για τον σκοπό αυτό το πρόγραμμα Audacity, θα δείτε και τη μορφή του κυματοπαλμού του, όπως παρουσιάζεται στην Εικόνα 20.4.



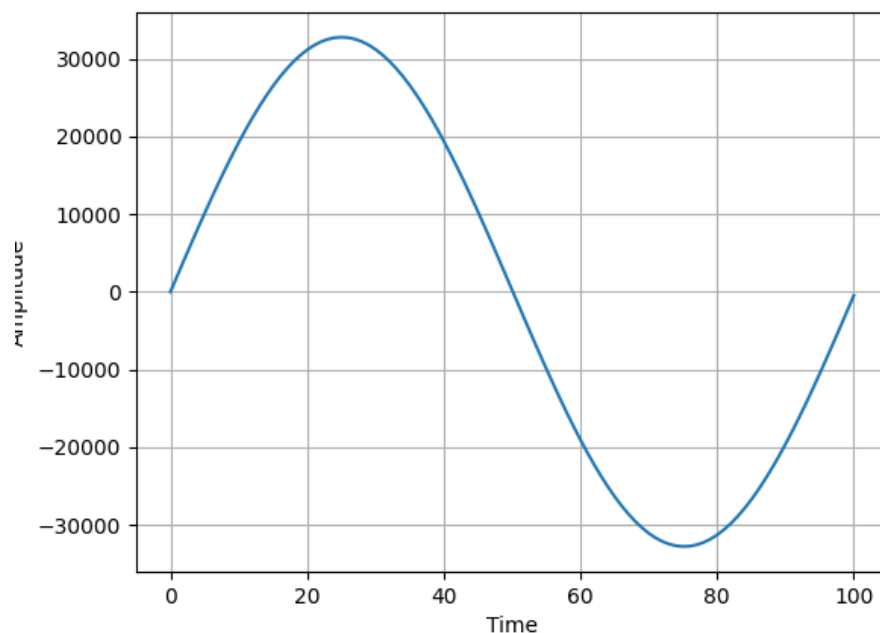
Εικόνα 20.4. Ο κυματοπαλμός του αρχείου `sine_wave_sound.wav` (16-bit PCM-Mono), όπως αναπαράγεται από το πρόγραμμα Audacity.

Εάν στο τέλος του παραπάνω κώδικα προσθέσετε τις γραμμές κώδικα:

```
import matplotlib.pyplot as plt
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.plot(data[0:(int(samplerate/fs)+1)])
plt.grid()
plt.show()
```

στην οθόνη θα εμφανιστεί η γραφική παράσταση του ημιτονοειδούς κύματος, όπως παρουσιάζεται στην Εικόνα 20.5, το οποίο αντιστοιχεί στη μορφή του κύματος μιας περιόδου ($1/fs = 1/440$ s), που λόγω του ρυθμού δειγματοληψίας των 44100 Hz αντιστοιχεί σε 100 μονάδες στον άξονα Time.

Το script του κώδικα, που παράγει τον ήχο του ημιτονοειδούς κυματοπαλμού και σχεδιάζει τη μορφή του, βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_20_Sine_Wave_Sound.py`



Εικόνα 20.5. Γραφική παράσταση μιας περιόδου του ημιτονοειδούς κύματος του αρχείου `sine_wave.wav`

20.3 Ο ήχος των πλήκτρων ενός πιάνου

Το πληκτρολόγιο του τυπικού (standard) πιάνου αποτελείται από 88 πλήκτρα τα οποία οργανώνονται σε ομάδες των 12 πλήκτρων (7 λευκά και 5 μαύρα) που ονομάζονται οκτάβες (Εικόνα 20.6).

Κάθε πλήκτρο του πιάνου παράγει ήχο μιας συγκεκριμένης συχνότητας, που δίνεται από τον τύπο [Σχετικά βλέπε: https://en.wikipedia.org/wiki/Piano_key_frequencies]:

$$f(n) = 2^{(n-49)/12} * 440 \quad [20.2]$$

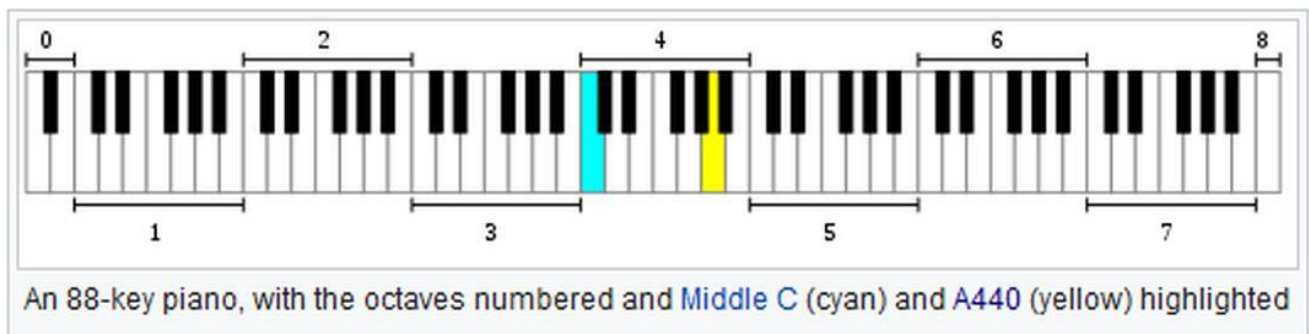
όπου:

$f(n)$: συχνότητα του ήχου που παράγεται από το πλήκτρο με αριθμό n , Hz

n : αριθμός πλήκτρου. Η αρίθμηση αρχίζει από το αριστερό άκρο του πληκτρολογίου

440: η συχνότητα του 49^{ου} πλήκτρου, ονομαζόμενου και A440 (το κίτρινο πλήκτρο στην Εικόνα 20.6), Hz

Από τις οκτάβες του πληκτρολογίου του πιάνου, χαρακτηριστική είναι η τέταρτη (κεντρική) οκτάβα, η οποία εκτός από το πλήκτρο A440 περιλαμβάνει και το πλήκτρο C4 (Middle C, το γαλάζιο πλήκτρο στην Εικόνα 20.6) που είναι η νότα Ντο.



Εικόνα 20.6. Τα 88 πλήκτρα του τυπικού (standard) πιάνου με τις 7 πλήρεις οκτάβες και τα 4 επιπλέον πλήκτρα (1/3 οκτάβας) [Πηγή: https://en.wikipedia.org/wiki/Piano_key_frequencies].

Γράφουμε τις παρακάτω γραμμές κώδικα:

```
import numpy as np
# Frequencies for octave No 4 piano keys
piano_Oct4_keys_name_number_Dix = {'C4':40, 'C#4':41, 'D4':42,
                                     'D#4':43, 'E4':44, 'F4':45, 'F#4':46,
                                     'G4':47, 'G#4':48, 'A4':49, 'A#4':50, 'B4':51}
base_frequency = 440 # 49th key (A440)
piano_Oct4_keys_frequency_Dix = {}
for i in piano_Oct4_keys_name_number_Dix.keys():
    piano_Oct4_keys_frequency_Dix.update(
        {i:base_frequency*np.power(2, ((piano_Oct4_keys_name_number_Dix[i]
                                         - 49)/12))})
piano_Oct4_keys_frequency_Dix.update({'':0.0}) # silent note
```

Καταρχάς, στο λεξικό `piano_Oct4_keys_name_number_Dix` έχουμε κάνει την αντιστοίχιση μεταξύ του ονόματος των πλήκτρων της 4^{ης} οκτάβας και του αύξοντα αριθμού τους στο πληκτρολόγιο του πιάνου.

Για την ονοματολογία των πλήκτρων και τον αύξοντα αριθμό τους ανατρέξετε στον πίνακα του website: https://en.wikipedia.org/wiki/Piano_key_frequencies

Στη συνέχεια, εκχωρούμε στη μεταβλητή `base_frequency` την τιμή της συχνότητας του 49^{ου} πλήκτρου (A440) και ορίζουμε το `piano_Oct4_keys_frequency_Dix` ως το (κενό) λεξικό στο οποίο θα κάνουμε την αντιστοίχιση μεταξύ ονόματος των πλήκτρων της 4^{ης} οκτάβας και της συχνότητάς τους.

Αυτό επιτυγχάνεται με τη βοήθεια του βρόχου `for` που ακολουθεί, όπου στο όνομα του κάθε πλήκτρου αποδίδεται η τιμή της συχνότητας σε Hz, που προκύπτει από την εφαρμογή της Εξίσωσης [20.2].

Το λεξικό `piano_Oct4_keys_frequency_Dix` συμπληρώνεται με την *άφωνη νότα*, που αντιστοιχεί σε ένα κενό διάστημα.

Το *Yankee Doodle* είναι ένας πολύ γνωστός αμερικάνικος ρυθμός/τραγούδι [Σχετικά βλέπε στο website: https://en.wikipedia.org/wiki/Yankee_Doodle], ο οποίος επιλέχθηκε γιατί μπορεί να αποδοθεί, με ικανοποιητική προσέγγιση, χρησιμοποιώντας αποκλειστικά τα πλήκτρα/νότες της 4^{ης} οκτάβας.

Εισάγουμε τη μεταβλητή `my_yankee_doodle_keys` στην οποία εκχωρούμε ως τιμή, υπό μορφή συμβολοσειράς, τα πλήκτρα του πιάνου που αντιστοιχούν στις νότες του Yankee Doodle:

```
my_yankee_doodle_keys = 'G4-G4-A4-B4-G4-B4-A4-D4-G4-G4-A4-B4-G4-F#4-D4-' \
                          'G4-G4-A4-B4-C4-B4-A4-G4-F#4-D4-E4-F#4-G4-G4-' \
                          'E4-F#4-E4-D4-E4-F#4-G4-D4-E4-D4-C4-B4-D4-E4-' \
                          'F#4-E4-D4-E4-F#4-G4-E4-D4-G4-F#4-A4-G4-G4'
```

Δηλαδή, έχουμε γράψει τα ονόματα των πλήκτρων της 4^{ης} οκτάβας, παρεμβάλλοντας μεταξύ τους μια παύλα (-), δημιουργώντας έτσι μια συμβολοσειρά (string), την οποία όταν την *παίζουμε* σε ένα πιάνο θα ακούσουμε τον ρυθμό του Yankee Doodle.

Η τιμή της μεταβλητής `my_yankee_doodle_keys` γράφτηκε από τον συγγραφέα, με τη βοήθεια του virtual πιάνου που είναι προσπελάσιμο στο website: <https://www.onlinepianist.com/piano-songs/children-song/yankee-doodle>

Ορίζουμε τη συνάρτηση:

```
def sine_wave(fs, duration=0.2):  
    """  
    Returns a numpy array of the points of a sine wave """  
    amplitude = np.iinfo(np.int16).max  
    t = np.linspace(0., duration, int(samplerate*duration))  
    data = amplitude*np.sin(2.*np.pi*fs*t)  
    return data
```

η οποία, όπως είδαμε στην παράγραφο 20.2 *Ταλαντώσεις και ήχοι*, παράγει ένα ημιτονοειδή κυματοπαλμό συγκεκριμένης συχνότητας, πλάτους και διάρκειας.

Επίσης, ορίζουμε τη συνάρτηση:

```
def music_synthesis(music_keys):  
    synthesis = [sine_wave(piano_Oct4_keys_frequency_Dix[i]) for i  
                  in music_keys.split('-')]  
    my_synthesis = np.concatenate(synthesis)  
    return my_synthesis
```

η οποία, για κάθε σειρά πλήκτρων, που διαχωρίζονται μεταξύ τους με μια παύλα, λαμβάνει τη συχνότητα που αντιστοιχεί σε κάθε πλήκτρο από το λεξικό `piano_Oct4_keys_frequency_Dix` και στη συνέχεια υπολογίζει τον αντίστοιχο κυματοπαλμό, καλώντας τη συνάρτηση `sine_wave()`

Η συνάρτηση `concatenate` της βιβλιοθήκης NumPy συνενώνει σειριακά τους κυματοπαλμούς και δημιουργεί μια ενιαία μουσική σύνθεση. Για την τεκμηρίωση της συνάρτησης ανατρέξτε στο website: <https://numpy.org/doc/stable/reference/generated/numpy.concatenate.html>

Αναδιαρθρώνοντας τα παραπάνω τμήματα, σε έναν ενιαίο κώδικα:

```
import numpy as np  
from scipy.io.wavfile import write  
samplerate = 44100  
# Frequencies for octave No 4 piano keys  
piano_Oct4_keys_name_number_Dix = {'C4':40, 'C#4':41, 'D4':42,  
                                     'D#4':43, 'E4':44, 'F4':45, 'F#4':46,  
                                     'G4':47, 'G#4':48, 'A4':49, 'A#4':50, 'B4':51}  
base_frequency = 440 # 49th key (A440)  
piano_Oct4_keys_frequency_Dix = {}  
for i in piano_Oct4_keys_name_number_Dix.keys():  
    piano_Oct4_keys_frequency_Dix.update(  
        {i:base_frequency*np.power(2, ((piano_Oct4_keys_name_number_Dix[i]  
                                         - 49)/12))})  
piano_Oct4_keys_frequency_Dix.update({'':0.0}) # silent note  
  
def sine_wave(fs, duration=0.2):  
    """  
    Returns a numpy array of the points of a sine wave """  
    amplitude = np.iinfo(np.int16).max  
    t = np.linspace(0., duration, int(samplerate*duration))  
    data = amplitude*np.sin(2.*np.pi*fs*t)  
    return data  
  
def music_synthesis(music_keys):  
    synthesis = [sine_wave(piano_Oct4_keys_frequency_Dix[i]) for i
```

```

        in music_keys.split('-')]
my_synthesis = np.concatenate(synthesis)
return my_synthesis

# My music
my_yankee_doodle_keys = 'G4-G4-A4-B4-G4-B4-A4-D4-G4-G4-A4-B4-G4-F#4-D4-' \
    'G4-G4-A4-B4-C4-B4-A4-G4-F#4-D4-E4-F#4-G4-G4-' \
    'E4-F#4-E4-D4-E4-F#4-G4-D4-E4-D4-C4-B4-D4-E4-' \
    'F#4-E4-D4-E4-F#4-G4-E4-D4-G4-F#4-A4-G4-G4'
my_yankee_doodle = music_synthesis(my_yankee_doodle_keys)
write('my_piano_yankee_doodle.wav', samplerate, \
    my_yankee_doodle.astype(np.int16))

```

έχουμε δημιουργήσει ένα δικό μας *πυθωνικό πιάνο*!

Εάν τρέξουμε το πρόγραμμα, παράγεται το αρχείο `my_piano_yankee_doodle.wav`, το οποίο υπάρχει στο website του βιβλίου.

Εάν ακούσετε το αρχείο `my_piano_yankee_doodle.wav` με το πρόγραμμα αναπαραγωγής ήχου του υπολογιστή σας, θα διαπιστώσετε ότι ο ρυθμός μεν μοιάζει αρκετά με εκείνον, που ακούγεται κατά την εκτέλεση του Yankee Doodle στο virtual πιάνο του website: <https://www.onlinepianist.com/piano-songs/children-song/yankee-doodle>, αλλά σίγουρα αυτό που ακούγεται δεν είναι ήχος πιάνου.

Αυτό οφείλεται στο ότι εμείς στο πρόγραμμά μας δεχόμαστε ότι τα πλήκτρα του πιάνου μας δημιουργούν πάντα κυματοπαλμούς ημιτονοειδούς μορφής διάρκειας 0.2 sec, πλάτους 32767, με ρυθμό δειγματοληψίας 44100 Hz και το μόνο που αλλάζει είναι η συχνότητα του κυματοπαλμού κάθε πλήκτρου, κάτι που βέβαια δεν συμβαίνει σε ένα πραγματικό πιάνο.

Το script του κώδικα που παράγει μουσική από το πρωτόγονο *virtual πιάνο* μας βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_20_My_Piano_Music.py`

Ο ήχος των πλήκτρων που έχουμε εκχωρήσει στη μεταβλητή `my_yankee_doodle_keys` μπορεί να βελτιωθεί σημαντικά με τη χρήση της βιβλιοθήκης `MIDIUtil`.

20.4 Σύνθεση μουσικής με τη βιβλιοθήκη `MIDIUtil`

Το MIDI (Musical Instrument Digital Interface) είναι ένα πρότυπο που περιγράφει το πρωτόκολλο επικοινωνίας, ψηφιακής διεπαφής και φυσικής διασύνδεσης μεταξύ ηλεκτρονικών μουσικών οργάνων, υπολογιστών και άλλων ηλεκτρονικών συσκευών, ανεξαρτήτως κατασκευαστή [Σχετικά με το πρότυπο MIDI βλέπε στο: <https://en.wikipedia.org/wiki/MIDI>].

Το MIDI δεν μεταδίδει ηχητικό σήμα, αλλά μηνύματα που περιέχουν πληροφορίες σχετικά με το τονικό ύψος και την ένταση της μουσικής νότας, καθώς και σήμα χρονισμού που προσδιορίζει την ταχύτητα (tempo) ενός κομματιού.

Η βιβλιοθήκη `MIDIUtil` της Python μας επιτρέπει να γράψουμε μουσική σε αρχεία σύμφωνα με το πρότυπο MIDI και με τη βοήθεια εντολών της Python.

Στο website της βιβλιοθήκης: <https://github.com/MarkCWirt/MIDIUtil> και στο website της τεκμηρίωσης: <https://midiutil.readthedocs.io/en/1.2.1/> θα βρείτε πληροφορίες για τη χρήση της.

Ως εξωτερική βιβλιοθήκη της Python πρέπει να εγκατασταθεί στον υπολογιστή μας κατά τα γνωστά:

```
pip install MIDIUtil
```

Γράφουμε τον κώδικα που ακολουθεί:


```

from midiutil import MIDIFile
# Synthesis section
piano_note_midi_key_number_Dix = {'C4':60, 'C#4':61, 'D4':62, 'D#4':63,
                                   'E4':64, 'F4':65, 'F#4':66, 'G4':67,
                                   'G#4':68, 'A4':69, 'A#4':70, 'B4':71}
my_yankee_doodle_keys = 'G4-G4-A4-B4-G4-B4-A4-D4-G4-G4-A4-B4-G4-F#4-D4-' \
                        'G4-G4-A4-B4-C4-B4-A4-G4-F#4-D4-E4-F#4-G4-G4-' \
                        'E4-F#4-E4-D4-E4-F#4-G4-D4-E4-D4-C4-B4-D4-E4-' \
                        'F#4-E4-D4-E4-F#4-G4-E4-D4-G4-F#4-A4-G4-G4'
synthesis = [piano_note_midi_key_number_Dix[i] for i in
             my_yankee_doodle_keys.split('-')]
# MIDI section
degrees = synthesis # MIDI note number
track    = 0
channel  = 0
time     = 0 # In beats
duration = 1 # In beats
tempo    = 250 # In BPM
volume   = 100 # 0-127, as per the MIDI standard
MyMIDI = MIDIFile(1) # One track
MyMIDI.addTempo(track, time, tempo)
for i, pitch in enumerate(degrees):
    MyMIDI.addNote(track, channel, pitch, time + i, duration, volume)
with open("my_midi_yankee_doodle.mid", "wb") as output_file:
    MyMIDI.writeFile(output_file)

```

Καταρχάς, κάνουμε import από τη βιβλιοθήκη midiutil την κλάση MIDIFile, με την οποία στη συνέχεια θα δημιουργήσουμε ένα αντικείμενό της.

Στη συνέχεια, ο κώδικας αποτελείται από δύο τμήματα:

Στο πρώτο τμήμα, κάτω από τη γραμμή σχολίου `# Synthesis section`, έχουμε δημιουργήσει το λεξικό `piano_note_midi_key_number_Dix`, το οποίο κάνει αντιστοίχιση του πλήκτρου-νότας της 4^{ης} οκτάβας του τυπικού πιάνου με τον αύξοντα αριθμό του πλήκτρου-νότας του πληκτρολογίου MIDI.

Για την αντιστοίχιση των πλήκτρων του πιάνου με εκείνα του MIDI ανατρέξτε στον πίνακα του website: https://en.wikipedia.org/wiki/Piano_key_frequencies

Στη συνέχεια, αφού αντιγράψαμε από τον κώδικα της παραγράφου 20.3 *Ο ήχος των πλήκτρων ενός πιάνου* τη μεταβλητή `my_yankee_doodle_keys`, που περιέχει τα πλήκτρα πιάνου της μουσικής του Yankee Doodle, δημιουργούμε τη λίστα `synthesis`, η οποία περιλαμβάνει τους αριθμούς των πλήκτρων MIDI που αντιστοιχούν στα πλήκτρα πιάνου της μουσικής του Yankee Doodle, δηλαδή η λίστα θα έχει τη μορφή: `[67, 67, 69, 71, 67,...]`.

Το δεύτερο τμήμα του κώδικα, κάτω από τη γραμμή σχολίου `# MIDI section`, είναι αντιγραφή του κώδικα που δίνεται στην παράγραφο Quick Start του website της βιβλιοθήκης MIDIUtil: <https://github.com/MarkCWirt/MIDIUtil> με τις ακόλουθες τροποποιήσεις:

- Στην παράμετρο `degrees` εκχωρήθηκαν οι τιμές της λίστας `synthesis`.
- Στην παράμετρο `tempo` εκχωρήθηκε η τιμή 250 αντί της 60 (δώσαμε ταχύτερο ρυθμό).
- Το `output_file` ονομάστηκε `my_midi_yankee_doodle.mid`.

Εάν τρέξουμε το πρόγραμμα, παράγεται το αρχείο `my_midi_yankee_doodle.mid`, το οποίο υπάρχει στο website του βιβλίου.

Εάν ακούσετε το αρχείο `my_midi_yankee_doodle.mid` με το πρόγραμμα αναπαραγωγής ήχου του υπολογιστή σας, θα διαπιστώσετε ότι ο ρυθμός μοιάζει πολύ περισσότερο με εκείνον που ακούγεται κατά την εκτέλεση του Yankee Doodle στο virtual πιάνο του website:

<https://www.onlinepianist.com/piano-songs/children-song/yankee-doodle>

Το script του κώδικα που συνθέτει μουσική με τη βοήθεια της βιβλιοθήκης MIDIUtil βρίσκεται στο website του βιβλίου στο αρχείο `Chapter_20_Synthesis_MIDI_Music.py`

20.5 Η Python στη μουσική

Η Python έχει πολύ στενούς δεσμούς με τη μουσική.

Στο website: <https://wiki.python.org/moin/PythonInMusic> ο αναγνώστης θα βρει πληροφορίες για λογισμικό που έχει γραφτεί στην Python, βιβλιοθήκες και εργαλεία προγραμματισμού μουσικής με την Python και έτοιμο λογισμικό που υποστηρίζει scripts της Python.

Τέλος, ενδιαφέρον παρουσιάζουν τα projects TunePad [<https://tunepad.com>] και EarSketch [<https://earsketch.gatech.edu>] που αναπτύχθηκαν από το Tangible Interaction Design and Learning (TIDAL) Lab του Northwestern University και το EarSketch Team του Georgia Institute of Technology, στις ΗΠΑ, για όσους επιθυμούν να μάθουν να συνθέτουν *πραγματική* ψηφιακή μουσική με τη βοήθεια της Python.

21 Για το τέλος

Την εποχή που γραφόταν αυτό το βιβλίο, κυκλοφόρησε από τον εκδοτικό οίκο Chapman and Hall / CRC το βιβλίο του Dr. Ali Arya με τίτλο “*Anyone Can Code: The Art and Science of Logical Creativity*” [<https://www.taylorfrancis.com/books/mono/10.1201/9780429244421/anyone-code-ali-arya>].

Πράγματι, η ανάπτυξη προγραμμάτων υπολογιστή πέρα από *Επιστήμη*, δηλαδή γνώση των αρχών και τεχνικών του προγραμματισμού ηλεκτρονικών υπολογιστών, είναι και *Τέχνη*, δηλαδή εμπεριέχει στοιχεία φαντασίας και δημιουργικότητας.

Είναι, λοιπόν, αναμενόμενο, για την επίλυση του ίδιου προβλήματος, να αναπτυχθούν από διαφορετικούς προγραμματιστές διαφορετικοί κώδικες, ιδίως όταν η ανάπτυξη του κώδικα γίνεται σε *πληθωρικές* γλώσσες προγραμματισμού, οι οποίες προσφέρουν πολλές εναλλακτικές επιλογές, όπως είναι η Python και οι βιβλιοθήκες της.

Πολλοί από τους αναγνώστες του βιβλίου μπορεί να υλοποιούσαν τον κώδικα των προγραμμάτων που περιέχονται σε αυτό με διαφορετικό τρόπο, πιο σύντομο, πιο κομψό, πιο ευφάνταστο, πιο *pythonic*...

Αλλά ο κώδικας των προγραμμάτων του βιβλίου έχει ως κυρίαρχο σκοπό να υπηρετήσει τον διδακτικό χαρακτήρα του κειμένου και όχι να επιδείξει την προγραμματιστική δεινότητα του συγγραφέα, η οποία είναι έτσι και αλλιώς περιορισμένη.

Είναι, επίσης, αληθές ότι ο *καθένας* από εμάς είναι ικανός να γράψει τον κώδικα ενός προγράμματος για να επιλύσει ένα πρόβλημα, για τις ανάγκες των σπουδών του ή της εργασίας του. Αυτό που θα χρειαστεί είναι χρόνο και κέφι για να μάθει μια γλώσσα προγραμματισμού, καθώς και *δημιουργική λογική*.

Η επιλογή της γλώσσας προγραμματισμού είναι προσωπικό θέμα. Όλες είναι καλές, αρκεί να εξυπηρετούν τον σκοπό για τον οποίο θα τις χρησιμοποιήσουμε. Και όπως οι ξένες γλώσσες, άλλες μαθαίνονται εύκολα και άλλες δύσκολα.

Εκείνο όμως που θα πρέπει να έχουμε κατά νου είναι ότι οι γλώσσες προγραμματισμού είναι εργαλεία. Εάν κατέχουμε τις αρχές του προγραμματισμού χρησιμοποιώντας μια γλώσσα, πολύ εύκολα μπορούμε να προγραμματίσουμε μαθαίνοντας μια άλλη, ίσως πιο κατάλληλη για την επίλυση του προβλήματός μας, γλώσσα. Εάν έχουμε μάθει να βιδώνουμε μια βίδα με γερμανικό κλειδί, ξέρουμε να τη βιδώσουμε και με ένα γαλλικό κλειδί!

Η Python είναι μια εύκολη στην εκμάθηση γλώσσα προγραμματισμού, η οποία υποστηρίζεται από έναν εξαιρετικά μεγάλο αριθμό έτοιμων βιβλιοθηκών για την εκτέλεση εξειδικευμένων υπολογιστικών εργασιών. Το γεγονός αυτό την έχει κάνει πολύ δημοφιλή στην κοινότητα των προγραμματιστών και ιδιαίτερα στην επιστημονική κοινότητα.

Αυτό αντικατοπτρίζεται στα ranking lists των γλωσσών προγραμματισμού:

- TIOBE Index [<https://www.tiobe.com/tiobe-index>]
- IEEE Spectrum [<https://spectrum.ieee.org/top-programming-languages>]
- PYPL Index [<https://pypl.github.io/PYPL.html>]

όπου καταλαμβάνει την πρώτη θέση (Μάρτιος 2022).

Στο βιβλίο αυτό, εκτός από την εισαγωγή του αναγνώστη στον προγραμματισμό των ηλεκτρονικών υπολογιστών με τη γλώσσα Python, έγινε προσπάθεια να αναδειχθεί ένα μέρος του εύρους των δυνατοτήτων της γλώσσας, με την ανάπτυξη κώδικα που επιλύει προβλήματα θεματολογίας Μηχανικού και ιδίως Μεταλλειολόγου.

Το βιβλίο θα κλείσει με την παράθεση του Zen της Python [<https://peps.python.org/pep-0020>]:

```
The Zen of Python, by Tim Peters
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

το οποίο εμφανίζεται στην οθόνη του υπολογιστή μας όταν τρέξουμε ένα script της Python που περιέχει, σκέτη, την εντολή:

```
import this
```

22 Βιβλιογραφία

Για τη συγγραφή αυτού του βιβλίου ο συγγραφέας ανέτρεξε σε έναν πολύ μεγάλο αριθμό πηγών.

Από αυτές μόνο δύο είχαν τη φυσική μορφή έντυπης έκδοσης:

1. A.M. Henderson “*The Role of Computers in Preparing Graduates for the Mineral Industry*”, APCOM 77, 15th International Symposium on the Application of Computers and Operations Research in the Mineral Industries, Brisbane, Australia, 4-8 July 1977.
2. Mining Engineering Handbook, SME-AIME, 1st edition, 1973.

Όλες οι υπόλοιπες ήταν πηγές που είναι διαθέσιμες στο Διαδίκτυο και η πρόσβαση σε αυτές έγινε, στις περισσότερες των περιπτώσεων, κατά την ώρα της συγγραφής του βιβλίου. Για τον λόγο αυτό, οι διαδικτυακές διευθύνσεις αυτών των πηγών αναφέρονται στη θέση του κειμένου από όπου και ανακτήθηκαν.

Οι διαδικτυακές διευθύνσεις (URL) αυτών των πηγών ήταν σε ισχύ τον Μάρτιο του 2022.

Το παρόν σύγγραμμα χρηματοδοτήθηκε
από το Πρόγραμμα Δημοσίων Επενδύσεων του Υπουργείου Παιδείας.